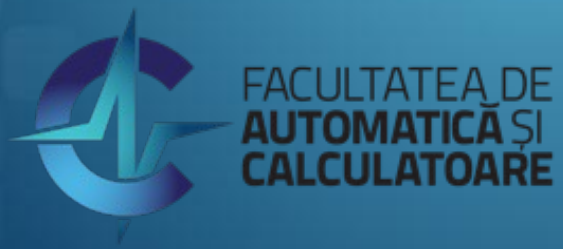
Algoritmi Paraleli și Distribuiți
Introducere

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



Despre curs

La fiecare 2 saptamani, vineri, 9-11 AM

- Interactivitate – dialog
- Open-office: ????, online MTeams
- Regulament afișat pe pagina cursului
  - Forma de evaluare a examenului final poate fi decisă/schimbată de către titularul de curs, în funcție de un context specific. Forma de evaluare poate fi: scris sau oral.
- Punctajele obținute pe parcurs sau examen se pot păstra pentru un an universitar (nu acumulare)
- Temele se pot trimite doar pe parcursul primului semestru universitar
- Rugamintea e sa ne vedem f2f pe MTeams

Despre curs

- Nota:

  10 = **6** (Parcurs) + **4** (Examen)

Parcurs **>= 3** Examen **>= 2** Teste **>= 1**

Parcurs: 3 teme(3p + 0.5p) + laborator (1p) +
Evaluare Teste (2p)

Întrebări?

Laborator Alg. Paraleli
Laborator Alg. Paraleli
....
Test 1
Laborator Alg. Distribuiti
Laborator Alg. Distribuiti
....
Test 2

Despre mine

CERN

MonALISA
MONitoring Agents using a Large Integrated Services Architecture

MONROE ALLIANCE

TEL-MONAER

SPERO

VINCI

https://cipsm.hpc.pub.ro

ciprian.dobre@upb.ro

http://www.facebook.com/ciprian.dobre

Bibliografia

G.R.Andrews
Concurrent Programming. Principles and Practice

C.A.R. Hoare
Communicating Sequential Processes

S.G. Akl
The Design and Analysis of Parallel Algorithms

G.R. Andrews, R.A. Olsson
The SR Programming Language. Concurrency in Practice

Ian Foster
Designing and Building Parallel Programs

A.S. Tanenbaum
Structured Computer Organization (Fourth Edition)

Allen B. Downey
The Little Book of Semaphores

Wan Fokkink
Distributed Algorithms: An Intuitive Approach (2nd edition)

Nancy A. Lynch
Distributed Algorithms

De ce calcul paralel și distribuit?

- Timp de execuție mai scurt
- Permite abordarea problemelor de dimensiuni mari
- Accesul resurselor aflate la distanță
- Reducerea costurilor
- Toleranță la defecte
- Ascunderea timpilor de așteptare
- Redundanța
- Scalabilitatea
- Scăderea timpului de răspuns
- Securitate

Limitele programării secvențiale?

Cramming More Components onto Integrated Circuits

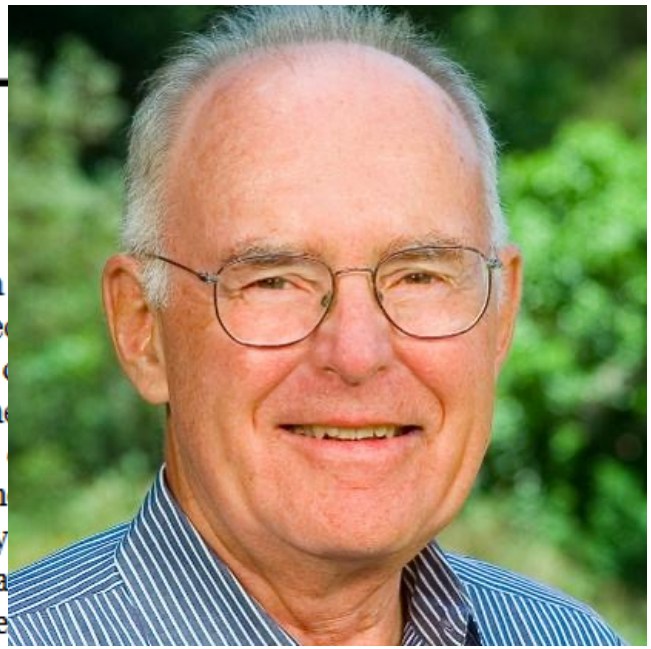
GORDON E. MOORE, LIFE FELLOW, IEEE

With unit cost falling as the number of components per circuit rises, by 1975 economics may dictate squeezing as many as 65 000 components on a single silicon chip.

The future of integrated electronics is the future of electronics itself. The advantages of integration will bring about a proliferation of electronics, pushing this science into many new areas.

Integrated circuits will lead to such wonders as home computers—or at least terminals connected to a central computer—automatic controls for automobiles, and personal portable communications equipment. The electronic wristwatch needs only a display to be feasible today.

But the biggest potential lies in the production of large systems. In telephone communications, integrated circuits

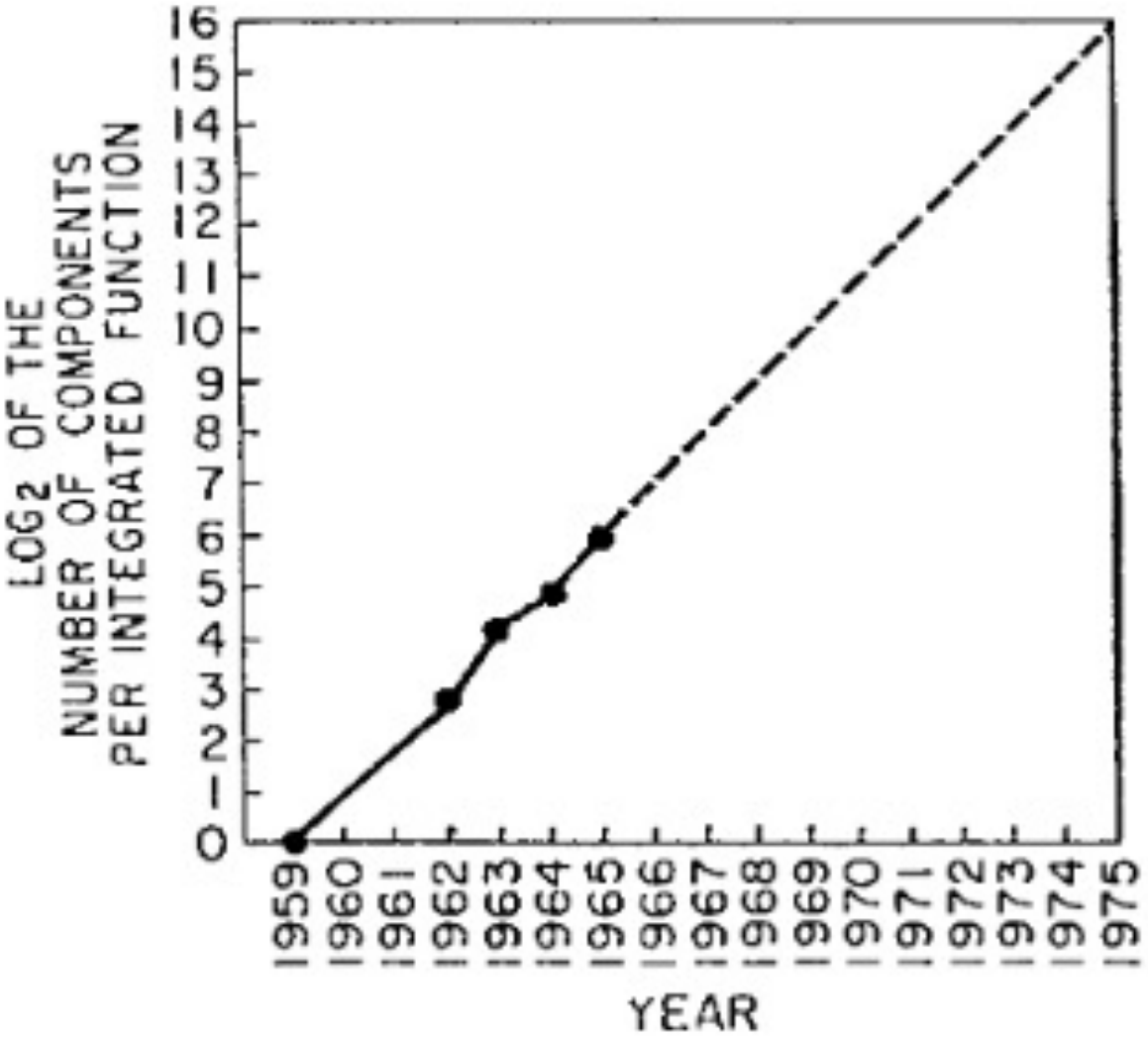


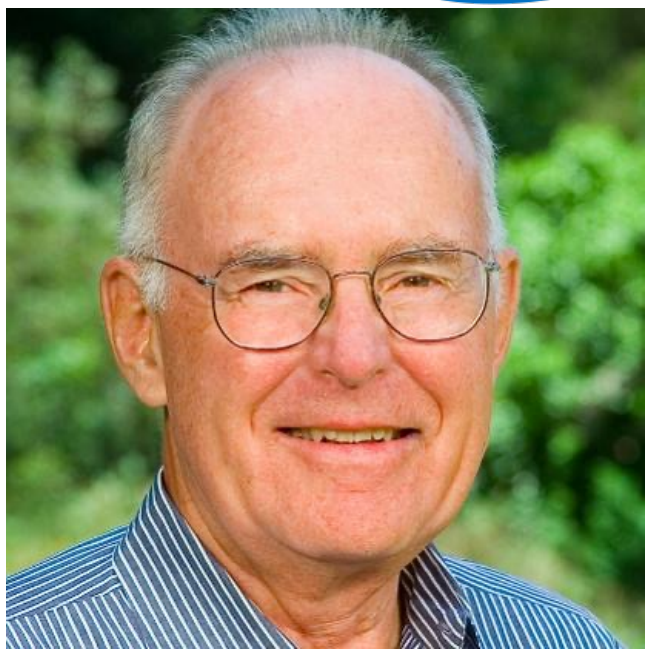
Each approach
each borrowed te
believe the way
various approache

The advocates
already using th
resistors by apply
conductor substra
upon films are de
attachment of active semiconductor devices to the passive film arrays.

Both approaches have worked well and are being used in equipment today.

Limitele programării secvențiale?





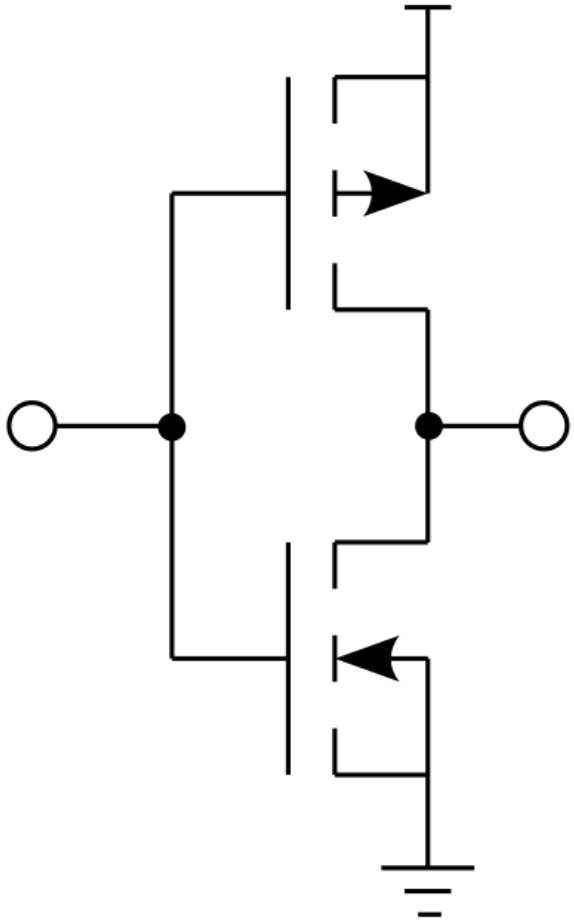
cofounded Intel

Limitele programării secvențiale

- Viteza de transmisie
  - Maxim c – viteza luminii
- Miniaturizare
  - Tranzistor de mărimea unui atom
- Economic
  - Costuri enorme pentru cercetare și proiectarea unui nou timp de procesor

Limitele programării secvențiale

- CMOS

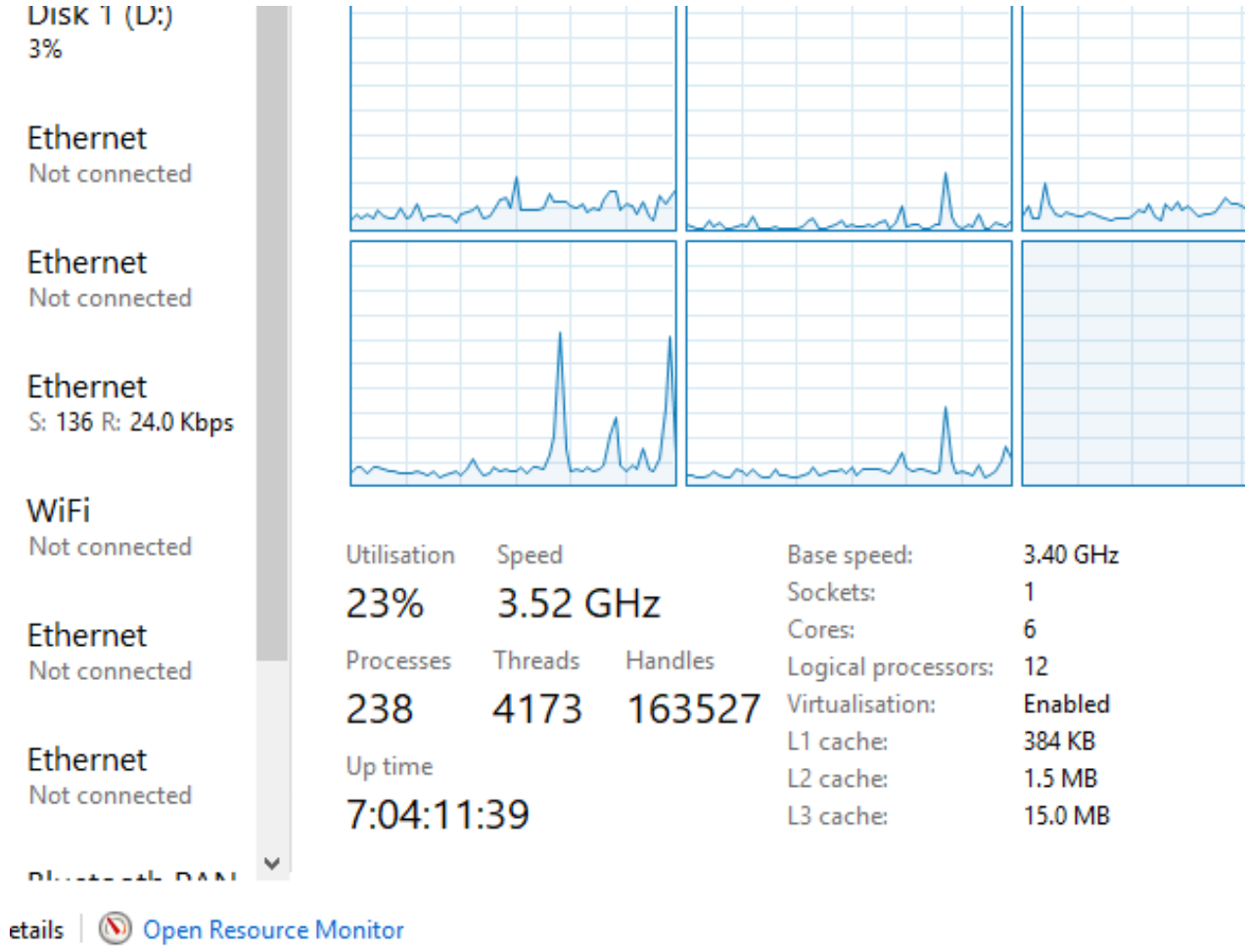


Importanța cursului

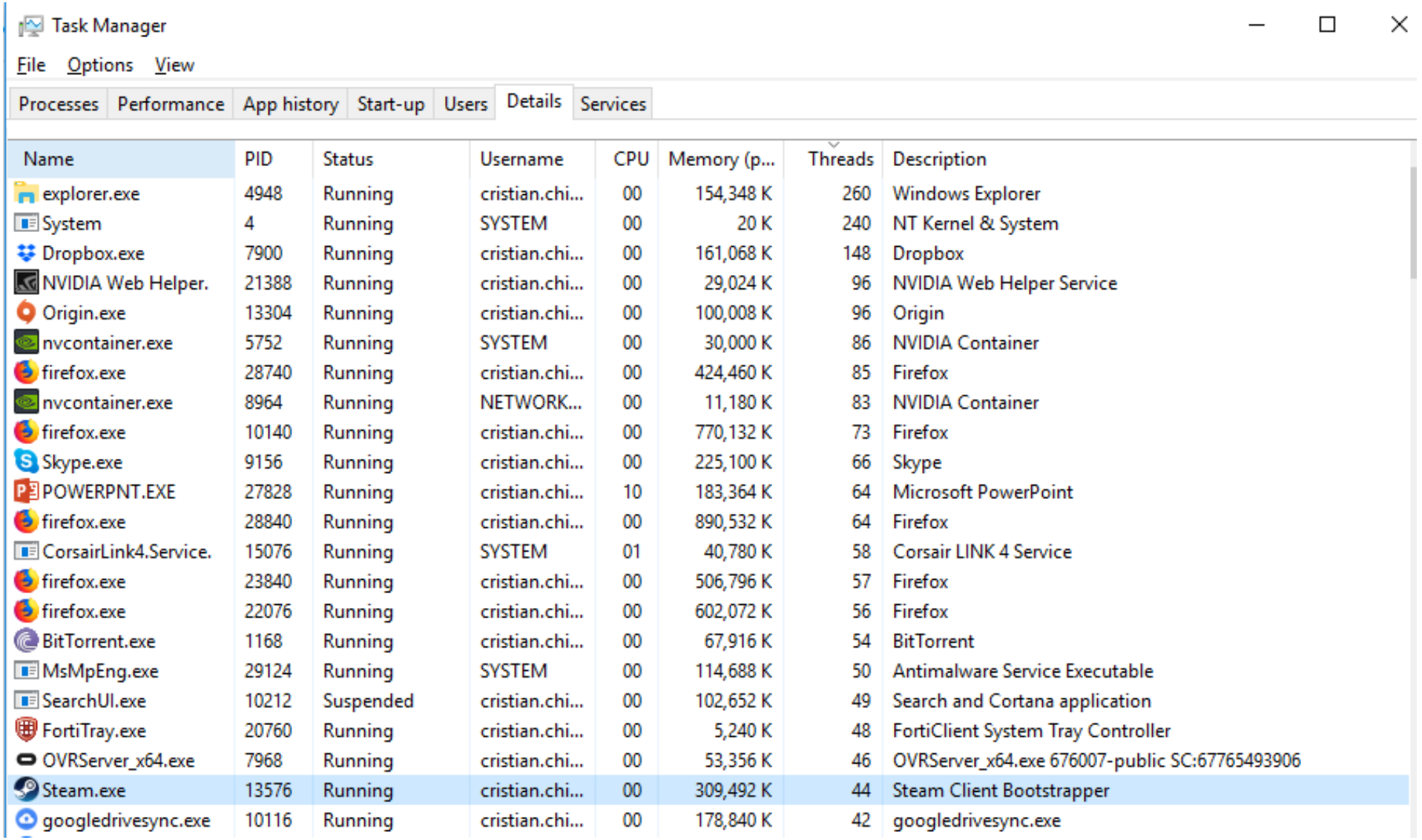
- Apar tot mai multe tehnologii paralele si distribuite
  - Blockchain; Peer-to-Peer
- Chiar și un procesor de ceas are mai multe core-uri
  - LG Watch Sport - MSM8909w Processor
    - Quad-Core
- Suport în noile IDE-uri
  - Eclipse; Visual Studio
- Număr mare de aplicații distribuite
  - Dropbox; Spark; Boinc

Importanța cursului

- Aproape toate aplicațiile folosesc mai multe thread-uri



Importanța cursului



Name	PID	Status	Username	CPU	Memory (p...	Threads	Description
explorer.exe	4948	Running	cristian.chi...	00	154,348 K	260	Windows Explorer
System	4	Running	SYSTEM	00	20 K	240	NT Kernel & System
Dropbox.exe	7900	Running	cristian.chi...	00	161,068 K	148	Dropbox
NVIDIA Web Helper.	21388	Running	cristian.chi...	00	29,024 K	96	NVIDIA Web Helper Service
Origin.exe	13304	Running	cristian.chi...	00	100,008 K	96	Origin
nvcontainer.exe	5752	Running	SYSTEM	00	30,000 K	86	NVIDIA Container
firefox.exe	28740	Running	cristian.chi...	00	424,460 K	85	Firefox
nvcontainer.exe	8964	Running	NETWORK...	00	11,180 K	83	NVIDIA Container
firefox.exe	10140	Running	cristian.chi...	00	770,132 K	73	Firefox
Skype.exe	9156	Running	cristian.chi...	00	225,100 K	66	Skype
POWERPNT.EXE	27828	Running	cristian.chi...	10	183,364 K	64	Microsoft PowerPoint
firefox.exe	28840	Running	cristian.chi...	00	890,532 K	64	Firefox
CorsairLink4.Service.	15076	Running	SYSTEM	01	40,780 K	58	Corsair LINK 4 Service
firefox.exe	23840	Running	cristian.chi...	00	506,796 K	57	Firefox
firefox.exe	22076	Running	cristian.chi...	00	602,072 K	56	Firefox
BitTorrent.exe	1168	Running	cristian.chi...	00	67,916 K	54	BitTorrent
MsMpEng.exe	29124	Running	SYSTEM	00	114,688 K	50	Antimalware Service Executable
SearchUI.exe	10212	Suspended	cristian.chi...	00	102,652 K	49	Search and Cortana application
FortiTray.exe	20760	Running	cristian.chi...	00	5,240 K	48	FortiClient System Tray Controller
OVRServer_x64.exe	7968	Running	cristian.chi...	00	53,356 K	46	OVRServer_x64.exe 676007-public SC:67765493906
Steam.exe	13576	Running	cristian.chi...	00	309,492 K	44	Steam Client Bootstrapper
googledrivesync.exe	10116	Running	cristian.chi...	00	178,840 K	42	googledrivesync.exe

Supercomputers

Rank	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148,600.0	200,794.9	10,096
2	**Sierra** - IBM Power System S922LC, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband , IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94,640.0	125,712.0	7,438
3	**Sunway TaihuLight** - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway , NRCPC National Supercomputing Center in Wuxi China	10,649,600	93,014.6	125,435.9	15,371
4	**Tianhe-2A** - TH-IVB-FEP Cluster, Intel Xeon E5-2692v2 12C 2.2GHz, TH Express-2, Matrix-2000 , NUDT National Super Computer Center in Guangzhou China	4,981,760	61,444.5	100,678.7	18,482
5	**Frontera** - Dell C6420, Xeon Platinum 8280 28C 2.7GHz, Mellanox InfiniBand HDR , Dell EMC Texas Advanced Computing Center/Univ. of Texas United States	448,448	23,516.4	38,745.9	

Summit

Sierra

Sunway TaihuLight

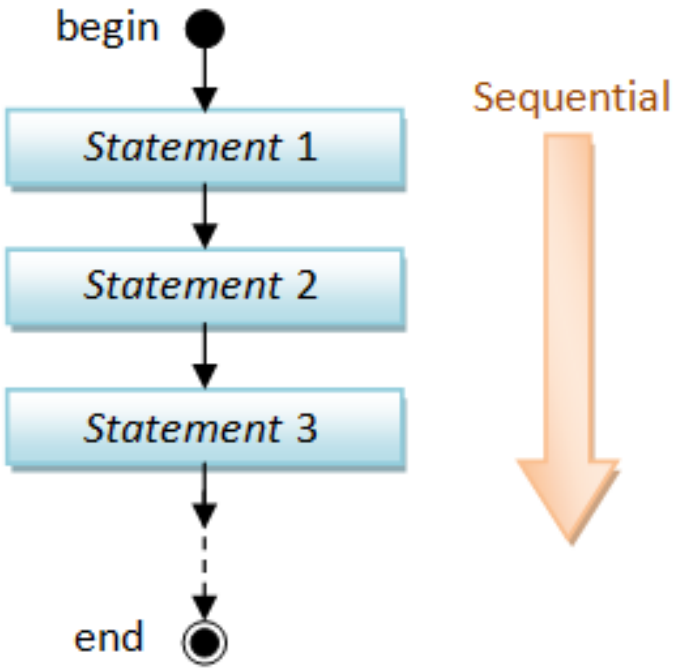
Computing power 2018

24-hour average: 28.019 PetaFLOPS.
Active: 156,141 volunteers, 648,324 computers.

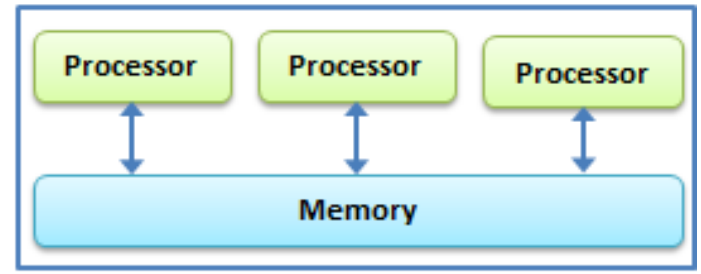
Computing power 2020

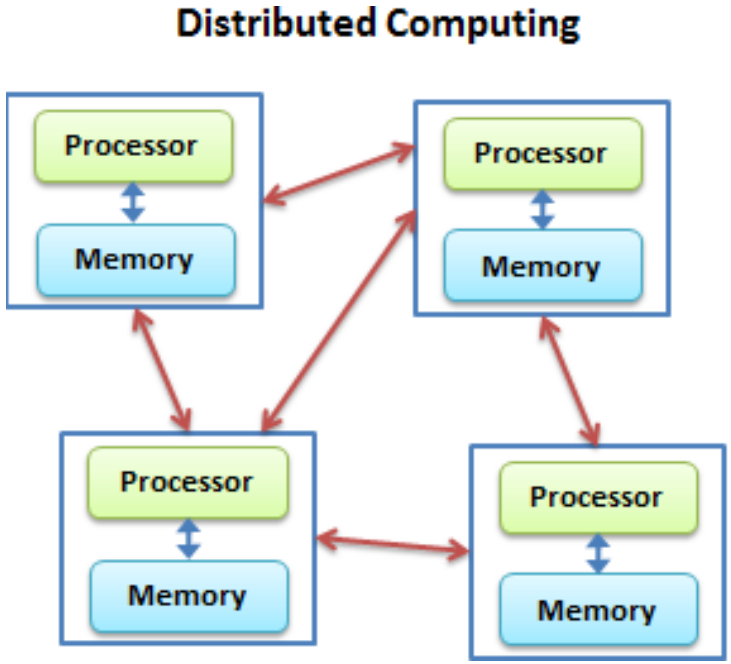
24-hour average: 26.392 PetaFLOPS.
Active: 87,025 volunteers, 826,092 computers.
Daily change: +43 volunteers, 0 computers.

Algoritmi Paraleli/Distribuiți vs Secvențiali



Sequential



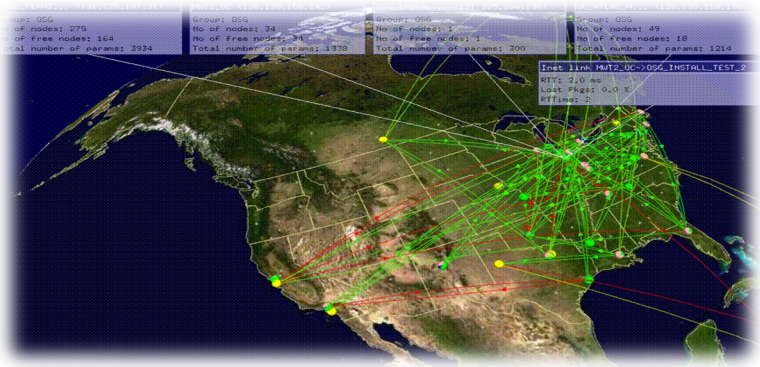


Resurse fizice

- Procesor – multi-core

High End Desktop (HEDT)	MSRP / Retail	Cores / Threads	Base / Boost GHz	L3 Cache	Memory
Threadripper 3990X	$3,990 / $3,750	64 / 128	2.9 / 4.3	256	Quad DDR4-3200
Intel W-3175X	$2,999 / N/A	28 / 56	3.1 / 4.8	38.5	Six-Channel DDR4-2666

- Cluster



- Grid/Cloud

Taxonomia Flynn

Some Computer Organizations and Their Effectiveness

MICHAEL J. FLYNN, MEMBER, IEEE

Abstract—A hierarchical model of computer organizations is developed, based on a tree model using request/service type resources as nodes. Two aspects of the model are distinguished: logical and physical.

General parallel- or multiple-stream organizations are examined as to type and effectiveness—especially regarding intrinsic logical difficulties.

The overlapped simplex processor (SISD) is limited by data dependencies. Branching has a particularly degenerative effect.

The parallel processors [single-instruction stream–multiple-data stream (SIMD)] are analyzed. In particular, a nesting type explanation is offered for Minsky's conjecture—the performance of a parallel processor increases as log M instead of M (the number of data stream processors).

Multiprocessors (MIMD) are subjected to a saturation syndrome based on general communications lockout. Simplified queuing models indicate that saturation develops when the fraction of task time spent locked out (L/E) approaches $1/n$, where n is the number of processors. Resources sharing in multiprocessors can be used to avoid

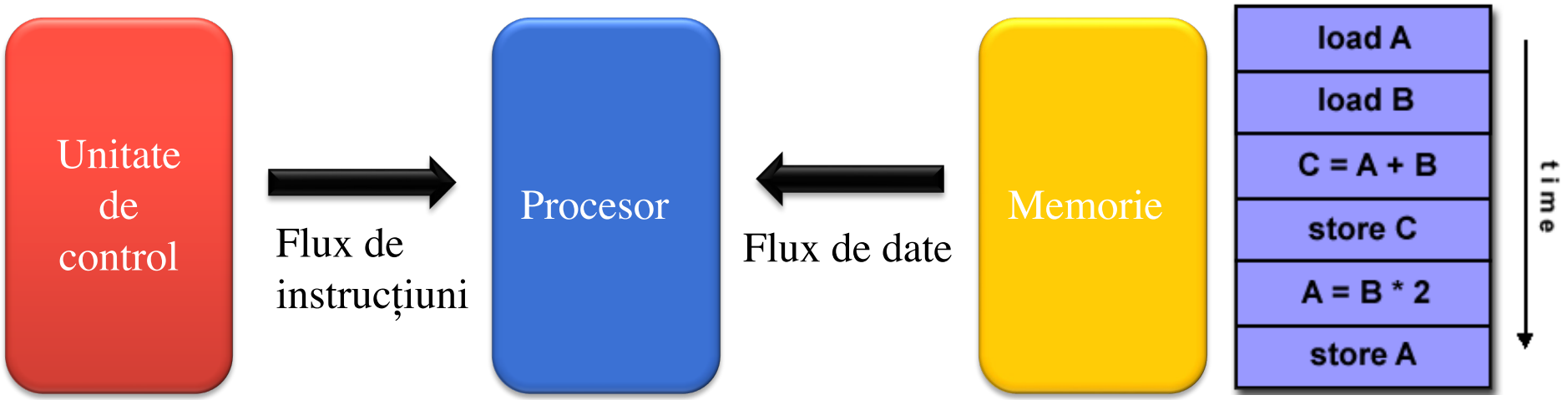
more "macr
particular us
must be sha
more signific
1) There i
limiting reso
est will eithe
tions will ap
figurations.
computer wi
cerned with
potential, wh
siderations.
2) We ma
sets. It is ass
set of instru

Taxonomia Flynn

- SISD
  - Single Instruction Stream, Single Data Stream
  - Calculatorul Clasic one-core
- SIMD
  - Single Instruction Stream, Multiple Data Streams
  - Suportul SSE; procesoare GPU
- MISD
  - Multiple Instruction Streams, Single Data Stream
  - Sisteme specializate
- MIMD
  - Multiple Instruction Streams, Multiple Data Streams
  - Procesoare actuale (ce aveți acasă și în buzunar)

SISD

- Model clasic Arhitectura *von Neumann*



SISD

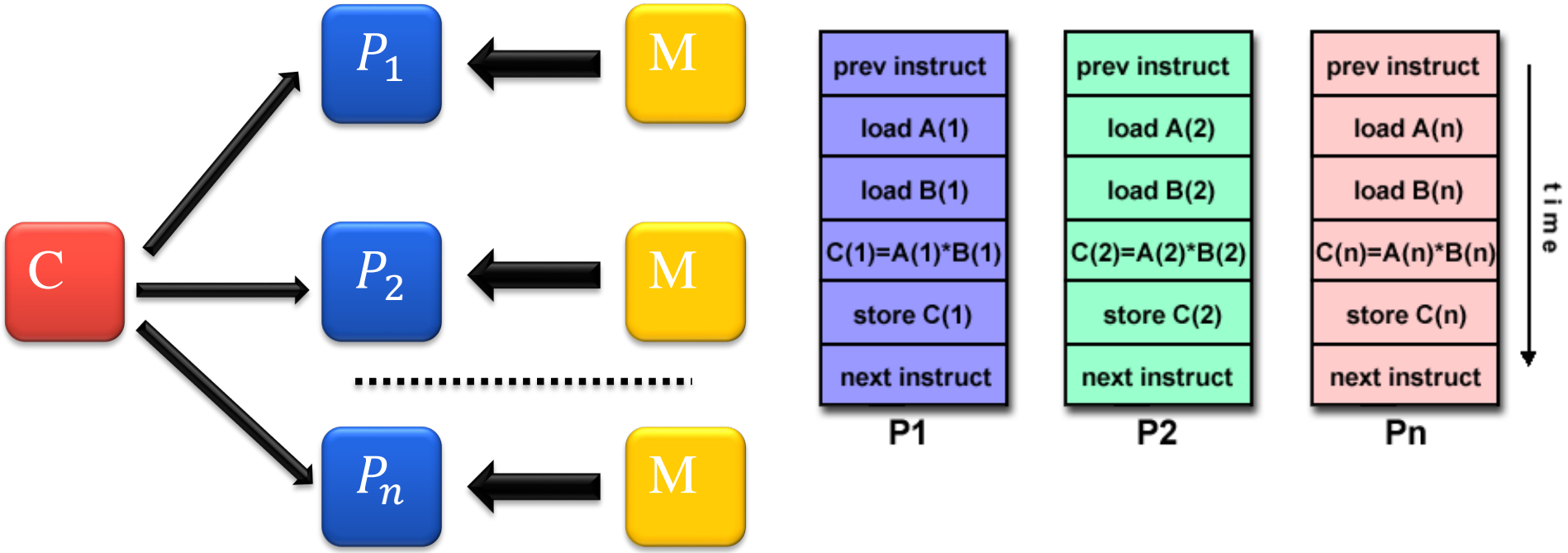
- Model clasic Arhitectura *von Neumann*

First Draft of a Report
on the EDVAC

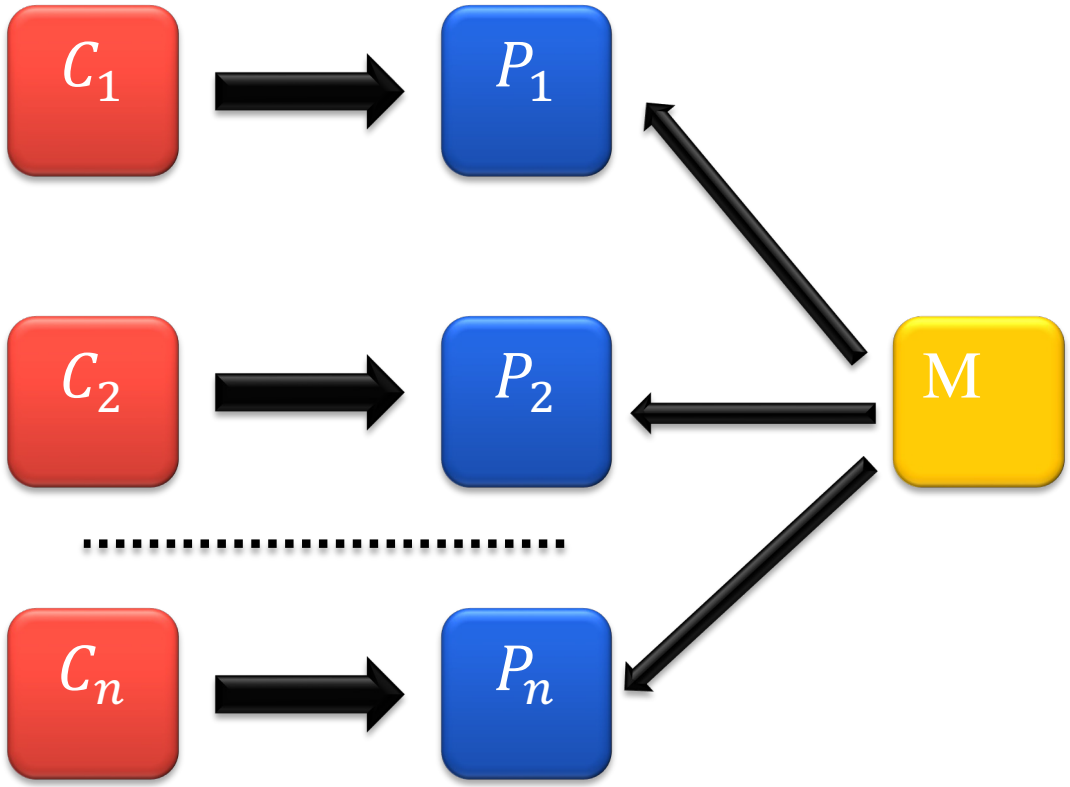
by

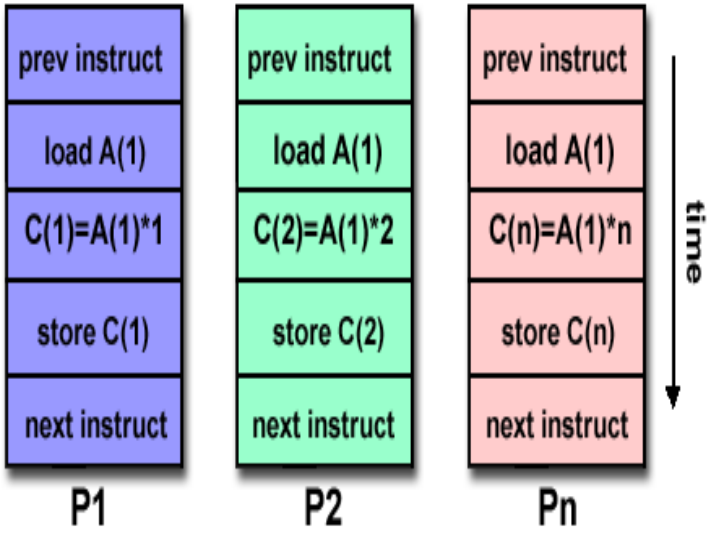
John von Neumann

SIMD

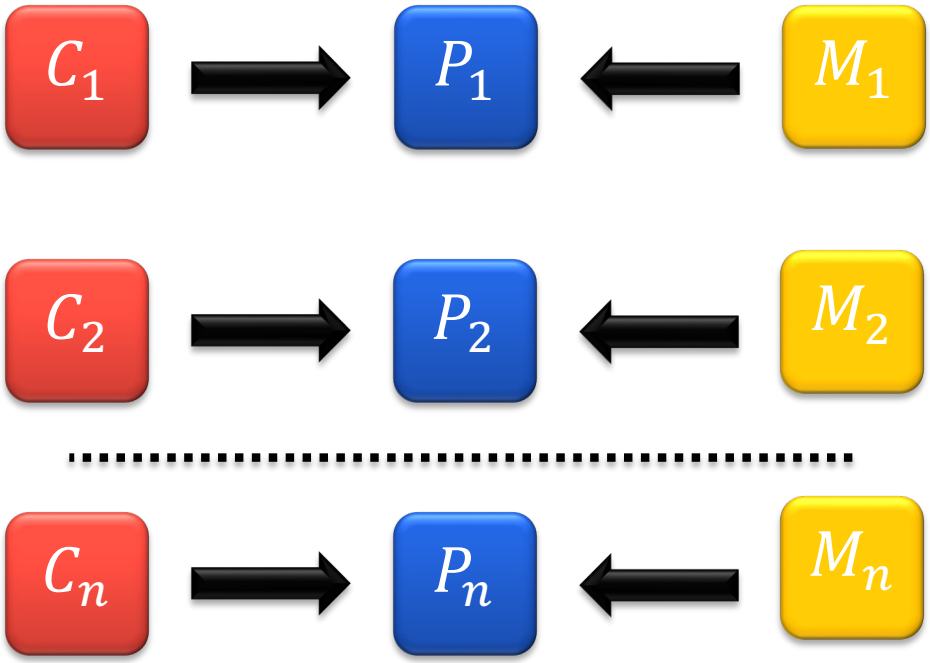

C
P_1
P_2
P_n
M
M
M
prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
load A(2)
load B(2)
C(2)=A(2)*B(2)
store C(2)
next instruct
P2
prev instruct
load A(n)
load B(n)
C(n)=A(n)*B(n)
store C(n)
next instruct
Pn
time

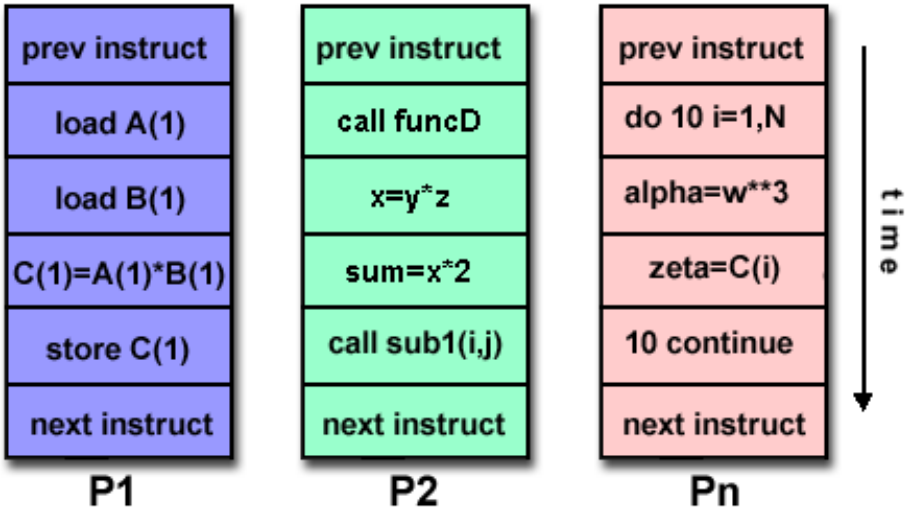
MISD


C_1
C_2
C_n
P_1
P_2
P_n
M


prev instruct
load A(1)
C(1)=A(1)*1
store C(1)
next instruct
P1
prev instruct
load A(1)
C(2)=A(1)*2
store C(2)
next instruct
P2
prev instruct
load A(1)
C(n)=A(1)*n
store C(n)
next instruct
Pn
time

MIMD


C_1
P_1
M_1
C_2
P_2
M_2
C_n
P_n
M_n

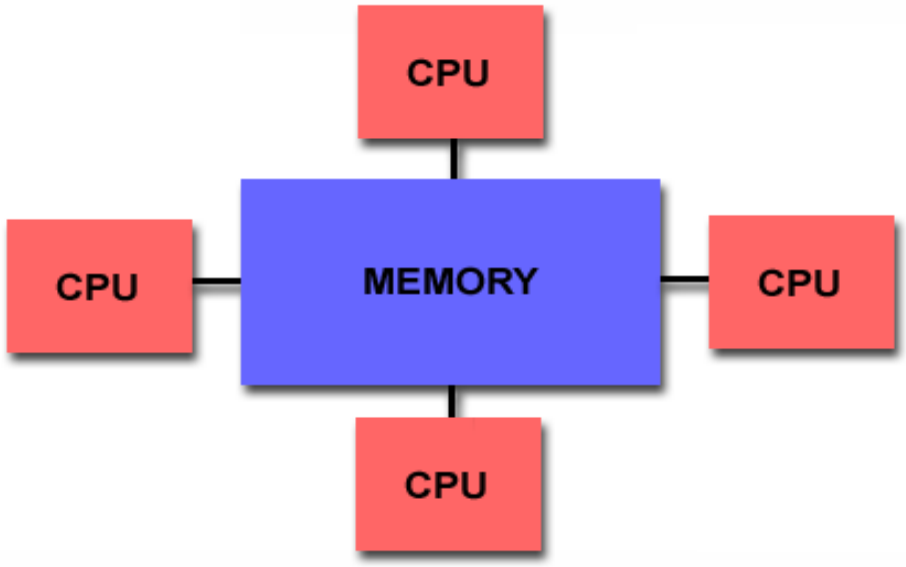

prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
call funcD
x=y*z
sum=x*2
call sub1(i,j)
next instruct
P2
prev instruct
do 10 i=1,N
alpha=w**3
zeta=C(i)
10 continue
next instruct
Pn
time

Memorie Partajată

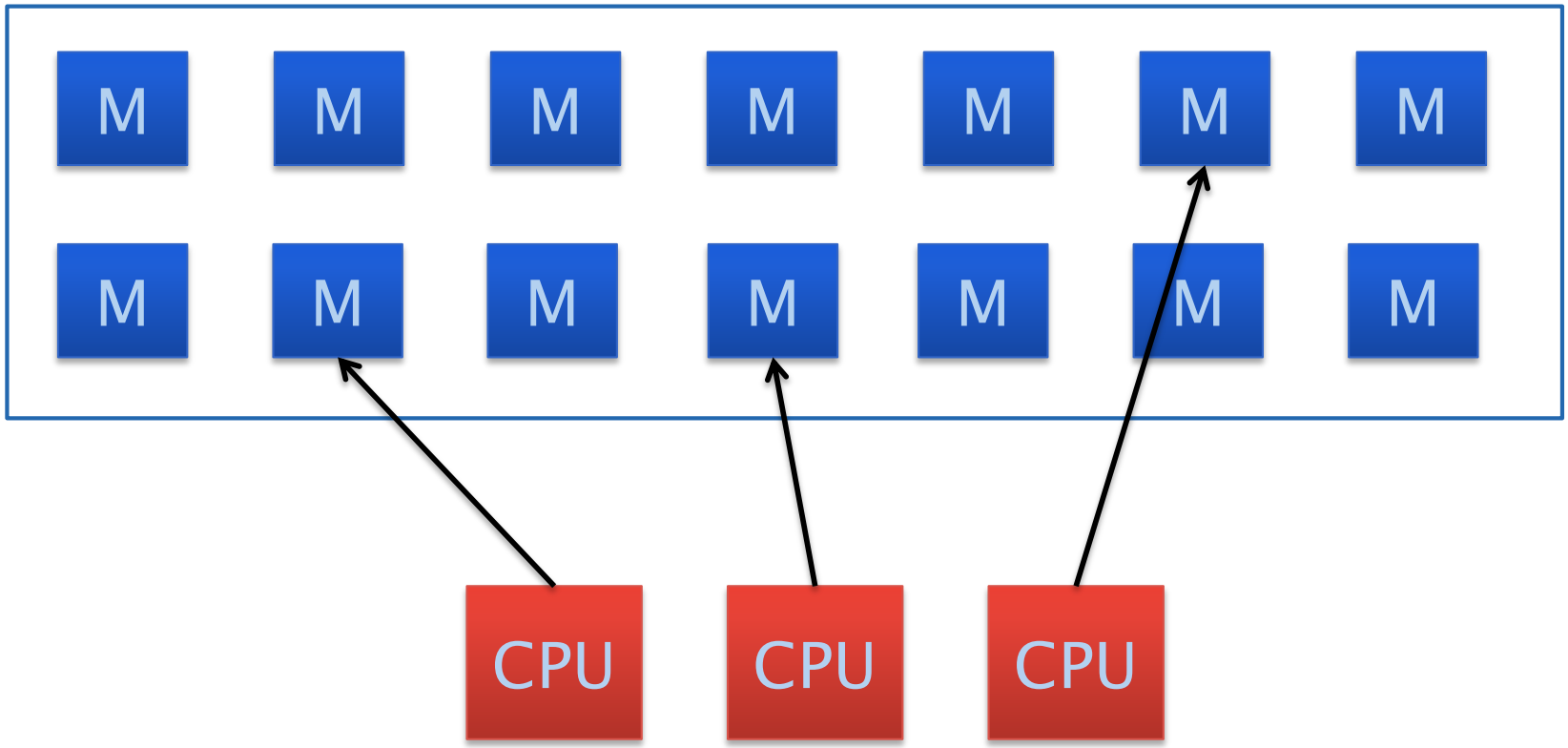
- Shared memory
  - Parallel Random Access Memory - **PRAM**
  - **EREW** – Exclusive Read Exclusive Write
  - **CREW** – **Concurrent Read Exclusive Write --** cel mai des întâlnit
  - **ERCW** – Exclusive Read Concurrent Write
  - **CRCW** – Concurrent Read Concurrent Write
  - O variabilă poate fi citită într-un pas în model **CR** dar în log(N) în model **ER**.

Memorie Partajată

- Uniform Memory Access
  - **UMA**

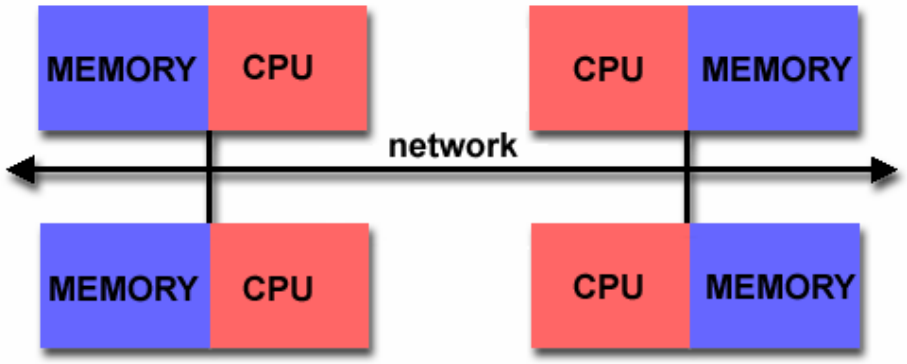


Memorie Partajată - Acces

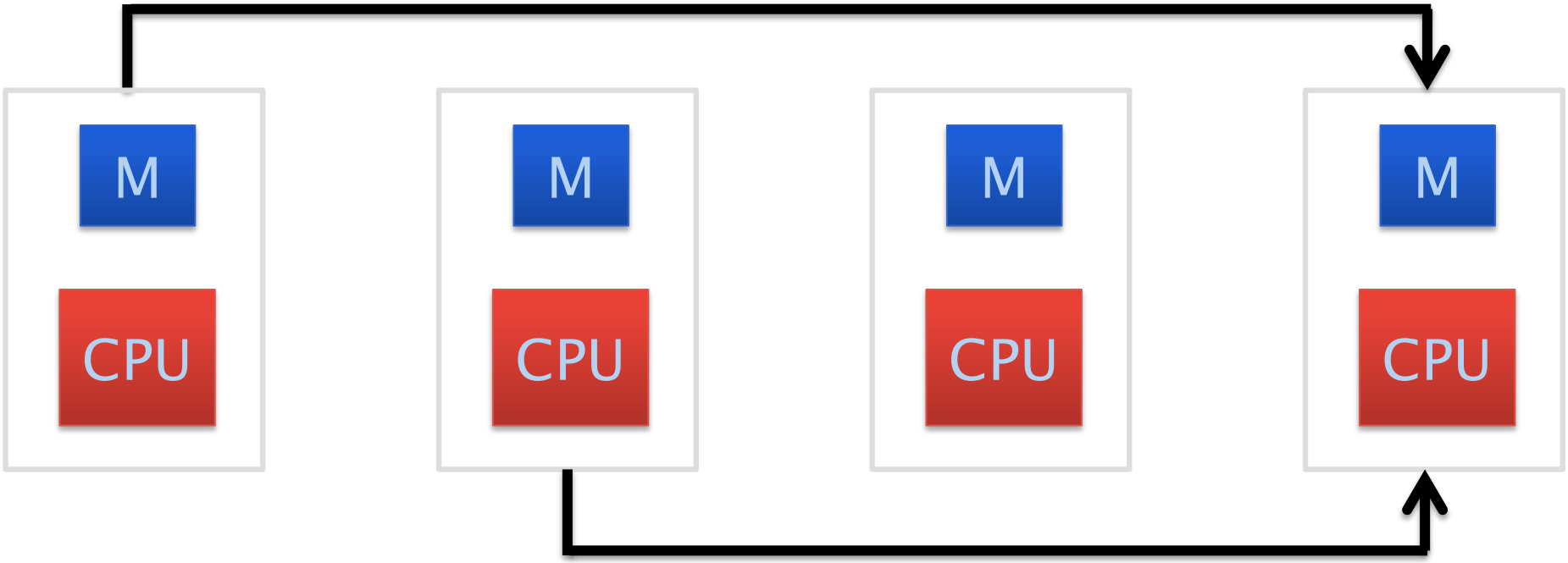


Memorie Distribuită

- Massively Parallel Processors
- Network of workstations
- Non-Uniform Memory Access
  - **NUMA**

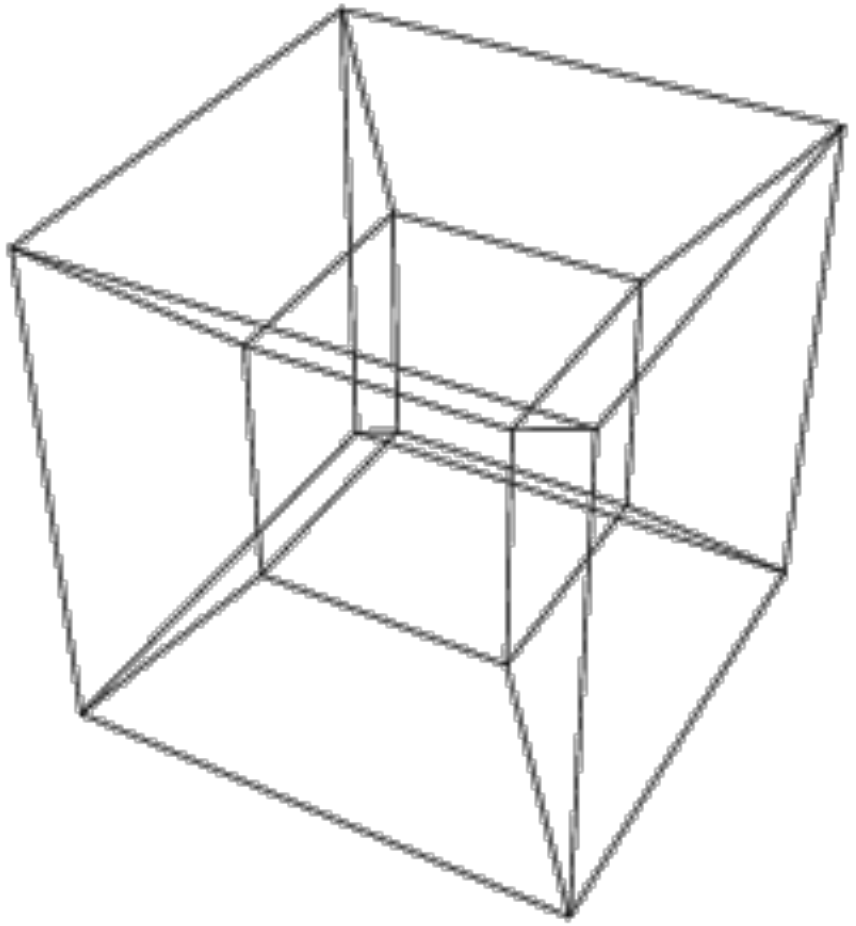


Memorie Distribuită - Acces



Rețele de configurare

- Topologii
  - Tablou
  - Arbore
  - Cub
  - Hipercub



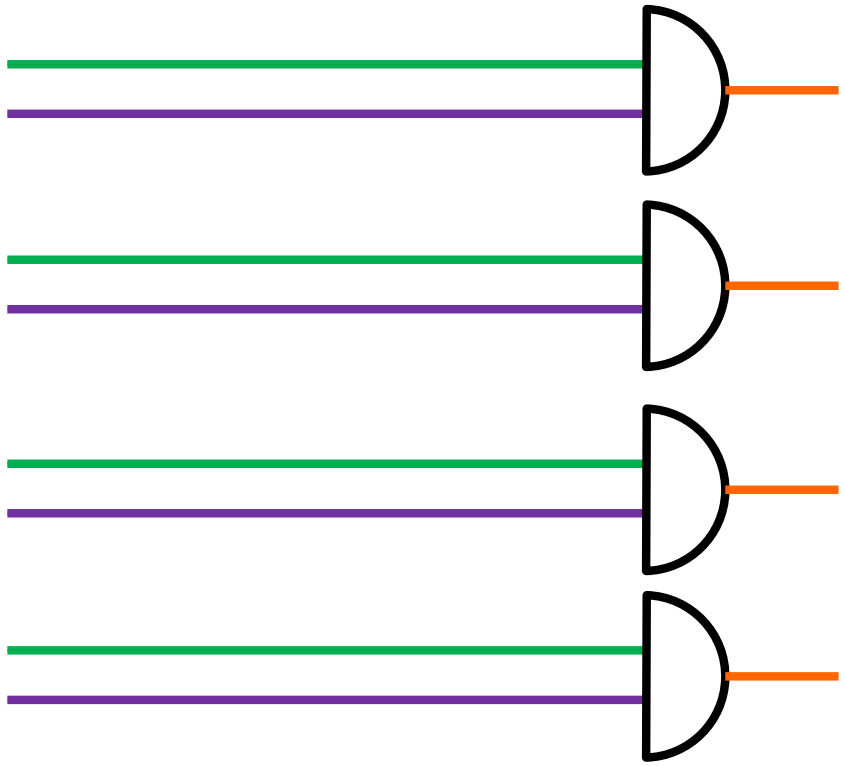
- Depinde de
  - Aplicație
  - Performanțe dorite
  - Număr procesoare disponibile
- Exemple: IBM 9000, Cray C90, Fujitsu VP

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task

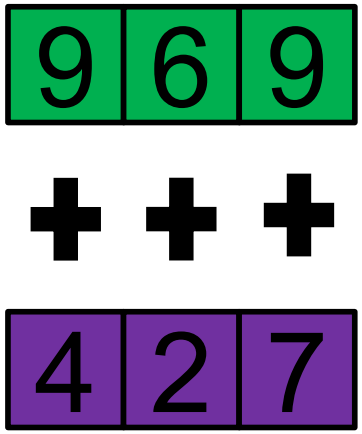
Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task



Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task



C = A + B

Adunarea a doi vectori

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

Tipuri de paralelism

- La nivel de bit
- La nivel de instrucțiune
- La nivel de task
  - **Multi-Tasking (pot comunica și procesele)**
  - **Multi-Threading**

Cum pot comunica două procese?

Notații pseudocod

```
co S1 || S2 || ... || Sn oc
```

Ex.1:

```
x=0; y=0;
co x=x+1 || y=y+1 oc
z=x+y;
```

Notații pseudocod

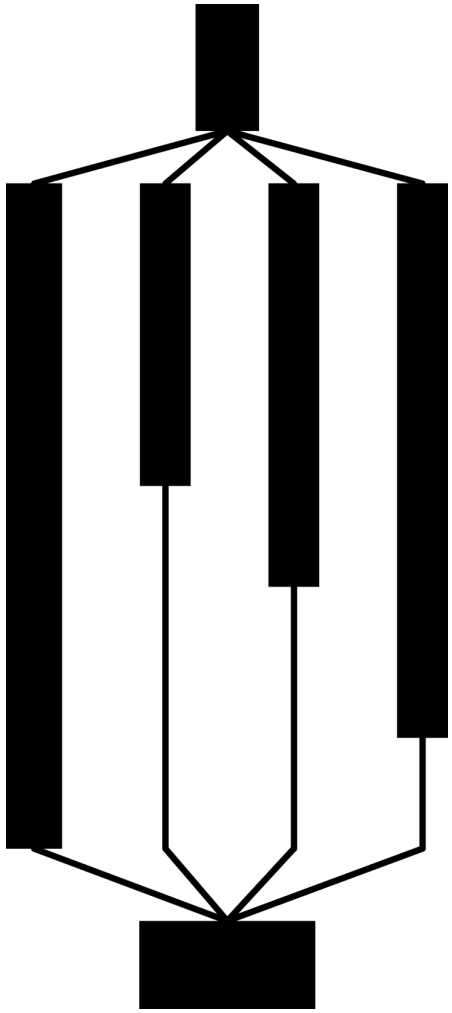
```
co [cuantificator]{Sj}
```

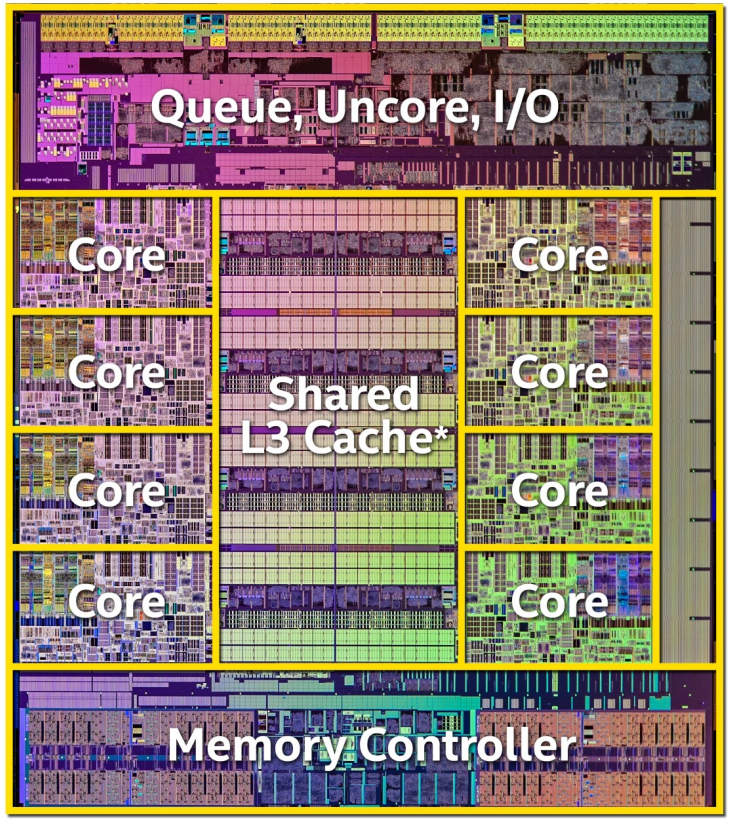
Ex. 2:

```
    co [j=1 to n] {a[j]=0; b[j]=0;}
```

```
process Name[cuatificatori] { Sj }
```

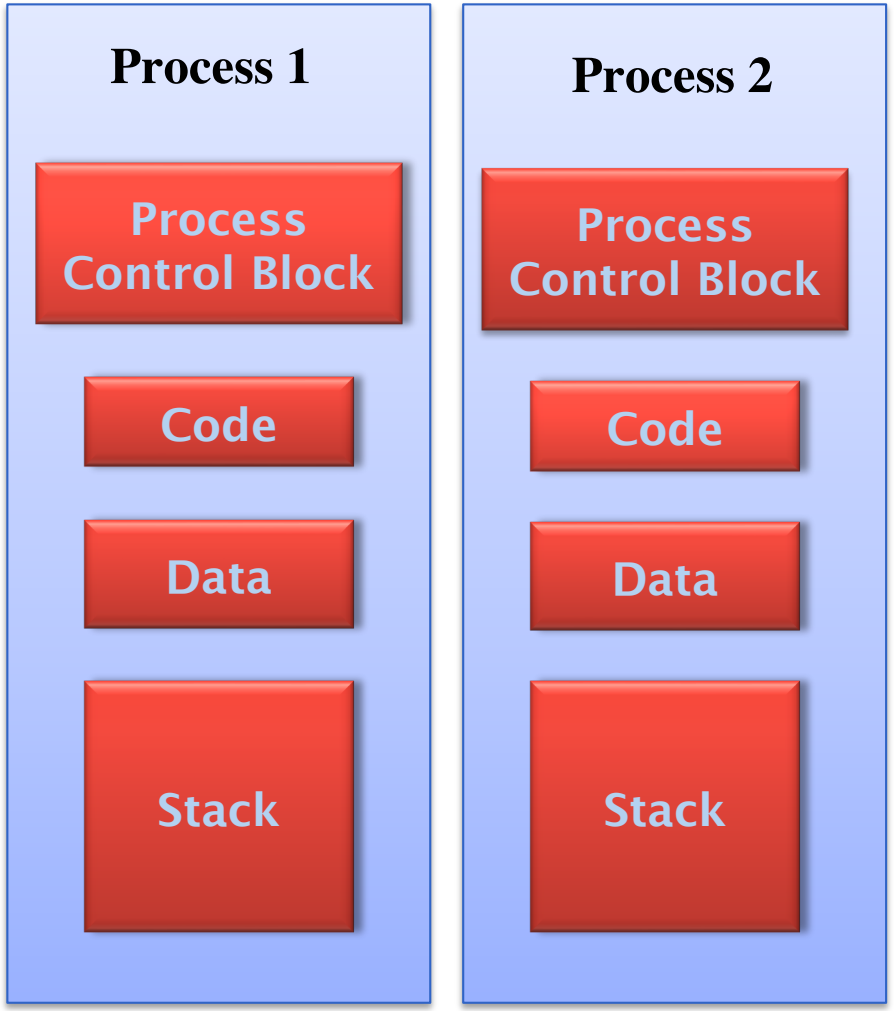
Threads vs cores





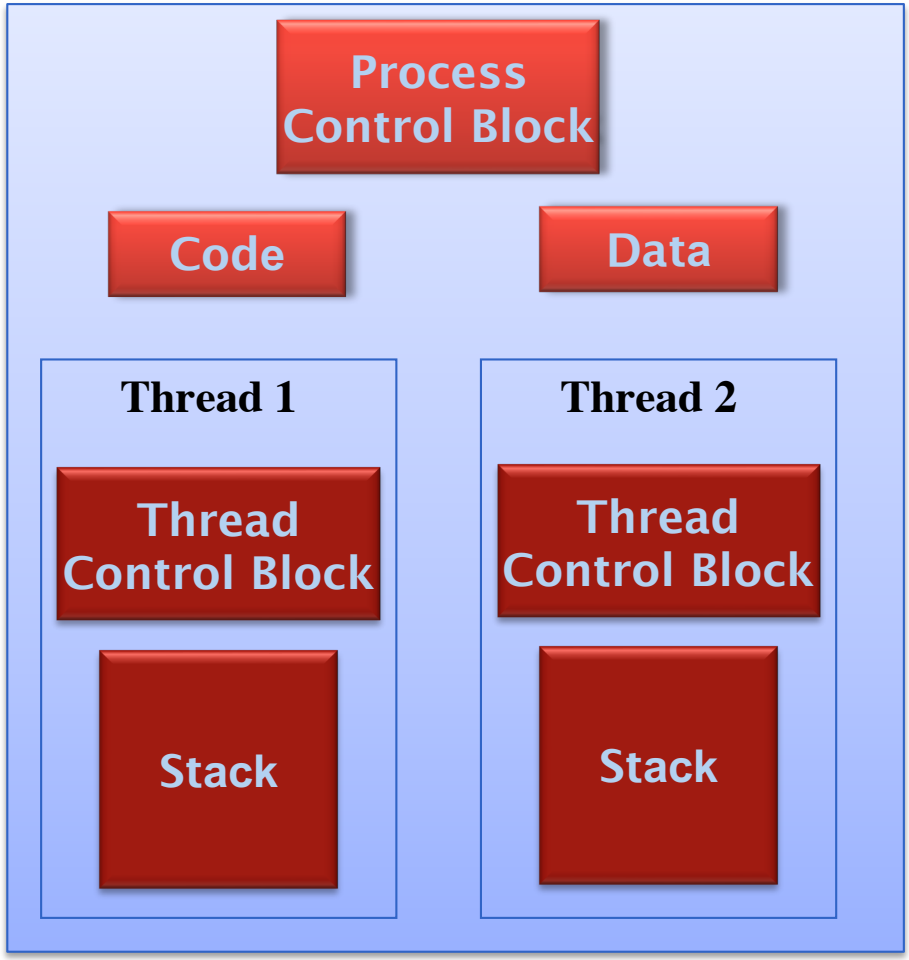
Procese și thread-uri

- Proces
  - Instanță a unui program în execuție
  - Pentru un proces, SO alocă:
    - Un spațiu în memorie (codul programului, zona de date, stiva)
    - Controlul anumitor resurse (fișiere, dispozitive I/O, …)

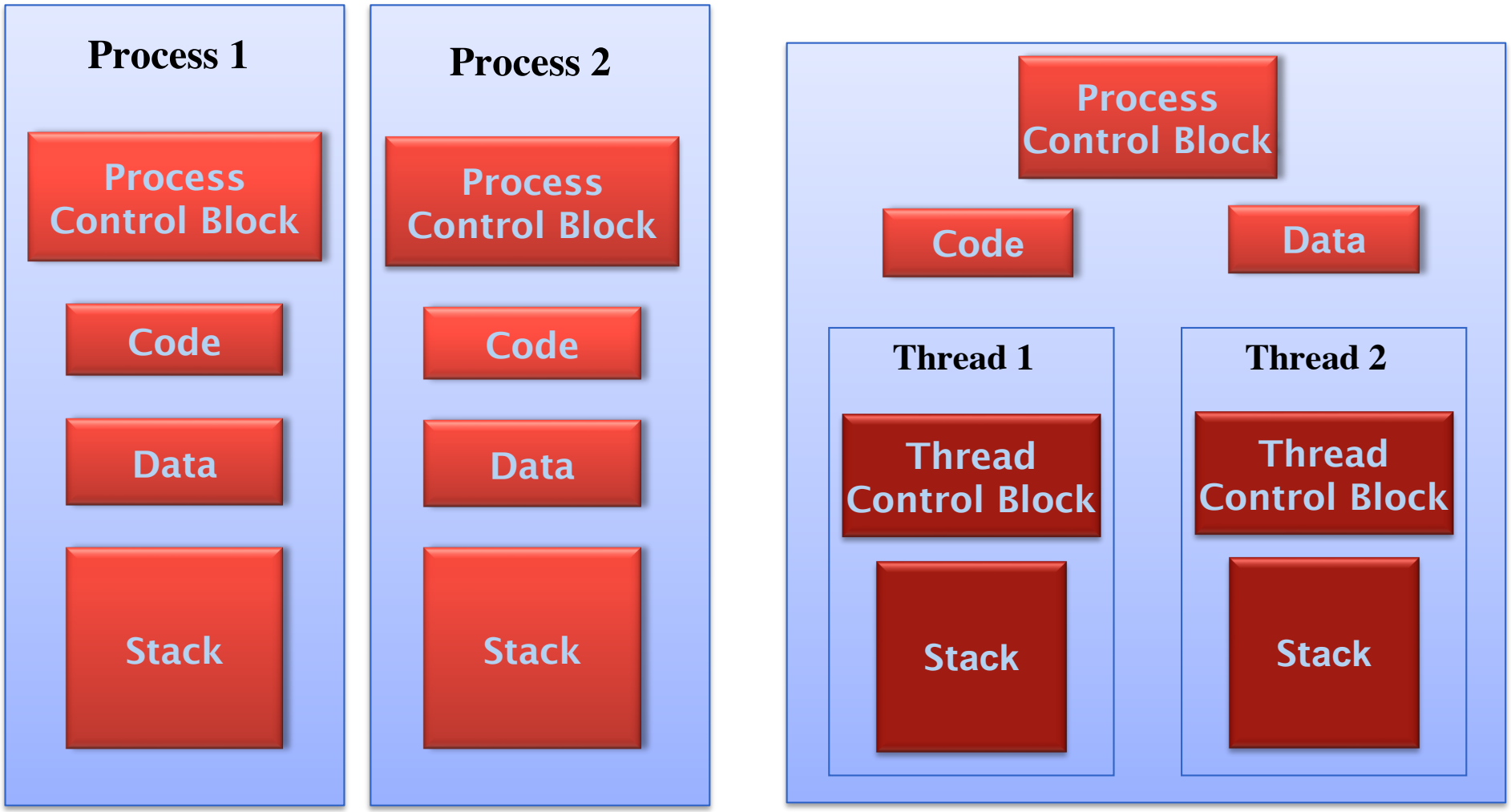


Thread (fir de execuție)

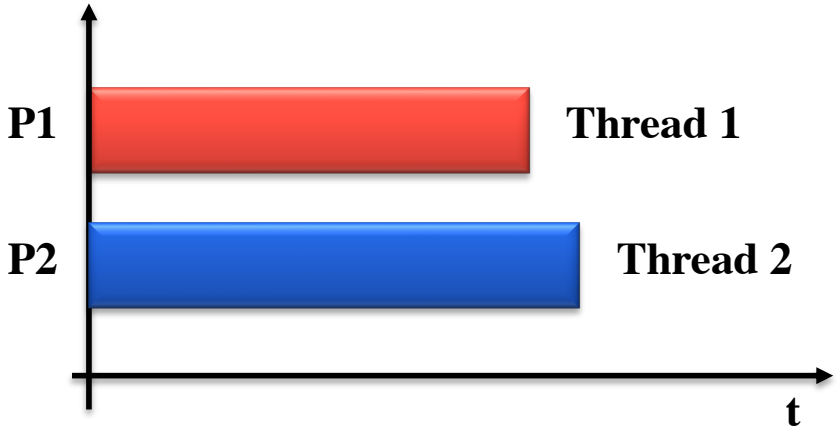
- Un proces poate avea mai multe fire de execuție
- Fir de execuție (thread):
  - Flux de control secvențial în cadrul unui proces
- Firele de execuție
  - împart același spațiu de adrese;
  - au în comun: zonele de cod și date din memorie, resursele procesului
  - zona de stivă – reprezintă starea curentă a thread-ului și este proprie thread-ului



Multi-proces vs. multi-thread

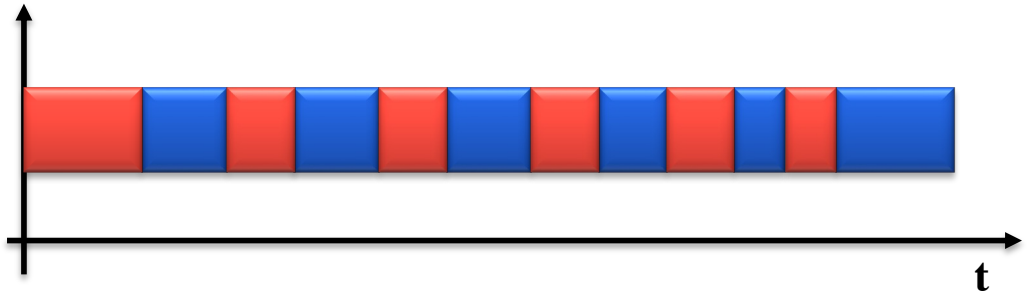


Execuția thread-urilor



→ Putem considera execuție în paralel

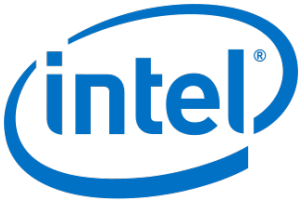
Defapt:



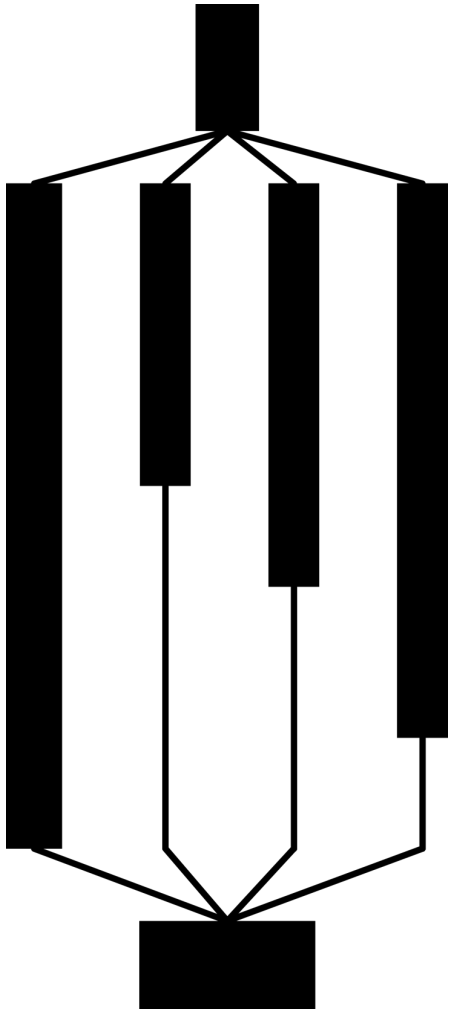
→ planificarea este realizată de către sistemul de operare

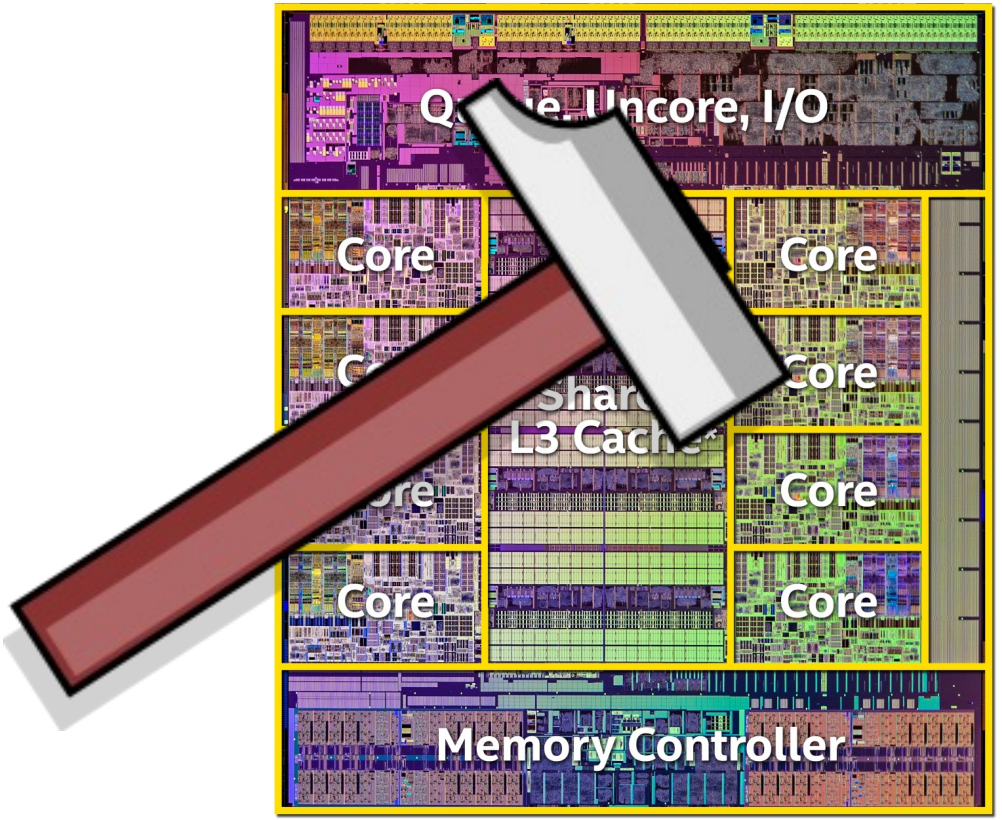
→ politici: divizarea timpului (*time sharing*), priorități

Hyperthreading – the confusion

Thank you 

Hyperthreading – the confusion





Multi-tasking vs Multi-threading

Multi-tasking

Task 1

Task 2

Multi-tasking

Task 2

Task 1

Multi-tasking

Task 1

Task 2

Multi-tasking

Task 2

Task 1

Multi-threading

Thread 2

Thread 3

Thread 4

Thread 1

Multi-threading

Thread 1

Thread 2

Thread 4

Thread 3

Multi-threading

Thread 1

Thread 2

Thread 4 Thread 3

Multi-threading

Thread 2

Thread 3

Thread 4

Thread 1

Multi-threading

Thread 1

Thread 4

Thread 2 Thread 3

Multi-tasking vs Multi-threading

Care sunt diferențele?

Un proces are mai multe thread-uri

Thread-urile unui proces împart memoria

Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri?

Poți avea mai multe thread-uri pe un singur core?

Multi-tasking vs Multi-threading

Poți avea multi-tasking pe mai multe core-uri? **DA**

Poți avea mai multe thread-uri pe un singur core? **DA**

Notiuni cu care vom lucra

Race condition

Thread 1	Thread 2
a = a + 2	**a = a + 2**

Care este valoare lui a în acest moment?

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

Care este valoare lui a în acest moment?

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

What is the value of a?

4

Race condition

a = 0

Thread 1	Thread 2
a = a + 2	**a = a + 2**

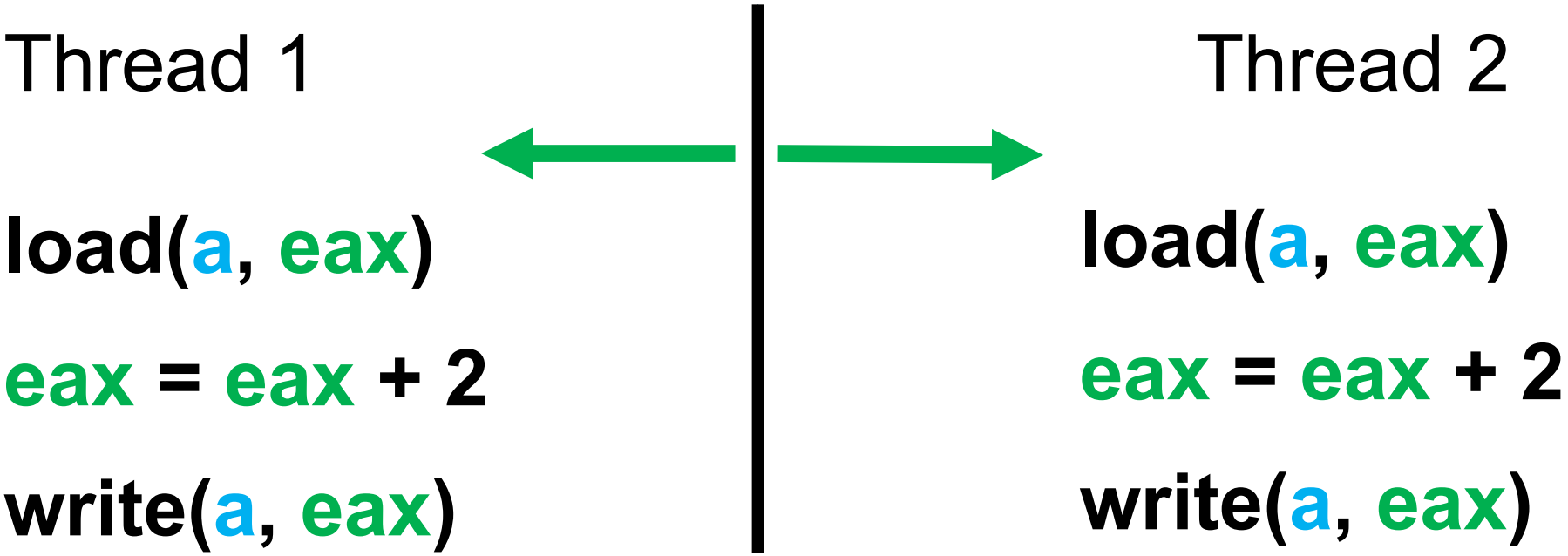
What is the value of a?

4 **AND** 2

Sa ne uitam in detaliu

Race condition

a = 0

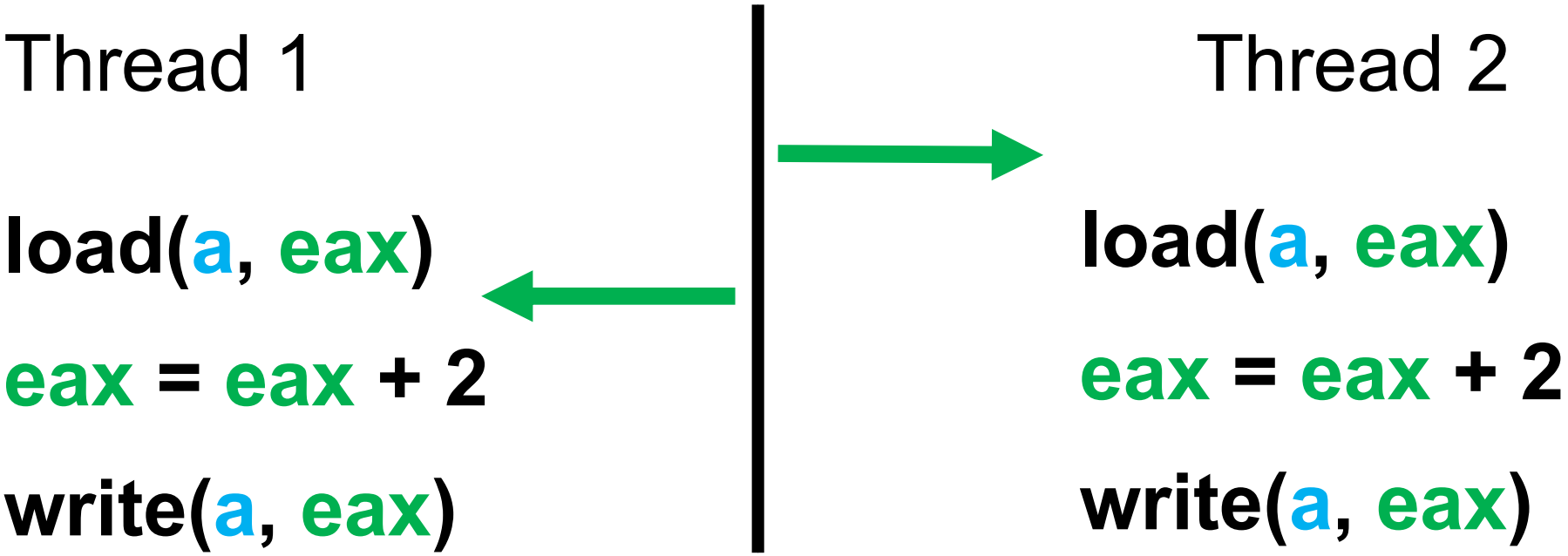


eax =

eax =

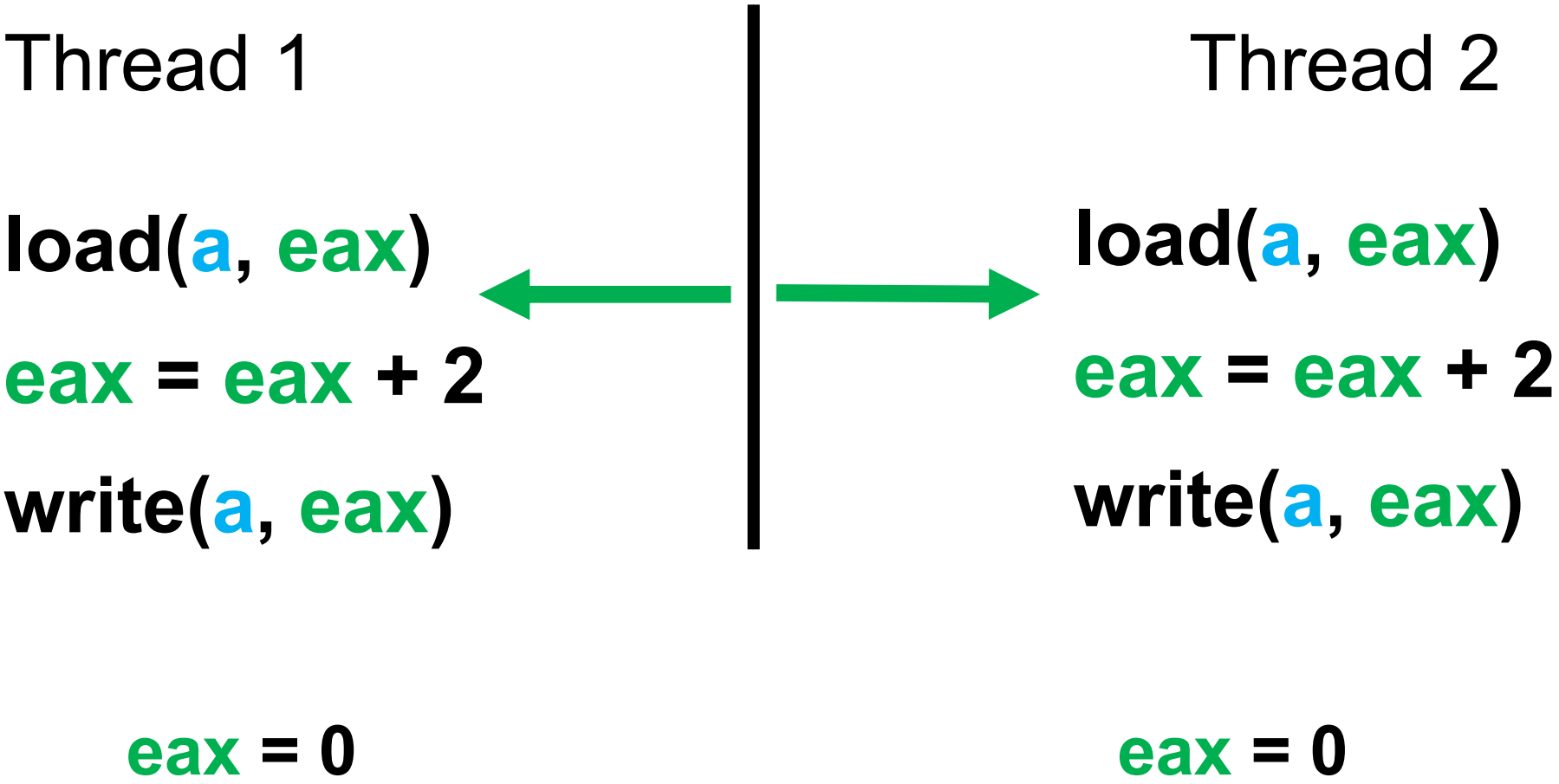
Race condition

a = 0



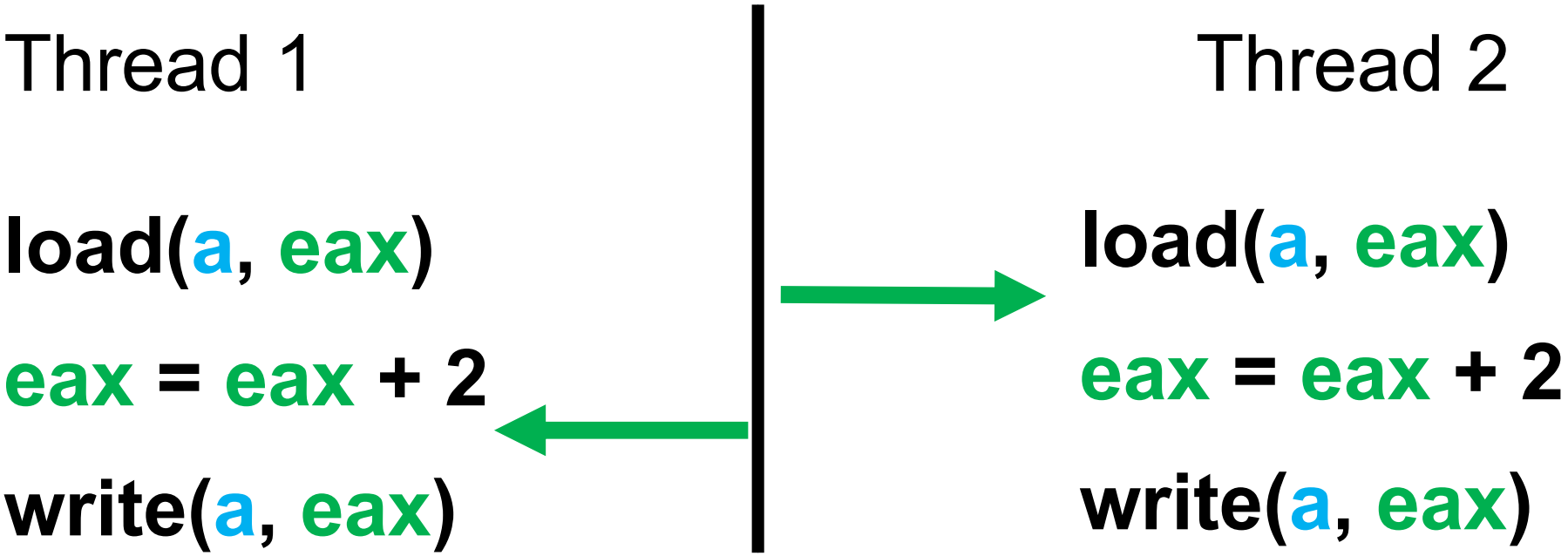
Race condition

a = 0



Race condition

a = 0

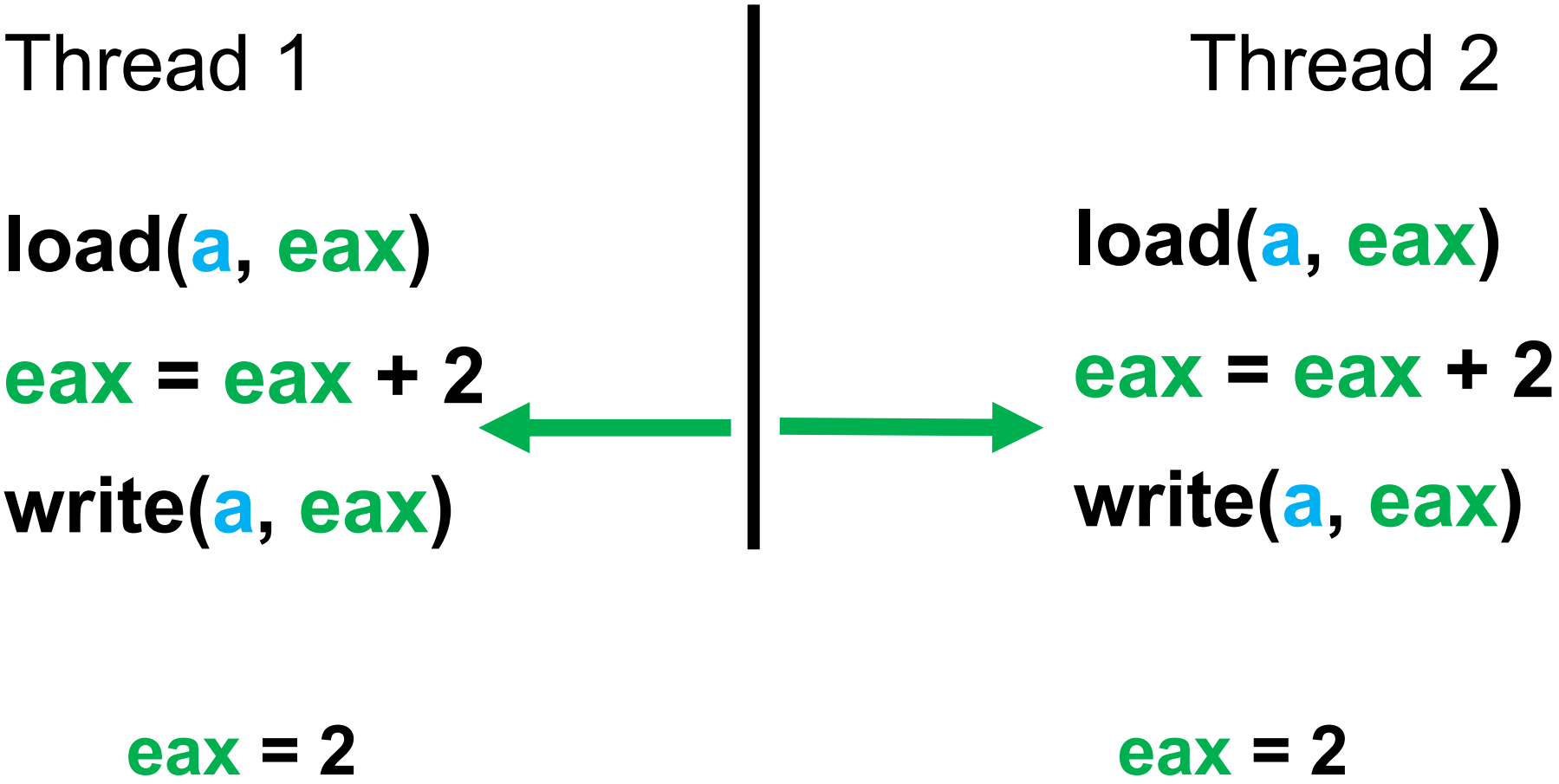


eax = 2

eax = 0

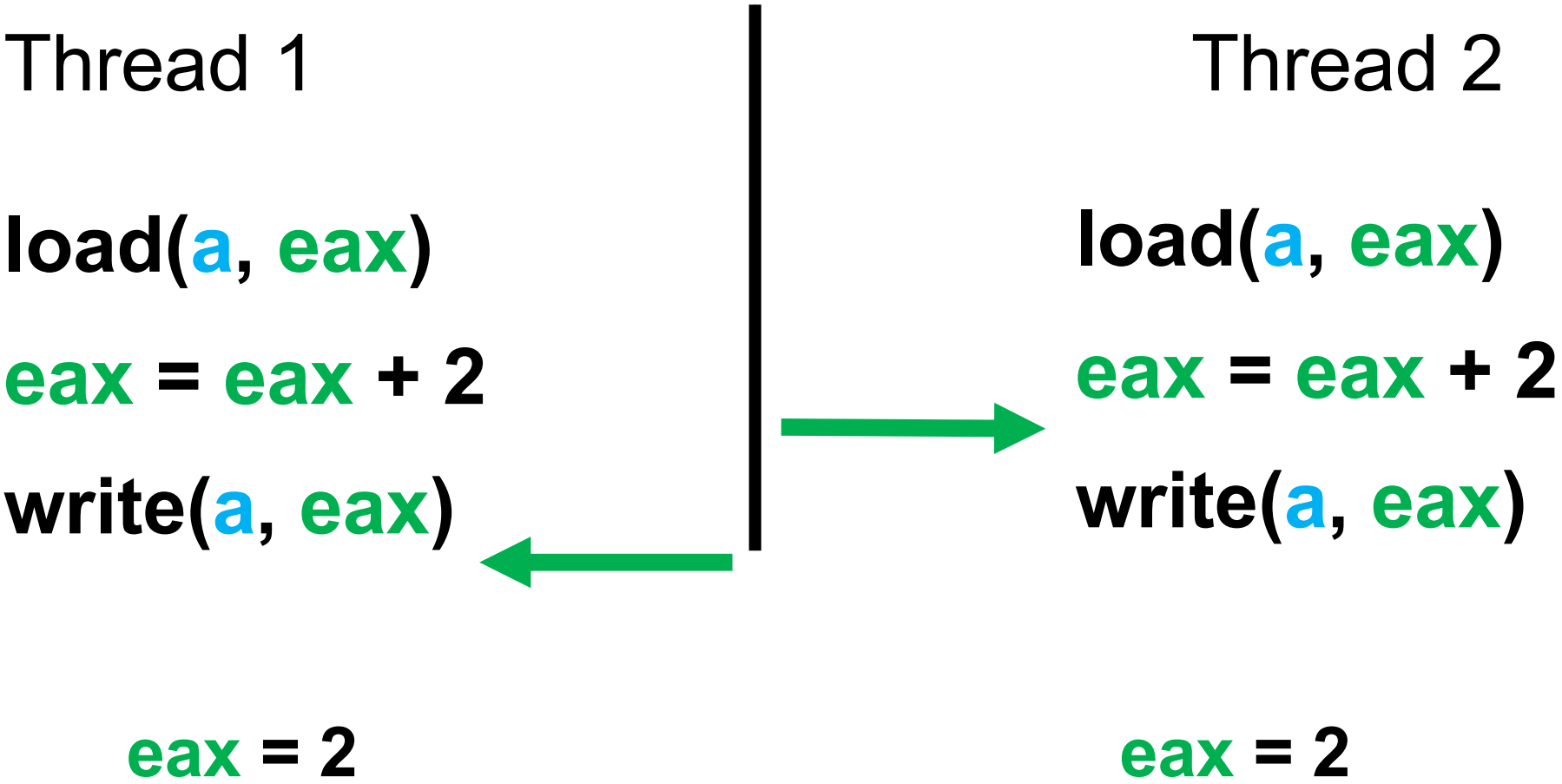
Race condition

a = 0



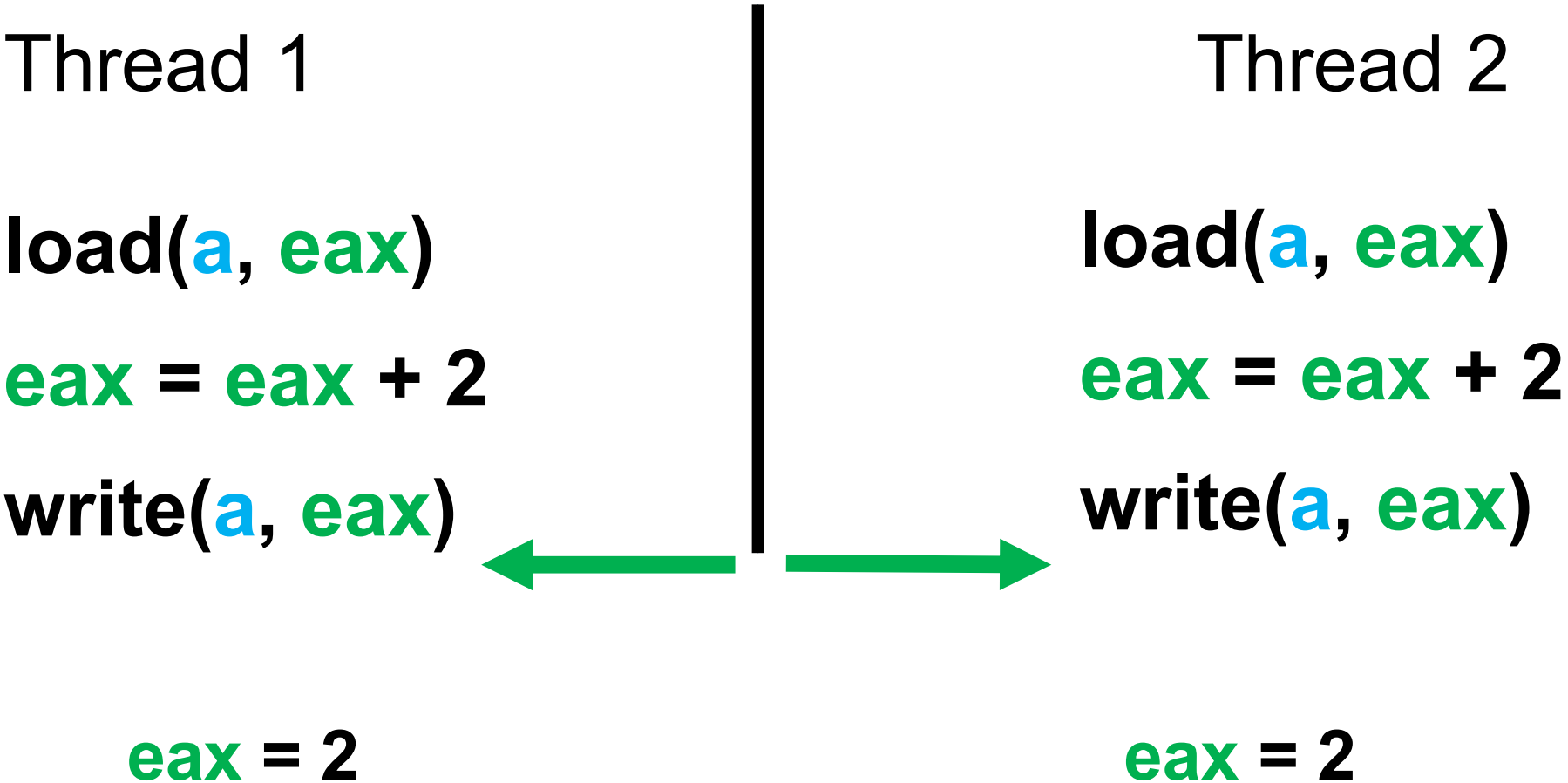
Race condition

a = 2



Race condition

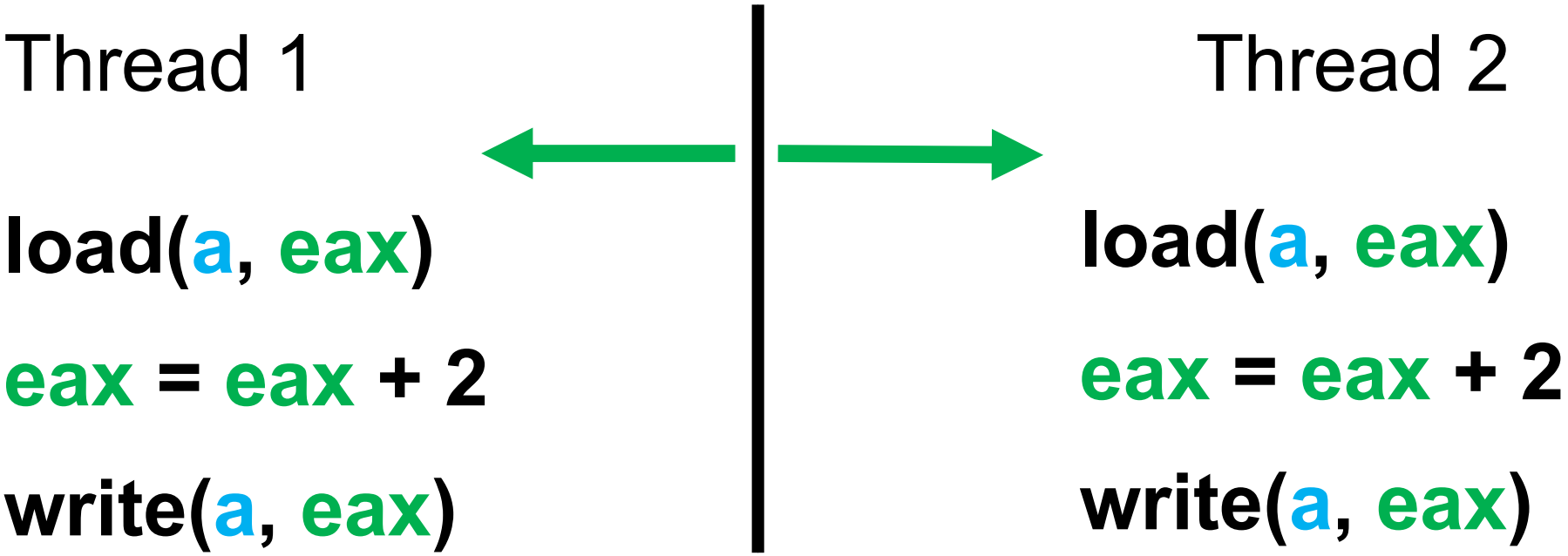
a = 2



Un pas si mai departe...

Race condition

a = 0

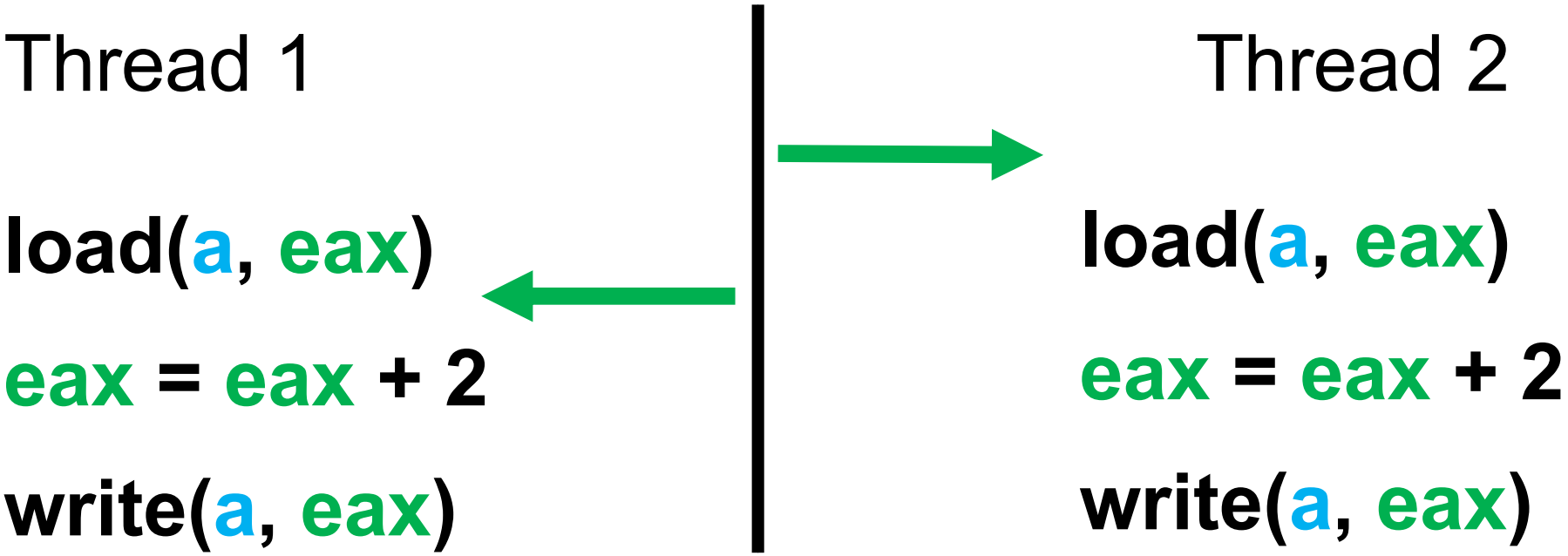


eax =

eax =

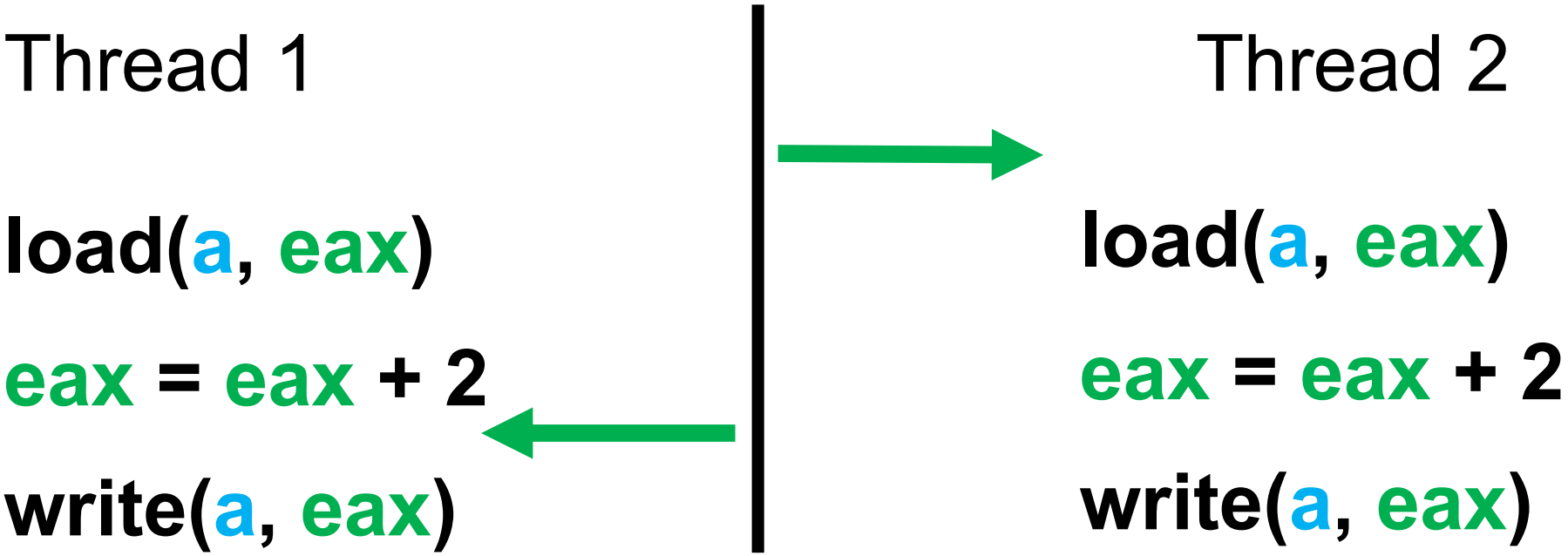
Race condition

a = 0



Race condition

a = 0

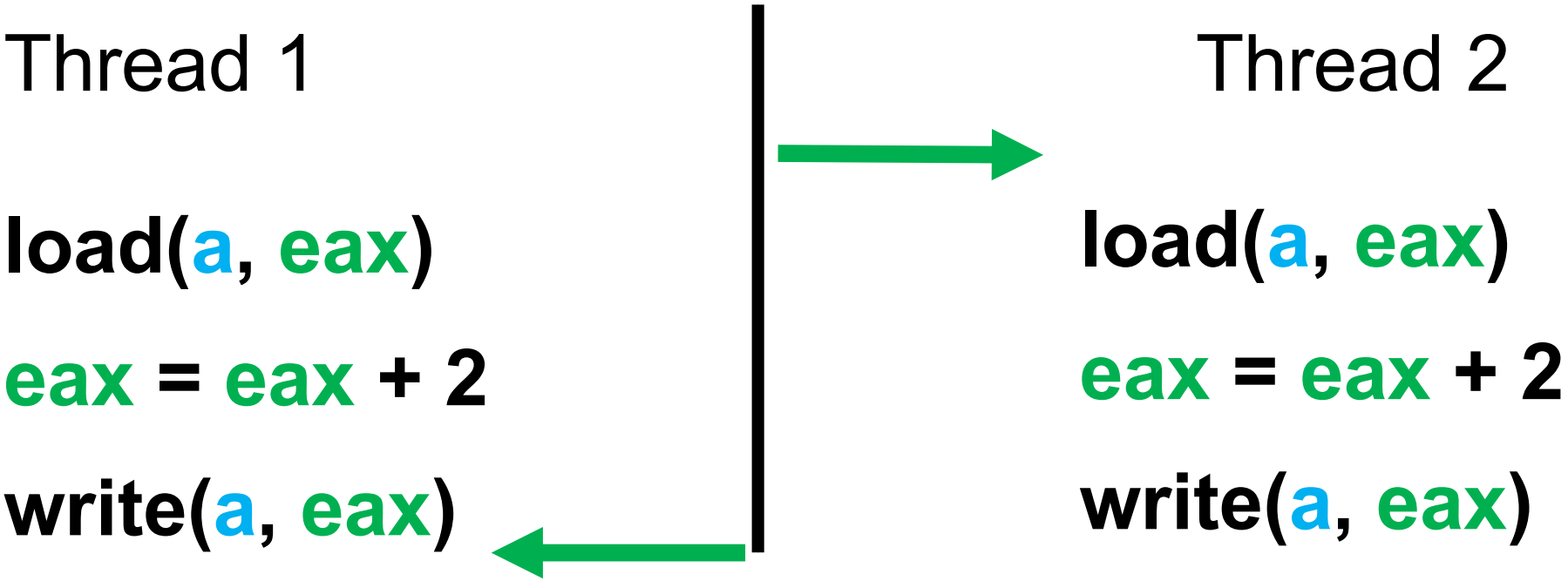


eax = 2

eax =

Race condition

a = 2

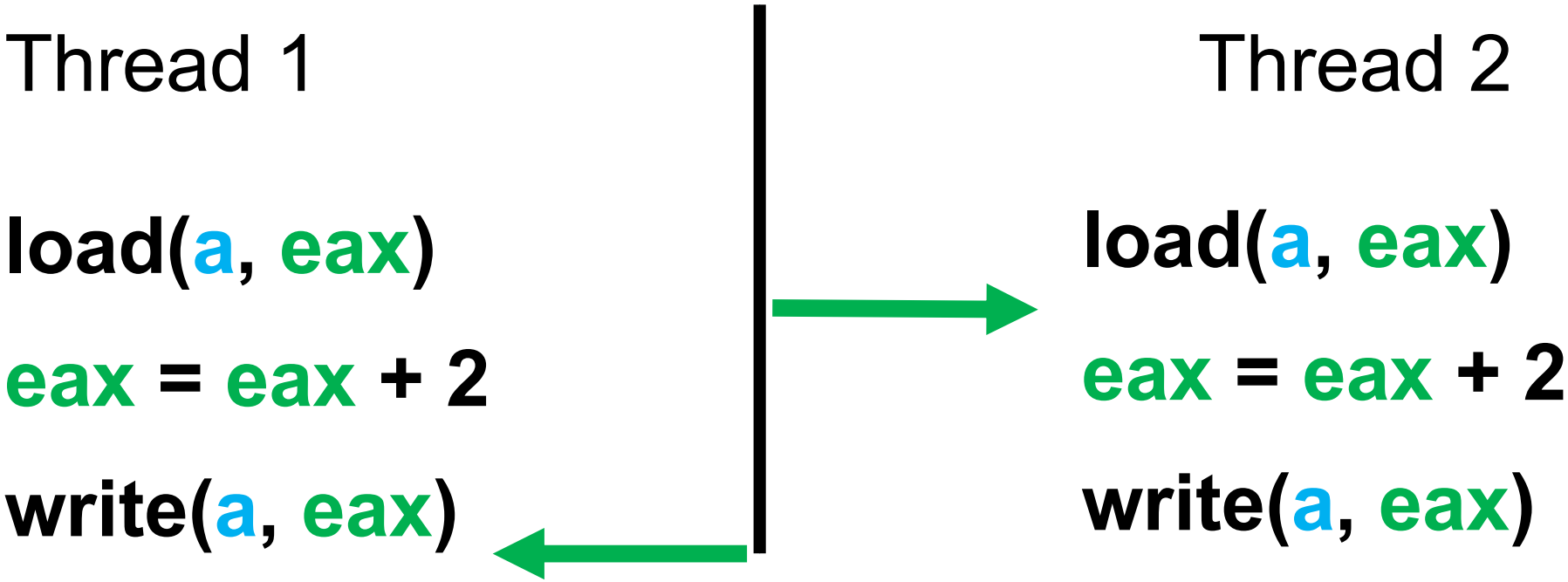


eax = 2

eax =

Race condition

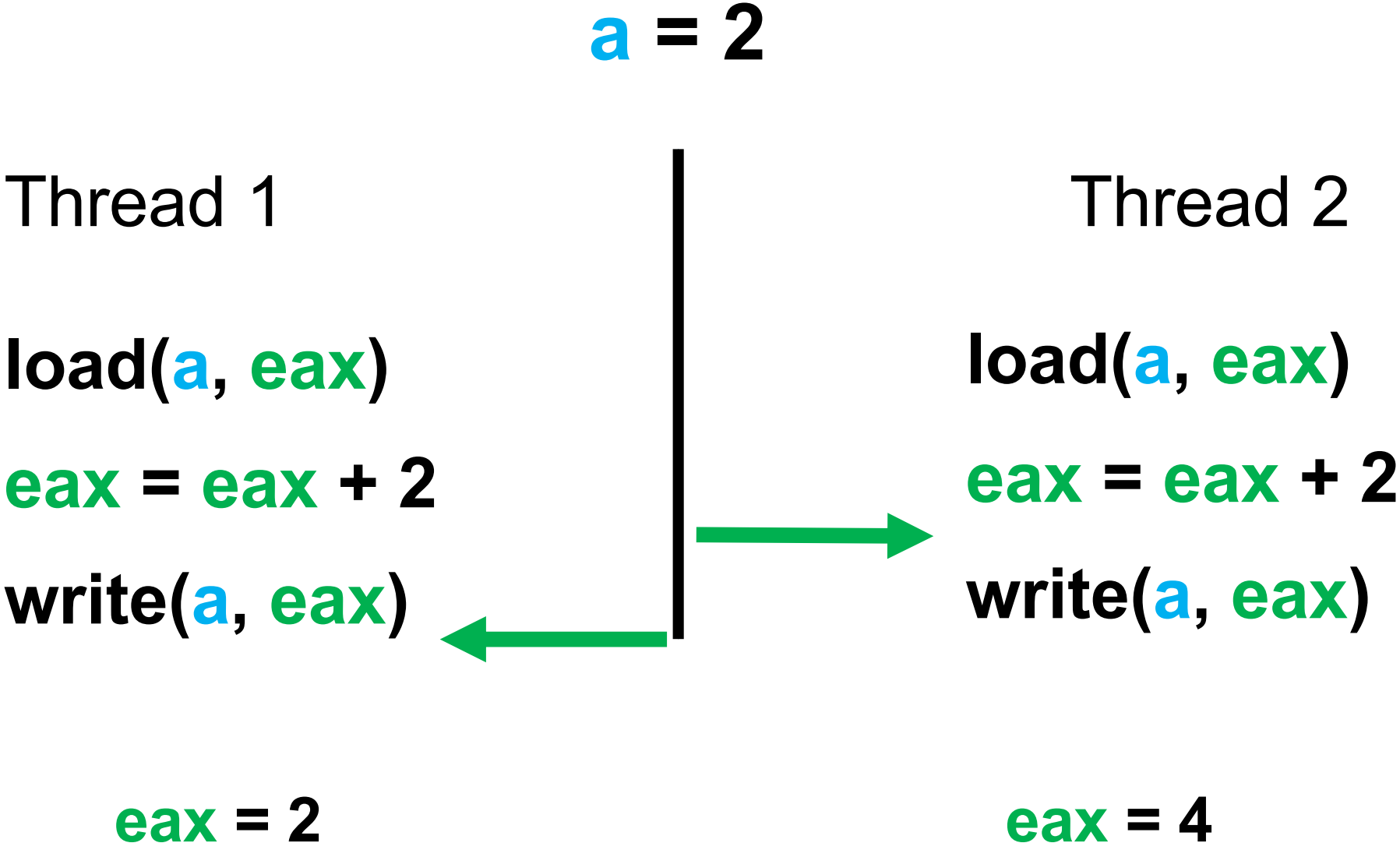
a = 2



eax = 2

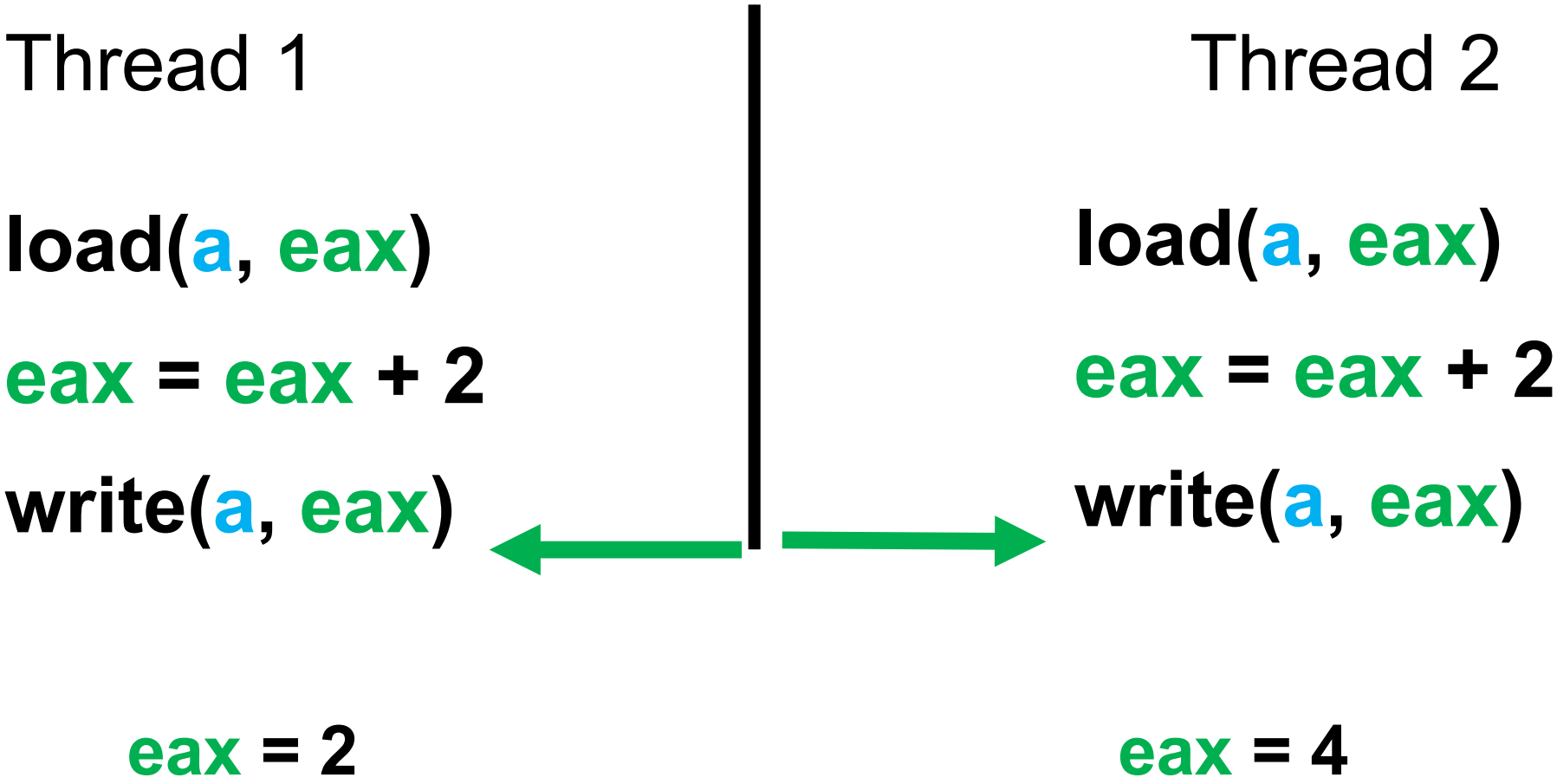
eax = 2

Race condition



Race condition

a = 4



Primitive de sincronizare

Synchronization primitives

- Atomics
- Semaphore
  - Binary semaphore (Mutex)
  - Critical section
- Barrier

Atomics

- Fie operația de 64 biți pe un procesor de 64 biți **C = A + B**

```
load(A, eax)
load(B, ebx)
eax = eax + ebx
write(C, eax)
```

Atomics

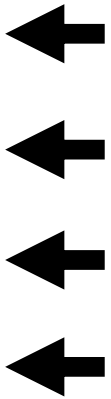
- Fie operația de 64 biți pe un procesor de 32 biți C = A + B

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C** = **A** + **B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)
```



Putem avea doar jumătate de **C** modificat

Atomics

- Fie operația de 64 biți pe un procesor 32 biți **C** = **A** + **B**

```
load(A[0], eax)
load(B[0], ebx)
eax = eax + ebx
write(C[0], eax)   <-
load(A[1], eax)
load(B[1], ebx)
eax = eax + ebx
write(C[1], eax)   <-
```

Atomicitatea asigură că **C** va fi vizibil doar complet modificat, sau complet nemodificat.

Excludere mutuală - mutex

Mutual exclusion – soluția lui Dekker

EWD35 - 1

EWD35.htm

About the sequentiality of process descriptions.

Over de sequentialiteit van procesbeschrijvingen.

Het is niet ongebruikelijk, wanneer een spreker zijn
een inleiding. Omdat ik mij hier mogelijk richt tot een g
minder vertrouwd is met de problematiek, die ik wil aansn
die ik zal moeten gebruiken, wilde ik in dit geval ter in
inleidingen houden, nl. een om de achtergrond van de prob
een tweede, om U een gevoel te geven voor de aard van de
wij tegen het lijf zullen lopen.

Theodorus (Dirk) J. Dekker

Mutual exclusion – soluția lui Dijsktra

Solution of a Problem in Concurrent Programming Control

E. W. Dijkstra
Technological University, Eindhoven, The Netherlands

A number of mainly independent sequential-cyclic processes with restricted means of communication with each other can be made in such a way that at any moment one and only one of them is engaged in the "critical section" of its cycle.

Introduction

Given in this paper is a solution to a problem for which, to the knowledge of the author, has been an open question since at least 1962, irrespective of the solvability. The paper consists of three parts: the problem, the solution, and the proof. Although the setting of the problem might seem somewhat academic at first, the author trusts that anyone familiar with the logical problems that arise in

computer can only request one one-way message at a time. And only this w
this problem is f

The Solution

The common
"Boo
The integer
will only be set
by the others. It
well outside the
mentioned set to
The program

```
"integer j;
Li0:  b[i] := fals
Li1:  if k ≠ i th
Li2:  begin c[i] :
Li3:  if b[k] then
      go to Li1
      end
        else
```

Soluția lui Dekker

```
wants_to_enter[0] = true
while (wants_to_enter[1]) {
        if (turn != 0) {
                wants_to_enter[0] = false
                while (turn != 0) { // busy wait }
                wants_to_enter[0] = true
        }
}
// critical section ...
turn = 1
wants_to_enter[0] = false
```

Soluția lui Dijsktra

```
b[i] = fals
while(sw[i]) {
        sw[i] = F
        if (k!=i) {
                c[i] = true
                if(b[k])
                        k = i
                sw[i] = T
        } else {
        c[i] = false
        for(j=0;j<N;j++)
                if(j!=i && !c[j])
                        sw[i] = T
        }
 }
//critical
b[i] = true
c[i] = true
```

lock() - P

unlock() - V

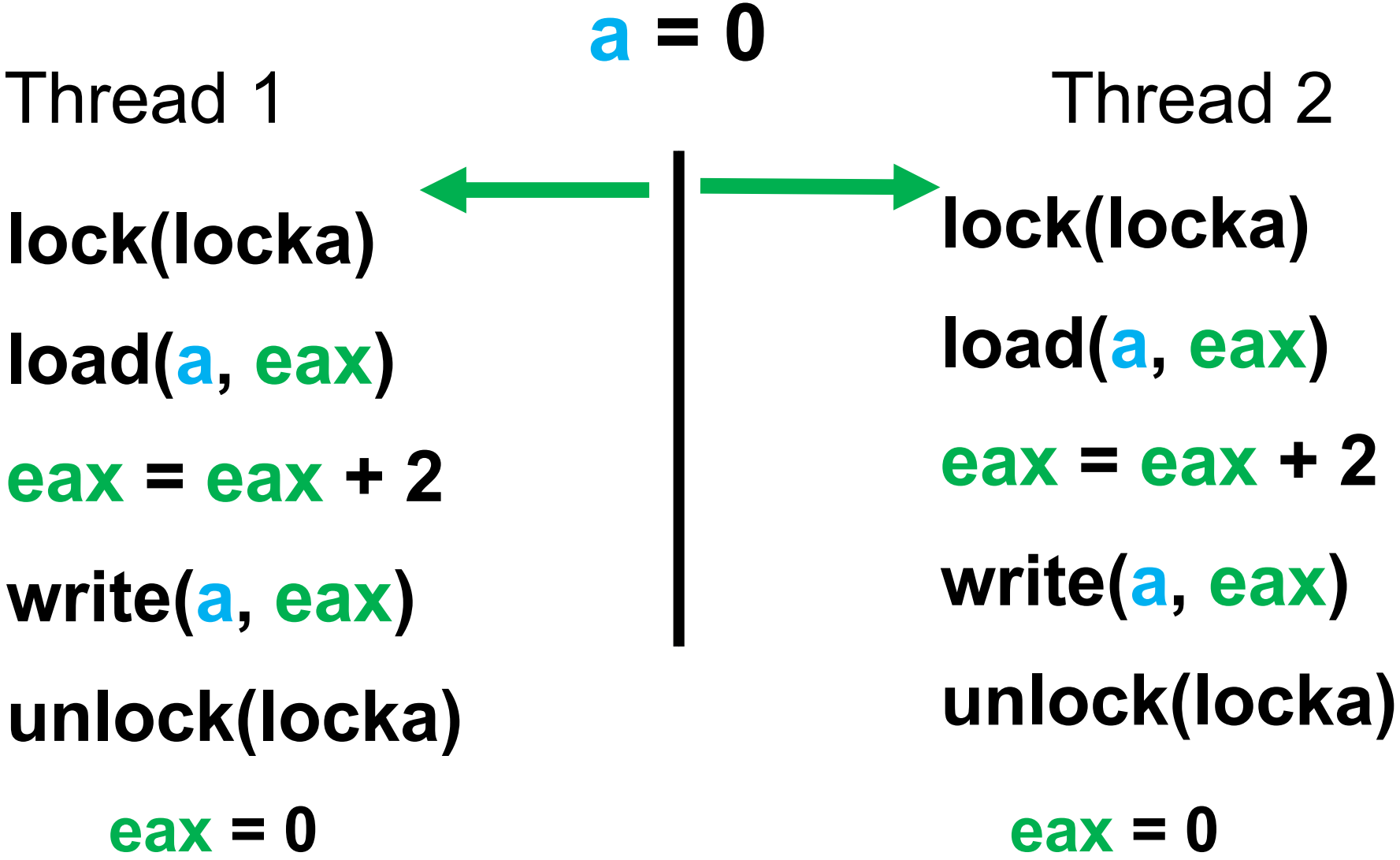
Soluția lui Peterson

```
do {
        // entry condition
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j) { // busy wait }
        // critical section...
        // exit condition
        flag[i] = false;
        // remainder section
} while (true);
```

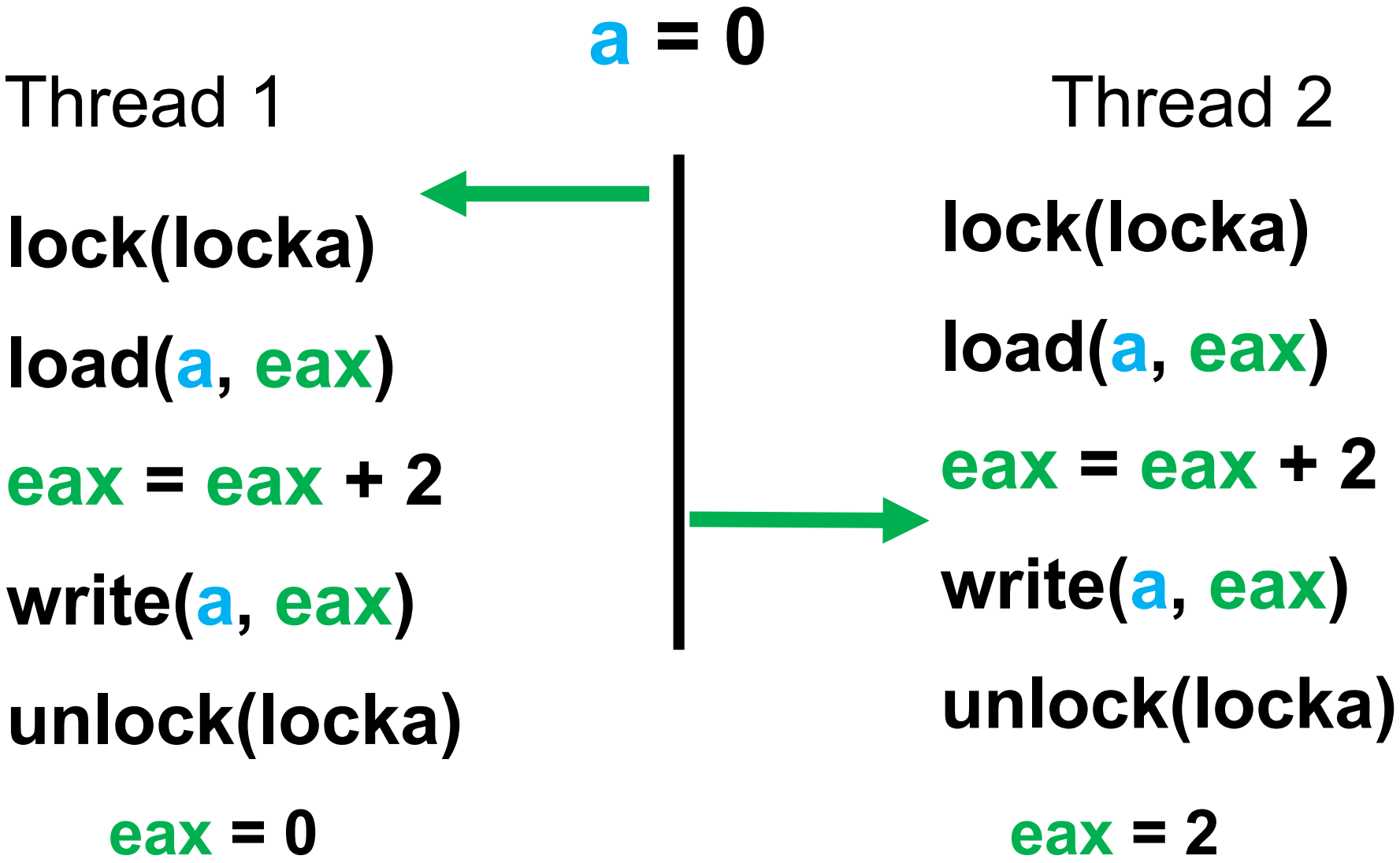
Soluție cu asistare hardware

```
while (test_and_set(lock));
// critical section
lock = 0
```

Race condition – soluție



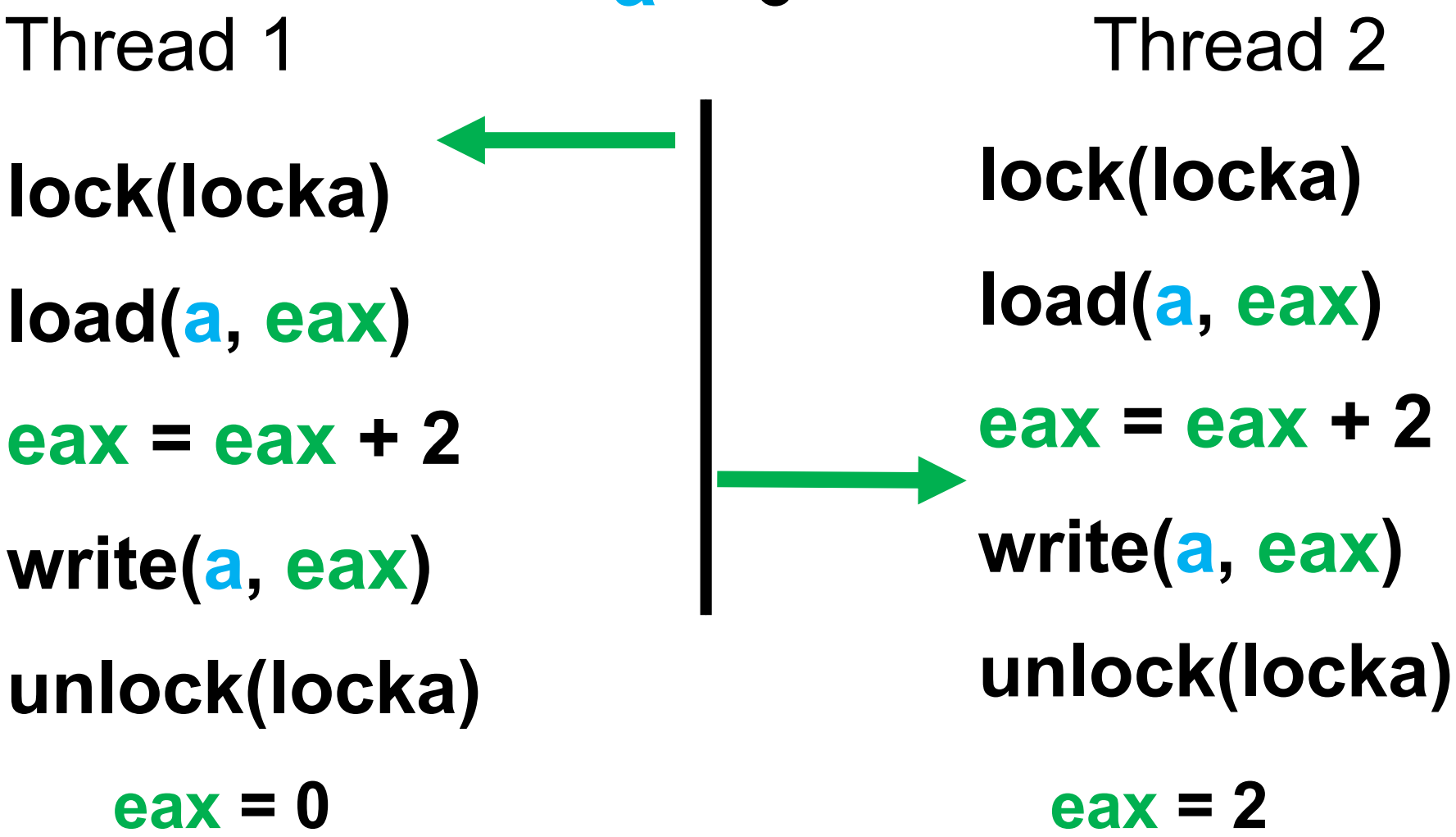
Race condition – soluție



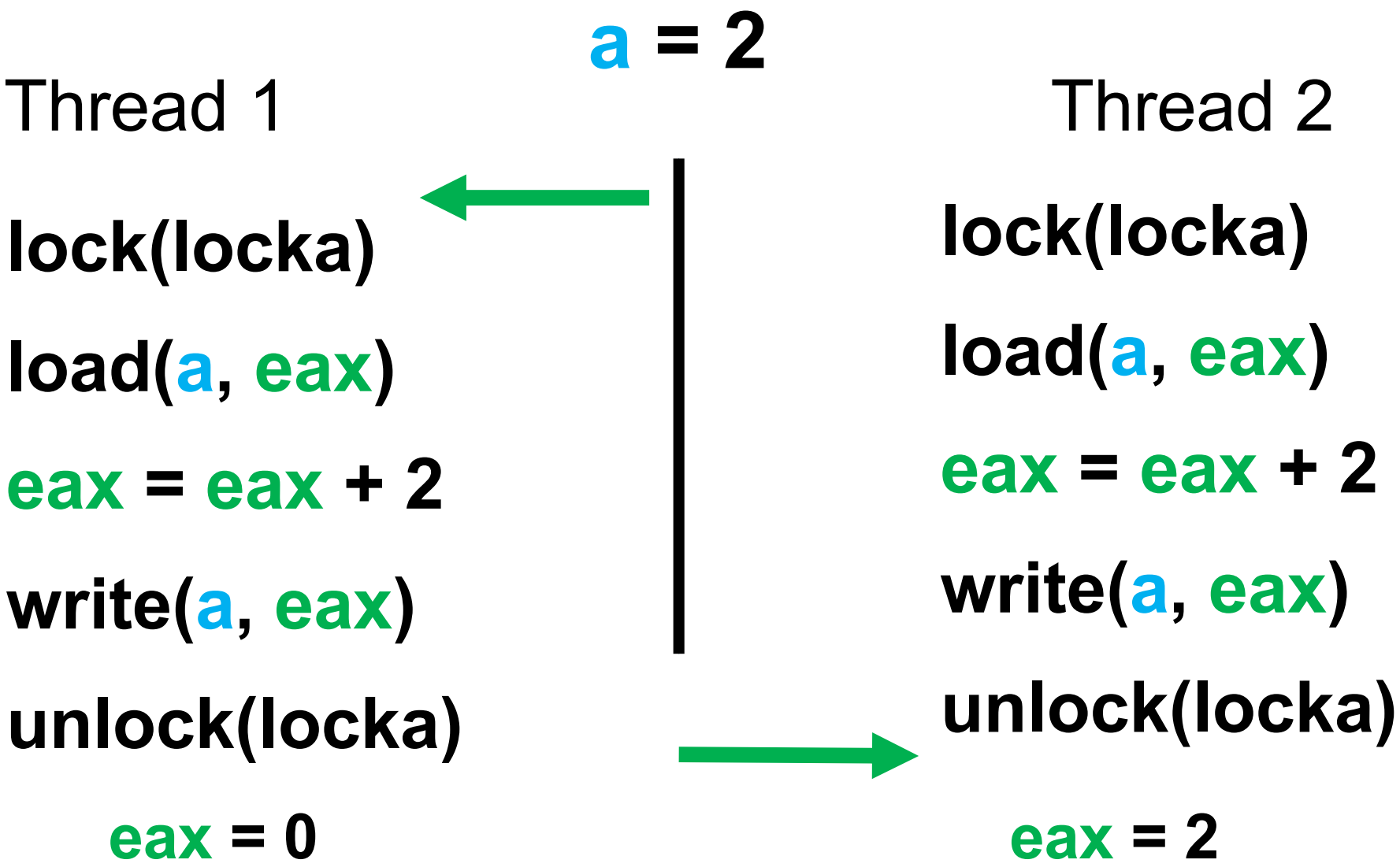
Race condition – soluție

OK

a = 0



Race condition – soluție



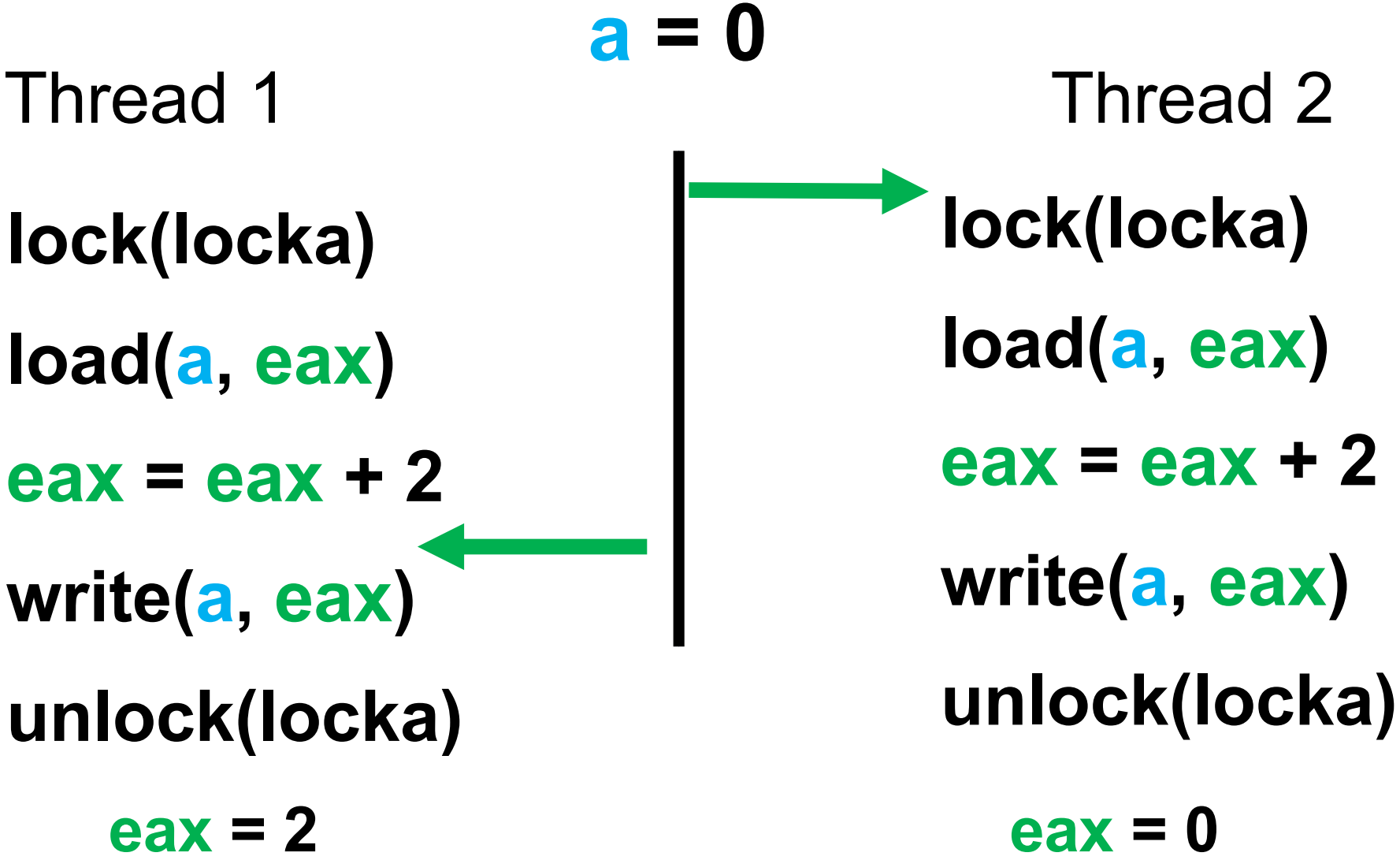
Race condition – soluție

OK

a = 2

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 0	eax = 2

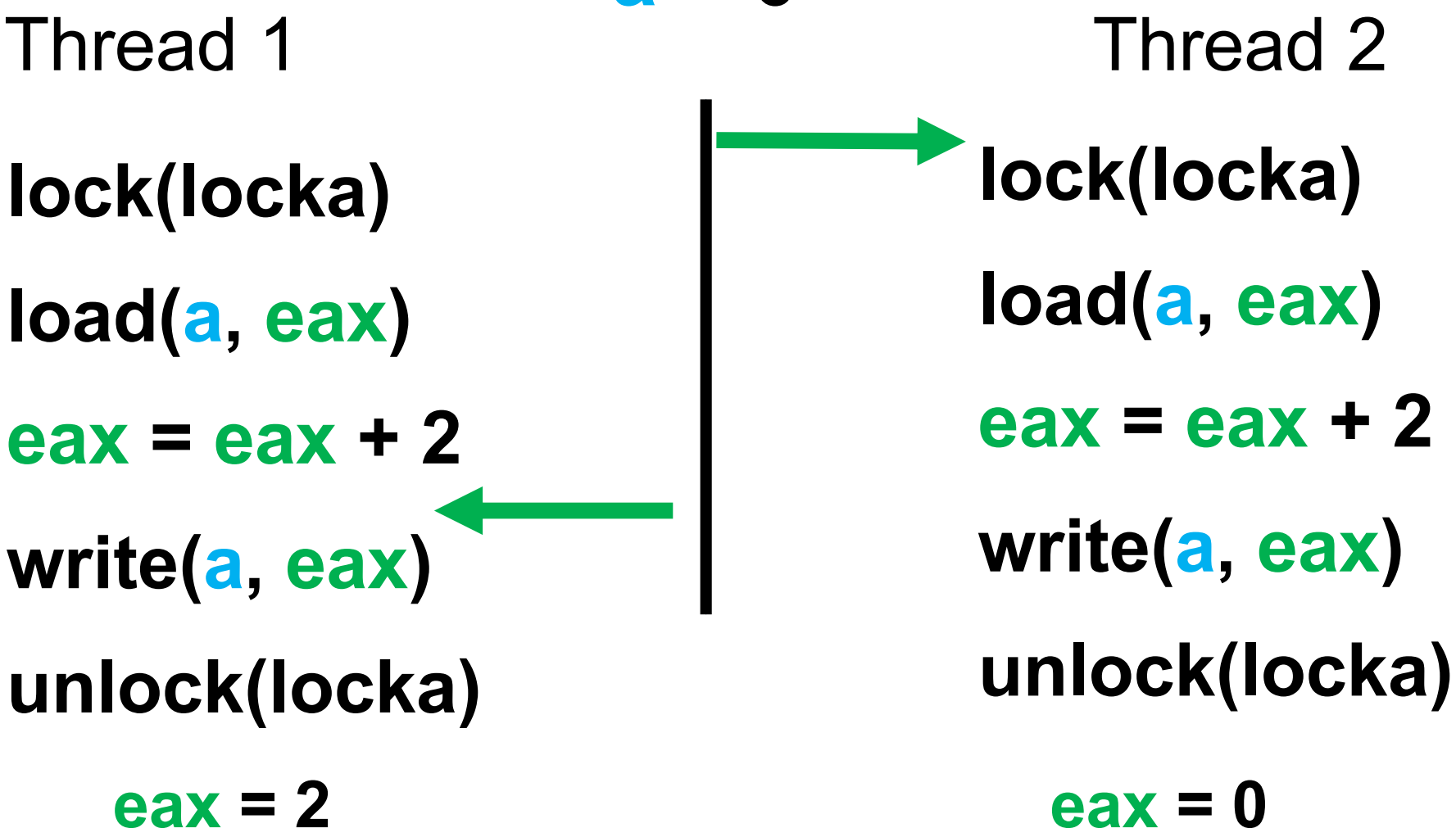
Race condition – soluție



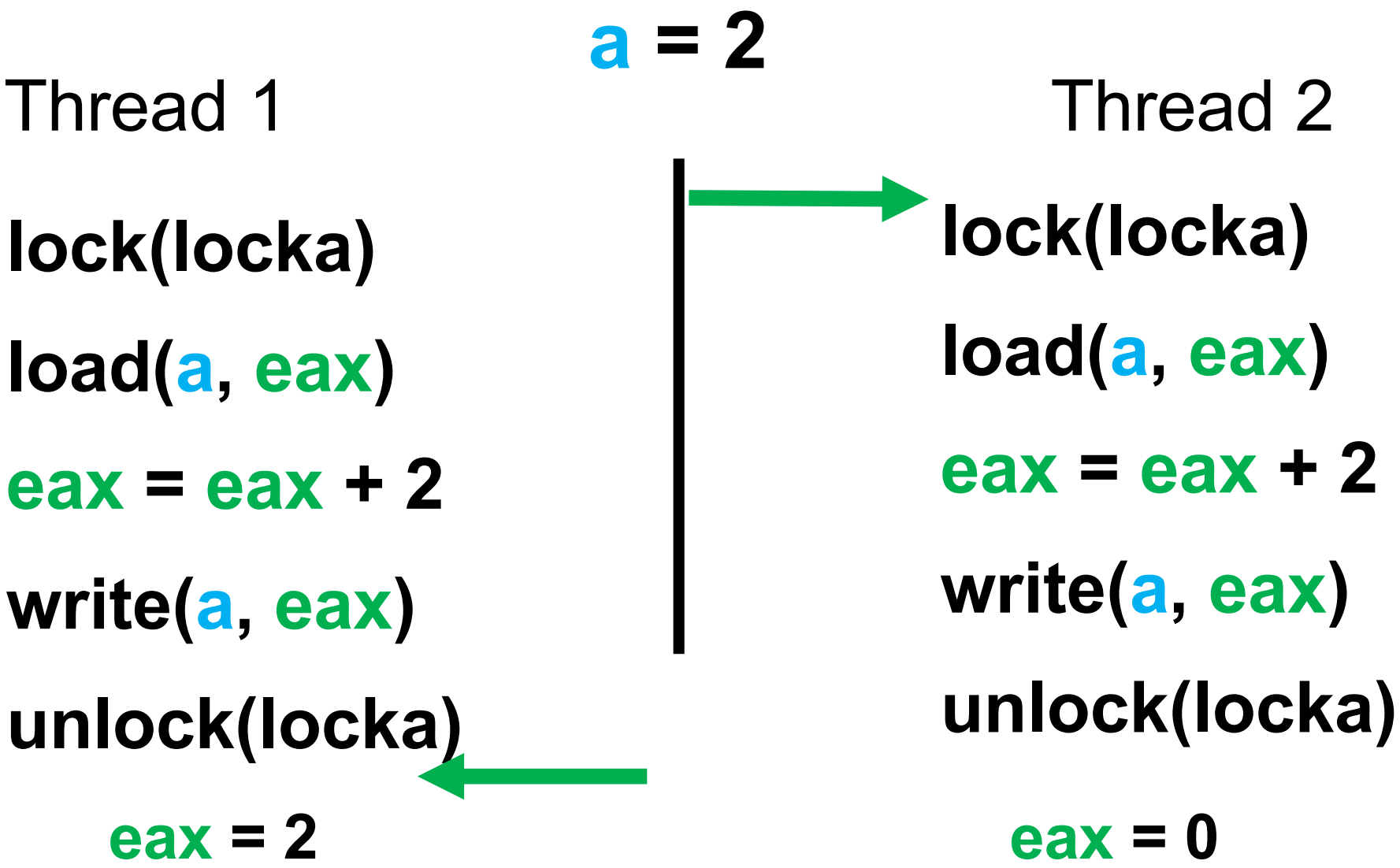
Race condition – soluție

OK

a = 0



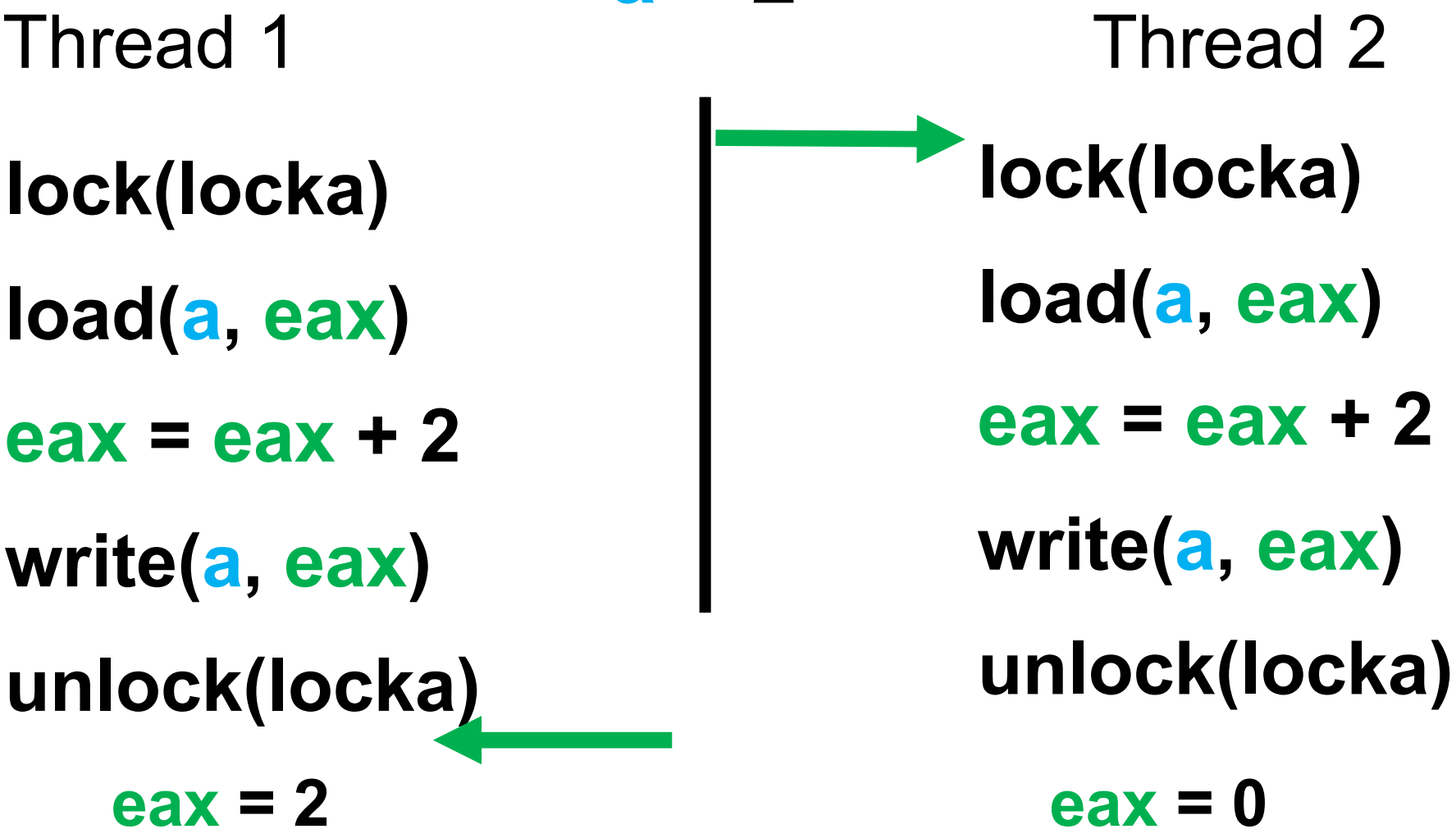
Race condition – soluție



Race condition – soluție

OK

a = 2



Race condition – soluție

a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 2

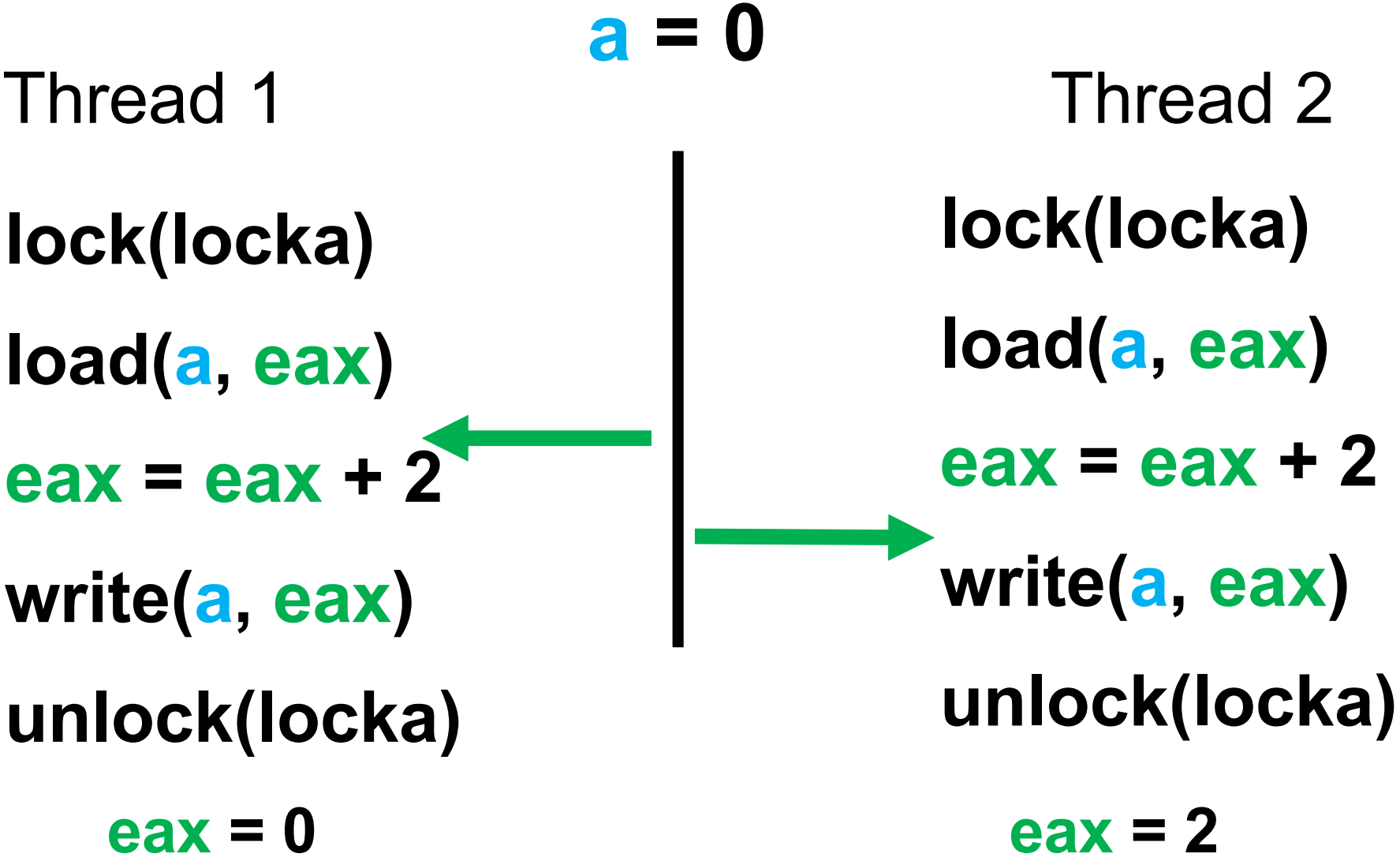
Race condition – soluție

OK

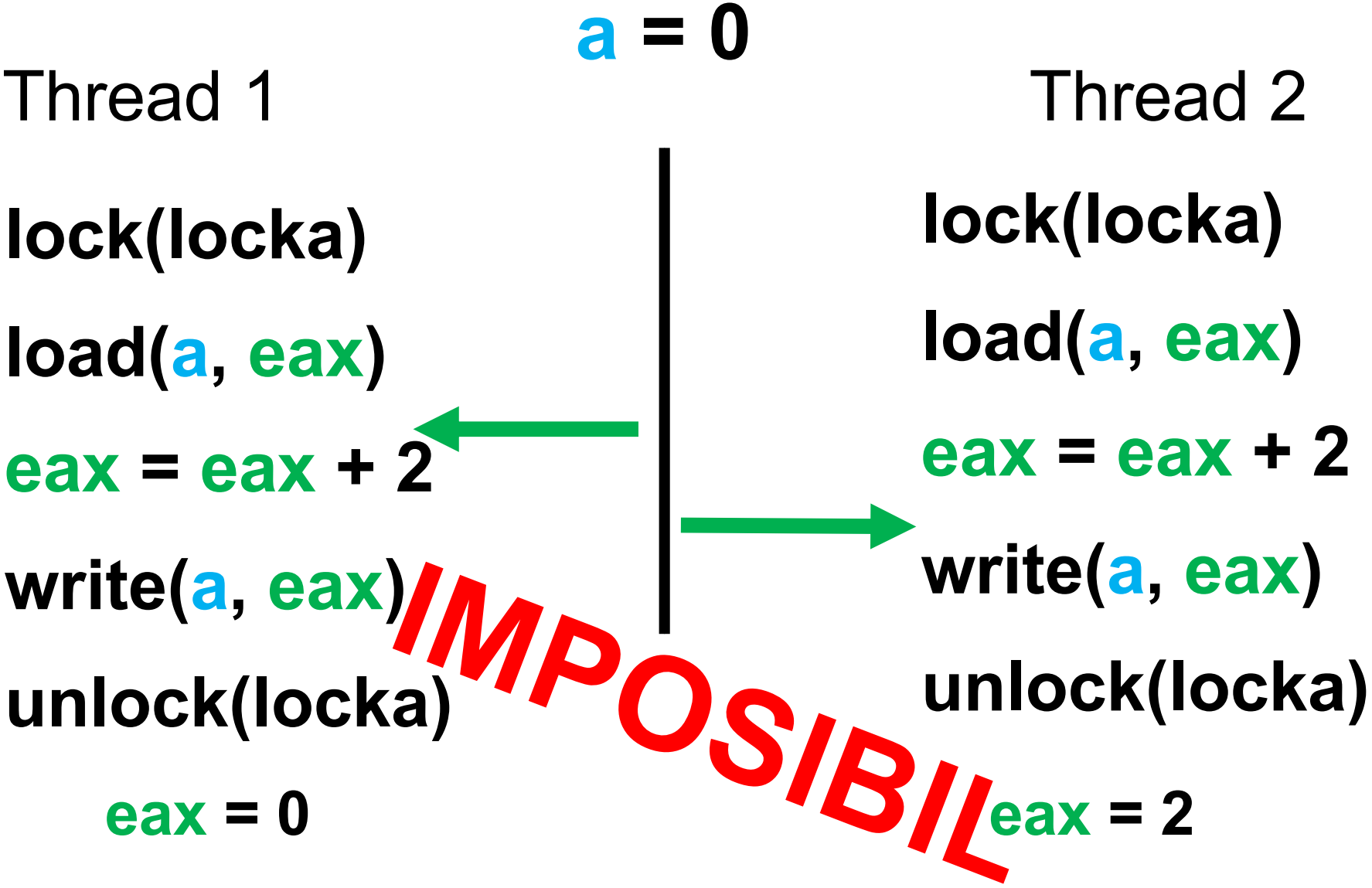
a = 4

Thread 1	Thread 2
lock(locka)	lock(locka)
load(a, eax)	load(a, eax)
eax = eax + 2	eax = eax + 2
write(a, eax)	write(a, eax)
unlock(locka)	unlock(locka)
eax = 2	eax = 2

Race condition – soluție

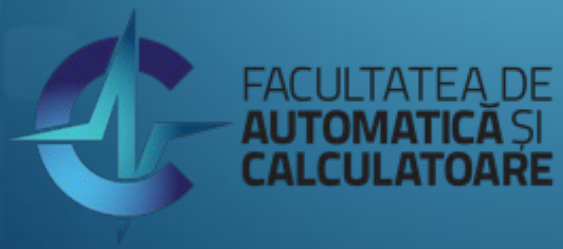


Race condition – soluție



Algoritmi Paraleli și Distribuiți
Complexitate algoritmi paraleli

Prof. Ciprian DOBRE
ciprian.dobre@upb.ro



Performanța

- Timp de execuție
- Memorie ocupată
- Număr de procese (thread-uri)
- Scalabilitate
- Toleranță la defecte
- Cost
- Eficiență
- Fiabilitate
- Portabilitate

Măsuri

- T - Timpul total necesar execuției programului paralel
- P - Numărul de procesoare utilizate
- G – Timp execuție **cel mai rapid algoritm secvențial**
- S – Speedup
  - $S = \frac{G}{T}$

- Costul $C = T * P$

- Eficienţa $E = \frac{G}{C} = \frac{G}{TP} = \frac{S}{P}$

Complexitate

* 3

Complexitate secvențială?

Complexitate

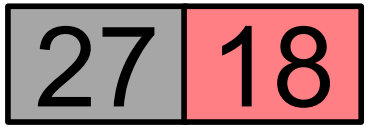
* 3

27

Complexitate secvențială?

Complexitate

* 3



Complexitate secvențială?

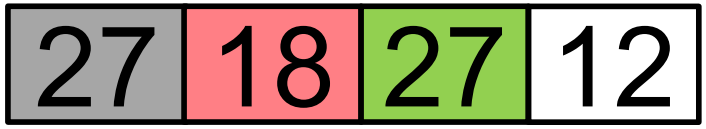
Complexitate

* 3

Complexitate secvențială?

Complexitate

* 3



Complexitate secvențială?

Complexitate

* 3

Complexitate secvențială? O(**N**)

Complexitate

* 3

Complexitate paralelă?

Complexitate

* 3

Complexitate paralelă? **O(1)** ?

Complexitate

* 3

Complexitate paralelă? **O(1)** ? **P** = **N**

Complexitate

* 3

Complexitate paralelă? $\mathbf{O}(\frac{N}{P})$

Complexitate

Speedup?

Complexitate

Speedup?

T = O($\frac{N}{P}$)

Complexitate

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

Complexitate

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})}$$

Complexitate

Speedup?

T = O($\frac{N}{P}$)

G = O(N)

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Complexitate

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Eficiența?

T = O($\frac{N}{P}$)

G = O(N)

Complexitate

$$S = \frac{O(N)}{O(\frac{N}{P})} = O(P)$$

Eficiența?

$$E = \frac{S}{P} = \frac{O(P)}{P} = 1$$

T = O($\frac{N}{P}$)

G = O(N)

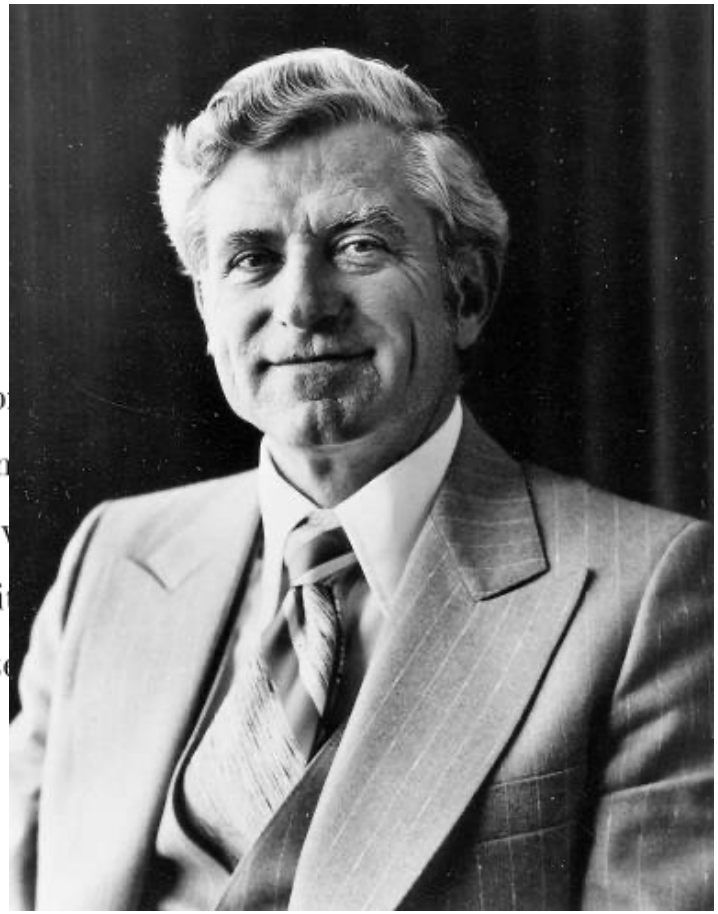
Legea lui Amdhal

Amdahl's law

Validity of the single processor approach to achieving large scale computing capabilities[1]

Gene M. Amdahl

IBM Sunnyvale, California



1 INTRODUCTION

For over a decade prophets have voiced the contention that the o
has reached its limits and that truly significant advences can be m
multiplicity of computers in such a manner as to permit cooperati
direction has been pointed out as general purpose computers wi
of memories, or as specialized computers with geometrically relat
controlled by one or more instruction streams.

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

$$S = \frac{G}{T}$$

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

$$S = \frac{G}{T} = \frac{G}{fG + \frac{(1-f)G}{P}} = \frac{1}{f + \frac{(1-f)}{P}}$$

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Legea lui Amdahl

$$S = \frac{1}{f + \frac{(1-f)}{P}}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{S \leq \frac{1}{f}}$$

Legea lui Amdahl

f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

Legea lui Amdahl

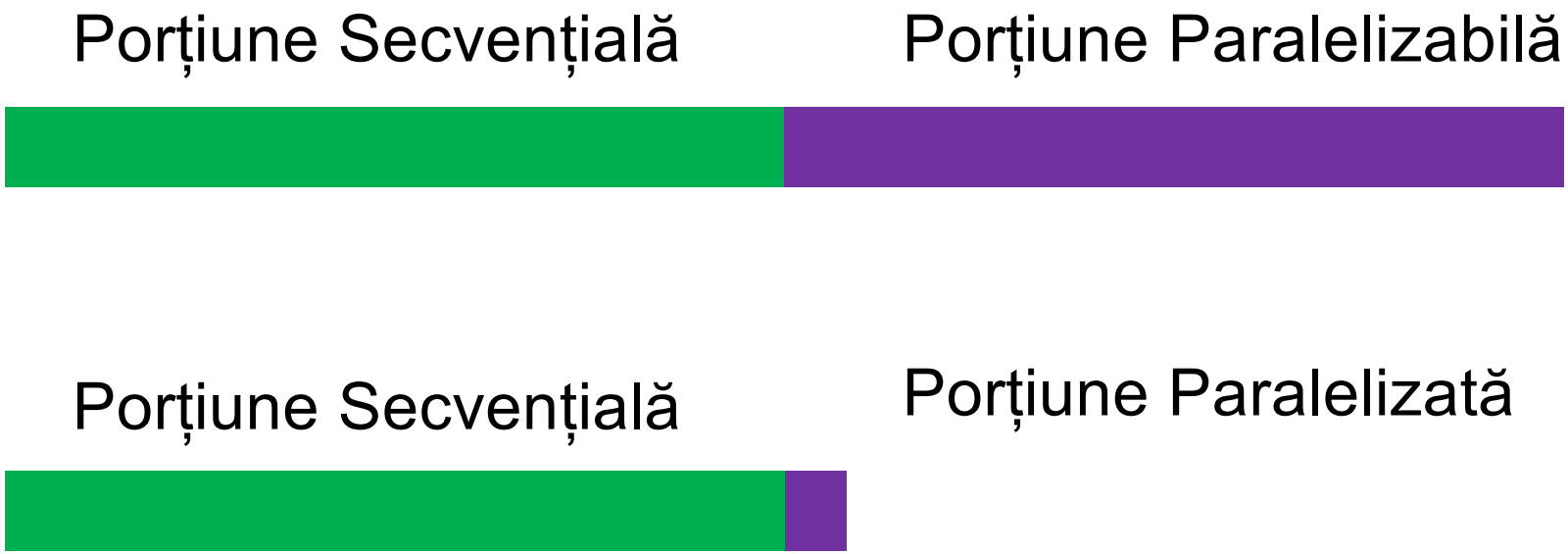
f - Procent de operații din algoritmul secvențial care **NU** se pot executa paralel.

$$T = fG + (1 - f)\frac{G}{P}$$

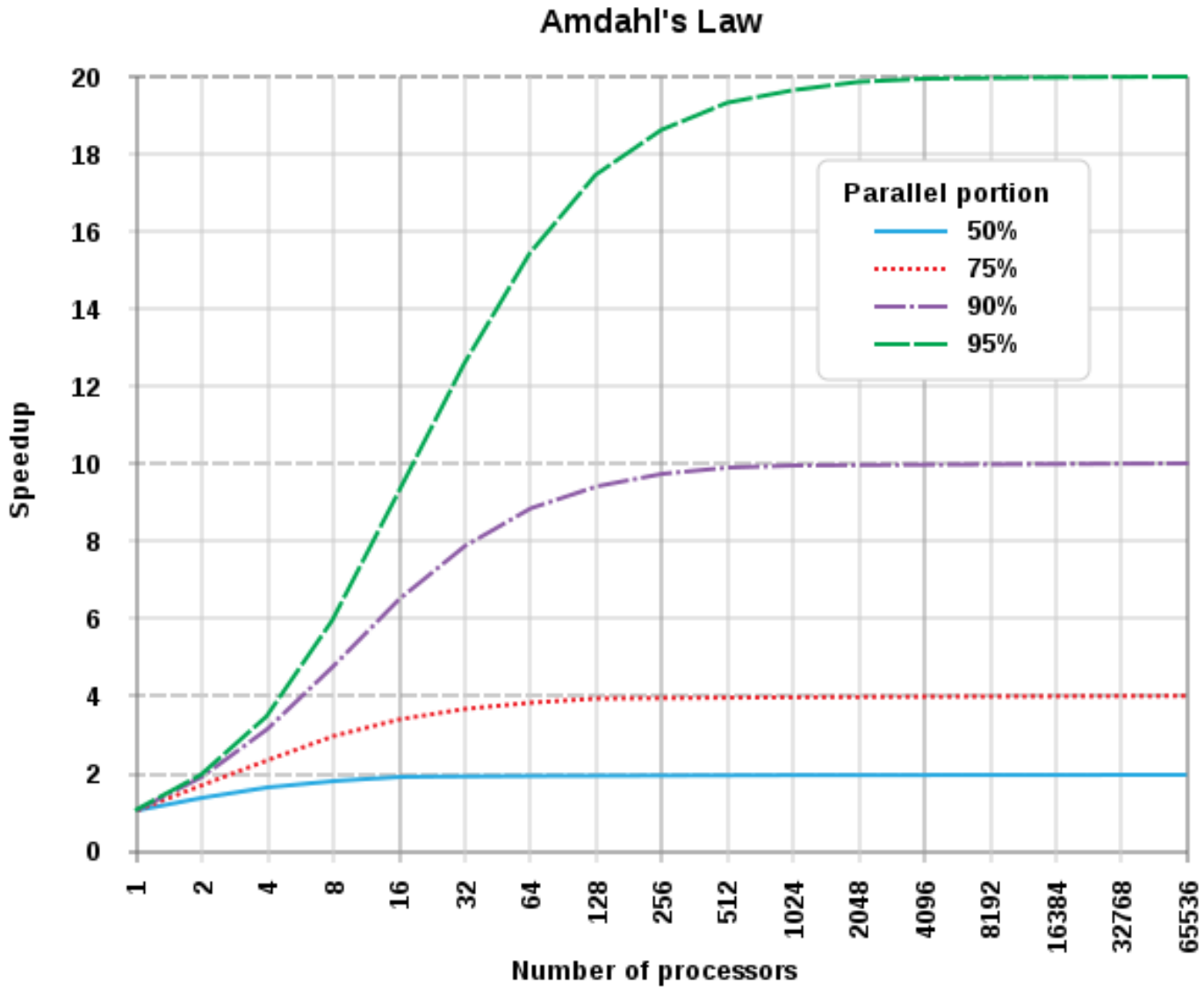
Ce se întâmplă dacă P e foarte mare (chiar mai mare decât N)?

$$\boldsymbol{T \geq fG}$$

Legea lui Amdahl

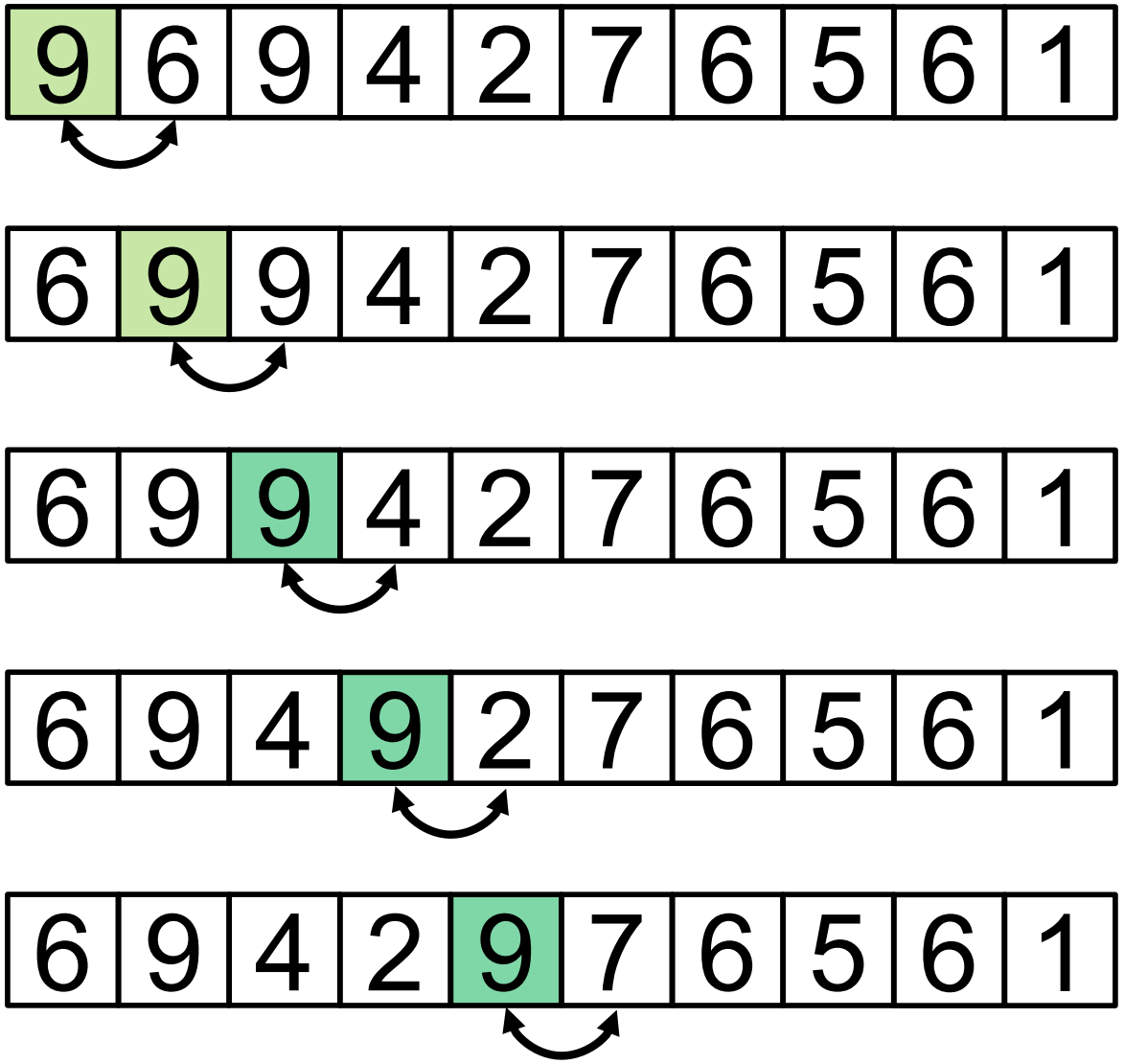


Legea lui Amdahl

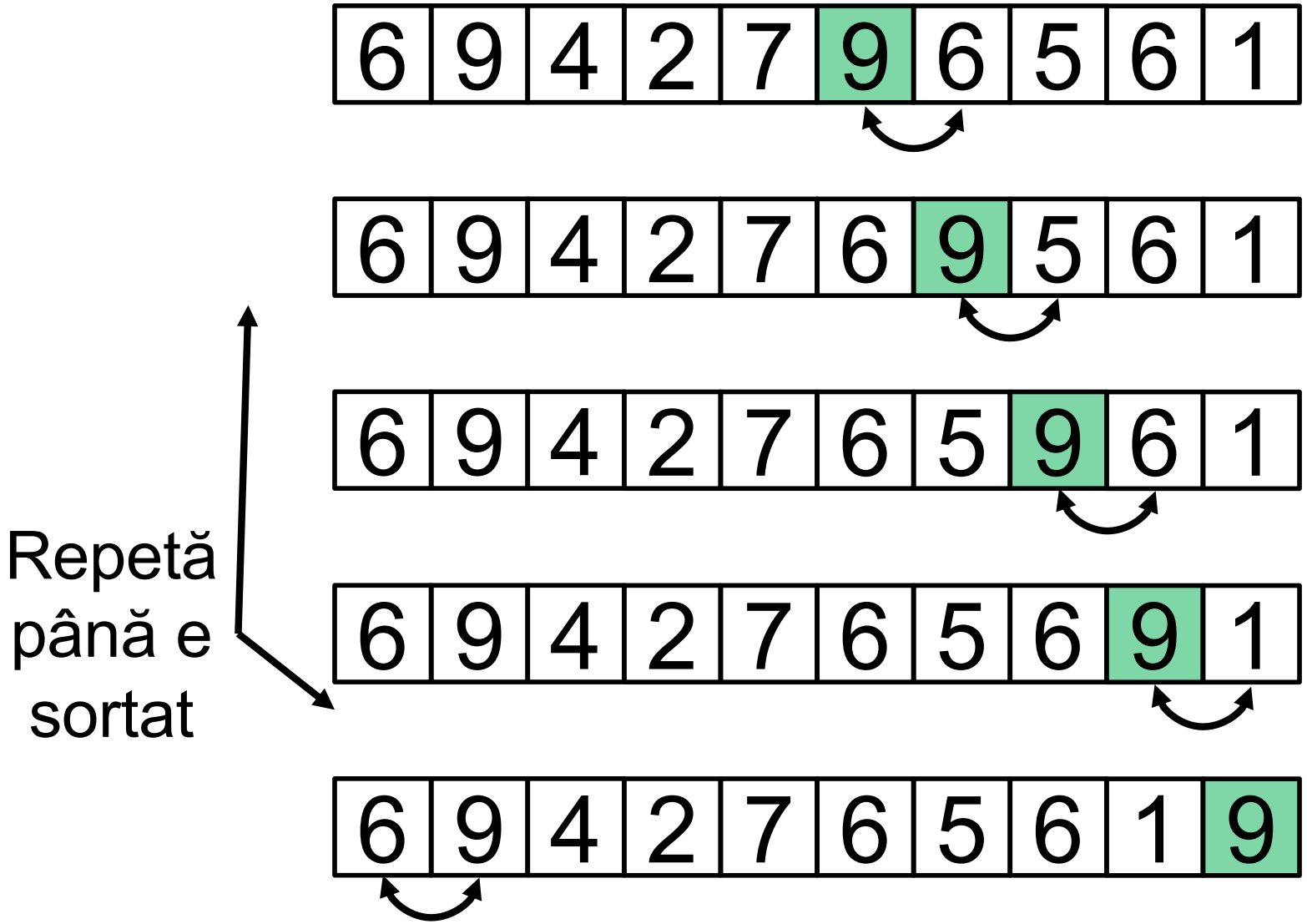


Exemplu: sortarea unui vector

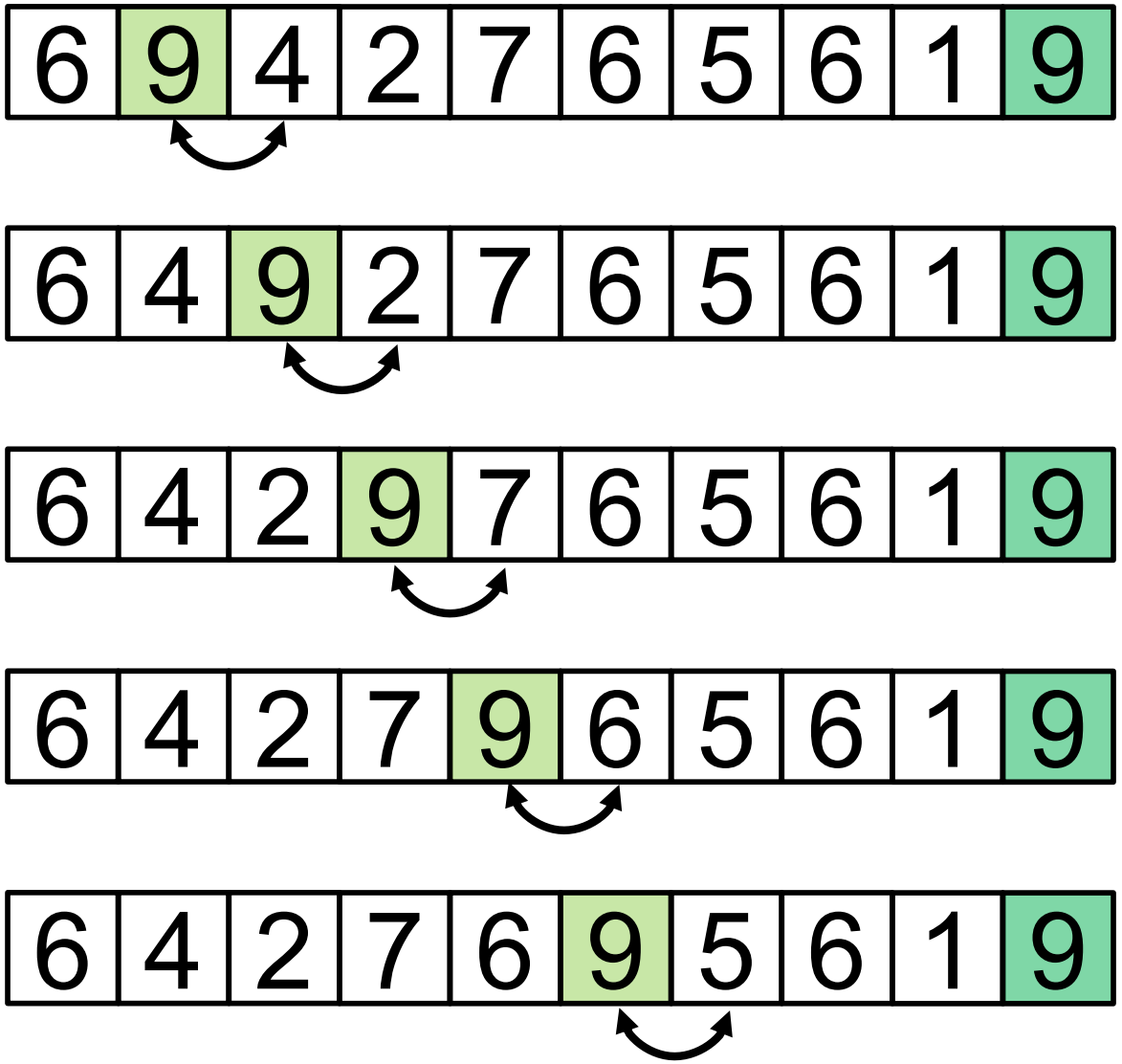
Bubble sort


9 6 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 4 9 2 7 6 5 6 1
6 9 4 2 9 7 6 5 6 1

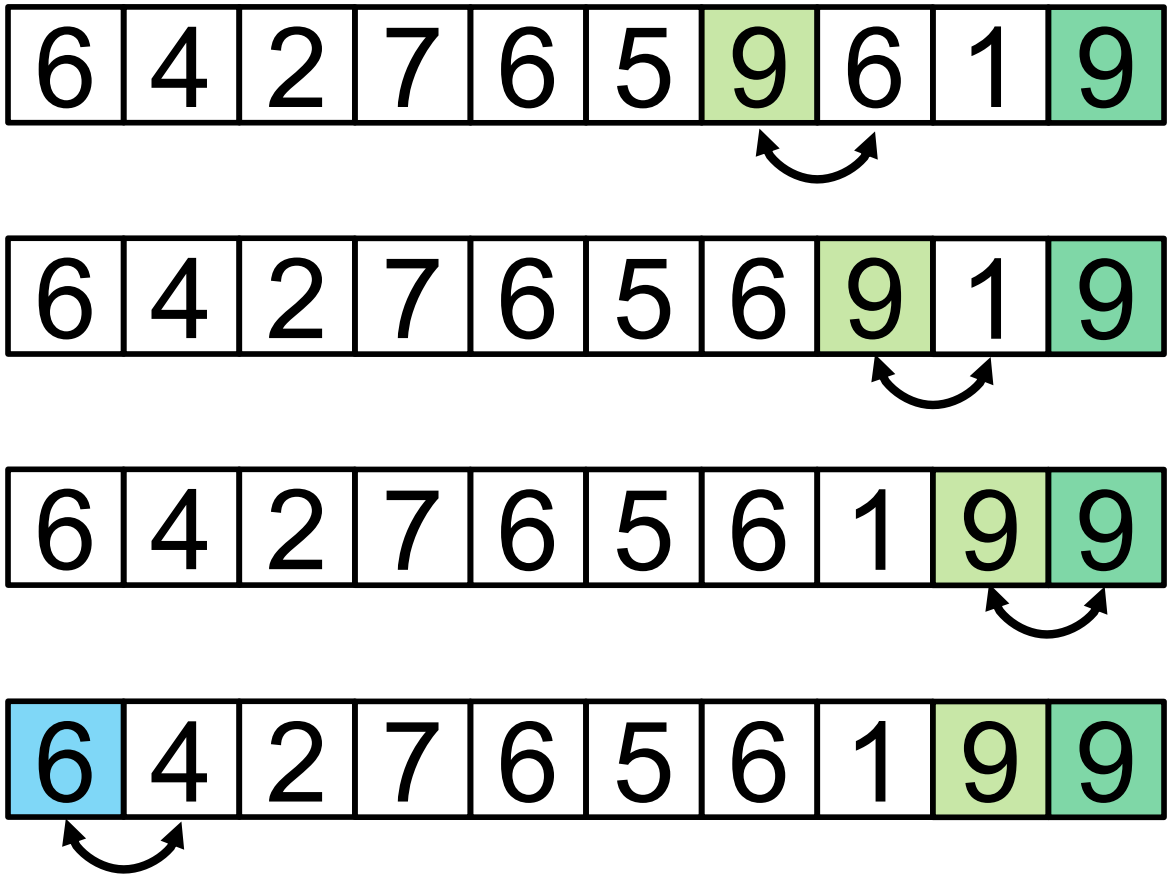
Bubble sort


6 9 4 2 7 9 6 5 6 1
6 9 4 2 7 6 9 5 6 1
6 9 4 2 7 6 5 9 6 1
6 9 4 2 7 6 5 6 9 1
6 9 4 2 7 6 5 6 1 9
Repetă până e sortat

Bubble sort

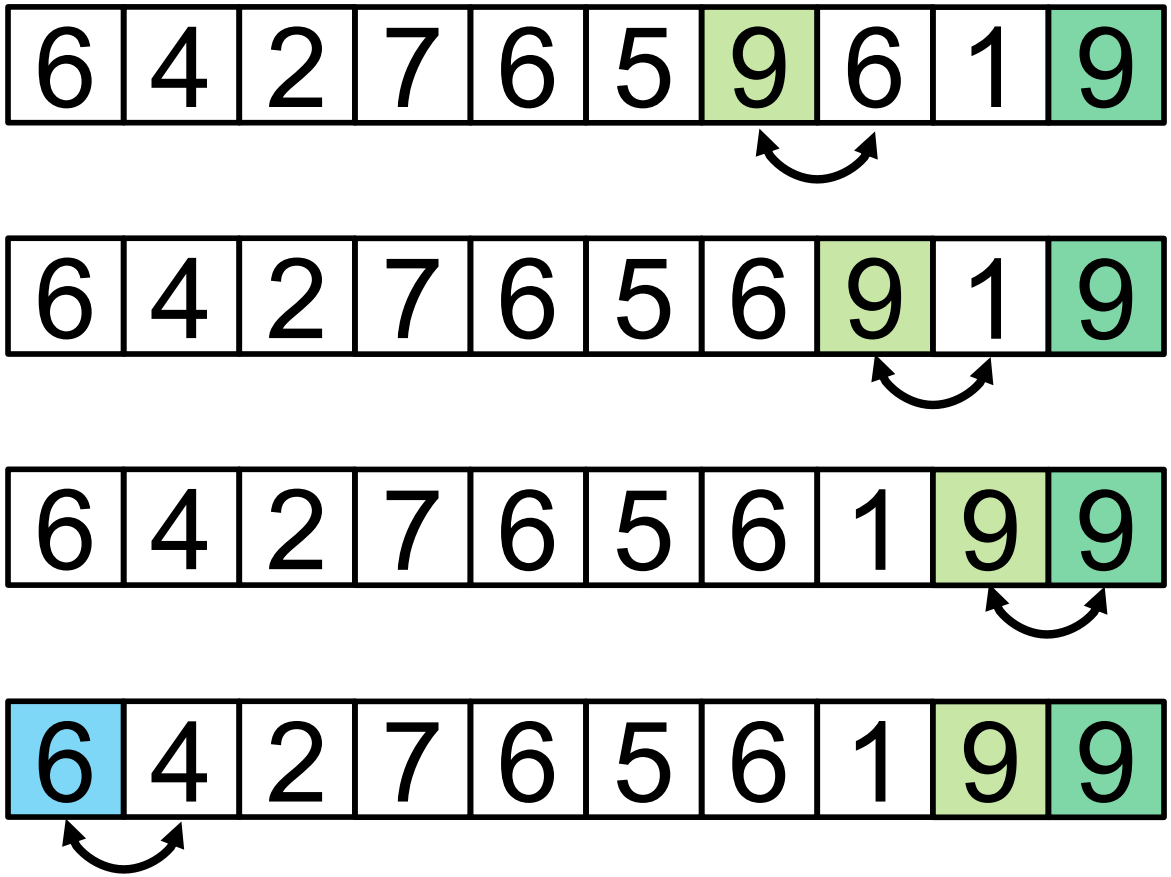

6 9 4 2 7 6 5 6 1 9
6 4 9 2 7 6 5 6 1 9
6 4 2 9 7 6 5 6 1 9
6 4 2 7 9 6 5 6 1 9
6 4 2 7 6 9 5 6 1 9

Bubble sort



.....

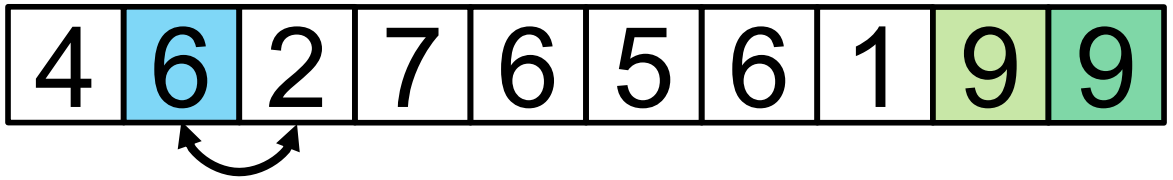
Bubble sort



.....

Complexitate?

Bubble sort

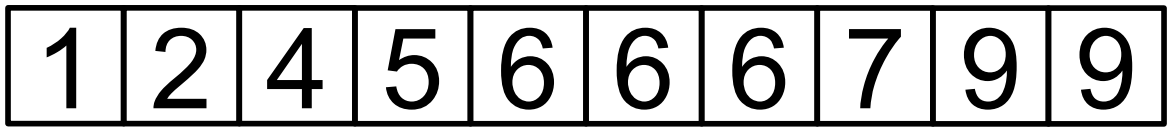


.....

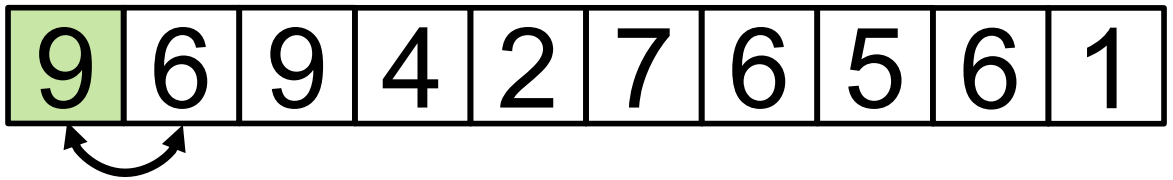
Complexitate: **O(n^2)**

Un pas trece prin toate elementele

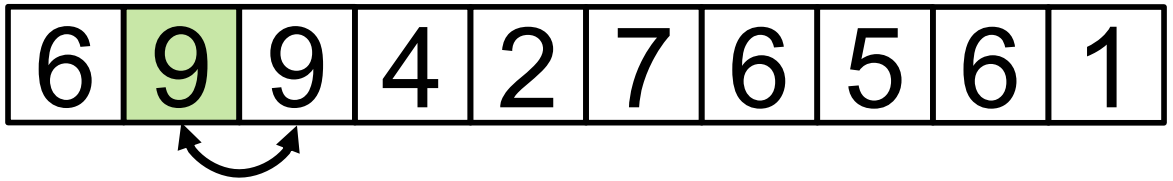
Garantat să termine după n repetiții



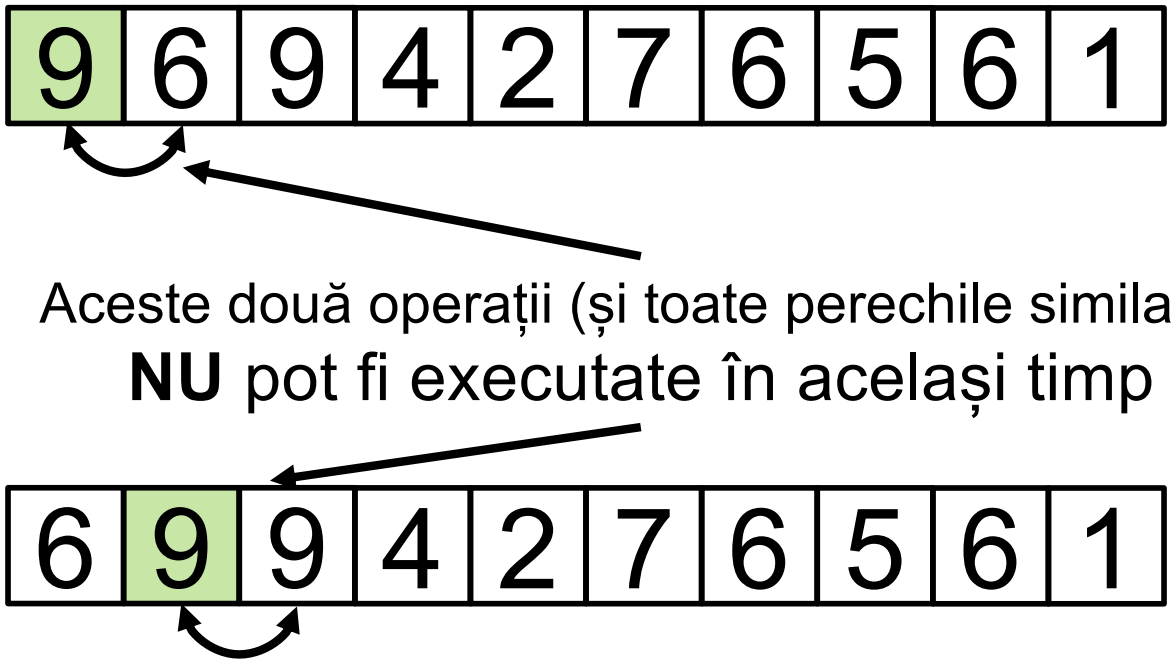
Parallel bubble sort



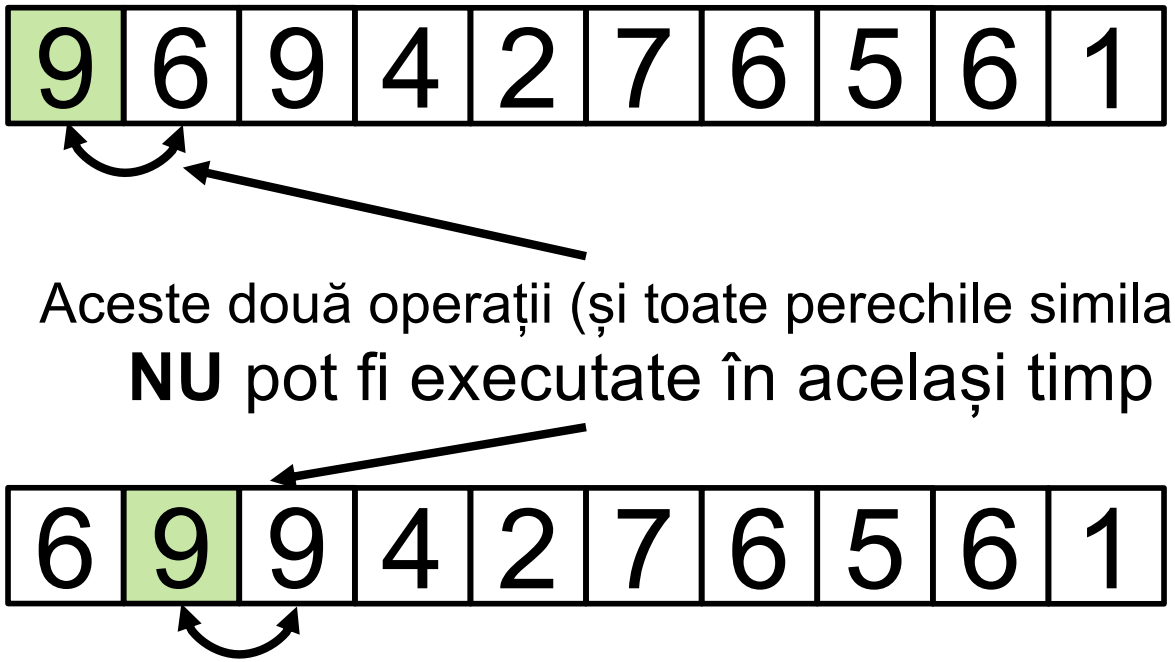
Cum paralelizăm?



Parallel bubble sort

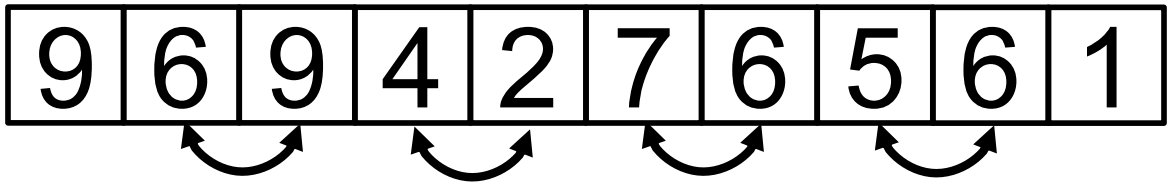


Parallel bubble sort



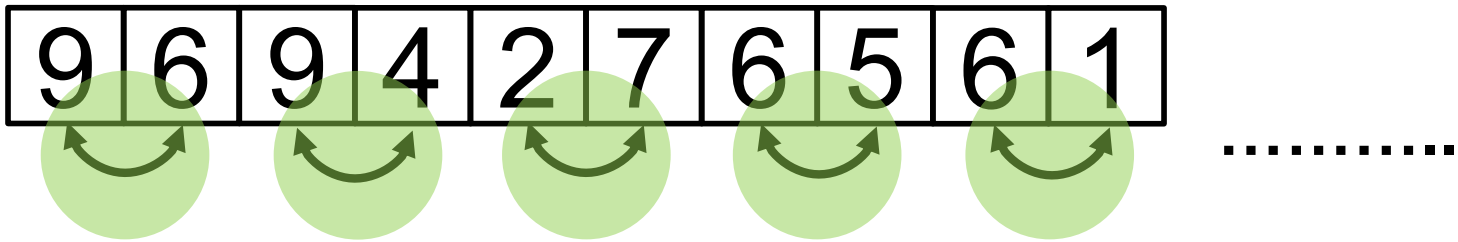
Solution hint: Nu e necesară execuția operațiilor într-o anumită ordine

OETS: Odd-Even Transposition Sort

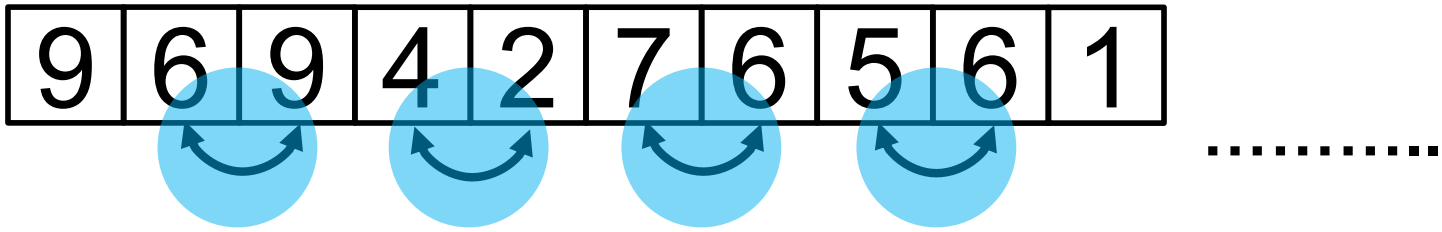


OETS: Odd-Even Transposition Sort

Pot fi executate în paralel →

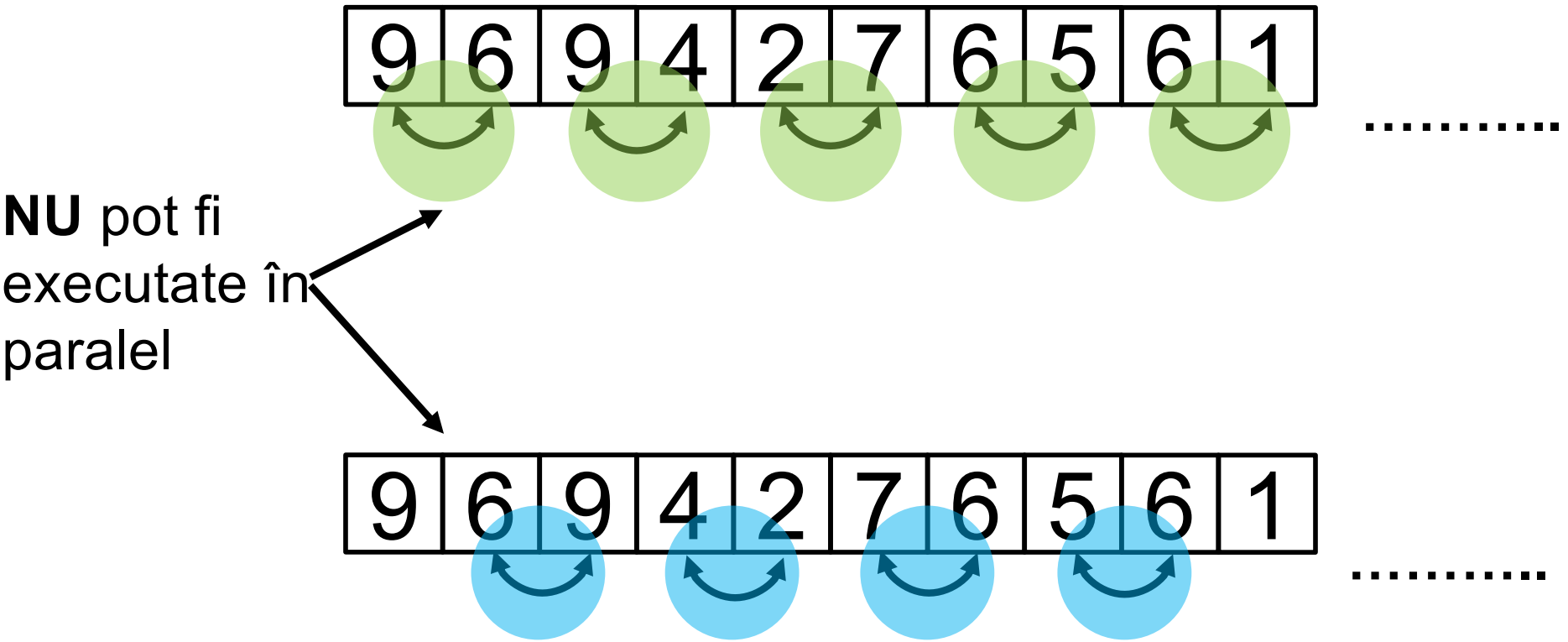


Pot fi executate în paralel →

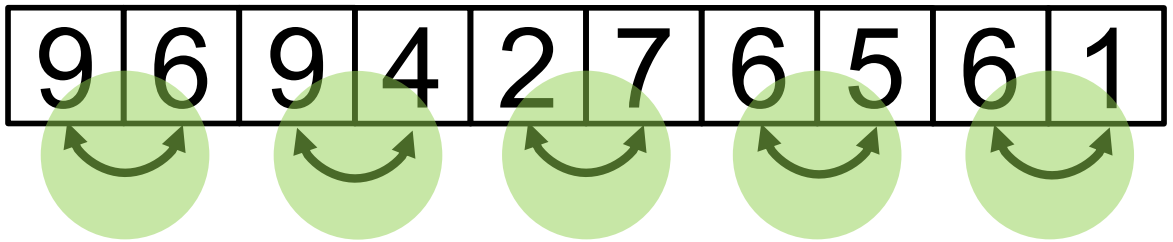


.....

OETS: Odd-Even Transposition Sort

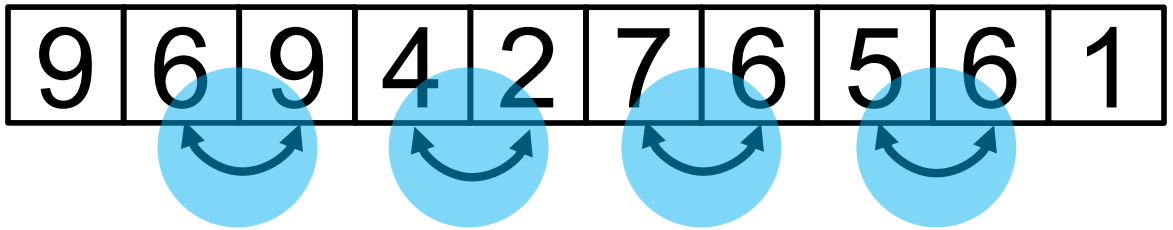


OETS: Odd-Even Transposition Sort



.....

NU pot fi executate în paralel

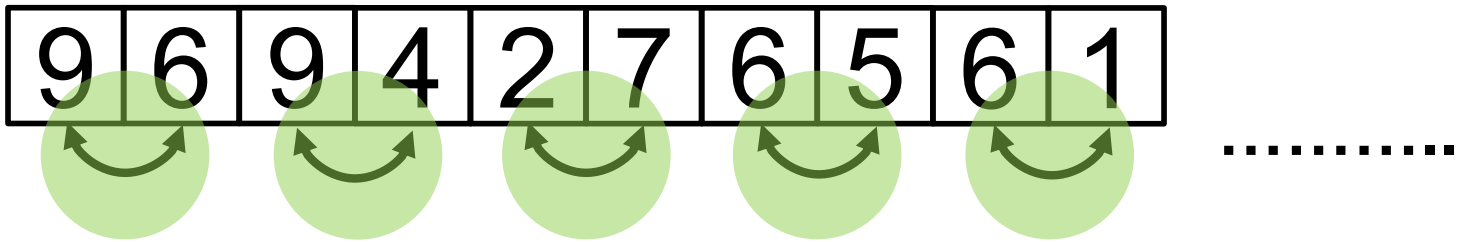


.....

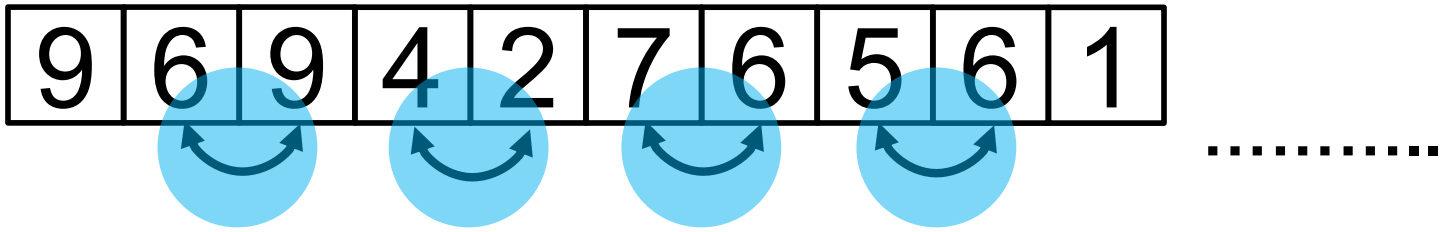
Ce facem? Ce folosim?

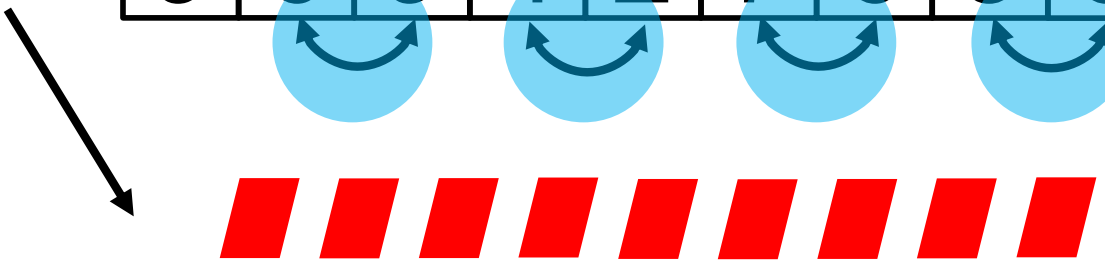
.....

OETS: Odd-Even Transposition Sort



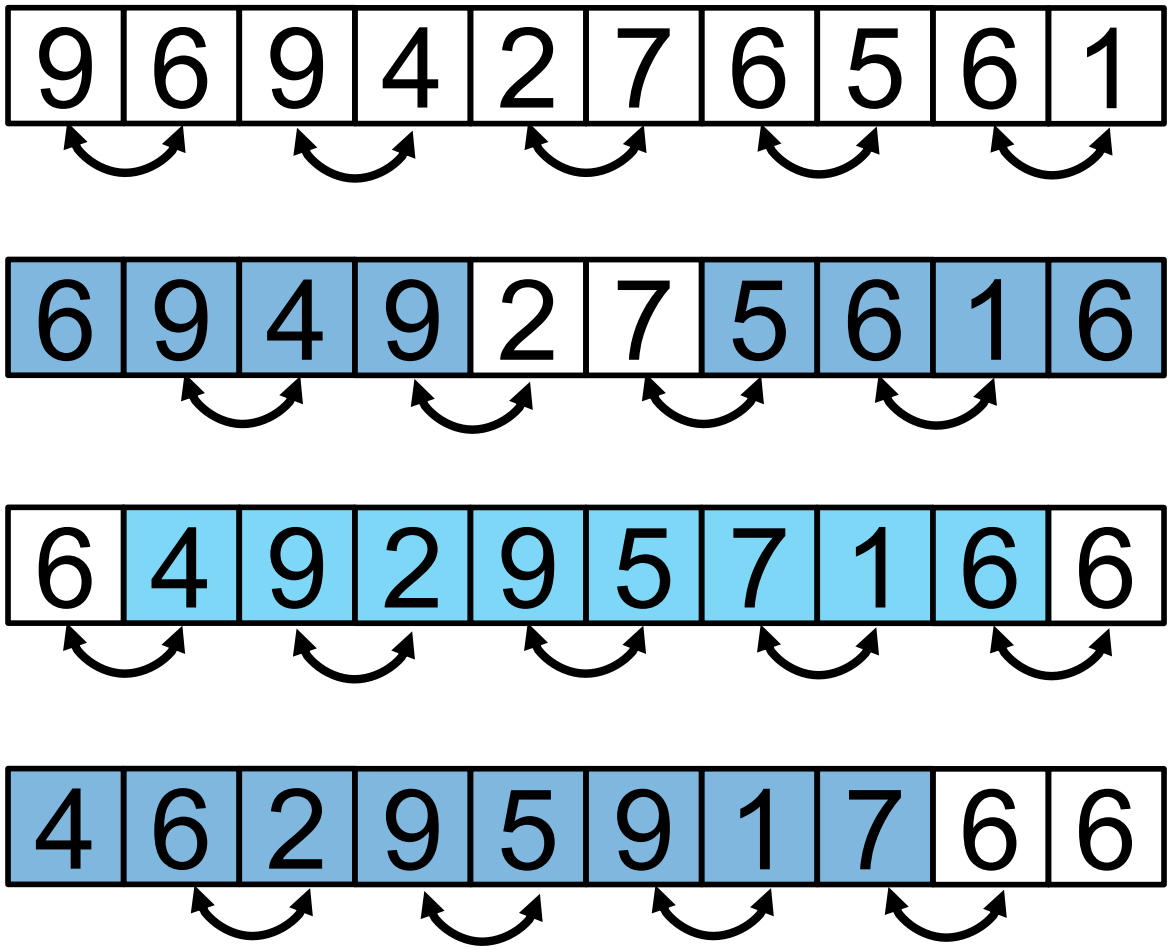
Soluția:
Barrier
între
fiecare
repetiție



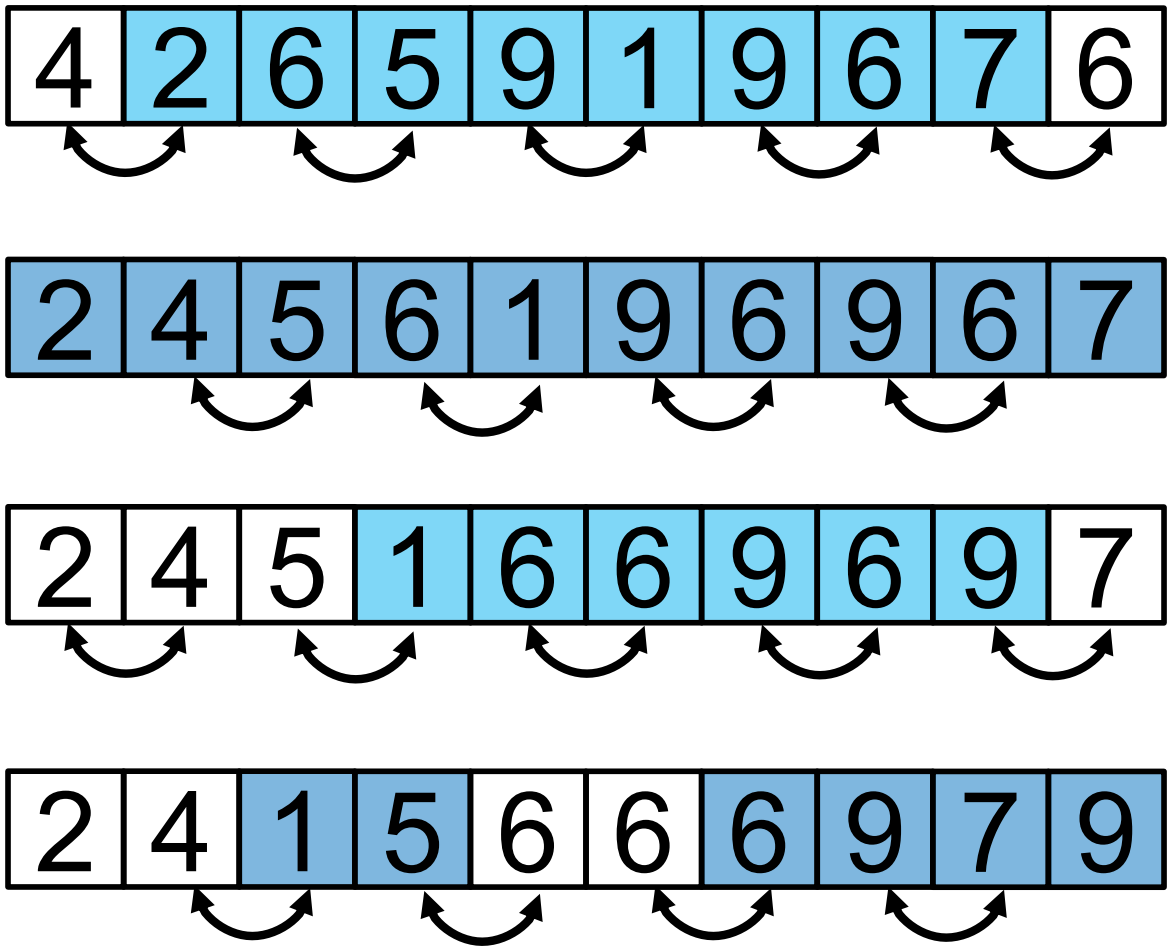


.....

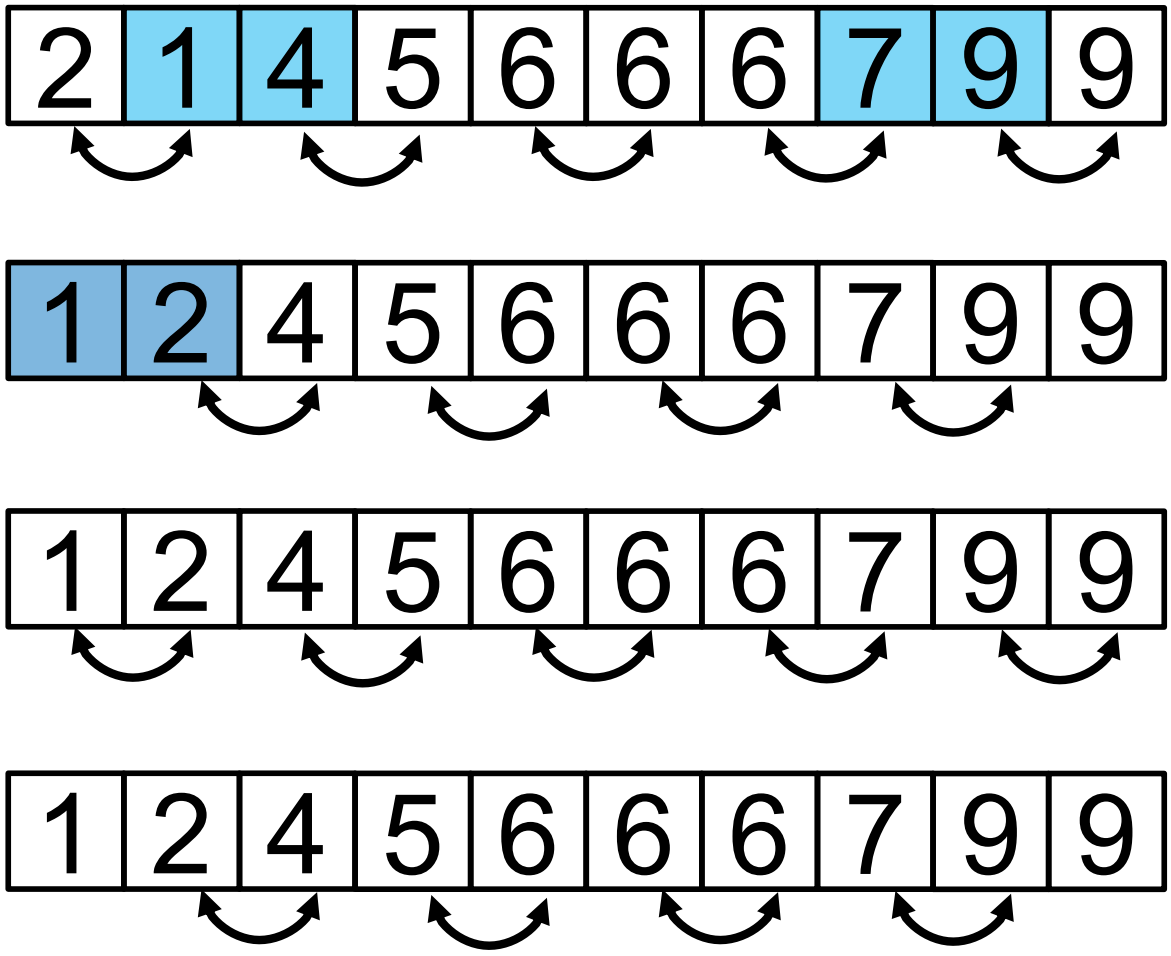
OETS: Odd-Even Transposition Sort



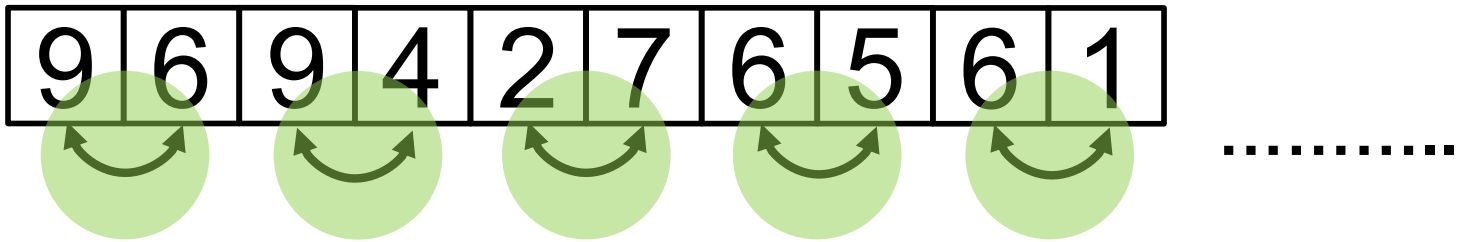
OETS: Odd-Even Transposition Sort



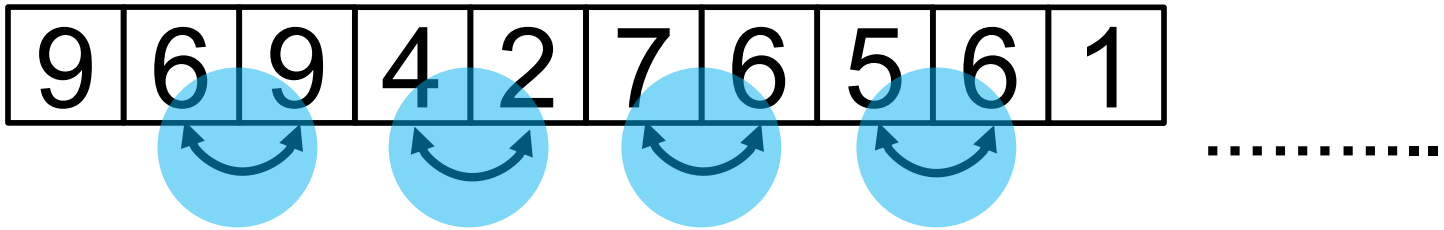
OETS: Odd-Even Transposition Sort



OETS: Odd-Even Transposition Sort

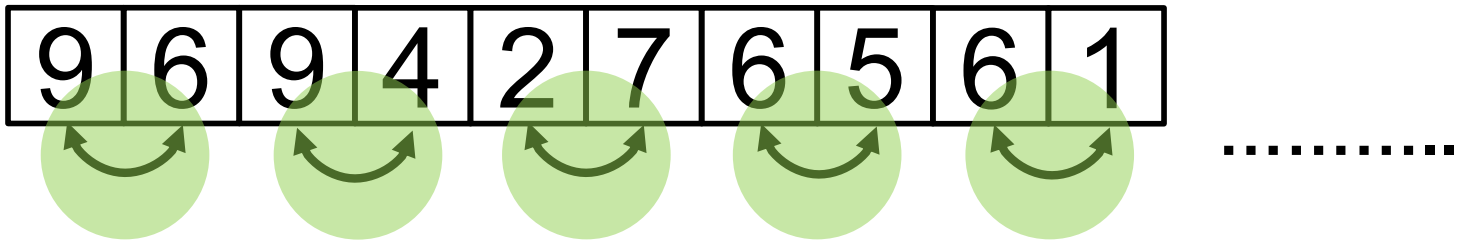


Complexitate a soluției paralele?

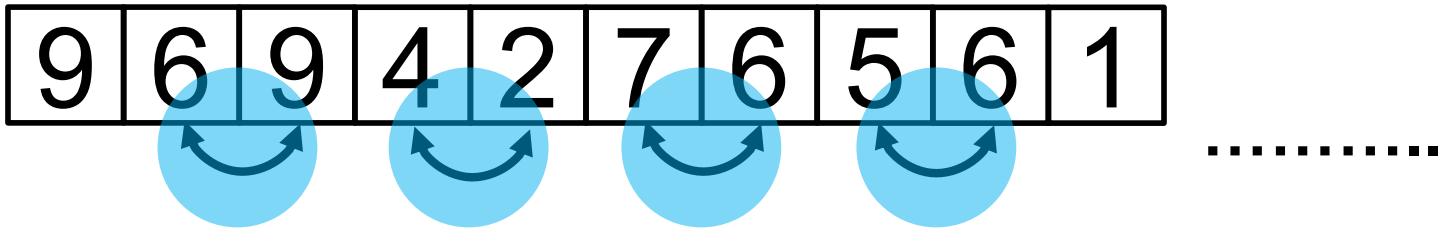


.....

OETS: Odd-Even Transposition Sort

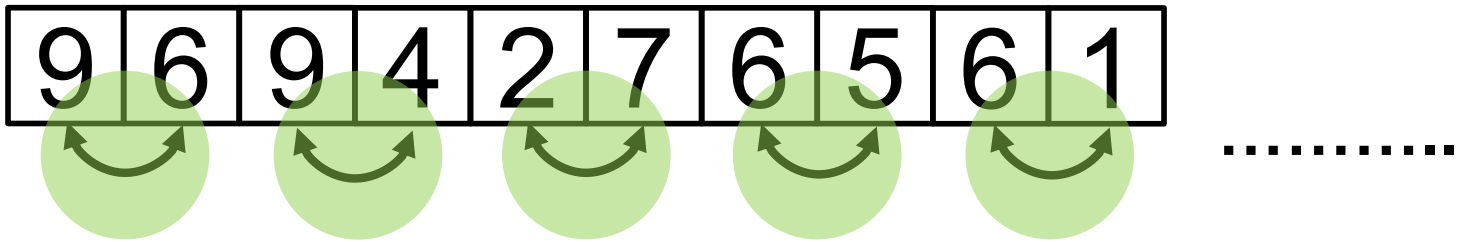


Complexitate a soluției paralele?

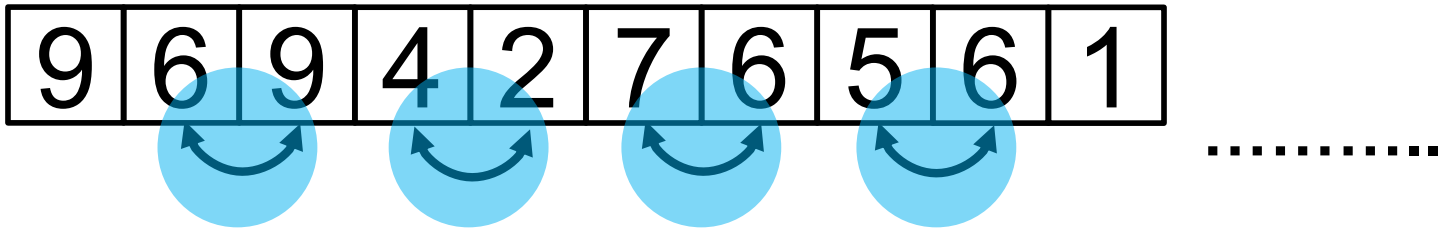


$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

OETS: Odd-Even Transposition Sort



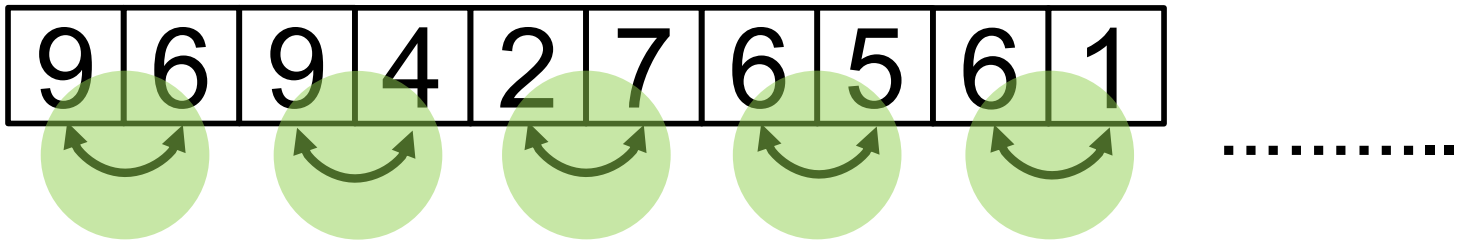
Speedup?



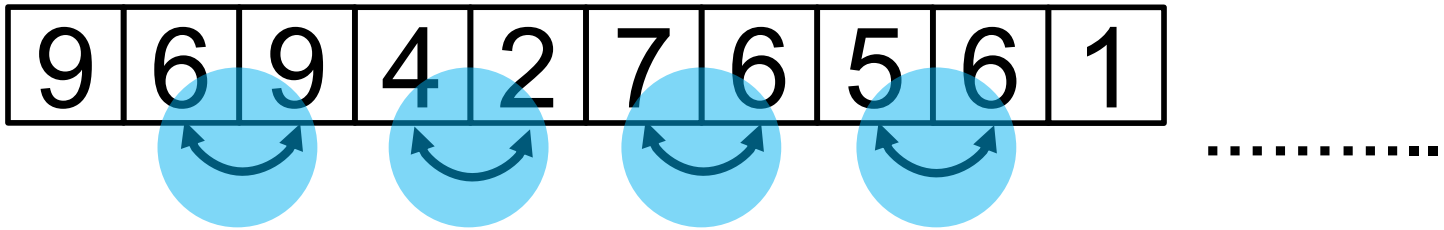
$$T = O\left(\frac{N}{P} * N\right) (= O(N) \text{ pentru } P = N)$$

.....

OETS: Odd-Even Transposition Sort

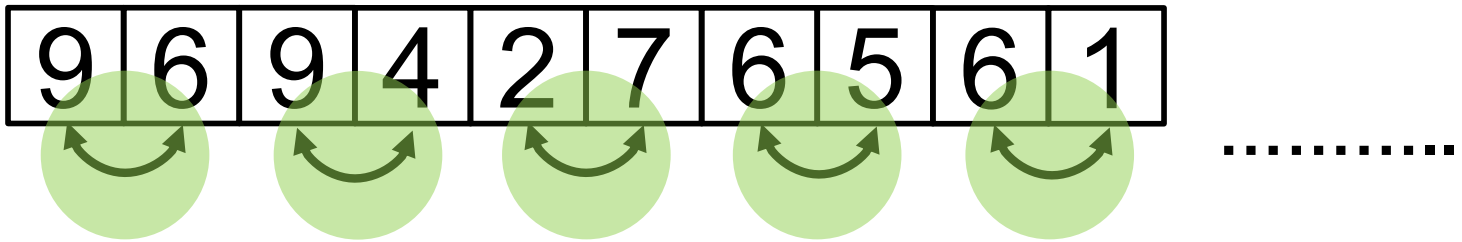


Speedup?

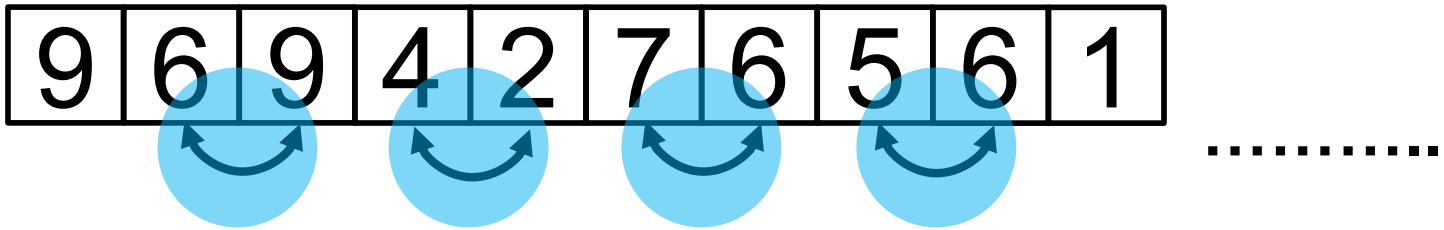


$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

OETS: Odd-Even Transposition Sort

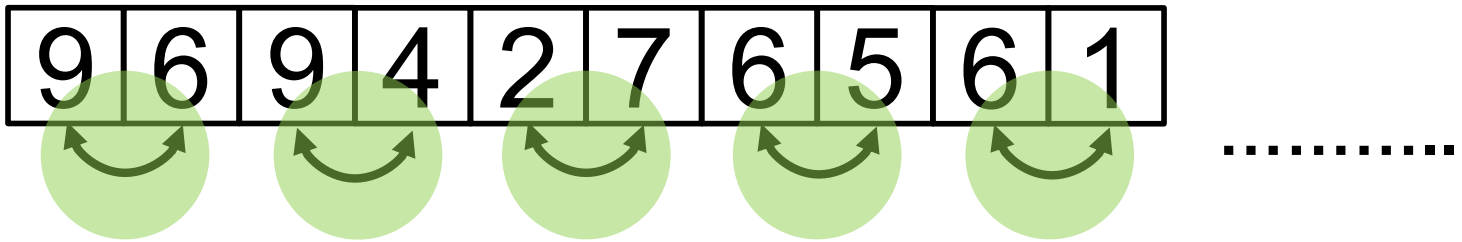


Speedup? **Sigur**?

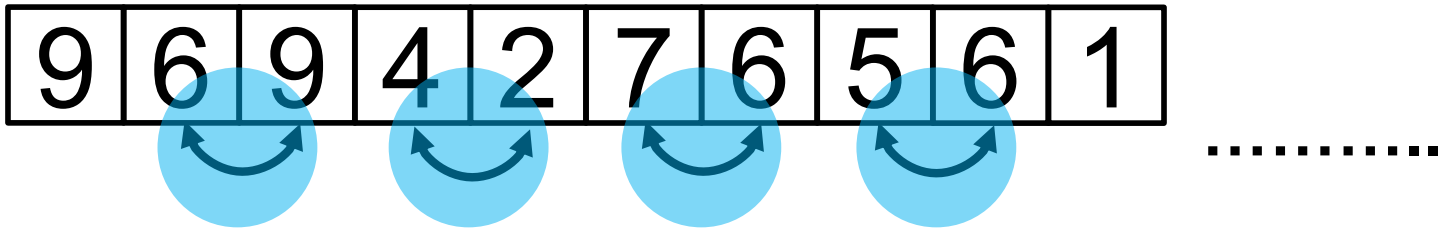


$$S = \frac{N^2}{\frac{N^2}{P}} = P$$

OETS: Odd-Even Transposition Sort

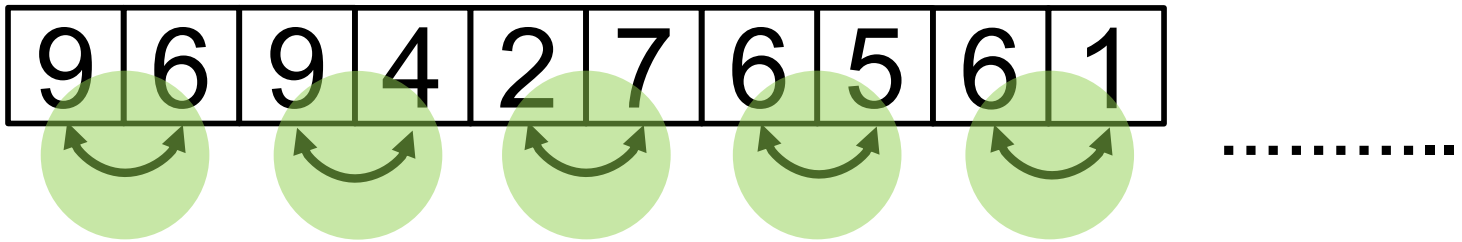


Speedup?



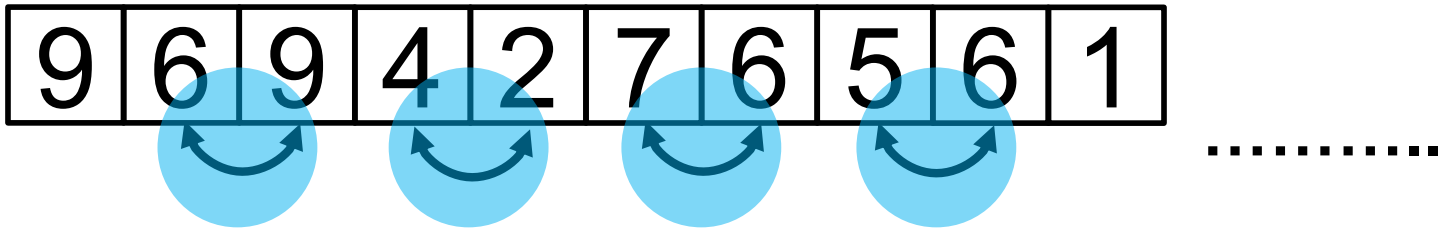
$$S = \frac{N log_2 N}{\frac{N^2}{P}}$$

OETS: Odd-Even Transposition Sort



.....

Speedup?

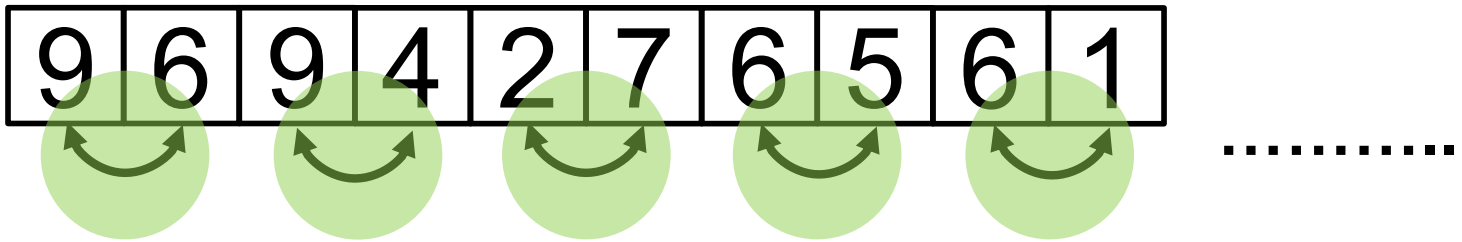


.....

$$S = \frac{Plog_2N}{N}$$

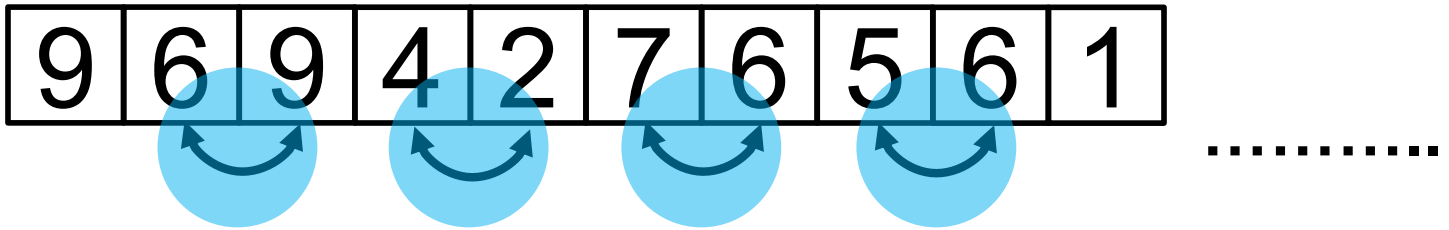
.....

OETS: Odd-Even Transposition Sort



.....

Nu uitați: Așteptatul la barieră poate introduce timpi mari!

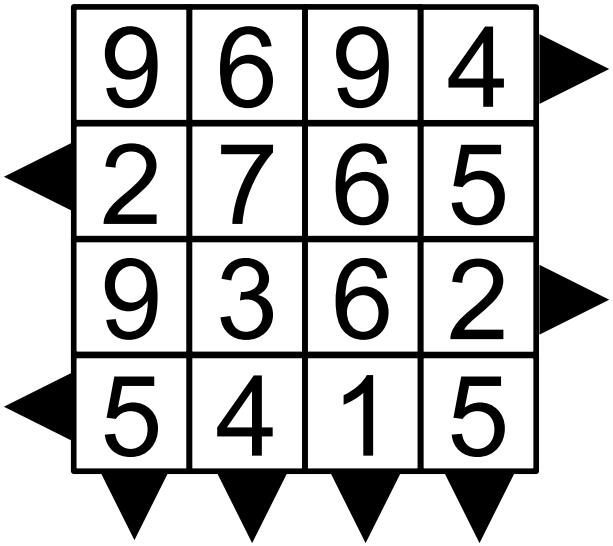


.....

$$S = \frac{Plog_2N}{N}$$

.....

Shear sort
(Row-column sort)
(Snake sort)



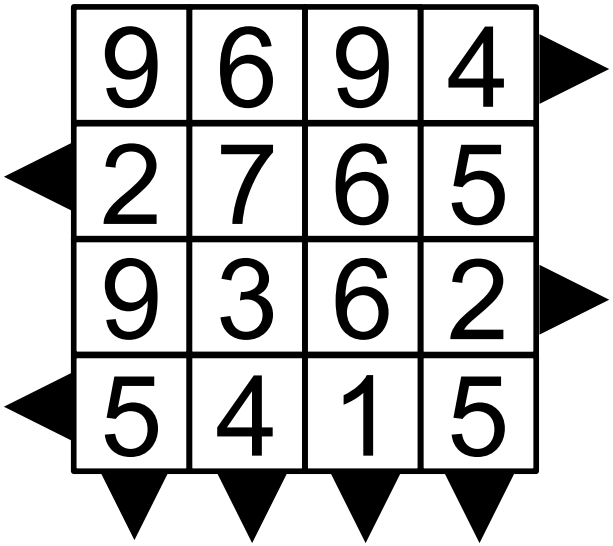
Sortează fiecare linie **pară** în mod **ascendent**
Sortează fiecare linie **impară** în mod **descendent**

Shear sort

9	6	9	4
2	7	6	5
9	3	6	2
5	4	1	5

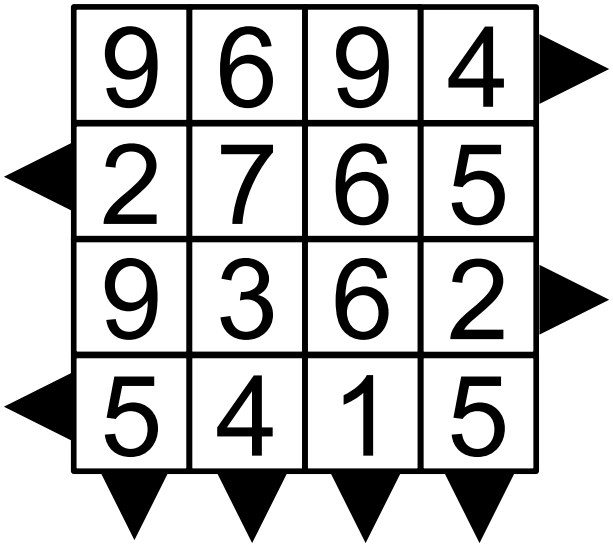
Sortează crescător coloanele

Shear sort



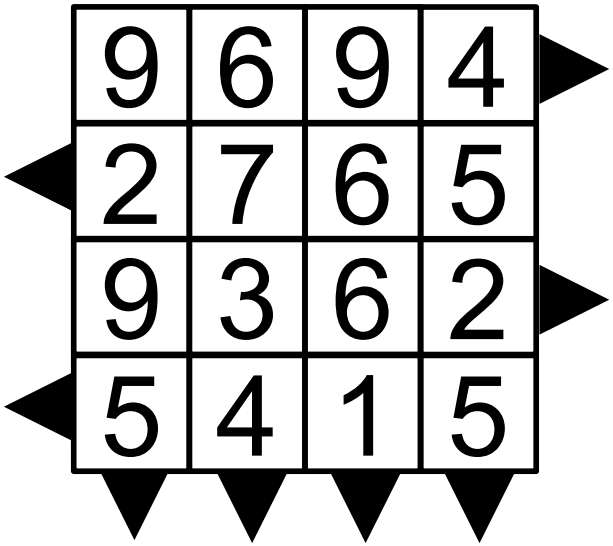
Repetă tot de $log_2 n$ ori

Shear sort



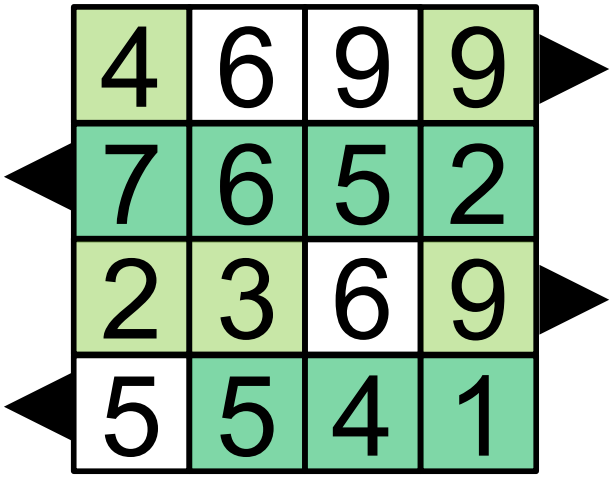
De ce sortăm liniile pare crescător și cele impare descrescător?

Shear sort

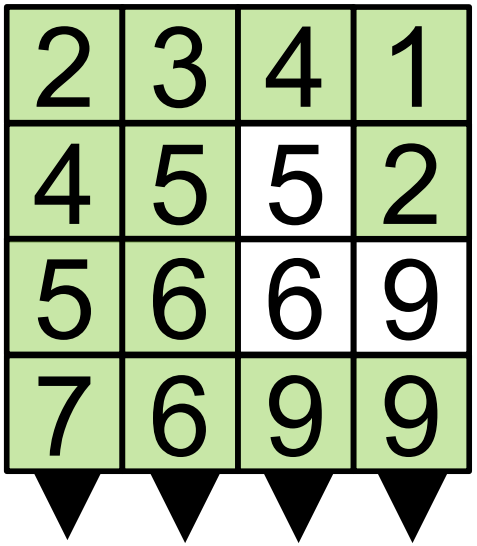


De ce liniile pare crescător și celelalte descrescător?
Dorim compararea celui mai mare element de pe linia i cu cel mai mic de pe linia i+1

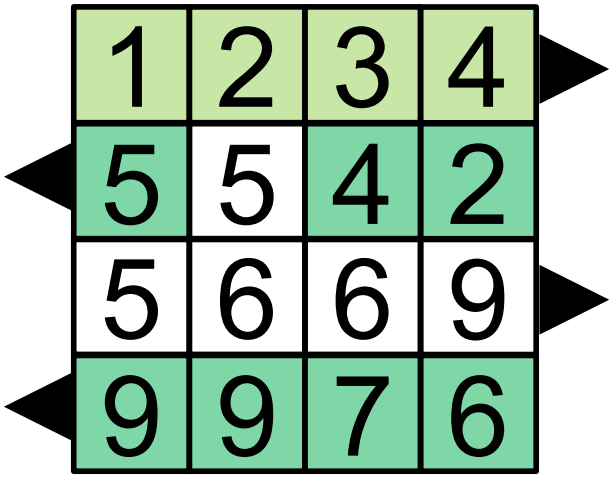
Shear sort



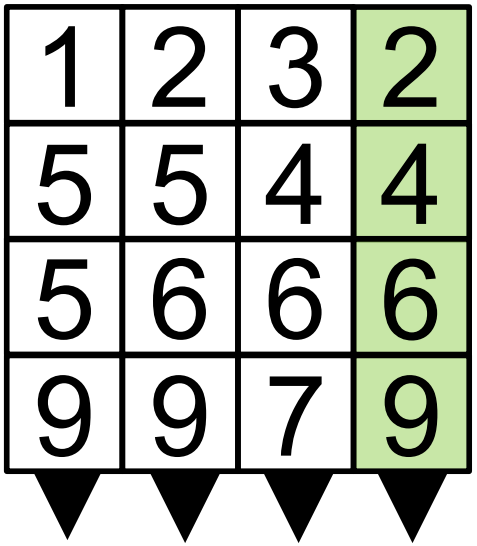
Shear sort



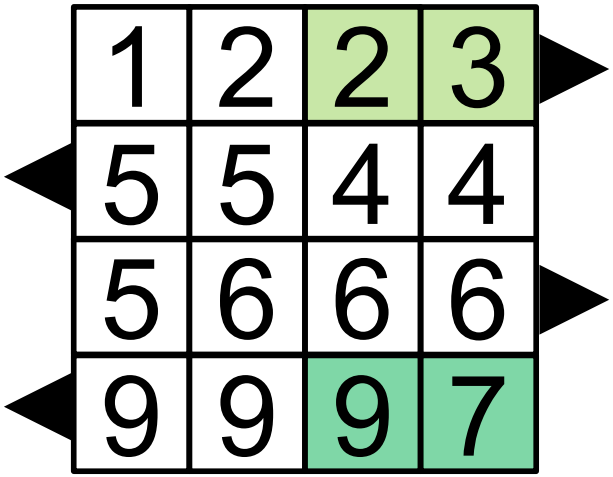
Shear sort



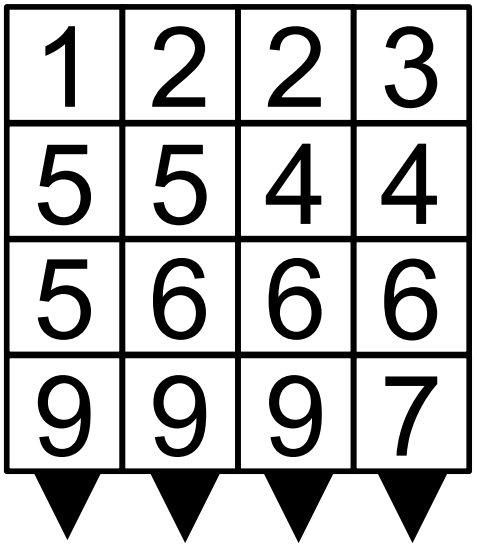
Shear sort



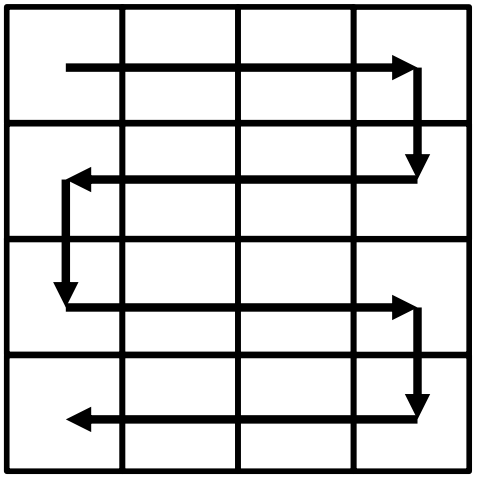
Shear sort



Shear sort

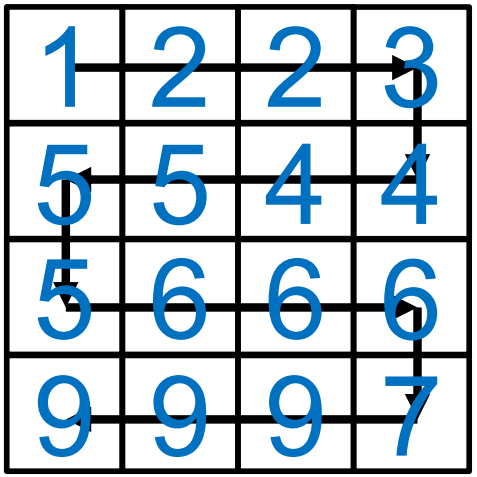


Shear sort



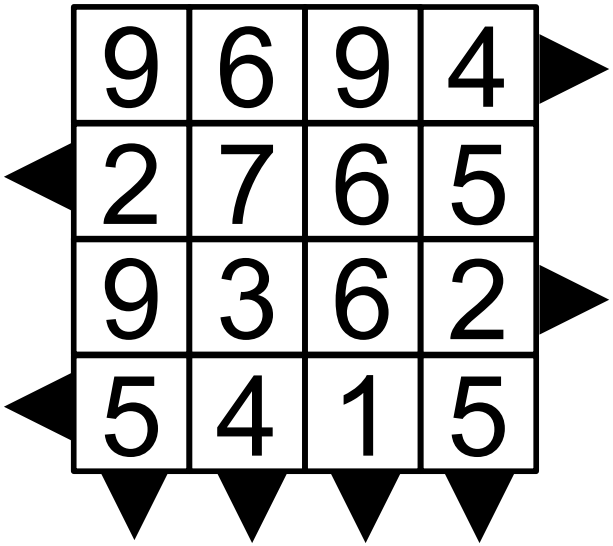
Lista finală se obține citind în formă de șerpuită (snake sort)

Shear sort



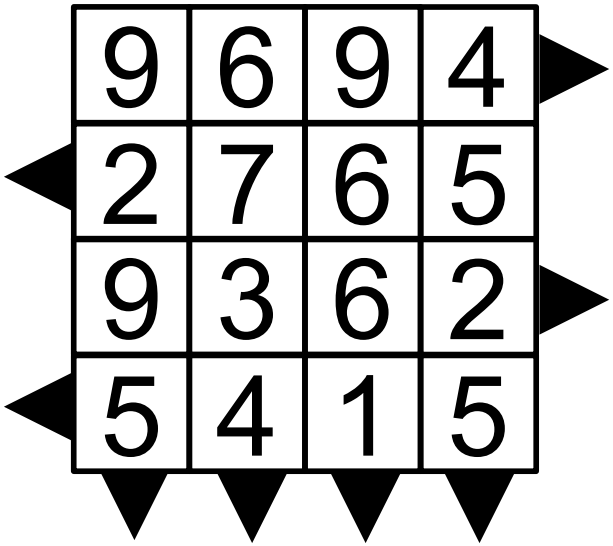
1	2	2	3	4	4	5	5	5	6	6	6	7	9	9	9

Shear sort



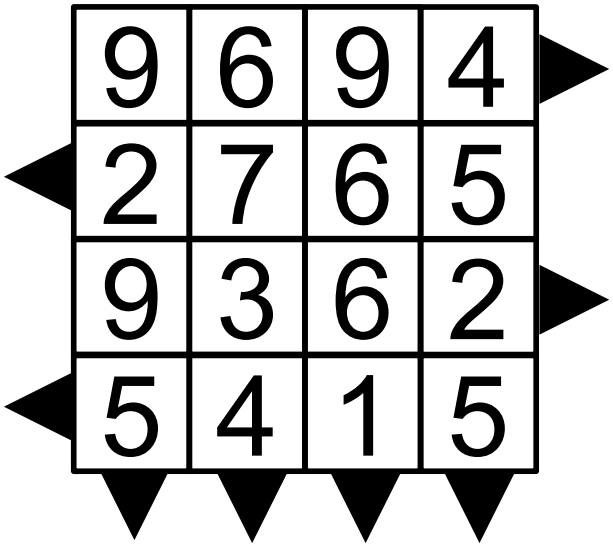
Complexitate? $G =$

Shear sort



Complexitate? $G = log_2 N \ *$
$log_2 N$ repetiții

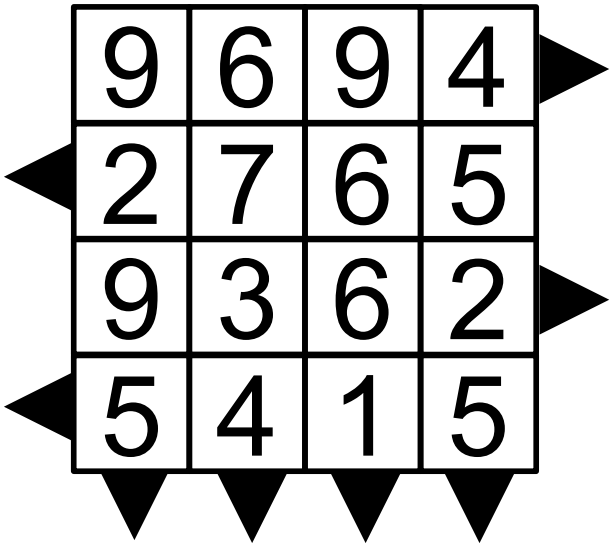
Shear sort



Complexitate? $G = log_2 N * 2 * \sqrt{N}$

$\sqrt{N}$ linii/coloane

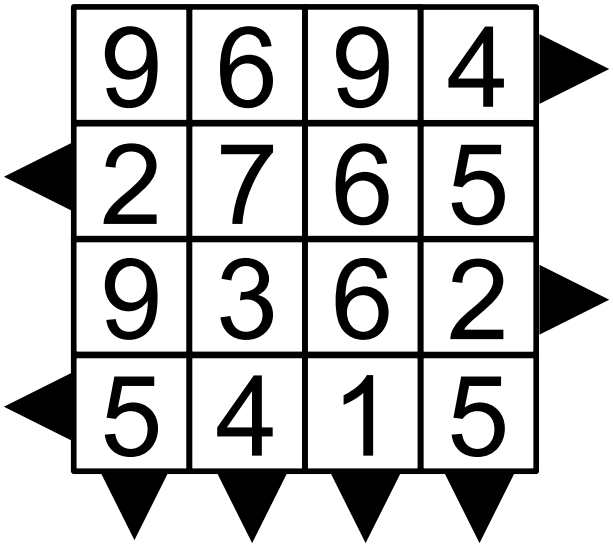
Shear sort



Complexitate? $G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$

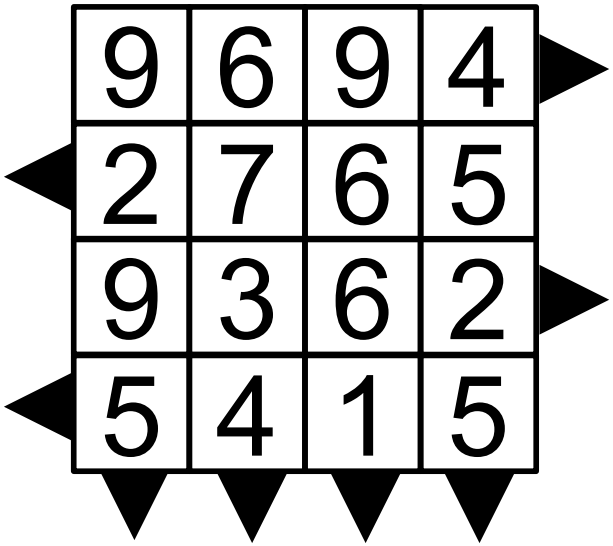
$\sqrt{N} * log_2 \sqrt{N}$ cel mai bun algoritm secvențial de sortare

Shear sort



Complexitate? $G = log_2 N * N * log_2 \sqrt{N}$

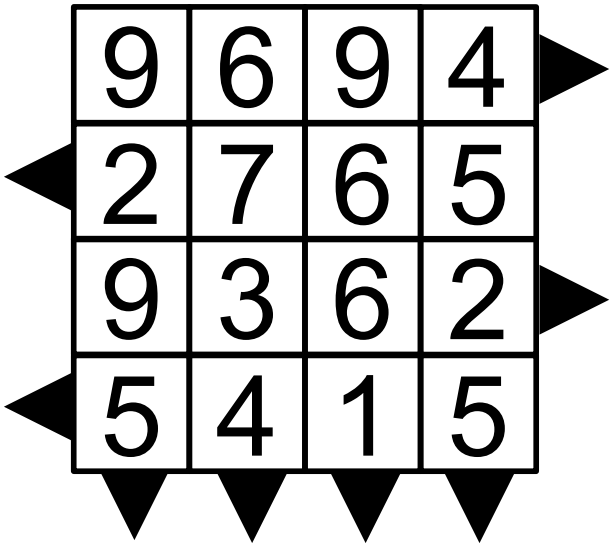
Shear sort



Complexitate? $G = Nlog_2N * log_2\sqrt{N}$

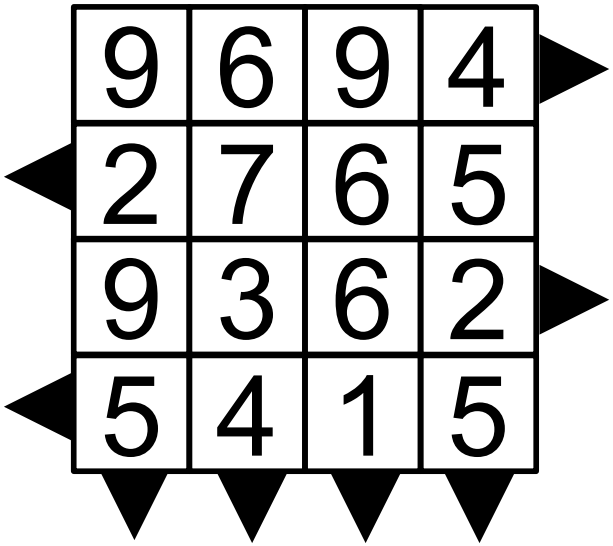
Cel mai bun algoritm secvențial rămâne $Nlog_2N$

Shear sort



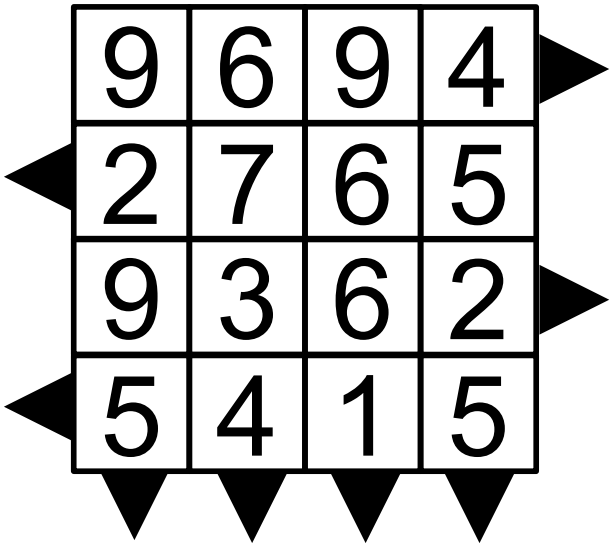
Cum paralelizăm?

Shear sort



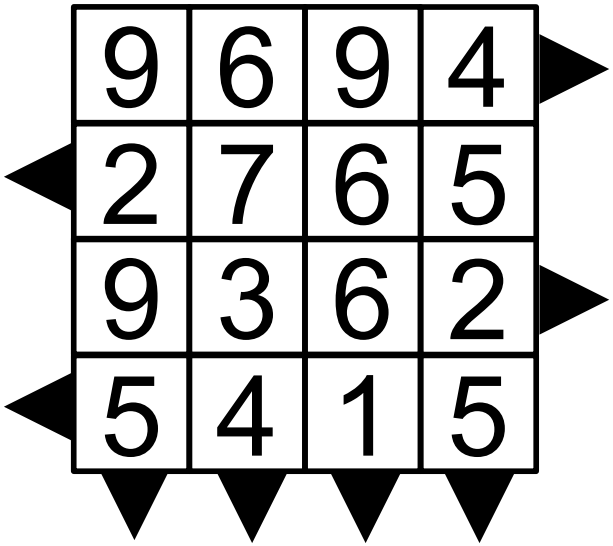
Cum paralelizăm?
Toate liniile în paralel, barieră, apoi toate coloanele, apoi barieră, și repetăm.

Shear sort



Cum paralelizăm?
Toate liniile în paralel, barieră, apoi toate coloanele, apoi barieră, și repetăm.

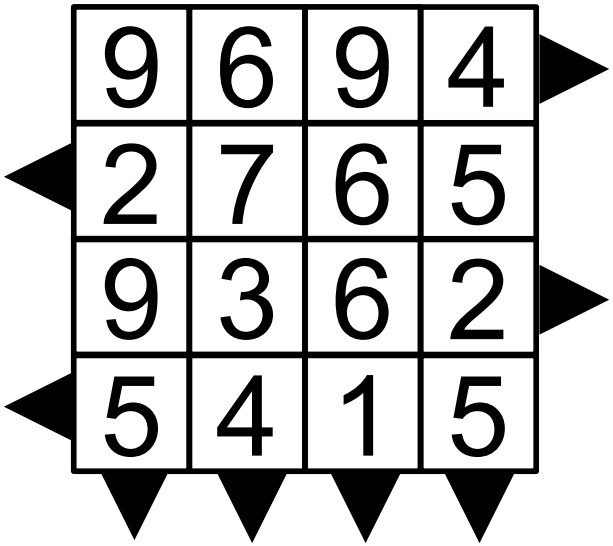
Shear sort



Complexitate versiune paralelă?

$$G = log_2 N * 2 * \sqrt{N} * \sqrt{N} * log_2 \sqrt{N}$$

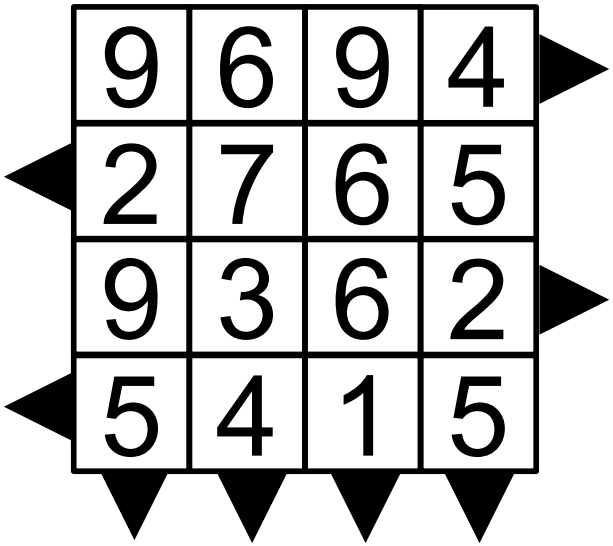
Shear sort



Complexitate versiune paralelă? N = P

$$T = log_2 N * 2 * \sqrt{N} * log_2 \sqrt{N}$$

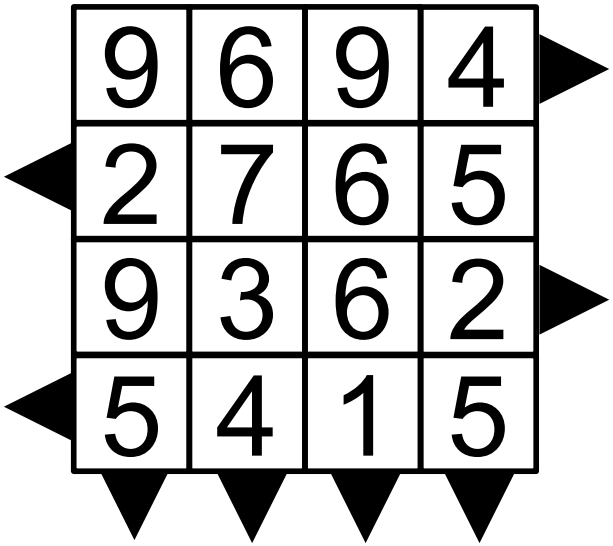
Shear sort



Complexitate versiune paralelă? N = P

$$T = \sqrt{N}\, log_2 \sqrt{N} * log_2 N$$

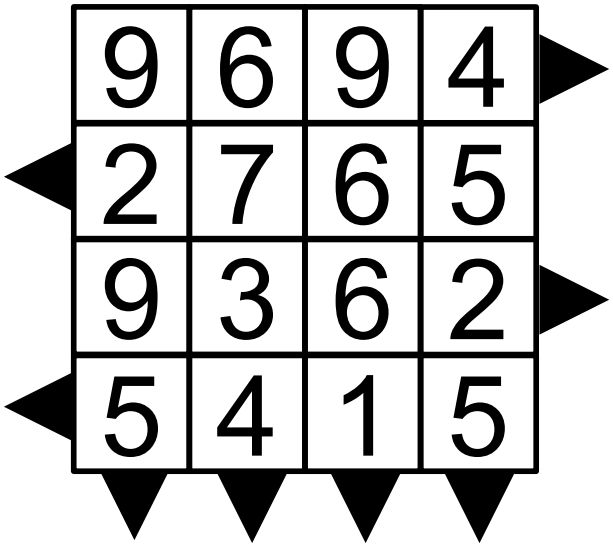
Shear sort



Complexitate versiune paralelă?

$$T = log_2 N * 2 * \frac{\sqrt{N}}{P} * \sqrt{N} * log_2 \sqrt{N}$$

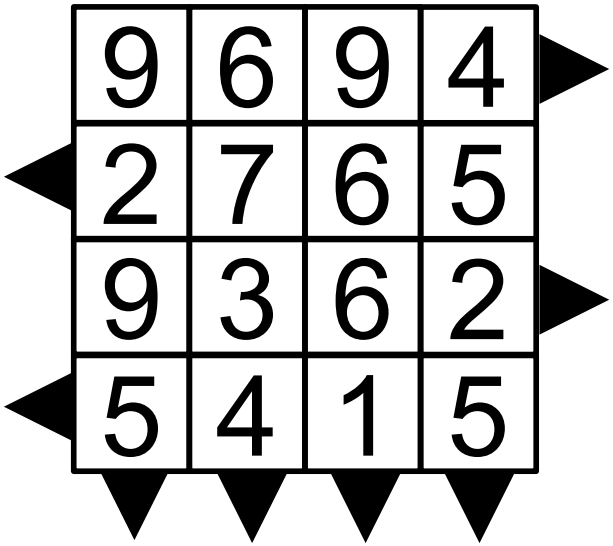
Shear sort



Complexitate versiune paralelă?

$$T = \frac{N}{P} log_2 N * log_2 \sqrt{N}$$

Shear sort



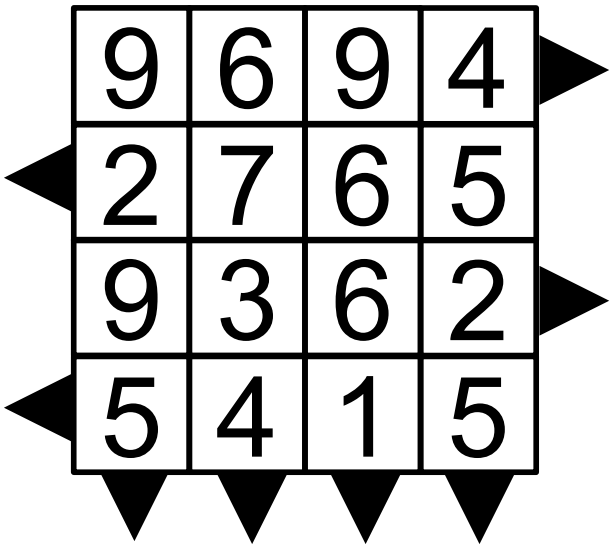
Speedup?

$T = \frac{N}{P} log_2 N \ * \ log_2 \sqrt{N}$

$G = \ N log_2 N$

Shear sort

$$S = \frac{Nlog_2N}{\frac{N}{P}log_2N \ * \ log_2\sqrt{N}}$$



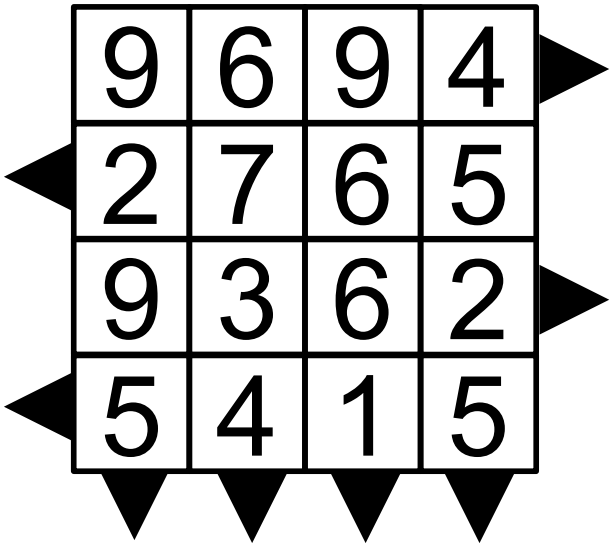
Speedup?

$$T = \frac{N}{P}log_2N \ * \ log_2\sqrt{N}$$

$$G = \ Nlog_2N$$

Shear sort

$$S = \frac{P}{log_2\sqrt{N}}$$



Speedup?

$$T = \frac{N}{P} log_2 N \ * \ log_2\sqrt{N}$$

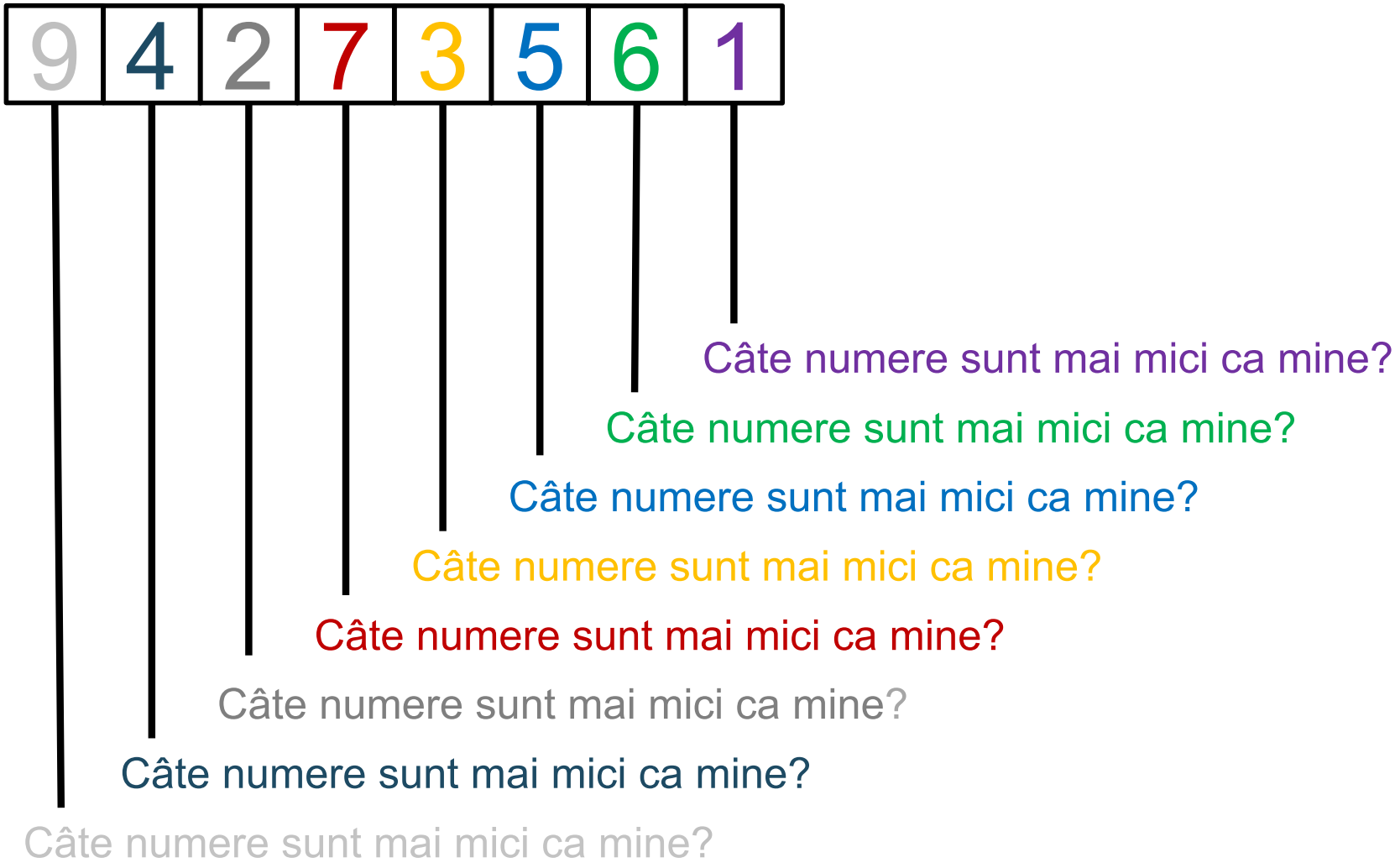
$$G = \ N log_2 N$$

Rank Sort

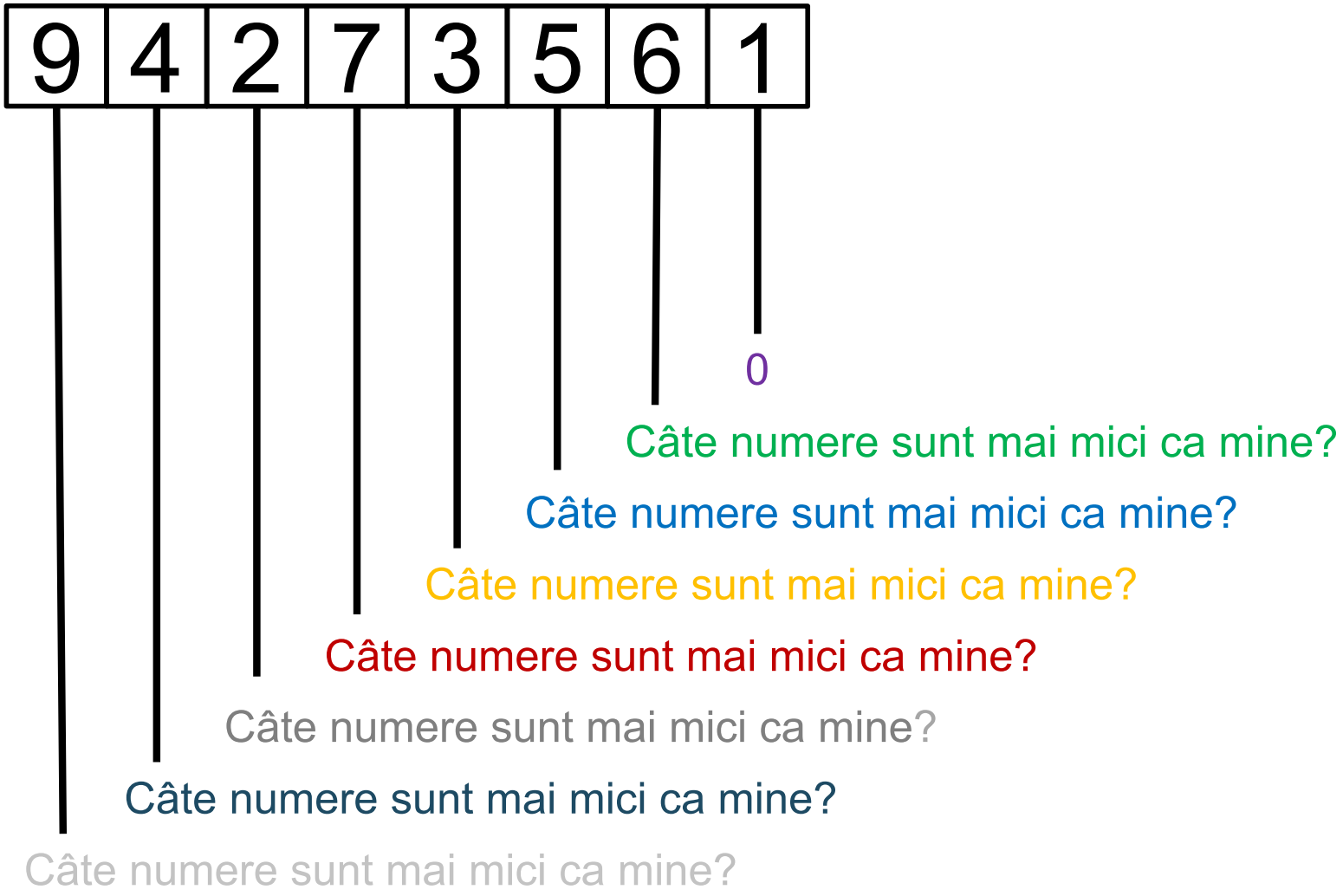
9	4	2	7	6	5	6	1

1	2	4	5	6	6	7	9

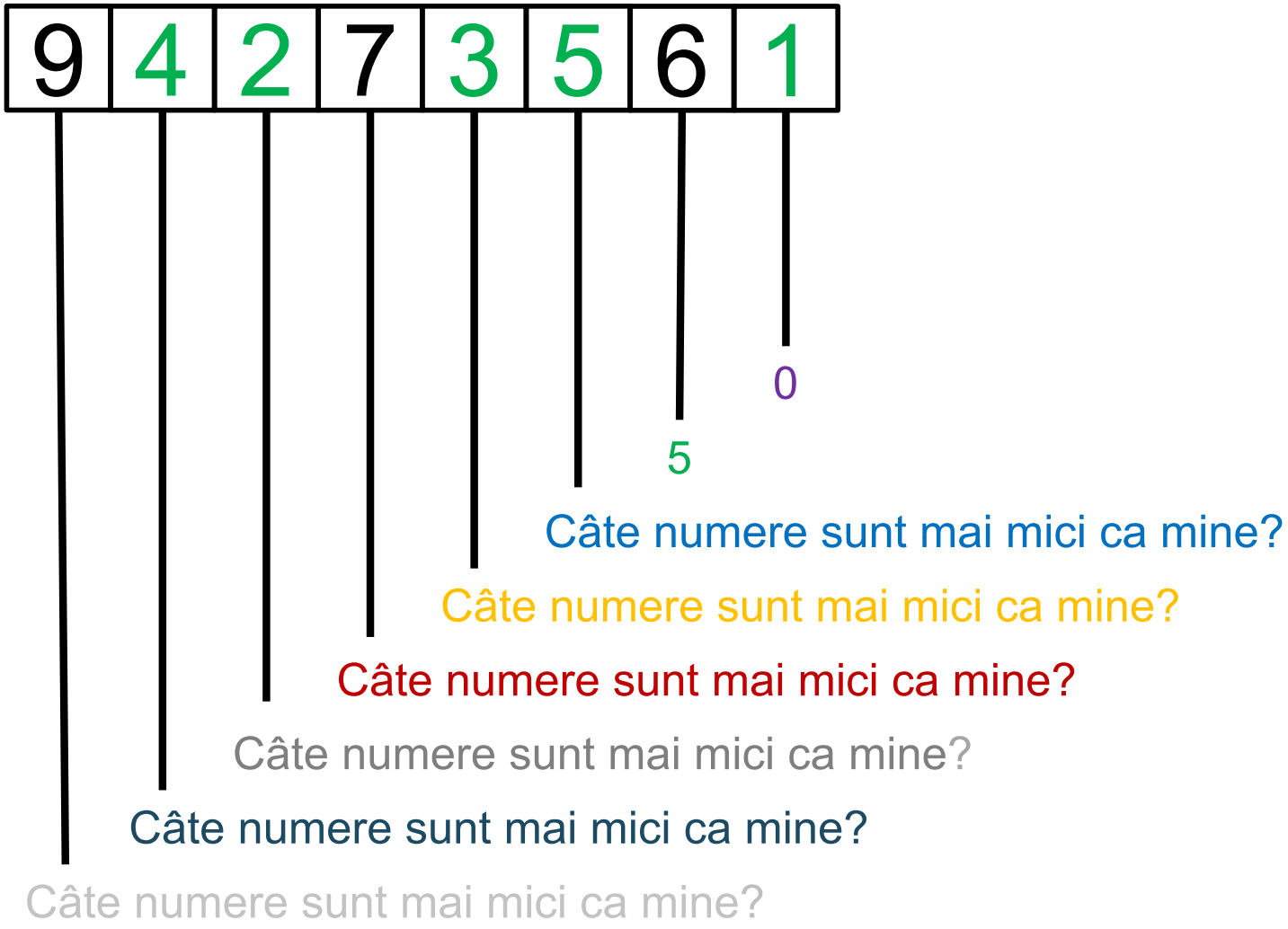
Rank Sort


9 4 2 7 3 5 6 1
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

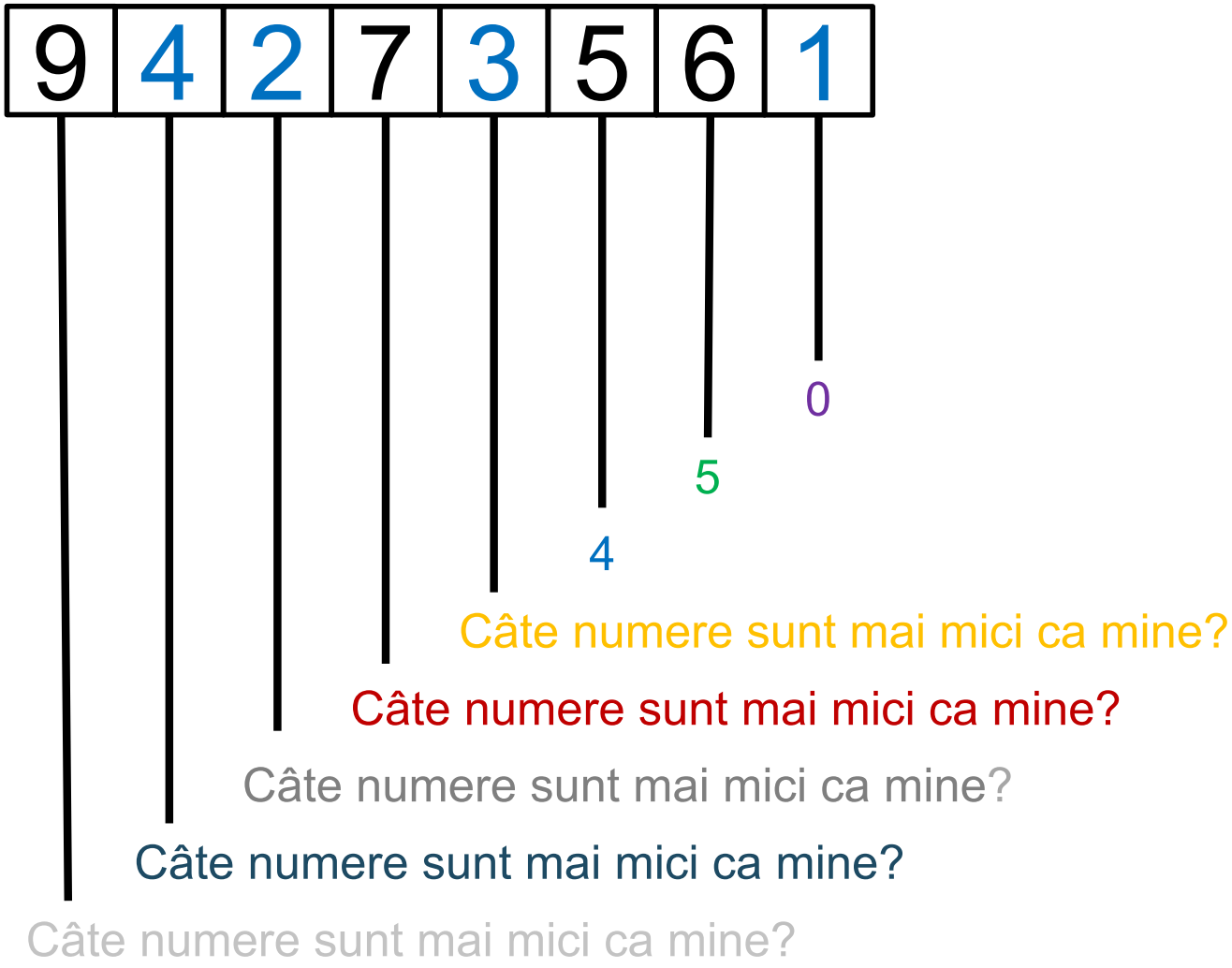
Rank Sort


9 4 2 7 3 5 6 1
0
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

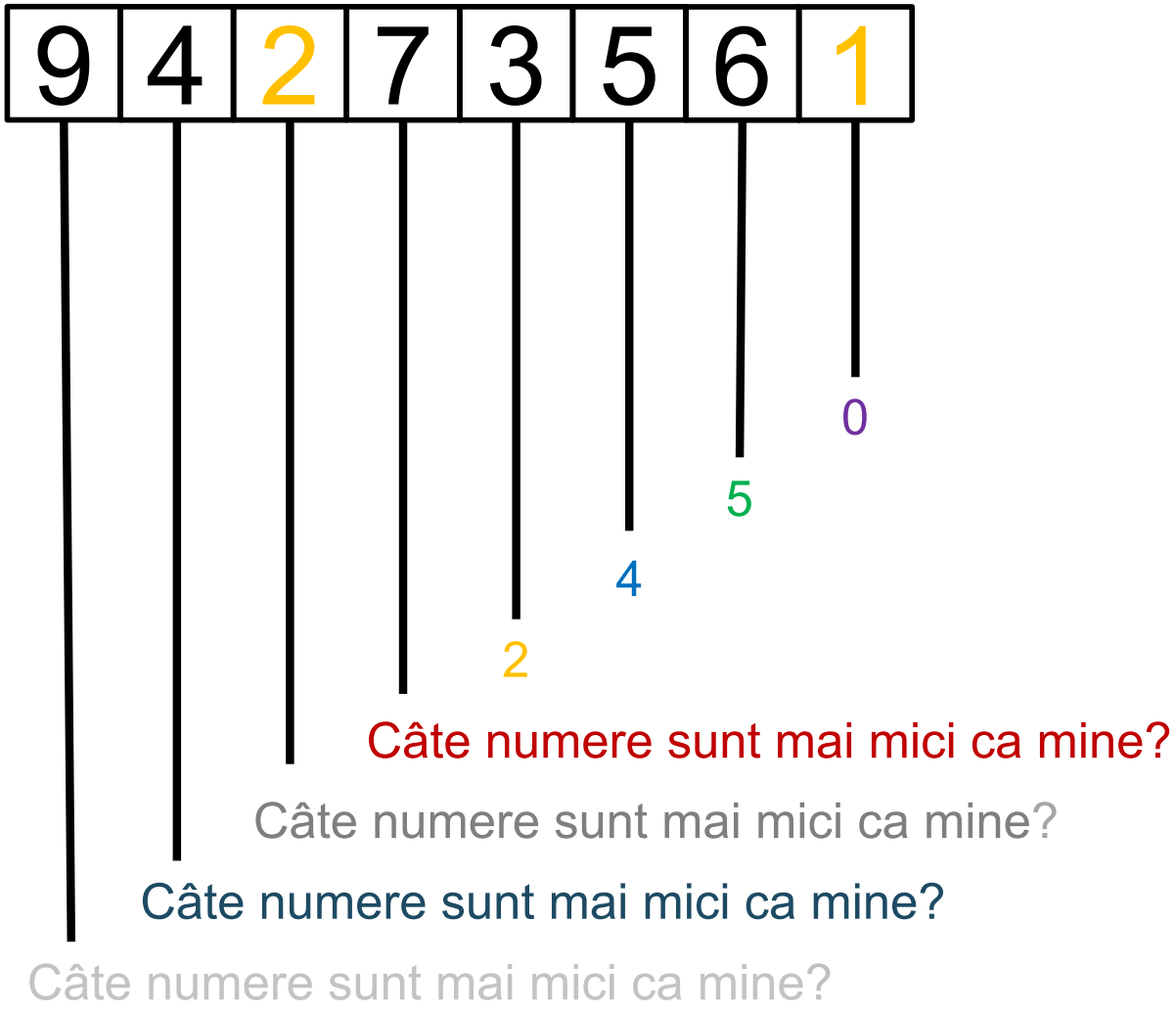
Rank Sort


9 4 2 7 3 5 6 1
0
5
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

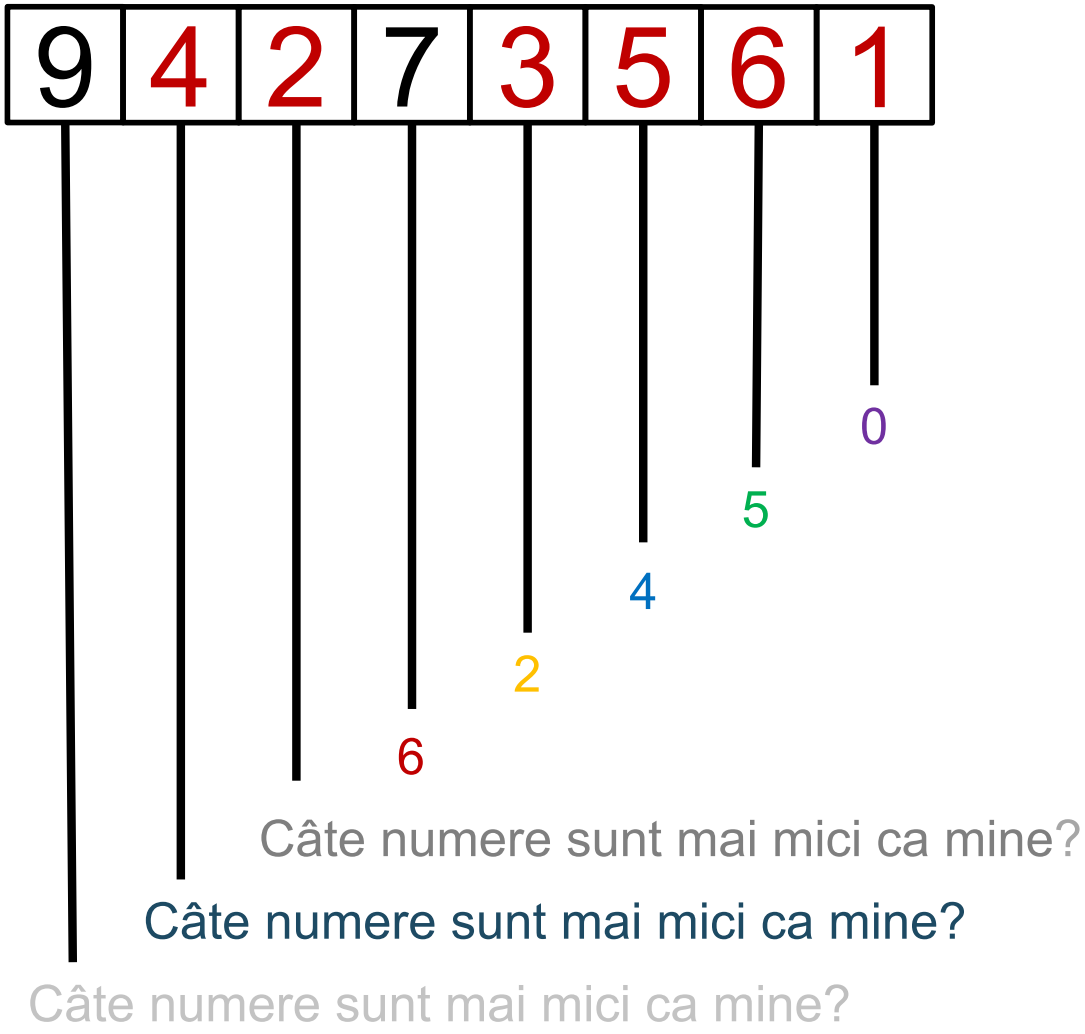
Rank Sort


9 4 2 7 3 5 6 1
0
5
4
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

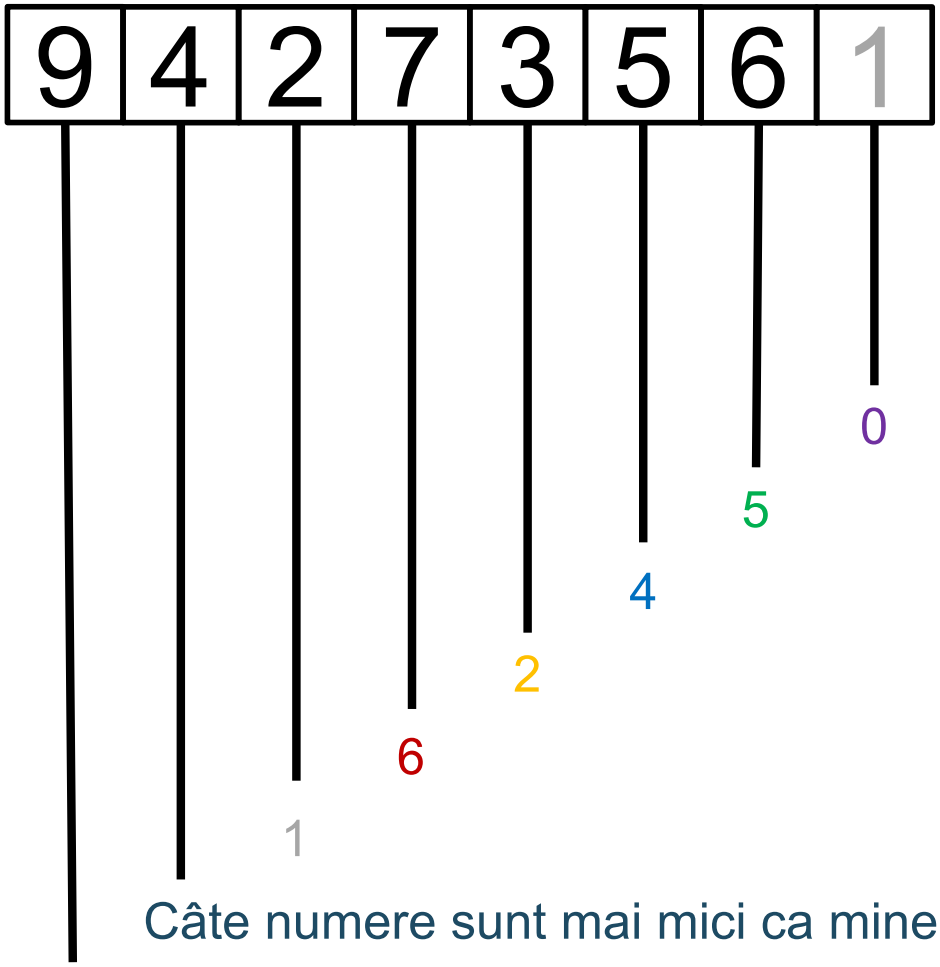
Rank Sort


9 4 2 7 3 5 6 1
0
5
4
2
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

Rank Sort

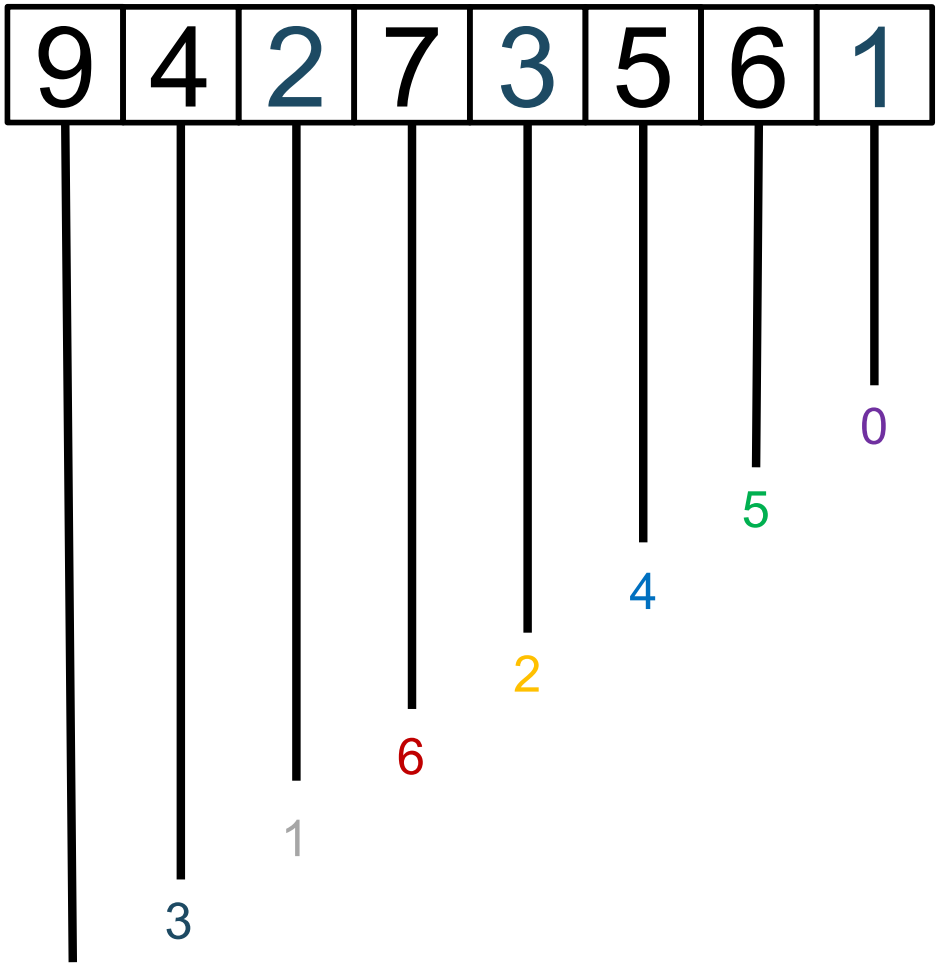

9 4 2 7 3 5 6 1
6
2
4
5
0
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?
Câte numere sunt mai mici ca mine?

Rank Sort



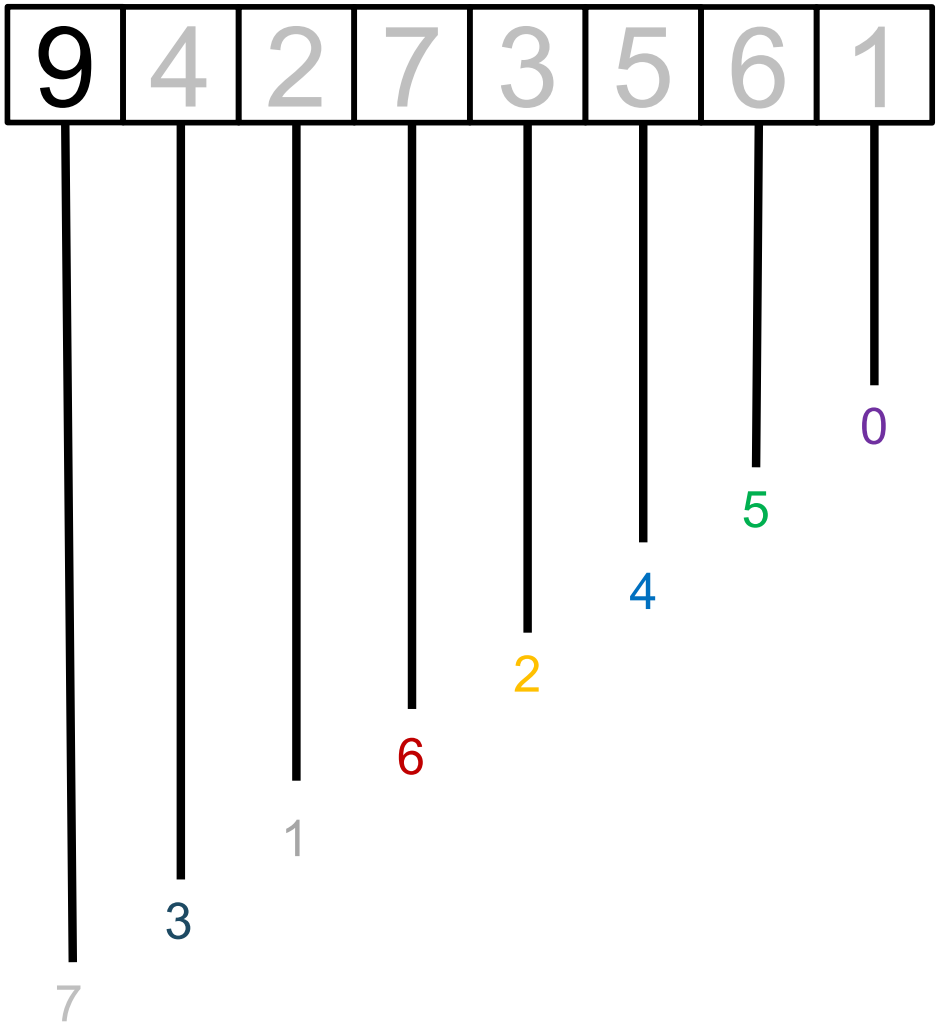
Câte numere sunt mai mici ca mine?

Rank Sort

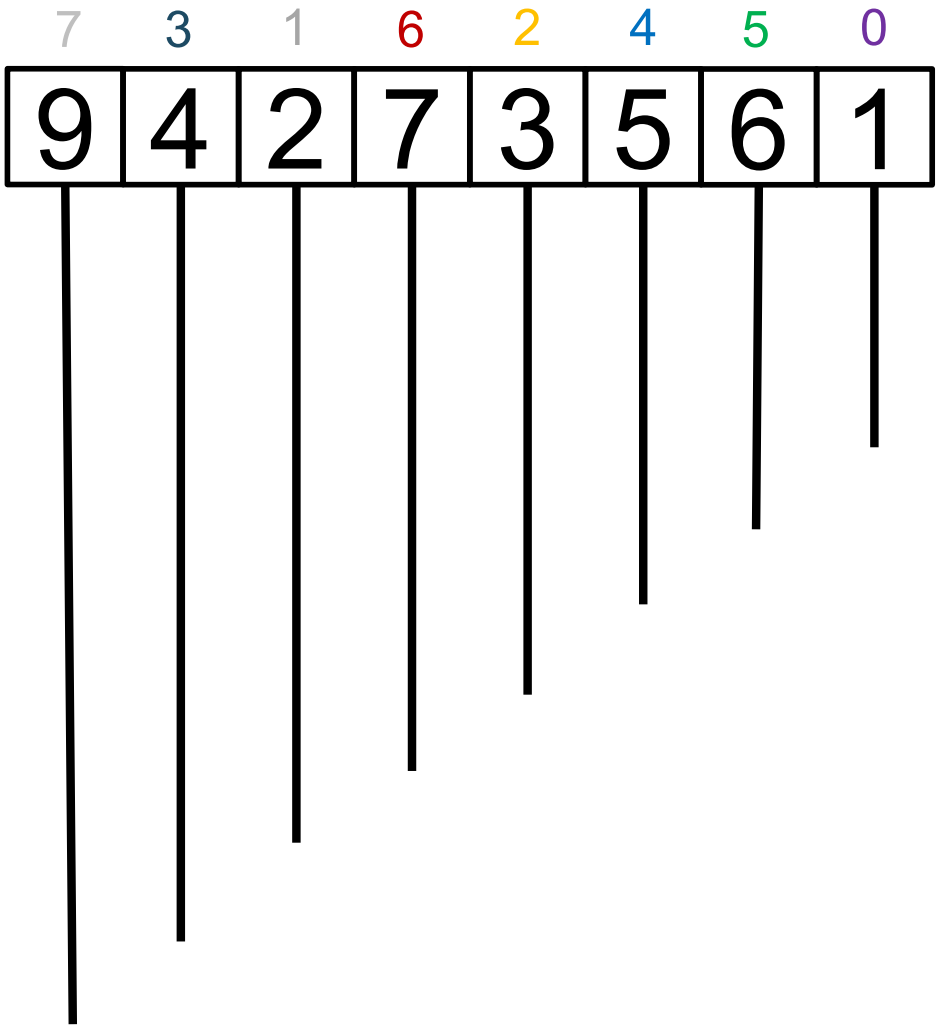


Câte numere sunt mai mici ca mine?

Rank Sort

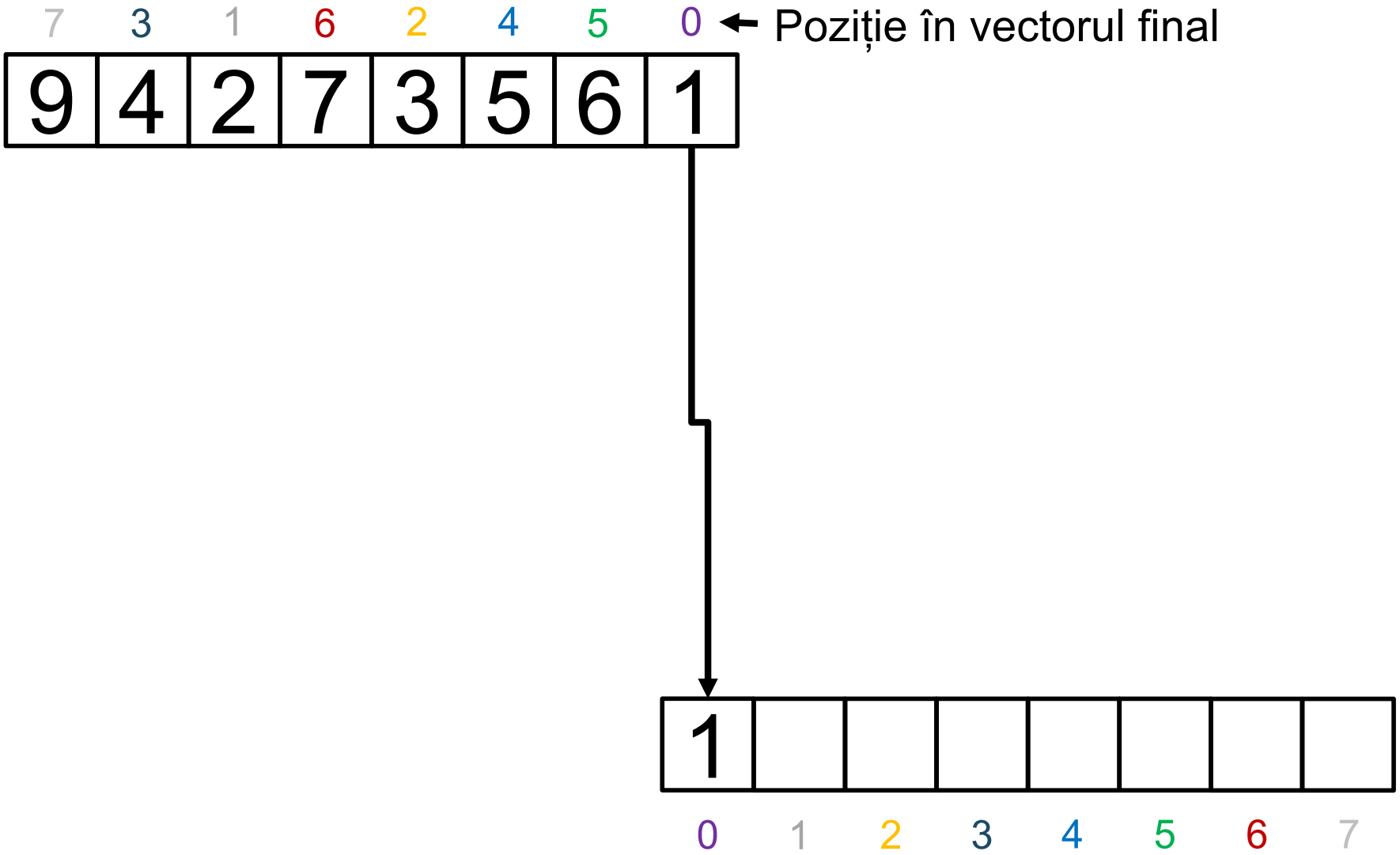


Rank Sort

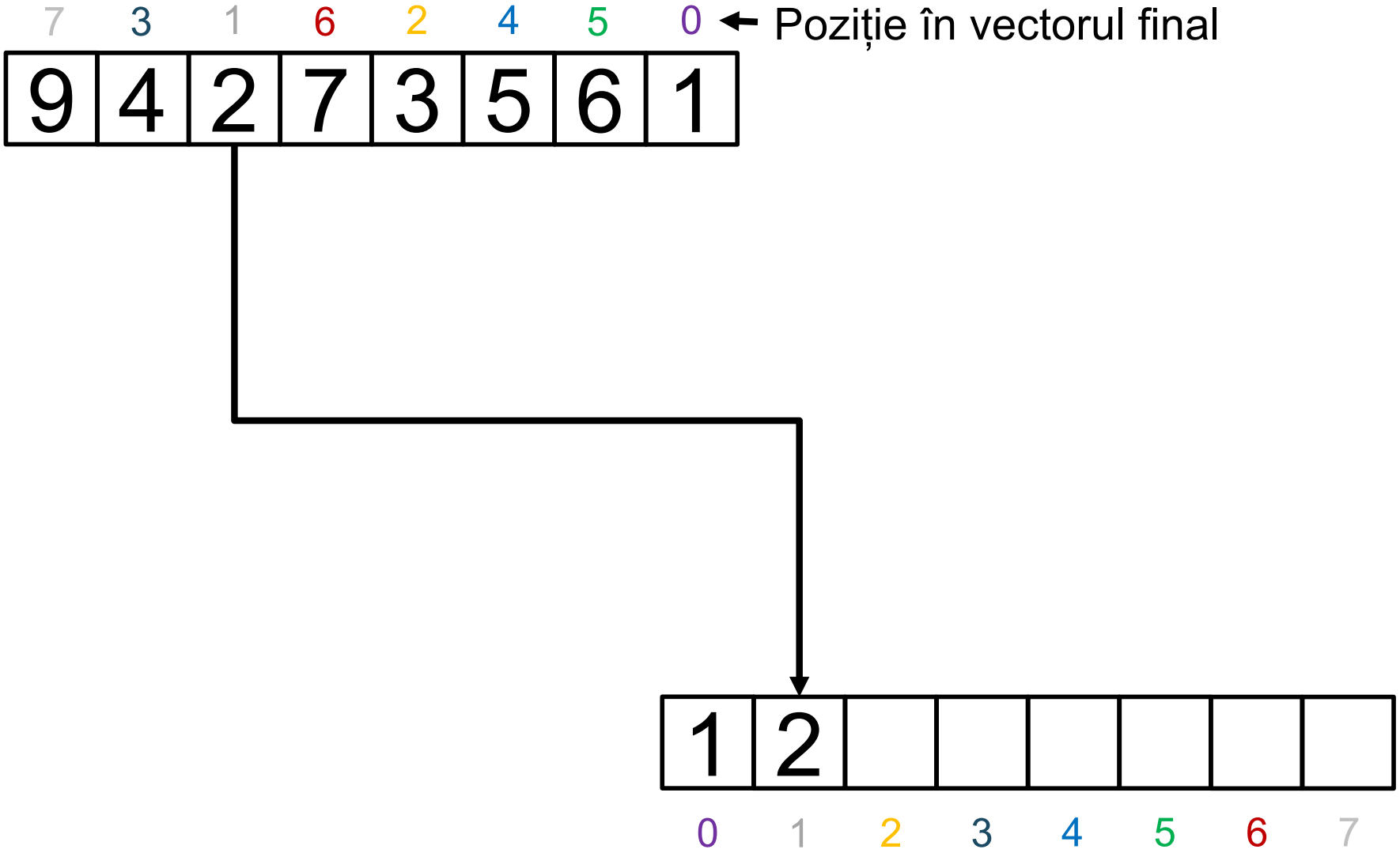


Poziții în vectorul final

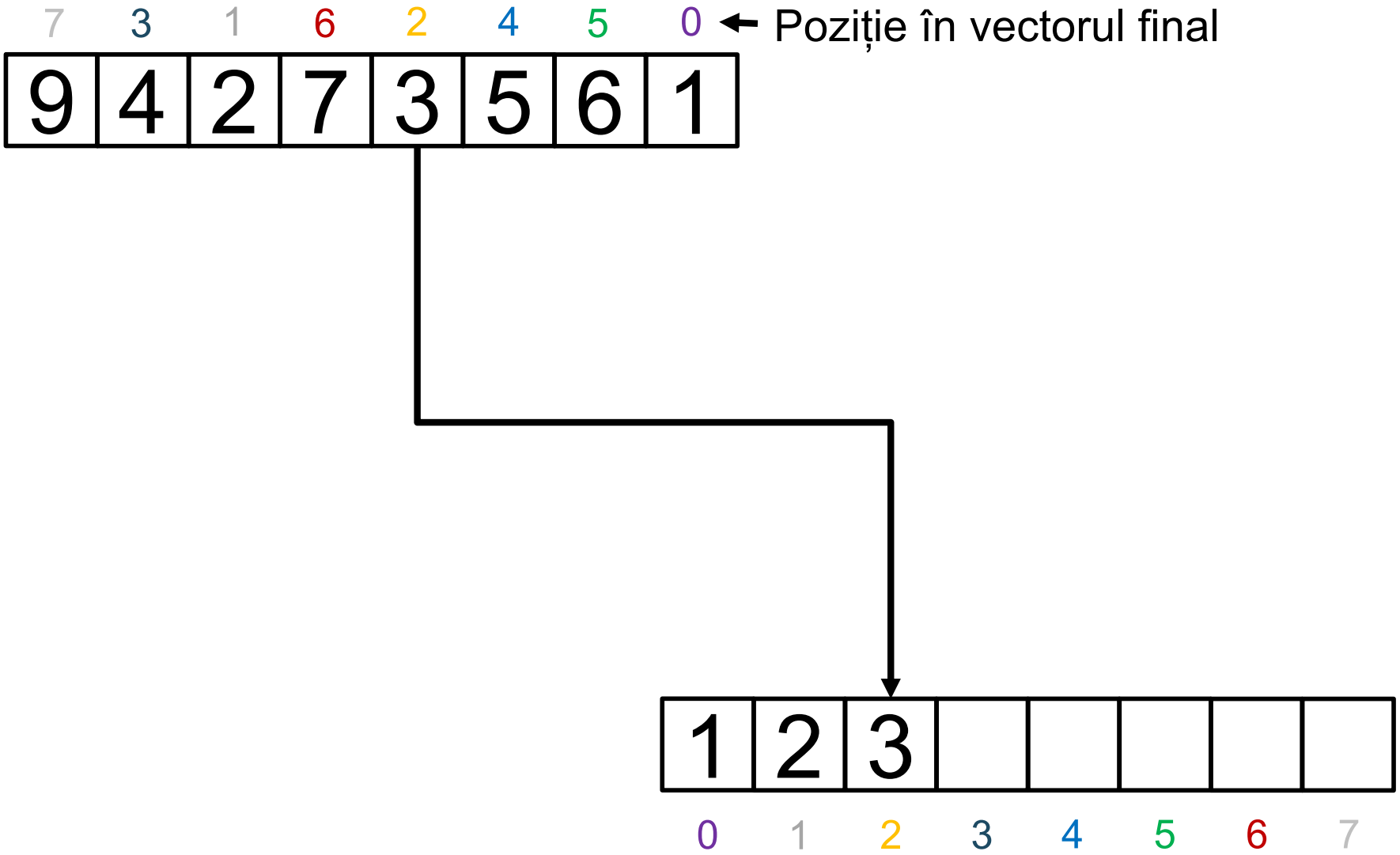
Rank Sort



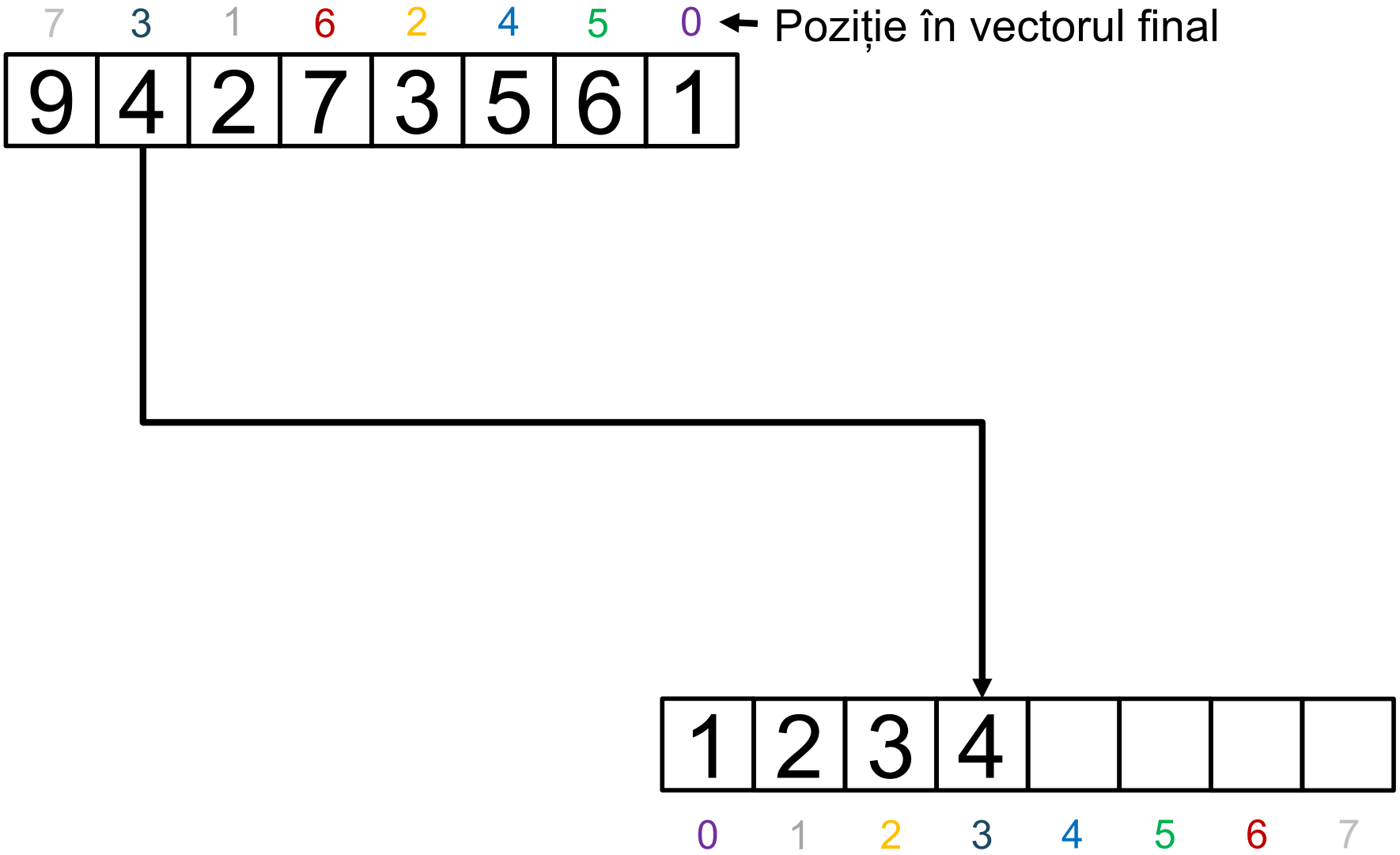
Rank Sort



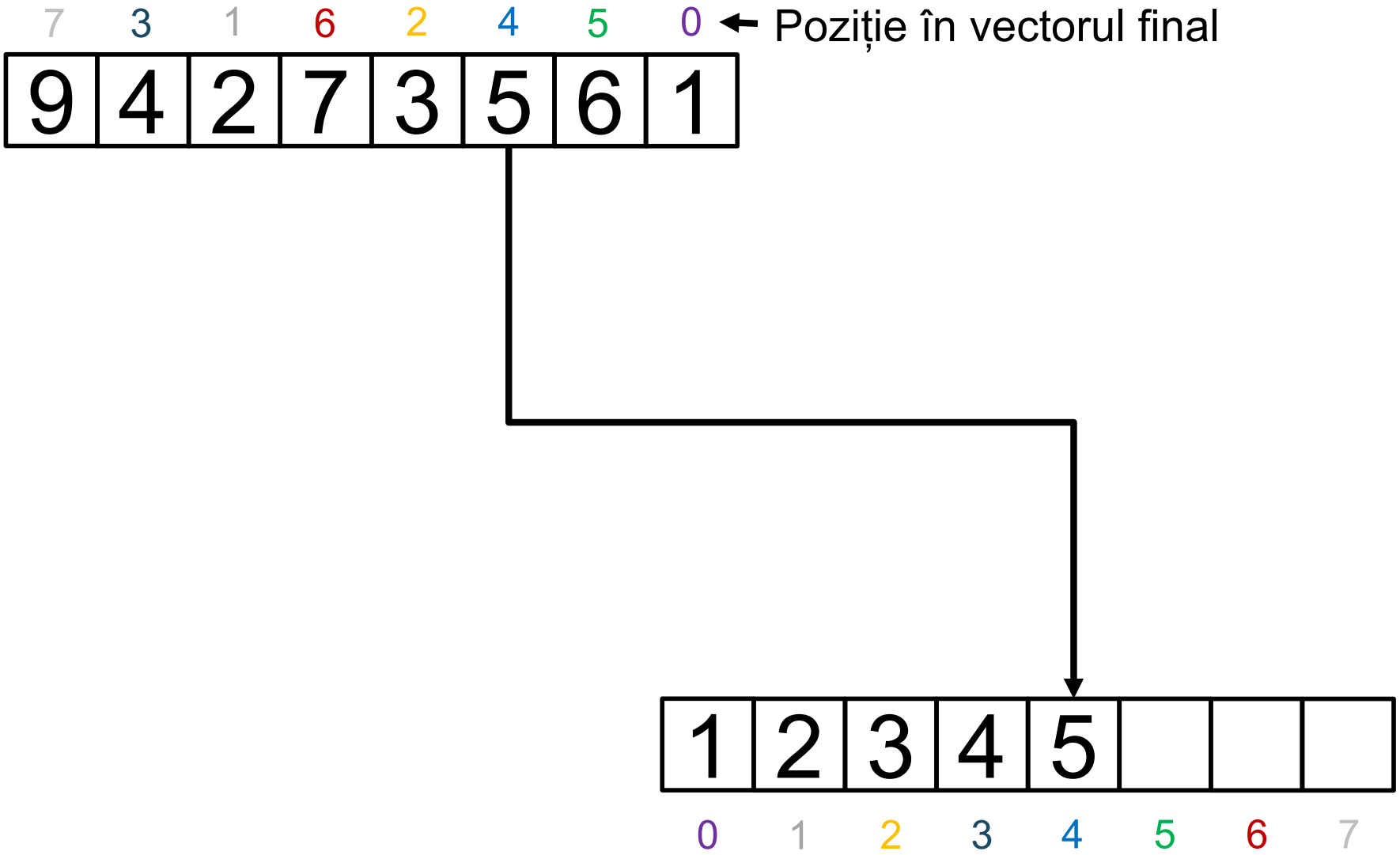
Rank Sort



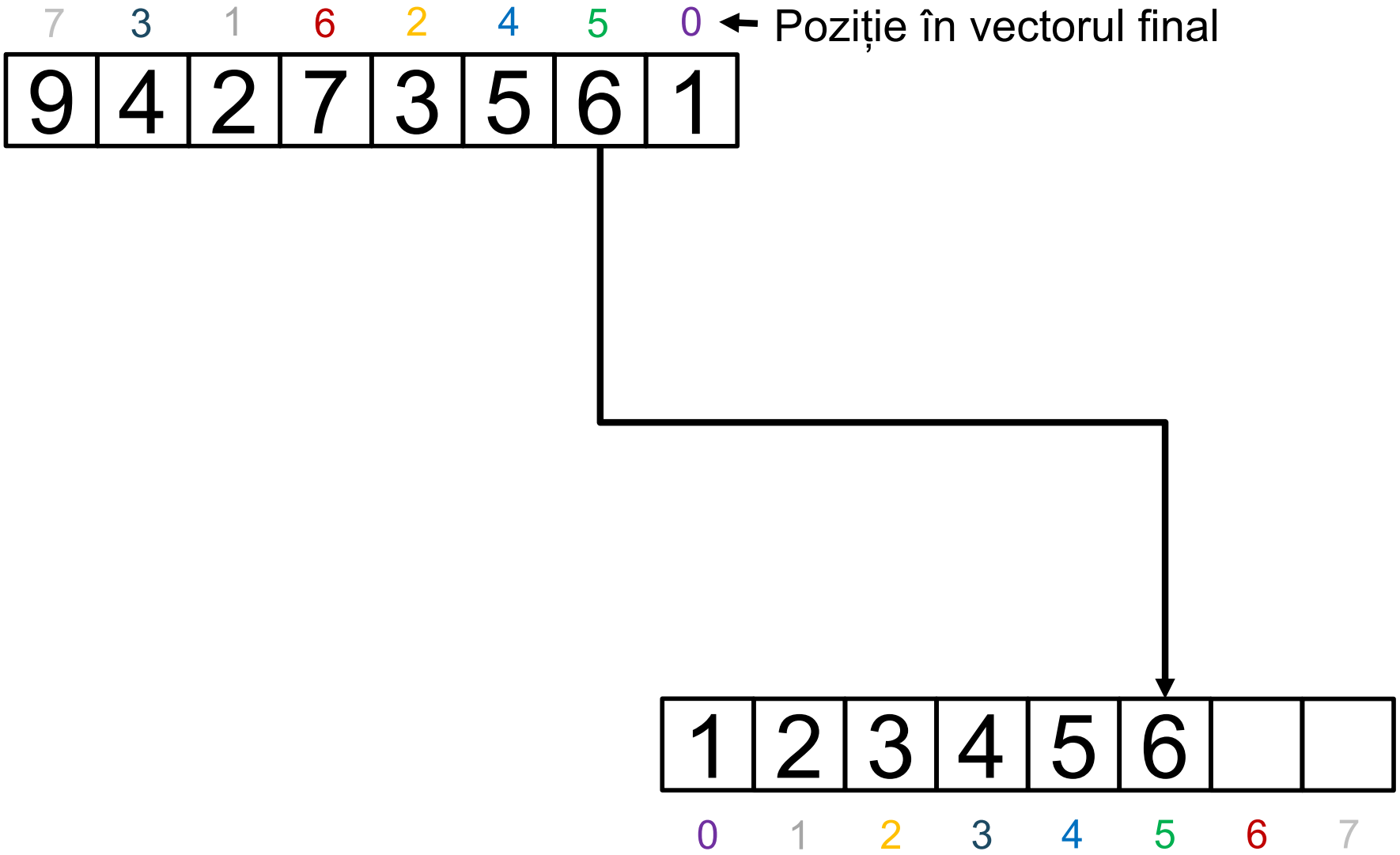
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4
0 1 2 3 4 5 6 7

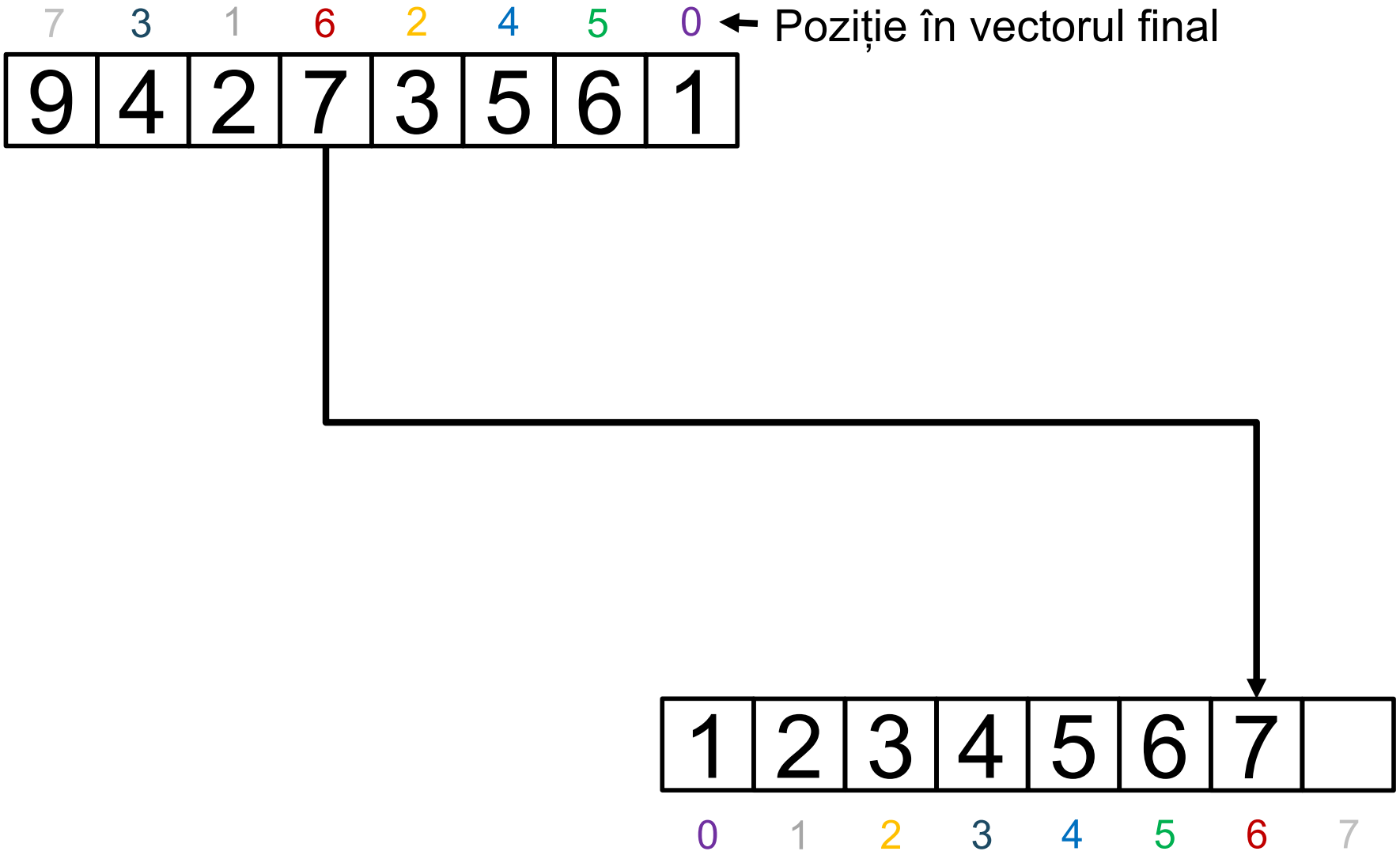
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5
0 1 2 3 4 5 6 7

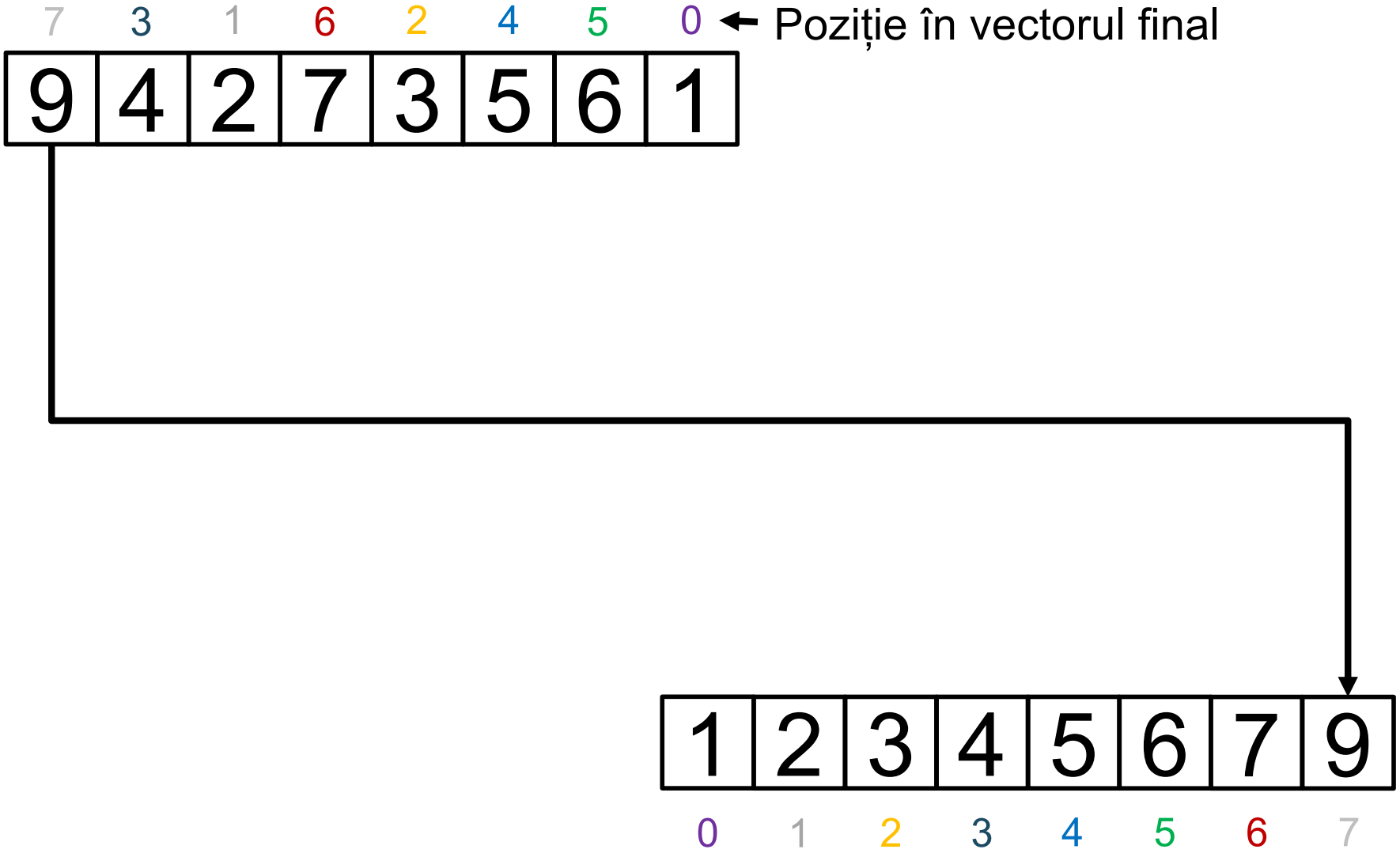
Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6
0 1 2 3 4 5 6 7

Rank Sort


7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6 7
0 1 2 3 4 5 6 7

Rank Sort

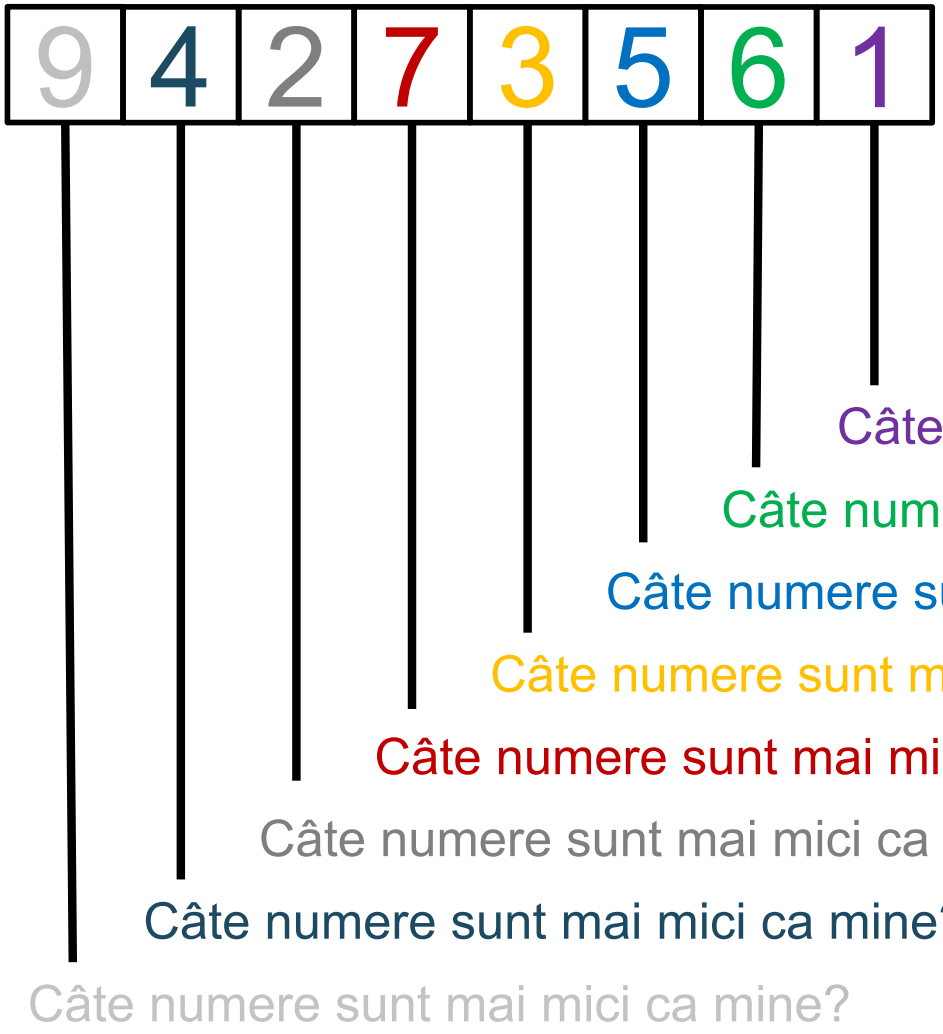

7 3 1 6 2 4 5 0
Poziție în vectorul final
9 4 2 7 3 5 6 1
1 2 3 4 5 6 7 9
0 1 2 3 4 5 6 7

Parallel Rank Sort

9	4	2	7	6	5	6	1

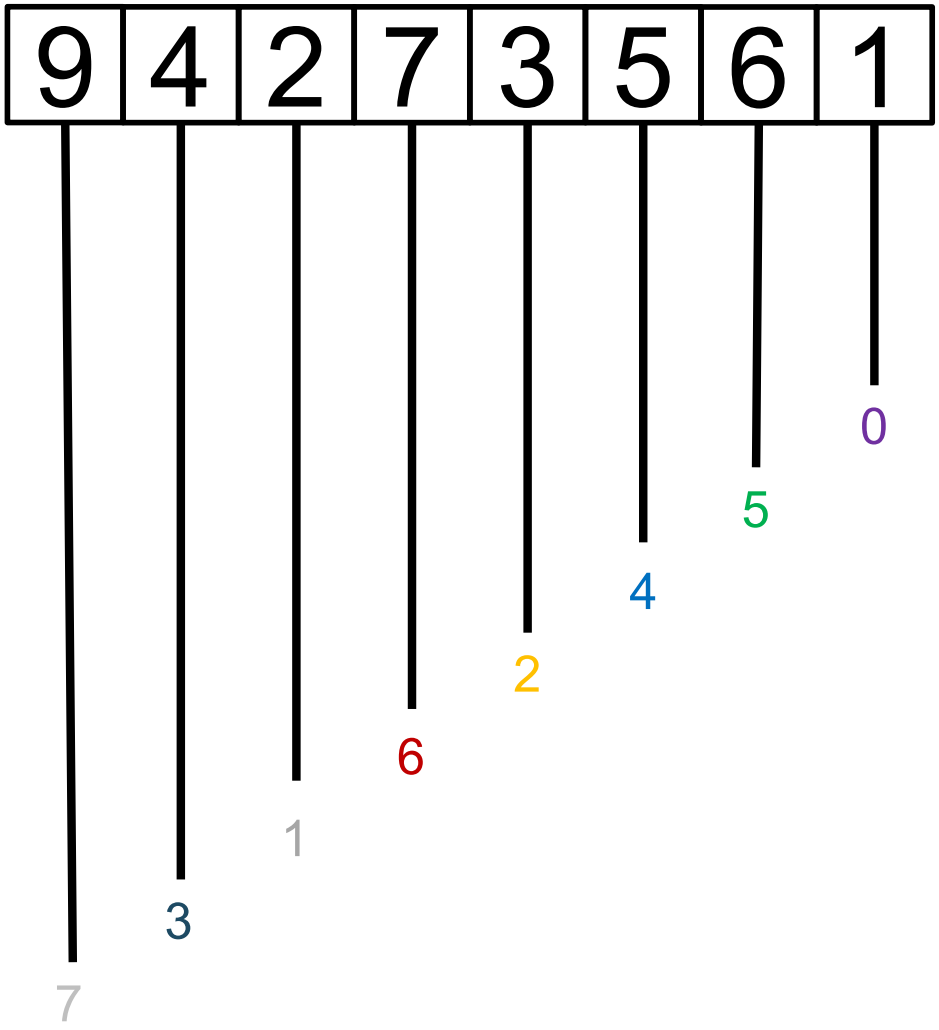
1	2	4	5	6	6	7	9

Parallel Rank Sort



Toate întrebările pot fi răspunse în paralel.

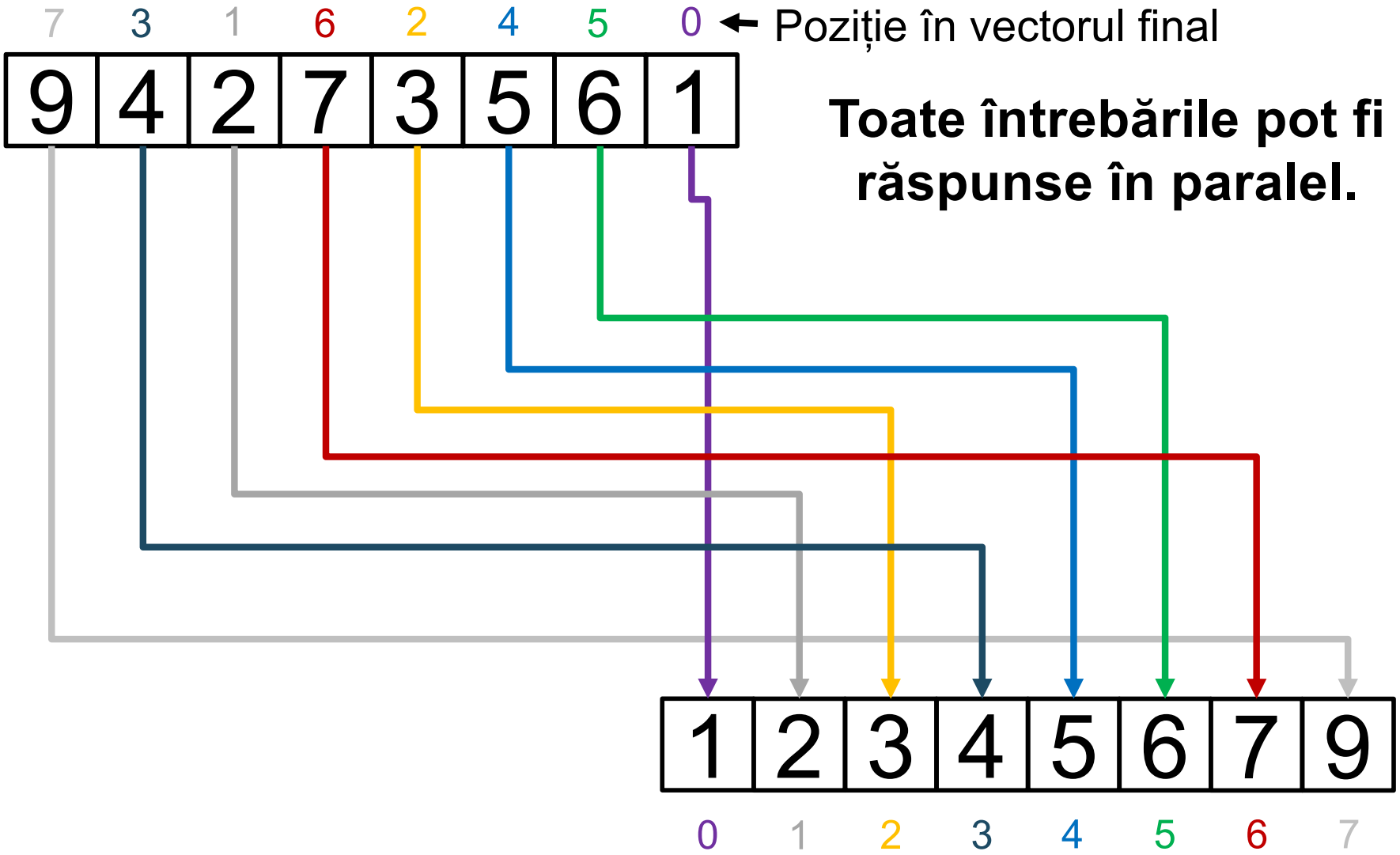
Parallel Rank Sort



Toate întrebările pot fi răspunse în paralel.

Parallel Rank Sort

Parallel Rank Sort



Inmultiri de matrici

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

Matrix multiply

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

$$G = N^3$$

Matrix multiply

Atenție avem N^2 elemente

```
for(i = 0; i < N; i++)
        for(j = 0; j < N; j++)
                for(k = 0; k < N; k++)
                        c[i][j] += a[i][k] * b[k][j]
```

Complexitate?

$$G = N^3$$

Matrix multiply

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Complexitate?

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Complexitate?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$T = \frac{N^3}{P}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$S = \frac{N^3}{\frac{N^3}{P}}$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Matrix multiply

Speedup?

$$S = P$$

```
co[Tid = 1 to P]
{
        start = Tid * ceil(N/P)
        end = min((Tid+1) * ceil(N/P),N)
        for(i = start; i < end; i++)
                for(j = 0; j < N; j++)
                        for(k = 0; k < N; k++)
                                c[i][j] += a[i][k] * b[k][j]
}
```

Super-linear speedup?

$S > P?$

- *"... the effort expended on achieving high parallel processing rates is wasted unless it is accompanied by achievements in sequential processing rates of very nearly the same magnitude."*

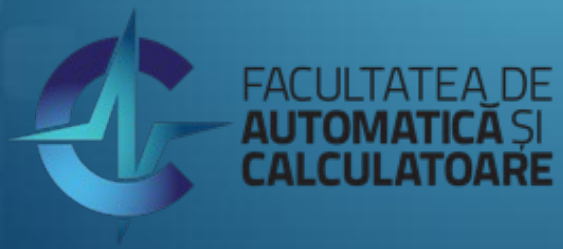
Amdhal, 1967

- *"... speedup should be measured by scaling the problem to the number of processors, not by fixing the problem size."*

Gustafson, 1988

Algoritmi Paraleli și Distribuiți
Lucru cu semafoare

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



Sincronizare: Notiuni introductive

Dezvoltarea algoritmilor folosind variabile partajate (MIMD)

- Sincronizarea: excluderea mutuală şi sincronizarea condiţionată.
- Câteva rezultate notabile:
  - mutex
  - semafoare
    - regiuni critice
  - bariere
- Câteva dintre problemele *clasice*:
  - producători-consumatori
  - problema filozofilor
  - cititori-scriitori
  - problema bărbierului

Sectiuni critice

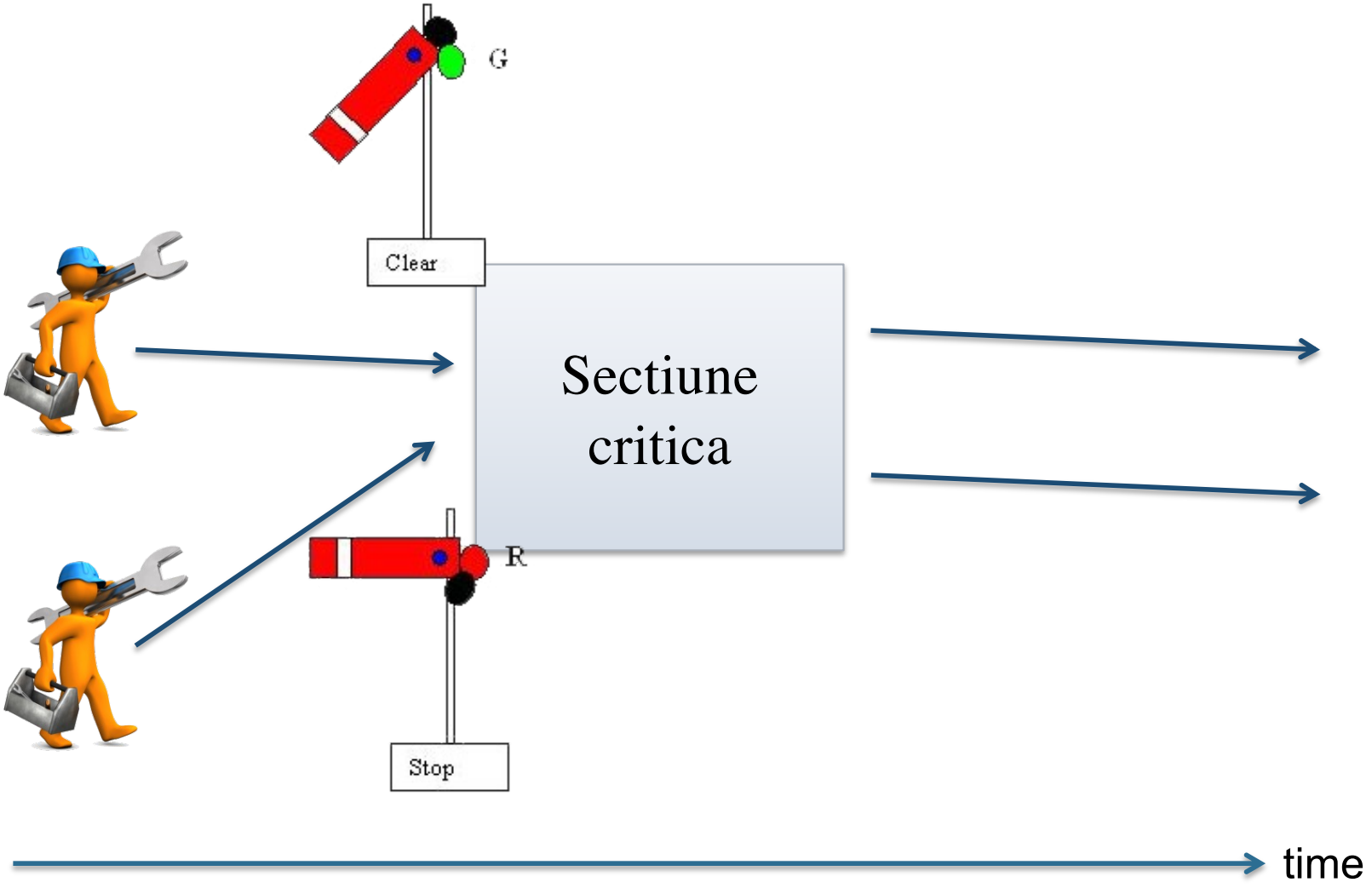
- Problema: fiecare proces P(i) al unei colectii de procese P(i:1..n) executa ciclic o sectiune critica, in care are acces exclusiv la anumite resurse partajate, urmata de o sectiune necritica in care foloseste doar resurse locale.

- Semafor = tip special de variabile partajate manipulate prin 2 operații atomice: P și V *(Dijkstra, 1965)*
  - Asigura excluderea mutuala intre procesele care acceseaza sectiunea critica.

Secțiuni critice

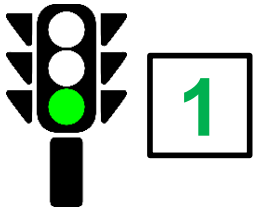
```
sem mutex= 1;
process P[i=1 to n] {
    while (true) {
      P(mutex); /* acaparare secțiune critică */
      Secțiune critică;
      V(mutex); /* eliberare secțiune critică */
      Secțiune necritică
    }
}
```

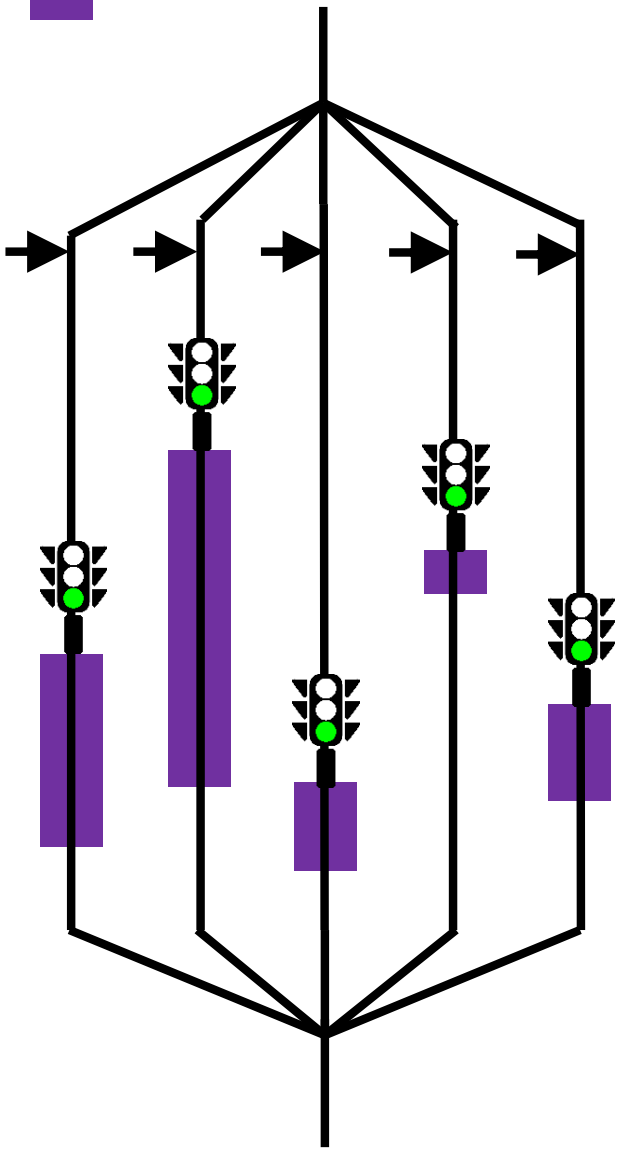
Exemplu executie…



Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

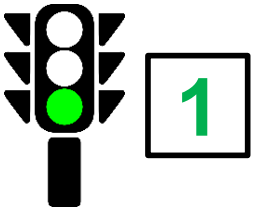


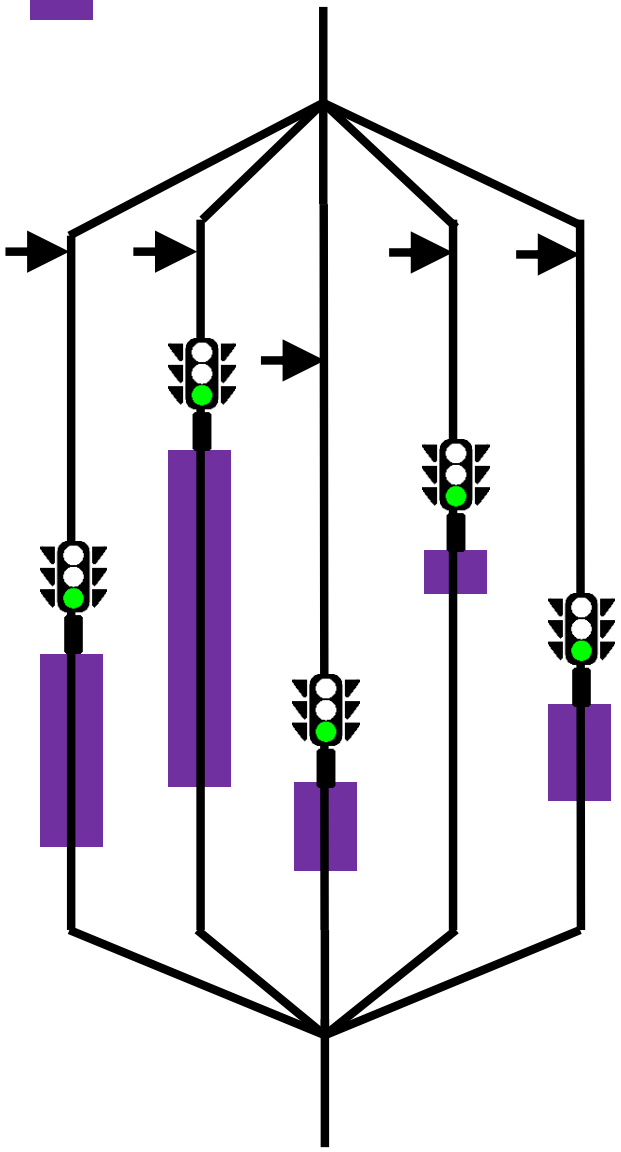


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra

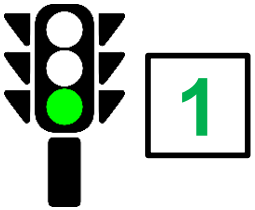


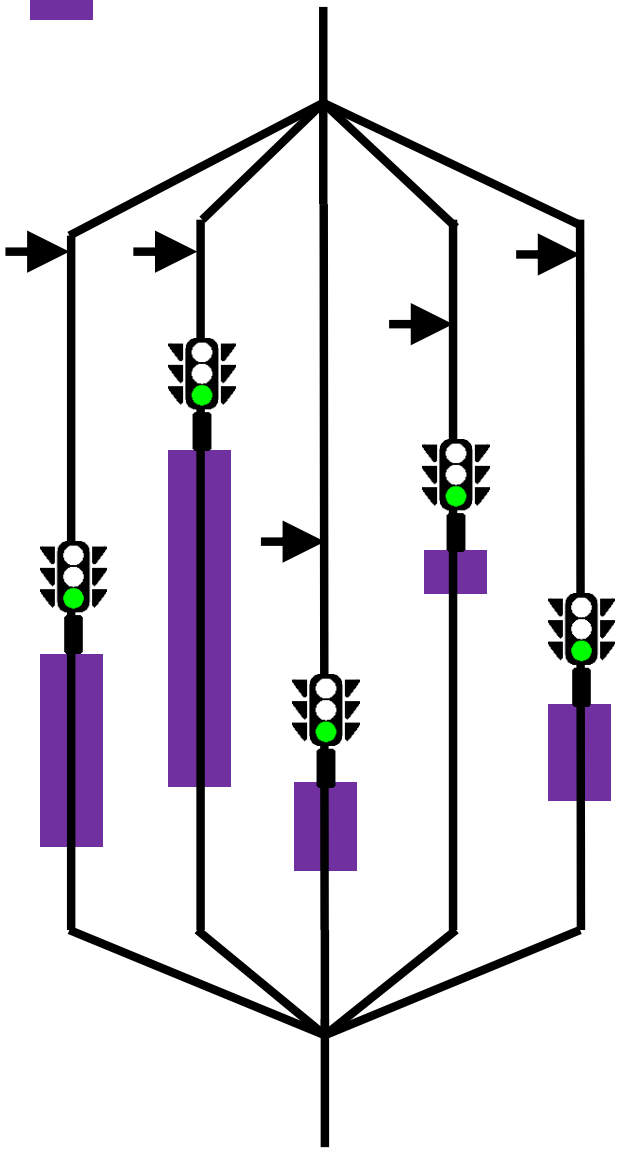


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra

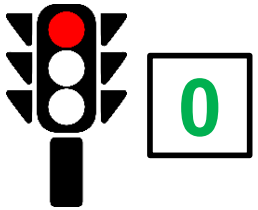


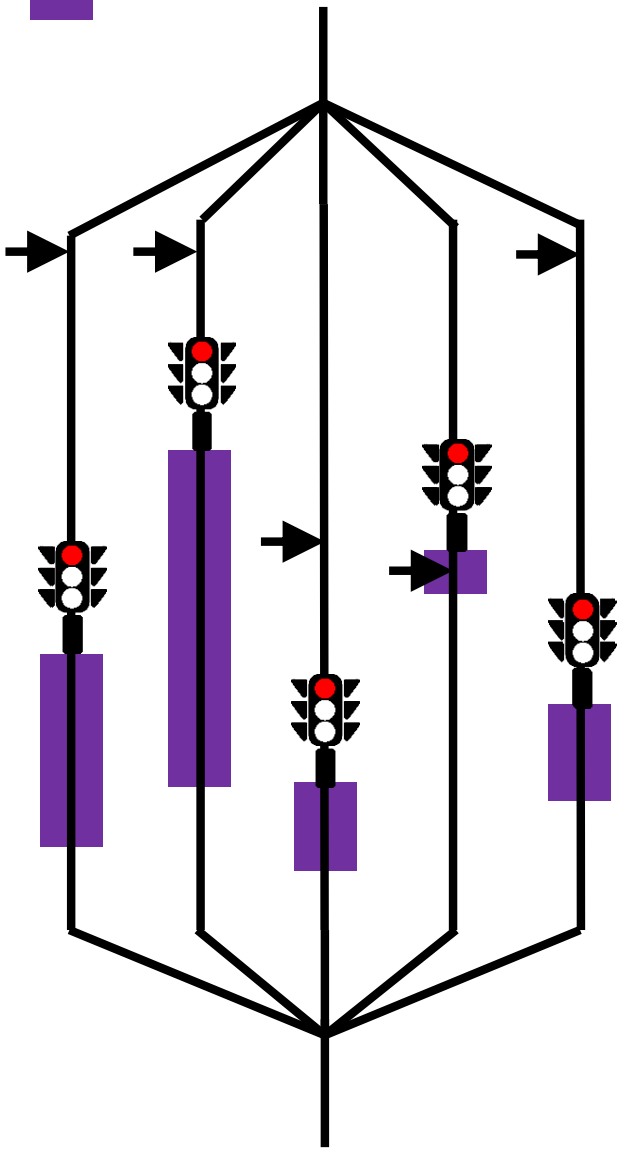


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

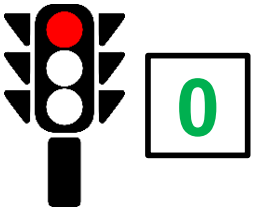


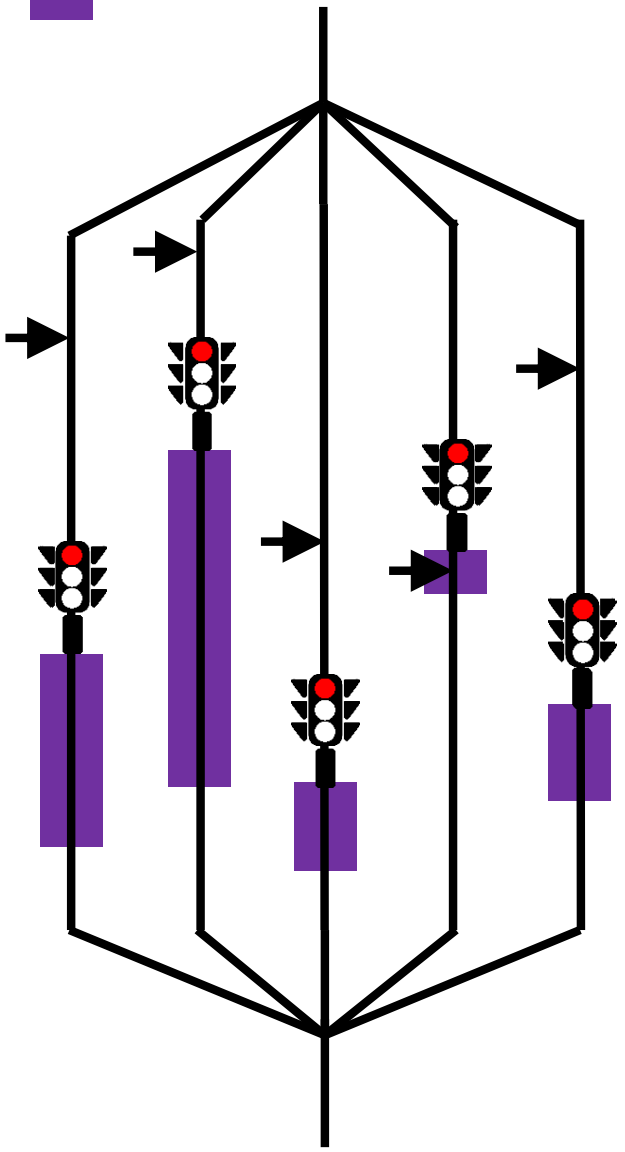


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

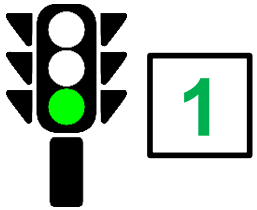


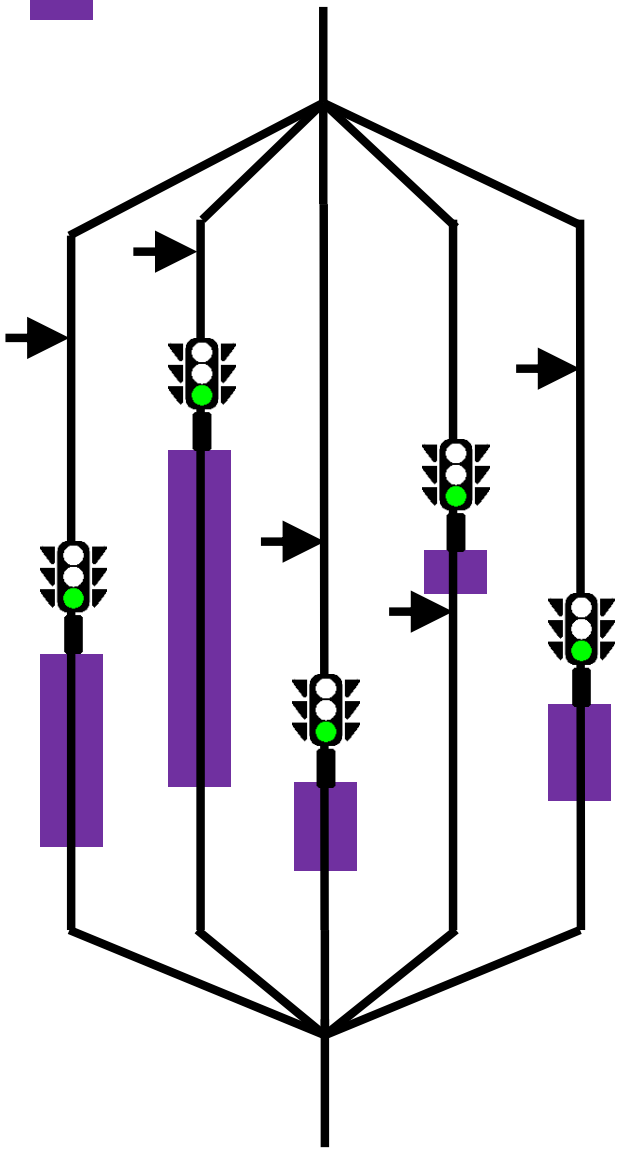


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

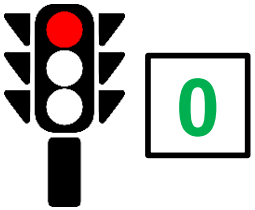


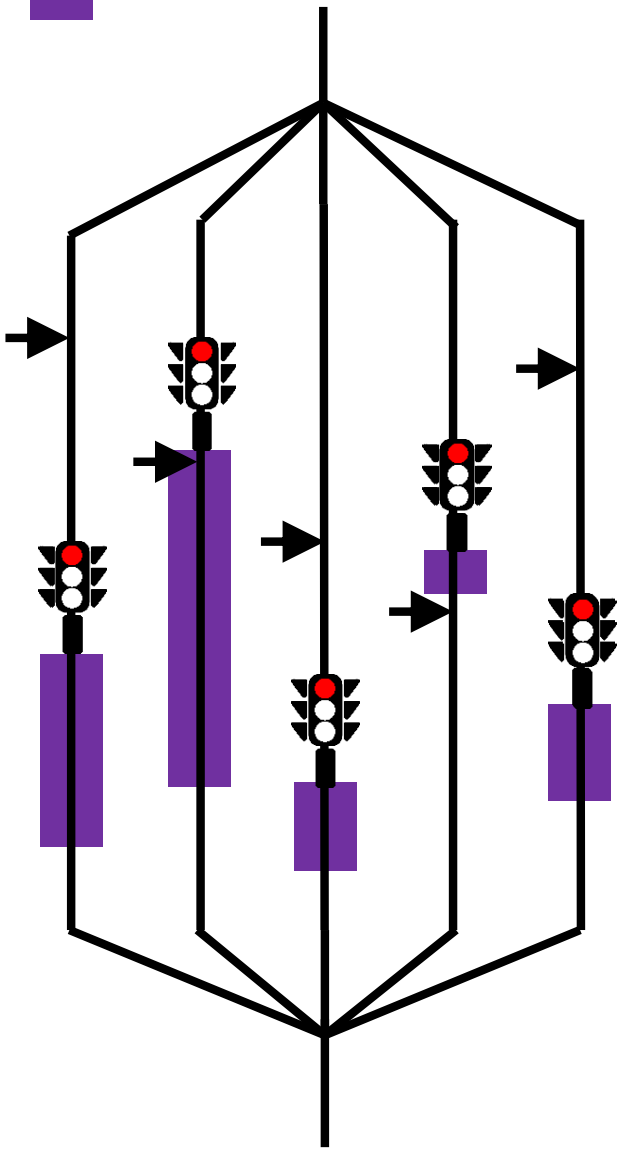


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

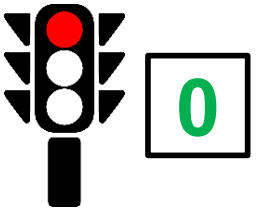


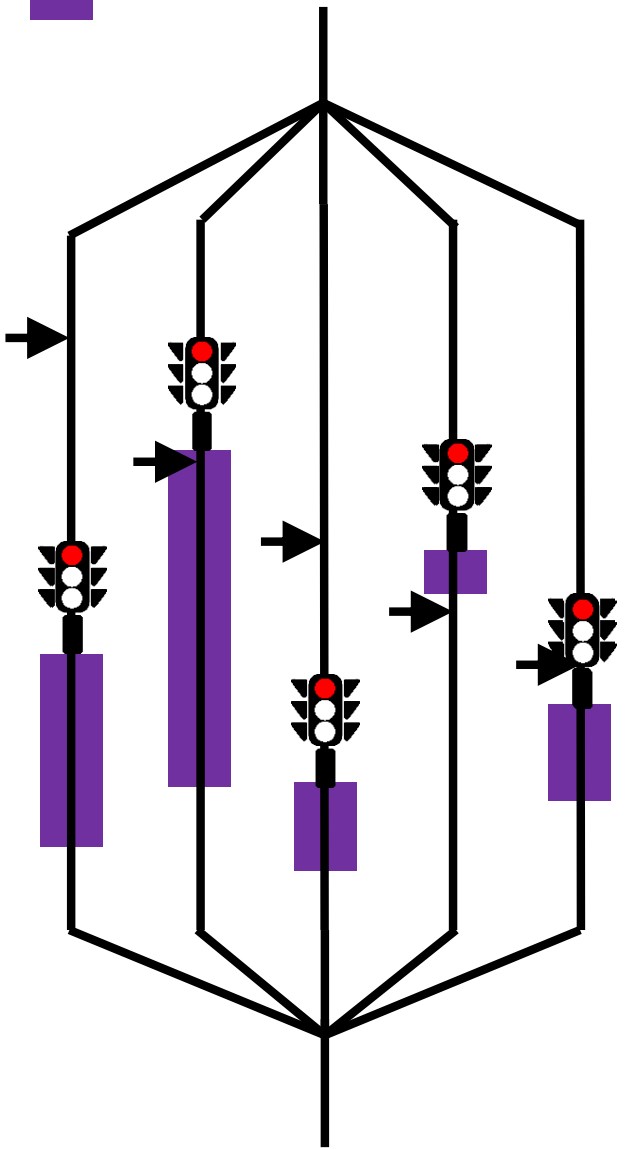


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

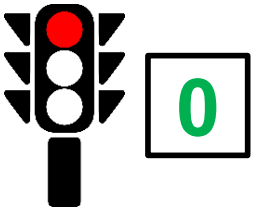


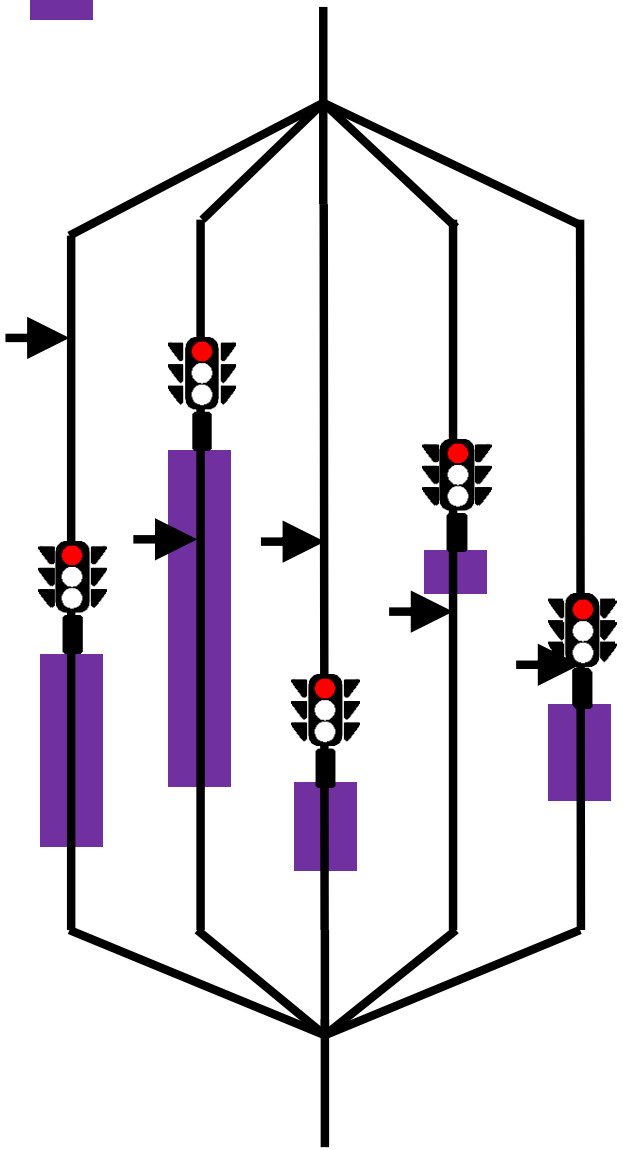


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

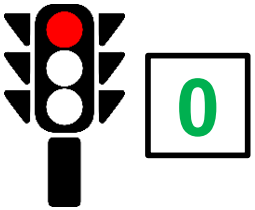


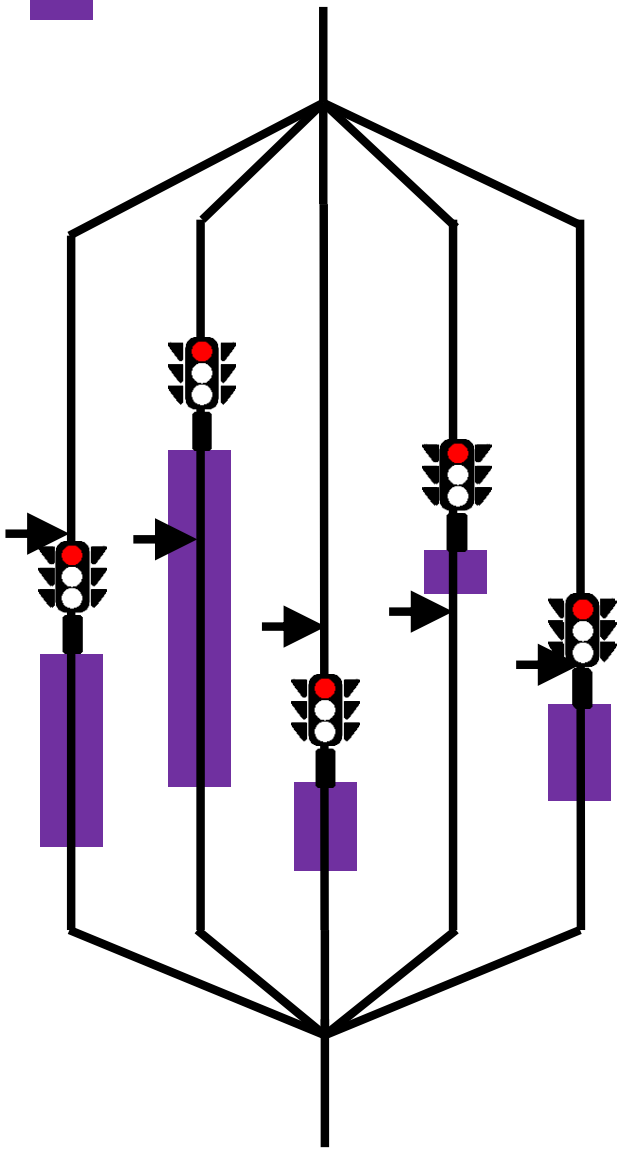


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra

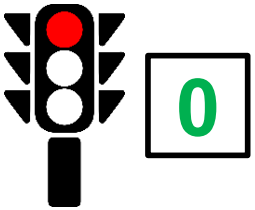


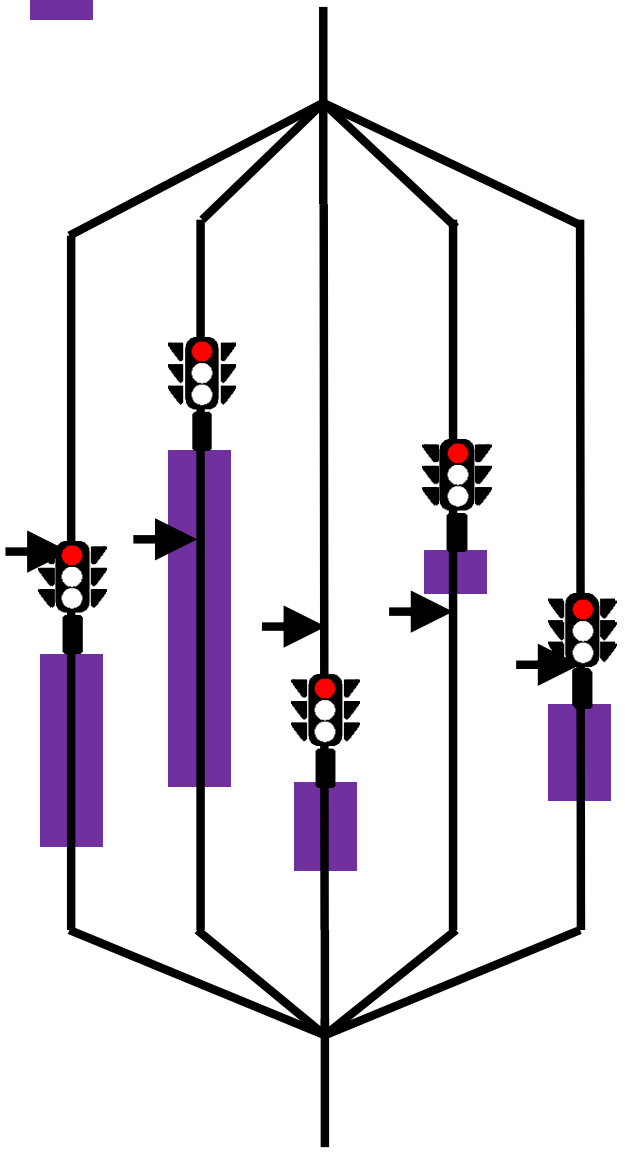


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
- Dijsktra

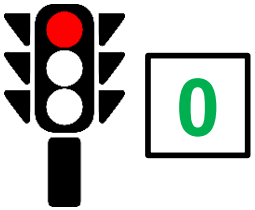


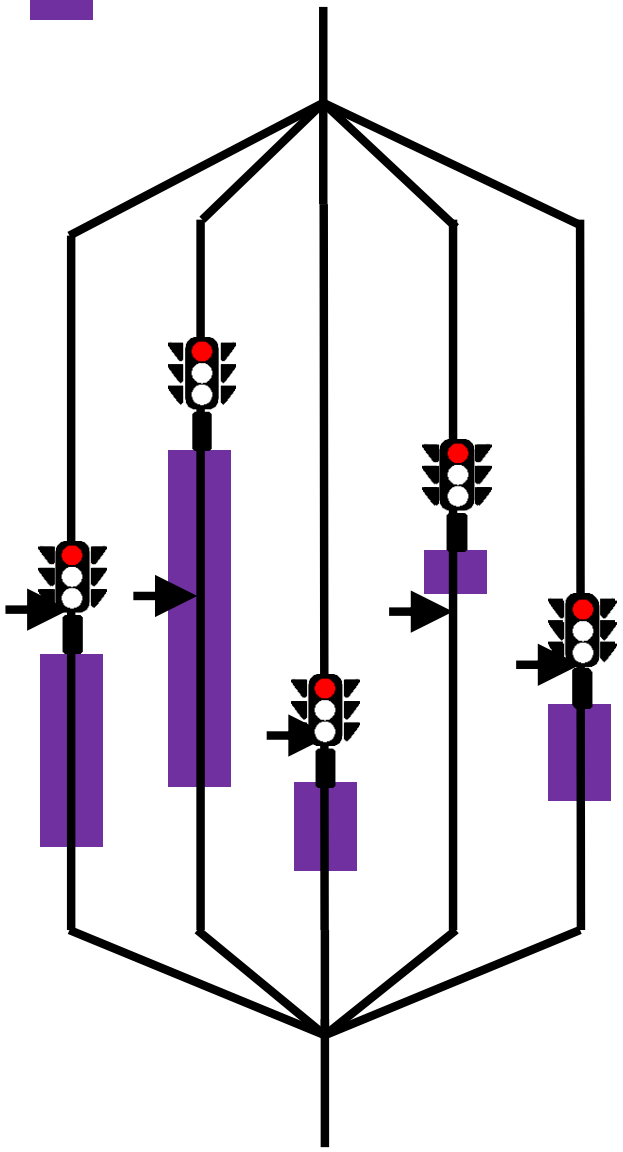


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

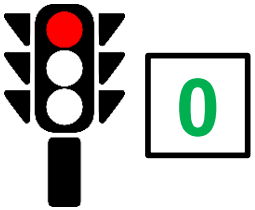


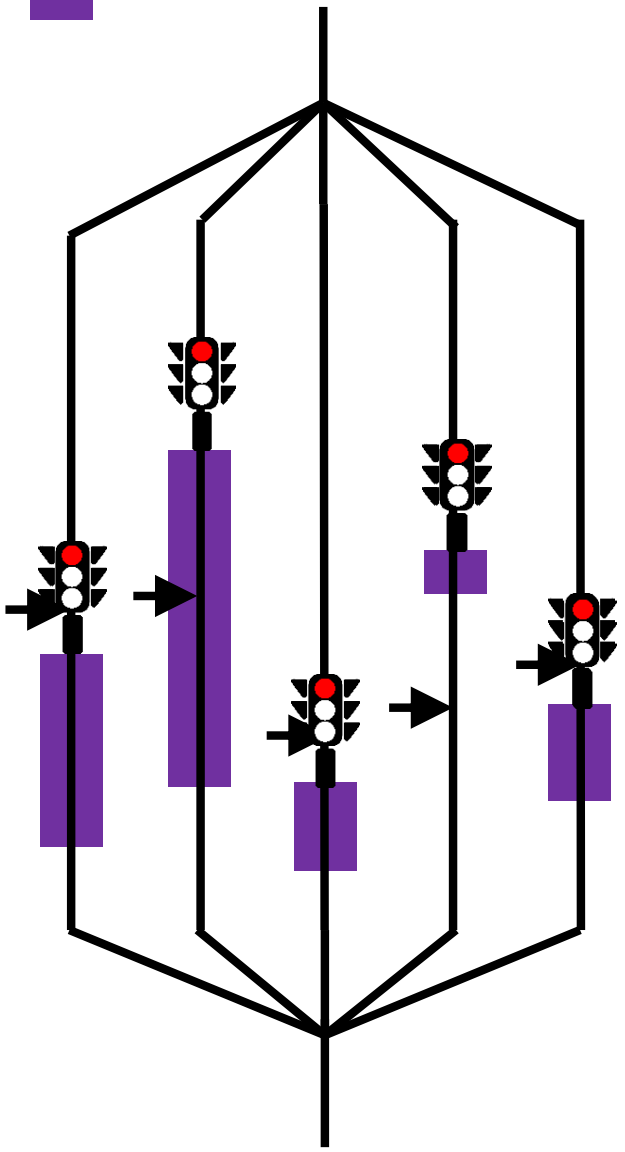


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

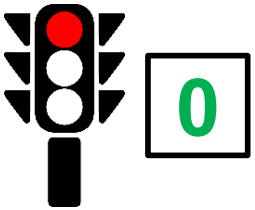


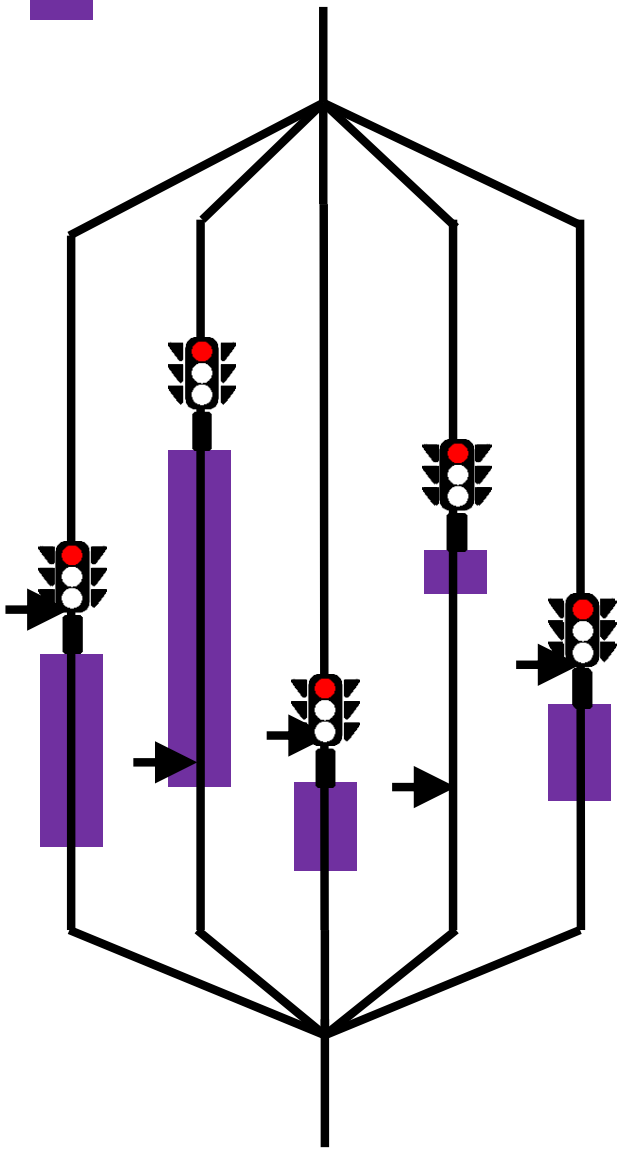


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

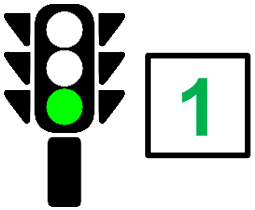


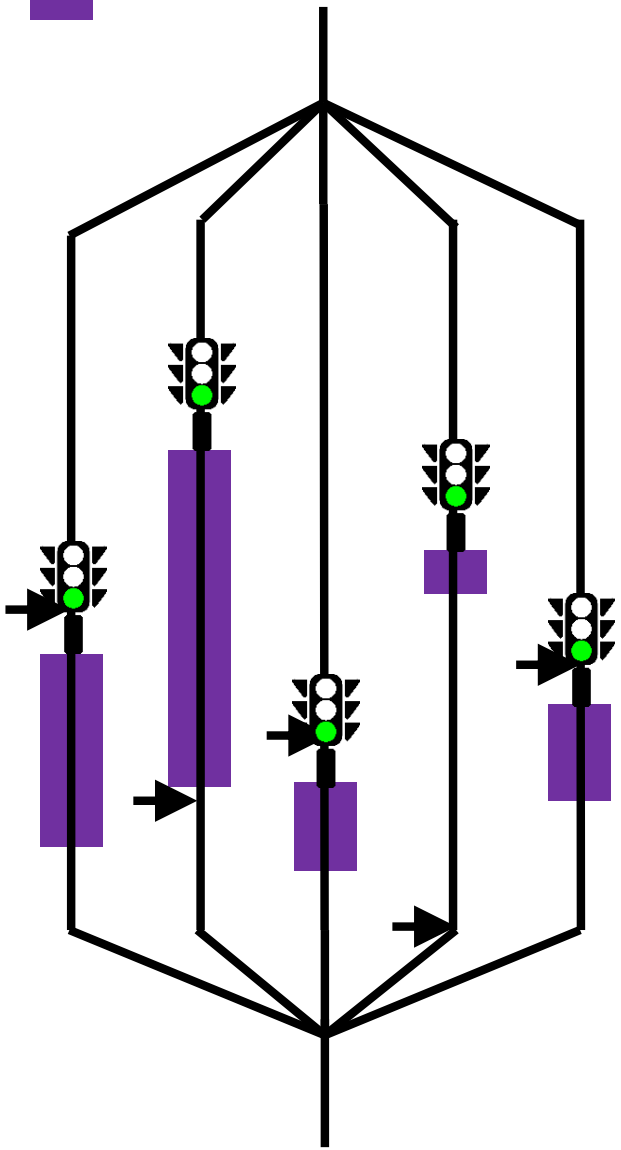


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

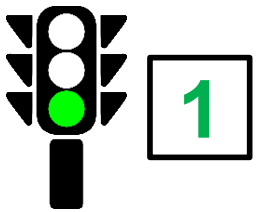


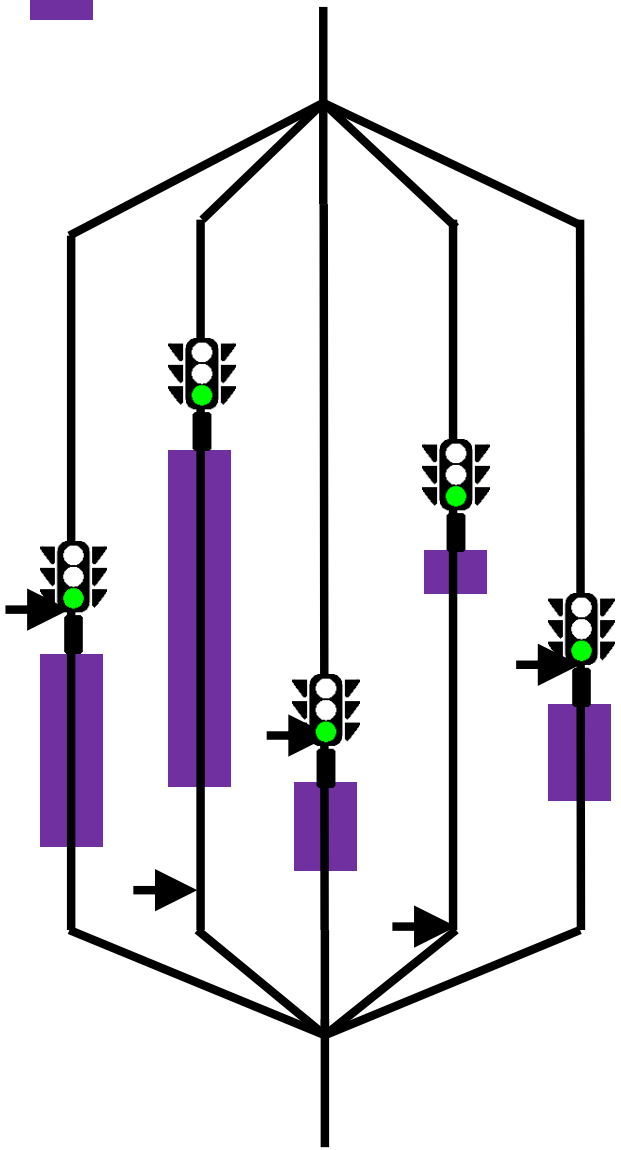


Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra

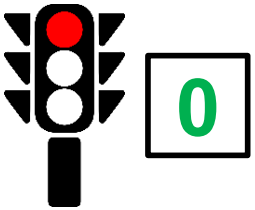




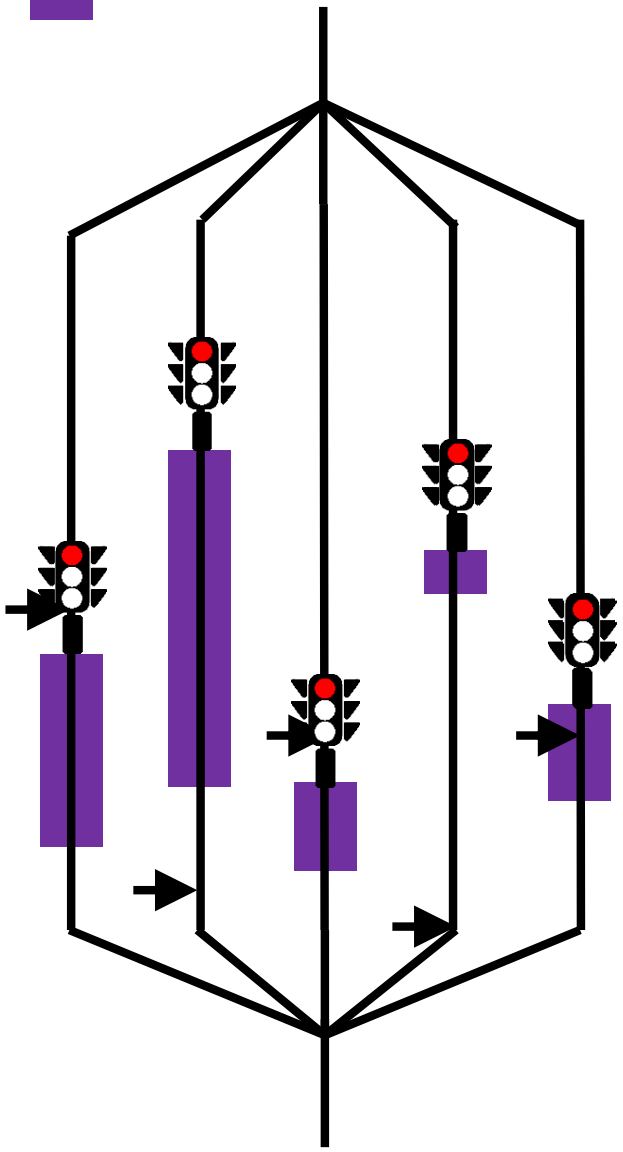
Doar un thread poate fi în zona critică la un moment dat

Zonă Critică

P() sau Proberen
V() sau Verhogen
 - Dijsktra



0



Doar un thread poate fi în zona critică la un moment dat

Producator-consumator

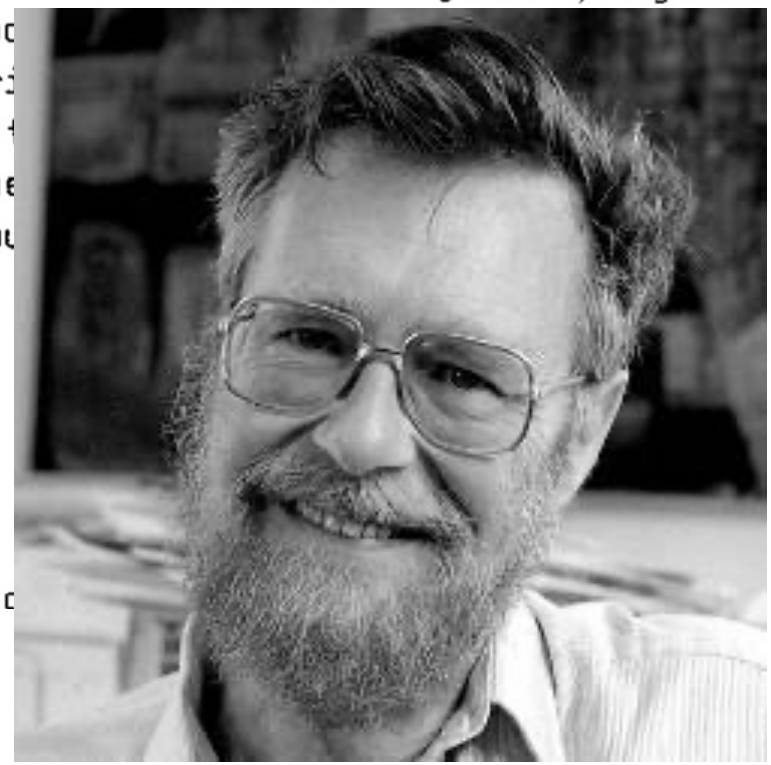
Producer - Consumer

EWD209 - 0

EWD209.html

A Constructive Approach to the Problem of Program Correctness.

Summary. As an alternative to methods by which the correctness of given programs can be established a posteriori, this paper propo
of program generation such as to produce a priori
is treated to show the form that such a control
comes from the field of parallel programming; the
is representative for the way in which a whole mu
actually been constructed.

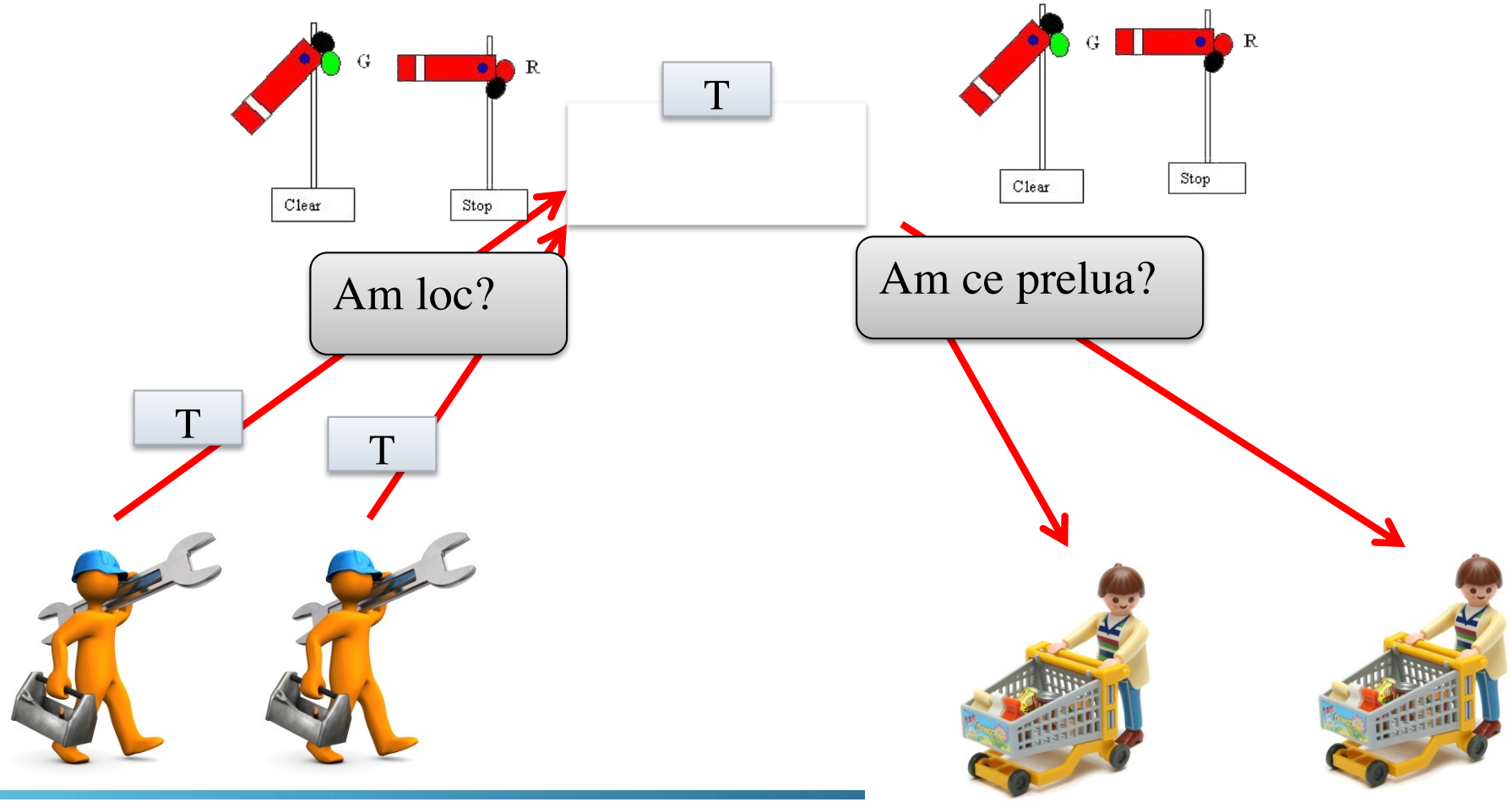


Introduction.

The more ambitious we become in our machine
becomes the problem of program correctness. The

Producători și consumatori

- Problema
  - Se consideră M producători și N consumatori care comunică printr-un singur buffer partajat.



Producători și consumatori

```
typeT buf; /* T - tipul datelor */
sem gol= 1, plin= 0; /* sem binare */
```

```
process Producător[i=1 to M]{
    typeT v;
    while(true) {
      v = produce();
      P(gol);
      buf = v;
      V(plin);
    }
}
```

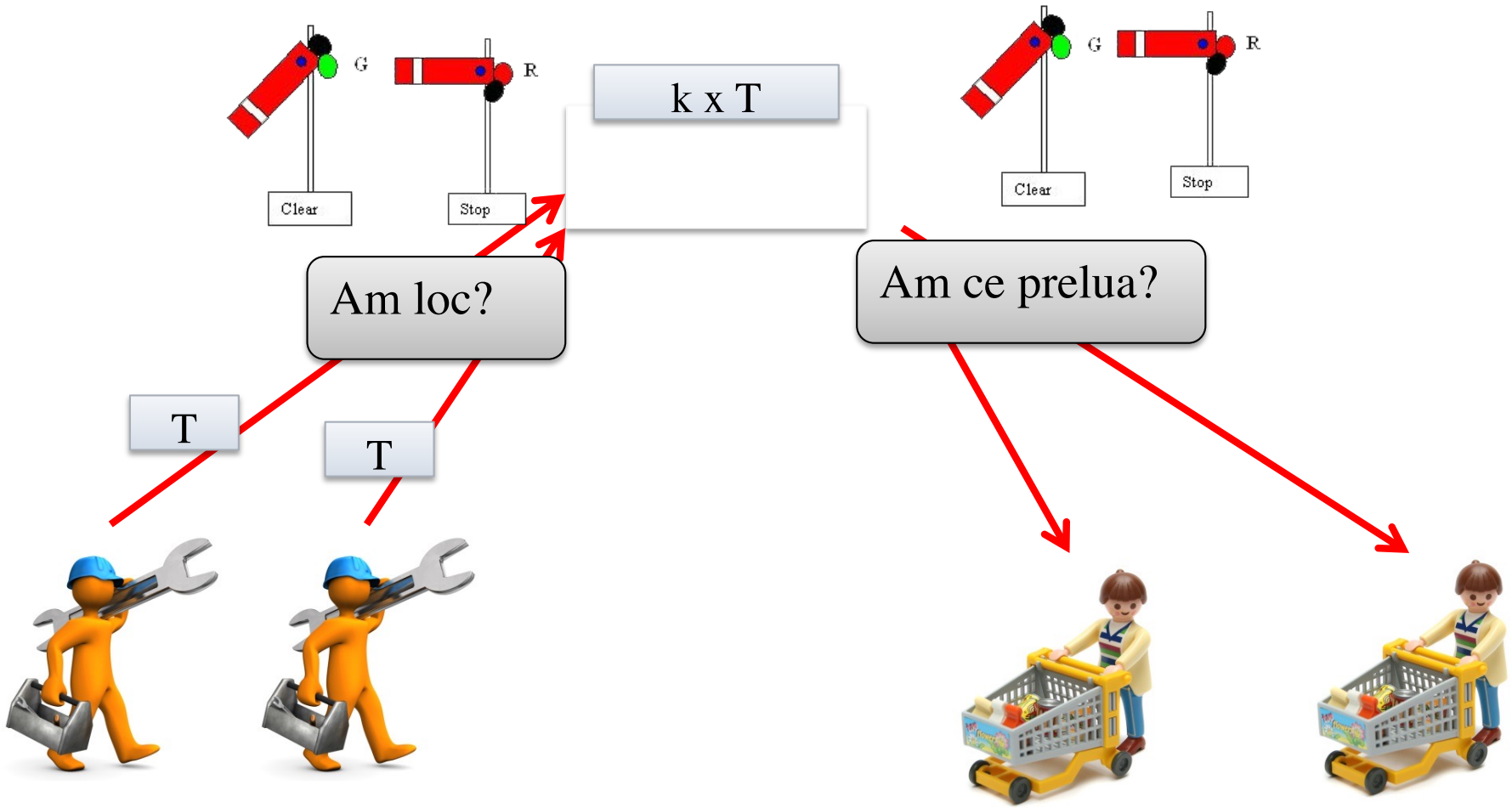
```
process Consumator[i=1 to N]{
    typeT w;
    while(true) {
      P(plin);
      w = buf;
      V(gol);
      consumă(w);
    }
}
```

semnalizare

Comunicarea producător și consumator prin tampon limitat

```
typeT buf[1:k];
sem gol = k, plin = 0;   /* semafoare generale */
process Producător {
      typeT v;
      int ultim= 1;
      while (true) {
        v = produce();
        P(gol);                      /* există locuri goale? */
        buf[ultim] = v;
        ultim = ultim mod k + 1; /* circular */
        V(plin);
      } }
process Consumator {
      typeT w;
      int prim= 1;
      while (true) {
        P(plin);                     /* există valori în buffer? */
        w = buf[prim]; prim = prim mod k + 1;
        V(gol);
        consumă(w);
      }
}
```

Comunicarea producător și consumator prin tampon limitat

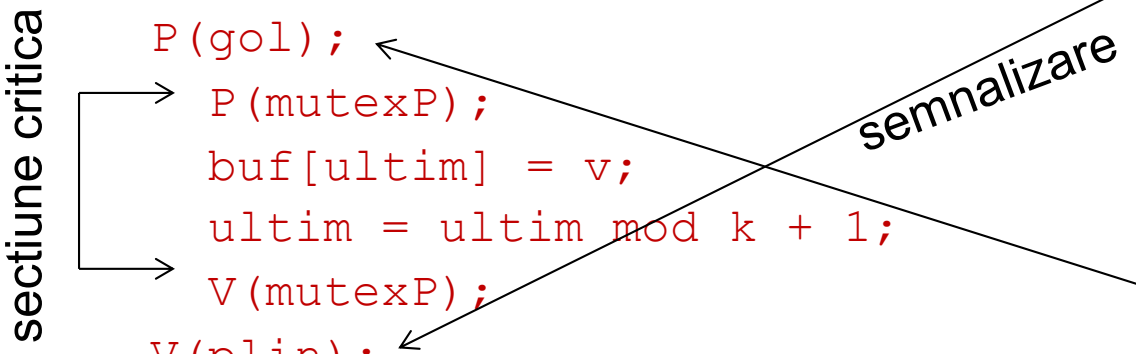


Mai mulți producători și mai mulți consumatori

```
typeT buf[1:k];
int prim = 1, ultim = 1;
sem gol = k, plin = 0;
sem mutexP = 1, mutexC = 1;
```

```
process Producător[i=1 to M]{
    typeT v;
    while (true) {
      v = produce();
      P(gol);
       P(mutexP);
       buf[ultim] = v;
       ultim = ultim mod k + 1;
       V(mutexP);
      V(plin);
    }
}
```

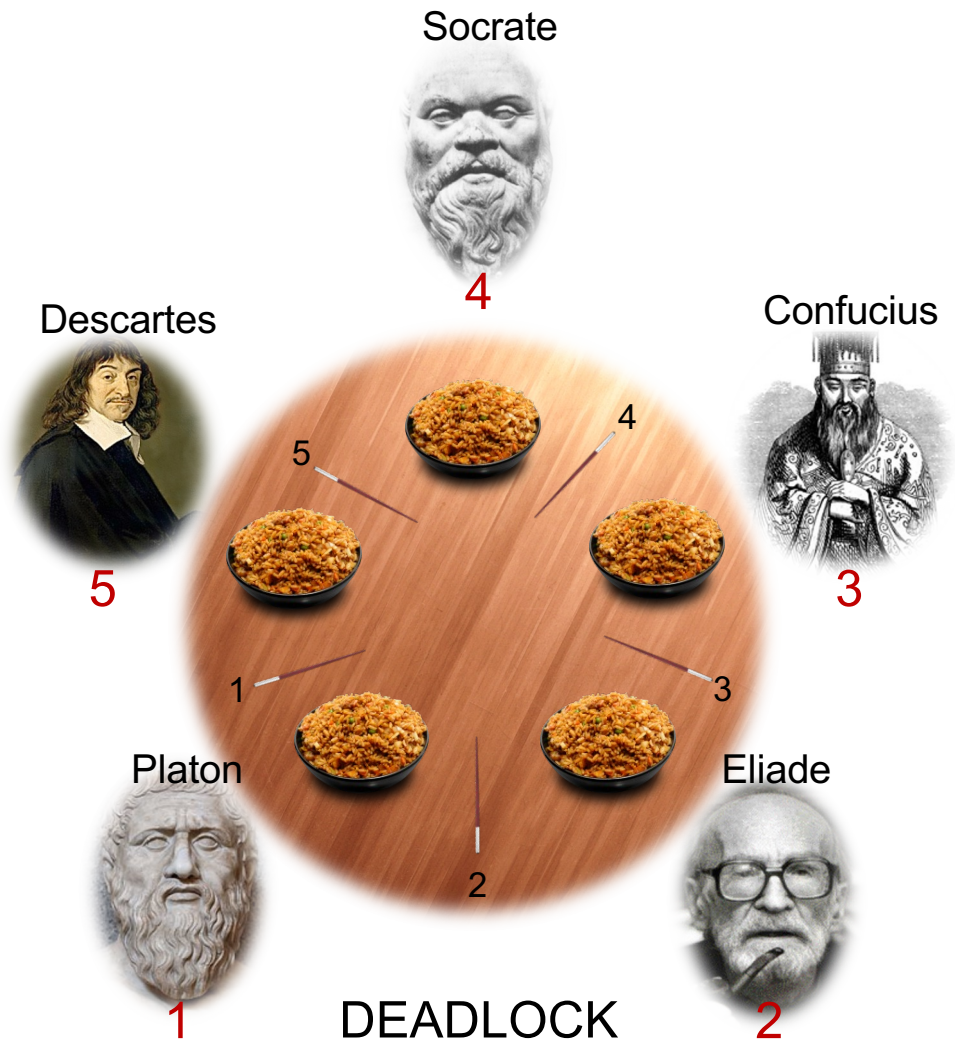
```
process Consumator[i=1 to N]{
      typeT w;
      while (true) {
P(plin);
        P(mutexC);
        w = buf[prim];
        prim = prim mod k + 1;
       V(mutexC);
     V(gol);
     consumă(w);
   }
}
```



Problema filozofilor

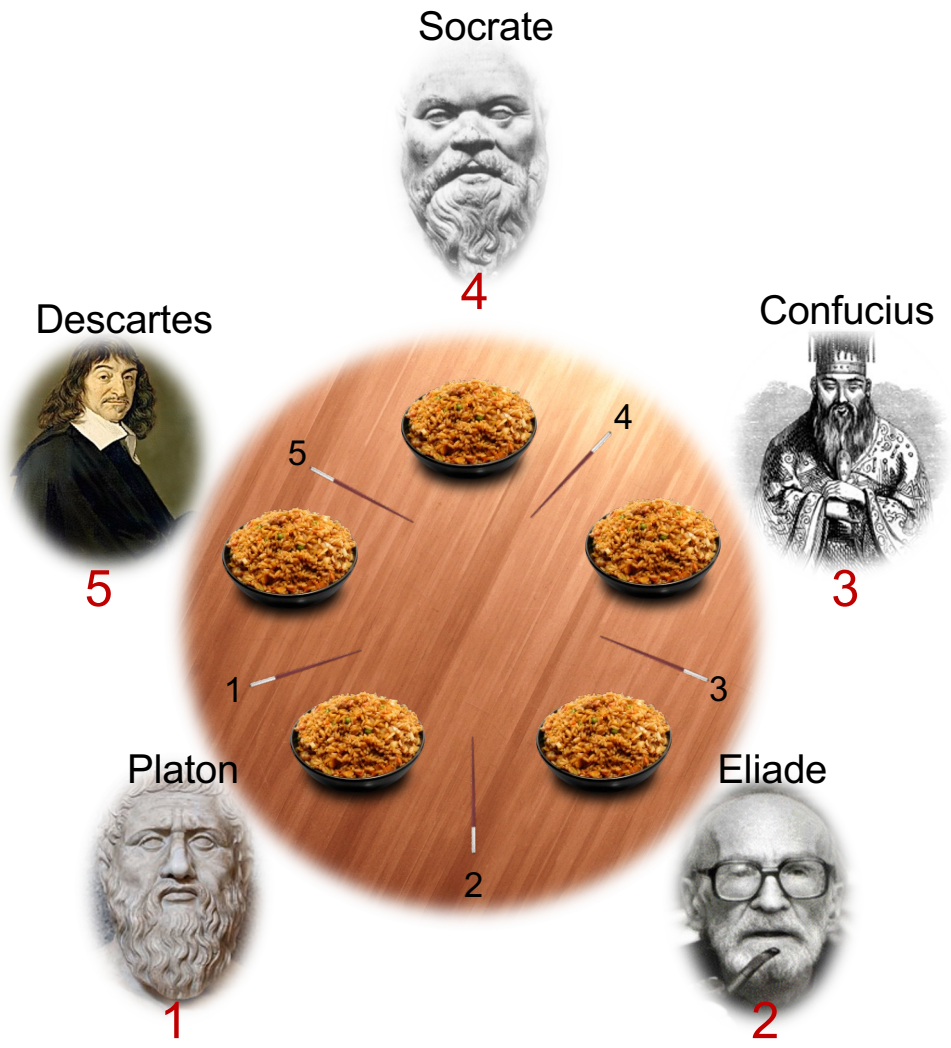
Problema filozofilor

```
process Filozof[i=1 to 5]{
    while (true) {
        ia bețișoare;
        mănâncă;
        eliberează bețișoare;
        gândește;
    }
}
```



Problema filozofilor (2)

```
sem f[1:5] = ([5]1);
process Filozof[i=1 to 4]{
      while (true) {
        P(f[i]); P(f[i+1]);
        mănâncă;
        V(f[i]); V(f[i+1]);
        gândește;
} }
process Filozof[i=5]{
      while (true) {
        P(f[1]); P(f[5]);
        mănâncă;
        V(f[1]); V(f[5]);
        gândește;
} }
```



Problema cititori-scriitori

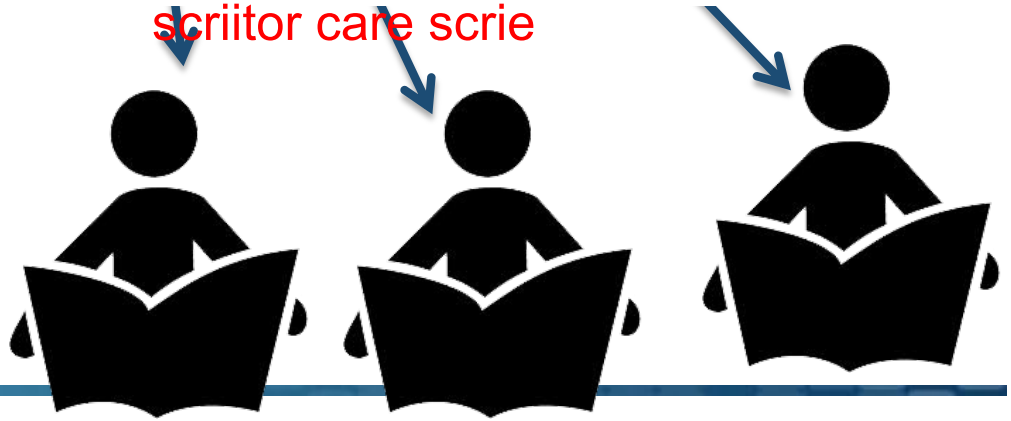
Problema cititorilor și scriitorilor
Excludere mutuală

zona de memorie

(resursa critica)

1) Un singur scriitor are dreptul sa scrie la un moment dat,
2) Doar daca nu exista un cititor curent care citeste

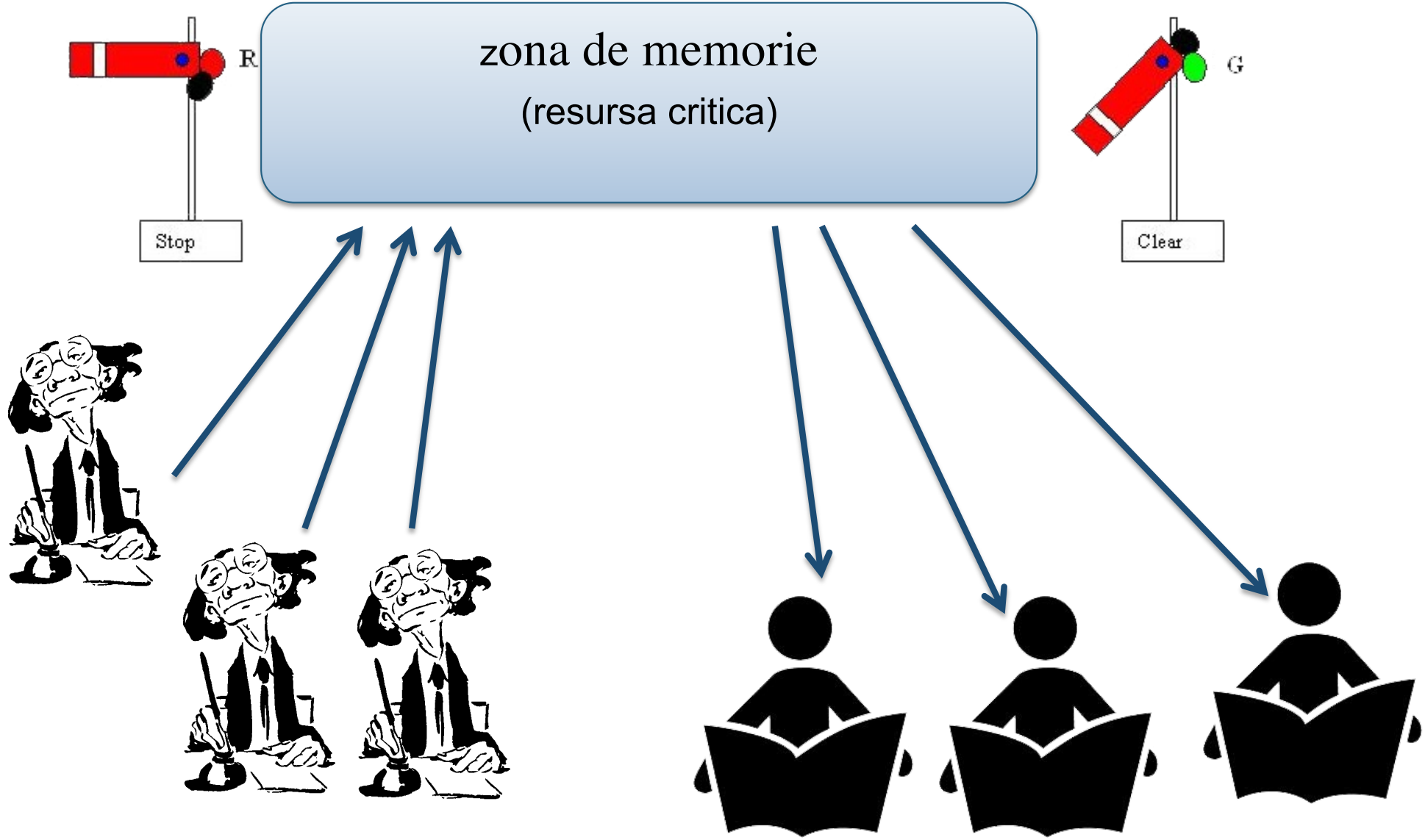
1) Un cititor poate citi din memorie, indiferent daca deja exista alti cititori ce deja citesc
2) Doar daca nu exista un scriitor care scrie



Problema cititorilor și scriitorilor
Excludere mutuală

```
sem rw = 1;
process Cititor[i=1 to m]{
      while (true){
        P(rw);
        citește din resursa comună;
        V(rw);
}   }
process Scriitor[j=1 to n]{
      while (true){
        P(rw);
        scrie în resursa comună;
        V(rw);
}   }
```

Excludere mutuala strica !
Un singur proces are voie sa execute opeartii pe resursa critica…

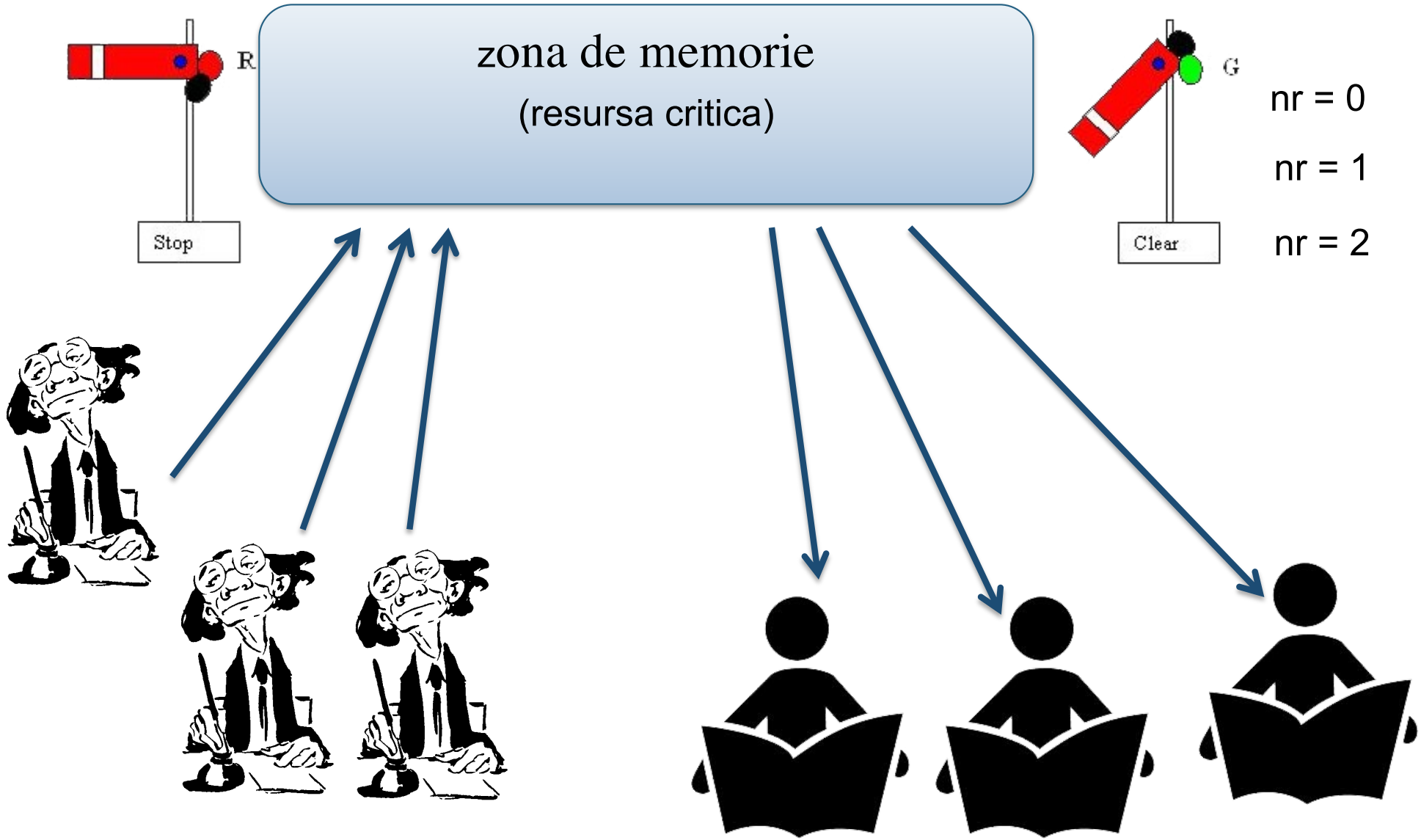

zona de memorie
(resursa critica)
R
Stop
G
Clear

Problema cititorilor si scriitorilor
Excludere mutuală (2)

```
int nr = 0; sem mutexR = 1, rw = 1;
process Cititor[i=1 to m]{
      while (true){
        P(mutexR);
          nr = nr + 1;
          if (nr == 1) P(rw);    /* dacă primul cititor */
        V(mutexR);
        citește din resursa comună;
        P(mutexR);
          nr = nr - 1;
          if (nr == 0) V(rw);    /* dacă ultimul cititor */
        V(mutexR);
}     }
process Scriitor[j=1 to n]{
      while (true){
        P(rw);
        scrie în resursa comună;
        V(rw);
}     }
```

instructiune cu garda

secțiune critică


zona de memorie
(resursa critica)
R
Stop
G
Clear
nr = 0
nr = 1
nr = 2

Problema cititorilor și scriitorilor
Sincronizare condiționată

- Invariant global:

```
RW: ( nr == 0 || nw == 0 ) && nw <= 1
```

- Proces Reader:
  - Incrementarea `nr` e condiționată de `(nw == 0)`
- Proces Writer:
  - Incrementarea `nw` e condiționată de `(nr == 0 && nw == 0)`
- Decrementarea nu trebuie condiționată

Problema cititorilor și scriitorilor
Sincronizare condiționată (2)

```
int nr = 0, nw = 0;
process Cititor[i=1 to m]{
      while (true){
        <await (nw == 0) nr = nr + 1>
        citește din resursa comună;
        <nr = nr - 1>
}     }
process Scriitor[j=1 to n]{
      while (true){
        <await (nr == 0 && nw == 0) nw = nw + 1>
        scrie în resursa comună;
        <nw = nw - 1>
}     }
```

Problema cititorilor și scriitorilor
Sincronizare condiționată (3)

Politici:

- noile cereri de la cititori sunt întârziate dacă un scriitor așteaptă
- un cititor întârziat este trezit doar dacă nu există un scriitor în așteptare

```
int nr = 0;   /* nr. cititori care folosesc resursa */
int nw = 0;   /* nr. scriitori care folosesc resursa */
sem e = 1;    /* intrare secțiune atomică */
sem r = 0;    /* intarzie cititori daca nw>0 sau dw>0 */
sem w = 0;    /* intarzie scriitori daca nw>0 sau nr>0 */
int dr = 0;   /* nr. cititori întârziați */
int dw = 0;   /* nr. scriitori întârziați */
```

Problema cititorilor și scriitorilor
Sincronizare condiționată (4)

Split binary semaphore

- Folosit pentru a implementa atât excluderea mutuală cât și sincronizarea condiționată.
- Semafoarele e, r și w formează împreună un semafor ***splitat*** *(split binary semaphore)*:
  - cel mult un semafor este 1 la un moment dat $0 \leq e + r + w \leq 1$
  - fiecare cale de execuție începe cu un P și se termină cu un singur V
  - instrucțiunile între P și V se execută în excludere mutuală.
- Tehnica se numește ***pasarea ștafetei***:
  - Inițial un semafor este 1 și un proces poate prelua ștafeta printr-o operație P asupra semaforului
  - când un proces deține ștafeta (se execută într-o secțiune critică și toate semafoarele sunt 0), el poate pasa ștafeta altui proces printr-o operație V asupra unuia din cele trei semafoare.

Problema cititorilor și scriitorilor
Sincronizare condiționată (5)

```
process Cititor[i=1 to m]{
    while (true){
      P(e);
      if (nw > 0 or dw > 0){
        dr = dr + 1; V(e); P(r);
      }
      nr = nr + 1;
      if (dr > 0) { dr = dr - 1; V(r); }
      else if (dr == 0) V(e);
      citește din resursa comună;
      P(e);
      nr = nr - 1;
      if (nr == 0 and dw > 0) { dw = dw - 1; V(w); }
      else if (nr > 0 or dw == 0) V(e);
    }
}
```

Problema cititorilor și scriitorilor
Sincronizare condiționată (6)

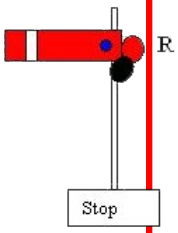
```
process Scriitor[j=1 to n]{
  while (true){
    P(e);
    if (nr > 0 or nw > 0) {
      dw = dw + 1; V(e); P(w)
    }
    nw = nw + 1;
    V(e);
    scrie în resursa comună;
    P(e);
    nw = nw - 1;
    if (dr > 0 and dw == 0) { dr = dr - 1; V(r); }
    else if (dw > 0) { dw = dw-1; V(w); }
    else if (dr == 0 and dw == 0) V(e);
  }
}
```

Problema barbierului

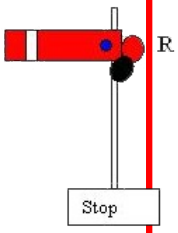
Problema bărbierului

- Problema:
  - O frizerie cu un bărbier, un scaun de bărbier, n scaune de așteptare.
  - Când nu sunt clienți, bărbierul doarme.
  - Când sosește un client fie trezește bărbierul, fie așteaptă dacă acesta e ocupat.
  - Dacă toate scaunele sunt ocupate, clientul pleacă.

Pas 1

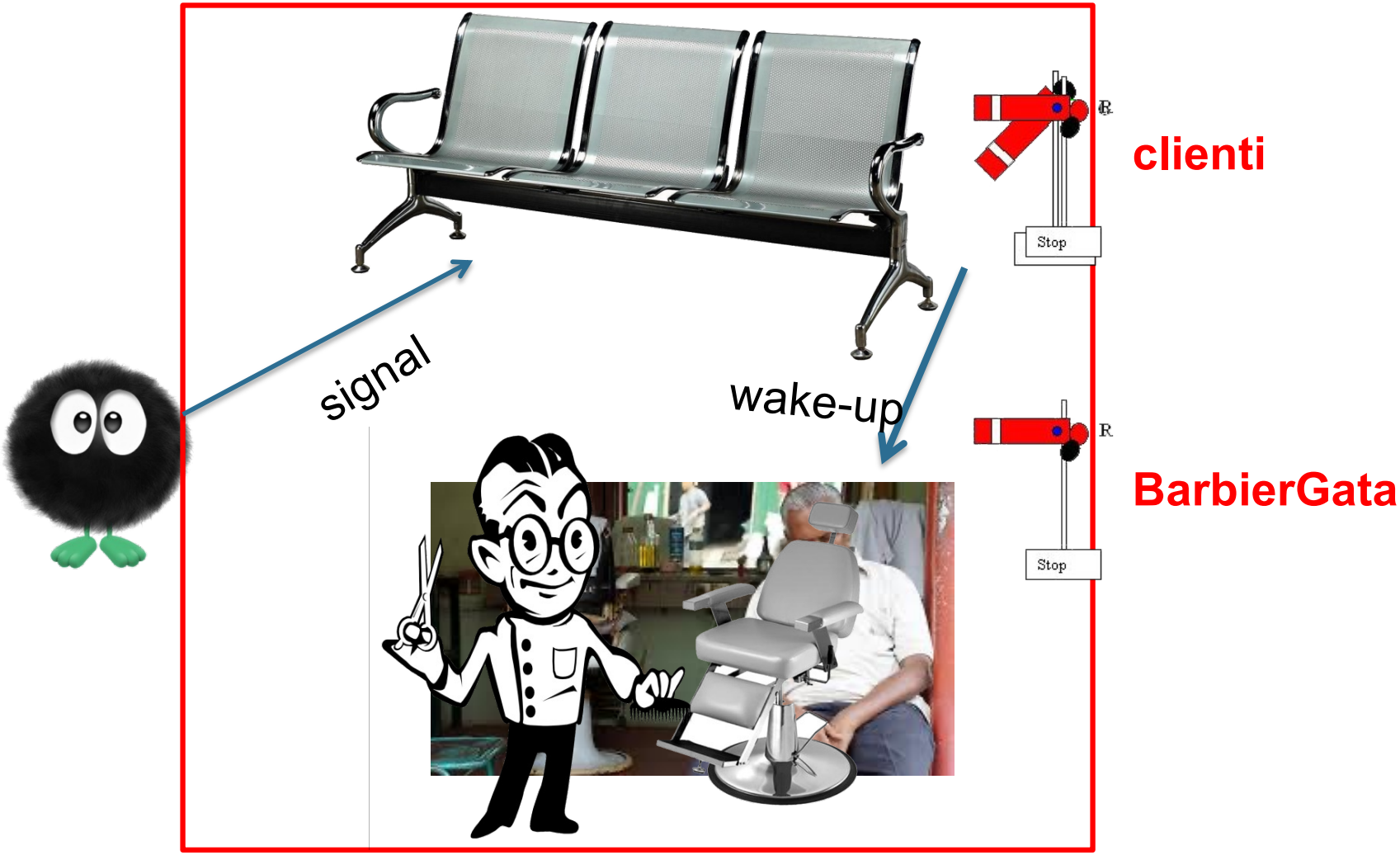


clienti

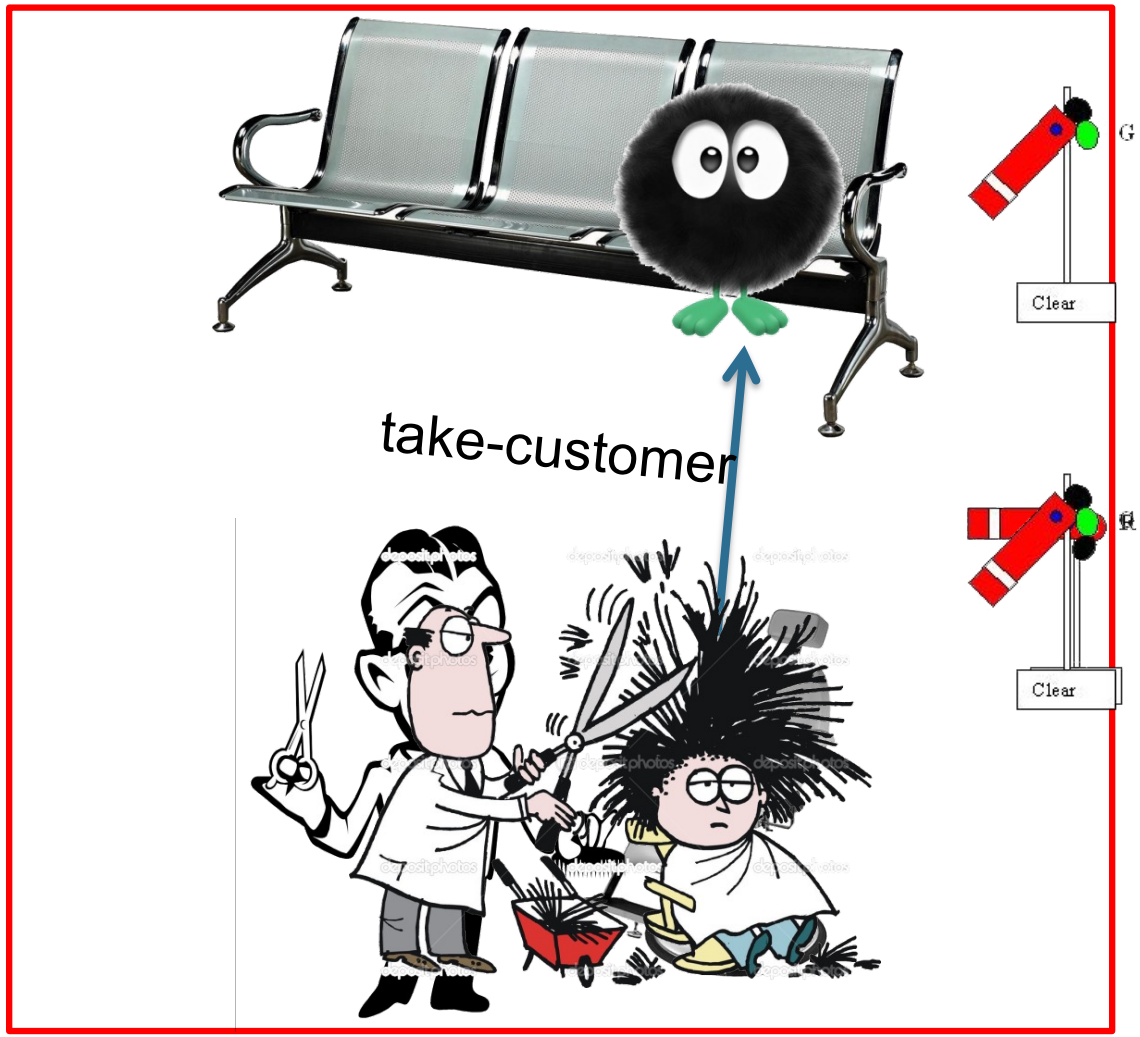


BarbierGata

Pas 2



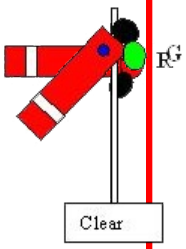
Pas 3



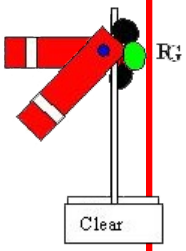
clienti

BarbierGata

Pas 4



clienti



BarbierGata

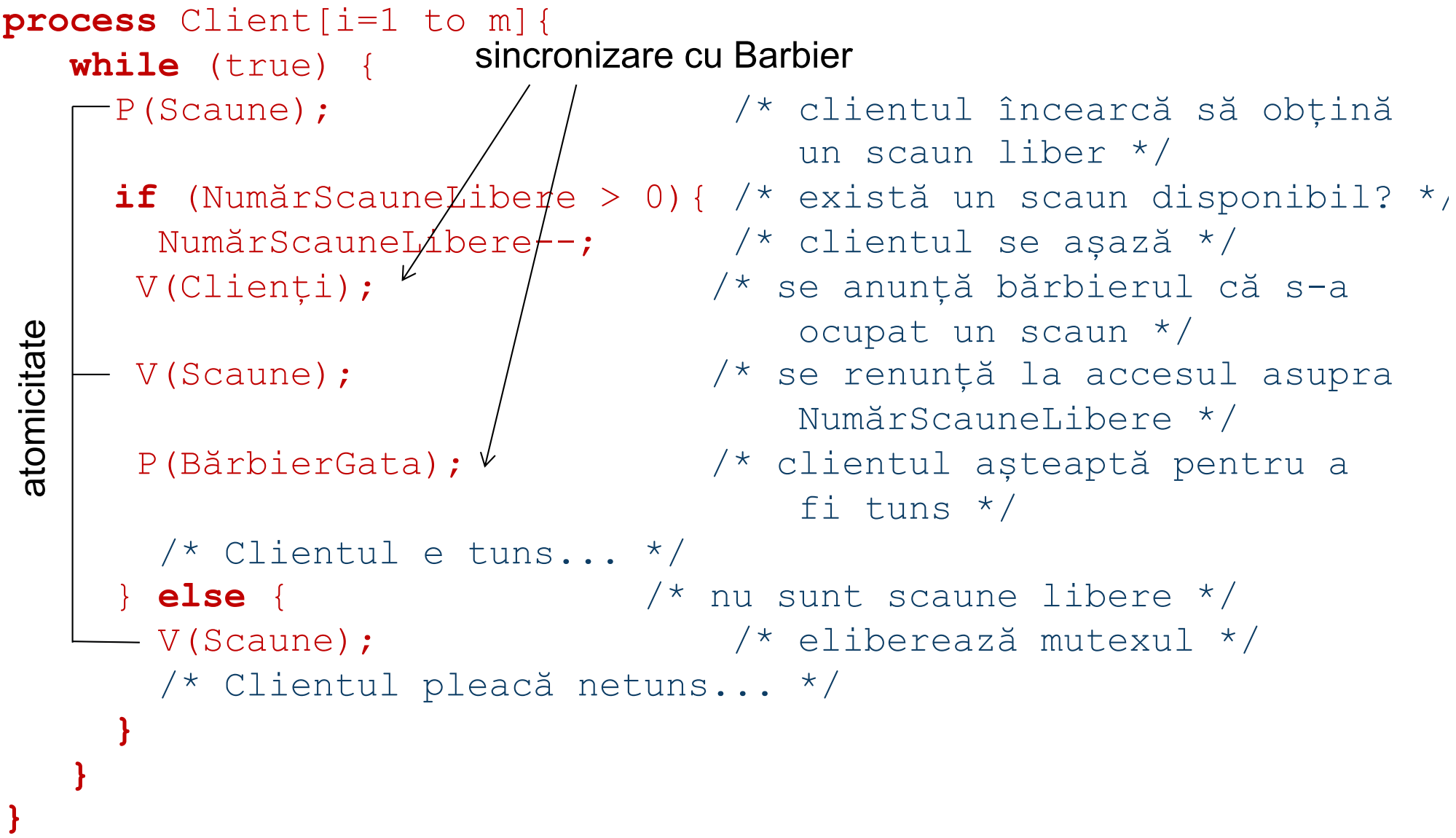
Problema bărbierului (1)

```
int NumărScauneLibere = n;
sem Clienți = 0;
sem BărbierGata = 0;
sem Scaune = 1;

process Bărbier{
   while (true){
      P(Clienți);                  /* se caută un client;
                                      dacă există, este chemat */
      P(Scaune);                   /* are client, va modifica
                                      NumărScauneLibere */
      NumărScauneLibere++;         /* se eliberează un scaun */
      V(BărbierGata);              /* bărbierul e gata să tundă */
      V(Scaune);                   /* a terminat de modificat
                                      NumărScauneLibere */
      /* Bărbierul tunde... */
   }
}
```

sincronizare cu Client

atomicitate

Problema bărbierului (2)



Problema bărbierului (var. mutex)

Bărbier

emptyChairs = N
Clients = 0
BarberReady = 0
Chairs = 1

Client

```
while(true) {
    Clients.lock();
    Chairs.lock();
    emptyChairs++;
    BarberReady.unlock();
    Chairs.unlock();
    cutHair();
}
```

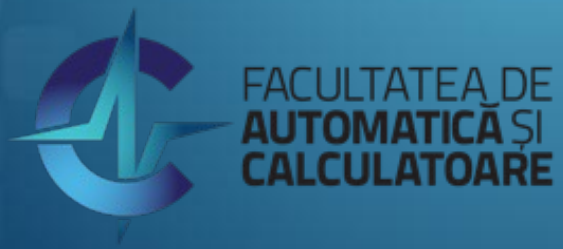
```
Chairs.lock();
if(emptyChairs>0) {
    emptyChairs--;
    Clients.unlock();
    Chairs.unlock();
    BarberReady.lock();
    getHairCut();
} else {
    Chairs.unlock();
}
```

Sumar

- Dezvoltarea algoritmilor folosind variabile partajate (MIMD)
- Semafoare
- Secțiuni critice
- Probleme:
  - Producători și consumatori
  - Problema filozofilor
  - Problema cititorilor și scriitorilor
  - Problema bărbierului

Arhitecturi Paralele
Abordări probleme paralele

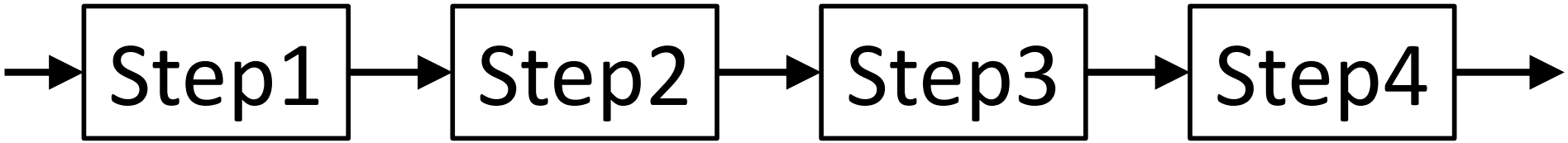
Prof. Ciprian Dobre
ciprian.dobre@upb.ro



Paralelism: modele Pipeline

Pipeline

- CPU Instruction pipeline
- Graphics pipeline
- Various algorithms



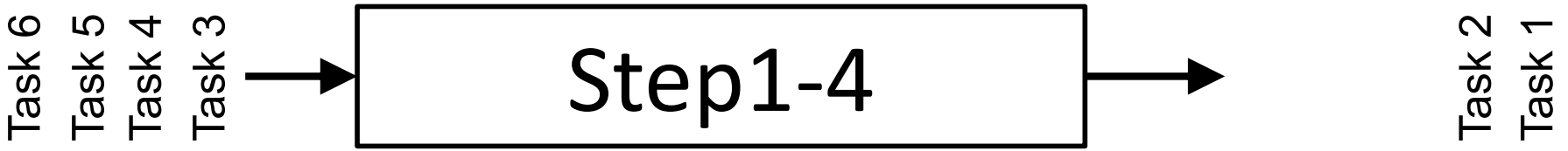
Un pas poate fi executat de:

- thread
- process
- element hardware

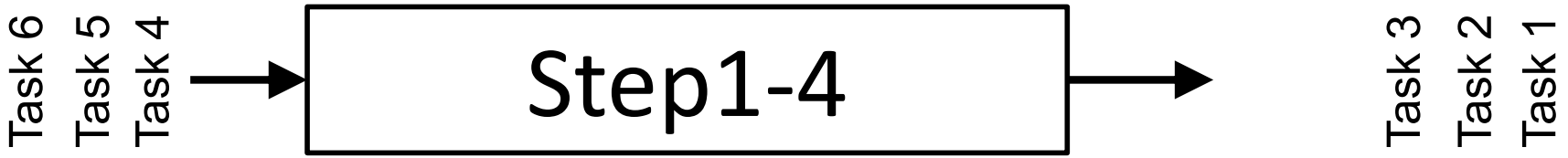
Without Pipeline

Without Pipeline

Without Pipeline



Without Pipeline



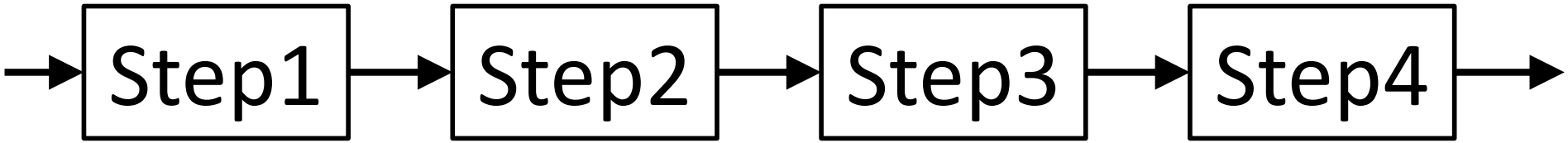
Without Pipeline

Without Pipeline

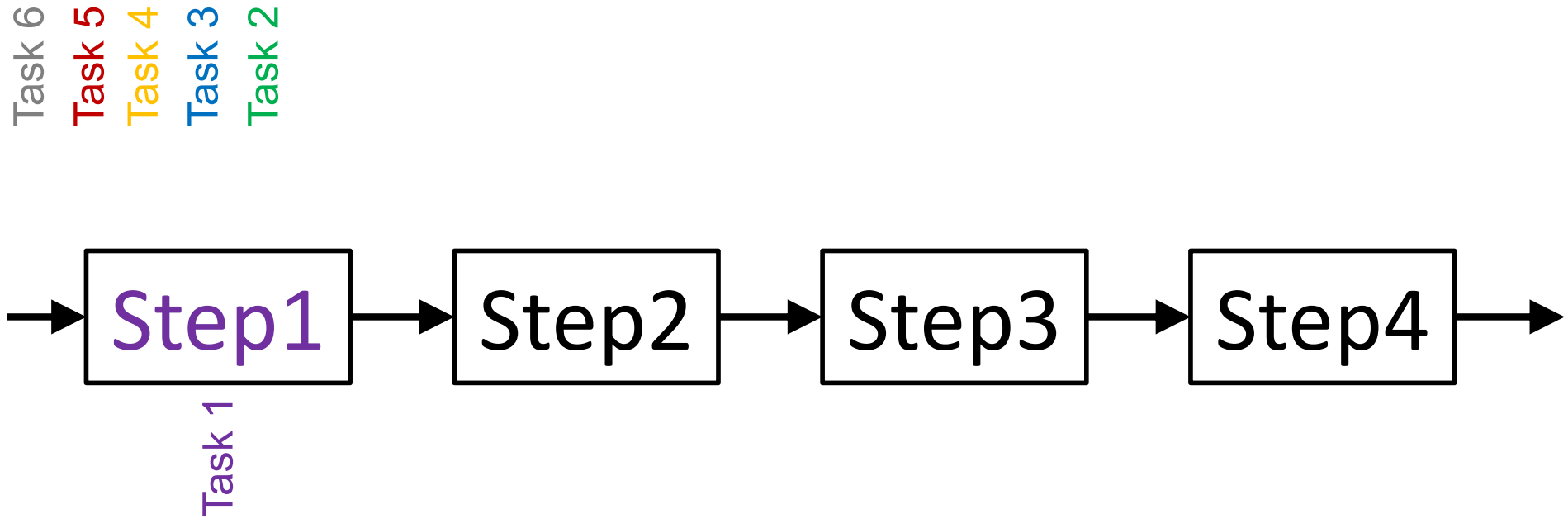
$$total_execution_time = task_execution_time * number_of_tasks$$

Pipeline

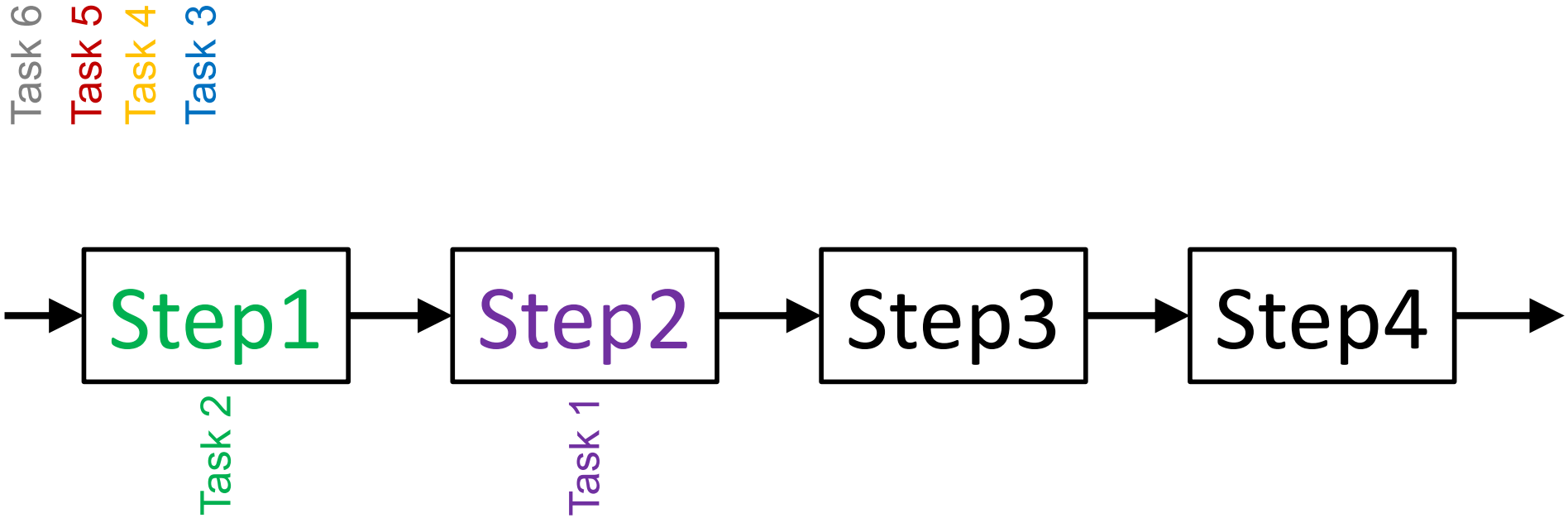
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1



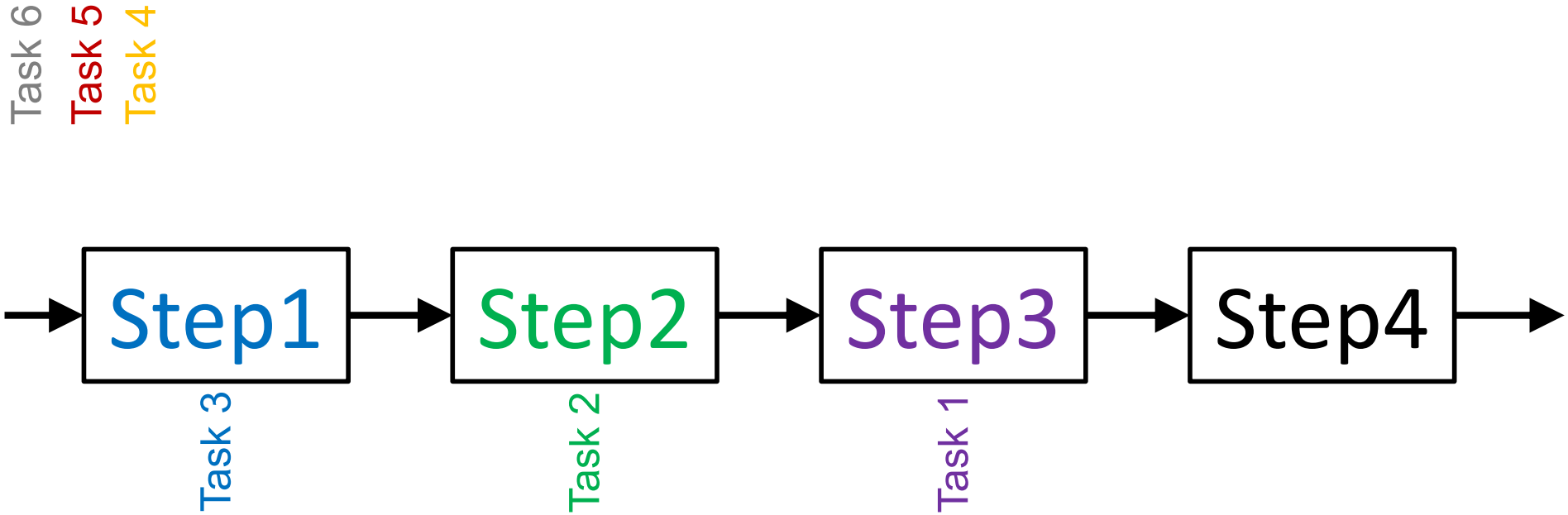
Pipeline


Task 6
Task 5
Task 4
Task 3
Task 2
Step1
Step2
Step3
Step4
Task 1

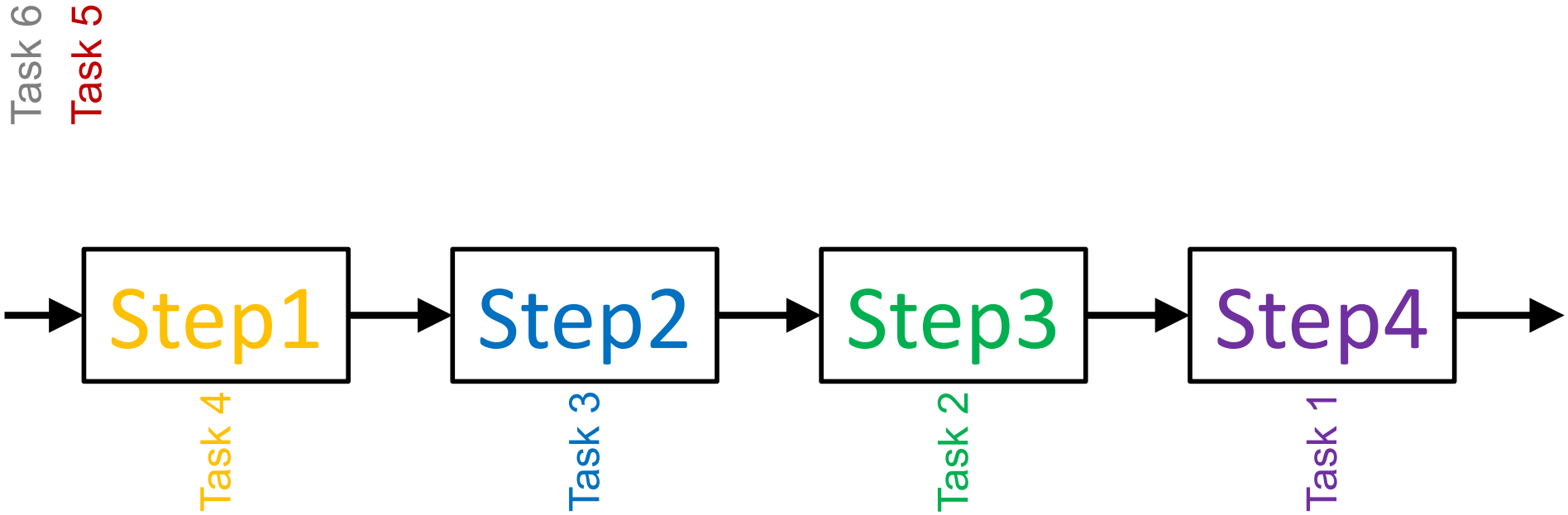
Pipeline


Task 6
Task 5
Task 4
Task 3
Step1
Step2
Step3
Step4
Task 2
Task 1

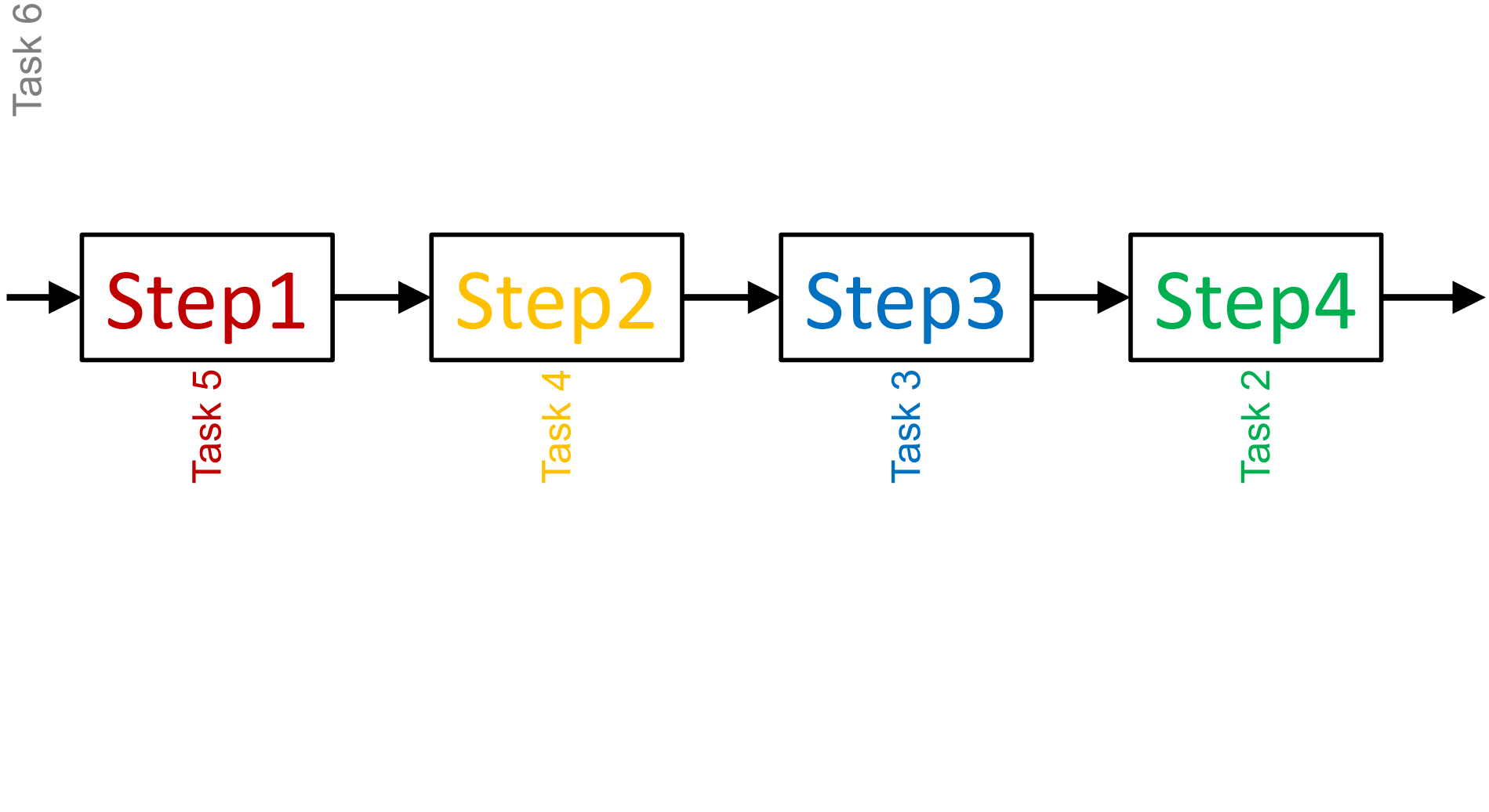
Pipeline


Task 6
Task 5
Task 4
Step1
Step2
Step3
Step4
Task 3
Task 2
Task 1

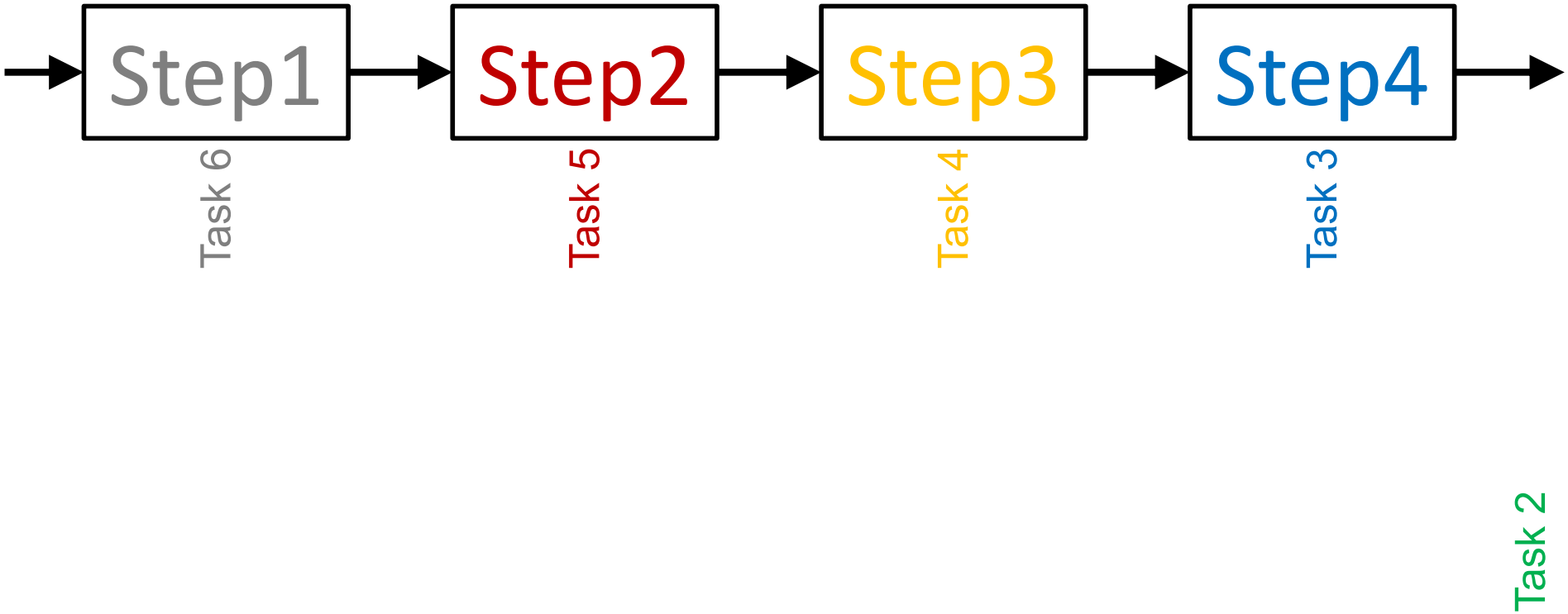
Pipeline


Task 6
Task 5
Step1
Step2
Step3
Step4
Task 4
Task 3
Task 2
Task 1

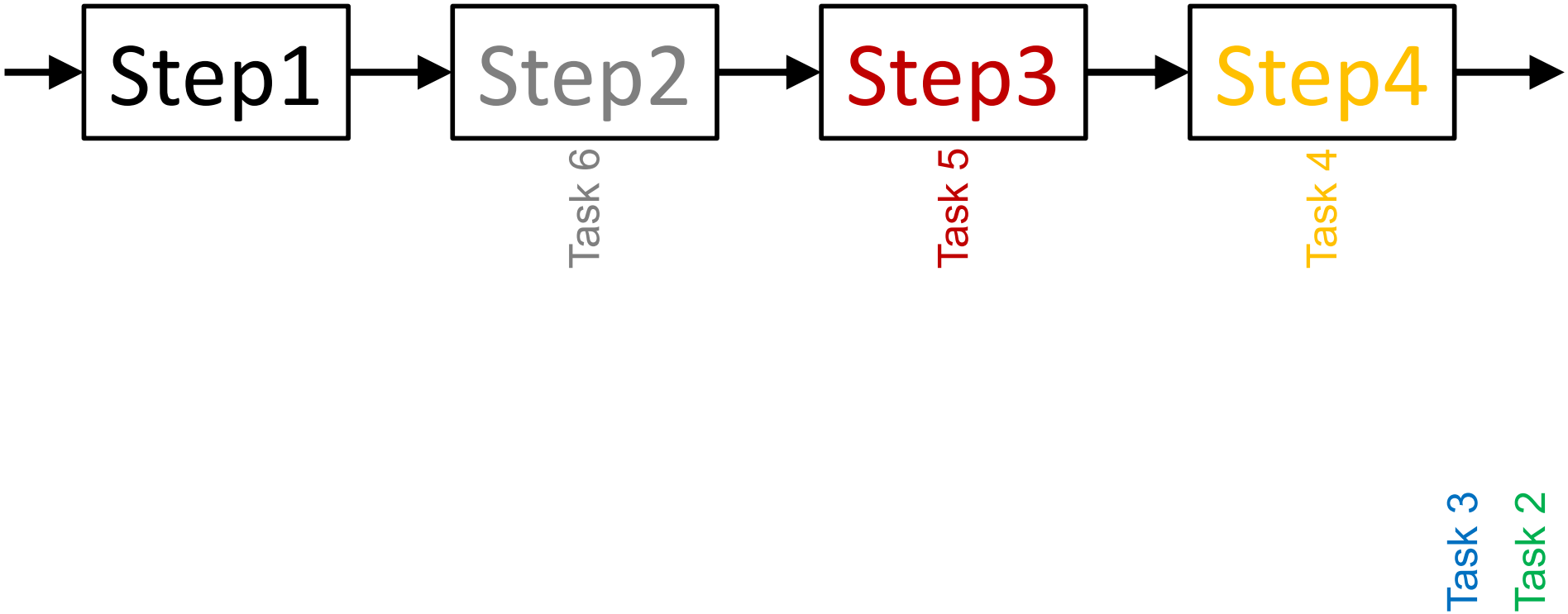
Pipeline


Task 6
Step1
Step2
Step3
Step4
Task 5
Task 4
Task 3
Task 2
Task 1

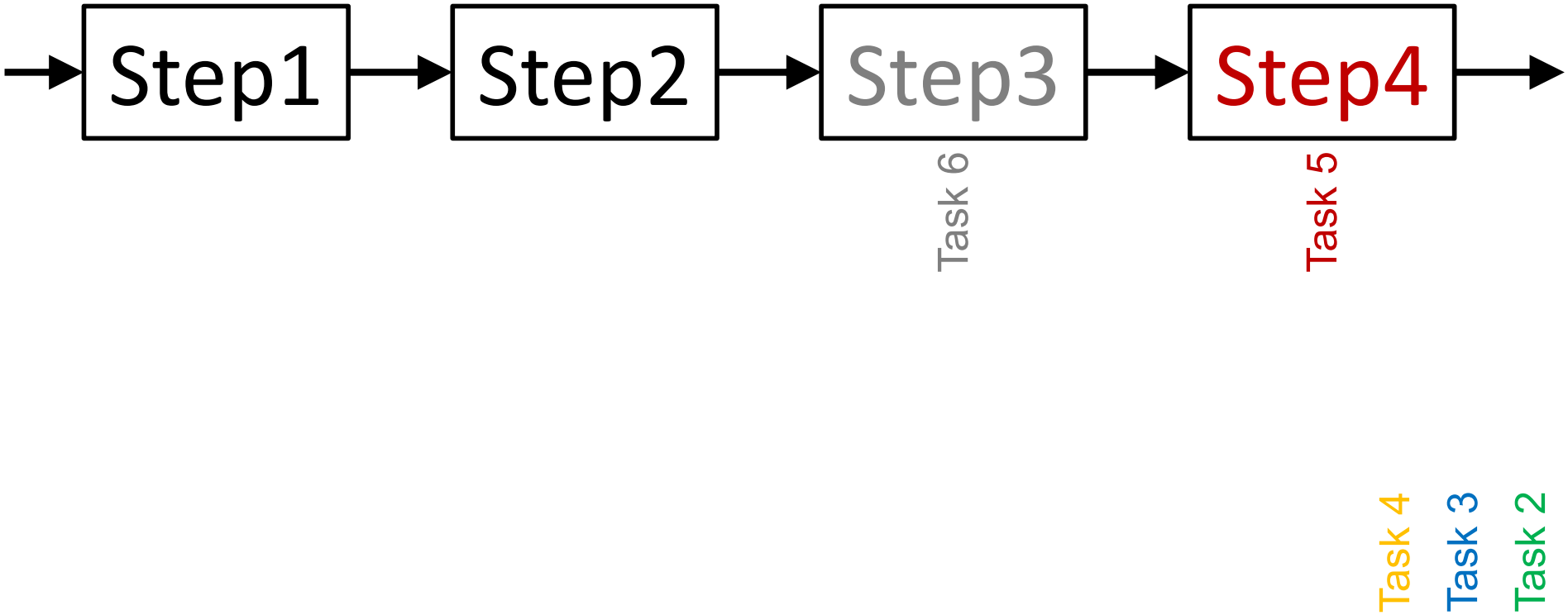
Pipeline


Step1
Step2
Step3
Step4
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1

Pipeline

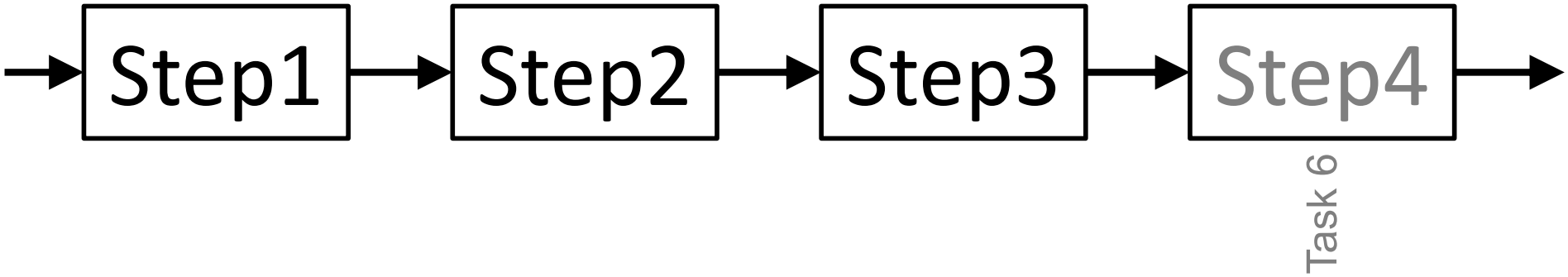

Step1
Step2
Step3
Step4
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1

Pipeline


Step1
Step2
Step3
Step4
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1

Pipeline

Ideal: $step_execution_time = \frac{task_execution_time}{number_of_steps}$



După $number_of_steps$ tascuri:
$total_execution_time = number_of_tasks * step_execution_time$

Un task se termină la fiecare “step tick”

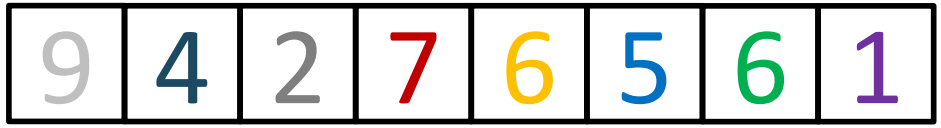
Sortare...

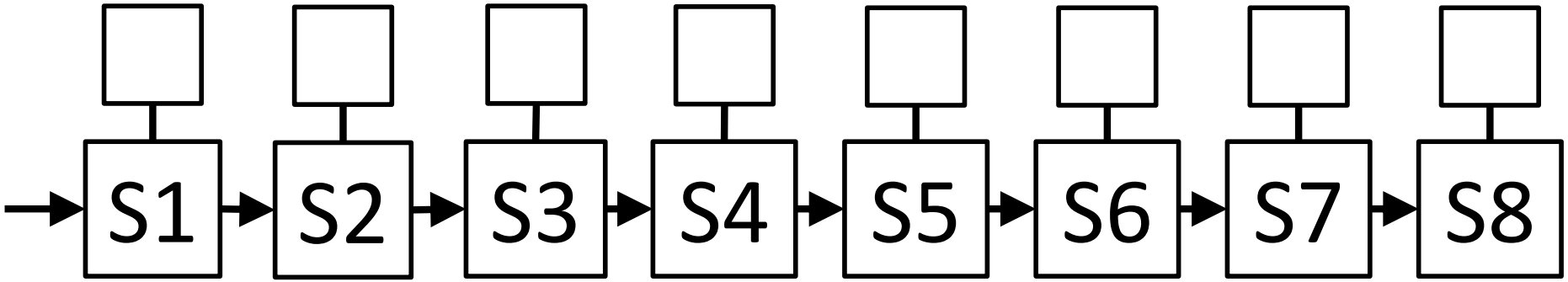
Sorting

9	4	2	7	6	5	6	1

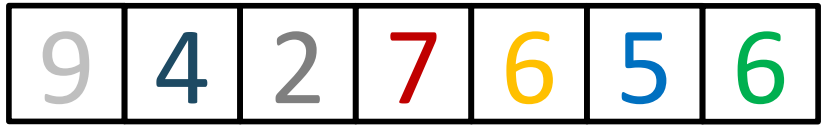
1	2	4	5	6	6	7	9

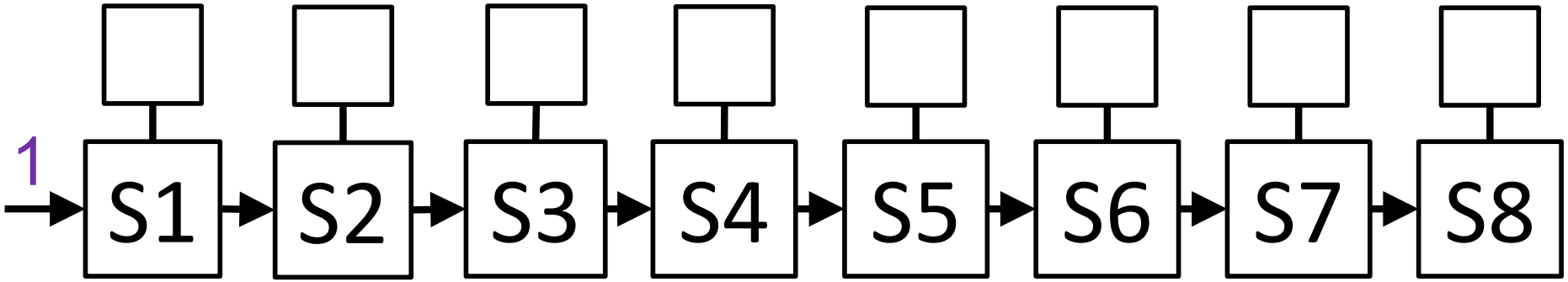
Sorting with pipeline



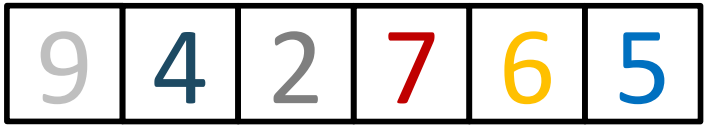


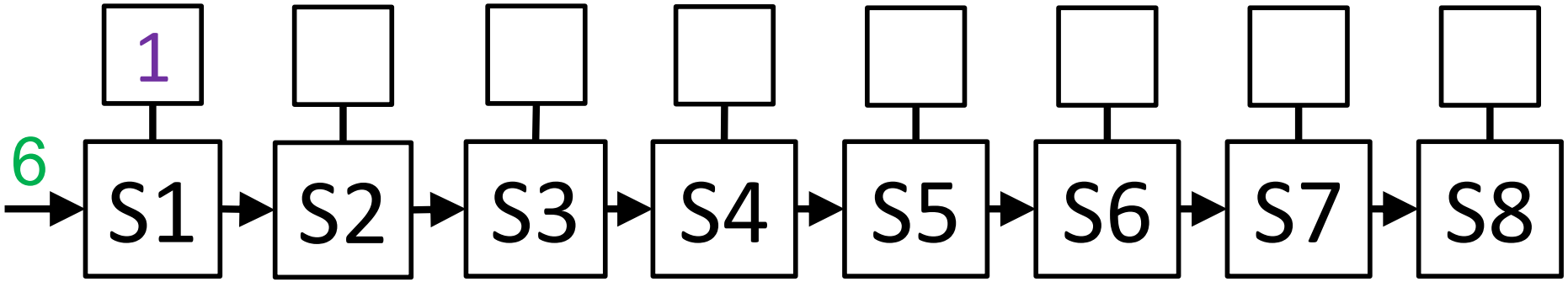
Sorting with pipeline



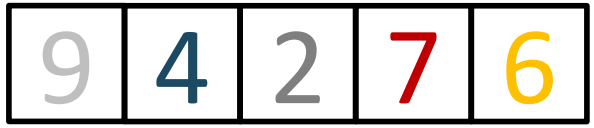


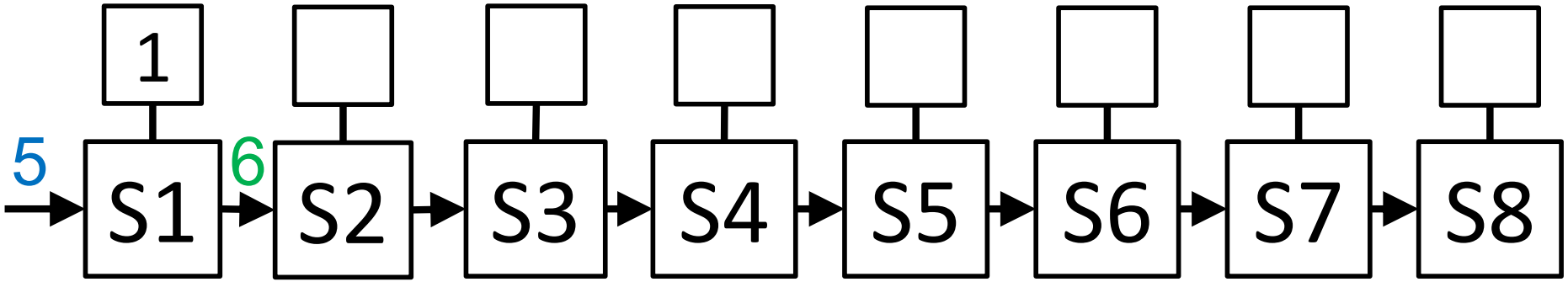
Sorting with pipeline



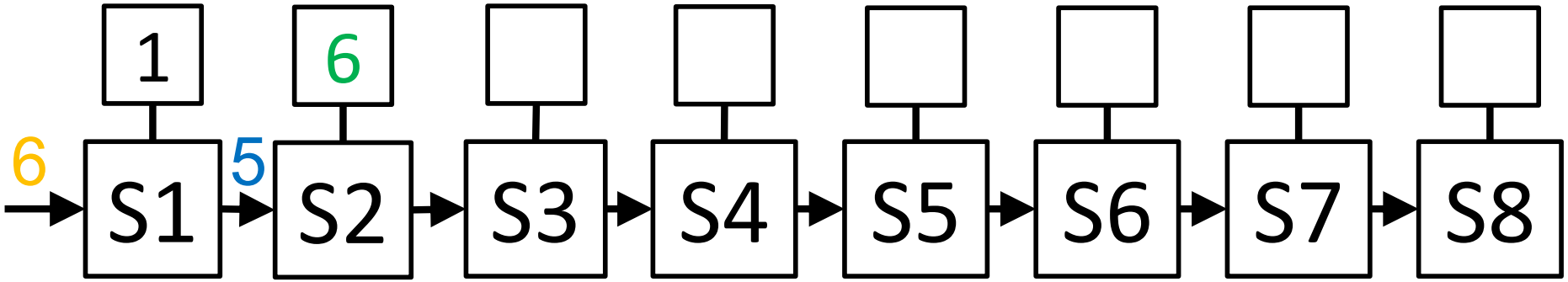


Sorting with pipeline

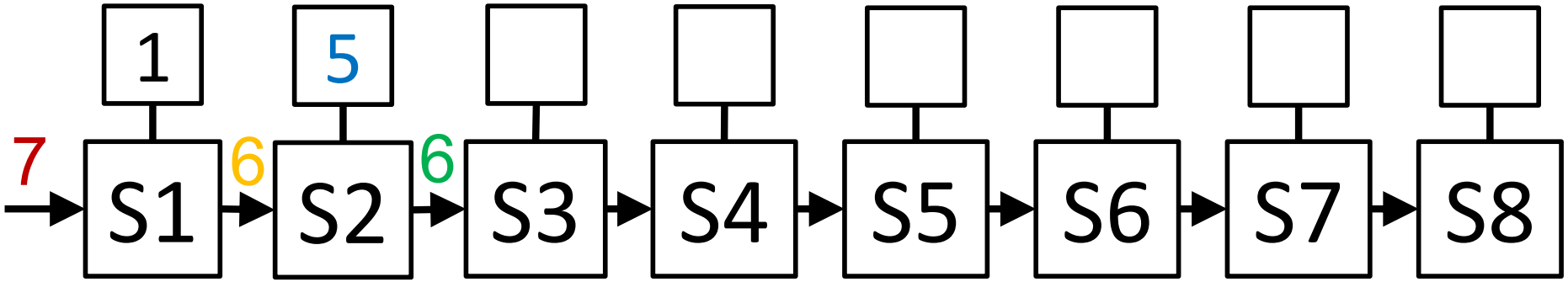




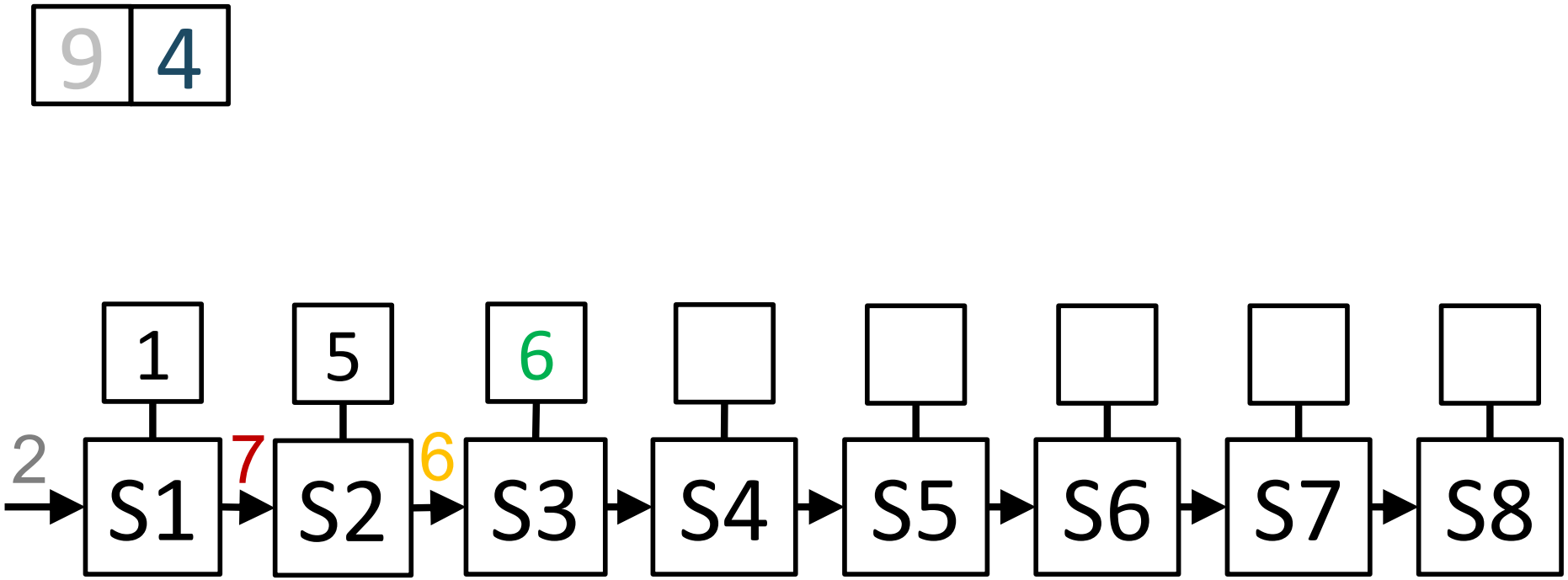
Sorting with pipeline



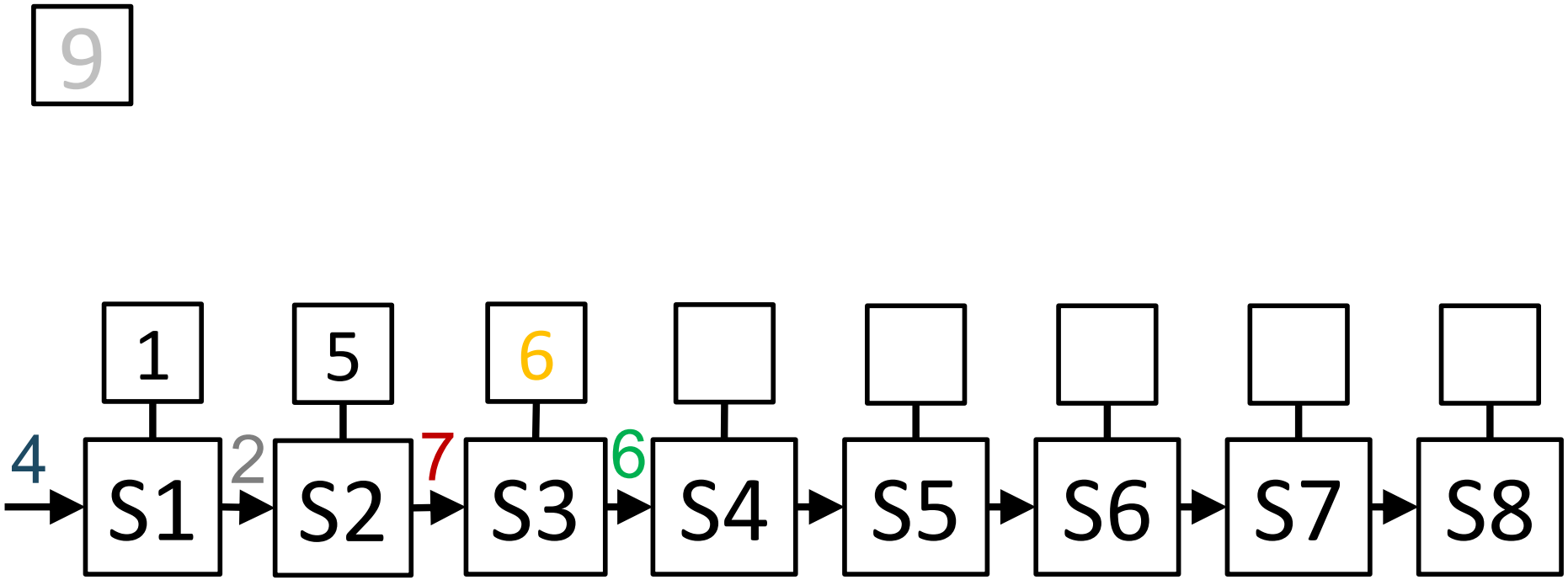
Sorting with pipeline



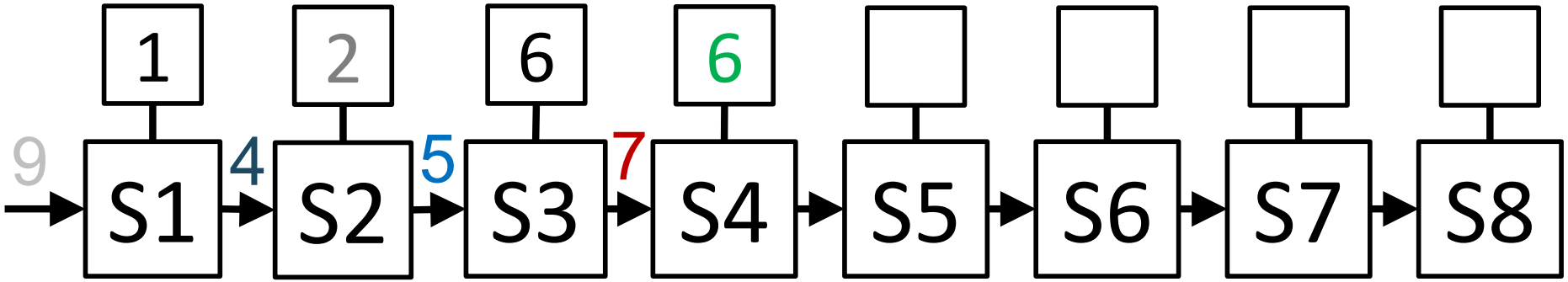
Sorting with pipeline



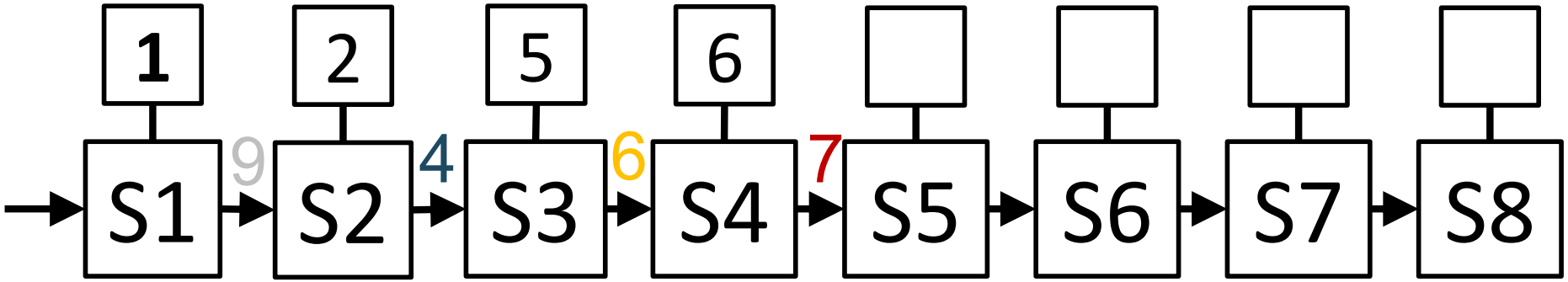
Sorting with pipeline



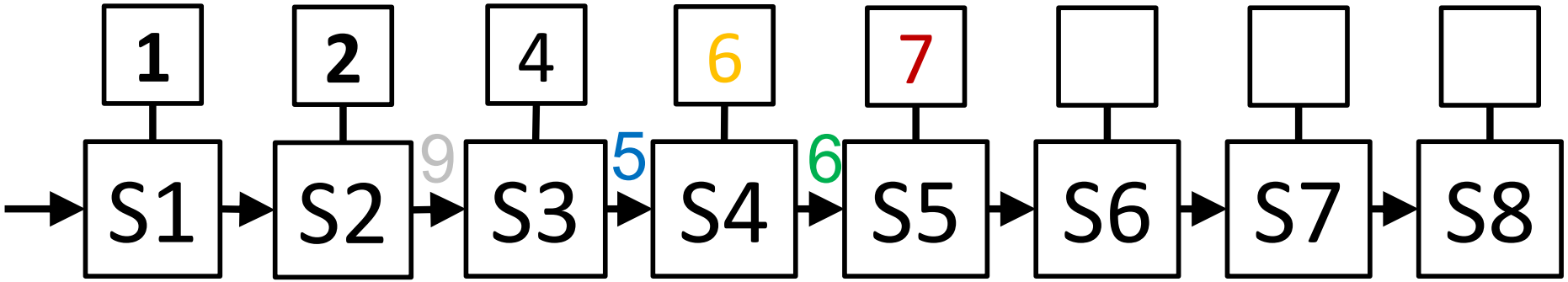
Sorting with pipeline



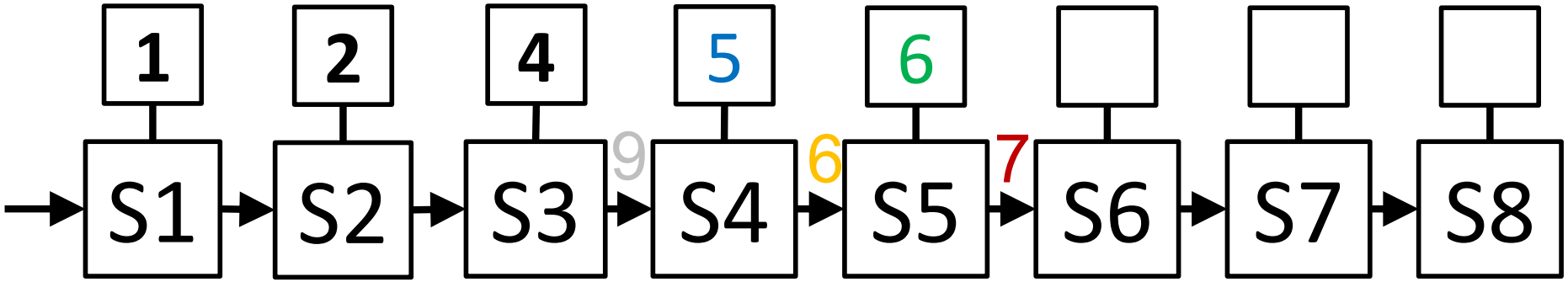
Sorting with pipeline



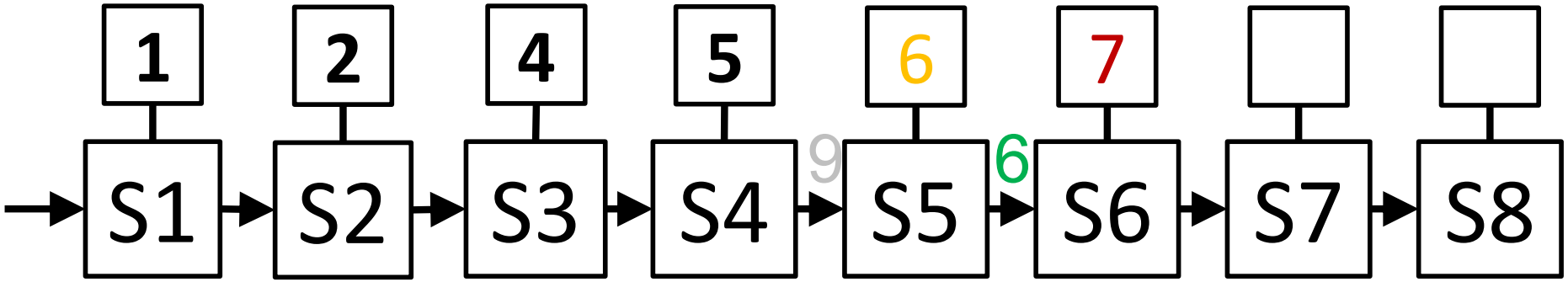
Sorting with pipeline



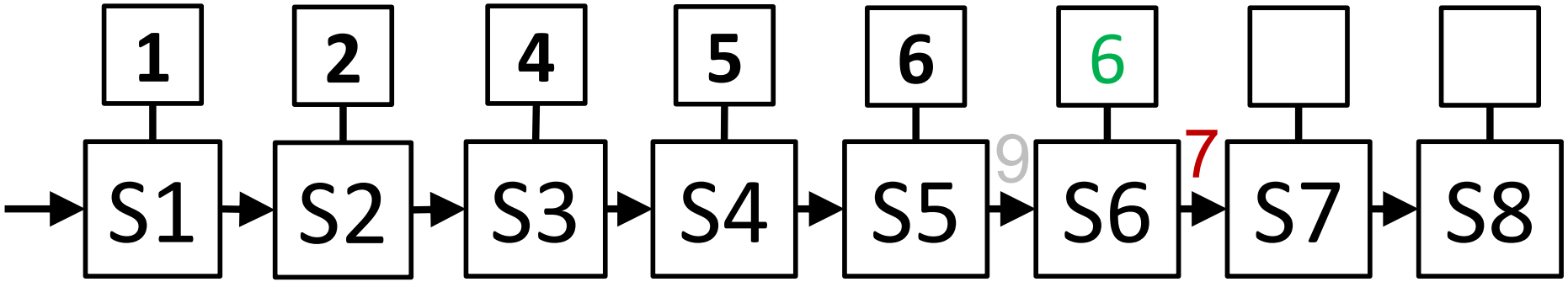
Sorting with pipeline



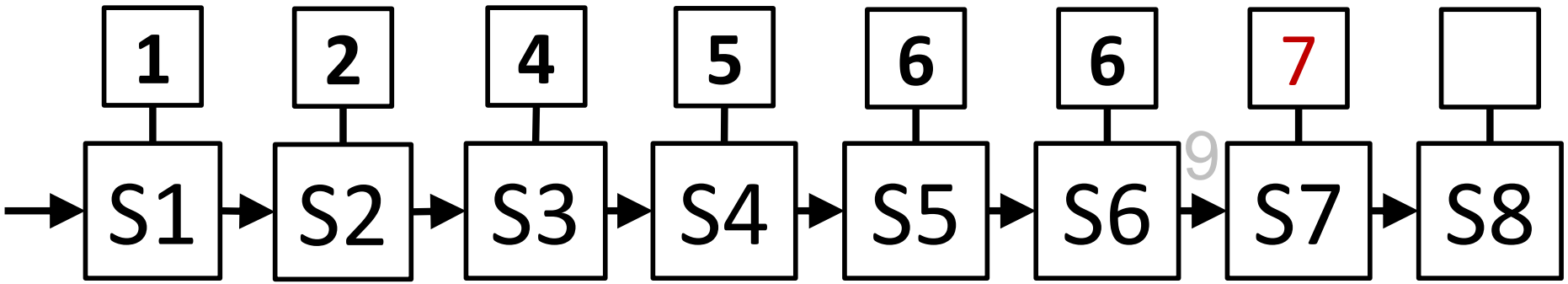
Sorting with pipeline



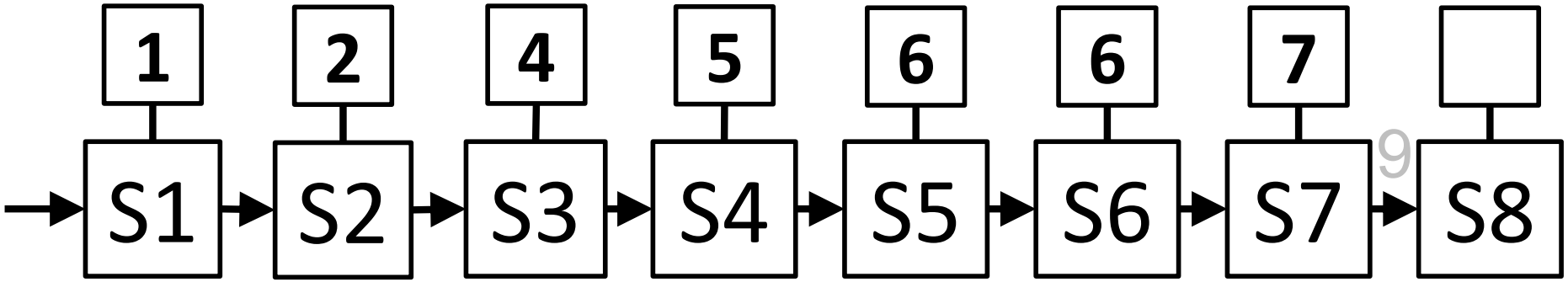
Sorting with pipeline



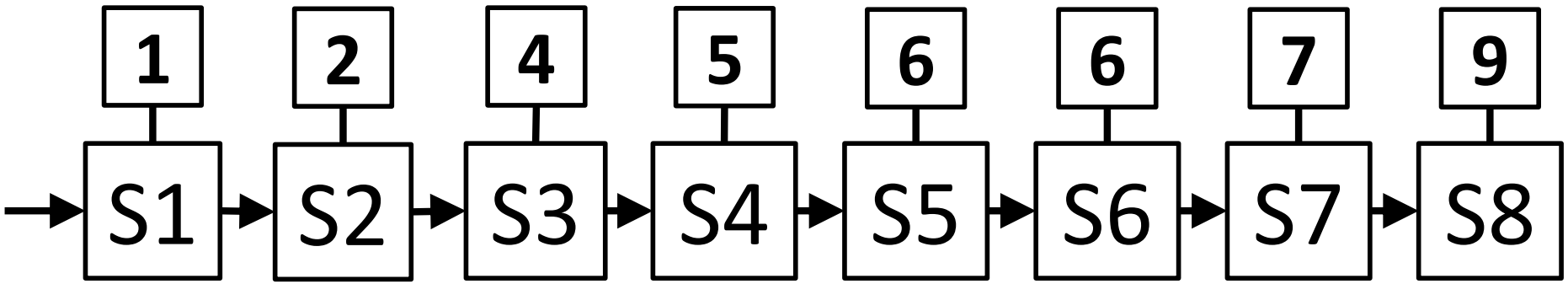
Sorting with pipeline



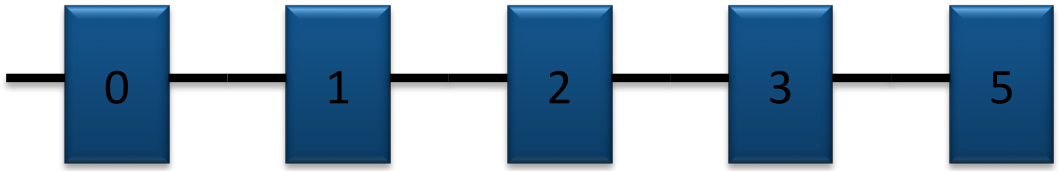
Sorting with pipeline



Sorting with pipeline



Pașii algoritmului de sortare



Pentru **N** valori, **faza 1** durează **2N-1** pași

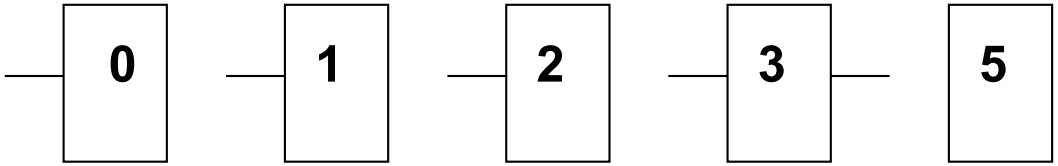
Faza a doua

- Valorile ordonate sunt scoase prin celula din stânga
- Variante:
  1. fiecare procesor începe să transmită spre stânga, imediat ce numerele sunt sortate;
  2. cunoscând poziţia sa din vector şi numărând valorile inspectate, fiecare procesor calculează momentul când începe să transmită spre stânga;
  3. procesorul din dreapta începe să transmită spre stânga imediat ce primeşte o valoare; toate celelalte încep să transmită spre stânga imediat ce primesc o valoare din dreapta;
  4. fiecare procesor începe să transmită spre stânga imediat ce nu mai primeşte o valoare dinspre stânga.

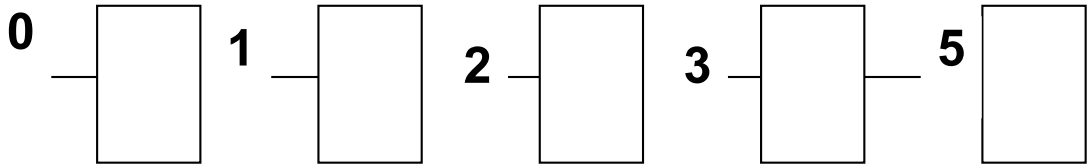
- Complexitate:
  - metoda 3 → **4N-3** pași
  - metoda 4 → **3N-2** pași

- Variantele 1 și 2 impun cunoașterea
  - Pozitiei procesoarelor in vector
  - Contorizarea valorilor

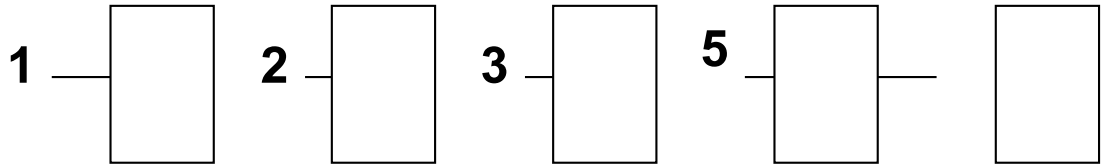
Metoda 4 faza 2



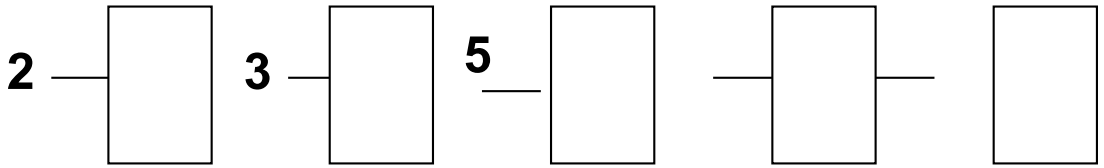
sfarsit faza 1

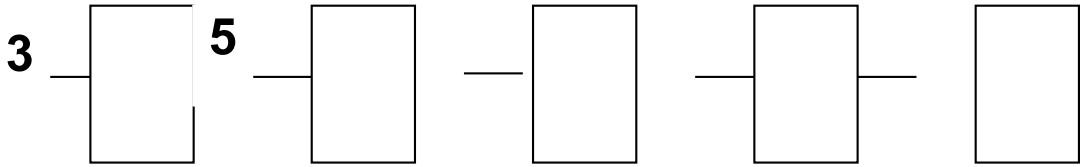


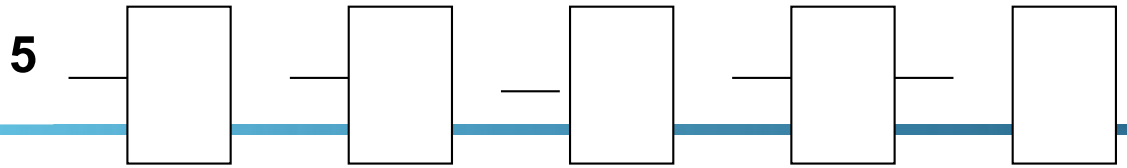
un pas - procesoarele observa ca nu mai primesc din stanga si transmit in stanga



un pas – valoarea din dreapta trece la iesirea din stanga





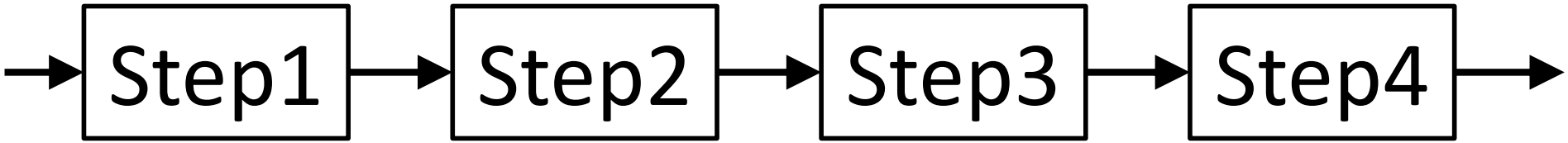


Calcule polinomiale...

Polynomial

$$P(x) = \sum_{i=0}^{n} a_i x^i = a_0 x^0 + a_1 x^1 + a_2 x^2 + \cdots + a_{n-1} x^{n-1} + a_n x^n, n \geq 0$$

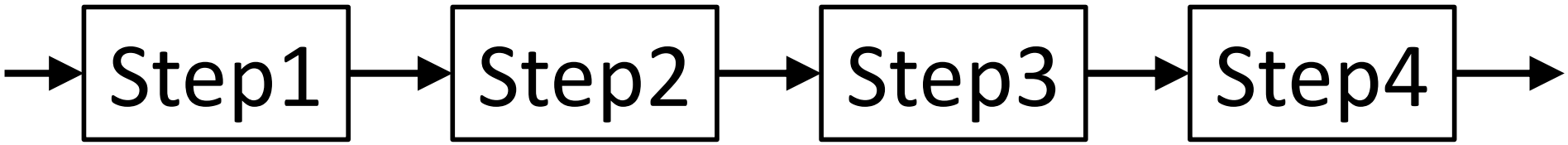
Pipeline



Polynomial calculation + Pipeline

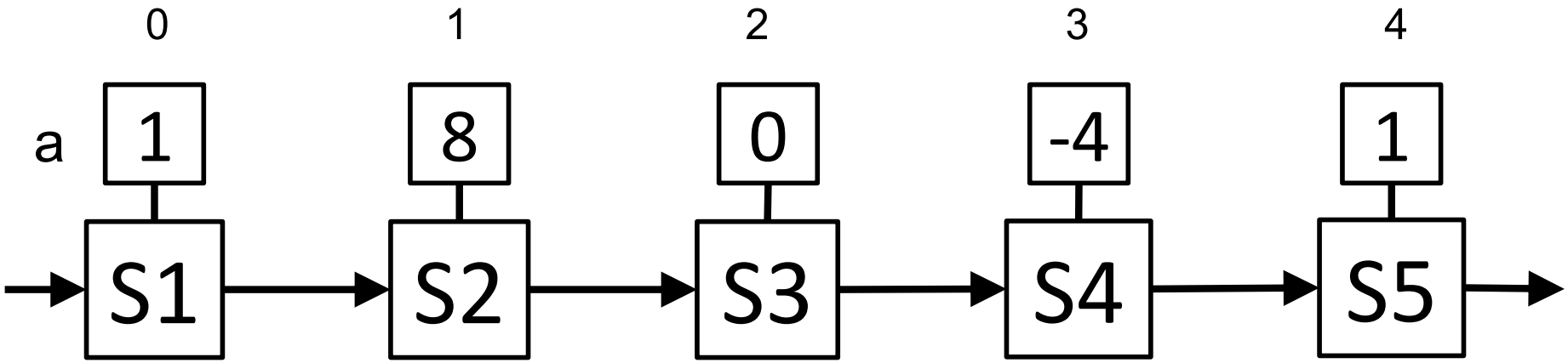
$$P(x) = 1 + 8x + (-4)x^3 + x^4$$

	0	1	2	3	4
a	1	8	0	-4	1



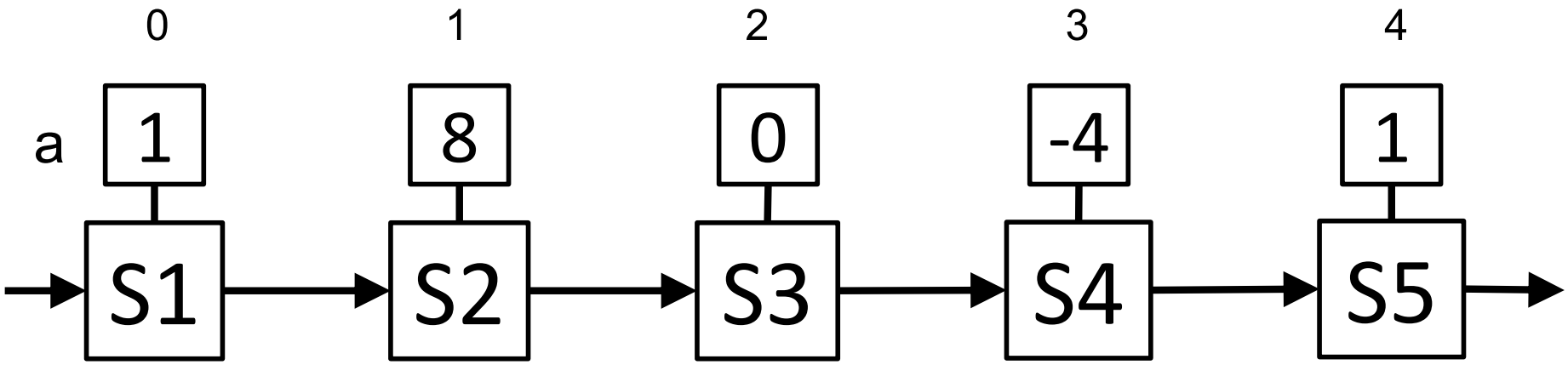
Polynomial calculation + Pipeline

$$P(x) = 1 + 8x + (-4)x^3 + x^4$$



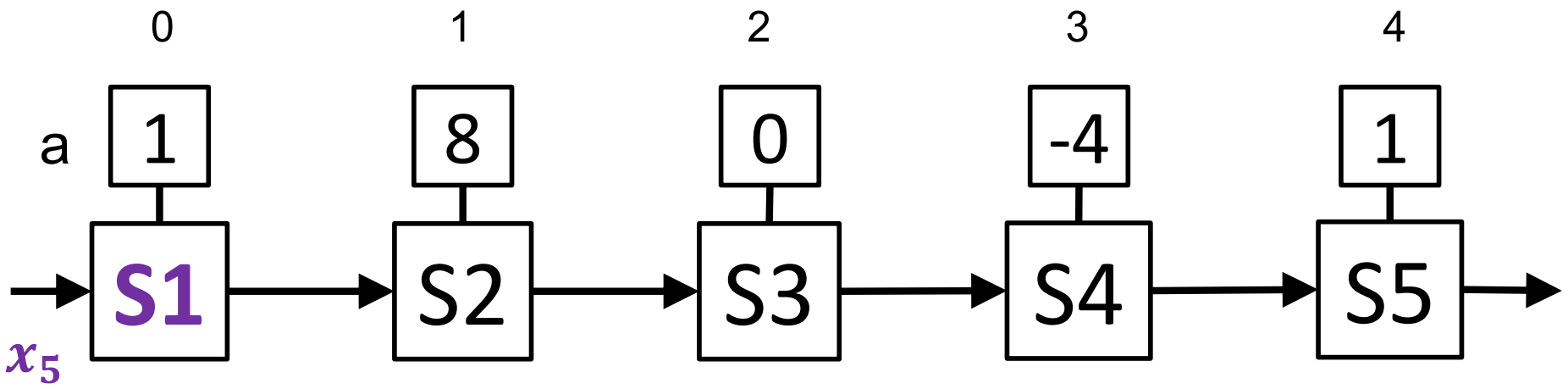
Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4 x_5



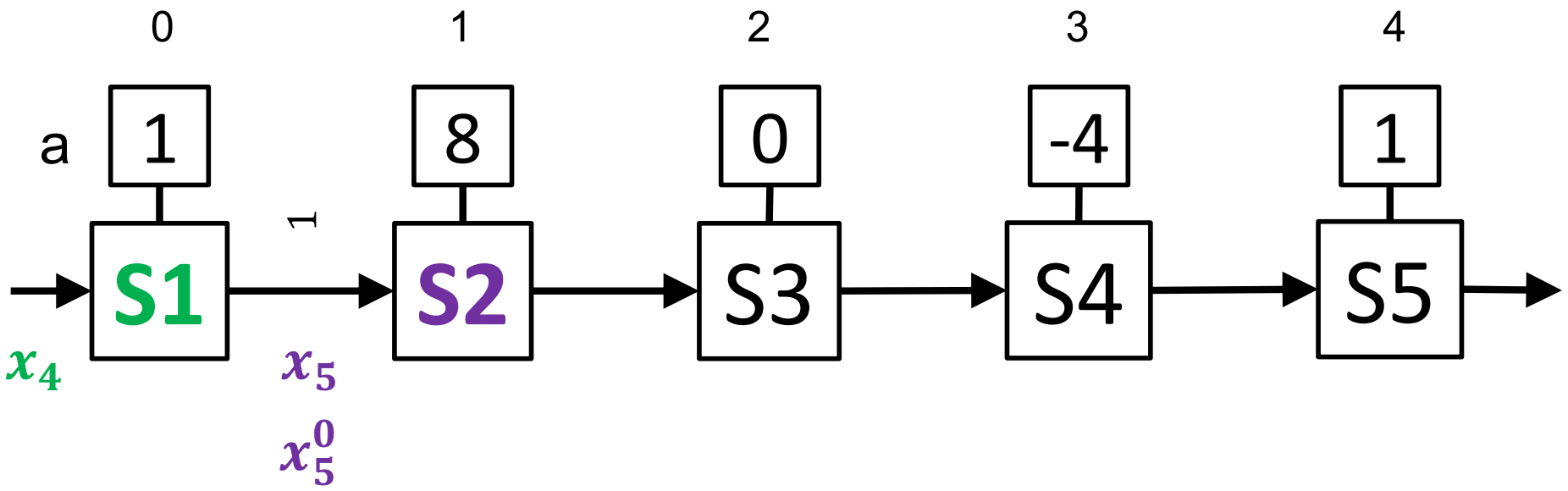
Polynomial calculation + Pipeline

x_1 x_2 x_3 x_4



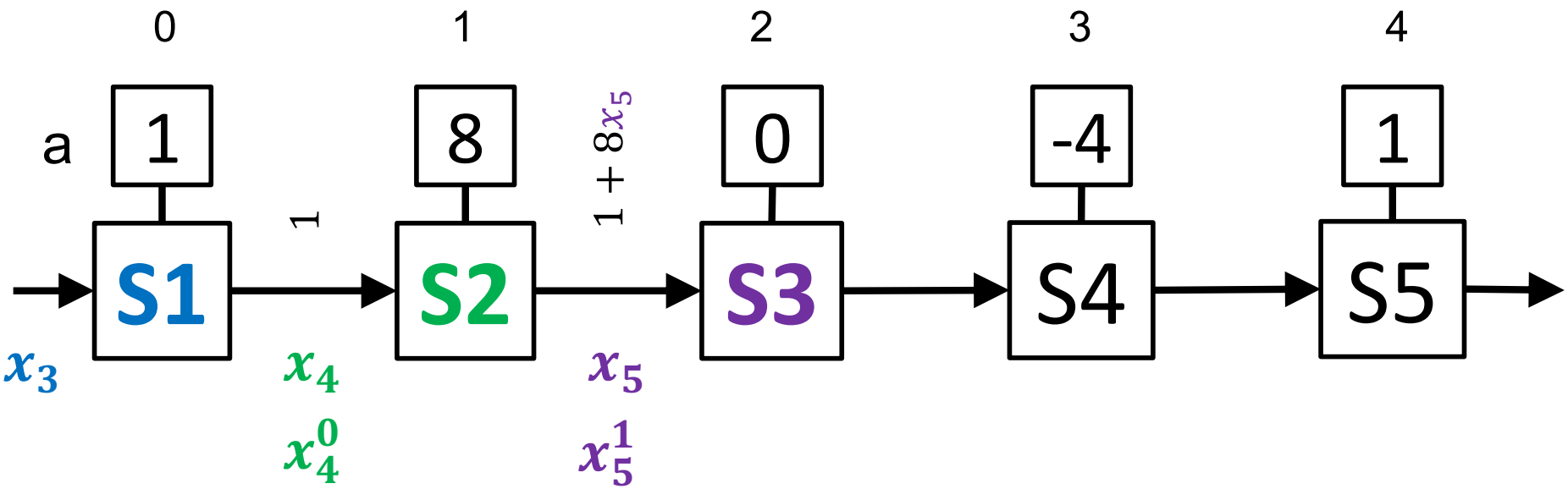
Polynomial calculation + Pipeline

x_1 x_2 x_3



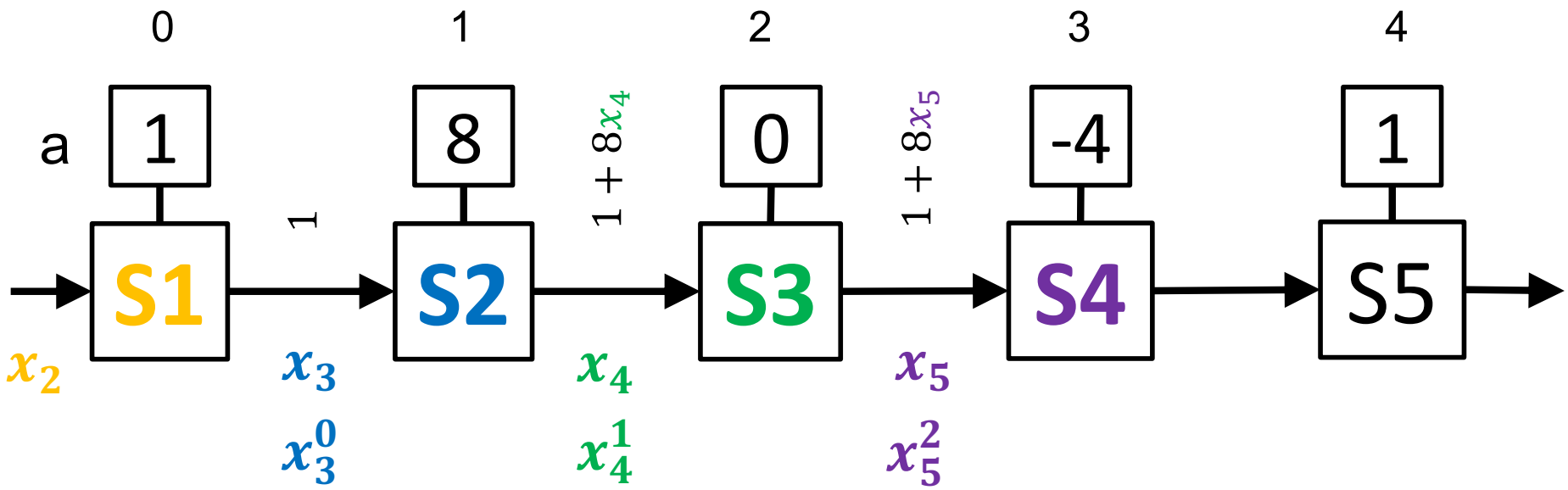
Polynomial calculation + Pipeline

x_1 x_2

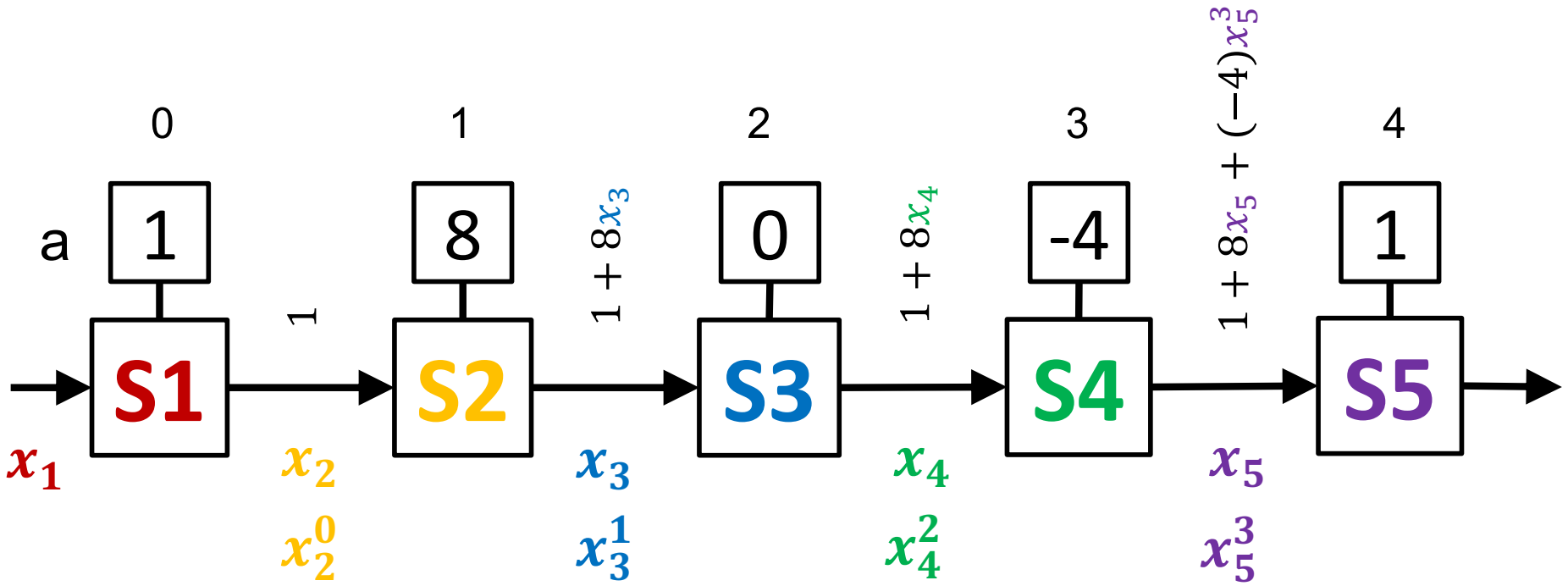


Polynomial calculation + Pipeline

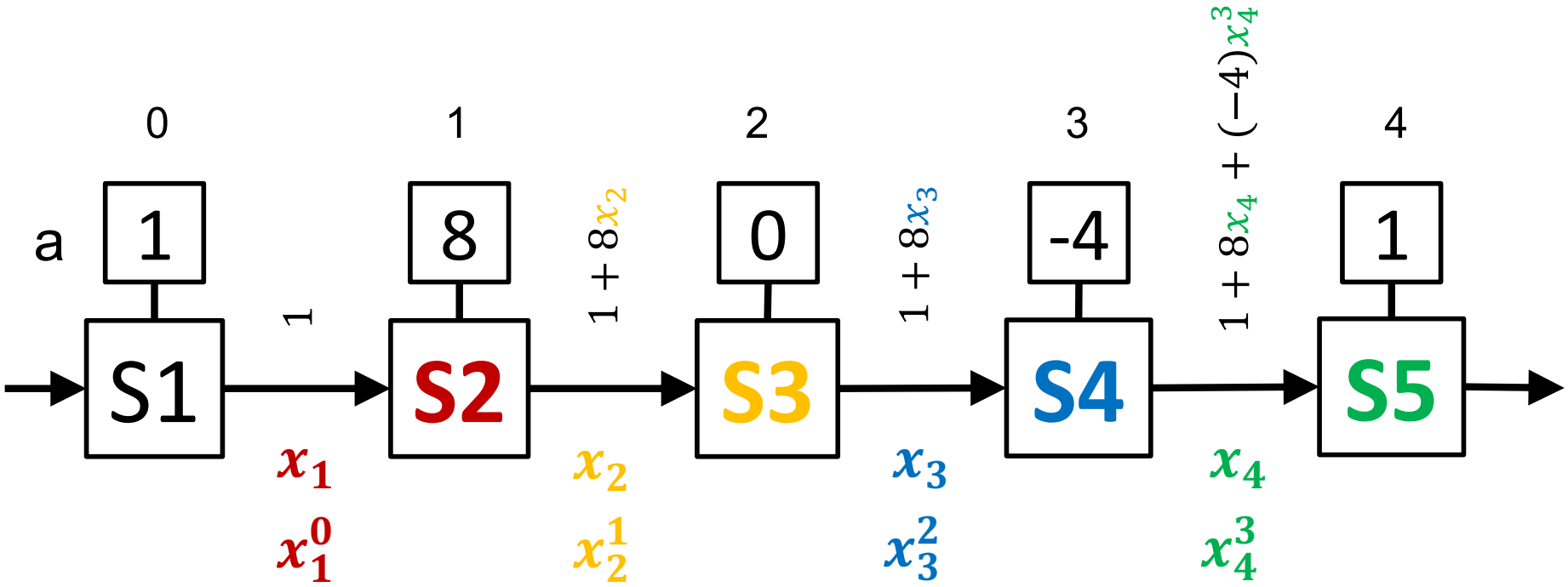
x_1



Polynomial calculation + Pipeline

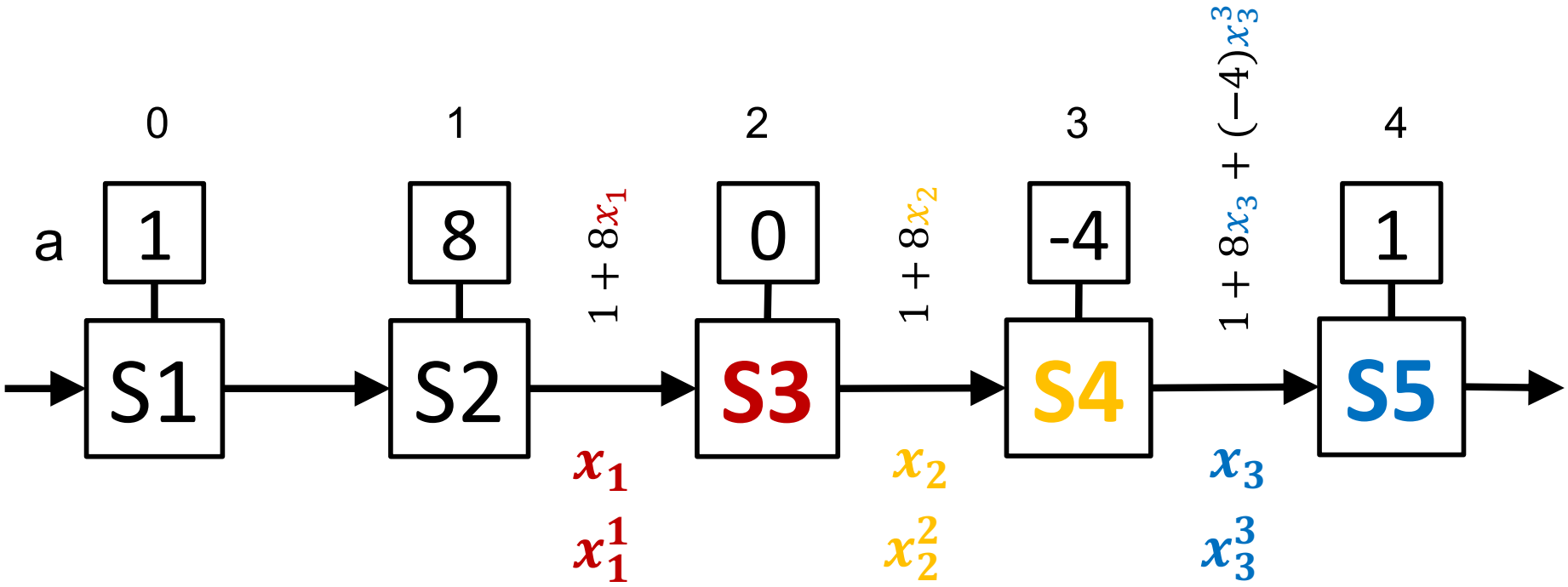


Polynomial calculation + Pipeline



$$1 + 8x_5 + (-4)x_5^3 + x_5^4$$

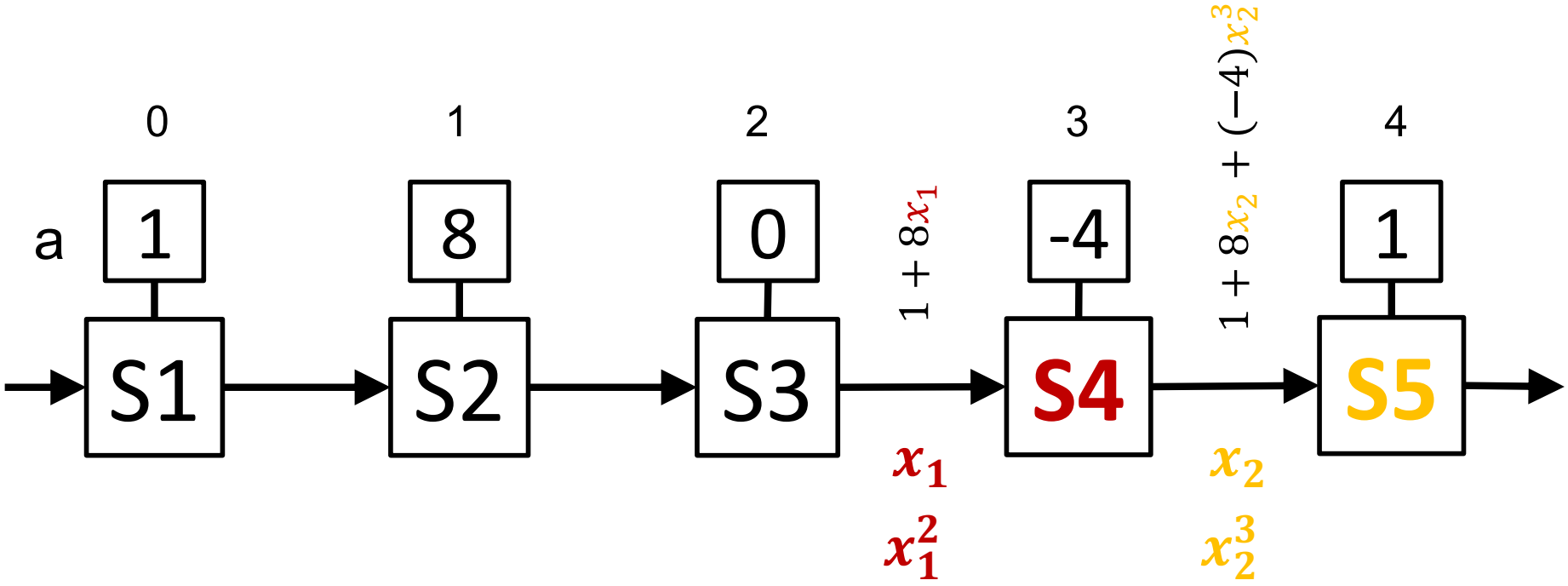
Polynomial calculation + Pipeline



$$1+8x_4+(-4)x_4^3+x_4^4$$

$$1+8x_5+(-4)x_5^3+x_5^4$$

Polynomial calculation + Pipeline

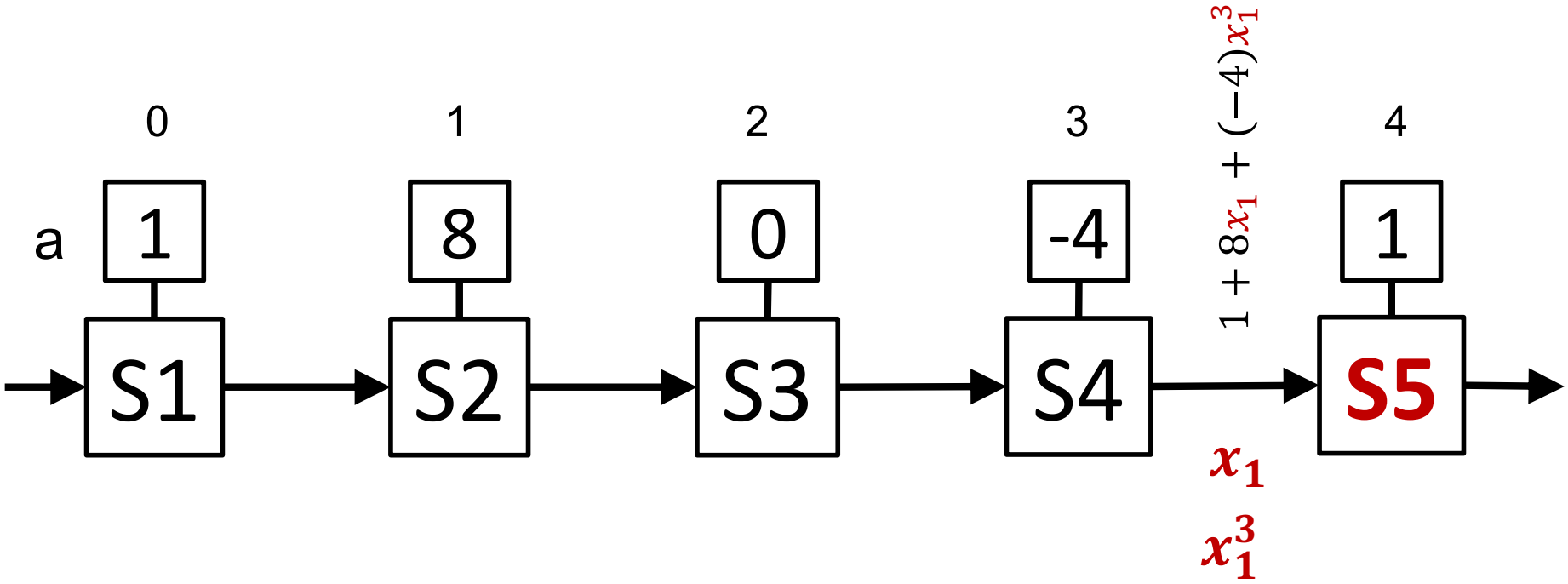


$$1+8x_3+(-4)x_3^3+x_3^4$$

$$1+8x_4+(-4)x_4^3+x_4^4$$

$$1+8x_5+(-4)x_5^3+x_5^4$$

Polynomial calculation + Pipeline



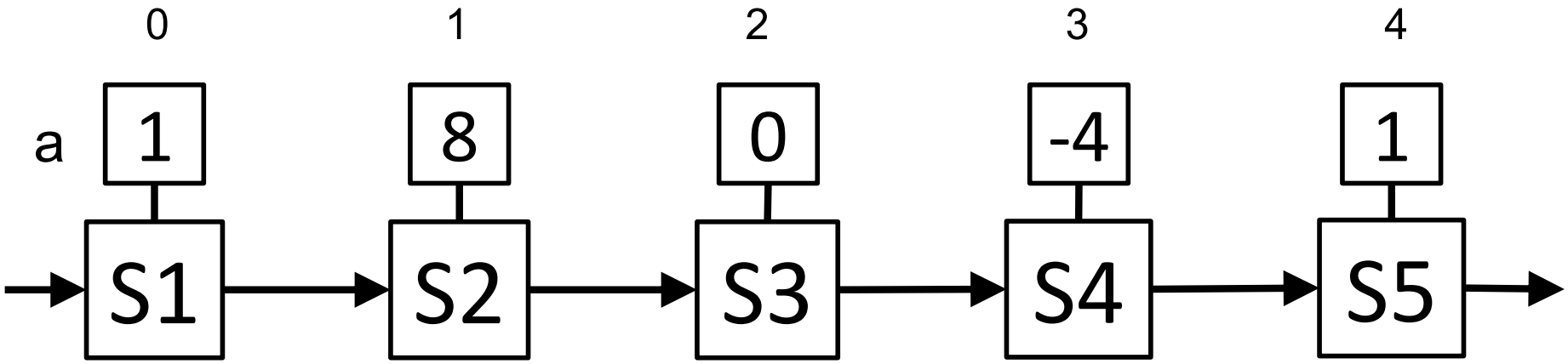
$1 + 8x_2 + (-4)x_2^3 + x_2^4$

$1 + 8x_3 + (-4)x_3^3 + x_3^4$

$1 + 8x_4 + (-4)x_4^3 + x_4^4$

$1 + 8x_5 + (-4)x_5^3 + x_5^4$

Polynomial calculation + Pipeline



$$1 + 8x_1 + (-4)x_1^3 + x_1^4$$

$$1 + 8x_2 + (-4)x_2^3 + x_2^4$$

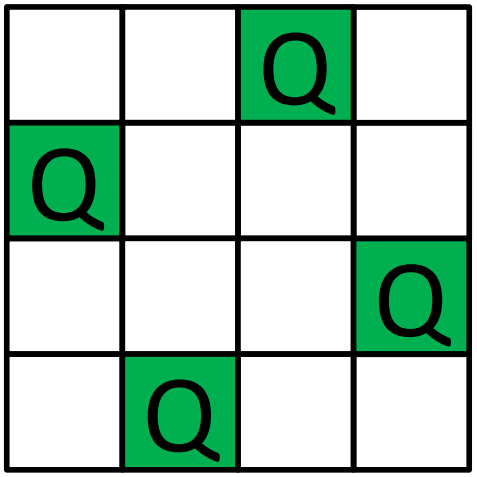
$$1 + 8x_3 + (-4)x_3^3 + x_3^4$$

$$1 + 8x_4 + (-4)x_4^3 + x_4^4$$

$$1 + 8x_5 + (-4)x_5^3 + x_5^4$$

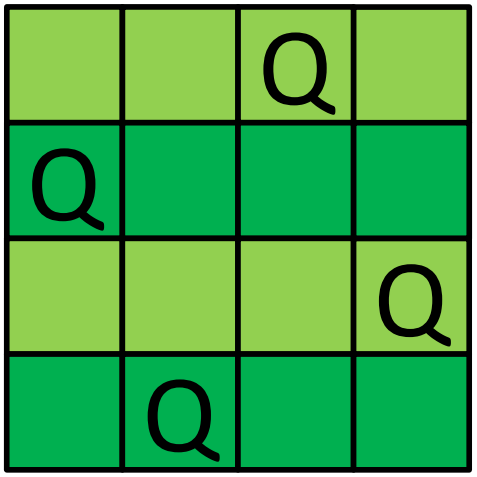
Aplicație: problema Damelor

N Queens Problem



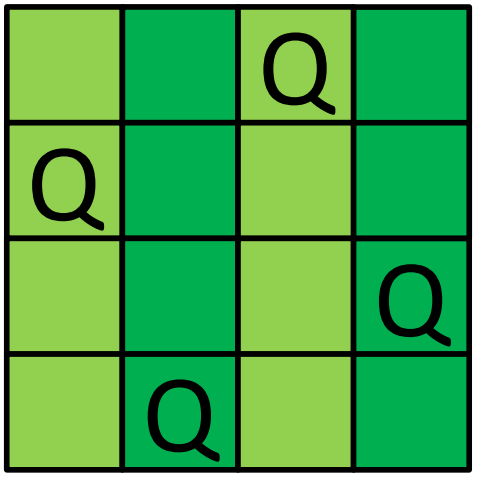
N Queens Problem

No more then one queen per line



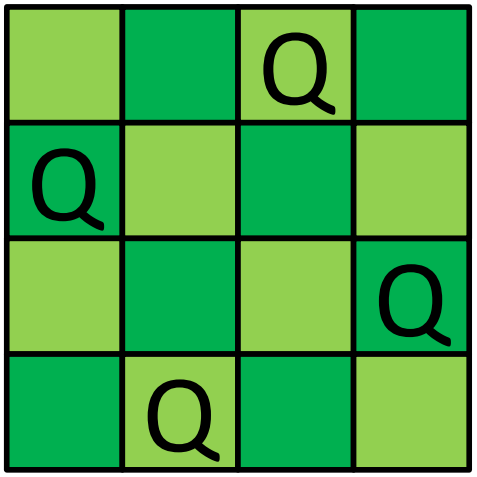
N Queens Problem

No more then one queen per column

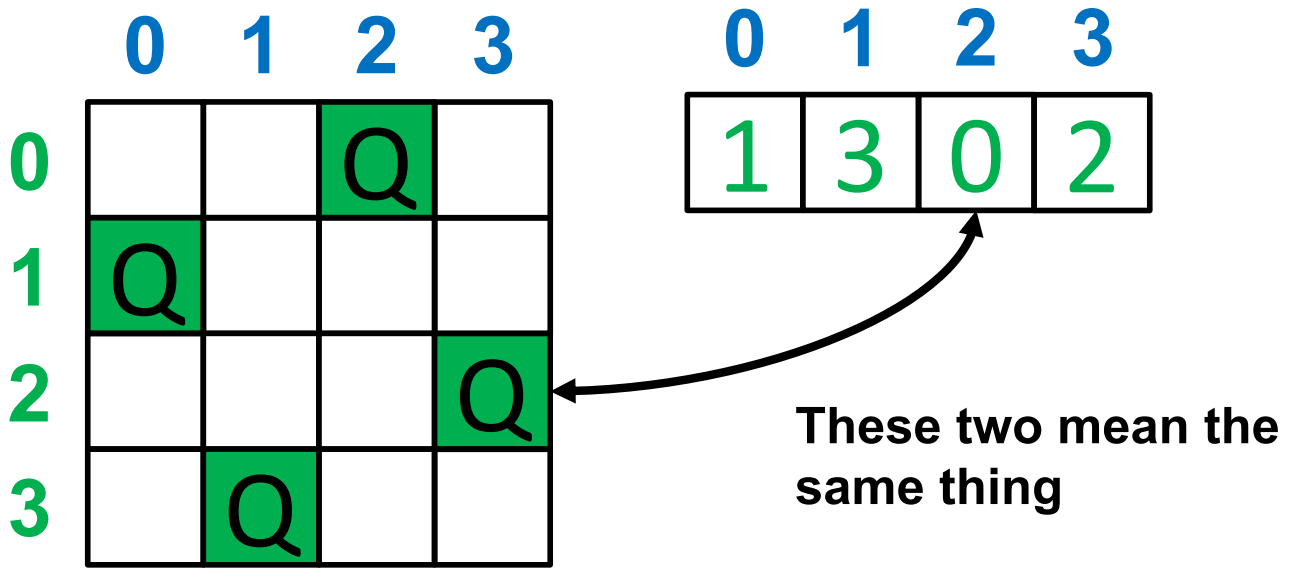


N Queens Problem

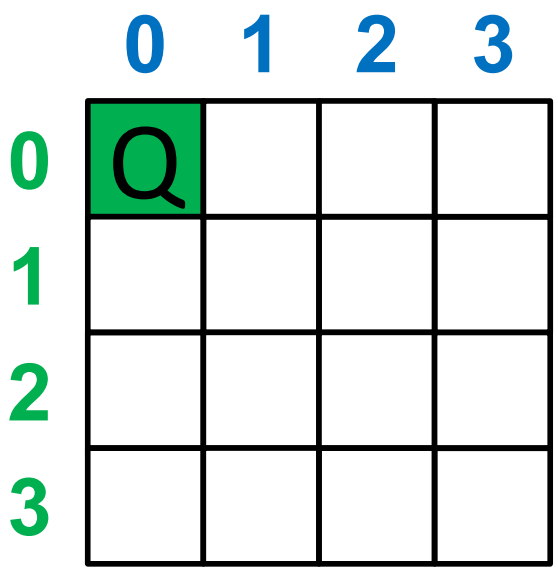
No more then one queen per diagonals

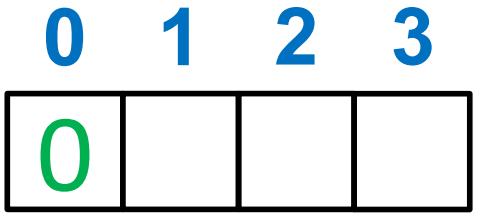


N Queens Problem

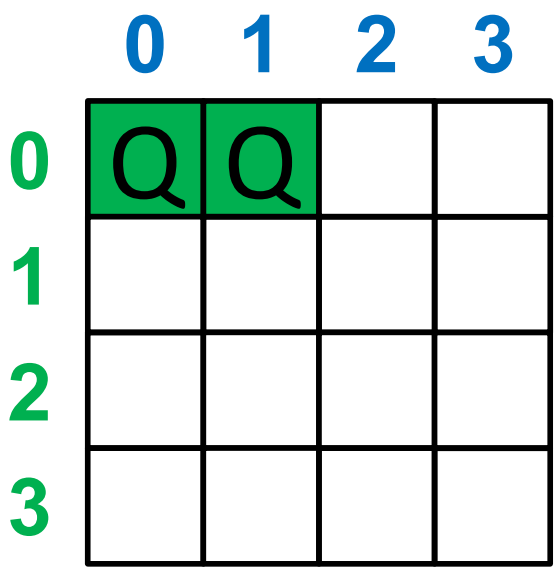


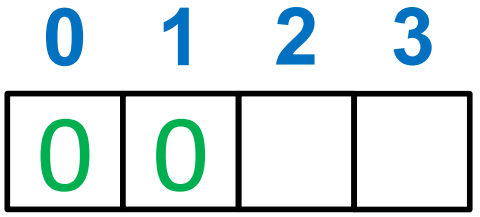
N Queens Problem - Solution





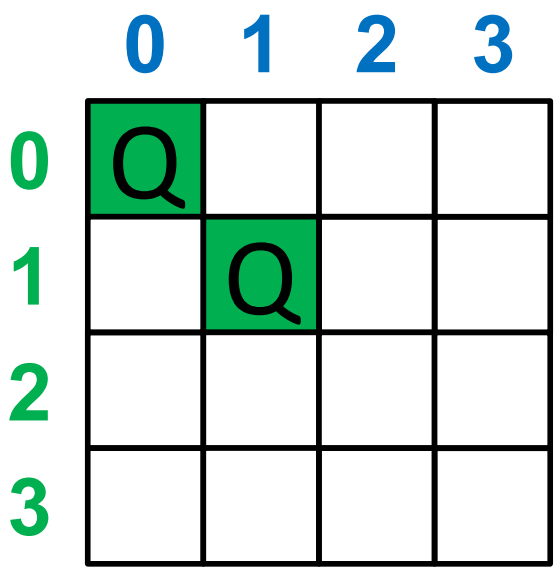
N Queens Problem - Solution

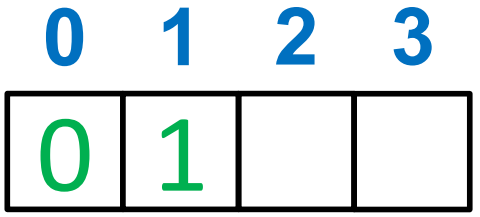




Line conflict

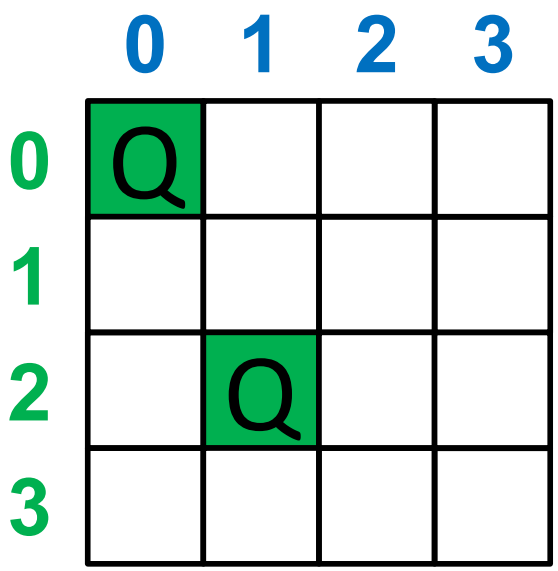
N Queens Problem - Solution





Diagonal conflict

N Queens Problem - Solution

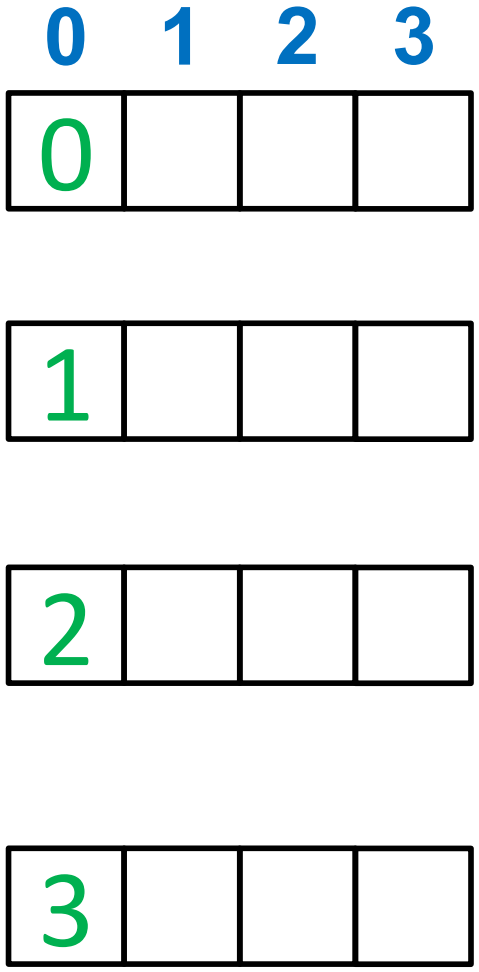


0	1	2	3
0	2		

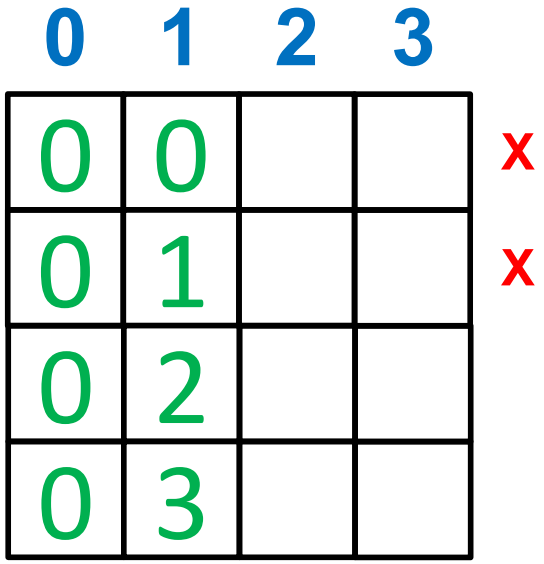
OK.

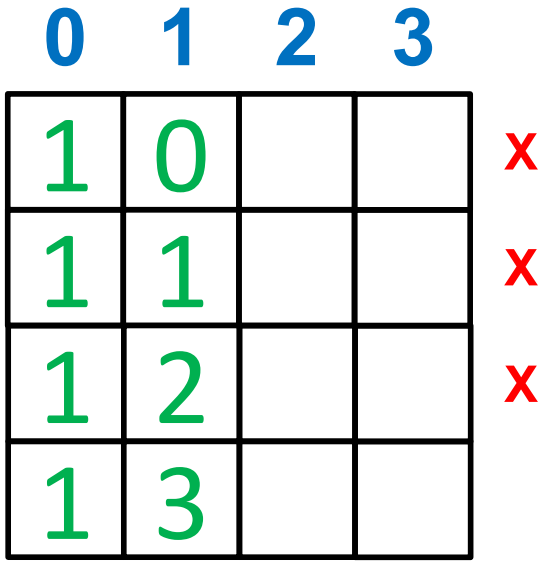
And so on…

N Queens Problem – Parallel Solution

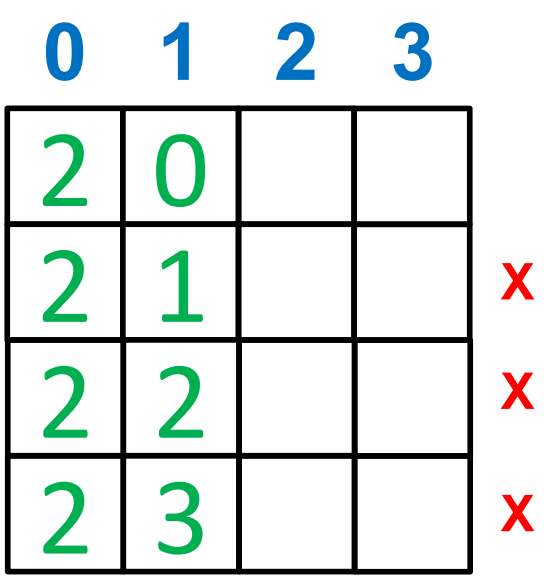


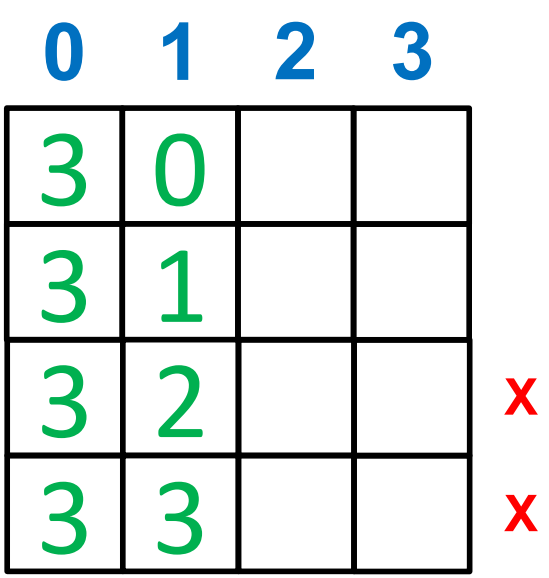
N Queens Problem – Parallel Solution





And so on…

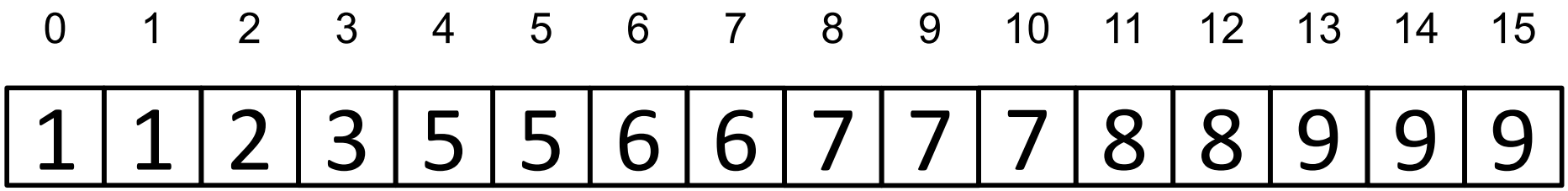




Cautare binara

Binary Search

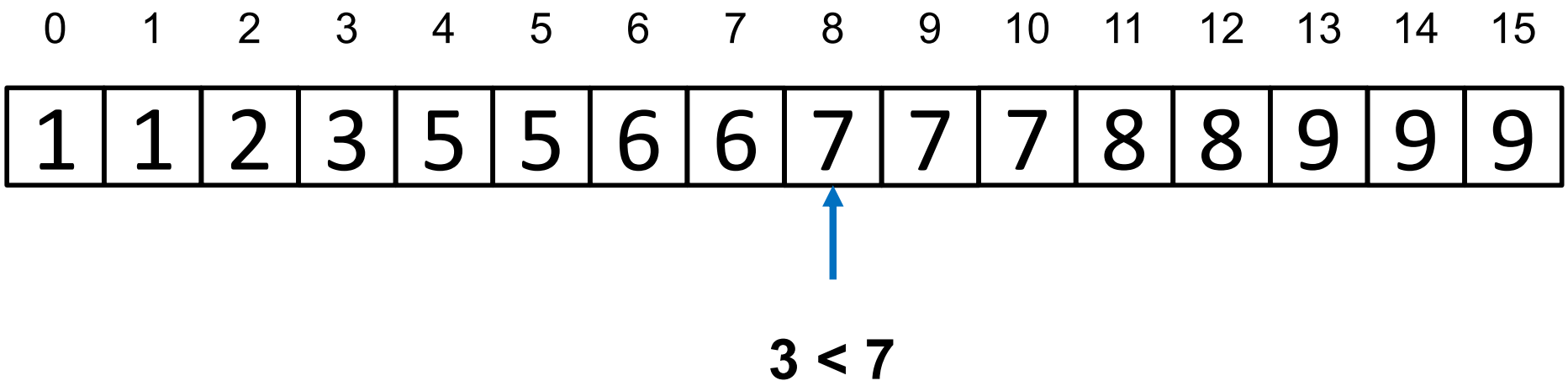
Searching for 3



Binary Search

Searching for | 3 |

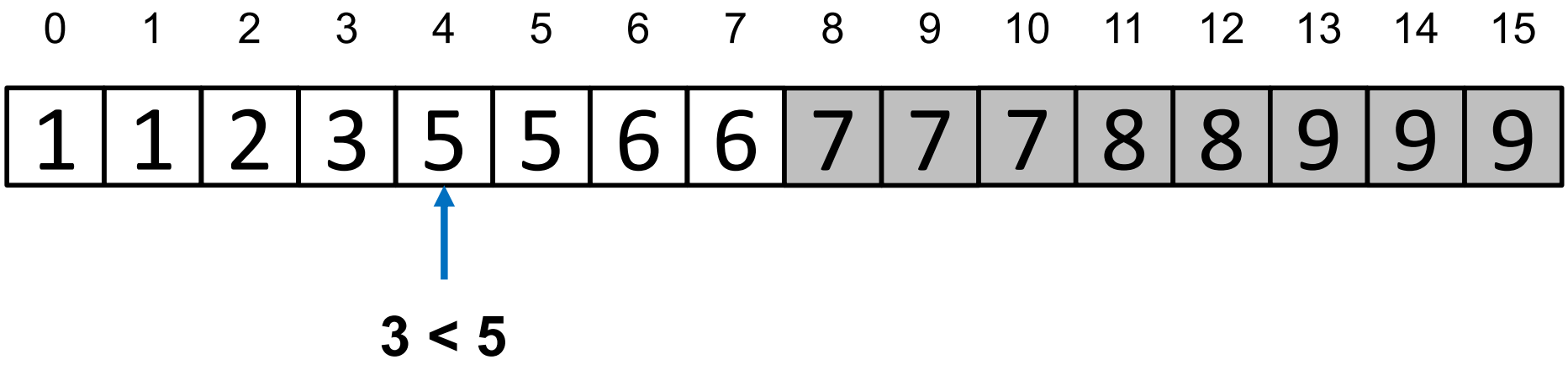
Interest area | 0 | 15 |



Binary Search

Searching for 3

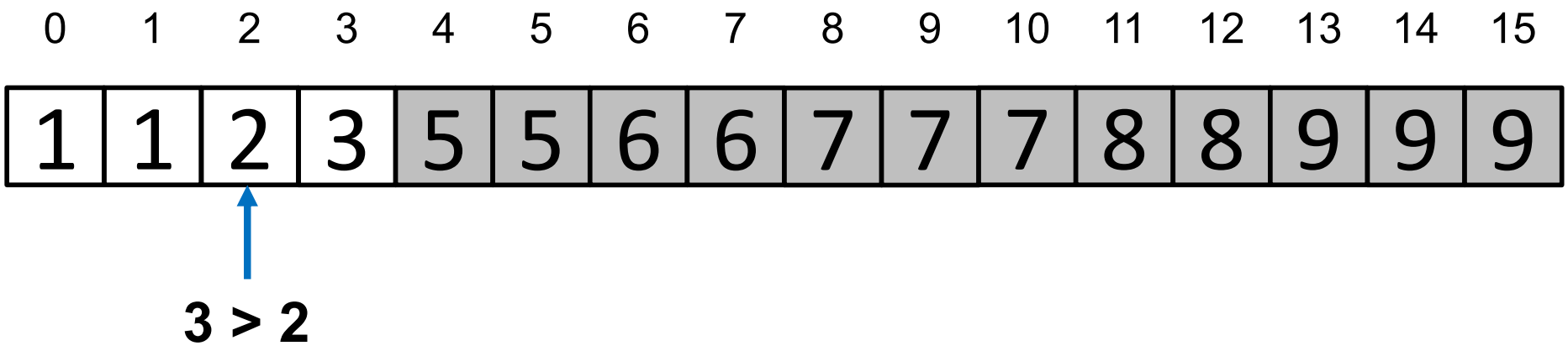
Interest area 0 7



Binary Search

Searching for 3

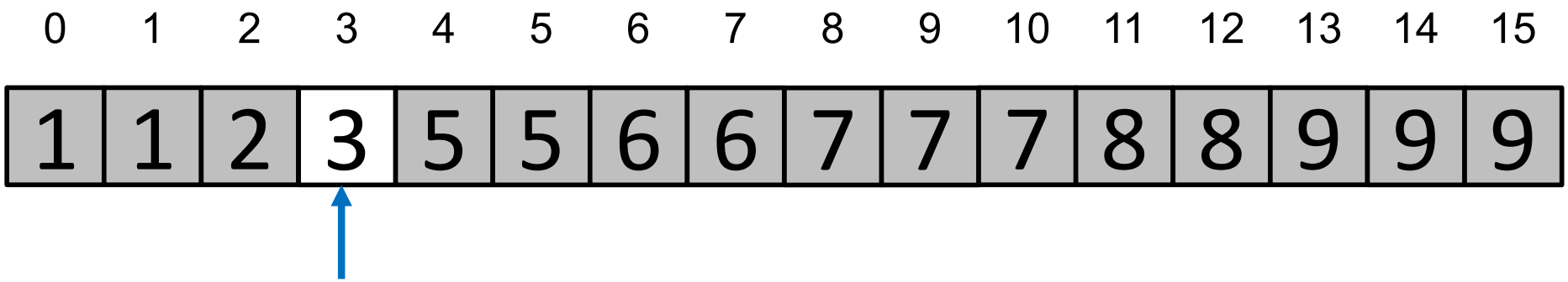
Interest area 0 3



Binary Search

Searching for | 3 |

Interest area | 3 | 3 |



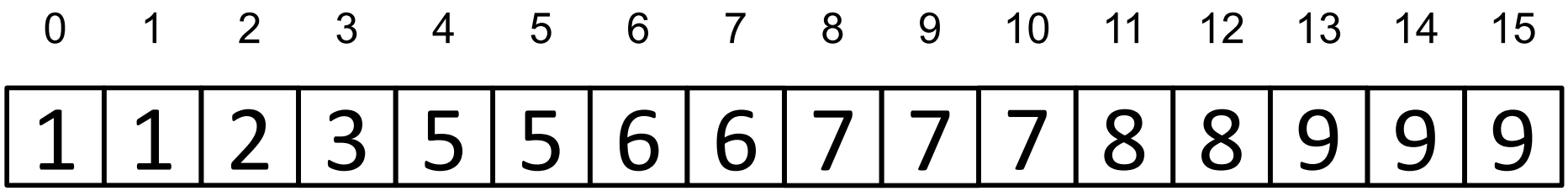
3 = 3

end

O($log_2(n)$) time

Parallel Search

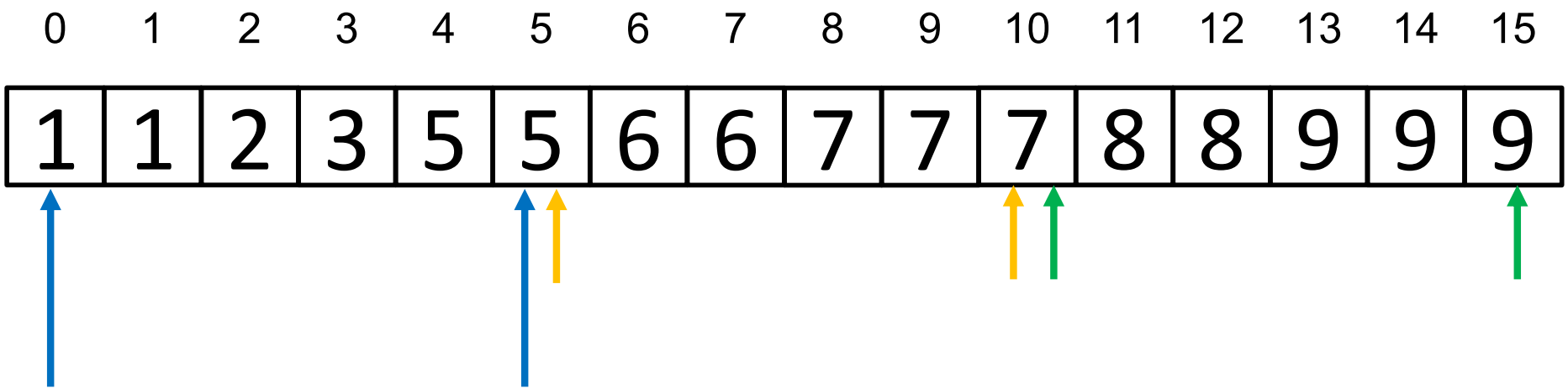
Searching for 3



Parallel Search

Searching for 3

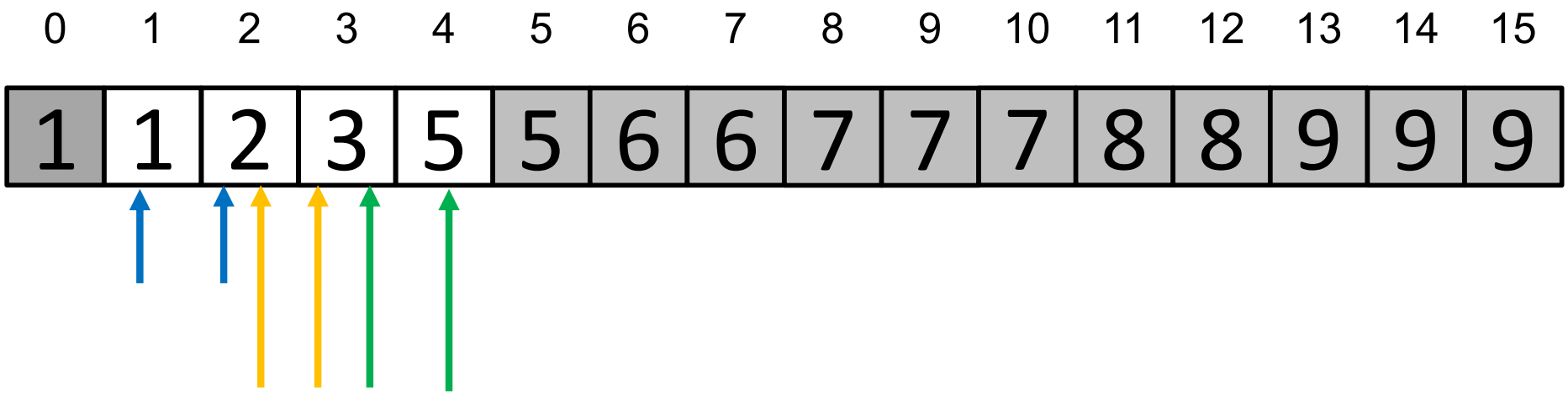
Interest area 0 15



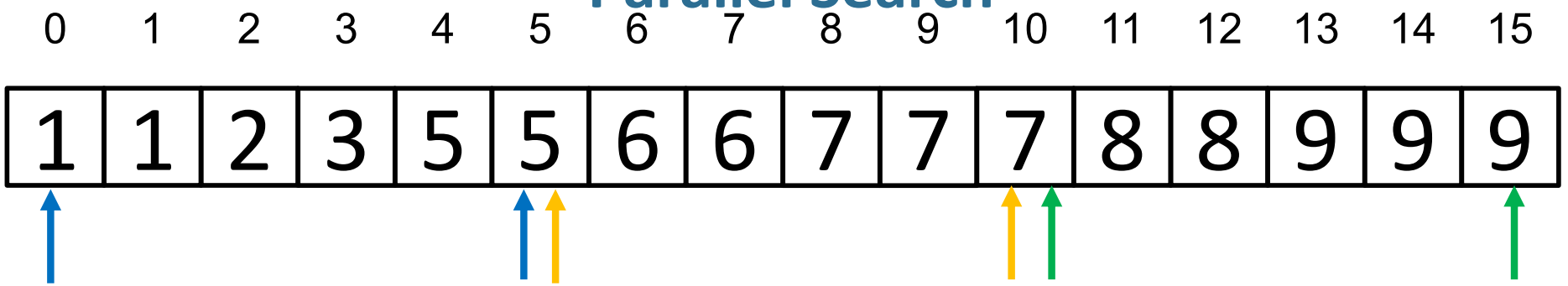
Parallel Search

Searching for 3

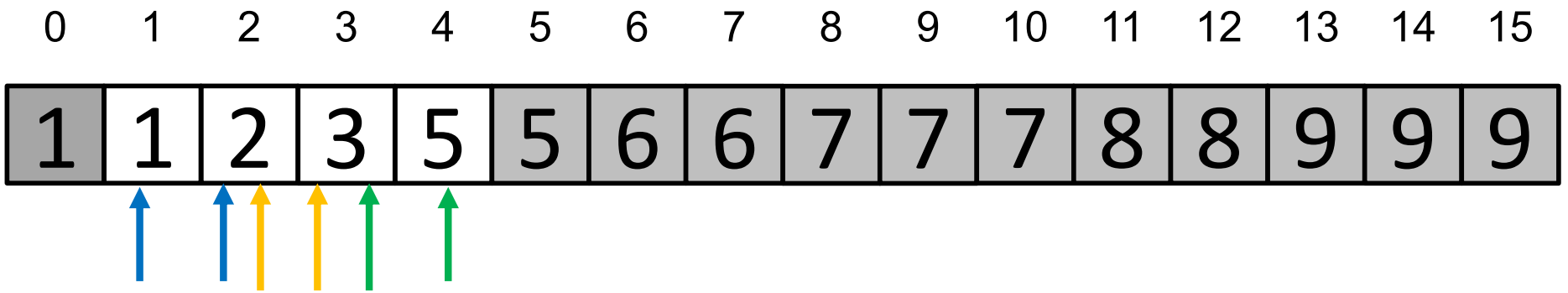
Interest area 1 4



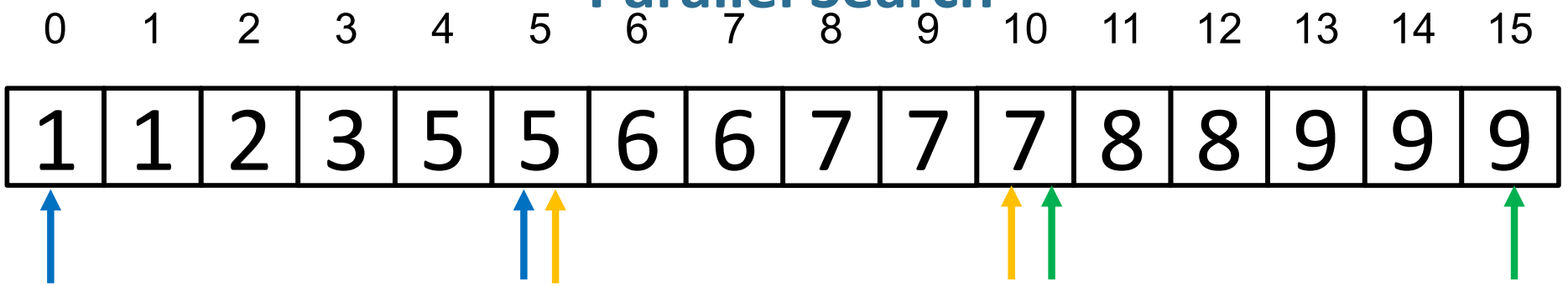
Parallel Search

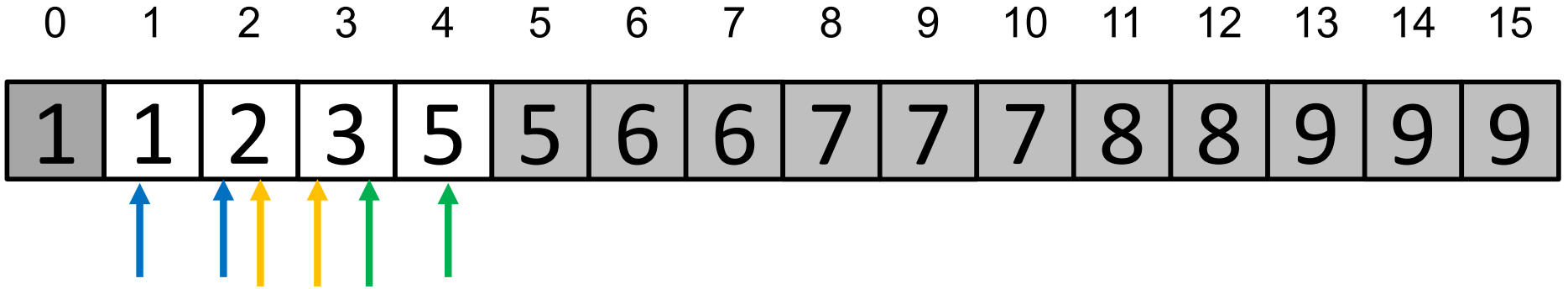


Operations can**NOT** be executed in parallel



Parallel Search



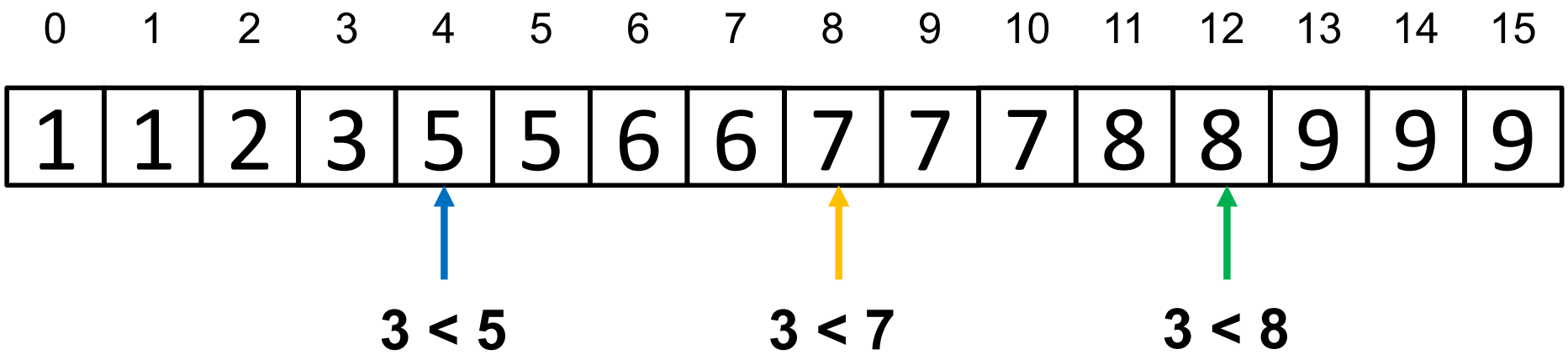


Parallel Search – solutia 2 (cu mai putine threaduri)

Parallel Search – solution 2 (fewer threads)

Searching for | 3 |

Interest area | 0 | 15 |

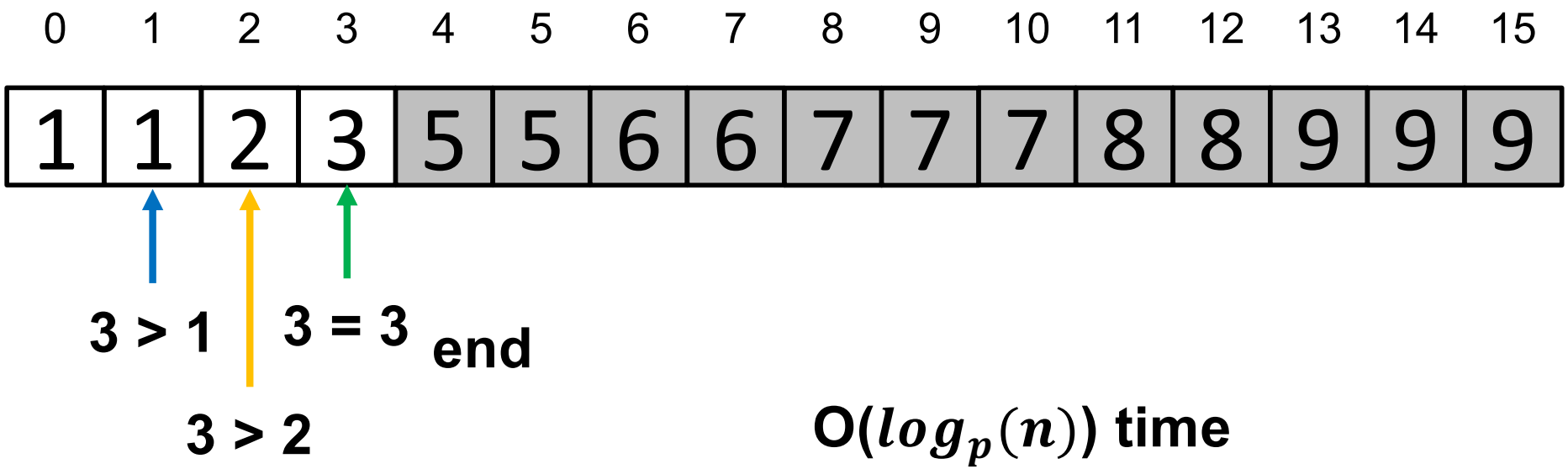


Operations can be executed in parallel

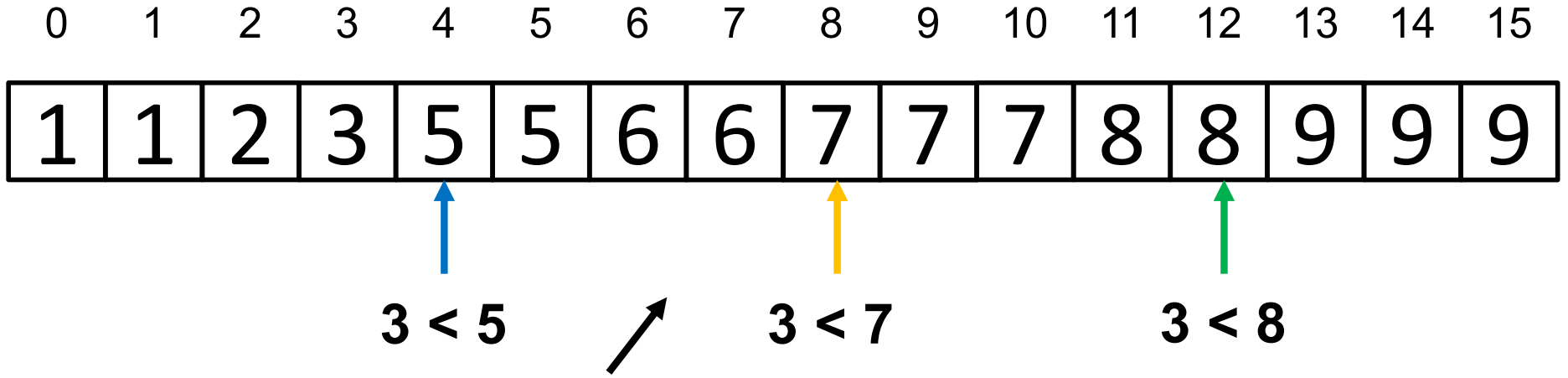
Parallel Search

Searching for 3

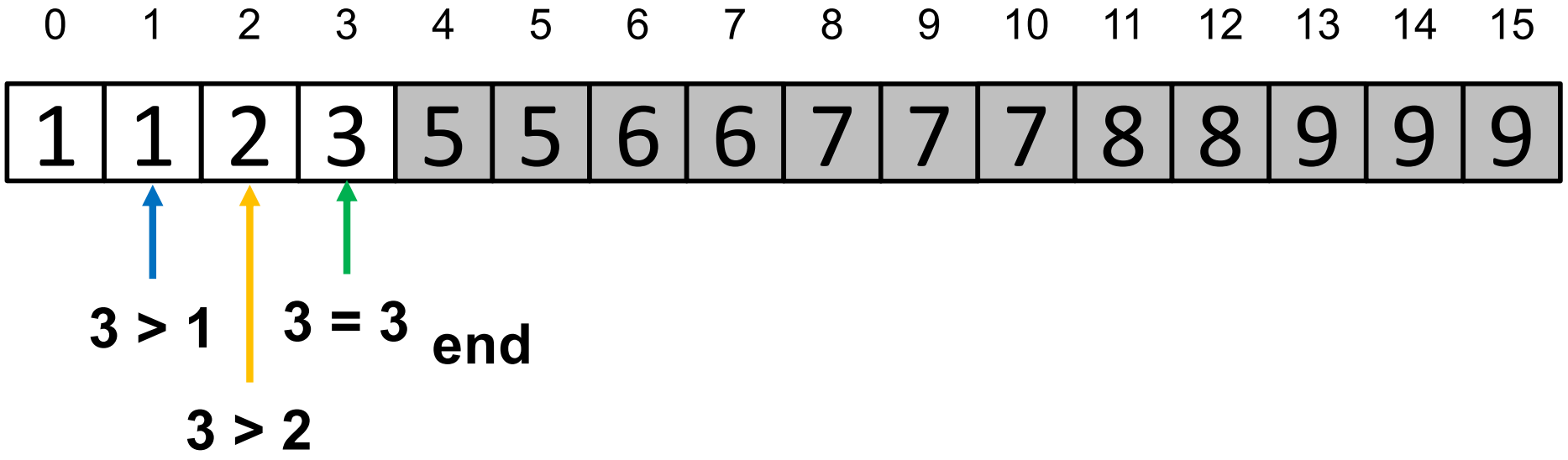
Interest area 0 3



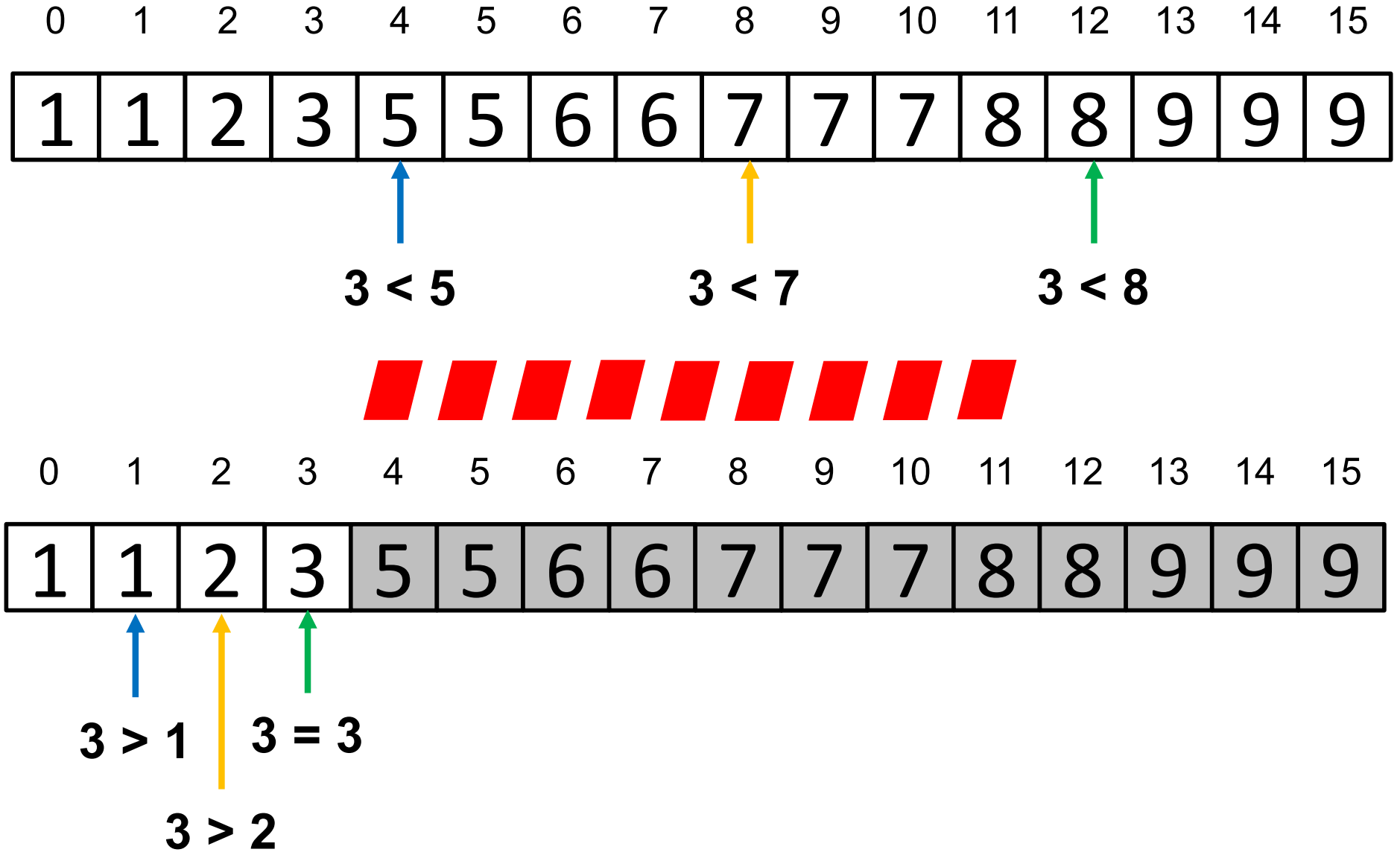
Parallel Search



Operations can**NOT** be executed in parallel

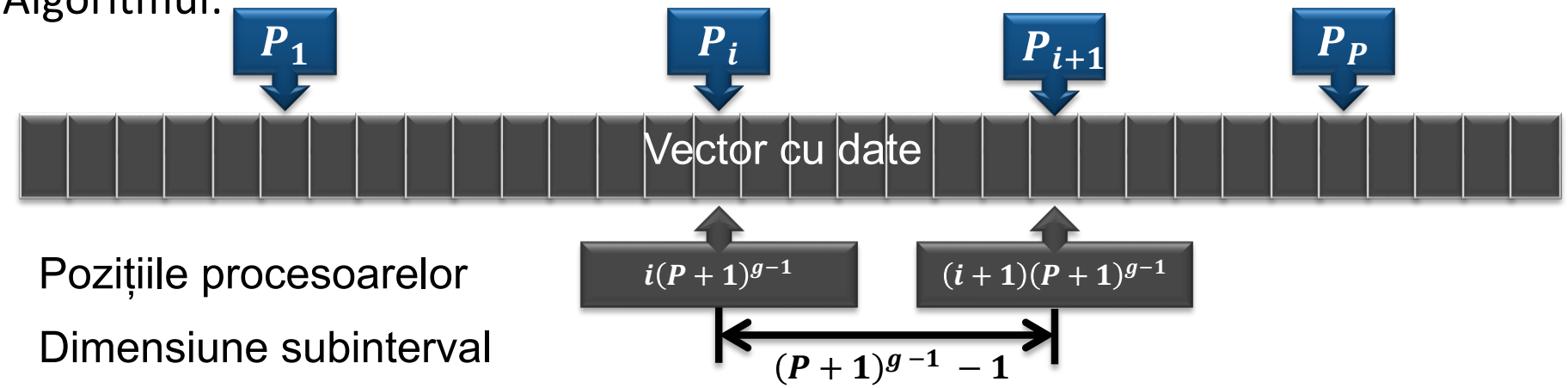


Parallel Search



Performanta?

- Algoritmul:

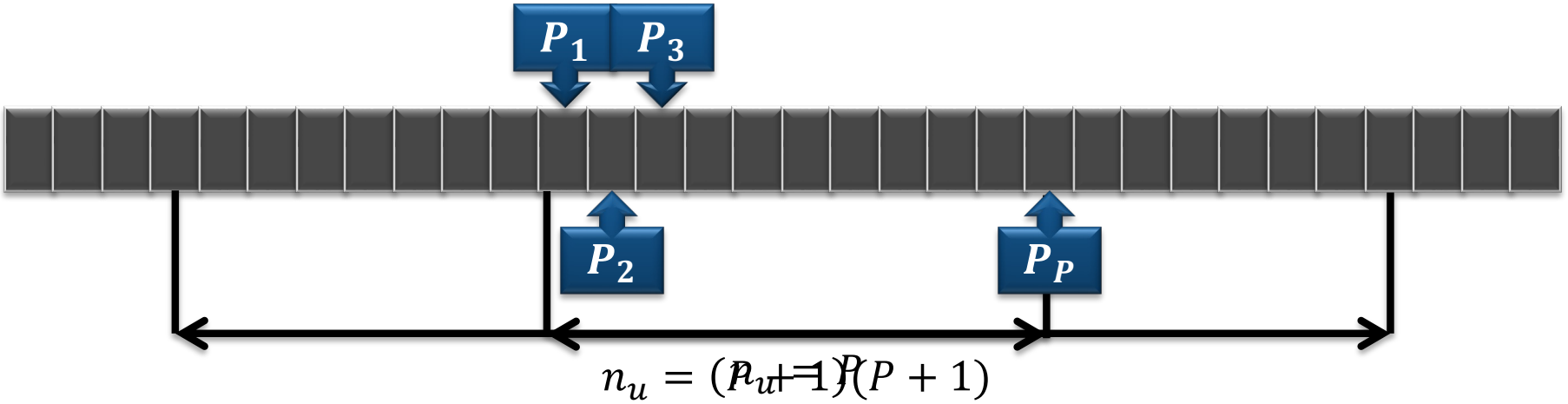


  - **g** : numărul de paşi
  - **P** : numărul de procesoare
  - **n** : numărul de elemente ale vectorului
  - P_i : procesorul cu indicele i

- Deoarece fiecare etapă se aplică unei secvenţe de dimensiune $\frac{1}{P+1}$ din secvenţa precedentă, sunt necesare:

$O(\log_{P+1}(n+1))$ etape.

Justificare



în ultimul pas P procesoare acoperă P elemente

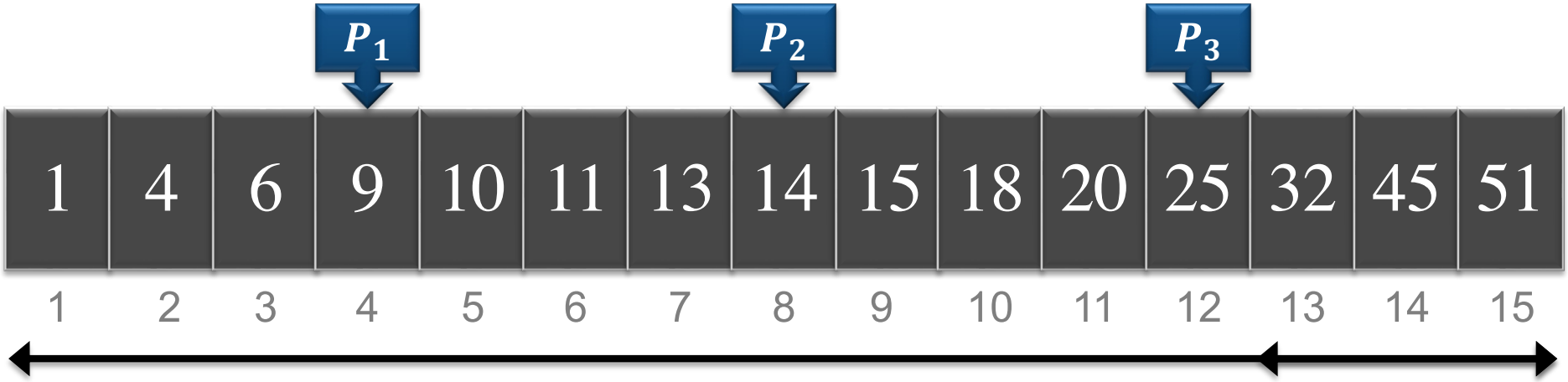
în pasul anterior $(P+1)^2 - 1$

în g paşi $(P+1)^g - 1$

din $(P+1)^g - 1 \geq n$ *(n = nr elemente şir)*

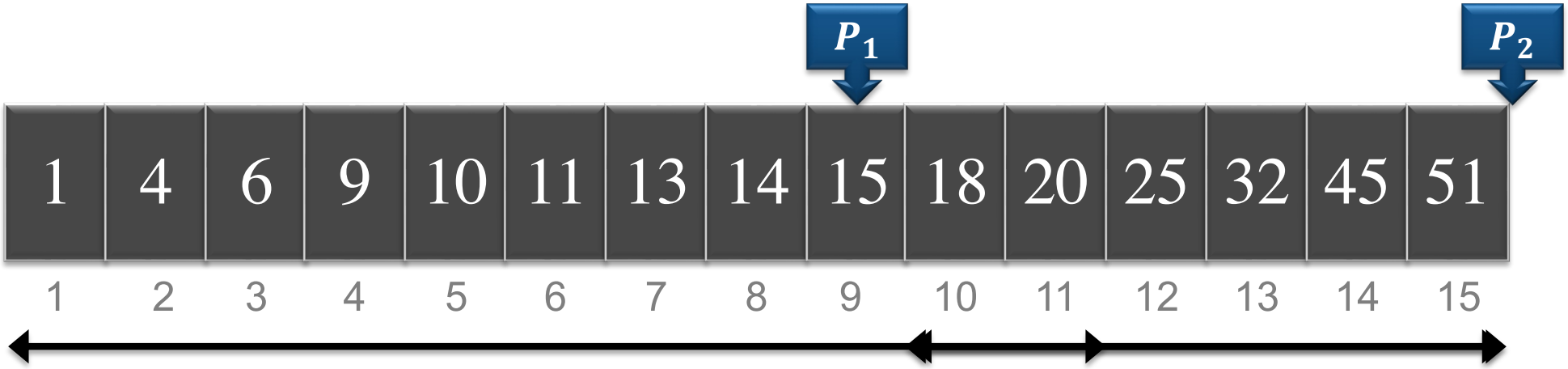
rezultă $\boldsymbol{g = sup\left(\frac{\log(n+1)}{\log(P+1)}\right)}$

Exemplu 1



- X = 25 *(numărul căutat)*
- P = 3 *(numărul de procesoare)*
- n = 15 *(dimensiunea vectorului)*
- Din $i(P+1)^{g-1} \Rightarrow P_1$ va ocupa poziția 4 din vector
  - $i = 2$
- Din $(P+1)^{g-1} \Rightarrow g = 2$
- Din $i(P+1)^{g-1} \Rightarrow P_2$ va ocupa poziția 8 din vector
- Lungimea unei subsecvențe în primul pas este $(P+1)^{g-1} = 3$.
  - Din $i(P+1)^{g-1} \Rightarrow P_3$ va ocupa poziția 12 din vector

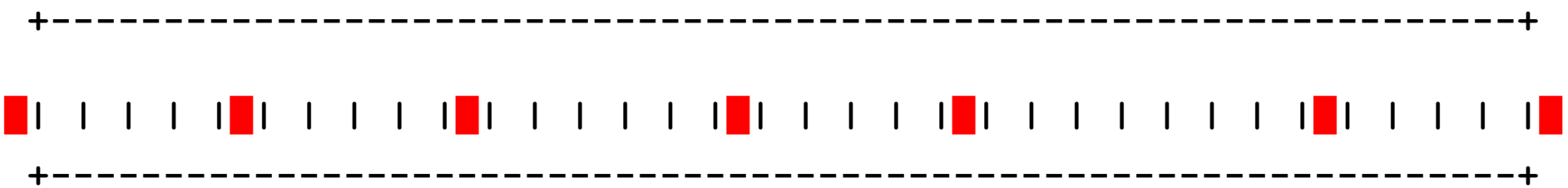
Exemplu 2



- X = 21 *(numărul căutat)*
- P = 2 *(numărul de procesoare)*
- n = 15 *(dimensiunea vectorului)*
- Din $i(P+1)^{g-1} \Rightarrow P_1$ va ocupa poziția 9 din vector
  - $i = 2$
  - Din $i(P+1)^{g-1} \Rightarrow g = 2$
- Din $(P+1)^{g-1} \Rightarrow P_2$ va ocupa poziția **18** din vector
- Lungimea unei subsecvenţe în primul pas este $(P+1)^{g-1} = \mathbf{8}$.

Algoritm

```
/1/  /* Initializarea indicilor secventei de cautat */
   int q = 1;              /* limita stg interval crt */
   int r = n;              /* limita dr interval crt */
/2/  /* Initializarea rezultatului k si a numarului de pasi g */
   int x, k = 0;                        /* x = numar cautat, k = pozitia lui in sir */
   int g = sup(log(n+1)/log(P+1))
   enum sens {stg,dr};
   sens c[0:P+1];  /*c[i] – x este la stg / dr pozitiei procesului i*/
    c[0] = dr; c[P+1] = stg;
    int j[1:P];                    /* j[i] = pozitia din sir inspectata de proces i*/
```



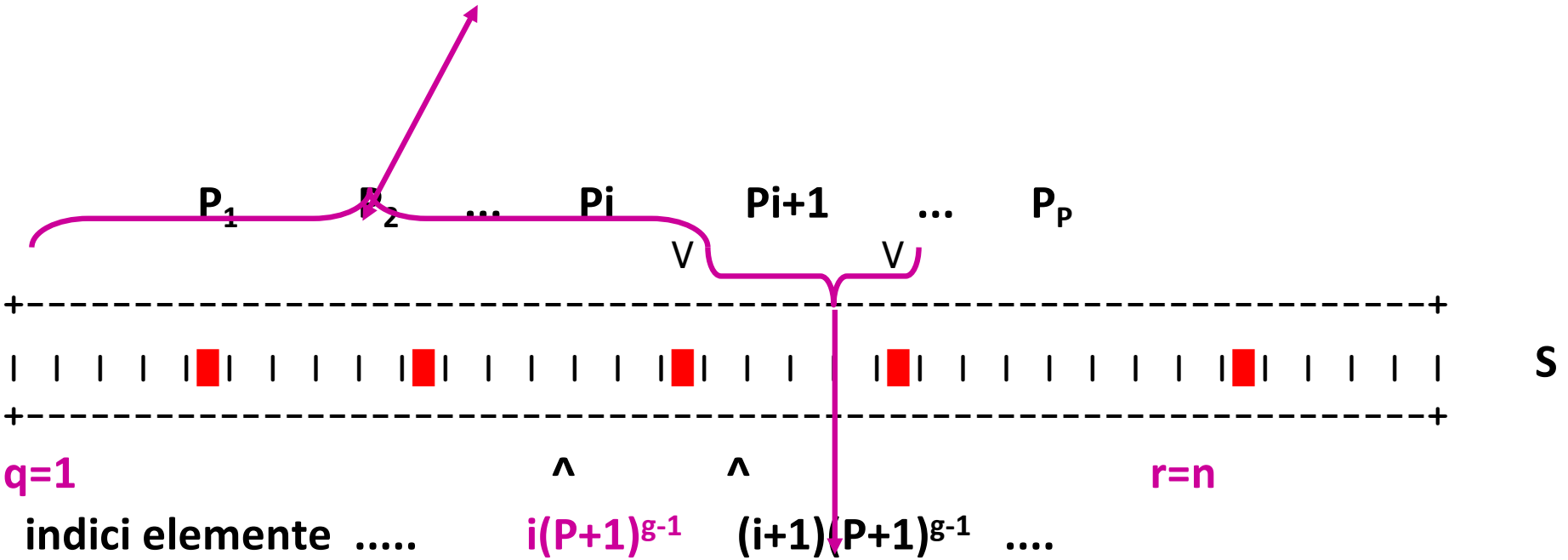
c[0]→ c[i] ← c[N+1]

process P[i = 1 to P]{

/3/ gaseste pozitia k unde se afla valoarea x

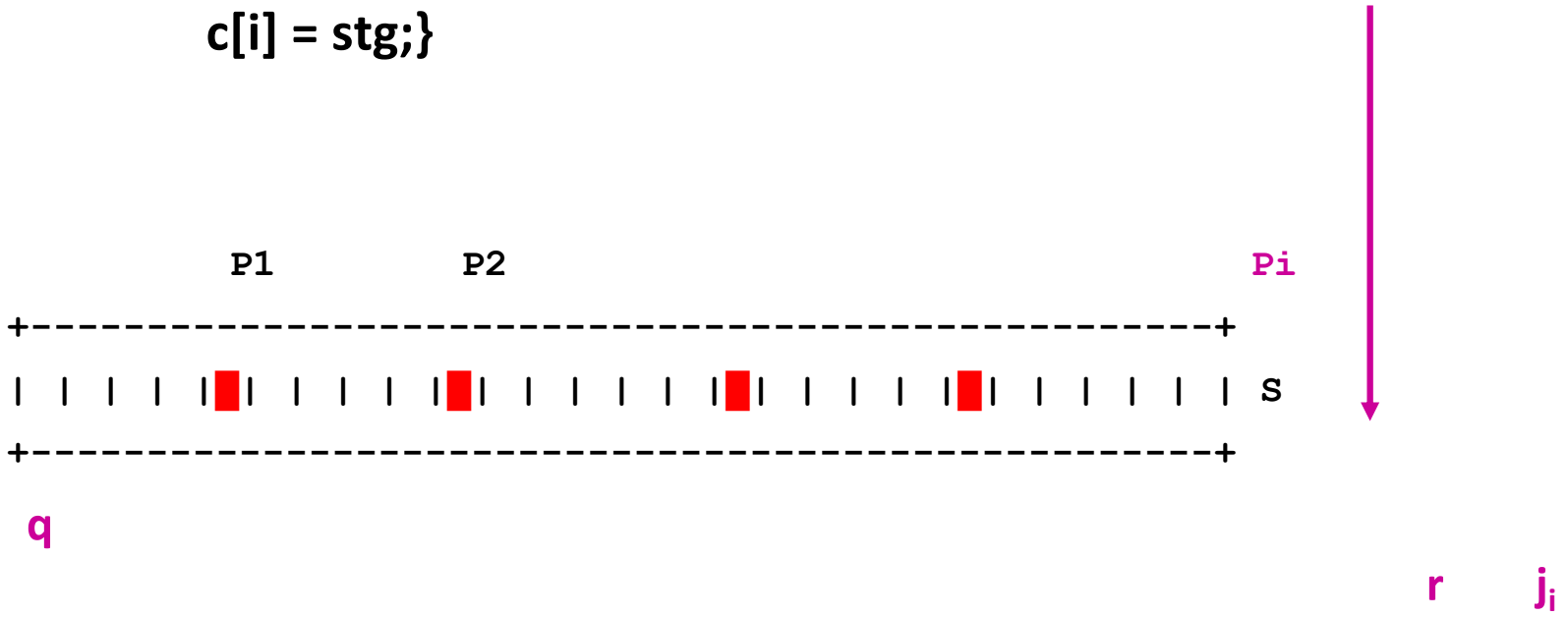
while (q<=r and k=0){ /* subsir exista si valoare negasita */

j[i] = (q-1) + $i*(P+1)^{g-1}$;



dimensiune subinterval $(P+1)^{g-1}-1$

```
while (q<=r and k=0){
    j[i] = (q-1) + i*(P+1)^(g-1)
/* Pi compara x cu s[j[i]] si determina partea de secv acceptata */
    if (j[i] <= r)                    /* pozitie in interior sir */
       if (s[j[i]] == x){ k = j[i];          /*pozitie gasita*/
       else if (s[j[i]] > x) c[i] = stg;
       else c[i] = dr;
    else {j[i] = r+1;                 /*j[i] indica o pozitie afara din sir*/
          c[i] = stg;}
```



```
/* calculeaza indicii subsecventei urmatoare */
/* doar un proces poate face modificarea intervalului de cautare */
        if (c[i] <> c[i-1]){
                r = j[i] – 1;
             if (i>1) q = j[i-1] + 1;
        }
/* doar procesul P executa operatia urmatoare*/
        if (i==P and c[i]<>c[i+1]) q = j[i] +1;
        g := g-1;
   }                    /*end while*/
}        /*end process*/
```

Observaţii

- Complexitatea

$$t(\mathrm{n}) = \mathrm{O}(\log_{P+1}(n+1)) \rightarrow$$ algoritmul este *timp* optimal

- Costul

$$O(P\ \log_{P+1}(n+1)) \rightarrow$$ algoritmul nu este *cost* optimal

- Notă
  - cerinţa este ca toate elementele secvenţei S să fie distincte

- În cazul în care șirul conține valori egale
  - Menţinerea eficienţei algoritmului → model CRCW.

Proprietati (dorite) ale algoritmilor paraleli

Număr de procesoare

- p(n) < n - uzual p(n) este funcţie sublineară de n
- p(n) adaptiv - algoritmul sa nu depindă de un număr fix de procesoare

Timp de execuţie

- t(n) mic - semnificativ mai mic dacat al variantei secvenţiale
- t(n) adaptiv - invers proportional cu p(n)

Cost

- c(n) sa fie minim, deci algoritmul sa fie cost-optimal

Reminder

Costul unui algoritm se definește ca produsul dintre numărul de procesoare folosite și timpul de rulare.
Pe o mașină secvențială, P=1, deci costul algoritmul este timpul (asimptotic) de execuție.

Un algoritm paralel cost optimal este unul pentru care costul este θ (timpul de execuție secvențial).
Dacă un algoritm paralel nu este cost optimal, el poate fi făcut cost optimal fie reducând timpul asimptotic de execuție, fie reduând numărul de procesoare.

Fie timpul de execuție al algoritmului secvențial pentru efectuarea unor task-uri de dimeniune n, $\theta(n)$, și timpul de execuție a versiunii paralele a acelui algoritm $\theta(\lg n)$, acesta folosind n procesoare, fiecare păstrând un element de date. Costul este așadar:

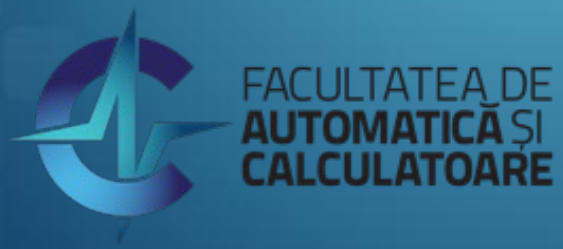
$$cost = n\theta(\lg n) = \theta(n \lg n)$$

Costul acesta nu este optimal. Pentru a obține un algoritm de cost optimal, putem fie reduce timpul de execuție la $\theta(1)$, fie reduce numărul de procesoare (la $n/\lg n$, astfel încât produsul $n/\lg n$ cu $\theta(\lg n)$ să dea $\theta(n)$).

Să presupunem acum că fiecare procesor deține $\lg n$ date. Dacă procesul secvențial este liniar cu numărul de elemente, atunci fiecare dintre cele $n/\lg n$ procesoare va procesa în paralel cele $\lg n$ date locale într-un timp de $\theta(\lg n)$. Cele $n/\lg n$ procesoare vor efectua algoritmul (non-cost optimal) anterior asupra celor $n/\lg n$ rezultate din pasul secvențial într-un timp de $lg(n/\lg n) = \lg n - lg(\lg n) = \theta(\lg\ n)$. Costul fiecărui pas este $\theta(\lg n)$, dar costul final este $(n/\lg n)\theta(\lg n) = \theta(n)$.

Arhitecturi Paralele
Abordări probleme paralele log(N)

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



Sume prefix

Aplicații folosind paralelismul de date
Calculul sumelor prefix

Problemă:

Se dă tabloul **a[1:n]**, se cere **s[1:n]**, unde:

$$s[i] = \sum_{k=1}^{i} a[k]$$

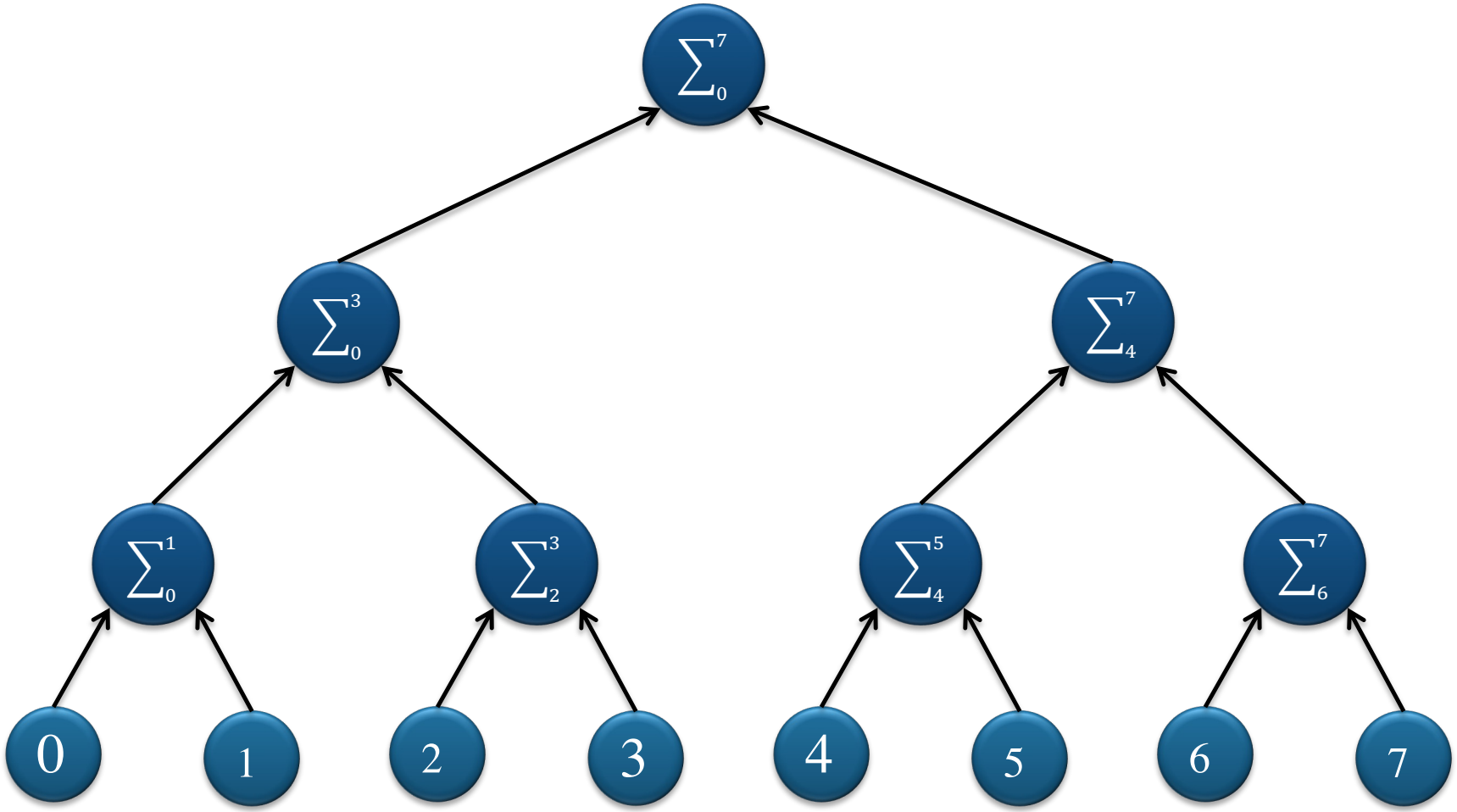
- Algoritm secvențial:

```
s[1] = a[1];
for [i = 2 to n]
    s[i] = a[i] + s[i-1];
```

- Algoritm paralel:

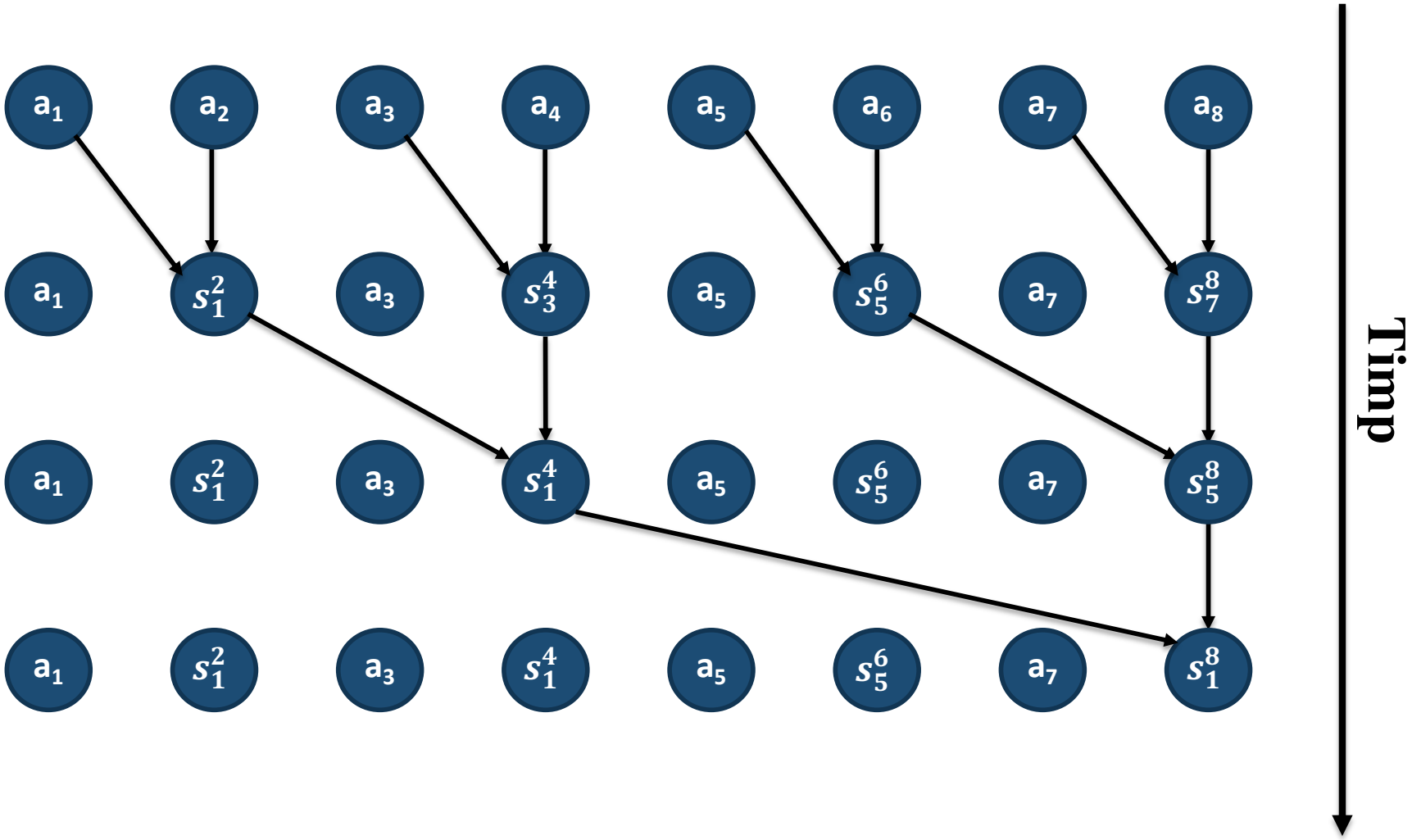
  - derivat din algoritmul ***sumei elementelor unui vector***

Suma elementelor unui vector



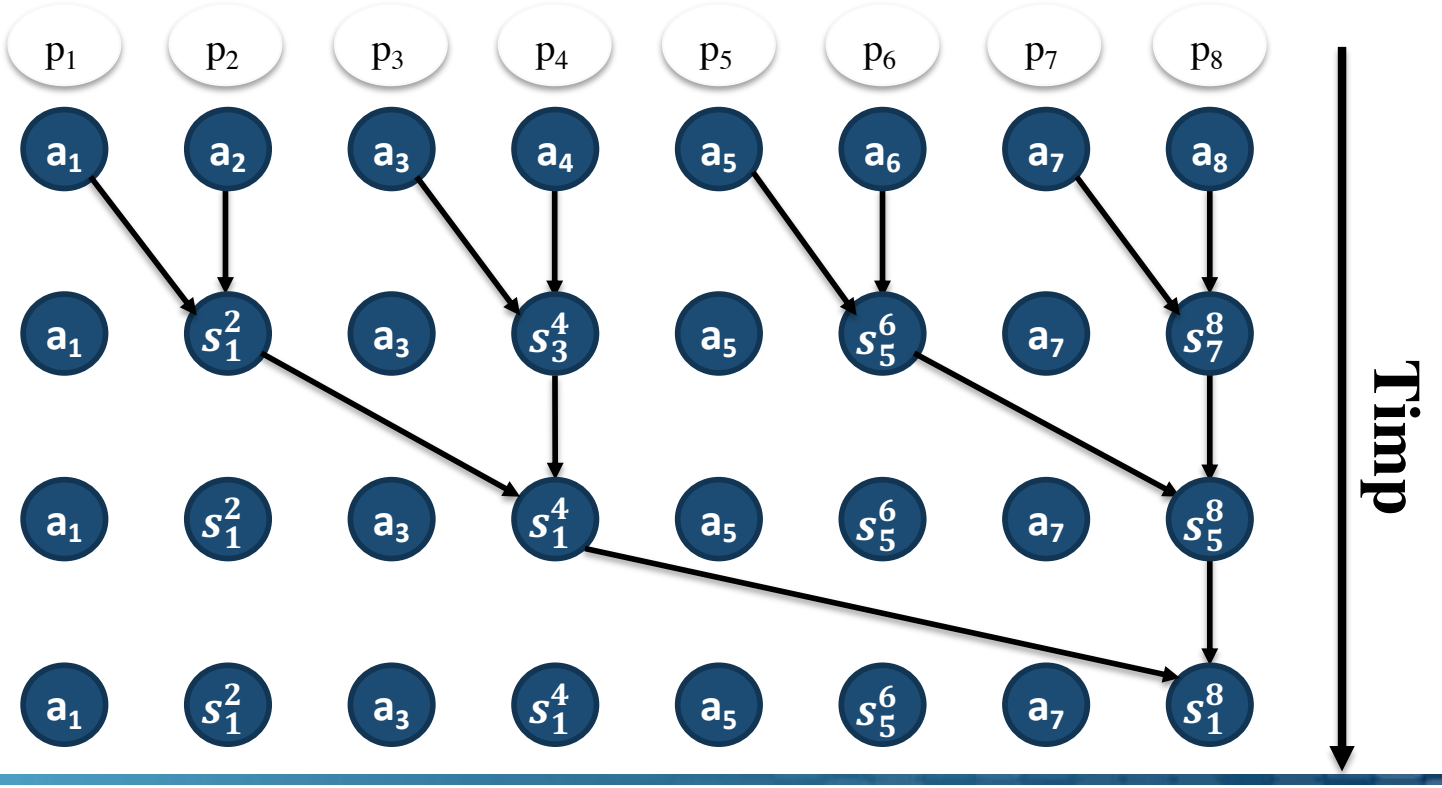
$\sup\left(\frac{n}{2}\right)$ procesoare, $\sup(\log_2 n)$ pași

Suma elementelor unui vector

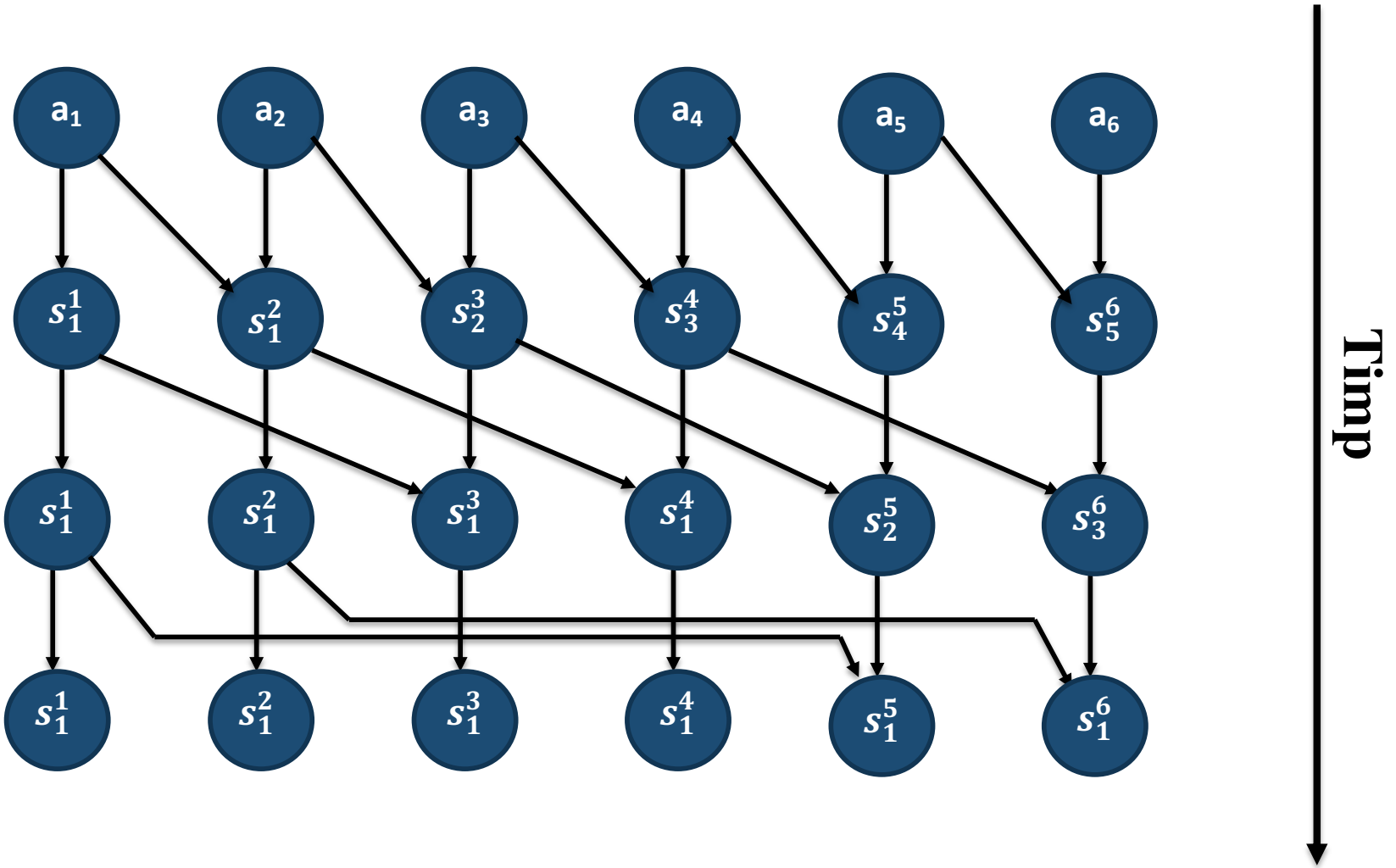


Suma elementelor unui vector

```
int a[1:n];
process suma[k=1 to n] { // suma proceselor p din figur
      for [j = 1 to sup(log2 n)] {
            if (k mod 2^j == 0)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```



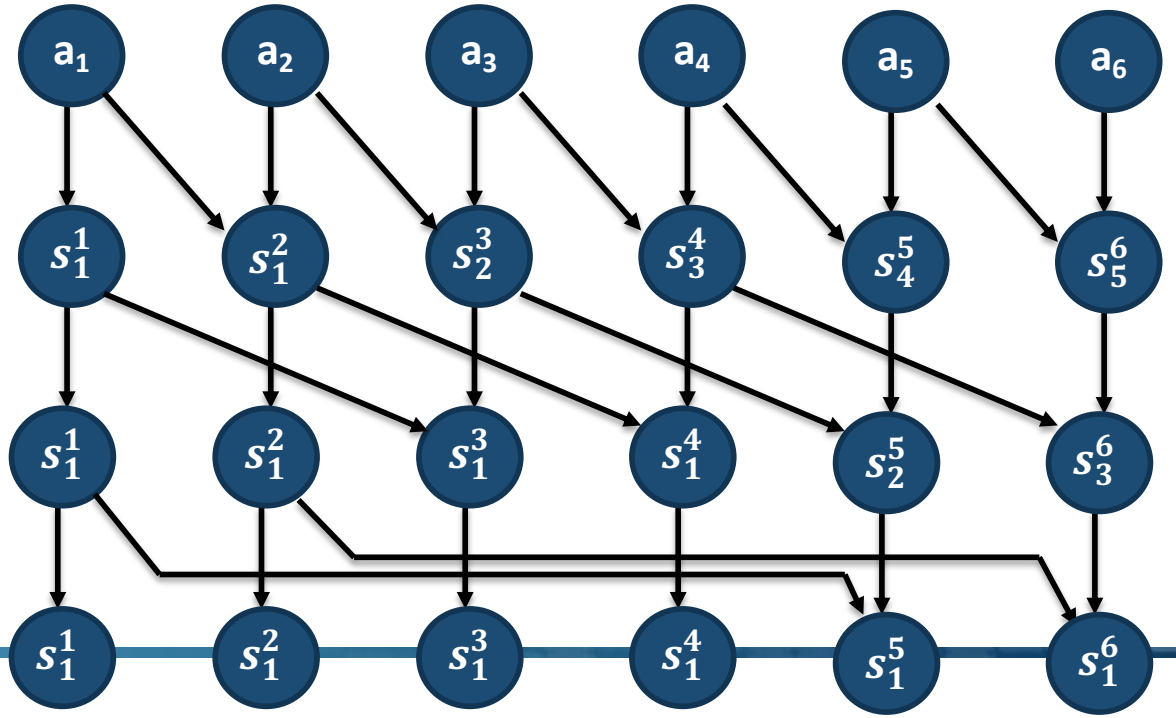
Sume prefix (varianta 1)



Sume prefix – varianta 1

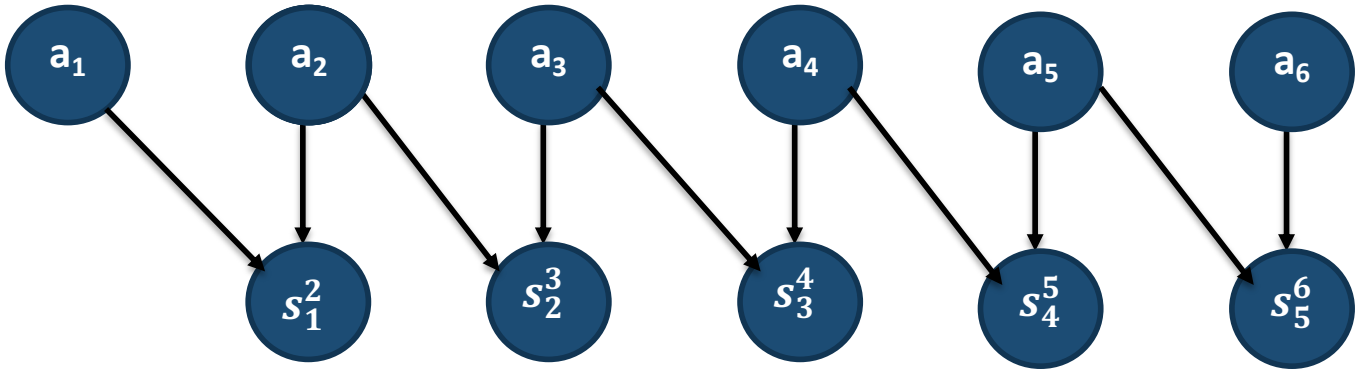
```
int a[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            if (k - 2^(j-1) >= 1)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```

Eroare de sincronizare...



Sume prefix – varianta 1 – probleme

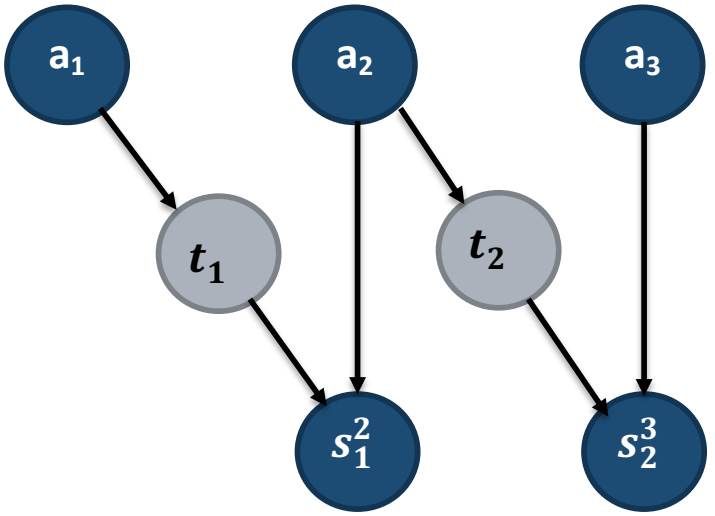
- Presupunem că procesorul numărul 3 este mai lent decât restul.
  - Suprascriere a locației de memorie



$$s_2^3 = a_3 + s_1^2$$

Sume prefix – varianta 2

```
int a[1:n], temp[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            temp[k] = a[k];
            barrier;
            if (k - 2^(j-1) >= 1)
                  a[k] = temp[k-2^(j-1)] + a[k];
            barrier;
      }
}
```



Sume prefix – varianta 3

```
int a[1:n], temp[1:n];
process suma[k:1 to n] {
    int d = 1;
    while (d < n) {
      temp[k] = a[k];
      barrier;
      if (k - d >= 1) a[k] = temp[k-d] + a[k];
      barrier;
      d = 2 * d;
    }
}
```

Parallel Scan

Parallel Scan

scan (o) < x1, ..., xn >

==

< x1, x1 o x2, ..., x1 o ... xn >

Exemplu: Filtru

Considerați la **intrare** un vector, trebuie găsit la ieșire un alt **vector** care să conțină doar elementele pentru care **(f elt)** este **true**

Fie f x = x > 10

filter f < 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >
= < 17, 11, 13, 19, 24 >

Paralelizabil?

- Găsirea elementelor este o operație ușoară
- Dar punerea lor în secvența corectă pare mai dificilă

Prefixe paralele - to the rescue

1. Folosim mapare paralelă pentru a calcula un bit-vector pentru elementele ce se potrivesc operației

   input: < 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >
   bits: < 1, 0, 0, 0, 1, 0, 1, 1, 0, 1 >

2. Aplicăm sume prefix pe elementele bit-vectorului

   bitsum < 1, 1, 1, 1, 2, 2, 3, 4, 4, 5 >

3. Aplicăm parallel map pentru a produce ieșirea

   output < 17, 11, 13, 19, 24 >

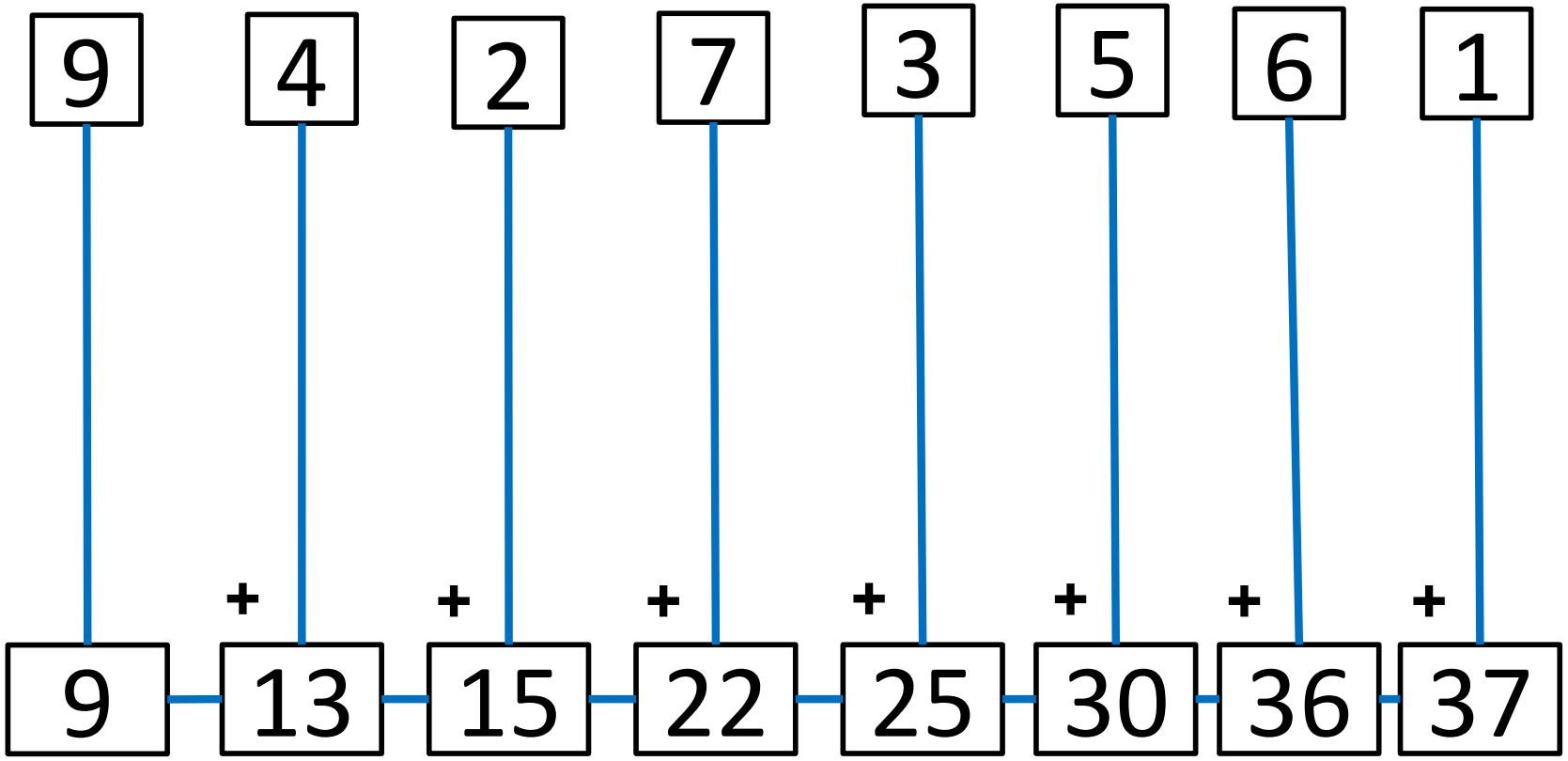
Generic: Scan

Se aplică aceeași operație între elementele unui vector.
Se colectează toate rezultatele parțiale

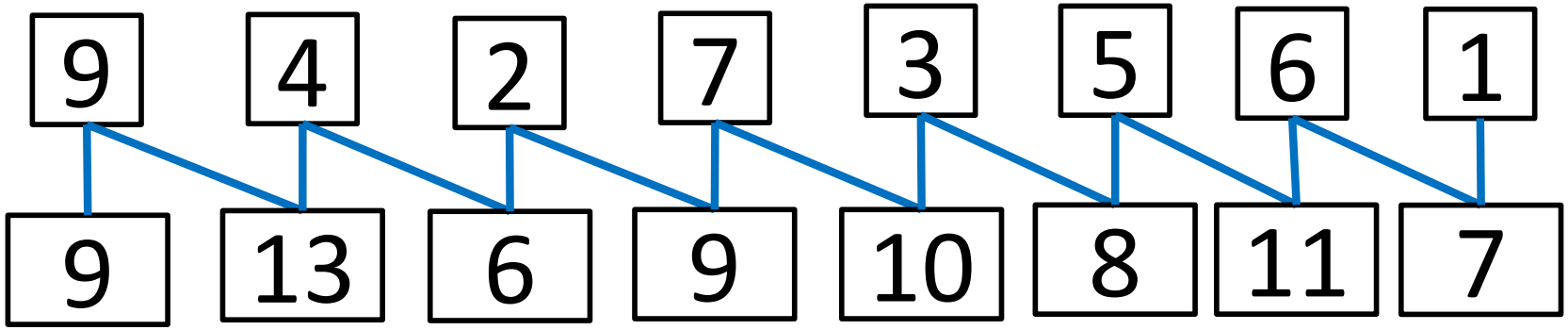
Poate fi executat în log(n) pași folosind un arbore

Operația poate lua orice formă (+, *, min, max, and, etc.)

Scan - sum

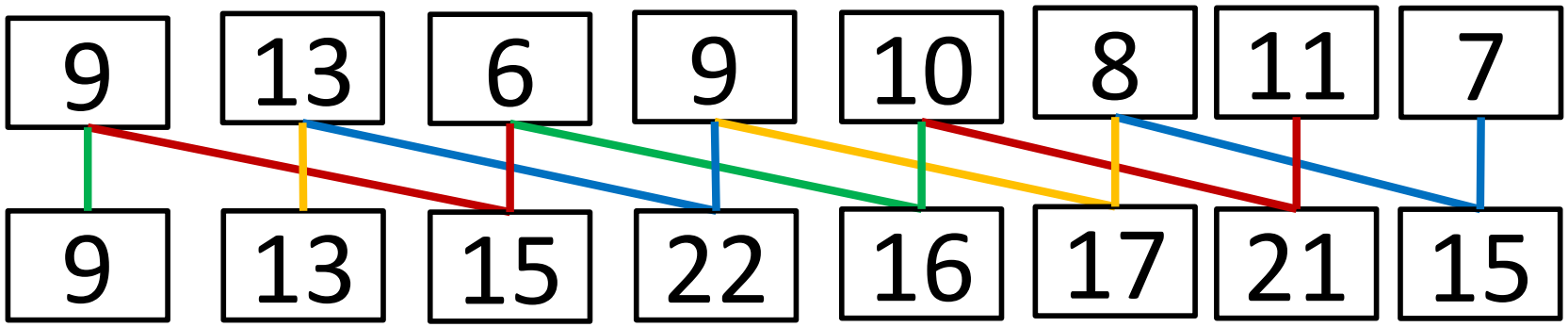


Scan - sum



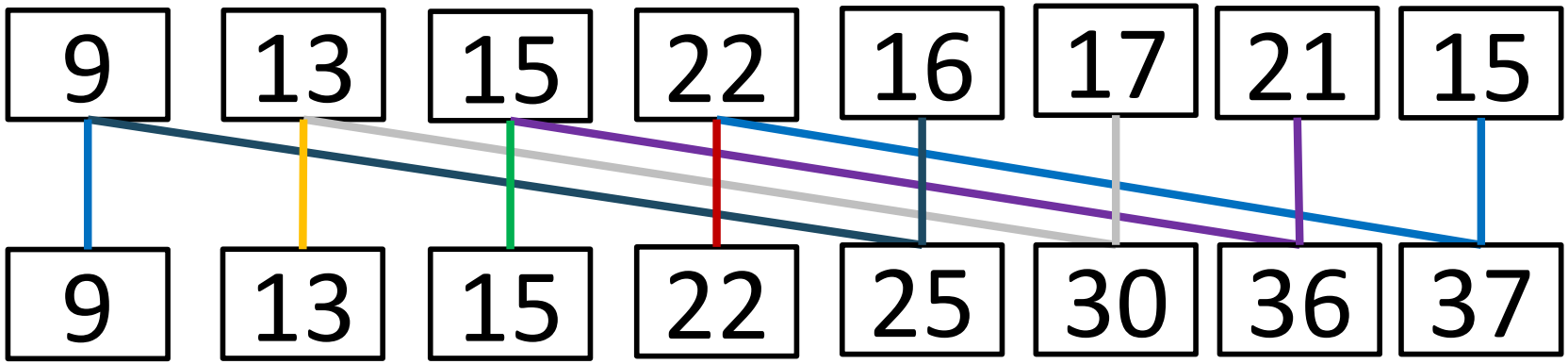
Pot fi executate în paralel

Scan - sum



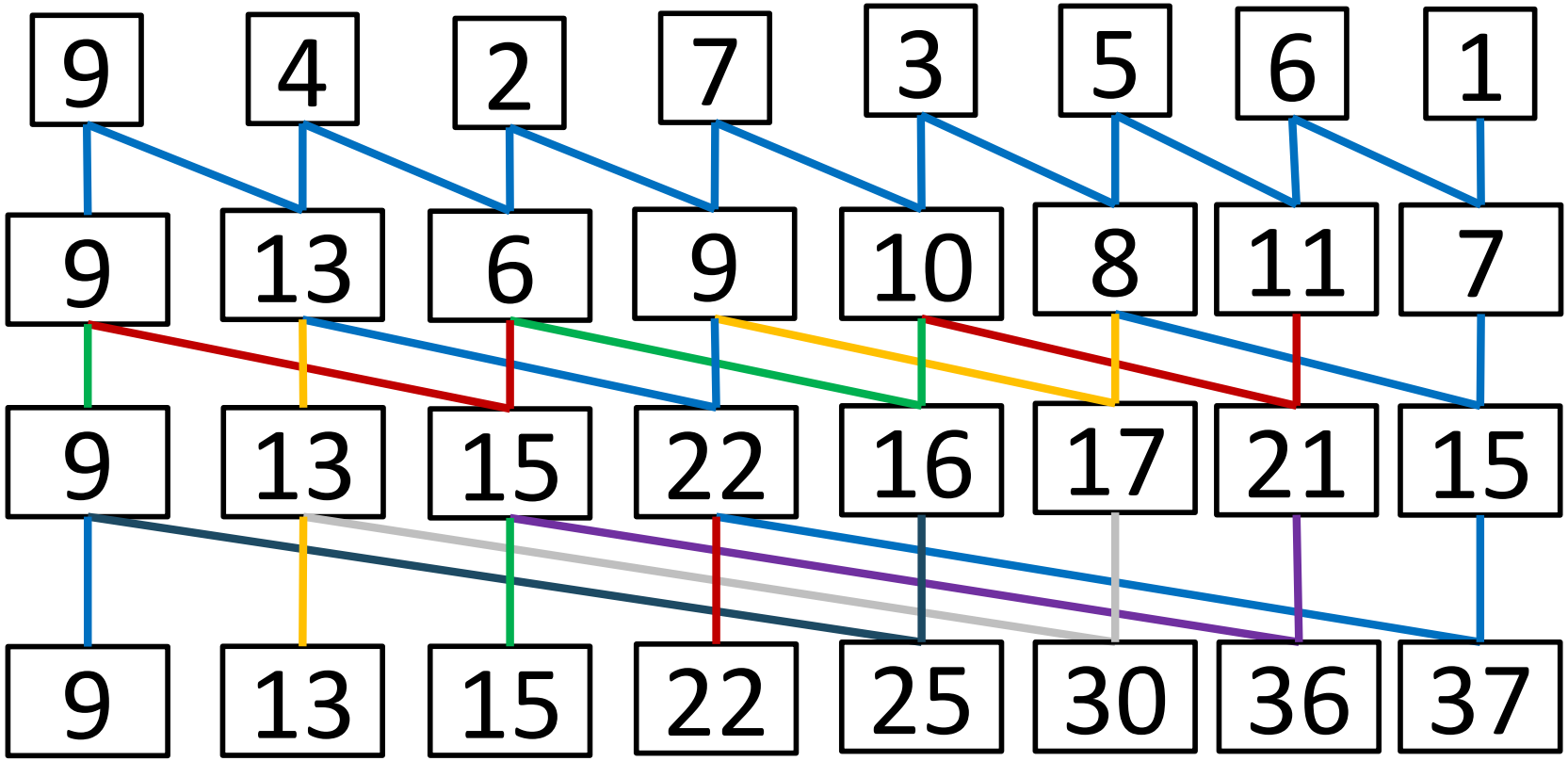
Pot fi toate executate în paralel

Scan - sum

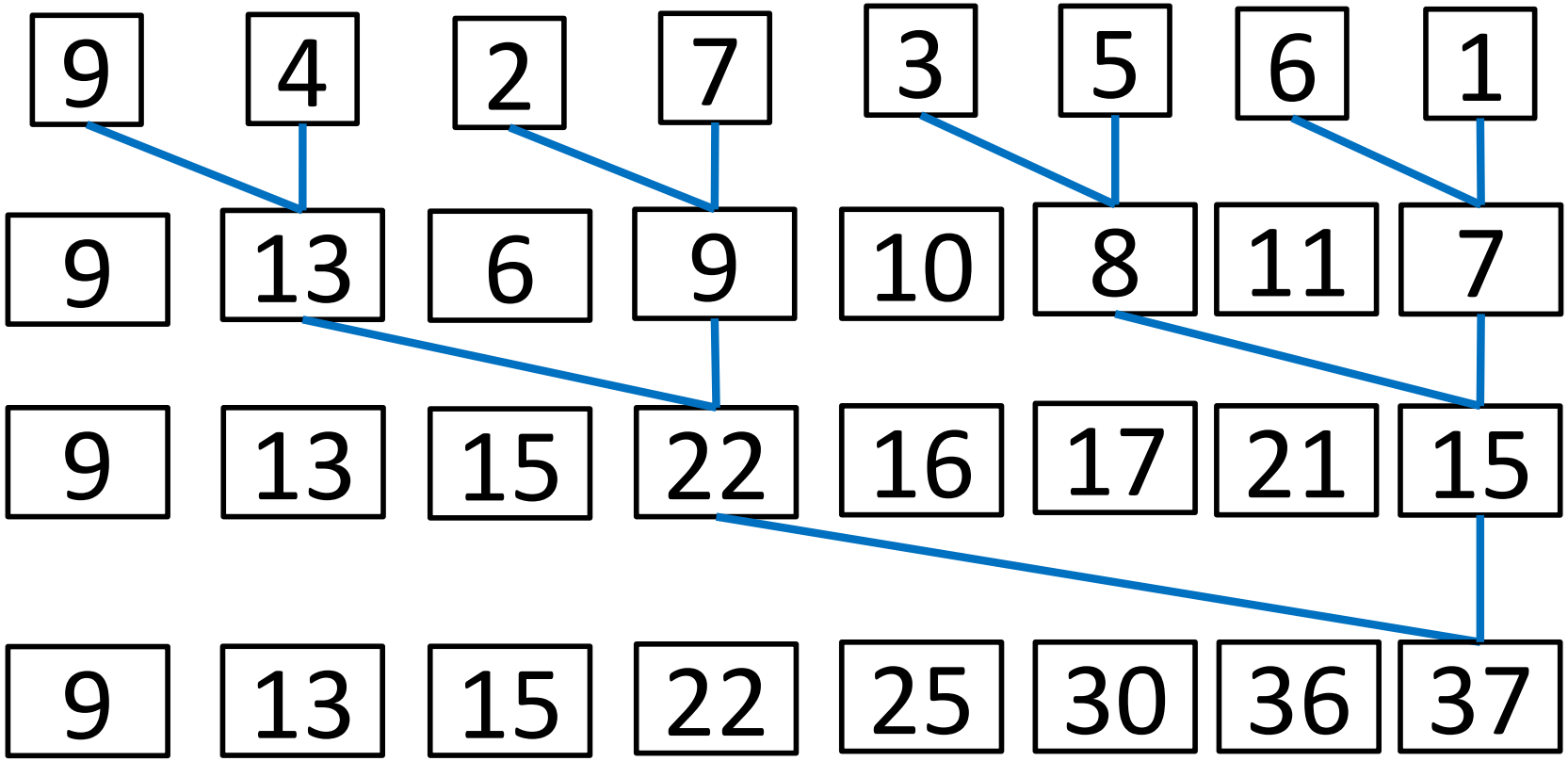


Pot fi toate executate în paralel

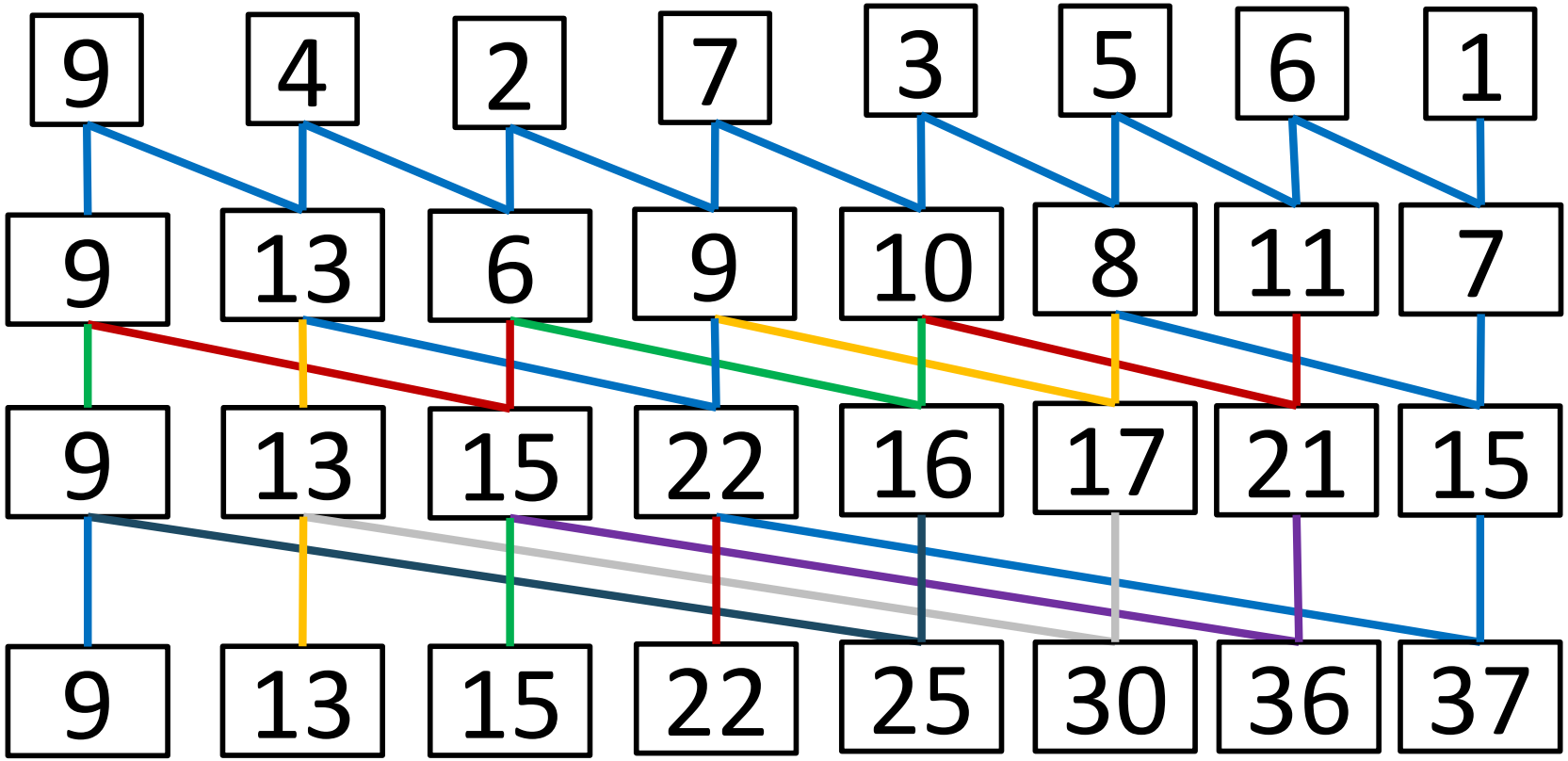
Scan – Cum funcționează?



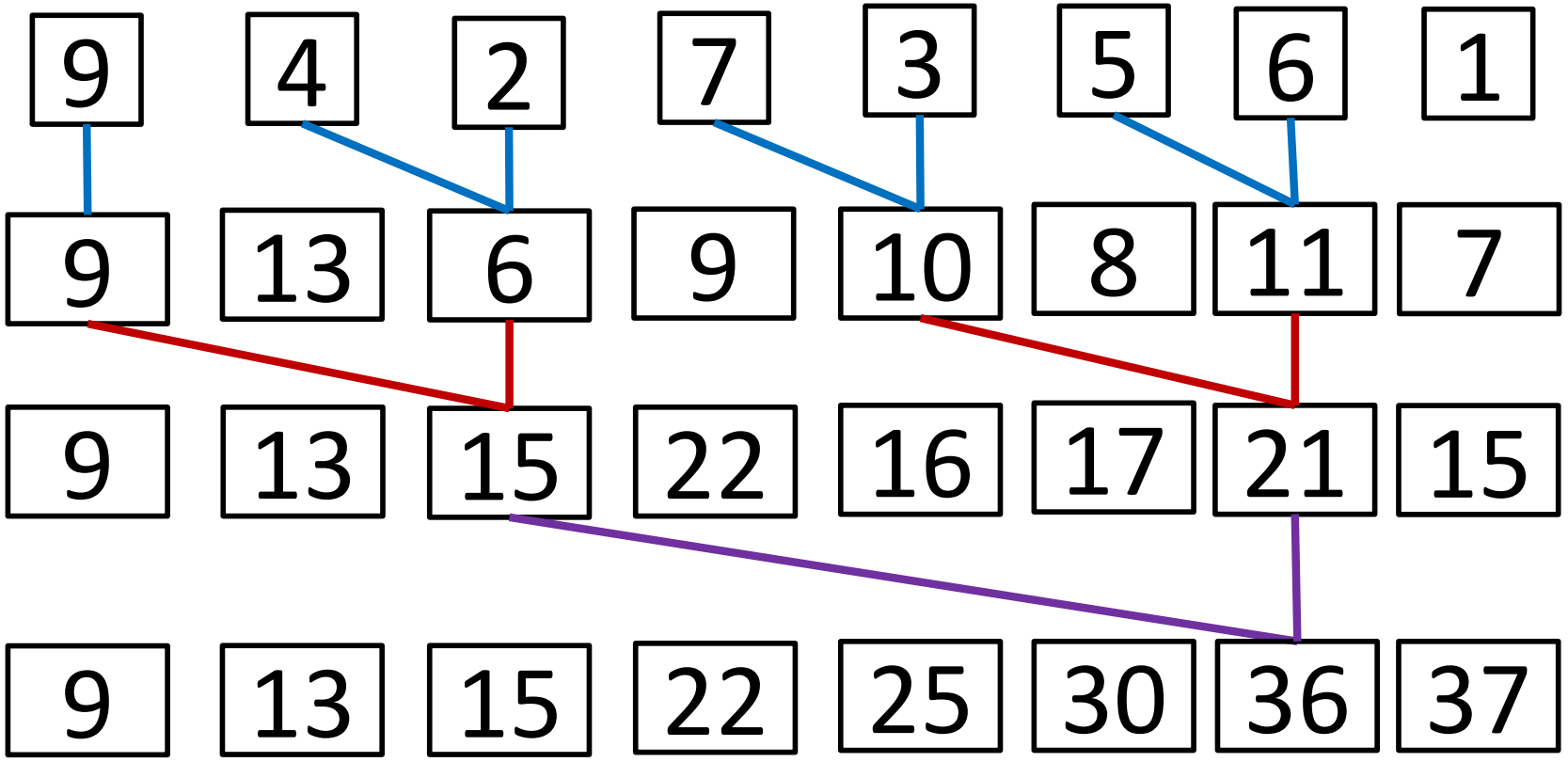
Scan – Cum funcționează



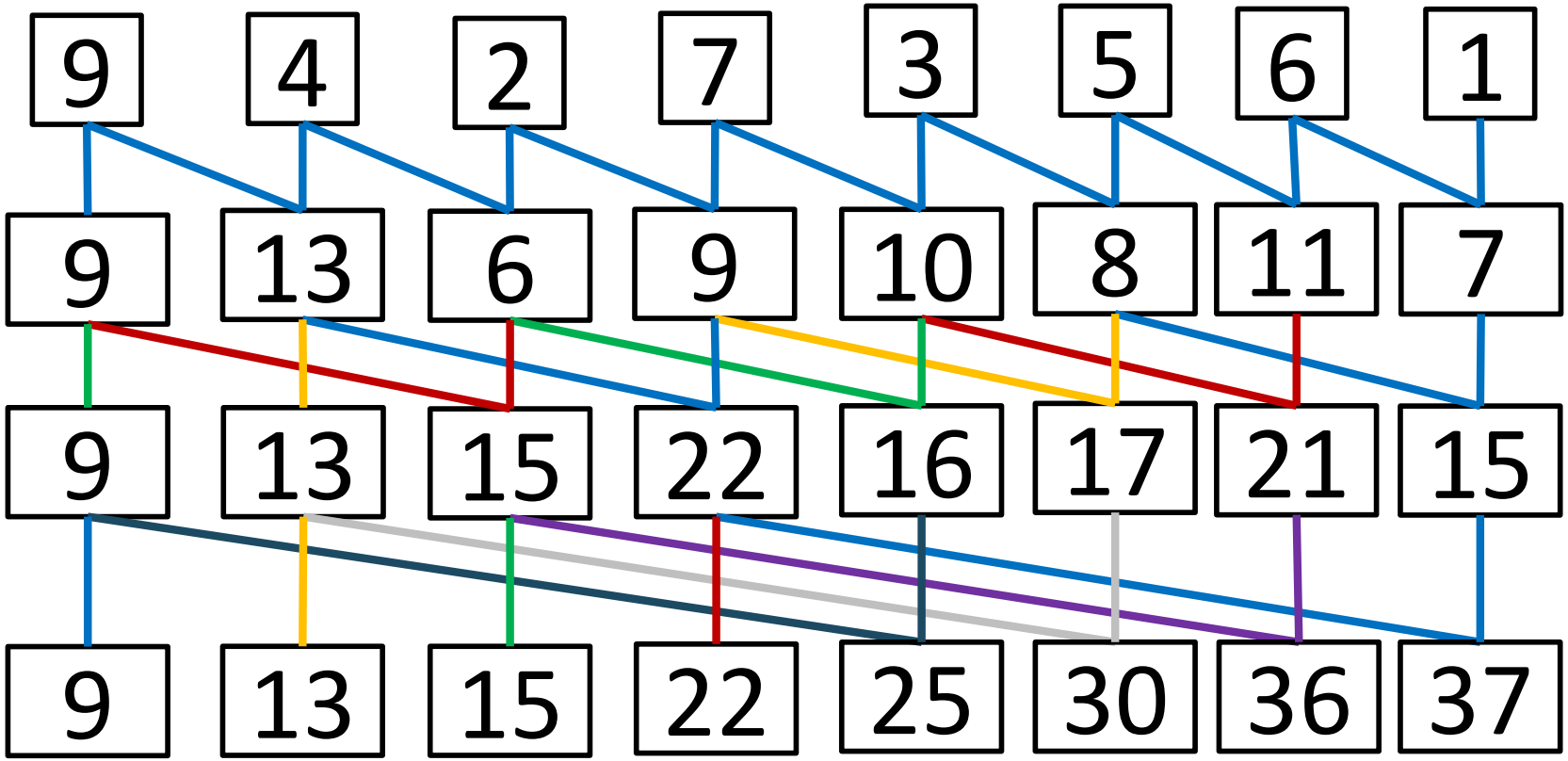
Scan – Cum funcționează?



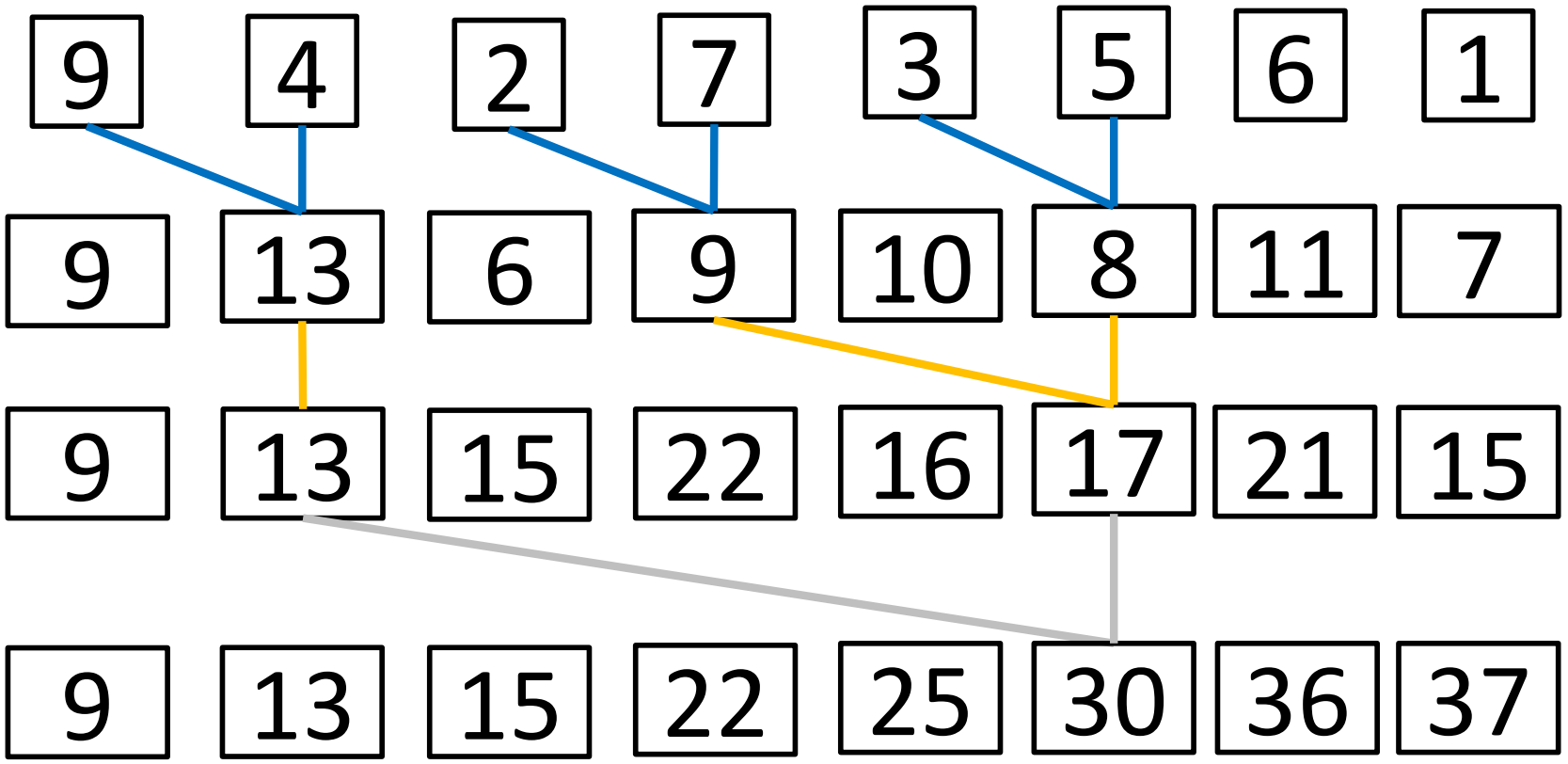
Scan – Cum funcționează?



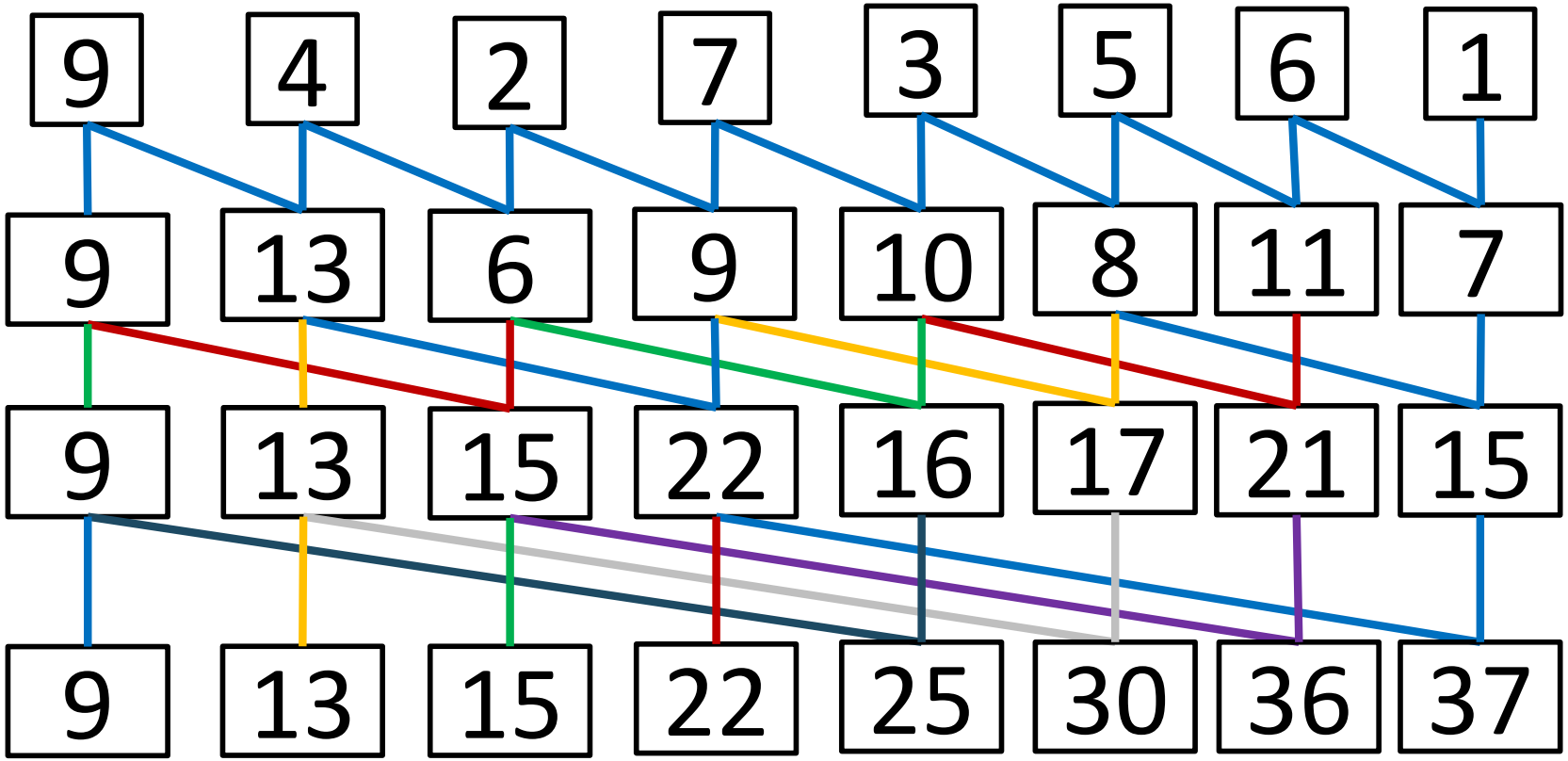
Scan – How it works?



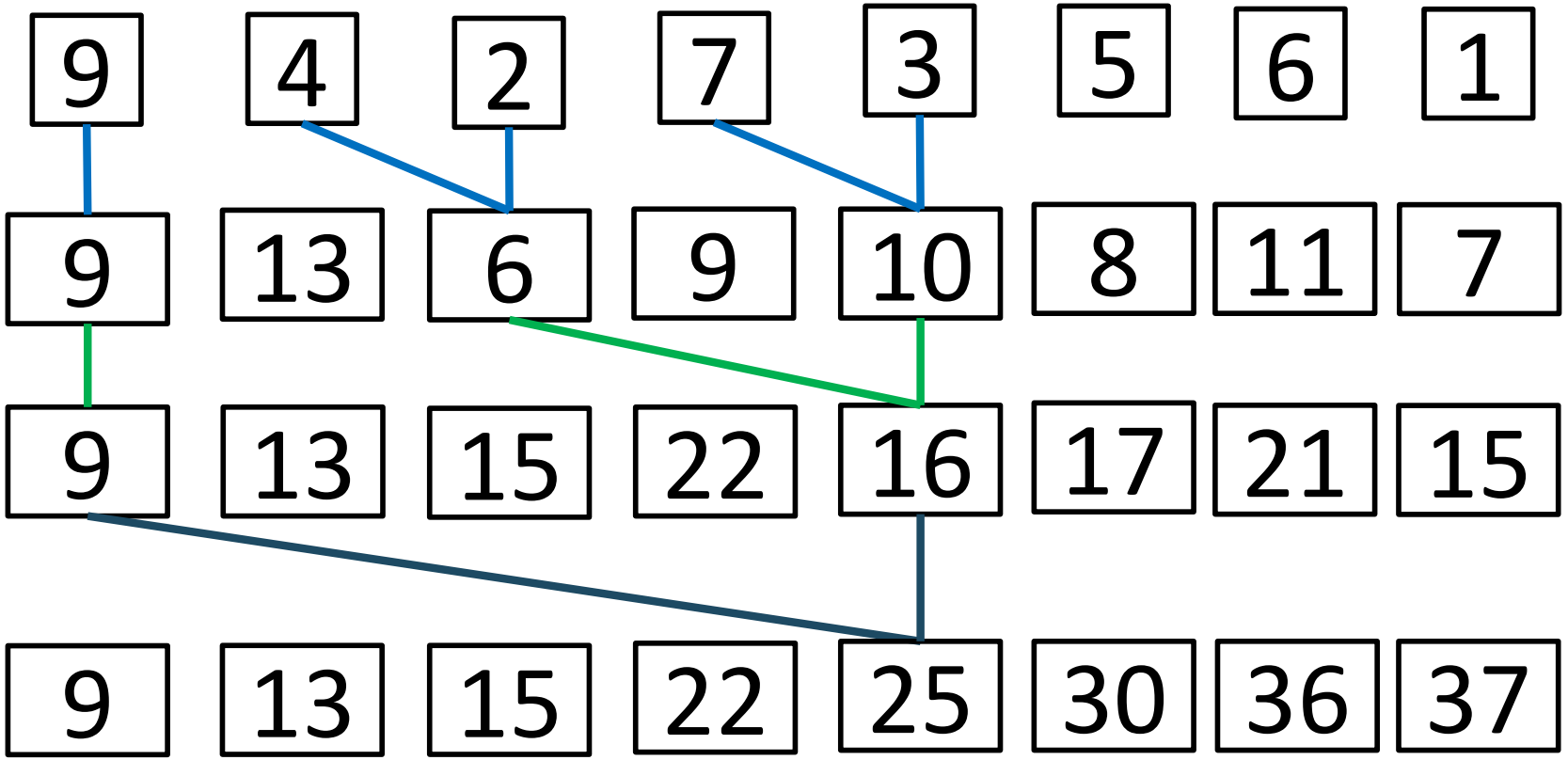
Scan – How it works?



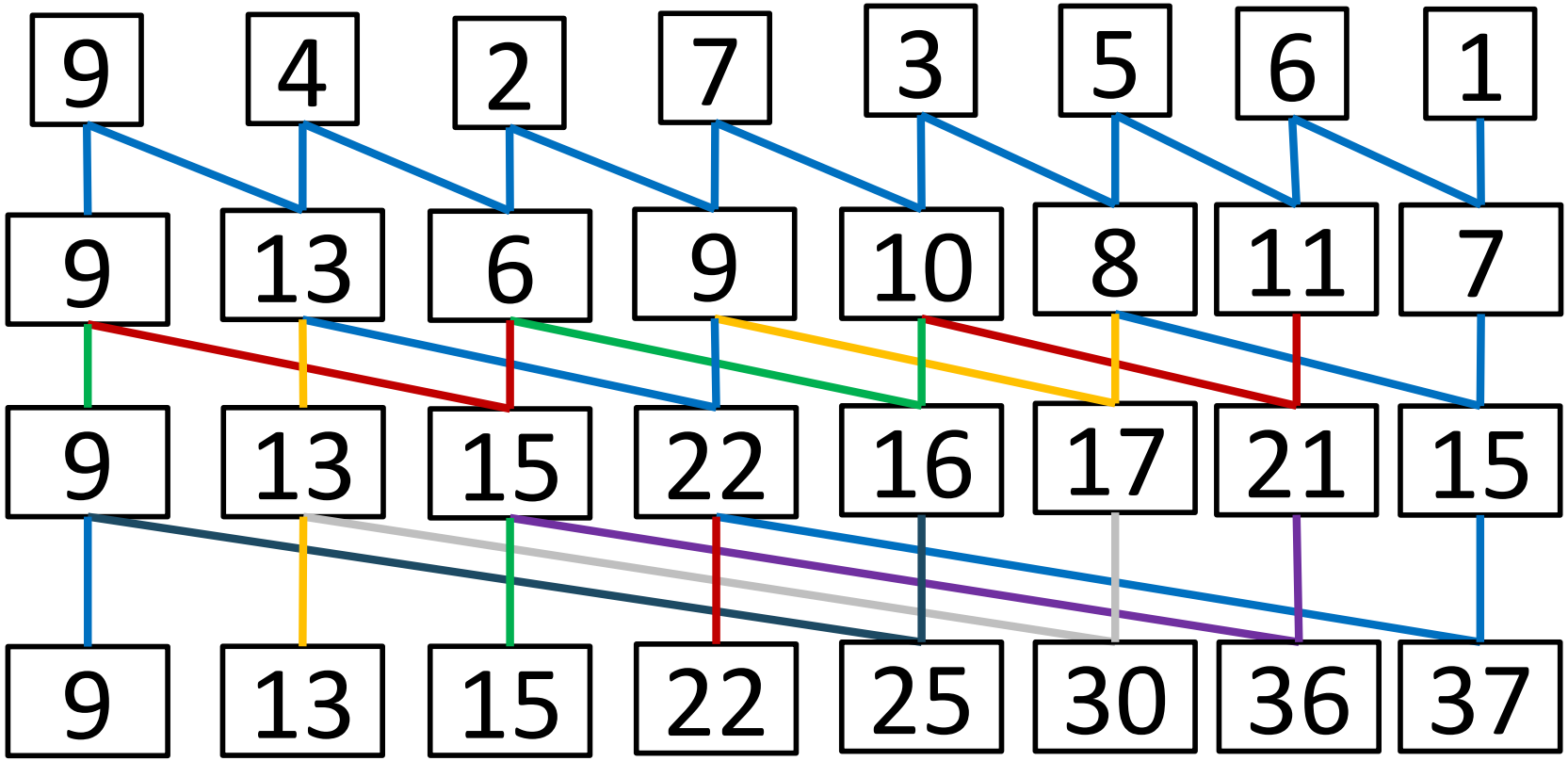
Scan – How it works?



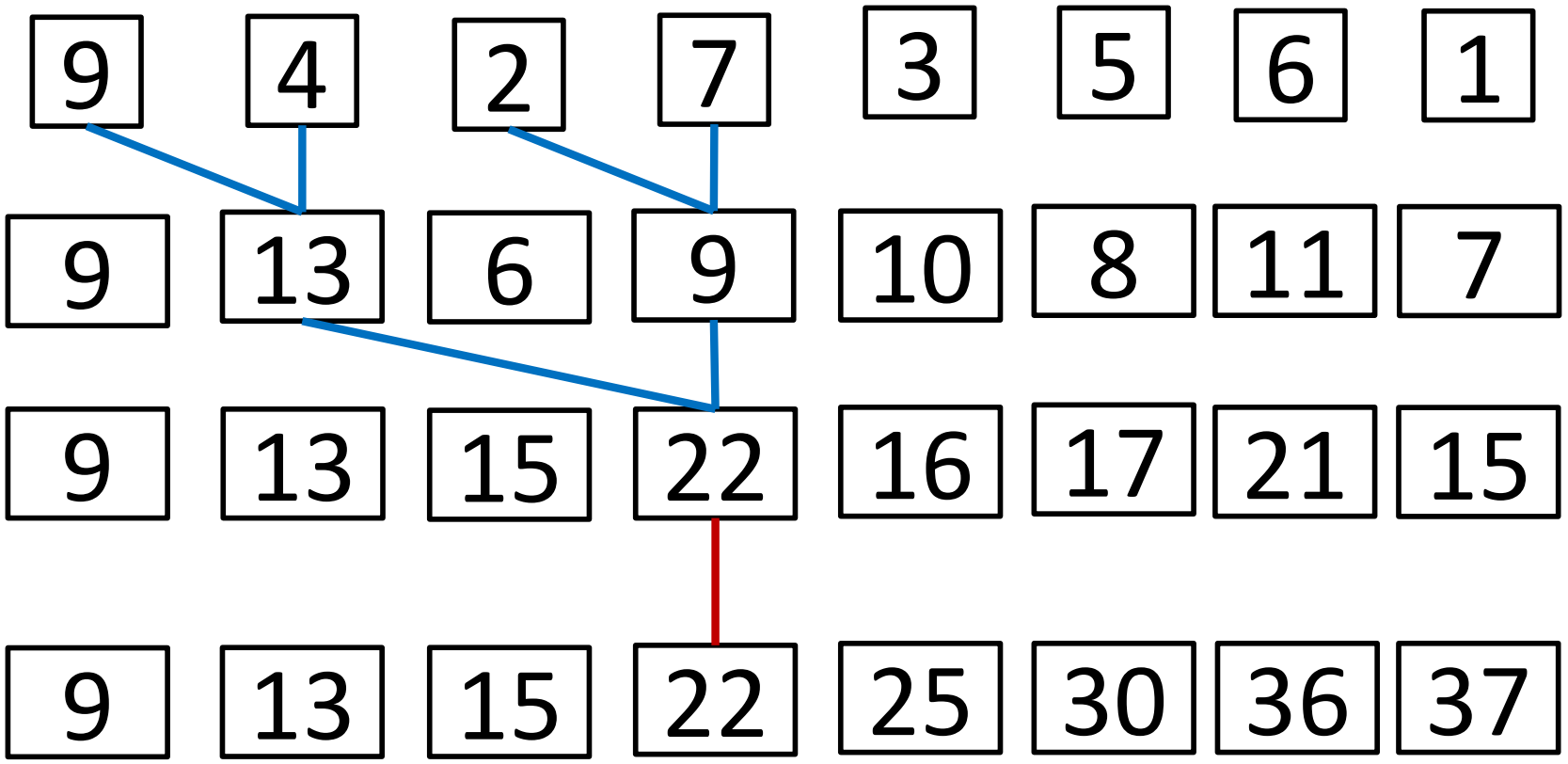
Scan – How it works?



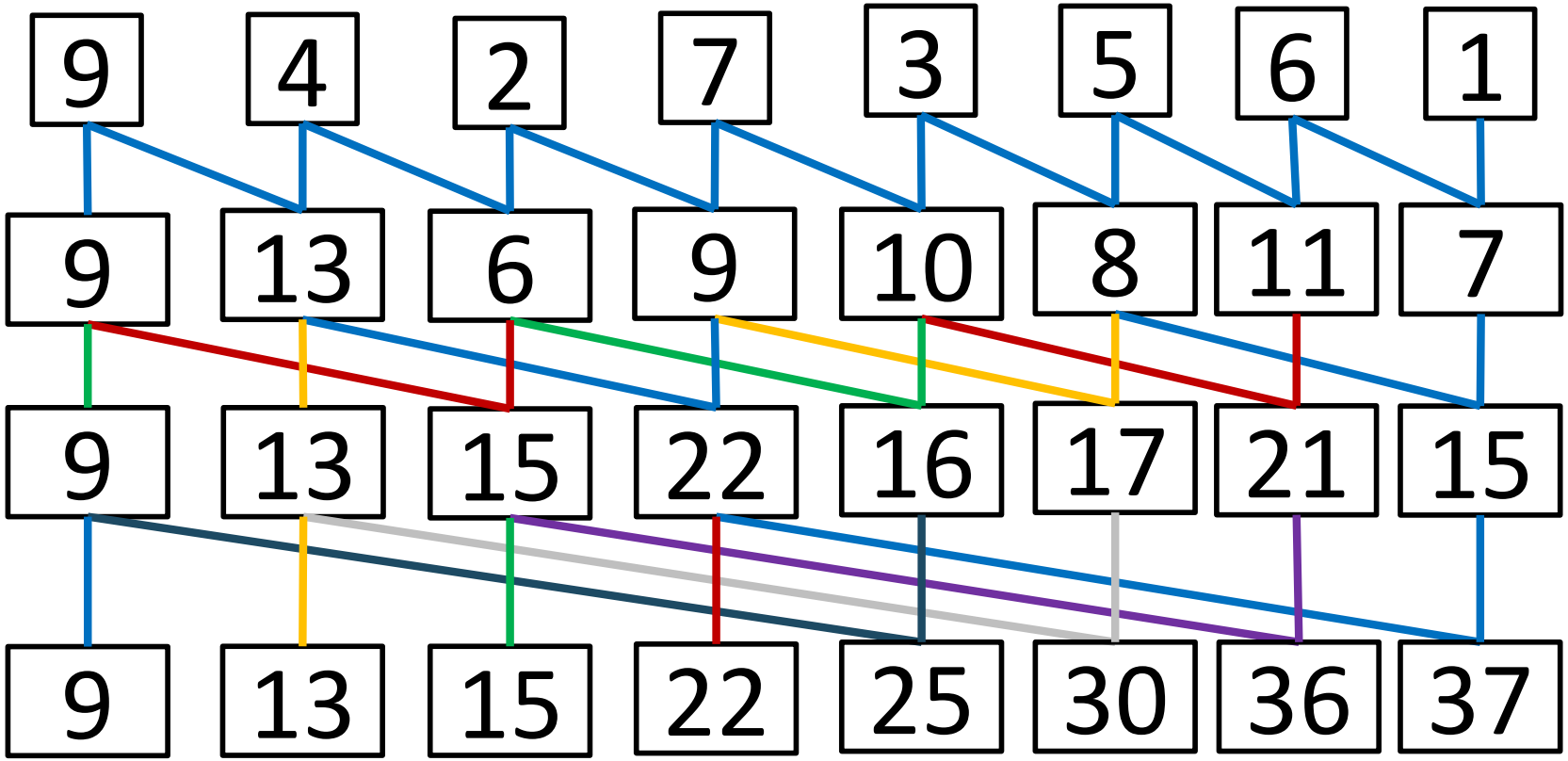
Scan – How it works?



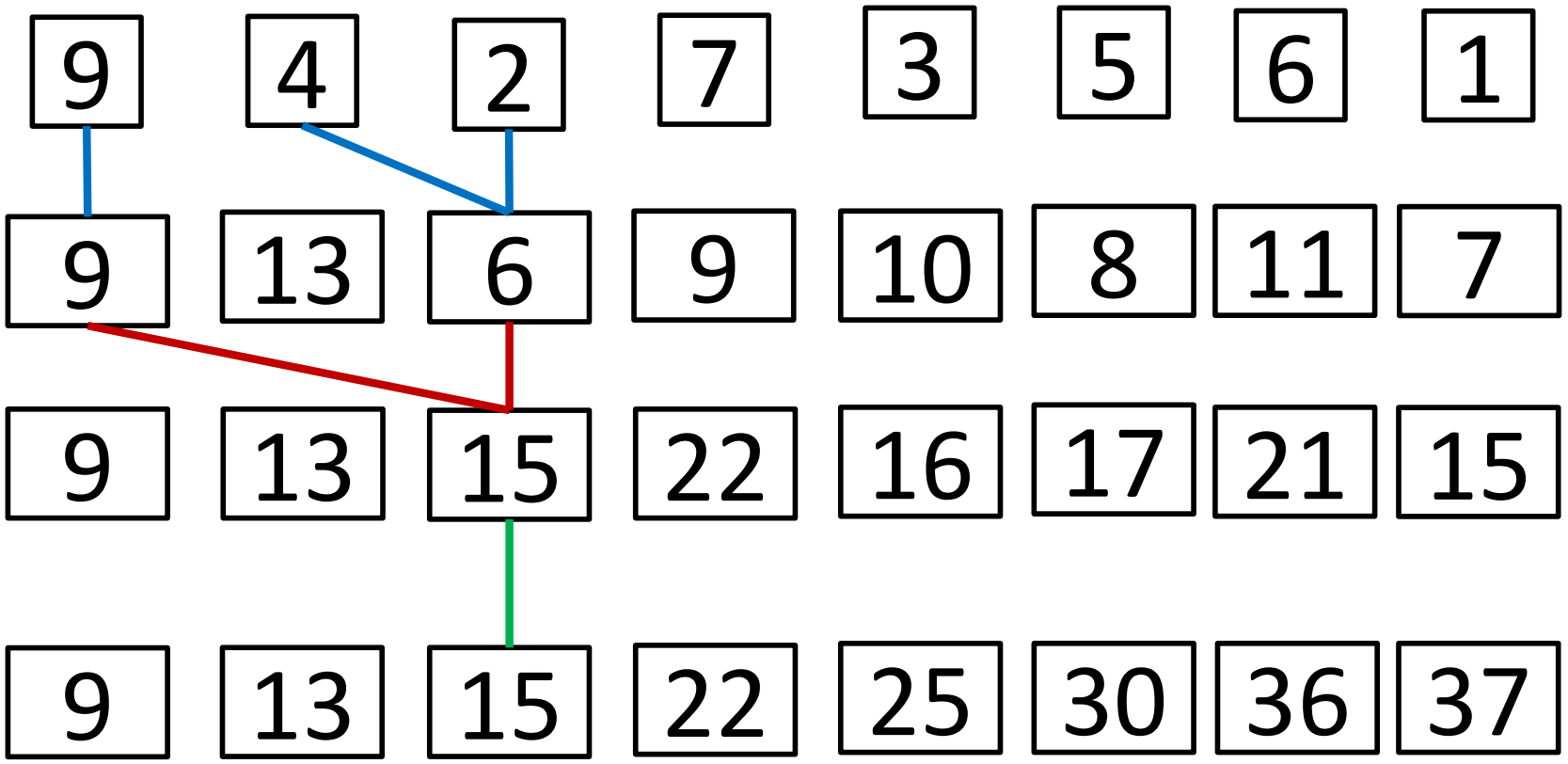
Scan – How it works?



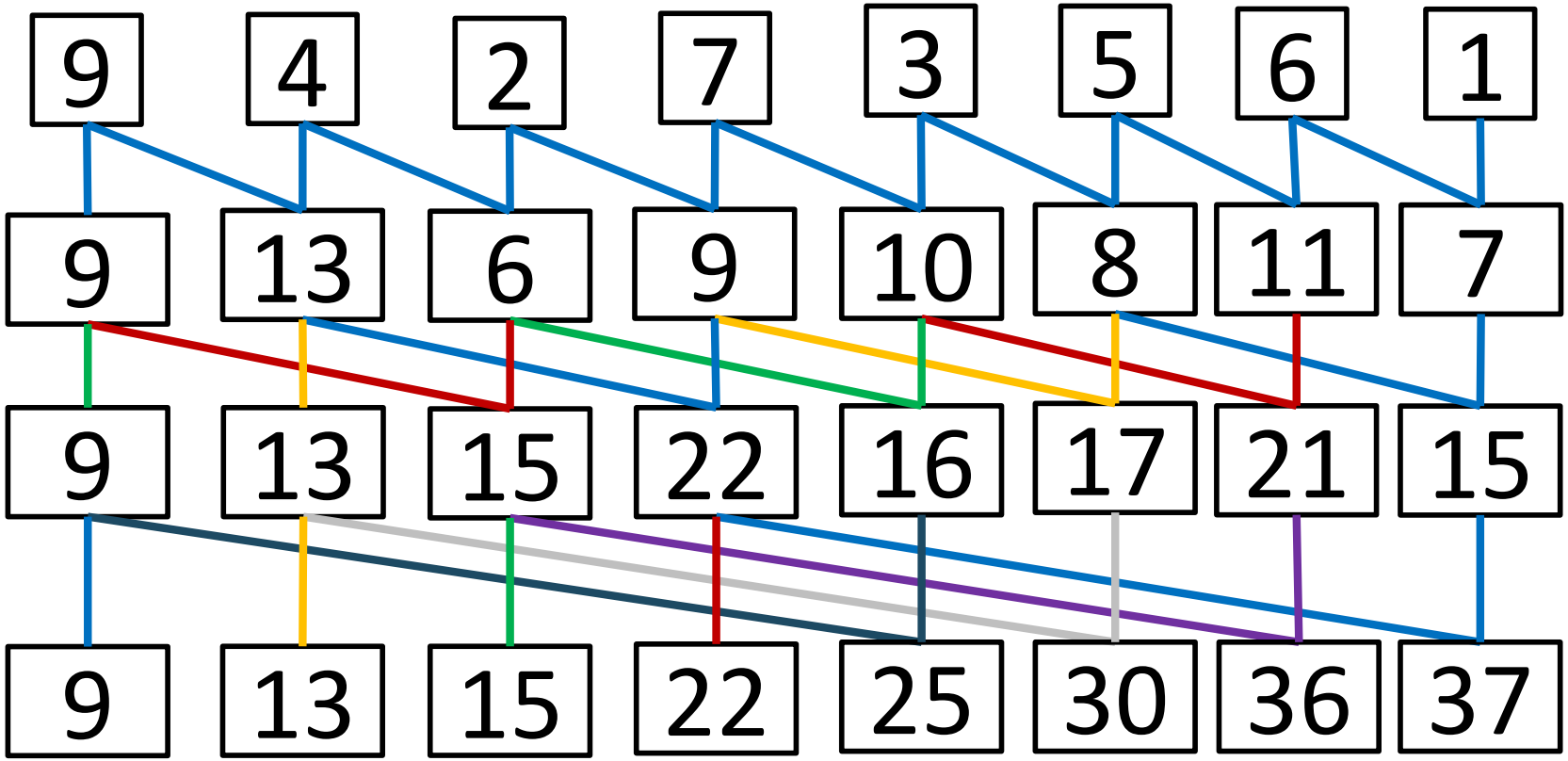
Scan – How it works?



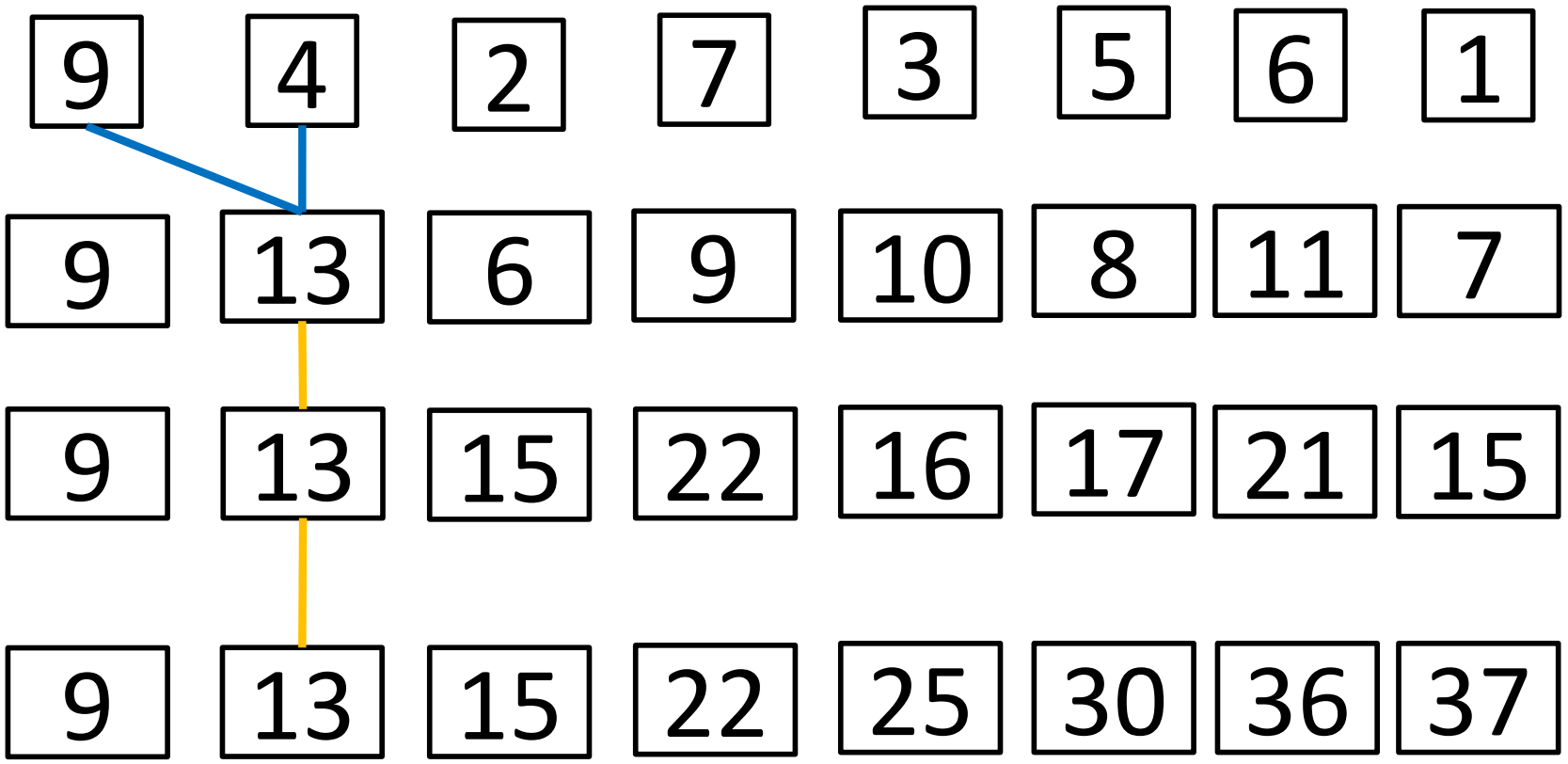
Scan – How it works?



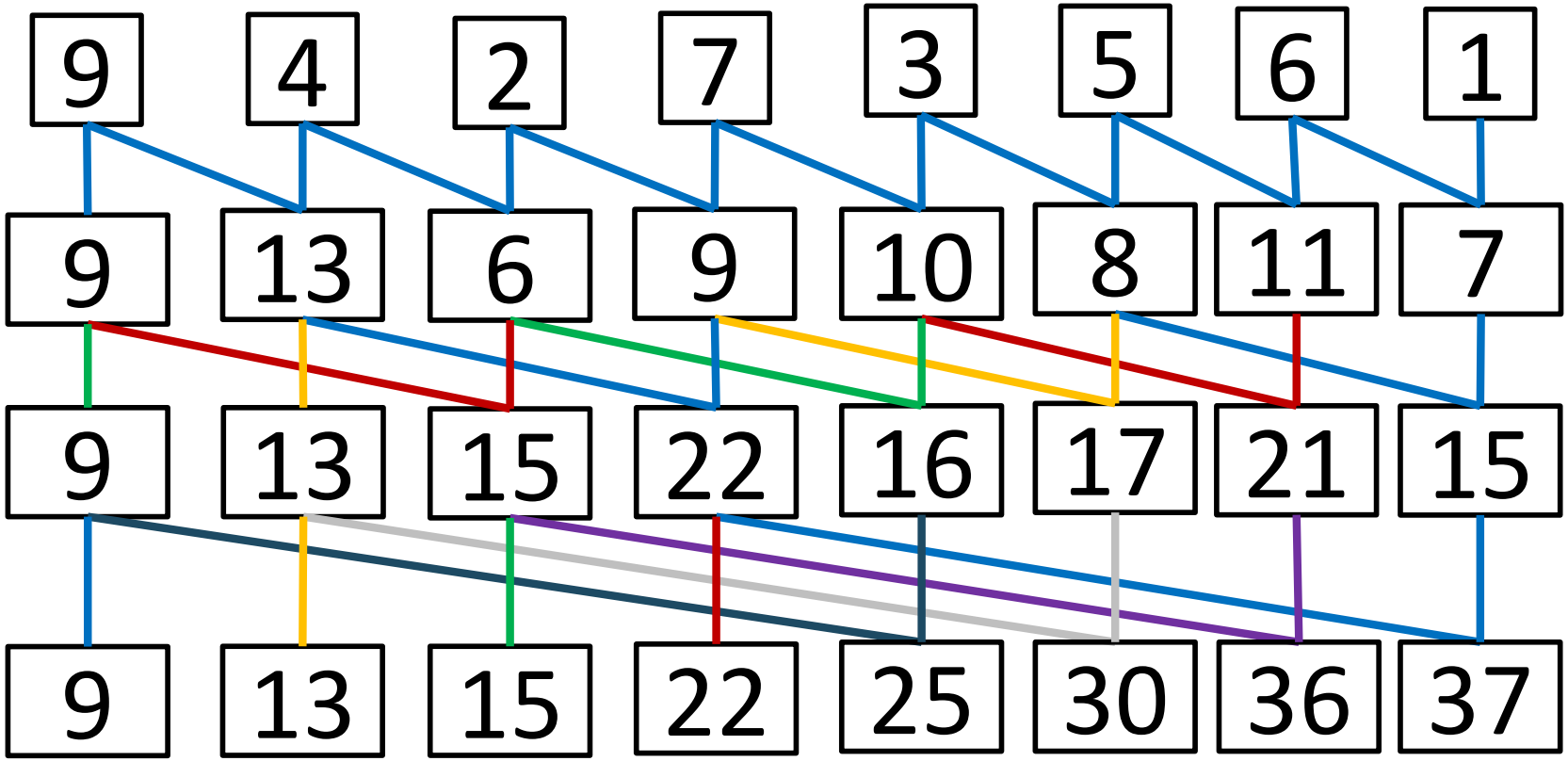
Scan – How it works?



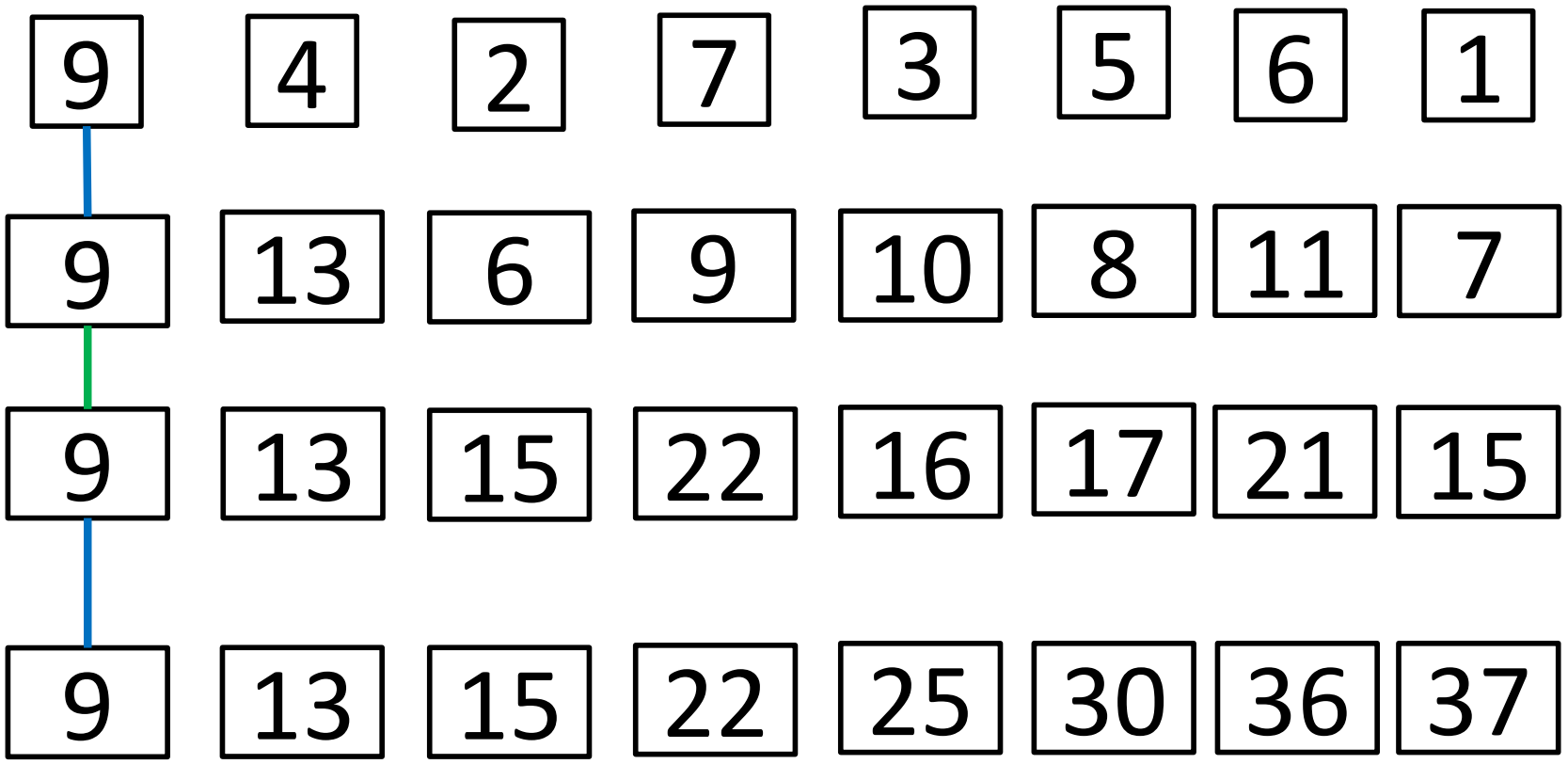
Scan – How it works?



Scan – How it works?



Scan – How it works?



Difuzarea unei valori

Difuzarea unei valori – justificare O(log P)

- D – celula din memoria comună ce trebuie difuzată
- Folosim un tablou A[1:N]

procedure BROADCAST (D, N, A)

Pas 1: Procesorul P1

1.1. citește valoarea din D

1.2. o memorează în propria memorie

1.3. o scrie în A[1]

Pas 2:

fa i = 0 to (log N -1) ->

fa j = 2^i+1 to $2^{(i+1)}$ do in parallel

Procesor Pj

2.1. citește valoarea din A[j-2^i]

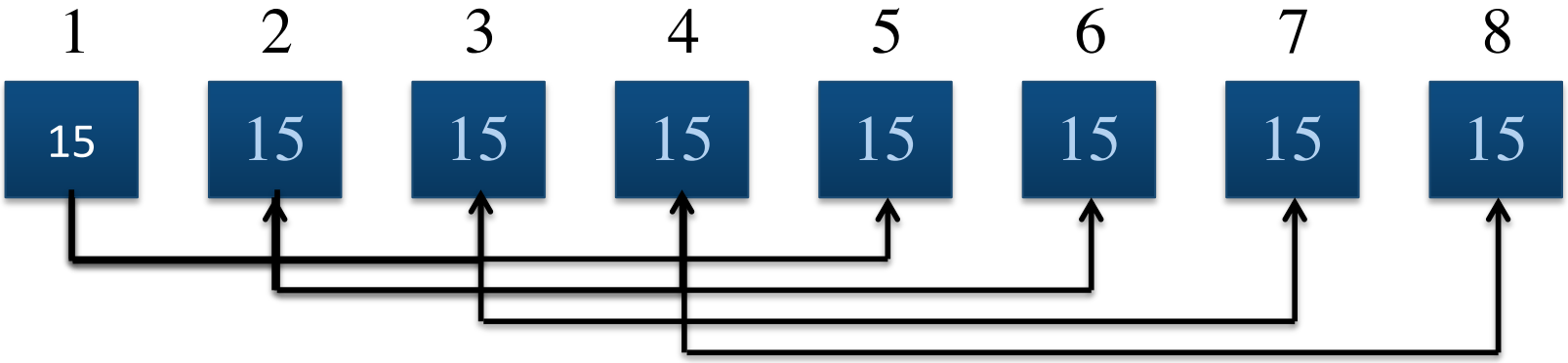
2.2. o memorează în propria memorie

2.3. o scrie în A[j]

af

af

Difuzarea unei valori



Operații cu vectori – broadcast

Pas 1: (Procesorul P_1)

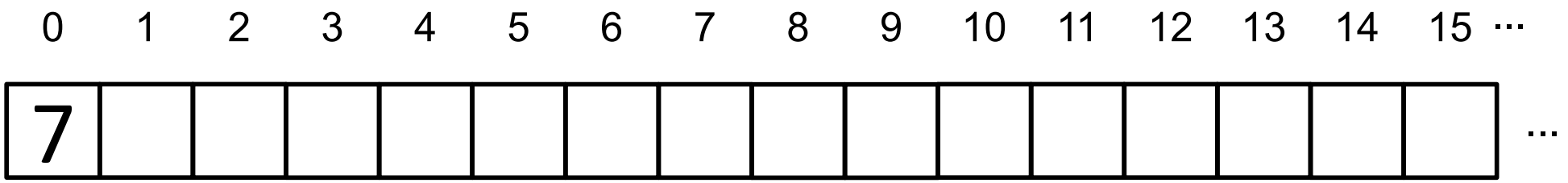
```
int t;/* t locala P1 */
t = D;
A[1] = t;
```

Pas 2:

```
for [i = 0 to (log N -1)] {
    process Pj[j = 2^i+1 to 2^i+1]{ /*(Procesor Pj)*/
        int t; /* t locală Pj */
        t = A[j-2^i];
        A[j] = t;
    }
}
```

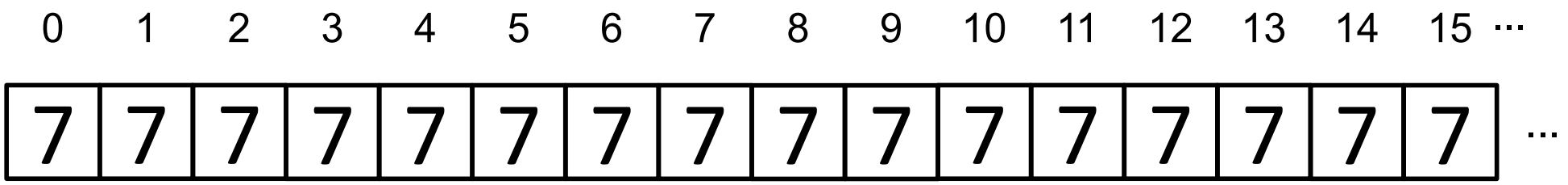
Value Broadcast

Start



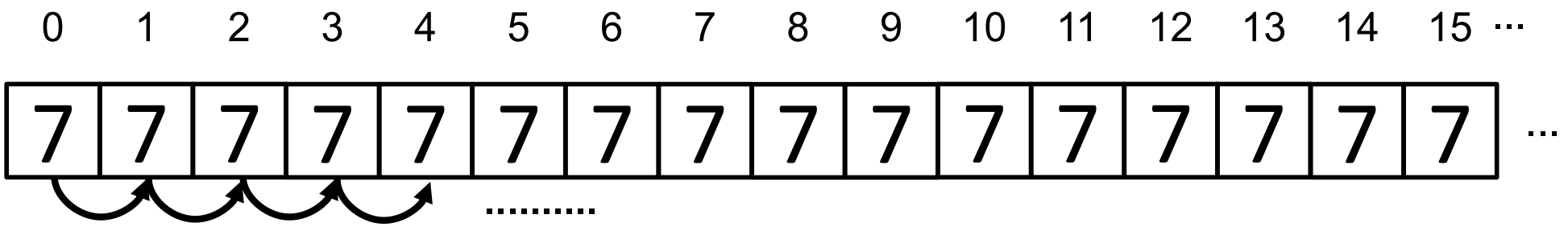
Value Broadcast

End



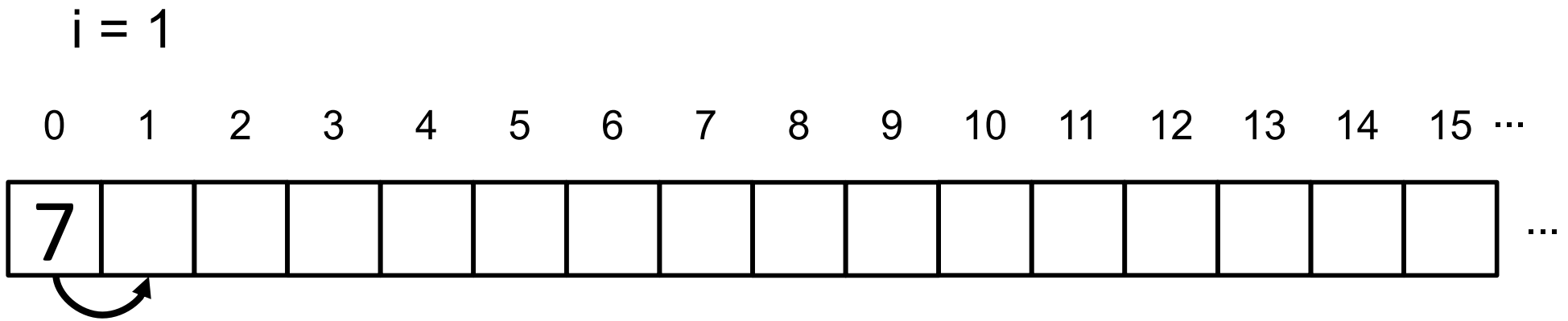
Inefficient Value Broadcast

O(n) time



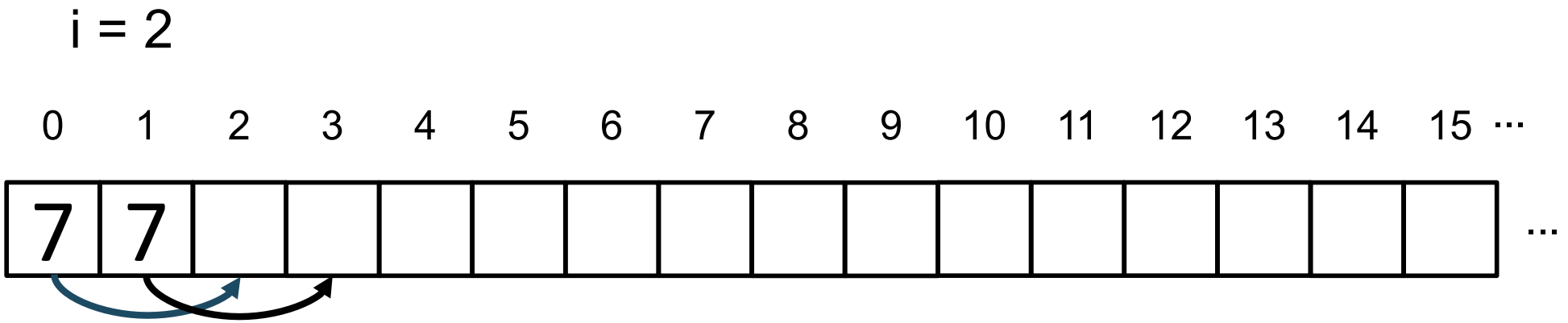
Efficient Value Broadcast

Every element that has the value
copies it to its current position + i



Efficient Value Broadcast

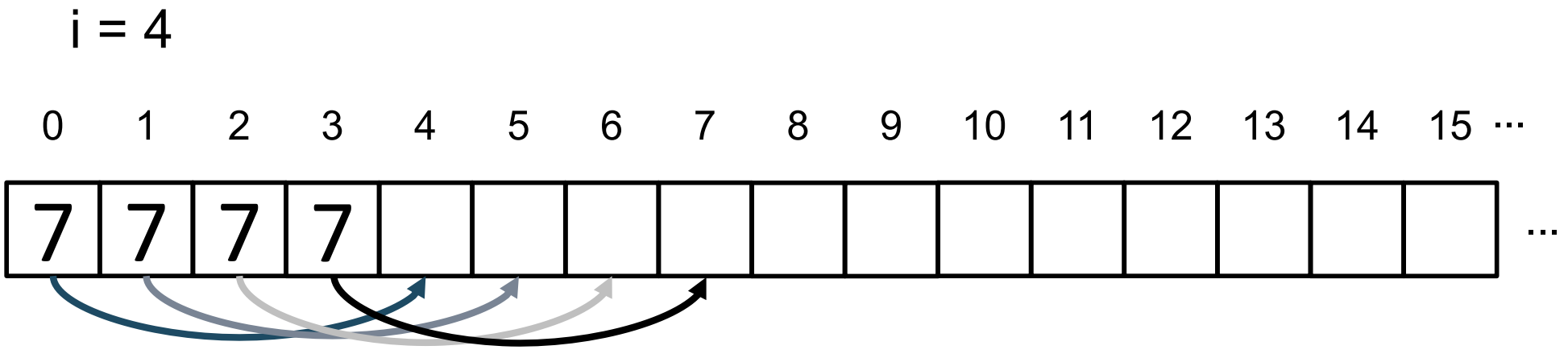
Every element that has the value
copies it to its current position + i



These operations can be executed in parallel

Efficient Value Broadcast

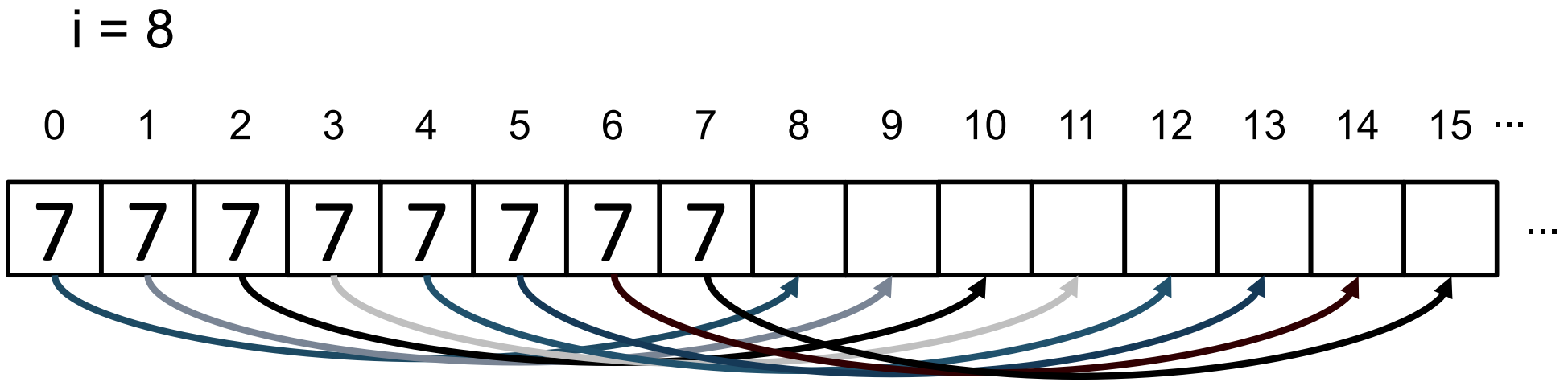
Every element that has the value
copies it to its current position + i



These operations can be executed in parallel

Efficient Value Broadcast

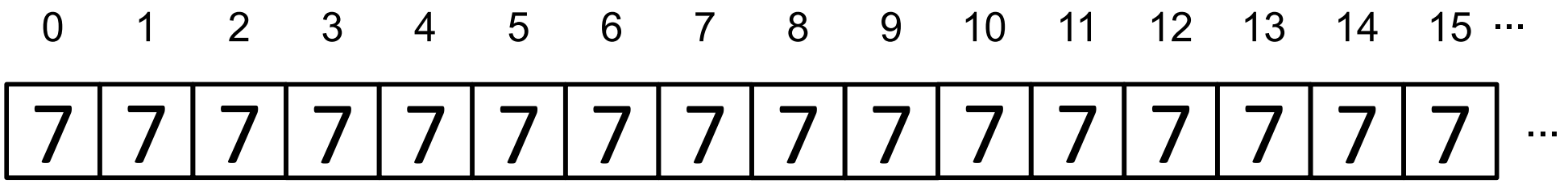
Every element that has the value
copies it to its current position + i



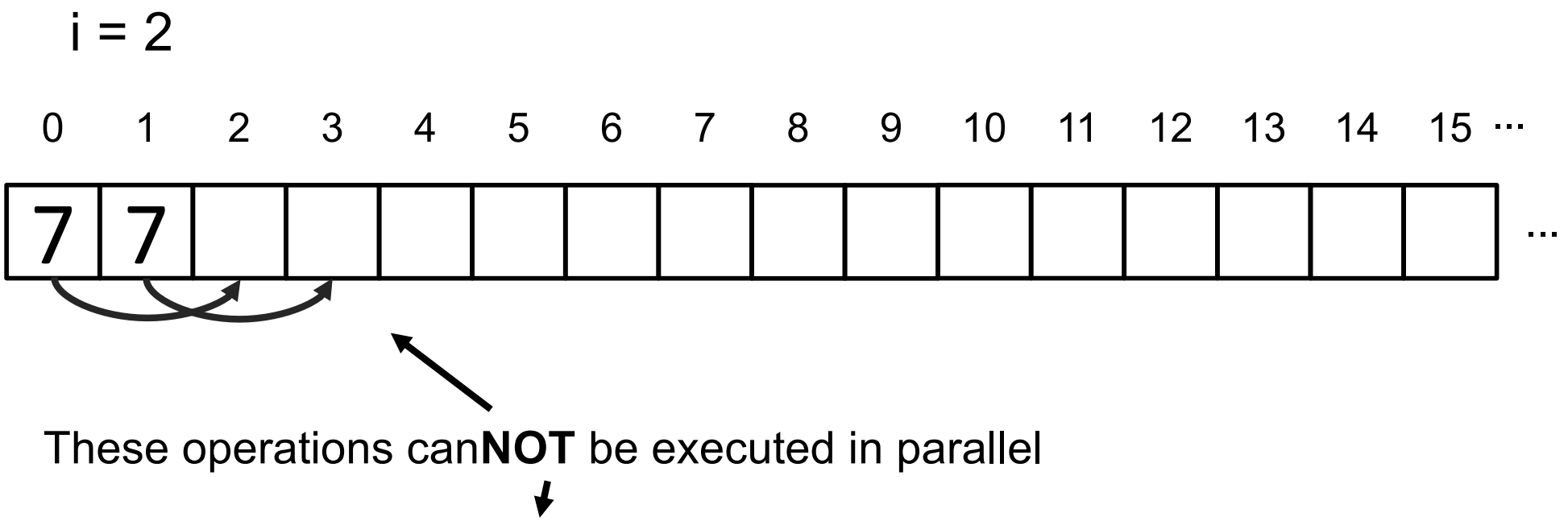
These operations can be executed in parallel

Efficient Value Broadcast

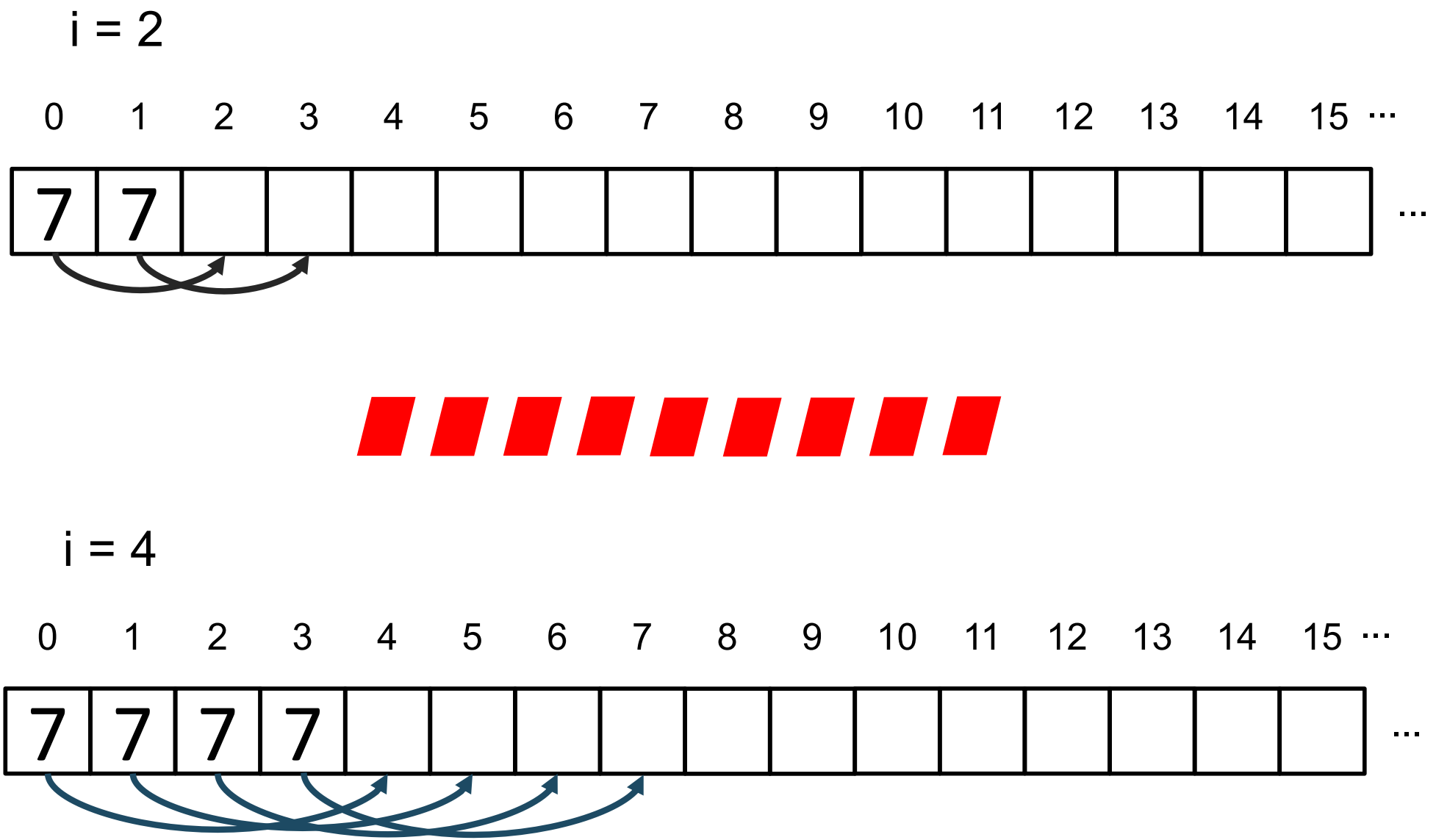
O(log2(n)) time with p = n/2 processors



Efficient Value Broadcast



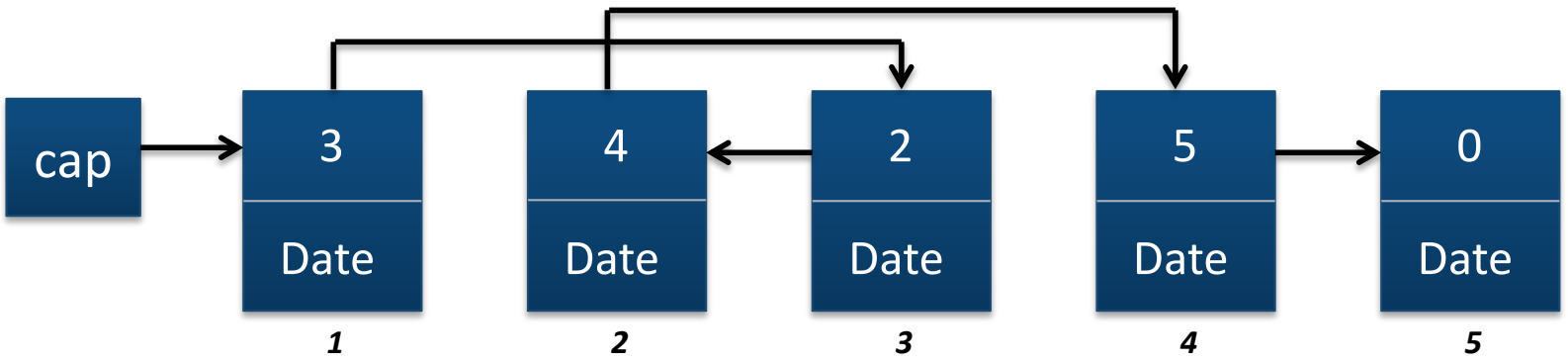
Efficient Value Broadcast



Operații cu liste

Operații cu liste

- Listă *n* elemente: `data[1:n]`
- Legăturile între elemente: `leg[1:n]`
- Capul listei: `cap`
- Elementul `i` face parte din listă ⇔ fie
  - `cap = i`
  - există un element `j`, între `1` și `n`, a.î. `leg[j]=i`

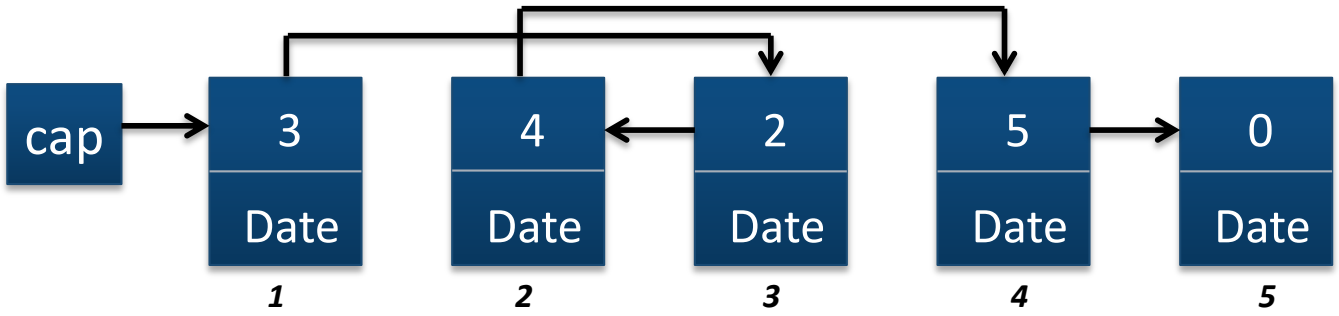


- Problemă: vrem ca fiecare procesor să afle capătul listei

Operații cu liste

```
int leg[1:n], end[1:n];
process Află[i=1 to n] {
      int nou, d=1;
      end[i] = leg[i];
      barrier;
      while (d<n) {
            nou = 0;
            if (end[i]<>0 and end[end[i]]<>0)
                  nou=end[end[i]]
            barrier;
            if (nou<>0) end[i]=nou;
            barrier;
            d = 2*d;
      }
}
```

Operații cu liste

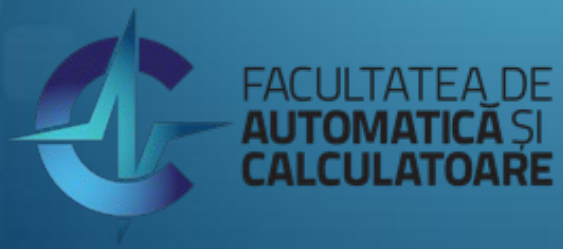


Procesor	End[i]	End[i]	End[i]	End[i]
1	3	2	5	5
2	4	5	5	5
3	2	4	5	5
4	5	5	5	5
5	0	0	0	0
	init	*d = 1* *d < 5*	*d = 2* *d < 5*	*d = 4* *d < 5*

```
int leg, end[1:n];
process Află[i=1 to n] {
    int nou, d=1;
    end[i] = leg[i];
    barrier;
    while (d < n) {
        nou = 0;
        if (end[i]<>0 and end[end[i]]<>0)
            nou = end[end[i]]
        barrier;
        if (nou <> 0)
            end[i] = nou;
        barrier;
        d = 2 * d;
    od
oc
```

Algoritmi Paraleli și Distribuiți
Comunicarea prin mesaje.
Complexitatea algoritmilor distribuiți

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



1. Comunicarea prin mesaje

Transmiterea de mesaje

- Îndeplinește cele două funcții:
  - **comunicarea** - datele ajung de la transmiţător la receptor
  - **sincronizarea** - un mesaj nu poate fi recepţionat înainte de a fi transmis
- **Canalele** sunt obiectele puse în comun de procese
- **Programe distribuite**

Comunicare asincrona prin mesaje

Comunicarea asincronă prin mesaje

Modelul

- Canalele păstrează ordinea mesajelor
- Canalele nu pierd mesajele
- **canalele** sunt cozi FIFO de mesaje, de capacitate nelimitata
- operaţia **send** nu este blocantă
- operaţia **receive** este blocantă

Declaraţie – forma generală

- **chan** nume_canal (tip1 id1,..., tipn idn)

Grup indexat de canale

- **chan** rezultat [1:n](int)

Operaţii

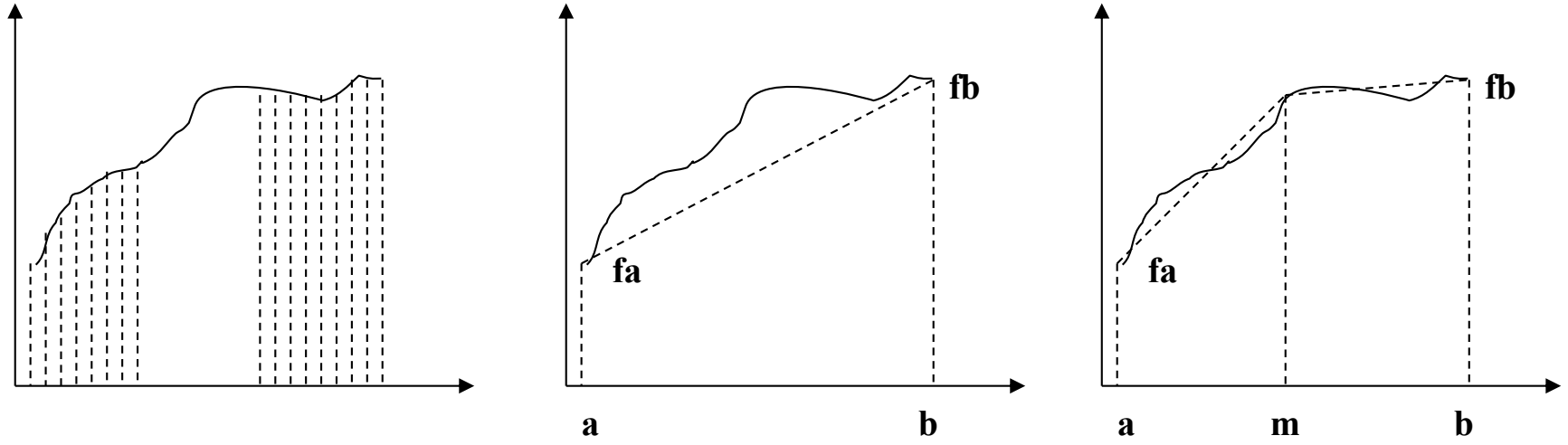
- **send** nume_canal (expresie$_1$,..., expresie$_n$)
- **receive** nume_canal (variabila$_1$,..., variabila$_n$)
- **empty** (nume_canal)

Exemplu - Filtru

```
chan   input (char),
       output (char [1:Maxl]);

process car_linii{
  char line[1:Maxl]; int i=1;
  while (true) {
    receive input (line[i]);
    while (line[i] <> CR and i < Maxl) {
      /* line[1:i] = ultimele i caractere introduse*/
      i = i + 1;
      receive input (line[i]);
    }
    send output (line);
    i = 1;
  }
}
```

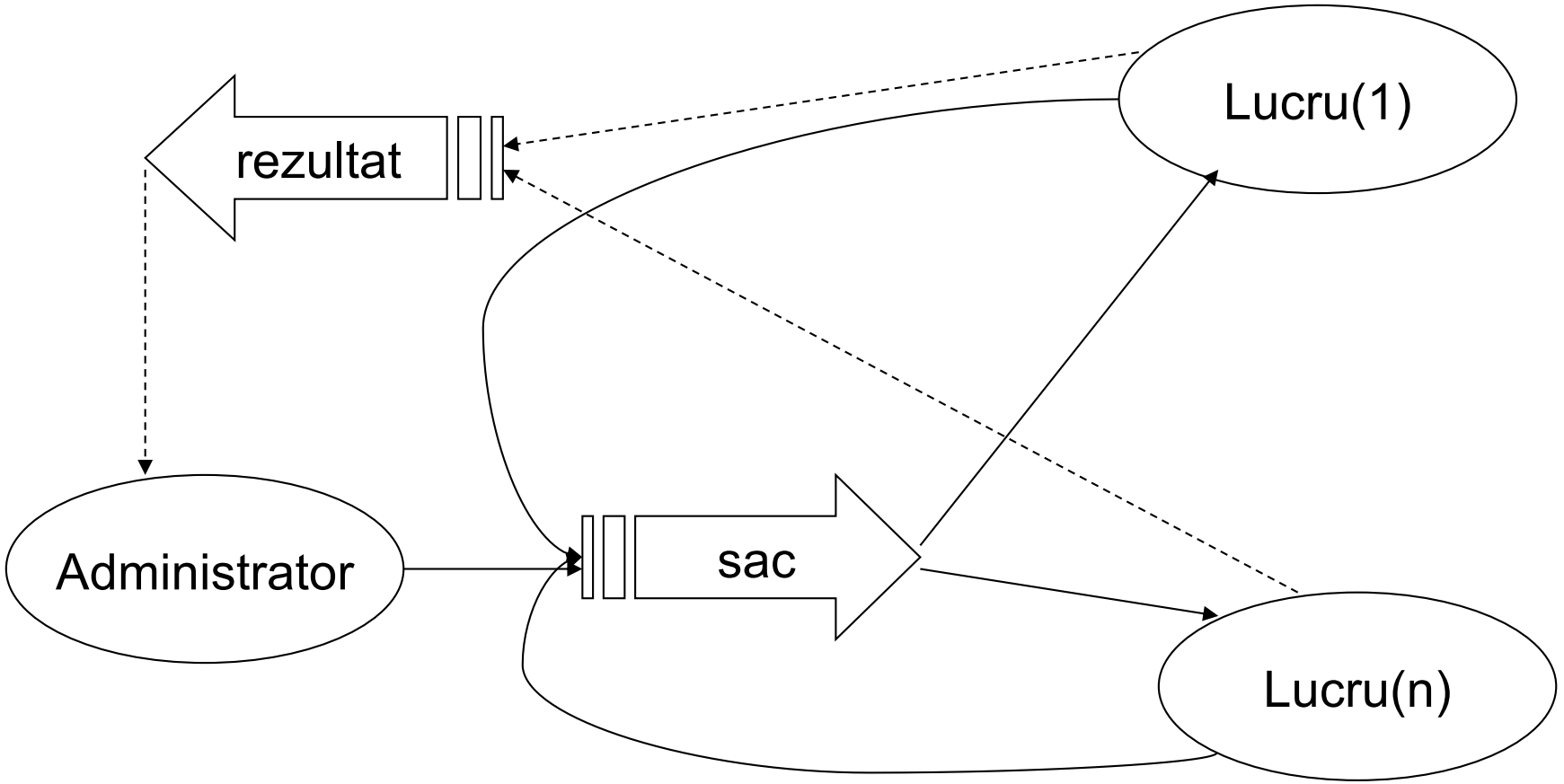
Modelul Replicated Workers

Integrare numerica



- Stanga – varianta secventiala
  - calcul iterativ - la fiecare pas se dubleaza numarul de subintervale
  - se termina cand diferenta intre doua valori succesive este acceptabila
- Dreapta – varianta distribuita
  - un proces primeste un subinterval [a,b] si aria corespunzatoare
  - imparte intervalul in doua sub-intervale [a.m], [m,b] si calculeaza ariile
  - compara suma celor doua arii cu aria primita
  - continua calculul separat pe cele doua subintervale, daca este cazul

Replicated workers



Exemplu - Replicated Workers (2)

```
chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);

process Administrator{
  real xl, xr, yl, yr, a, b, aria, total: real;
  /* alte variabile marcare intervale terminate */

  yl = f(xl); yr = f(xr);
  aria = (yl + yr) * (xr - xl) / 2;
  send sac(xl, xr, yl, yr, aria);

  while (nu s-a calculat toata aria) {
    receive rezultat (a, b, aria);
    total = total + aria;
    marchează intervalul [a, b] terminat;
  }
}
```

Exemplu - Replicated Workers (3)

```
process Lucru[i=1 to n]{
   real a, b, m, ya, yb, ym;
   real ariaSt, ariaDr, ariaTot, dif;
   while (true) {
      receive sac (a, b, ya, yb, ariaTot);
      m = (a + b) / 2;
      ym = f(m);
      calculează ariaSt și ariaDr;
      dif = ariaTot - ariaSt - ariaDr;
      if (dif mic) send rezultat (a, b, ariaTot);
      else { send sac (a, m, ya, ym, ariaSt);
             send sac (m, b, ym, yb, ariaDr);
      }
   }
}
```

Comunicare sincrona prin mesaje

Comunicarea sincrona prin mesaje

Modelul

Caracteristici similare cu comunicarea asincrona

Diferenta: si operaţia **send** **este** blocantă – transmitatorul se blocheaza pana cand mesajul este receptionat

Operaţii

synch_send nume_canal (expresie$_1$,..., expresie$_n$)

receive nume_canal (variabila$_1$,..., variabila$_n$)

empty (nume_canal)

Neajunsuri

reduce concurenta

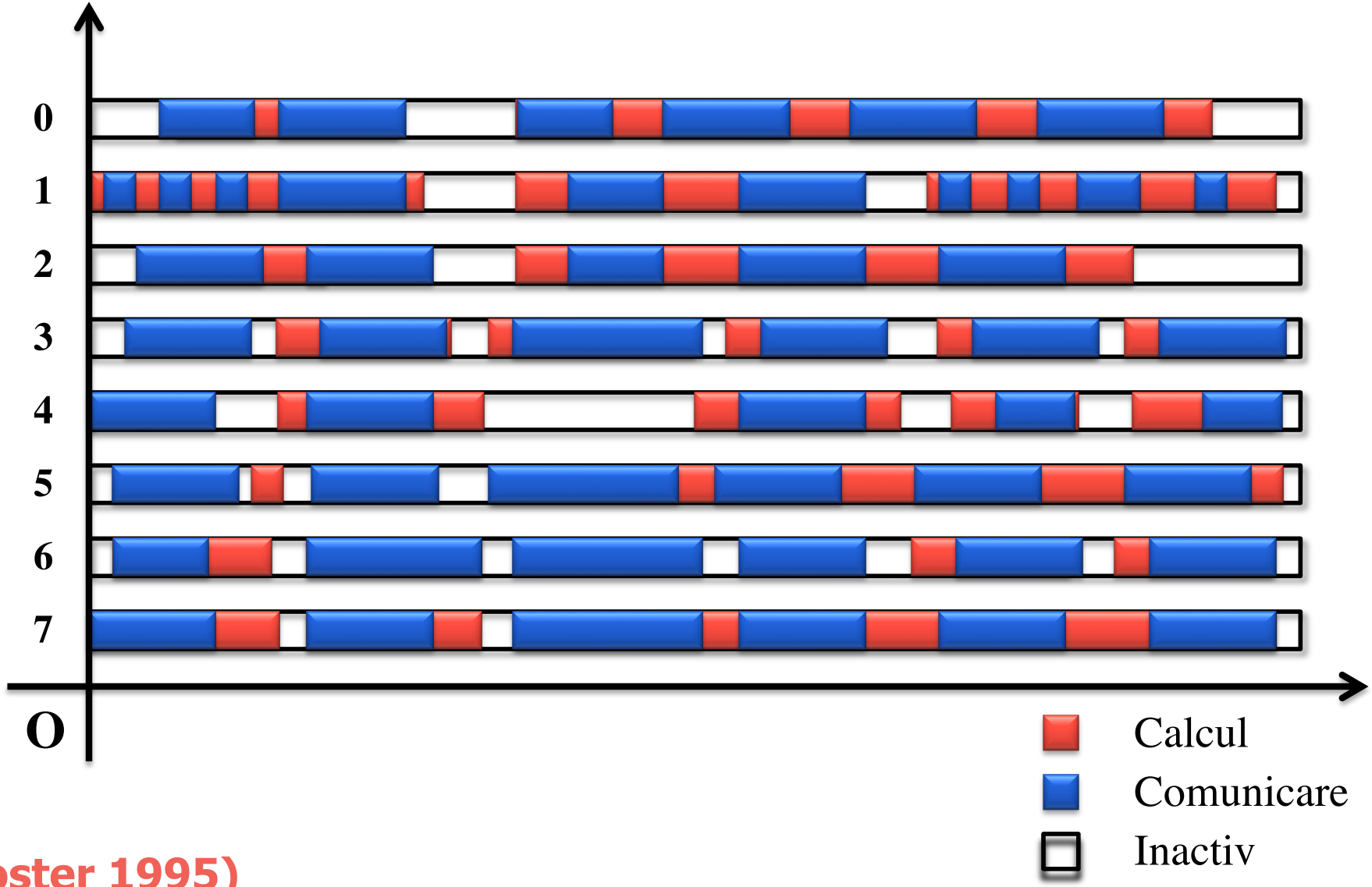
Exemplu – comunicare producator / consumator

```
chan prodcons (real);

process prod {
    real date[n];
    for [i = 1 to n] {
        calculeaza date[i];
        synch_send prodcons (date[i]);    /* intarzie transmitatorul */
    }                                     /* daca receptorul este mai lent */
}
process cons {
    real buf[n];
    for [i = 1 to n] {
        receive prodcons (buf[i]);
        prelucreaza datele primite;
    }
}
```

2. Complexitatea algoritmilor distribuiți

Modelul Foster

Modelul Foster



(Foster 1995)

Timpul total de execuţie

- Definiţie
  - Timpul scurs de la începerea execuţiei primului proces până la terminarea execuţiei ultimului proces.

$$T = f\,(N,\ P,\ U, \ldots)$$

$$= T^{j}_{comp} + T^{j}_{commun} + T^{j}_{idle}$$

$$= \left(\frac{1}{P}\right) * \left(\sum_{i=0}^{P-1} T^{i}_{comp} + \sum_{i=0}^{P-1} T^{i}_{commun} + \sum_{i=0}^{P-1} T^{i}_{idle}\right)$$

$$= \left(\frac{1}{P}\right) * \left(T_{comp} + T_{commun} + T_{idle}\right)$$

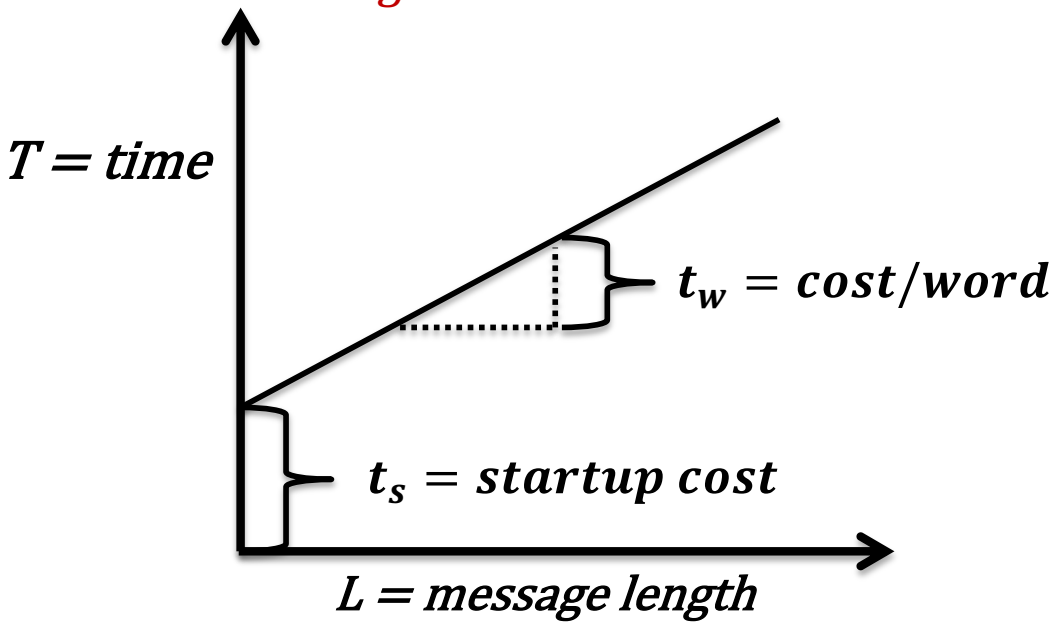
Timpul de calcul - T_{comp}

- Depinde de:
  - dimensiunea problemei N
  - numărul de task-uri sau procesoare
  - caracteristicile procesoarelor şi memoriei
- Exemplu:
  - scalarea dimensiunii problemei sau a numărului de procesoare poate modifica performanța cache-ului sau eficacitatea pipelining-ului procesorului

Timpul de comunicare – T_{commun} (1)

- Timpul petrecut cu transmiterea - recepţia de date
- Tipuri:
  - inter-procesor
  - intra-processor
- Presupunere: au costuri comparabile

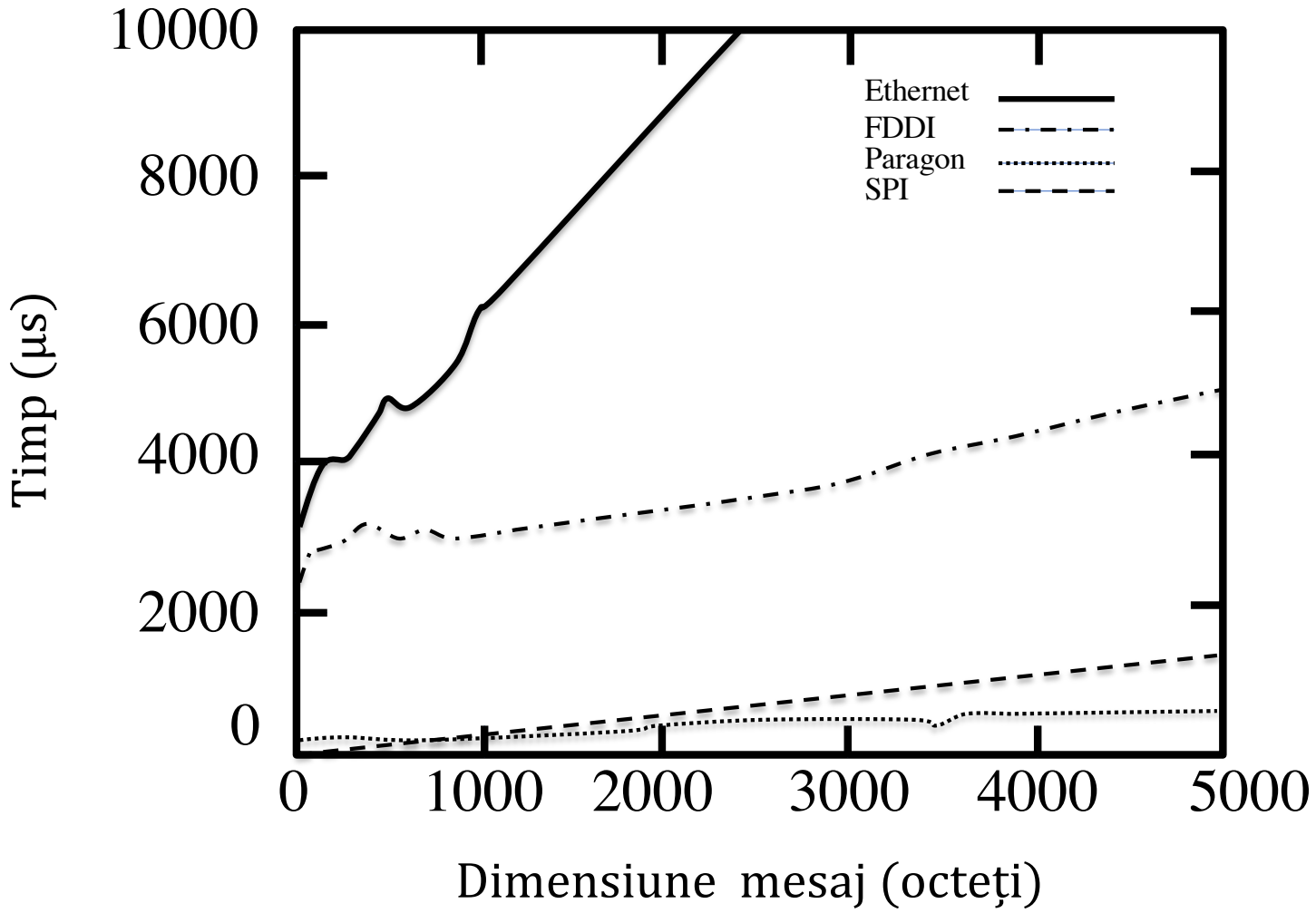
$$T_{msg} = t_s + t_w L$$



Machine	t_s (µs)	t_w(µs)
IBM SP2	40	0.11
Intel DELTA	77	0.54
Intel Paragon	121	0.07
Meiko CS-2	87	0.08
nCUBE-2	154	2.4
Thinking Machines CM-5	82	0.44
Workstations on Ethernet	1500	5.0
Workstations on FDDI	1150	1.1

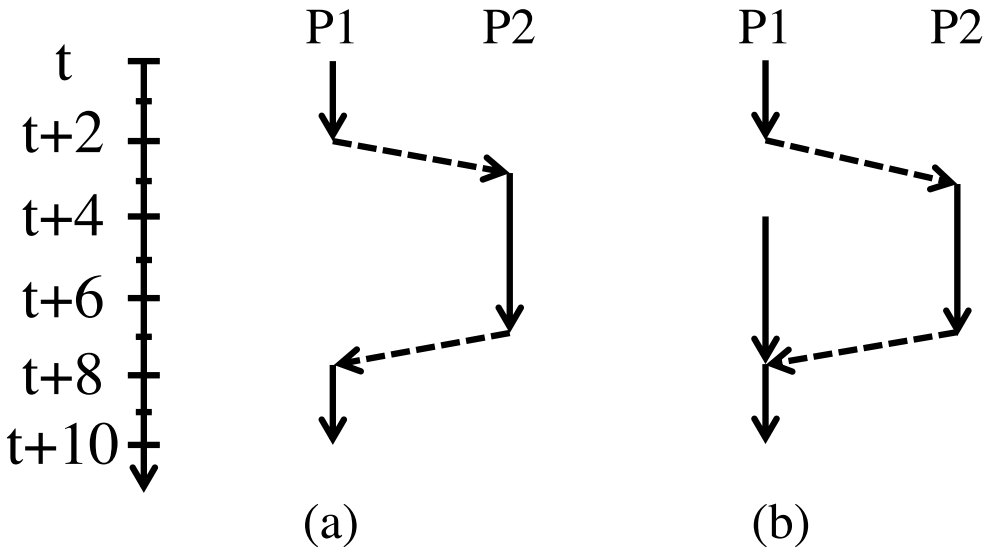
Timpul de comunicare – T_{commun} (1)

- Măsurători experimentale: timp dus-întors pentru mesaje (RTT)



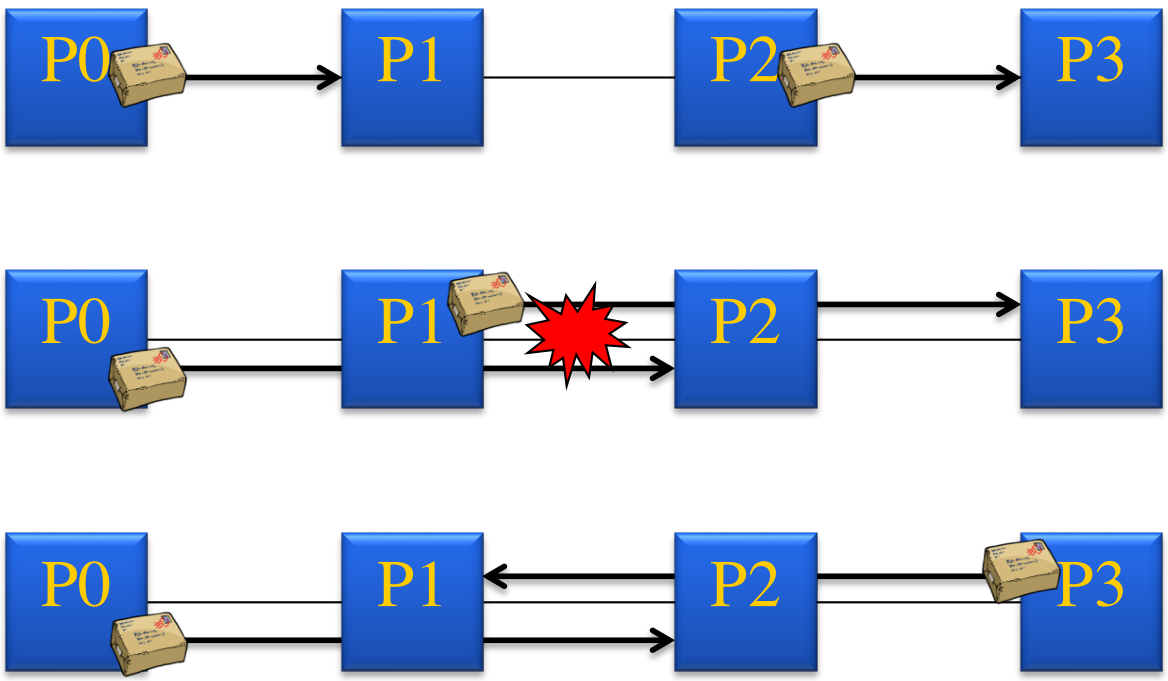
Timpul idle - T_{idle}

- Datorită:
  - lipsei calculelor → *load balancing*
  - lipsei datelor → *overlapping computation and communication*



Model revizuit pentru cost comunicare

- Model comunicare simplu: $T_{msg} = t_s + t_w L$
- Competiţie pentru bandă: $T_{msg-b} = t_s + t_w SL$
- S = număr de procesoare care comunică concurent pe acelaşi canal (și în același sens)



Exemplu – Sequential Floyd

- $I[i,j]_0 = 0$, daca i == j
- $I[i,j]_0 = cost(i,j)$, dacă există muchie și i != j
- $I[i,j]_0 = MAX$, altfel

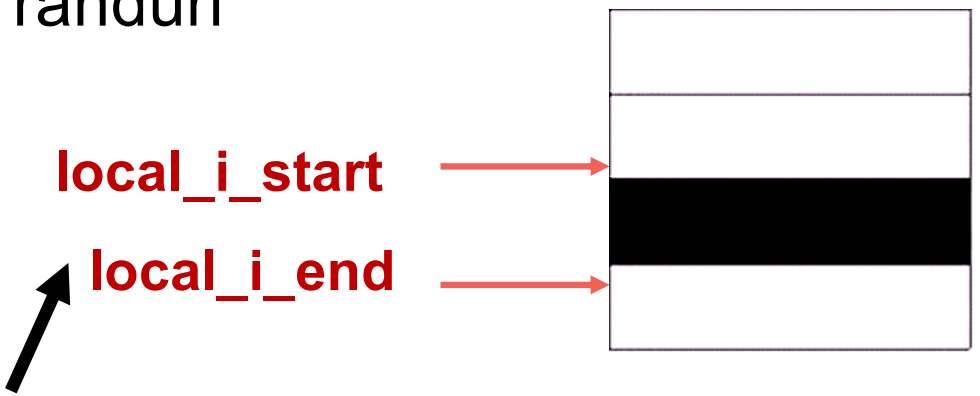
```
for [k = 0 to N - 1]
  for [i = 0 to N - 1]
    for [j = 0 to N - 1]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

Durata unei iterații

$$T = t_c N^3$$

Exemplu – Parallel Floyd 1 (1)

- Bazat pe o descompunere uni-dimensională pe rânduri a matricei I (algoritmul poate folosi cel mult P procesoare, P<=N)
- Fiecare task are unul sau mai multe rânduri adiacente din I şi răspunde de calculul acestor rânduri




```
for [k = 0 to N-1]
   for [i = local_i_start to local_i_end]
     for [j = 0 to N-1]
     I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

Exemplu – Parallel Floyd 1 (2)

- În pasul k, procesele au nevoie de linia k din matricea I.
- Procesorul cu această linie o difuzează tuturor în log P paşi folosind o structură arborescentă.
- Deoarece fiecare mesaj are N cuvinte, timpul de difuzare a unei linii este: log P ($t_s + t_w N$)

a)

b)

Exemplu – Parallel Floyd 1 (3)

- Fiecare task este rădăcină pentru cel puţin o difuzare (dacă P ≤ N).
- Folosind un hipercub, fiecare nod poate difuza în log P paşi
- Pentru N difuzări, cu mesaje de lungime N:

$$T_{Floyd1} = \frac{t_c N^3}{P} + N \log P \, (t_s + t_w N)$$

Timpul de comunicare

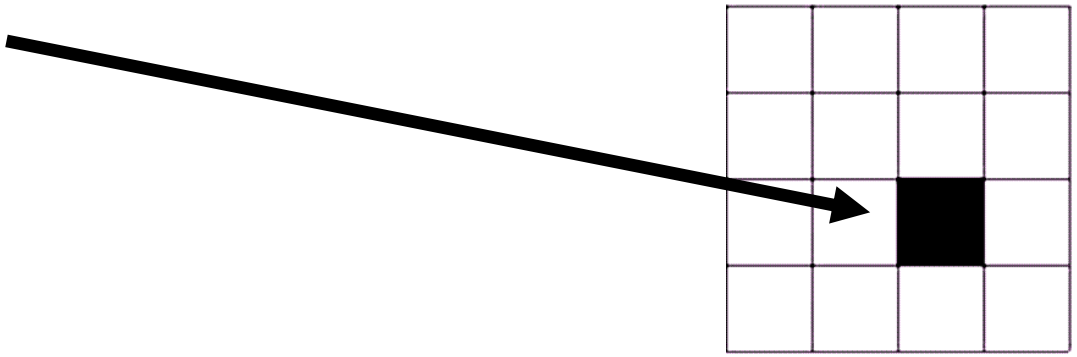
Timpul algoritmului secvențial împărțit la numărul de procesoa

Exemplu – Parallel Floyd 2 (1)

- Descompunere bi-dimensională a matricelor.
- Foloseşte până la N^2 procesoare.

```
for [k = 0 to N - 1]
  for [i = local_i_start to local_i_end]
    for [j = local_j_start to local_j_end]
      I[i,j]k+1 = min(I[i,j]k, I[i,k]k + I[k,j]k)
```

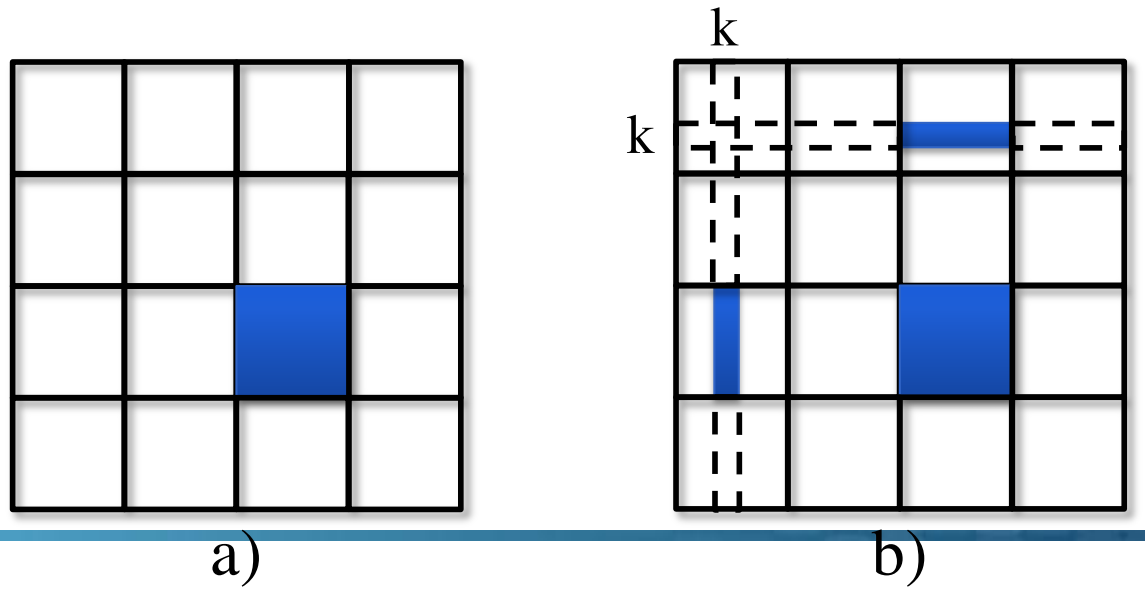
- Fiecare task răspunde de calculul elementelor situate intr-un patrat.



Exemplu – Parallel Floyd 2 (2)

- Cerinţe de comunicare în pasul k: două operaţii de difuzare.
- În fiecare pas, fiecare task cere, în plus faţă de datele locale, $\frac{N}{\sqrt{P}}$ valori de la două task-uri din aceeaşi linie şi aceeaşi coloană.
- Fiecare din cele doua mesaje are $\frac{N}{\sqrt{P}}$ elemente.
- Difuzarea unui mesaj durează (sunt $\sqrt{P}$ procese):

$$log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$



Exemplu – Parallel Floyd 2 (3)

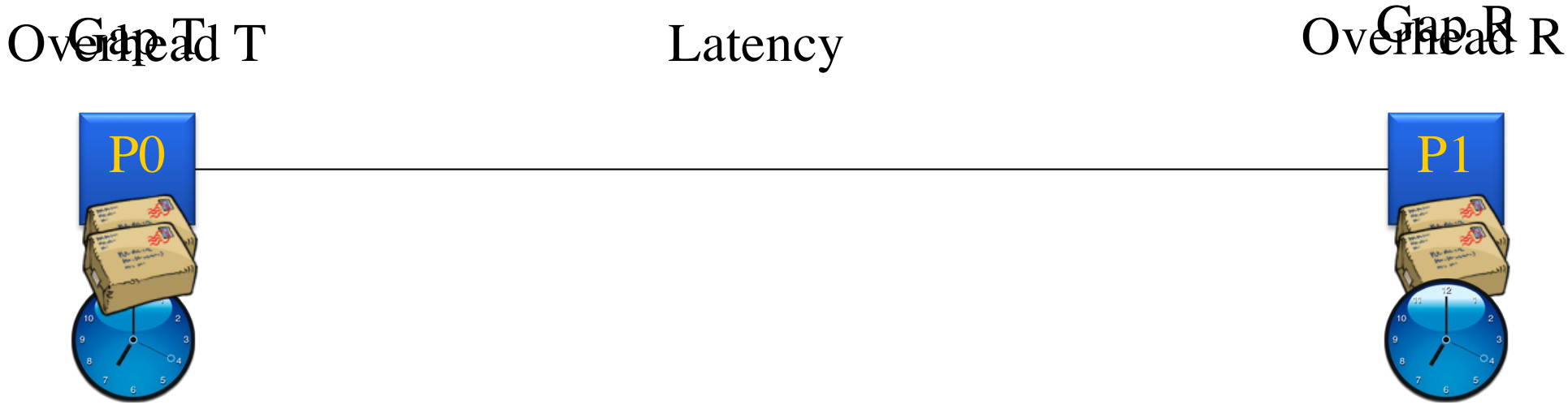
- În fiecare din N paşi, $\frac{N}{\sqrt{P}}$ valori trebuie difuzate la $\sqrt{P}$ task-uri în fiecare linie şi coloană (se poate folosi o structură hipercub); costul total este:

$$T_{Floyd\text{-}2} = t_c \frac{N^3}{P} + 2N \log\sqrt{P}\left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

$$= t_c \frac{N^3}{P} + N \log P \left(t_s + \frac{t_w N}{\sqrt{P}}\right)$$

Modelul LogP

Modelul LogP



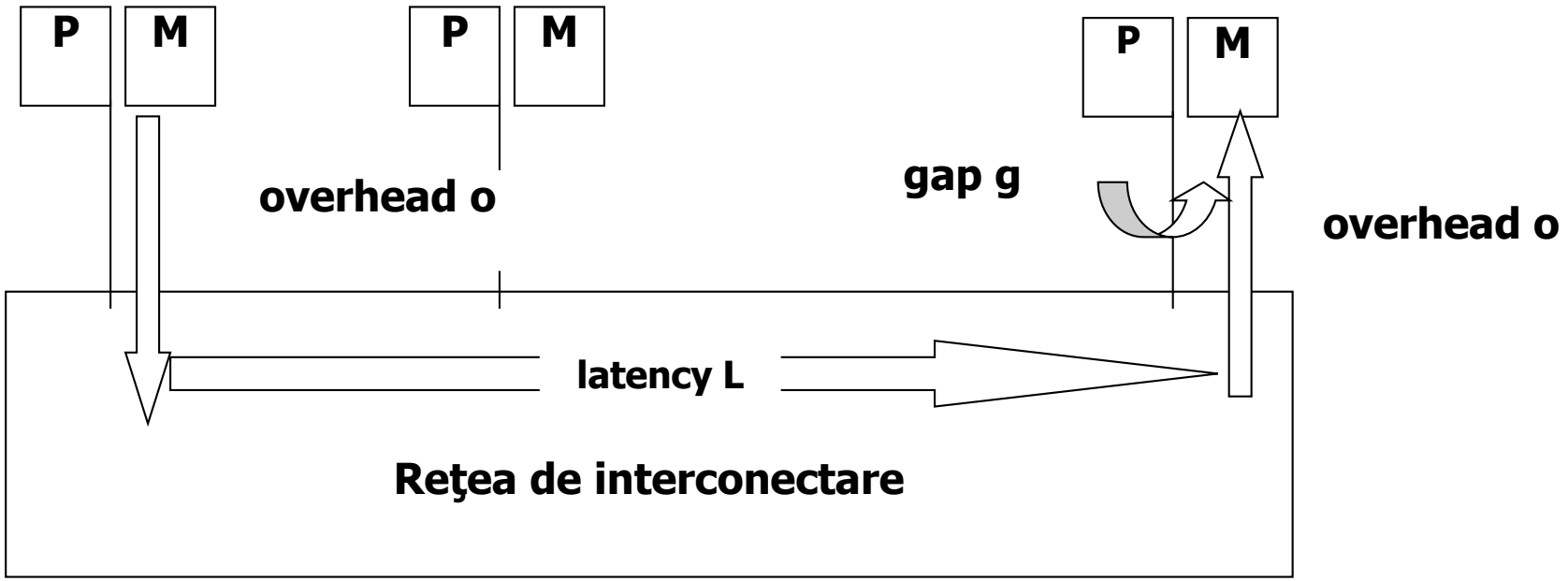
L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

P - numărul de module procesor / memorie.

Modelul LogP



L - latency sau întârzierea de transmitere a unui mesaj mic de la sursă la destinatar

o - overhead, durata pentru care procesorul este angajat în transmiterea sau recepţia fiecărui mesaj

g - gap, intervalul minim de timp între două transmiteri succesive sau două recepţii succesive la acelaşi modul

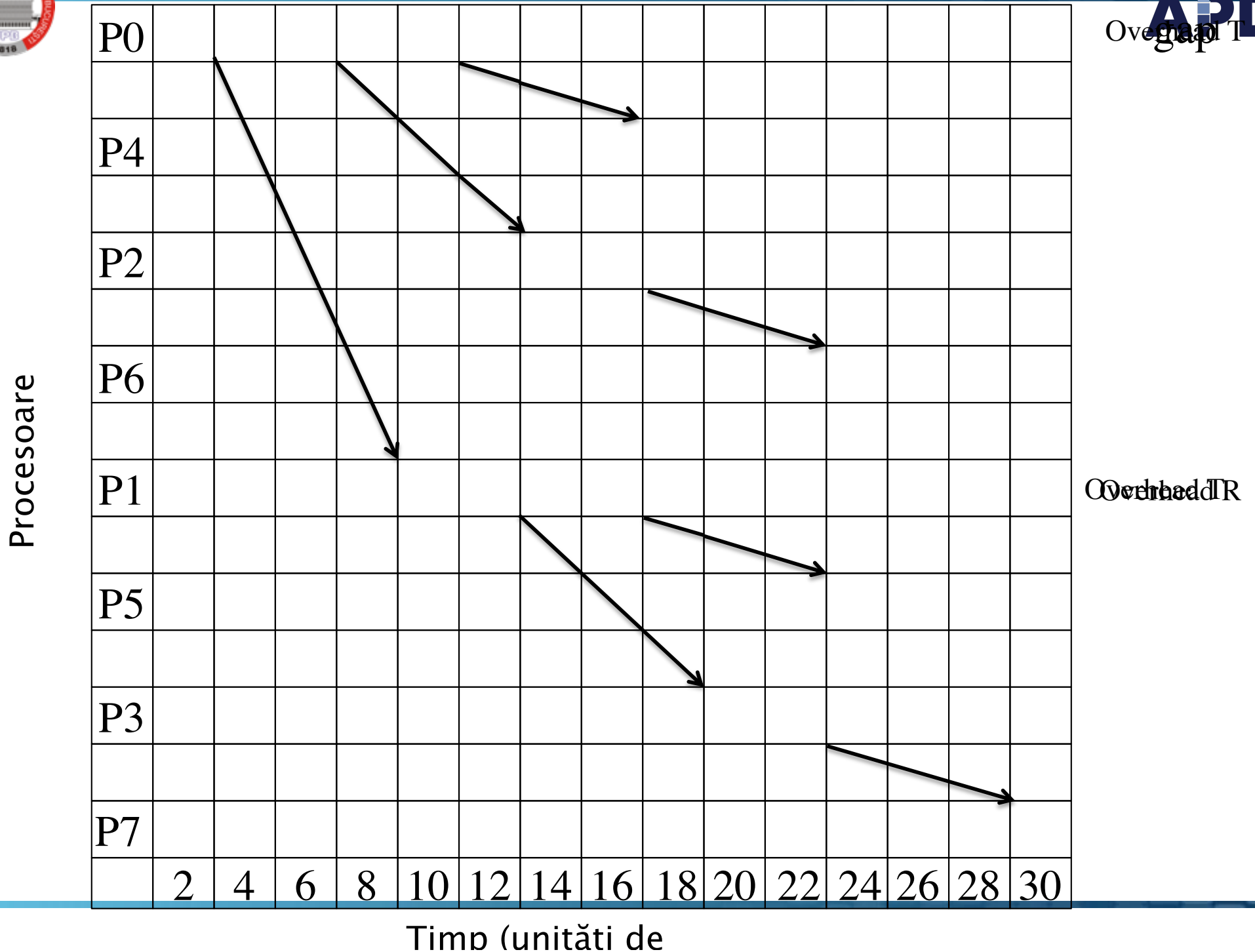
P - numărul de module procesor / memorie.

Exemplu - Difuzarea unei valori (1)

- Folosind modelului PRAM*, transmiterea are loc conform tiparului:
  - P0 → P1
  - P0, P1 → P2, P3
  - P0, P1, P2, P3 → P4, P5, P6, P7

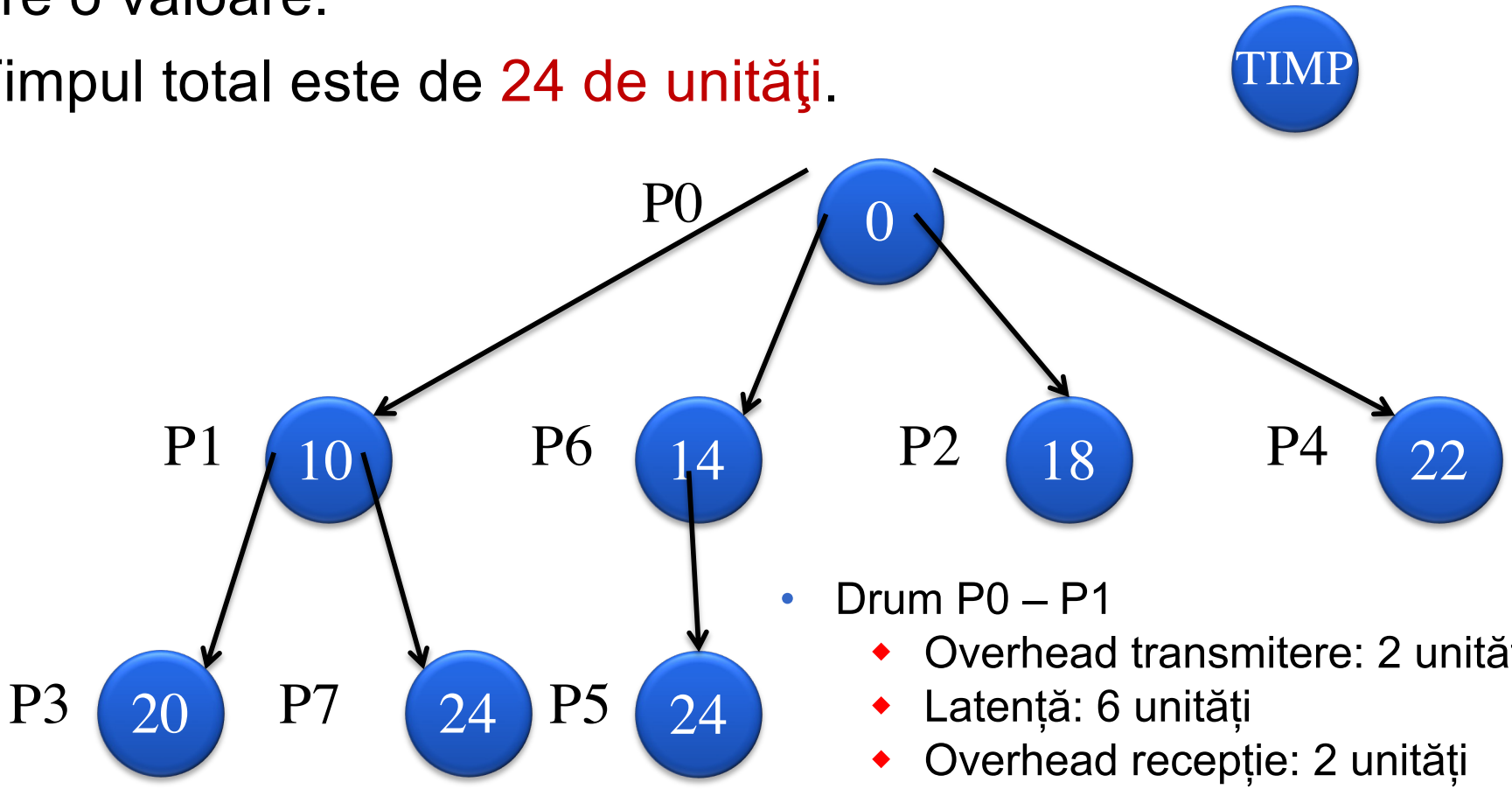
- Presupunând g ≥ 0 şi luând P = 8, o =2, g = 2, L = 6, obţinem timp total de difuzare = 30 de unităţi.

* Parallel Random Access Machine – vezi curs 1.

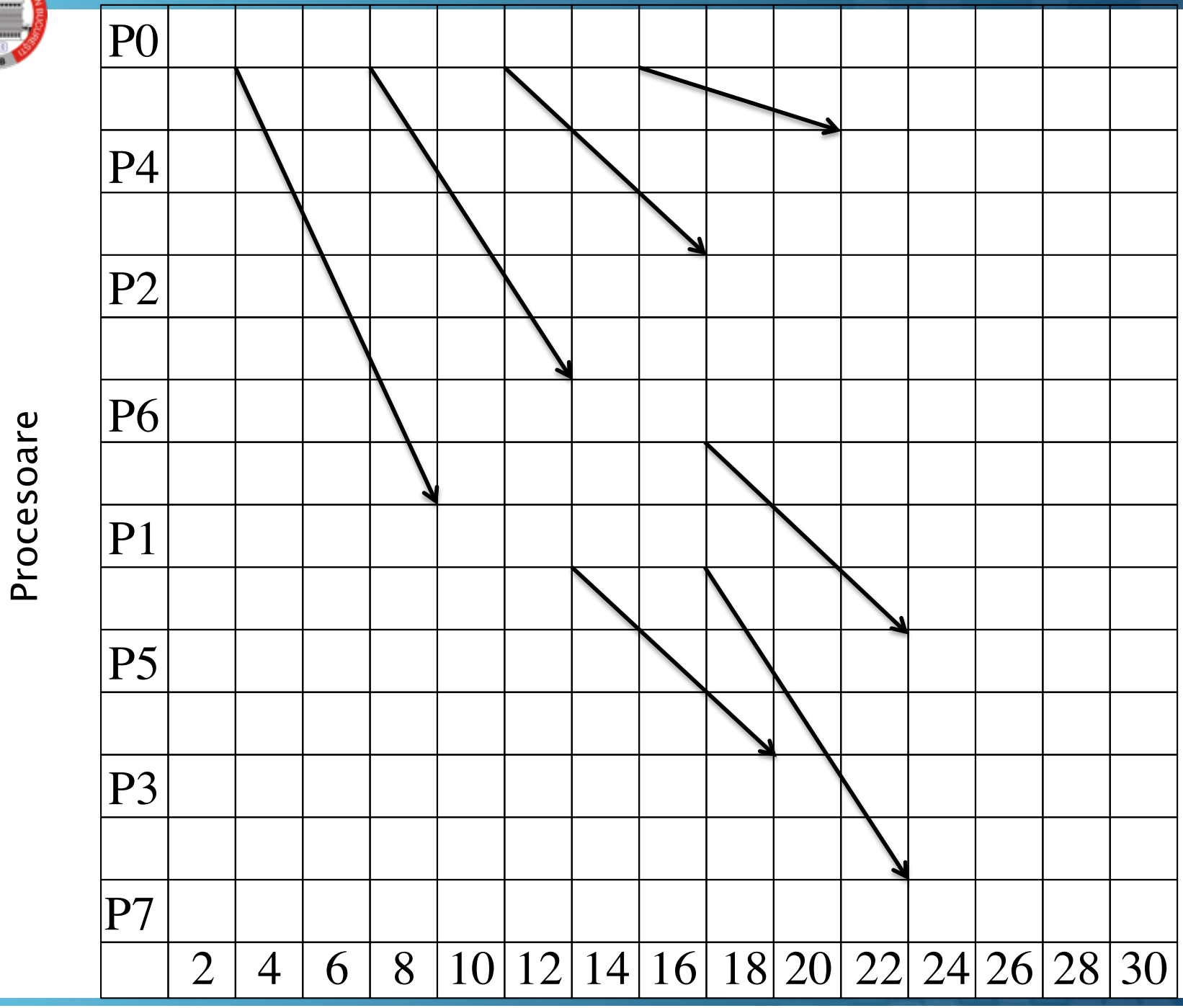

Overhead T
Procesoare
P0
P4
P2
P6
P1
P5
P3
P7
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
Timp (unități de

Exemplu - Difuzarea unei valori (2)

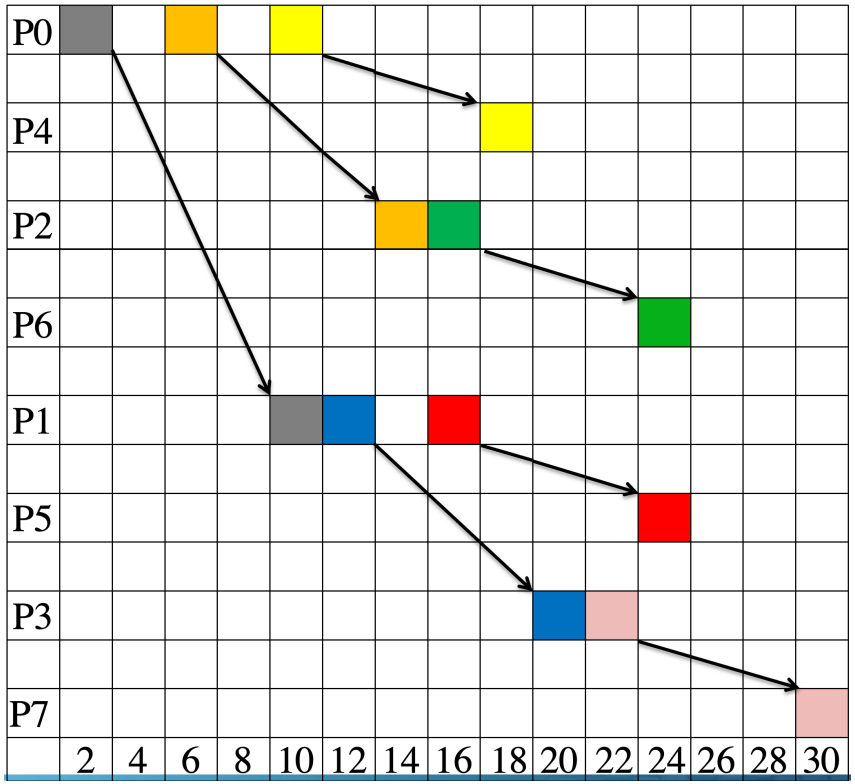
- Pentru modelul LogP, fiecare proces transmite imediat ce are o valoare.
- Timpul total este de 24 de unităţi.

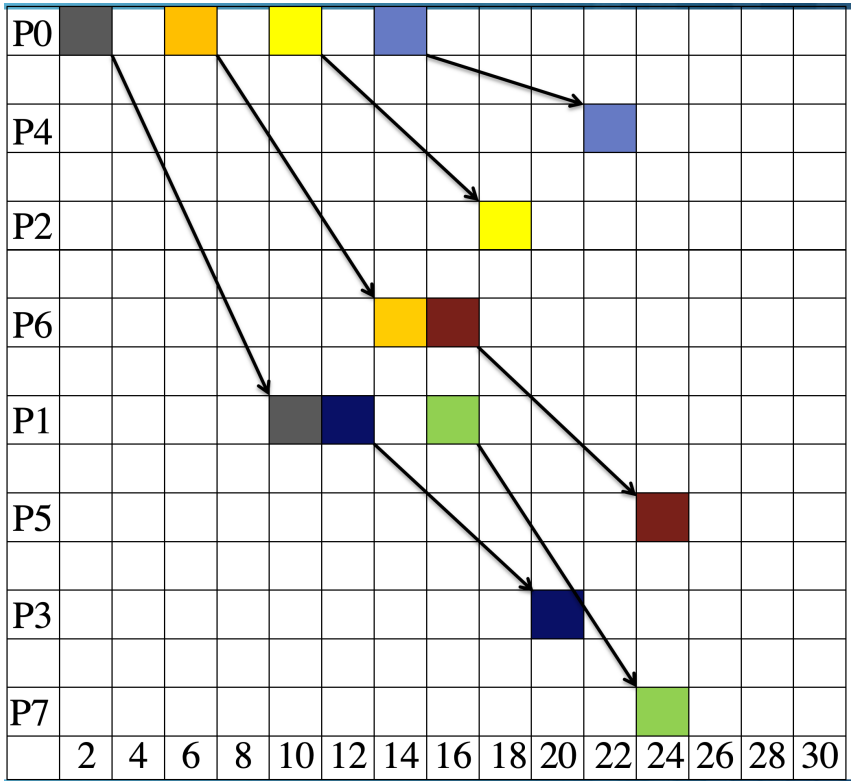


- Drum P0 – P1
  - Overhead transmitere: 2 unități
  - Latență: 6 unități
  - Overhead recepție: 2 unități
  - TOTAL: 10 unități


P0
P4
P2
P6
P1
P5
P3
P7
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
Procesoare
Timp (unități de timp)

Comparație


P0
P4
P2
P6
P1
P5
P3
P7
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30

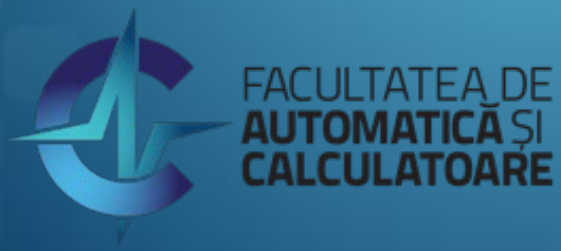

P0
P4
P2
P6
P1
P5
P3
P7
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30

Sumar

- Comunicarea prin mesaje
  - Comunicarea asincronă prin mesaje
  - Modelul Replicated Workers
  - Comunicarea sincronă prin mesaje
- Complexitatea algoritmilor distribuiți
  - Modelul Foster
  - Modelul LogP

Algoritmi Paraleli și Distribuiți
Ceasuri Logice
Ordonarea evenimentelor

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



De la ceasuri fizice la... ceasuri logice

Ceasuri fizice

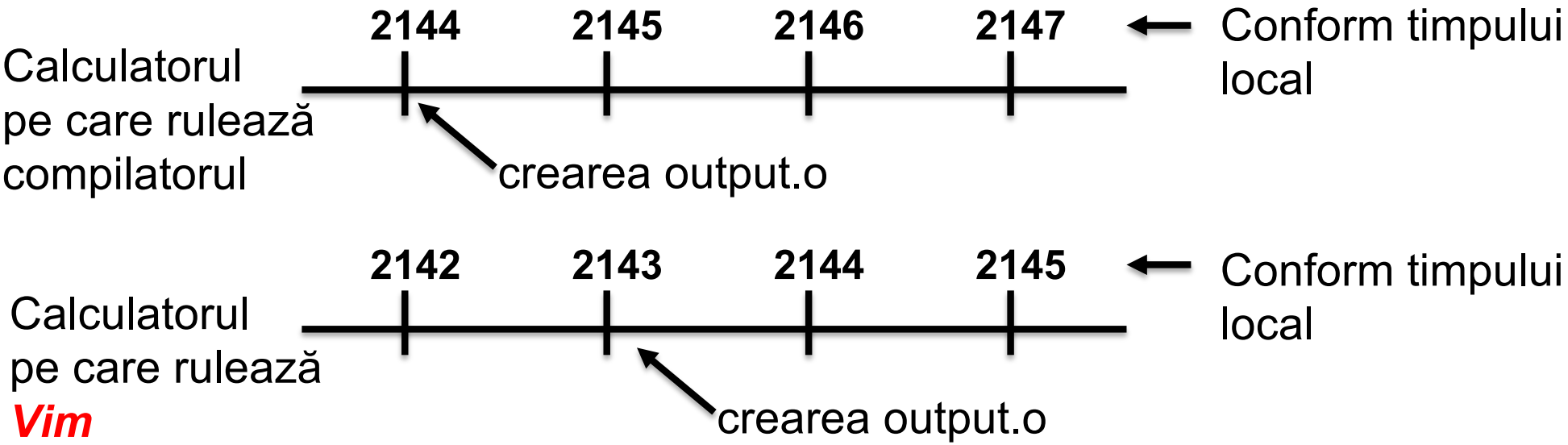
- Timpul este ne-ambiguu în sisteme ne-distribuite
  - există un singur ceas și (de obicei) kernelul face apeluri pentru a-i accesa valoarea
- Folosit implicit în multe aplicații
  - *Ex:* make *(pentru a determina fișierele care trebuie recompilate)*
- Sistemele distribuite introduc ambiguitatea în legătură cu timpul

Ceasuri fizice (2)

- Computerele folosesc un cristal de quartz care oscilează la o valoare bine definită și generează întreruperi (clock ticks) la intervale regulate
- Fiecare *clock tick* incrementează o valoare din memorie – kernelul convertește această valoare într-un format standardizat
  - *Ex:* nr. de milisecunde pornind de la Thu Jan 1 12:00:00 GMT 1970.
- Diferența în timp dintre calculatoare se numește clock skew.

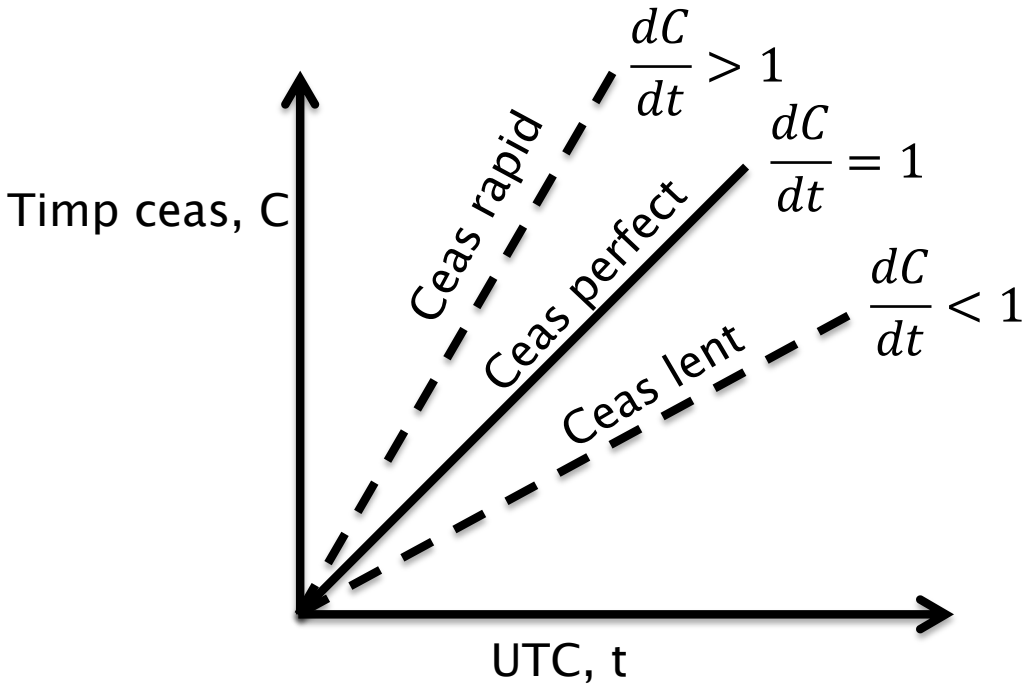
Ceasuri fizice (3)

- Clock skew dintre cele 2 computere confuzează make-ul:



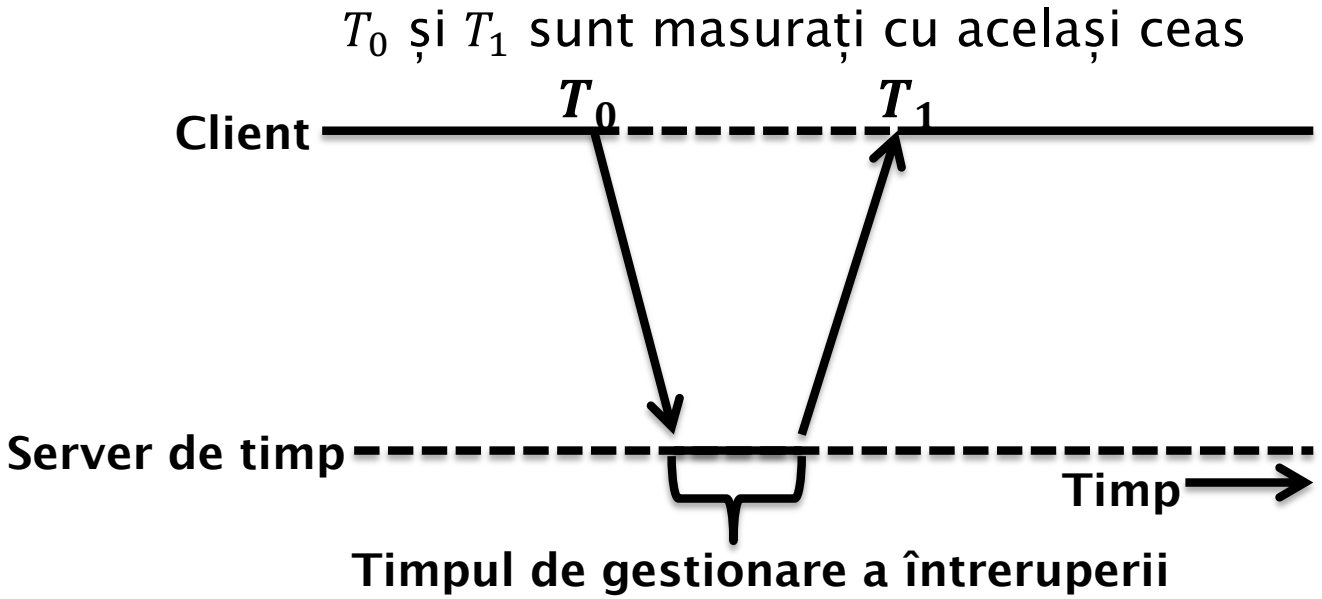
Sincronizarea ceasurilor fizice

- 2 tipuri:
  - doar între 2 calculatoare
  - sincronizare cu un standard *(ex: UTC)*
- Este imposibil de eliminat complet clock skew și imposibil de a asigura acuratețea perfectă a unui ceas.



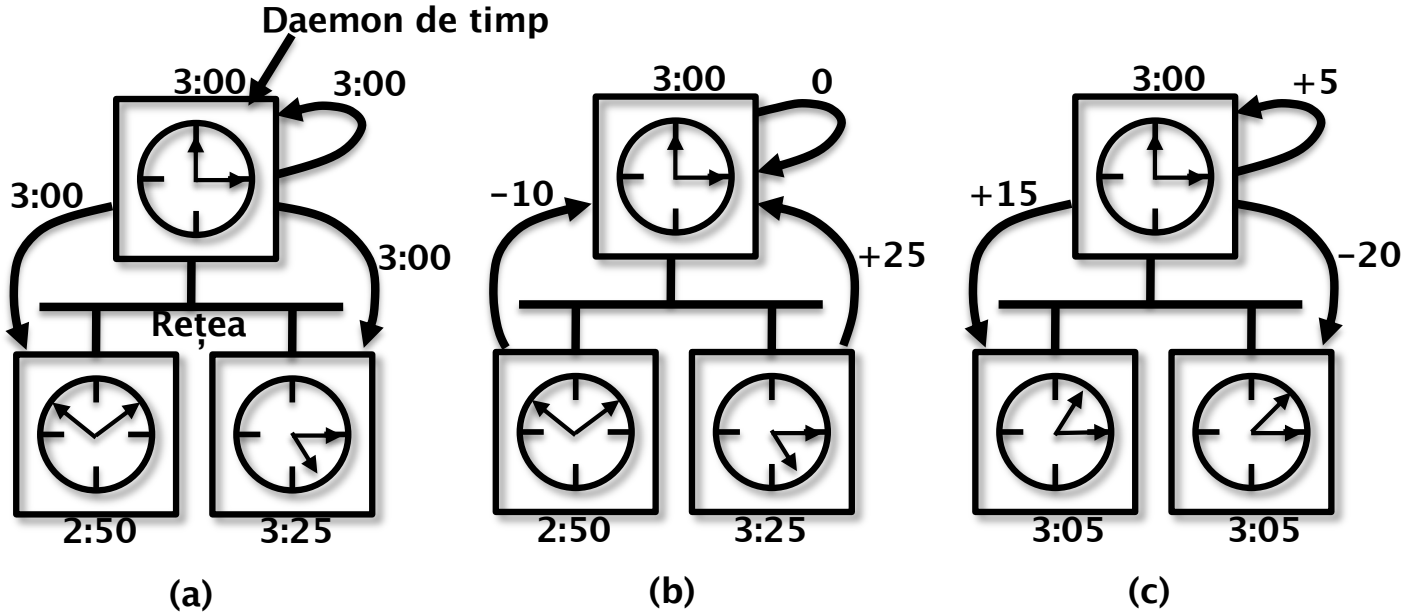
Sincronizarea ceasurilor fizice (2)

- Flaviu Cristian – adaptive internal clock synchronization:
  - time server, cu care se sincronizează celelalte ceasuri
  - probleme:
    - timpul nu trebuie sa curgă în sens invers
    - mesajul ajunge de la server la client într-un anumit timp



Sincronizarea ceasurilor fizice (3)

- Berkeley:
  - folosit în Berkeley UNIX
  - opus lui Flaviu Cristian*: un server de timp trimite mesaje periodic clienților pentru a afla timpul lor, calculează o medie și anunță clienții cum să-și actualizeze ceasurile*
  - scopul nu e de a se sincroniza cu time serverul, ci de a pune toate mașinile de acord asupra timpului (chiar dacă este greșit)



Sincronizarea ceasurilor fizice (4)

- Descentralizat: Network Time Protocol
  - Organizare ierarhică a serverelor de timp:
    - Stratul 1:
      - ceasuri de referință
        - *US Naval Observatory*
        - *Global Positioning System*
        - *Atomic Clocks*
    - Stratul 2:
      - routere, servere importante, etc.
  - Algoritmul și protocolul NTP extrem de complexe

    (RFC 1305 NTPv3 – 112 pagini , NTPv4 și mai complex)
  - Sincronizare precisă a ceasurilor: 1-50ms

Ceasuri logice (solutia Lamport)

Ceasuri logice și ordonarea evenimentelor

- Într-un algoritm distribuit, fiecare proces este caracterizat de:
  - o mulțime de stări
  - acțiuni care schimbă starea
- Eveniment *(producerea unei acțiuni)*
  - Evenimentele pot fi ordonate conform timpului fizic de producere.
  - Dificil pentru evenimente din procese diferite.
- Soluție mai simplă: ordinea relativă a evenimentelor
- Definită de relația petrecut înainte (→):
  - dacă **a** şi **b** sunt evenimente din acelaşi proces şi **a** îl precede in timp pe **b**, atunci a → b
  - când **a** reprezintă transmiterea unui mesaj de către un proces, iar **b** recepţia aceluiaşi mesaj de către un altul, atunci a → b
  - dacă a → b şi b → c, atunci a → c

Ceasuri logice și ordonarea evenimentelor (2)

- Mecanism de sincronizare bazat pe ***timp relativ***.
- ***timpul relativ*** poate să nu fie echivalent cu ***timpul real***.
  - *Exemplu:* Unix make
    - *Este important ca output.c să fie actualizat după generarea lui output.o ?*
- În aplicațiile distribuite ceea ce contează este ca procesele să *ajungă la un acord asupra ordinii de producere a* evenimentelor.
- Astfel de ***ceasuri*** se numesc ***ceasuri logice***.

Operating Systems

R. Stockton Gaines Editor

Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport
Massachusetts Computer Associates, Inc.

The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events. A distributed algorithm is given for synchronizing a system of logical clocks which can be used to totally order the events. The use of the total ordering is illustrated with a method for solving synchronization problems. The algorithm is then specialized for synchronizing physical clocks, and a bound is derived on how far out of synchrony the clocks can become.

Key Words and Phrases: distributed systems, computer networks, clock synchronization, multiprocess systems

CR Categories: 4.32, 5.29

Introduction

The concept of time is fundamental to our way of thinking. It is derived from the more basic concept of the order in which events occur. We say that something happened at 3:15 if it occurred *after* our clock read 3:15 and *before* it read 3:16. The concept of the temporal

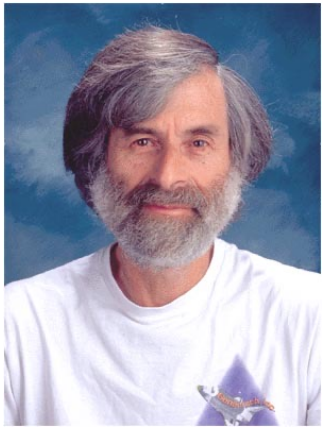
A distributed system consists of a collection of distinct processes which are spatially separated, and which communicate with one another by exchanging messages. A network of interconnected computers, such as the ARPA net, is a distributed system. A single computer can also be viewed as a distributed system in which the central control unit, the memory units, and the input-output channels are separate processes. A system is distributed if the message transmission delay is not negligible compared to the time between events in a single process.

We will concern ourselves primarily with systems of spatially separated computers. However, many of our remarks will apply more generally. In particular, a multiprocessing system on a single computer involves problems similar to those of a distributed system because of the unpredictable order in which certain events can occur.

In a distributed system, it is sometimes impossible to say that one of two events occurred first. The relation "happened before" is therefore only a partial ordering of the events in the system. We have found that problems often arise because people are not fully aware of this fact and its implications.

In this paper, we discuss the partial ordering defined by the "happened before" relation, and give a distributed algorithm for extending it to a consistent total ordering of all the events. This algorithm can provide a useful mechanism for implementing a distributed system. We illustrate its use with a simple method for solving synchronization problems. Unexpected, anomalous behavior can occur if the ordering obtained by this algorithm differs from that perceived by the user. This can be avoided by introducing real, physical clocks. We describe a simple method for synchronizing these clocks, and derive an upper bound on how far out of synchrony they can drift.

The Partial Ordering

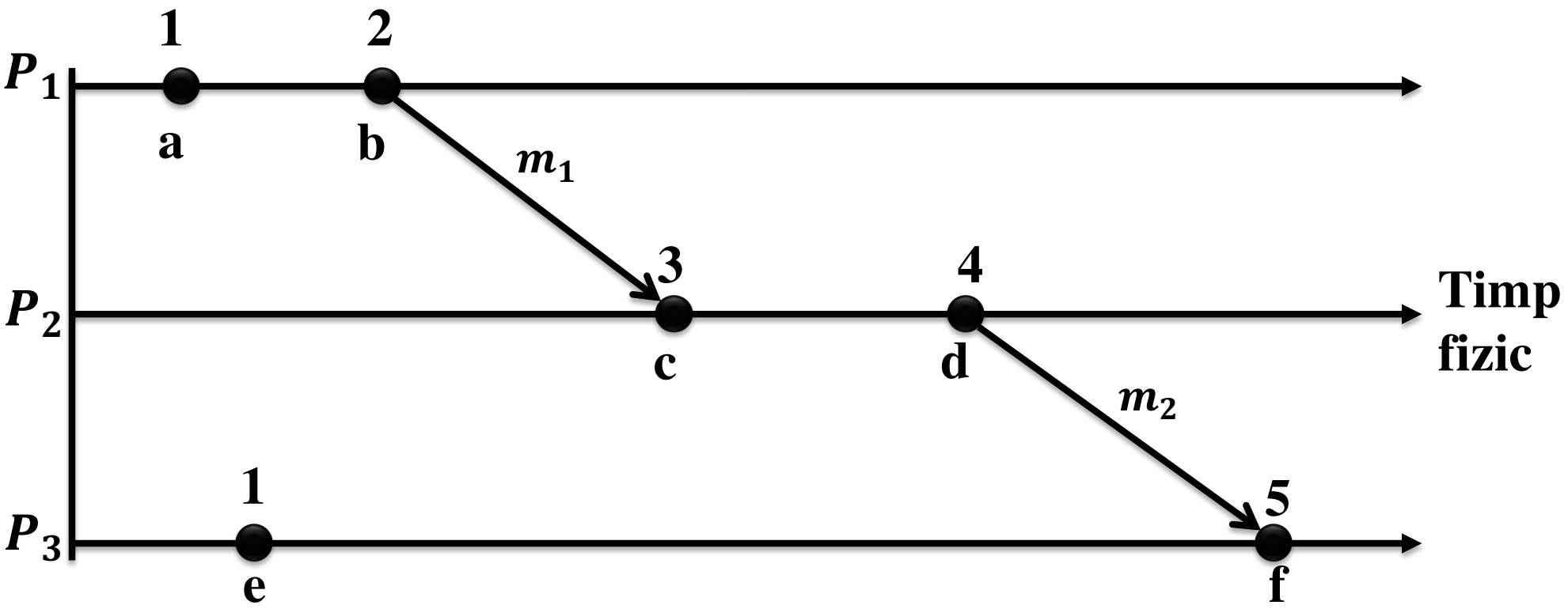


Lamport, Leslie. "Time, clocks, and the ordering of events in a distributed system." *Communications of the ACM* 21.7 (1978): 558-565.

Ceasuri logice și ordonarea evenimentelor (2)

- Ceas logic – întreg incrementat la producerea unui eveniment
- Algoritmul lui Lamport folosește relația → pentru a face ordonarea parțială a evenimentelor:
  - pentru fiecare proces o variabilă cl inițial 0
  - pentru fiecare mesaj un câmp tt
- Reguli:
  - la producerea unui eveniment intern:
    - cl este incrementat
    - valoarea ceasului logic este asociată evenimentului ca amprentă de timp
  - la transmiterea unui mesaj :
    - incrementează cl al procesului transmitator cu 1
    - actualizează tt = cl
  - la primirea unui mesaj cu amprenta de timp tt:
    - actualizează cl = maxim(cl, tt) + 1
- Ordonare parțială: $a \rightarrow b \Rightarrow cl(a) < cl(b)$

Exemplu



Aplicatie: semafoare distribuite

Semafoare distribuite

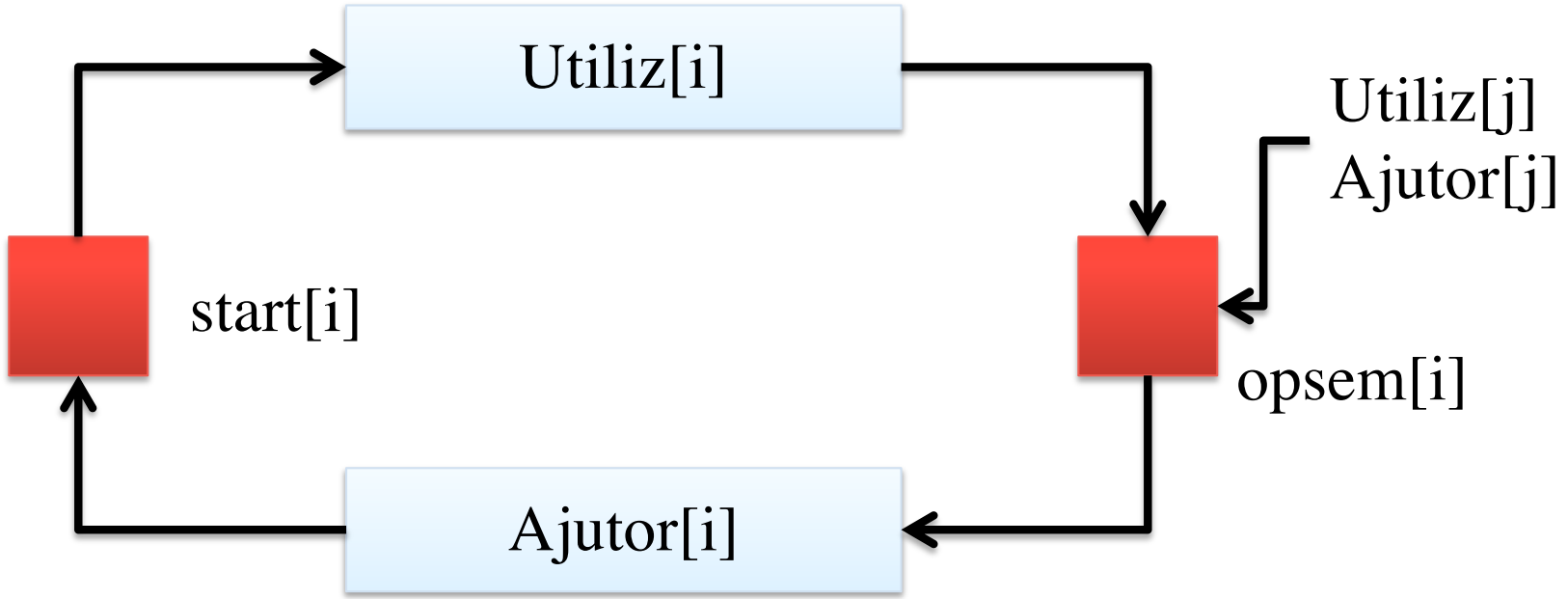
- Folosește primitive P și V
  - Când un proces execută o operaţie P sau V el difuzează mesaje m(id, tag, ts) celorlalte procese:

    broadcast ch(m) trimite m pe fiecare din ch[1:n]

    mesajul m are aceeasi amprenta de timp pe toate canalele
  - Analizează răspunsurile pentru a determina continuarea execuţiei
- Fiecare proces:
  - Stochează o coadă de mesaje mq și un ceas logic cl
  - La recepția unui mesaj P sau V îl stochează în mq, sortat crescător după ts
  - Transmite prin broadcast ACK *(folosit pentru a actualiza prefixele stabile din mq)*
  - Stochează o variabila s *(semaforul)*:
    - V – s este incrementat și mesajul șters
    - P – dacă $s > 0$, s decrementat și mesajul șters
      - mesajele P sunt procesate în ordinea în care apar în prefixul stabil deci fiecare proces ia aceeași decizie despre ordinea terminării operațiilor P

Aplicație: semafoare distribuite (2)

- Implementarea semafoarelor:
  - procesele Utiliz(i) inițiază operațiile P sau V
  - procesele Ajutor(i) implementează operațiile P și V



Aplicație: semafoare distribuite (3)

```
enum fel(V, P, ack);
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz[i= 1 to n] {
  int cl = 0;                          /* ceas logic */
  int ts;
  ...
  cl = cl + 1;
  broadcast opsem(i, V, cl);           /* operația V */
  ...
  cl = cl + 1;
  broadcast opsem(i, P, cl);           /* operația P */
  receive start[i](ts);
  cl = max(cl, ts)+1;
  ...
}
```

Aplicație: semafoare distribuite (4)

```
process Ajutor[i = 1 to n]{
    typedef struct{int transm, fel k, int ts) qelem;
    qelem qm[lmax]; /*coada ordonata dupa timestamp tm*/
    int cl = 0;
    int sem = valoare_inițială;
    int transm; fel k; int ts;
    while (true) {
        receive opsem[i](transm, k, ts);
        cl = max(cl, ts) + 1;
        if (k == P or k == V) {
            inserează (transm, k, ts) în locul corespunzător în qm;
            cl = cl + 1; broadcast opsem(i, ack, cl); }
        else if (k == ack) {
            înregistrează transmiterea unui ack;
            for [mesajele V complet confirmate] {
                scoate mesaj din qm;
                sem = sem + 1; }
            for [mesajele P complet confirmate st sem > 0] {
                scoate mesaj (transm, k, ts) din qm; // atentie la coliziuni
                sem = sem - 1;
                if (tansm == i) {cl = cl + 1; send start[i](cl);} }
        }
    }
}
```

Ceasuri logice vectoriale

Ceasuri logice vectoriale

Cu soluţia Lamport:

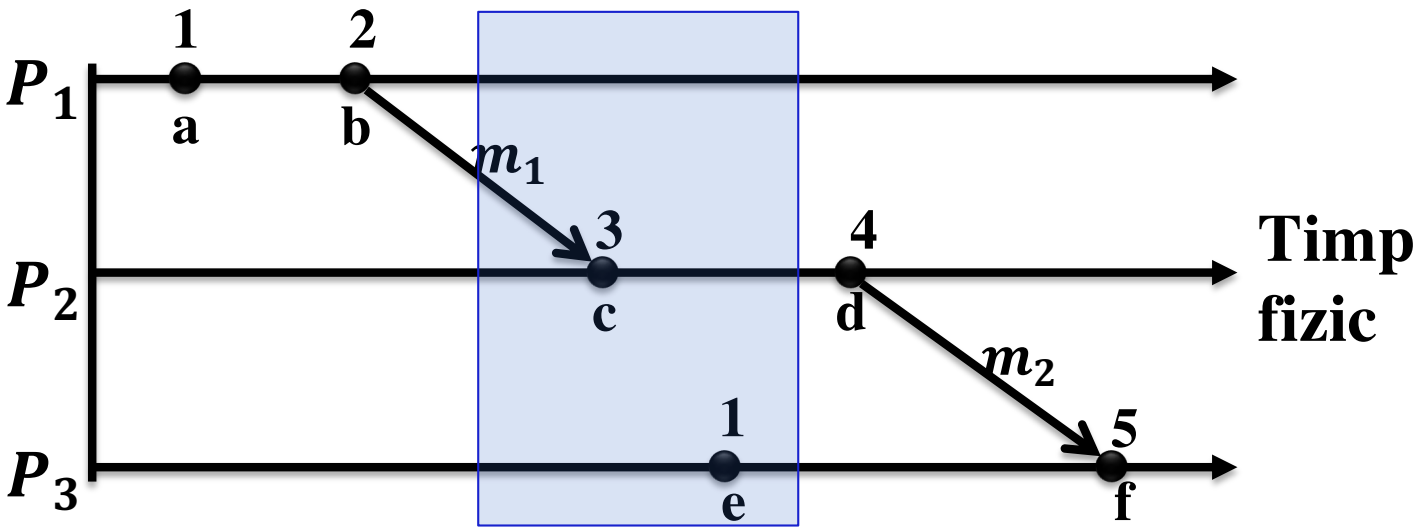
e precede f ⇒ amprenta_logică (e) < amprenta_logică (f)

Dar

amprenta_logică (e) < amprenta_logică (f) ⇏ e precede f

Ex: se poate spune ca **e** precede **c** ?

Soluția: ceasuri logice vectoriale.



Ceasuri logice vectoriale (2)

- Fiecare P_i are asociat un tablou $V_i[1..n]$ în care:
  - $V_i[i]$ este numărul de evenimente produse în procesul P_i
  - $V_i[j]$ este numărul de evenimente despre care P_i ştie *(a aflat)* că au avut loc la P_j.

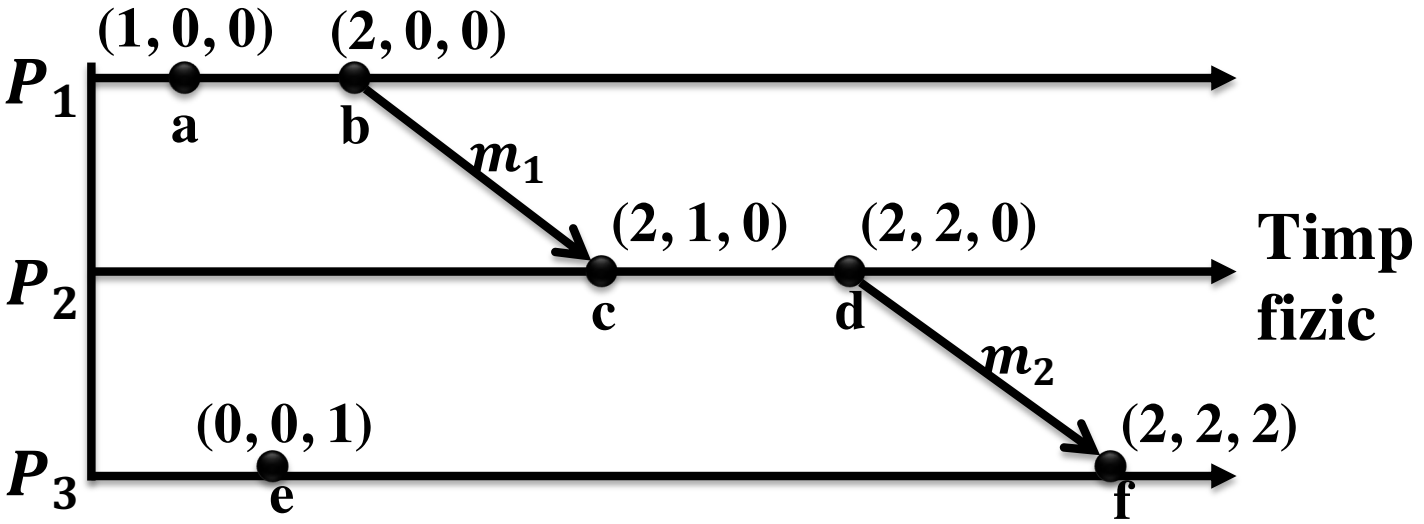
- Procesul P_i actualizează V_i la fiecare **eveniment** din P_i
  - e.g. pentru procesorul 3, (1,2,1,3) → (1,2,2,3)

- Când P_i **transmite** mesajul *m* *(eveniment send m):*
  - P_i incrementează $V_i[i]$
  - P_i adaugă V_i la *m* ca vector de amprente de timp curent $\boldsymbol{vt_m}$

- Când P_j **primeşte** *m* și $\boldsymbol{vt_m}$ *(eveniment receive m):*
  - P_j ajustează: $V_j[k] = max\{V_j[k], vt_m[k]\}$ pentru fiecare k

    e.g., P_2 primește un mesaj cu timpul (3,2,4) iar timpul curent al lui P_2 este (3,4,3), atunci P_2 ajusteaza timpul la (3,4,4)
  - P_j incrementează $V_j[i]$cu 1

Ceasuri logice vectoriale (3)



Aplicarea regulilor ceasurilor logice vectoriale

Reguli:

- $VT_1 = VT_2 \Leftrightarrow VT_1[i] = VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 \leq VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, pentru $i = 1, \dots, N$
- $VT_1 < VT_2 \Leftrightarrow VT_1[i] \leq VT_2[i]$, și $VT_1 <> VT_2$ *(de exemplu (1,2,2) < (1,3,2))*

Fie vt(a) şi vt(b) vectorii de amprente de timp asociaţi ev. a şi b. Atunci:

vt(a) < vt(b) $\Rightarrow$ **evenimentul a precede cauzal b**

vt(a) $\not<$ **vt(b) and vt(a)** $\not>$ **vt(b) and vt(a)** $\neq$ **vt(b)** $\Leftrightarrow$ **evenimentele a şi b sunt concurente**

Aplicație: Ordonare Cauzală Multicast

Ordonare Cauzală Multicast

- Procesele unei colecţii P comunică între ele doar prin mesaje cu difuzare
- Se cere ca mesajele să respecte **dependenţa cauzală**

$$m \rightarrow m' \Rightarrow livrarep(m) \rightarrow livrarep(m')$$

Protocolul *(vectori de timp)*:

- fiecare proces $P_i = (i = 1..n)$ are asociat un vector $V_i[1..n]$, cu toate elementele iniţial 0
- se numara doar operatiile de transmitere de mesaje
- $V_i[i]$, este nr ev. transmitere de mesaje produse de P_i;
- $V_i[j]$ este nr ev. transmitere de mesaje despre care $\boldsymbol{P_i}$ ştie (a aflat) că au avut loc la P_j.
- Procesul P_i actualizează V_i la fiecare eveniment de trimitere sau recepţie de mesaj din P_i.

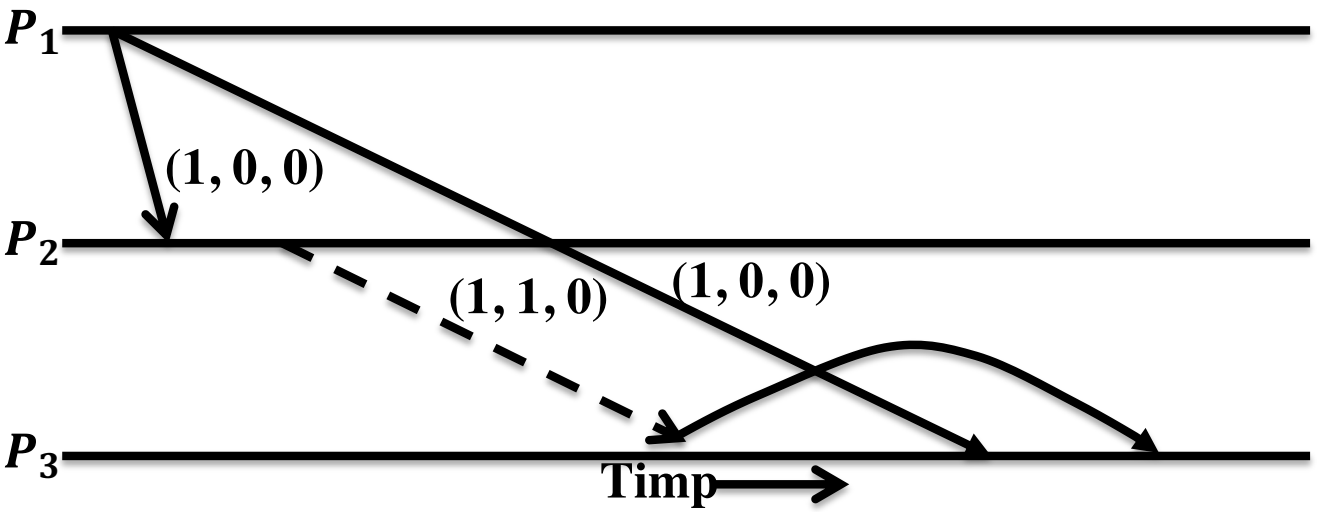
Ordonare Cauzală Multicast (2)

- Când P_S **transmite** mesajul *m*:
  - $\boldsymbol{P_S}$ **incrementează** $\boldsymbol{V_S[s]}$
  - $\boldsymbol{P_S}$ **adaugă** $\boldsymbol{V_s}$ **la** ***m*** **ca vector de amprente de timp curent** $\boldsymbol{vt_m}$
    - **Obs: Pentru ordonare cauzală, incrementarea lui $V_s[s]$ se face doar la transmiterea de mesaje de către s**
    - vt_m spune receptorului câte evenimente (din alte procese) au precedat **m** şi ar putea influenţa cauzal pe **m**.

- Când P_d **primeşte** mesajul *m* împreună cu vt_m, mesajul este păstrat într-o coadă de **întârziere** şi este **livrat** doar dacă:
  - $\boldsymbol{vt_s[s] = V_d[s] + 1}$ (acesta este următorul timestamp pe care **d** îl așteaptă de la **s**)
  - $\boldsymbol{vt_m[k] \leq V_d[k]\ pentru\ k <> s}$ (**d** a văzut toate mesajele ce au fost văzute de **s** la momentul când a trimis mesajul **m**)

- Când mesajul este livrat, V_d este actualizat conform regulilor vectorilor de timp:
  $\boldsymbol{V_d[k] = max\{V_d[k],\ vt_m[k]\}}$ **pentru fiecare** $\boldsymbol{k = 1, n}$.

Ordonare Cauzală Multicast (3)



Alte acţiuni ale protocolului la livrarea mesajelor:

- dacă $vt_m[k] > V_d[k]$ pentru un oarecare k atunci se întârzie *m*

P_S a primit mesaje de care mesajul curent poate fi cauzal dependent, dar pe care P_d încă nu le-a primit;

- dacă $vt_m[s] > V_d[s] + 1$ atunci se întârzie *m*

mai sunt mesaje de la P_S pe care P_s nu le-a primit

(asigură ordinea FIFO pentru canale nonFIFO)

- dacă $vt_m[s] < V_d[s]$ atunci rejectează *m*

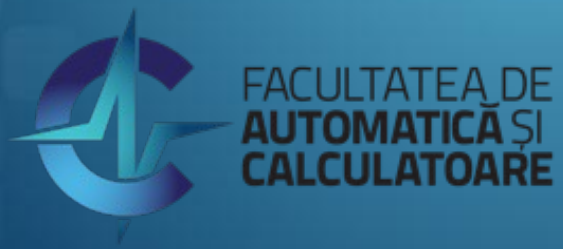
m este un duplicat al unui mesaj primit anterior

Sumar

- Ceasuri fizice
- Ceasuri logice și ordonarea evenimentelor
- Semafoare distribuite
- Ceasuri logice vectoriale
- Ordonare cauzală multicast

Algoritmi Paraleli și Distribuiți
Algoritmi undă

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro



Algoritmi undă

- În dezvoltarea algoritmilor distribuiți apar frecvent anumite tipuri de **probleme generale** pentru **rețele de procese**.
- Rezolvate prin transmitere de mesaje după o **schemă predefinită**, **dependentă de topologie** care asigură participarea **tuturor proceselor**.
- Justifică tratarea lor izolată de alți algoritmi în care aceste scheme pot fi folosite.

Notiuni preliminare - configuratii

- Un **program distribuit** consta dintr-o **colectie de procese** secventiale comunicante
- **Configuratie** = Ansamblul **starilor** tuturor proceselor la un moment dat
- O **operatie** modifica starea unui proces, implicit configuratia
- O executie a programului poate fi **modelata** printr-o secventa de **configuratii**.

- Ex.1 Program distribuit = colectie a doua procese, initial a si b au valoarea 0
- process P1{ int a =0; process P2{ int b =0;
  a = a+1;} b = b+1;}
- configuratia = ansamblul valorilor celor doua variabile modificate de procese → configuratia initiala este <0,0>
- daca operatia primului proces se executa mai intai, configuratia devine <1,0>
- O executie posibila = secventa de configuratii: (<0,0> , <1,0> , <1,1>)

Sisteme de tranziţii

- Un program distribuit poate fi modelat ca un **sistem de tranziții**:
  - Mulțimea tuturor stărilor (**configurațiilor**) posibile ale sistemului
  - Tranzițiile pe care sistemul le poate face între stări
  - Stările din care sistemul poate porni (inițiale)

- Formal, un **sistem de tranziții** este un triplet $S = (C, \rightarrow, I)$

  C este o mulțime de configurații

  $\rightarrow$ este relația de tranziție binară pe C

  I este setul configurațiilor inițiale (o submulțime a lui C)

- O **execuție** a lui S este o secvență maximală

  $$E = (\gamma_0, \gamma_1, \gamma_2, \ldots), \text{unde } \gamma_0 \in I \text{ și } \gamma_i \rightarrow \gamma_{i+1}, \ pentru\ i \geq 0.$$

- O configurație **terminală** γ nu are succesor:

  $$\nexists\ \delta \text{ astfel încât } \gamma \rightarrow \delta.$$

Sisteme de tranziţii (2)

- O secvență E este **maximală** dacă:
  - Este infinită sau
  - Sfârșește într-o configurație terminală.
- Configurația δ este **tangibilă** din γ dacă există o secvență $\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_k$ a.î.:

$$\gamma = \gamma_0$$
$$\delta = \gamma_k$$
$$\gamma_i \rightarrow \gamma_{i+1}, pentru\ i = \overline{0, k-1}$$

- Un **sistem distribuit** constă dintr-o colecție de procese și un subsistem de comunicații.
- **Convenții:**
  - pentru sistem: **tranziții** și **configurații**
  - pentru proces: **evenimente (interne / send,receive)** și **stări**
- Unei **execuții** E îi corespunde o **secvență de evenimente** din diferite procese.

Sisteme de tranziţii (3)

- Pentru o execuție E, **relația de ordine cauzală** $<$ este cea mai slabă relație care satisface:
  - dacă $\boldsymbol{a}$ și $\boldsymbol{b}$ sunt două evenimente diferite ale aceluiași proces și $\boldsymbol{a}$ se produce înaintea lui $\boldsymbol{b}$ atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a}$ este un eveniment send, iar $\boldsymbol{b}$ evenimentul receive corespunzător atunci $\boldsymbol{a} < \boldsymbol{b}$
  - dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{c}$ atunci $\boldsymbol{a} < \boldsymbol{c}$ (< este tranzitivă).
- Dacă $\boldsymbol{a} < \boldsymbol{b}$ și $\boldsymbol{b} < \boldsymbol{a}$ atunci a și b sunt **concurente**.

- O execuție E este **echivalentă** cu F ($E \sim F$) dacă:
  - au aceeași colecție de evenimente (ordinea diferă)
  - evenimentele respectă aceeași relație de ordine cauzală
  - ultima configurație a lui E coincide cu ultima configurație a lui F
- **Obs**: două execuții echivalente pot să nu aibă aceleași configurații.
- Un **calcul** (computation) al unui algoritm distribuit este o clasă de echivalență (sub relația $\sim$) a execuțiilor algoritmului.

Algoritmi undă (Wave algorithms)

- **Schema** de transmitere de mesaje:
  - dependentă de topologie
  - asigură participarea tuturor proceselor
- Algoritmii undă implementează astfel de scheme pentru:
  - difuzarea informației
  - realizarea unei sincronizări globale între procese
  - declanșarea unui eveniment în fiecare proces
  - calculul unei funcții în care fiecare proces participă cu o parte a datelor de intrare
- Proprietăți:
  - **terminare** – fiecare *calcul* este finit
  - **decizie** – fiecare *calcul* conține cel puțin un eveniment de decizie (*decide*)
  - **dependență** – în fiecare *calcul*, fiecare *decide* este precedat cauzal de un eveniment în fiecare proces

Definiții (1)

Calcul (computation) ⇔ **undă** (wave)

Categorii de procese:

Inițiatori (starters) – primul eveniment este unul intern sau un send

Neinițiatori (followers) – primul eveniment este un receive

Clasificare:

- centralizare
  - algoritmi centralizați – un inițiator
  - algoritmi descentralizați – set arbitrar de inițiatori

- topologie
  - inel, arbore, clică etc.
  - fixă (nu se produc modificări topologice)
  - nedirecționată (canale bidirecționale); excepțiile menționate explicit
  - conectată (există o cale între oricare două procese)

Definiții (2)

- cunoștințe inițiale – exemple:
  - identitatea proprie (nume)
  - identitățile vecinilor
  - **setul direcției (ordinea vecinilor)**
- numărul de decizii (regulă = cel mult o decizie în fiecare proces)
  - un singur proces decide
  - toate decid
  - unele decid
- complexitate
  - număr de mesaje schimbate
  - număr de biți interschimbați
  - timpul necesar pentru un calcul

Noțiuni preliminare
Sensul legăturilor

Complexitatea comunicării într-un algoritm distribuit depinde de topologie, dar și de:

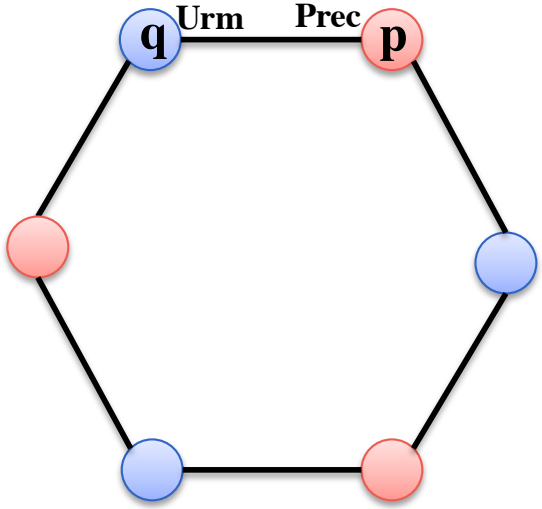
- cunoașterea topologiei la nivelul fiecărui nod (topological awareness)

- setul de direcții (sense of direction)
  - muchiile incidente unui nod sunt etichetate cu direcția spre care conduc în rețea
  - setul de etichete este același pentru fiecare nod
  - mărimea setului depinde de topologie (2 pt. inel, 4 pt. tor, etc)
  - trebuie respectată o **condiție suplimentară de consistență**

Noțiuni preliminare
Sensul legăturilor (2)

Inel:

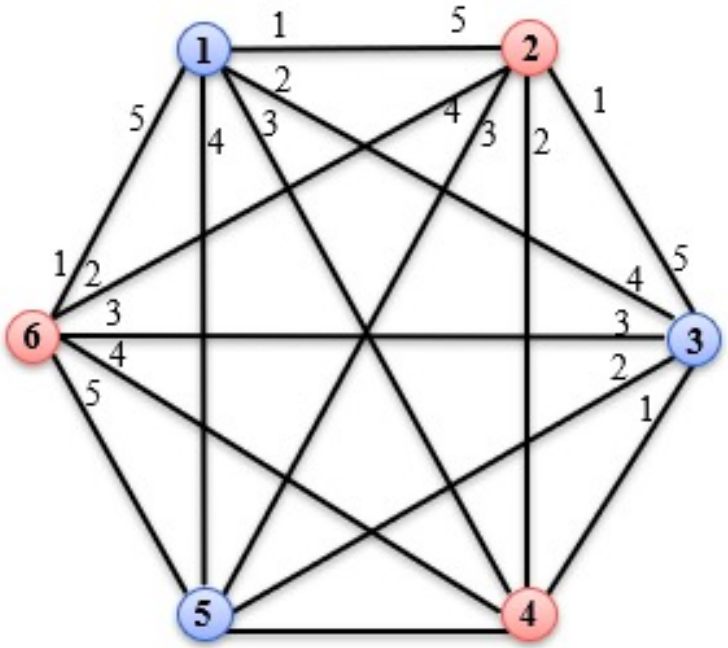
- 2 direcții: *Prec* (precedent) și *Urm* (următor)
- condiția de consistență: **precedentul lui p este q ⇔ următorul lui q este p**



Noțiuni preliminare
Sensul legăturilor (3)

Clică:

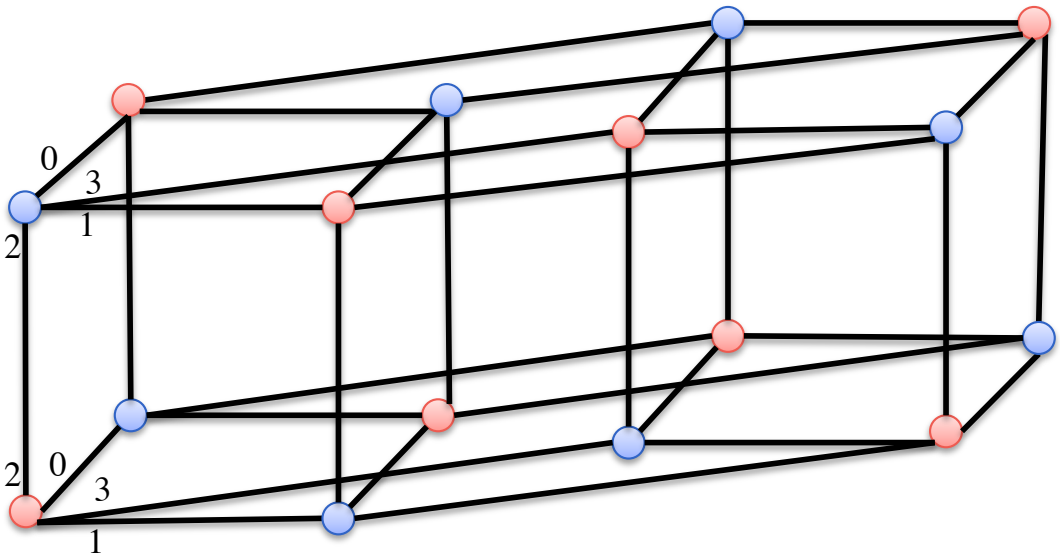
- N noduri de grad $N-1$
- set $direcții = \{1, 2, \ldots, N-1\}$
- condiția de consistență: **direcția de la nodul i la j are sensul $(j-i)\ mod\ N$**



Noțiuni preliminare
Sensul legăturilor (4)

Hipercub:

- pentru un hipercub n dimensional
- set $direcții = \{0, \dots n-1\}$
- condiția de consistență: **două noduri $(b_0, \dots, b_{n-1})$, $(c_0, \dots, c_{n-1})$ legate prin muchie cu eticheta i diferă doar în bitul i**



Notație transmitere mesaje

1. transmitere prin adresare directă

send ch[q] (mesaj)

- unde **q** este identitatea unică, globală a canalului către receptor

2. transmitere prin adresare indirectă

send ch[direcție] (mesaj)

- unde **direcție** este locală procesului care execută operația send
- identifică unul din canalele pe care procesul poate transmite

Algoritm inel

Algoritmul inel

- fiecare proces are un vecin dedicat, Urm
- transmiterea folosește **adresarea prin direcție** (Urm, Prec)
- toate canalele selectate prin Urm formează un ciclu Hamiltonian
- algoritmul este **centralizat**:
  - inițiatorul trimite un token (jeton) care este pasat de fiecare proces de-a lungul ciclului până ajunge înapoi la inițiator;
  - inițiatorul ia apoi decizia

Algoritmul inel

```
chan token[1..n](tok_type tok);
```

```
/* inițiator */
process P[I]{
        tok_type tok;
        send  token[Urm](tok);
        receive token[I](tok);
        decide
}
```

```
/* neinițiatori */
process P[k=1 to n, k<>I] {
        tok_type tok;
        receive token[k](tok);
        send token[Urm](tok)
}
```

Număr de mesaje = n, Timp = n

Algoritm arbore

Algoritmul arbore

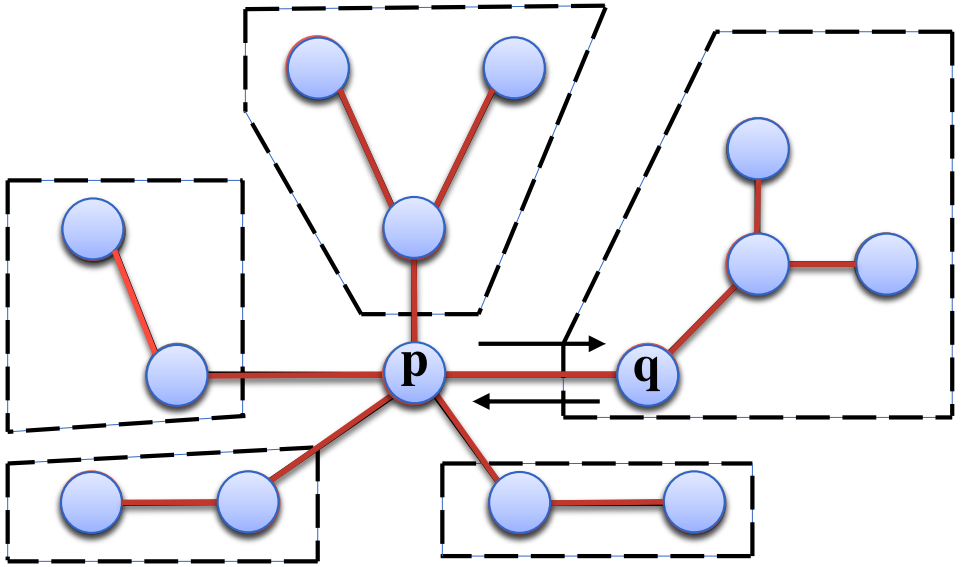
- Se aplică:
  - unei topologii arbore
  - unei topologii arbitrare în care se cunoaște un arbore de acoperire
- Fiecare nod cunoaște identitatea proprie și identitățile vecinilor
- Mulțimea tuturor identităților este **Ids**
- Pentru fiecare proces, se folosesc variabilele locale
  - Vecini – mulțimea identităților vecinilor (**q** = identitatea procesului q)
  - rec[q] – true dacă procesul a primit un mesaj de la vecinul q
- Inițiatorii sunt toate nodurile frunză
- **Algoritm:**
  - fiecare proces trimite exact un mesaj
  - când un proces a primit un mesaj pe fiecare canal incident mai puțin unul (condiție îndeplinită inițial de frunze) el trimite un mesaj pe canalul rămas
  - când un proces a primit câte un mesaj pe toate canalele sale atunci **decide**

Algoritmul arbore (2)

```
chan ch[1:n](int id, tok_type tok);
   /* fiecare proces are un canal propriu */
process Proc[p = 1 to n]{
   bool Vecini[1:n] = vecinii_lui_p;
   bool rec[1:n] = ([|n|] * false);
   int  r = numar_vecini_p;
   tok_type tok; int id, q0;
   while (r > 1){ receive ch[p](id, tok);
        rec[id] = true; r = r - 1 }
   /* de la un singur vecin, q0, nu s-a primit mesaj */
   afla q0 : Vecini[q0] and NOT rec[q0];
   send ch[q0](p, tok);
   receive ch[p](q0, tok); rec[q0] = true;
   decide
   /* informeaza celelalte procese despre decizie */
   /* for [q = 1 to n st Vecini[q] and q<>q0] send ch[q](p, tok); */
}
```

Număr de mesaje = N (egal cu nr. procese), Timp = O(D)

Algoritmul arbore (3)
exemplu de execuție

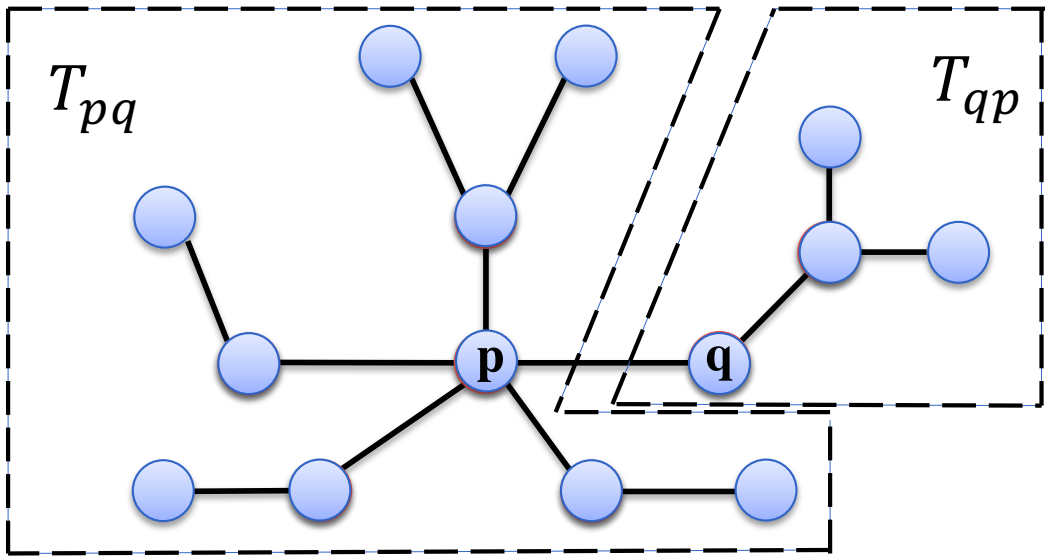


(b) q, după recepție, decid
transmit

Algoritmul arbore (4)

Algoritmul "arbore" este un algoritm undă.

1. calcul finit
   - algoritmul foloseşte cel mult N mesaje → algoritmul atinge o configuraţie terminală γ după un număr finit de paşi.
2. în γ, cel puţin un proces a executat un eveniment *"decide"*
3. *"decide"* este precedat de un eveniment în fiecare proces



Subseturile T_{pq} *şi* T_{qp}

Algoritm ecou

Algoritmul ecou

- se aplică unor **topologii arbitrare**
- este **centralizat**; există un singur inițiator, I
- propus de Chang; prezentam versiunea mai eficientă Segall
- bazat pe inundarea rețelei cu mesaje **tok**
  - se stabilește un arbore de acoperire
  - mesaje **tok** sunt transmise înapoi spre rădăcină prin canalele arborelui de acoperire

Algoritmul ecou

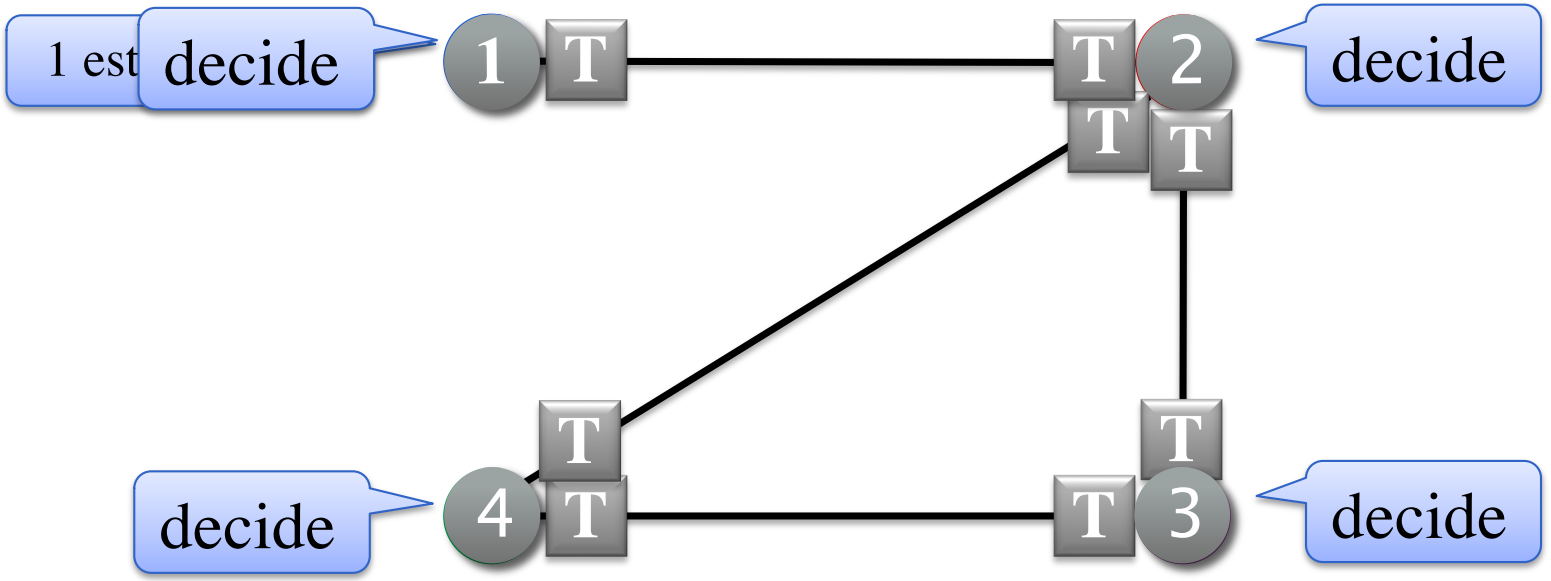
```
chan ch[1:n](int id, tik_type tok);
const int I = id_initiator;

process Proc(I){
   bool Vecini[1:n] = vecinii_lui_I;
   int r = numar_vecini_I, id;
   tok_type tok;

   for [i=1 to n st Vecini[i]] send ch[i](I, tok);
   while (r > 0) {
       receive ch[I](id, tok);
       r = r - 1;
   }
   decide;
}
```

Algoritmul ecou (2)

```
process Proc[p=1 to n, p <> I]{
   bool Vecini[1:n] = vecinii_lui_p;
   int r = numar_vecini_p, id, parinte;
   tok_type tok;

   receive ch[p](parinte, tok); r = r - 1;
   for [q=1 to n st Vecini[q] and q<> parinte)
       send ch[q](p, tok);
   while (r > 0) {
       receive ch[p](id, tok); r = r - 1; }
   send ch[parinte](p, tok)
}
```

$$Mesaje = 2|E|, Timp = O(N)$$

Algoritmul ecou (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Vecini =	{2}	{1, 3, 4}	{2, 4}	{2, 3}
părinte =		1	2	2
r =	0	0	0	0

Algoritmul fazelor
(pulsatiilor, heartbeat, gossiping..)

Algoritmul fazelor

- algoritm **descentralizat**
- **topologii arbitrare**
- **canale unidirecționale**
- vecinii sunt: **in**-vecini și **out**-vecini
- procesele cunosc diametrul grafului D (sau o valoare $D' > D$)
- fiecare proces trimite exact D mesaje fiecărui out-vecin
- mesajul $i + 1$ este trimis fiecărui out-vecin numai după ce i mesaje au fost primite de la fiecare in-vecin

Algoritmul fazelor

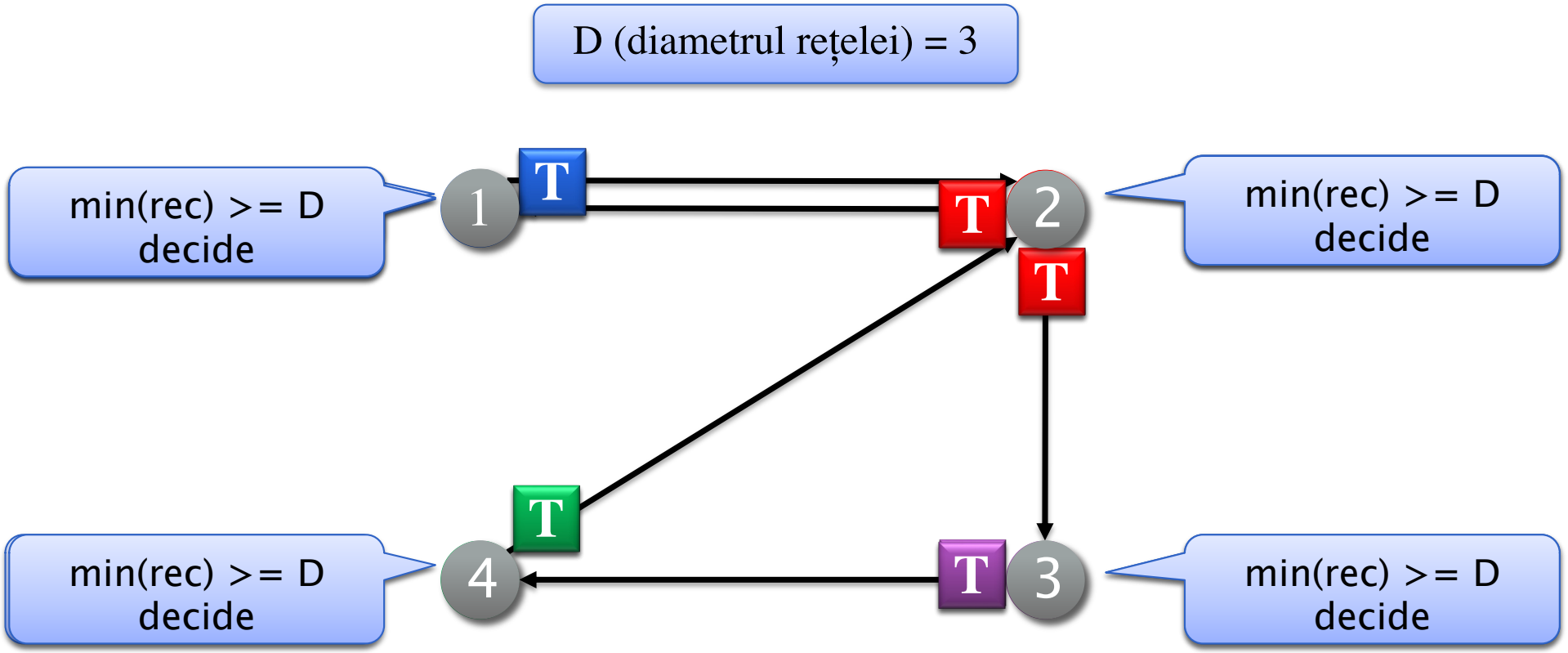
```
chan ch[1:n](int id, tok_type tok);
const int D = diametrul_retelei;

process Proc[p = 1 to n] {
   bool in[1:n] = in-vecinii_lui_p;
   bool out[1:n] = out-vecinii_lui_p;
   int rec[1:n] = ([n] 0);
   /* rec[q] = număr mesaje primite de la q */
   int sent = 0;
   /* număr de mesaje transmise fiecărui out-vecin */
   tok_type tok;
   int id;
```

Algoritmul fazelor (2)

```
    if (p este inițiator) {
        for [q = 1 to n st out[q]] send ch[q](p, tok);
        sent = sent + 1
    }

    /* min(rec) este min(rec[q], ∀q ∈ in) */
    while (min(rec) < D) {
        receive ch[p](id, tok); rec[id] = rec[id] + 1;
        if (min(rec) >= sent and sent < D) {
            for [q=1 to n st out[q]] send ch[q](p, tok);
            sent = sent + 1;
        }
    }
    decide;
}
```

Mesaje = 2D|E|, unde E este mulț. canalelor nedirijate (2 canale dirijate)
Timp = 2D

Algoritmul fazelor (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec[4] =	{2:3}	{1:3, 4:3}	{2:3}	{3:3}
sent =	3	3	3	3

Algoritmul fazelor (4)

Teoremă: Algoritmul "fazelor" este un algoritm undă

1. calcul finit

   fiecare proces trimite cel mult D mesaje prin fiecare canal ⇒

   algoritmul atinge o configurație terminală γ după un număr finit de pași

2. în γ, fiecare proces a executat un eveniment "decide"

   presupunem cel puțin un inițiator în C (pot fi mai mulți)

   2.1. fiecare proces a trimis cel puțin un mesaj

   2.2. fiecare proces a decis

3. "decide" este precedat de un eveniment în fiecare proces

- **T. Algoritmul "fazelor" este un algoritm unda**
- (1) **calcul finit**

  fiecare proces trimite cel mult D mesaje fiecarui out-vecin

  → algoritmul atinge o configuratie terminala **γ** intr-un calcul C cu numar finit de pasi
- (2) **in γ, fiecare proces a inregistrat un eveniment "decide"**

  presupunem cel putin un initiator in calculul C (pot fi mai multi)

  (2.1) fiecare proces a trimis cel putin un mesaj

```
if (p este initiator){
   for [q = 1 to n st out[q]]{
     send ch[q](p, tok); sent = sent+1;}

while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
  ...
```

daca p ne-initiator: **sent = 0** → cand p primeste primul mesaj, transmite unul tuturor out-vecinilor

(2.2) In configuratia terminala γ fiecare proces a decis

```
while (min(rec) < D){
  receive ch[p](id, tok); rec[id] = rec[id]+1;
  if (min (rec) >= sent and sent < D){
     for (q=1 to N st out[q]]
      send ch[q](p, tok);
     sent = sent+1;}
} decide;
```

Demonstram ca orice proces iese din ciclu, adica min(rec) = D , deci nu exista in-vecin care sa-i fi trimis mai putin de D mesaje

Fie p procesul cu cel mai mic $sent_p$ in γ ➔ ∀q $sent_q >= sent_p$ (1)

In particular, se aplica pentru toti in-vecinii q ai lui p

In γ, toate mes. trimise de q sunt recept. de p ➔ $sent_q = rec_p[q]$ (2)

(1)&(2) → $min_q (rec_p[q]) >= sent_p$ conditia din **if** este semi-indeplinita

➔ $sent_p$ =D; altfel p ar fi trimis mesaje suplimentare la ultima receptie

$sent_q$ =D ∀ q ➔ $rec_p[q]$ =D pentru orice canal qp

min(rec) = D ➔ fiecare proces a iesit din ciclu si a decis

- **(3) "decide" este precedat de un eveniment in fiecare proces**

  Ne referim la decizia din procesul **p**

  D este diametrul → pentru orice **q** exista o cale la **p** cu lungime <= D

  q = p(0), p(1), …, p(k) = **p**, cu lungimea k<=D

  Intr-o faza, transmisia precede receptia aceluiasi mesaj

  $f_{p(i)p(i+1)}^{(i+1)} \leq g_{p(i)p(i+1)}^{(i+1)}$ pentru 0<= i <k

  Din algoritm, receptia unui mesaj precede transmiterea lui in faza urmatoare

  $g_{p(i)p(i+1)}^{(i+1)} \leq f_{p(i+1)p(i+2)}^{(i+2)}$ pentru 0<= i <k-1

  Aplicand succesiv relatiile, rezulta $f_{p(0)p(1)}^{(1)} \leq g_{p(k-1)p(k)}^{(k)}$

  In plus, din algoritm, $g_{p(k-1)p(k)}^{(k)} \leq d_p$

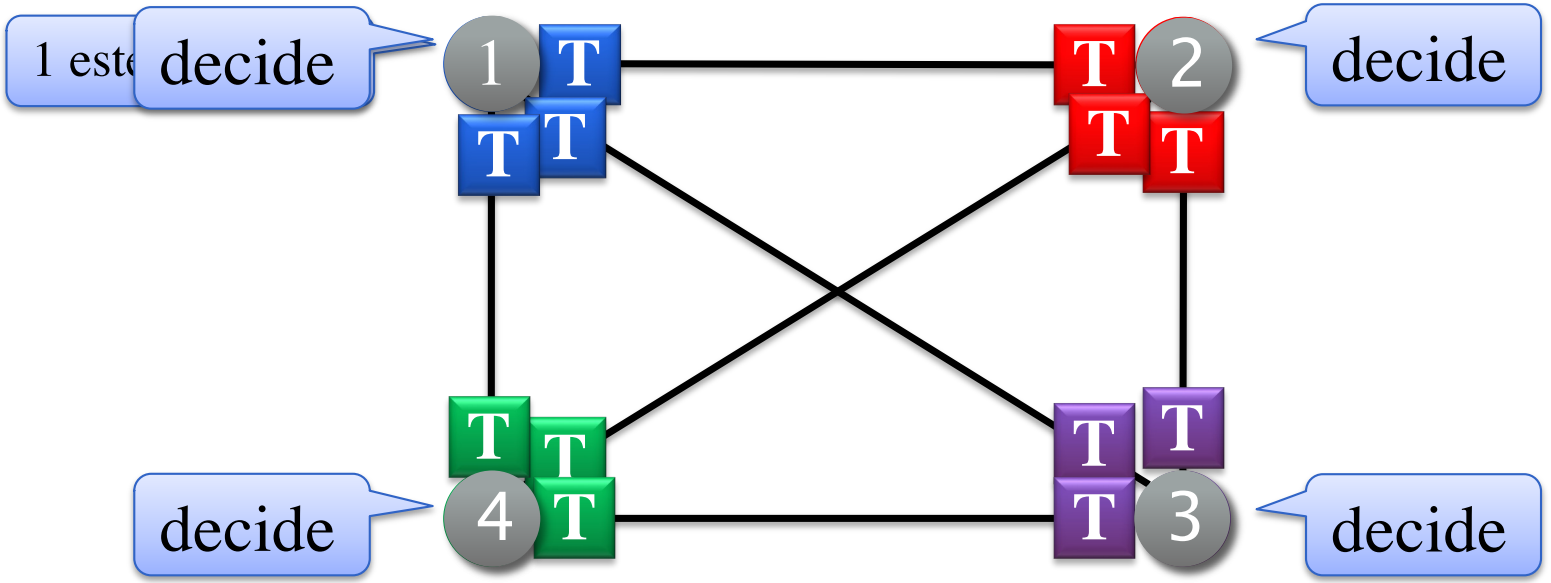
  ➔ decizia lui p este precedata de un eveniment in orice q: $f_{qp(1)}^{(1)} \leq d_p$

Algoritmul fazelor pentru clici

```
chan ch[1:n](int id, tok_type tok);
process Proc[p = 1 to N] {
   bool Vecini[1:n] = vecinii_lui_p;
   int n_vecini = numar-vecini_p;
   int rec = 0, sent = 0, id;
   tok_type tok;
   if (p este initiator) {
        for [q=1 to N st Vecini[q]] send ch[q](p, tok);
        sent = sent+1;
   }
   while (rec < n_vecini) {
        receive ch[p](id, tok); rec = rec + 1;
        if (sent == 0) {
              for [q=1 to N st Vecini[q]] send ch[q](p, tok);
              sent = sent + 1;
   }    }
   decide;
}
```

$$Mesaje = N(N-1), Timp = 2$$

Algoritmul fazelor pentru clici (2) exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
rec =	3	3	3	3
sent =	1	1	1	1

Algoritmul lui Finn

Algortimul lui Finn

- nu cere cunoașterea diametrului
- se bazează pe cunoașterea identificatorilor proceselor
- în mesaje se transmit seturi de identificatori de procese
- proc **p** păstreaza două mulțimi de ids

  Inc – mulțimea proceselor **q** pentru care un eveniment în **q** precede cel mai recent eveniment în **p**

  NInc – mulțimea proceselor **q** pentru care fiecare vecin **r** are un eveniment care precede cel mai recent eveniment în **p**

- **Algoritmul:**

  - inițial: $Inc = \{p\}, \quad NInc = \emptyset$
  - p trimite mesaje cu Inc și $NInc$ de fiecare dată când Inc sau $NInc$ crește
  - când p primește mesaje cu Inc și $\mathrm{N}Inc$, actualizează Inc și $\mathrm{N}Inc$ (reuniune)
  - când p a primit un mesaj de la toți in-vecini, p este inserat în $\mathrm{N}Inc$
  - când Inc devine egal cu $\mathrm{N}Inc$, p decide

Algortimul lui Finn (2)

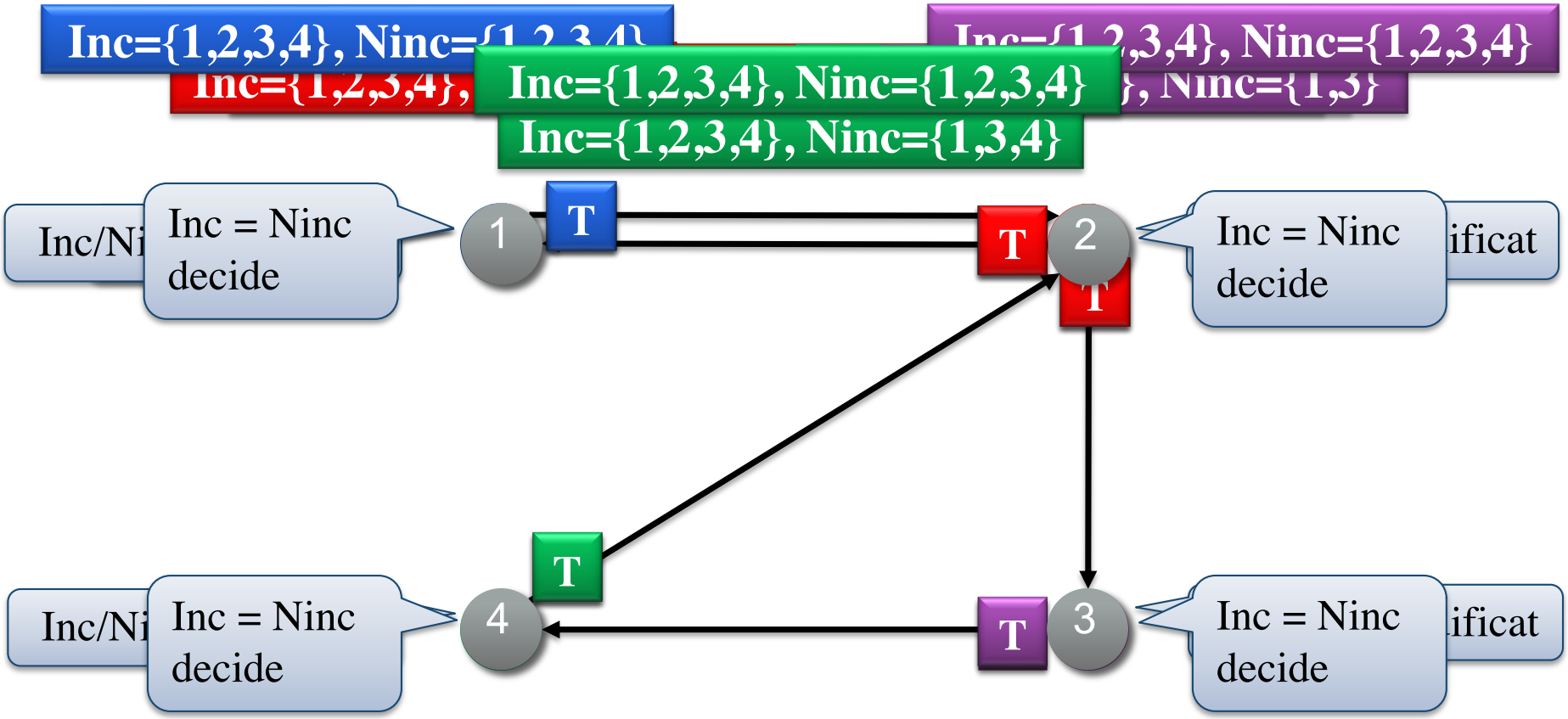
```
typedef SOP {multime identificatori};
chan ch[1:N](int id, SOP Inc, SOP NInc);

process Proc[p = 1 to n]{
   SOP Inc = {p};
   SOP Ninc = Ø;
   /* de la cine a recepționat */
   bool rec[1:n] = ([n] false);
   bool out[1:n] = out-vecinii_lui_p;
   bool in[1:n] = in-vecinii_lui_p;
   SOP rInc, rNInc;
   int id;

   if (p este initiator)
       for [q = 1 to n st out[q]]
          send ch[q](p, Inc, NInc);
```

Algortimul lui Finn (2)

```
   while (Inc <> Ninc) {
        receive ch[p](id, rInc, rNInc);

        Inc  = Inc  U rInc;
        NInc = NInc U rNInc;

        rec[id] = true;
        if (rec[id], for ∀id=1..n and in[id])
              NInc = NInc U {p};
        if (Inc sau NInc modificat)
              for [q=1 to n st out[q]]
                    send ch[q](p, Inc, NInc);
   }
   decide;
}
```

$$Mesaje \leq 2N|E|, \qquad Timp = O(D)$$

Algoritmul lui Finn (3)
exemplu de execuție



	Nodul 1	Nodul 2	Nodul 3	Nodul 4
Inc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
Ninc =	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}	{1, 2, 3, 4}
rec =	{2:T}	{1:T, 4:T}	{2:T}	{3:T}

Algoritmul lui Finn (4)

Teoremă: Algoritmul lui Finn este un algoritm undă

1. calcul finit

2. în γ, fiecare proces a executat un eveniment "decide“

   2.1. în γ fiecare proces a trimis cel puțin un mesaj pe fiecare canal (dem. similară alg. fazelor)

   2.2. în γ fiecare proces a decis

      2.2.1. $\forall p,\ Inc_p\ conține\ toate\ procesele\ (în\ \gamma)$

      2.2.2. $\forall p\ și\ q,\ NInc_p = NInc_q$

3. "decide" este precedat de un eveniment în fiecare proces

T. Algoritmul lui Finn este un algoritm unda

- (1) **calcul finit**
- Se trimite un mesaj la fiecare modificare Inc sau Ninc
- Dimensiunea celor doua seturi luate impreuna este initial 1 si ajunge la 2N
  → numarul de mesaje transmise este limitat superior la 4N|E|

- (2) **In γ, fiecare proces a executat un eveniment "decide"**
- 2.1. in **γ** fiecare proces a trimis cel putin un mesaj pe fiecare canal:

  p trimite tuturor vecinilor q → se schimba Inc_q → q transmite vecinilor sai; in plus graful este tare conex etc.
- 2.2. in **γ** fiecare proces a decis

  2.2.1. ∀ p, Inc_p contine toate procesele (in γ)

  daca exista arc pq, la modificare Inc_p, p il trimite lui q; q il include in Inc_q

  graf tare conex → pentru orice p si q: Inc_q include Inc_p

  impreuna cu Inc_p include Inc_q → $Inc_p = Inc_q$ pentru **∀** p si q

  → Inc_p contine identitatile tuturor proceselor

2.2.2. ∀ p si q, $NInc_p = NInc_q$

Fiecare proces a trimis un mesaj pe fiecare canal

→ fiecare proces a receptionat de la toti vecinii (∀ id € in: rec[id])

→ (cf alg) p este adaugat la NInc

$NInc_p$ contine toate procesele – demo similara cu Inc_p

→ $Inc_p = NInc_p$ in γ

→ fiecare proces a decis in γ

- (3) In procesul p, "decide" este precedat de un eveniment in fiecare proces

  Fie q un in-vecin al lui p

  Fie evenimentul intern lui q, **a_q: NInc = NInc U {q}**

  conform algoritmului, a_p se executa daca pentru fiecare in-vecin r al lui q un eveniment din r precede a_q

  Procesul p decide doar cand $Inc_p = NInc_p$ = setul tuturor proceselor

→ cand procesul p ia decizia:

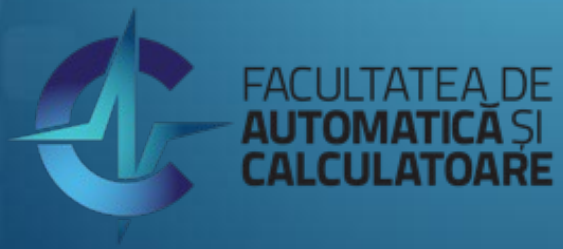
- p € Inc
- in-vecinul q € Inc si q € NInc
- deci decizia lui p este precedata de un eveniment din q si un eveniment din fiecare din in-vecinii lui q
- relatia este valabila pentru oricare doua procese din care unul este in-vecinul celuilalt
- retea conexa → decizia lui p este precedata de un eveniment din fiecare din celelalte procese

Sumar

- Sisteme de tranziții
- Caracteristici ale algoritmilor undă
- Algoritmul inel
- Algoritmul arbore
- Algoritmul ecou
- Algoritmul fazelor
- Algoritmul lui Finn

Algoritmi Paraleli și Distribuiți
Alegerea unui lider

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



Alegerea unui lider

- Gerard LeLann a propus problema alegerii unui lider într-un paper celebru din 1977
- Multe sisteme distribuite sunt bazate pe paradigma client-server:
  - Un server/coordonator (lider)
  - Mai mulți clienți
- Ce se întâmplă dacă liderul se defectează?
  - Soluție: se alege un nou lider
- Problema generică: dintr-o colecție de procese, se alege unul singur care urmează să joace un rol special *(lider)*:
  - Execută operații de inițializare
  - Controlează alte procese

INFORMATION PROCESSING 77, B. GILCHRIST, EDITOR
© IFIP, NORTH-HOLLAND PUBLISHING COMPANY (1977)

DISTRIBUTED SYSTEMS—TOWARDS A FORMAL APPROACH

GÉRARD LE LANN
IRISA—Université de Rennes—BP 25 A
35 031 Rennes Cedex, France

Packet-switching computer communication networks are examples of distributed systems. With the large scale emergence of mini and micro-computers, it is now possible to design special or general purpose distributed systems. However, as new problems have to be solved, new techniques and algorithms must be devised to operate such distributed systems in a satisfactory manner. In this paper, basic characteristics of distributed systems are analysed and fundamental principles and definitions are given. It is shown that distributed systems are not just simple extensions of monolithic systems. Distributed control techniques used in some planned or existing systems are presented. Finally, a formal approach to these problems is illustrated by the study of a mutual exclusion scheme intended for a distributed environment.

1. INTRODUCTION

Computer communication networks using packet-switching technology provide for the interconnection of data-processing equipments of any kind. Such systems, sometimes simply referred to as computer networks, may be viewed as multi-macroprocessors whenever the goals of resource-sharing are achieved. With the large-scale emergence of mini and microcomputers, it is now possible to envision building general or special purpose multimini and multimicrocomputers to be operated in a non-centralized manner. The need for automatic resource-sharing arises here as in a similar way it does for multimacroprocessor systems.

Two kinds of resources must be considered :
- system resources, multi-accessed by users and for which multiplexing is required (hidden sharing)
- user resources, which users agree to share according to some protocol of their own (explicit sharing).

This paper discusses the problems of system resource-sharing in a distributed environment. An example of a user-sharing problem is distributed data-base sharing.

2. DISTRIBUTED SYSTEMS-ELEMENTS FOR A FORMAL APPROACH

Experimental and public packet-switching computer

cesses called logical entities.

Let $F = \{f_i, i \in I\}$ be the set of the operating functions and $E_i = \{e_j^i, j \in J(i)\}$ be the set of entities participating in fuction f_i. At any instant t, it is possible to define $s_t(e_j^i)$ as the instantaneous state of entity e_j^i. It is therefore theoretically possible to define the global state of E_i at instant t as the vector $S_t(E_i) = \{..., s_t(e_j^i), ...$ for all $j \in J(i)\}$.

A system will be said to be f_i-centralized if there exists $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

An example is a system in which $\dim(E_i) = 1$; another example is a system in which entities run the operating function f_i by using a common "system table".

A system will be said to be totally centralized if it is f_i-centralized for any $i \in I$.

In contrast, a system will be said to be f_i-distributed if there does not exist $k \in J(i)$ such that $S_t(E_i)$ is known to e_k^i.

A system will be said to be totally distributed if it is f_i-distributed for any $i \in I$.

Typical cases of distributed systems are systems in

Le Lann, Gérard. "Distributed Systems-Towards a Formal Approach." *IFIP congress*. Vol. 7. 1977.

Alegerea unui lider

- Desemnarea statică a liderului ne-aplicabilă
  - nu se cunoaște compoziția exactă a grupului de procese
- Fiecare proces își cunoaște propria identitate și vecinii.
- Identitățile proceselor aparțin unei mulțimi total ordonate
- Alegerea liderului = determinarea procesului care are identitatea cea mai mică *(sau cea mai mare)*.
- Alegerea se face prin algoritmi descentralizați, cu participarea tuturor proceselor din colecție.

Alegerea unui lider

- Problemă:
  - Atunci când un proces se defectează, mai multe procese pot depista problema simultan
- Nu putem presupune că un singur proces detectează defectul
  - Algoritmul de alegere trebuie să conducă în final la situația în care toate procesele decid asupra aceluiași proces ca nou lider

Alegerea unui lider cu algoritmi undă

Caracteristici:

- toate procesele au același algoritm local
- algoritmul este **descentralizat** *(poate fi inițiat de oricare subset de procese)*
- în fiecare calcul algoritmul atinge o configurație finală în care:
  - un proces este lider
  - toate celelalte au pierdut
- varianta mai slabă:
  - un proces este lider
  - celelalte procese nu știu că au pierdut

Alegerea unui lider cu algoritmi undă (2)

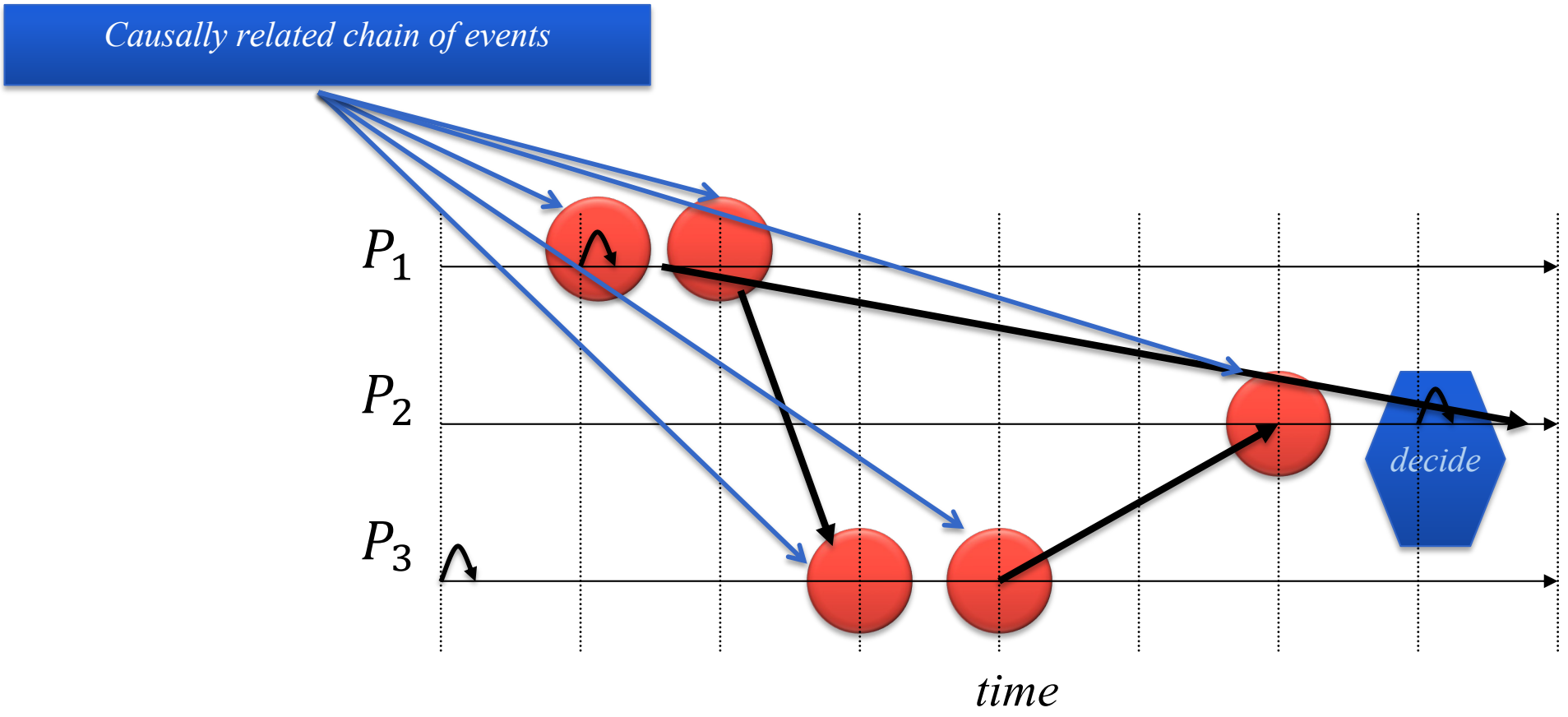
- Fiecare proces are o **stare** dintre:
  - leader
  - lost
  - sleep *(procesul nu a executat nici o acțiune)*
  - candidate *(a executat acțiuni dar nu e sigur dacă e leader sau lost)*

- Ipoteze:
  - Sistemul este asincron *(comunicație asincronă, fără timp global, timpi arbitrari de transmisie)*
  - fiecare proces este identificat printr-un nume unic
  - Identitățile sunt extrase dintr-o **mulțime total ordonată**
  - aflarea liderului se transformă în **aflarea procesului cu cel mai mic identificator**

Algoritmi undă pentru alegerea liderului

- Algoritmii undă au cele trei proprietăți:
  - Terminare
    - Algoritmul se termină într-un număr finit de pași
  - Decizie
    - Cel puțin un nod execută un eveniment *decide*
  - Consistență
    - *Oricare* proces execută cel puțin un eveniment ce precede cauzal evenimentul *decide*

Proprietățile algoritmilor undă



Folosirea algoritmilor undă pentru alegerea liderului

- Ideea:
  - Fiecare proces are atașată o identitate în cadrul undei
    - Un nod decide atunci când un eveniment a precedat cauzal fiecare eveniment prin care procesele adaugă identitatea respectivă în vectorul de identități cunoscute
    - Procesul ce decide va cunoaște identitatea tuturor proceselor
    - Se alege procesul având identitatea cea mai mică ca nou lider
    - Se anunță *(broadcast)* identitatea liderului tuturor proceselor

Exemplu: alegere leader folosind Algoritmul Arbore

Alegerea unui lider cu algoritmi undă (3)

Alegerea liderului cu algoritm tree

- algoritmul impune ca cel puțin toate frunzele să fie inițiatori ⇒ adaugă o fază de *wakeup:*
  - inițiatorii trimit mesaje *wakeup* tuturor proceselor
  - fiecare proces folosește variabilele:
    - **ws** – boolean, asigură că fiecare proces transmite mesaje *wakeup* cel mult o dată
    - **wr** – int, contorizeaza mesajele de *wakeup* recepționate
- când un proces a primit *wakeup* prin fiecare canal, el începe algoritmul de alegere
- când un proces decide, el află identitatea liderului
- în funcție de ea procesul trece în starea leader sau lost

Algoritmul tree

```
chan ch[1:n](int id_transm, in min_id);
chan wakeup[1:n]();
process Proc[p = 1 to n]{
    bool ws = false;
     int wr = 0, r = număr_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
     bool rec[1:n] = ([n] * false);
    int V = p;
    enum state{sleep, leader, lost};
     state stare = sleep;
     int id;
    /* aici începe faza wake-up */
    if (p este initiator) {
        ws = true;
        for [q = 1 to r st Vecini[q]]
            wakeup[q]();
    }
```

Algoritmul tree (2)

```
while (wr < număr_vecini_p){
     receive wakeup[p]();
     wr = wr + 1;
     if (NOT ws) {
          ws = true;
          for [q = 1 to n st Vecini[q]
               send wakeup[q]();
}    }

/* aici începe algoritmul tree */
while (r > 1){
     receive ch[p](q, id);
     rec[q] = true; r = r - 1;
     V = min(V, id);
}
```

Dacă este primul wakeup primit ⇒ trimit wakeup la toți vecinii

Aștept să primesc un răspuns de la N – 1 vecini și fac un minim între răspunsurile acestora.

N = numărul de vecini

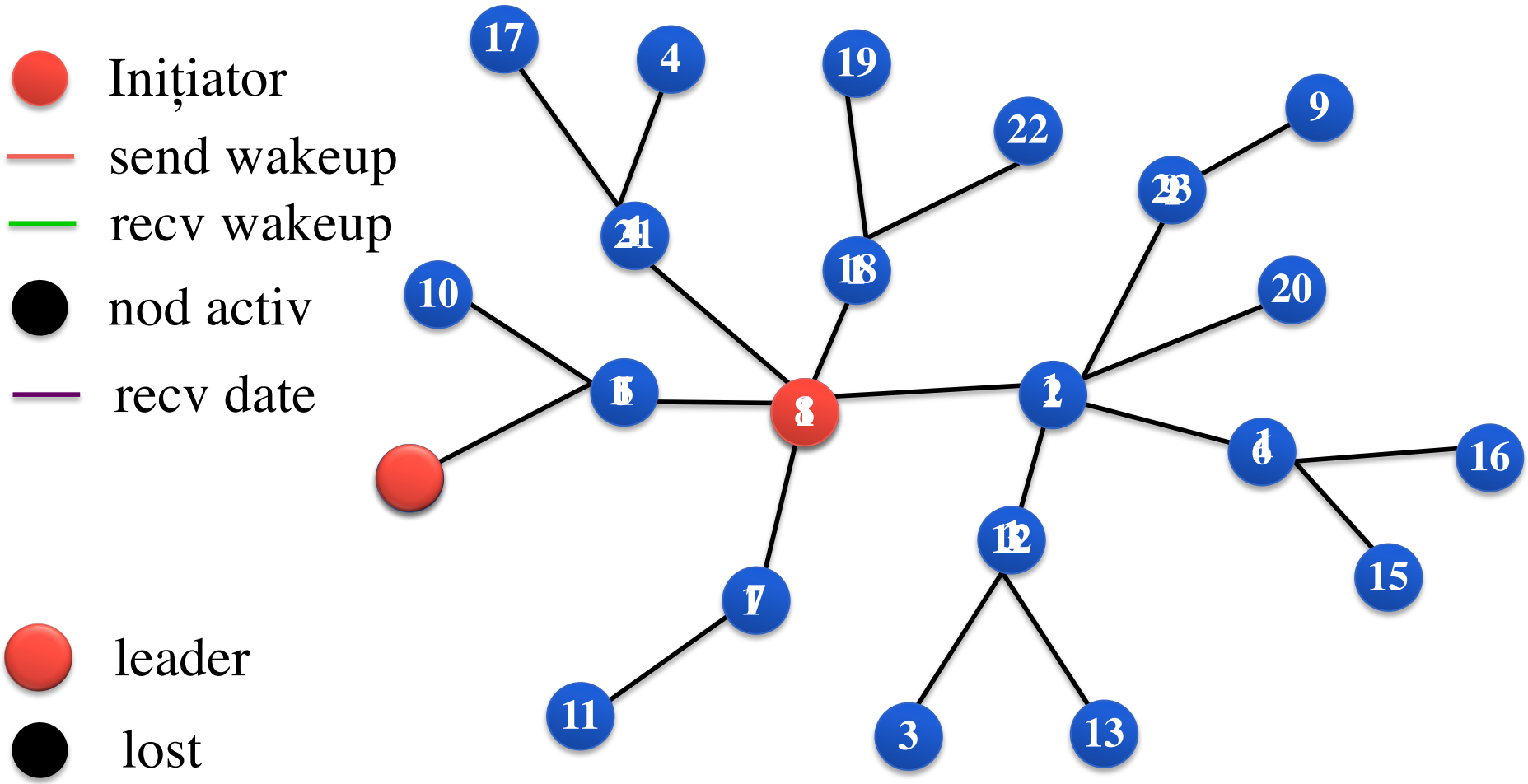
Algoritmul tree (3)

```
/* de la un singur vecin, q0 nu s-a primit mesaj */
afla q0: Vecini[q0] and NOT rec[q0];
send ch[q0](p, V);
receive ch[p](q0, id);
V = min(V, id);
if (V == p)
    stare = leader
else
    stare = lost

/* informeaza celelalte procese despre decizie */
for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,V);
```

Aștept rezultatul final de la ultimul nod

primit răspuns

Algoritmul tree (4)



Algoritmul tree (5)

- *Notații :*
  - **N** : Numărul de procese implicate
  - **D** : Diametru rețelei formate de procesele implicate

- *Numărul de mesaje* = O(N)
  - pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ⇒ total = 4N - 4

- *Timp* = O(D)
  - în D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup* ⇒ în D+1 pași fiecare proces a pornit unda.
  - Prima decizie se ia la D pași după pornire, ultima după alți D pași ⇒ totalul = 3D+1 pași

Alegere lider – cazul topologiilor inel

Alegerea liderului - topologie inel

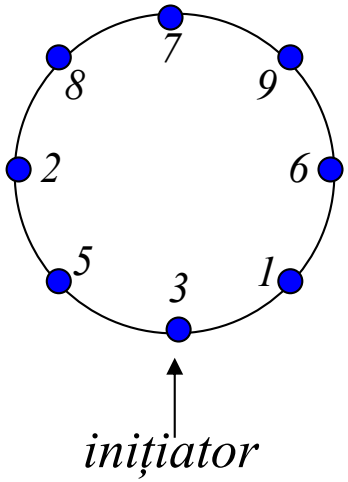
Specificarea problemei:

- Se dă un aranjament circular de procese identificate prin numere distincte între ele.
- Dispunerea proceselor în inel este oarecare.
- Se cere desemnarea prin conses a procesului cu id **maxim**.
- Nu se cunoaște apriori numărul de procese.
- Nu există un control centralizat.

- Idee simplă:
  - Permitem fiecărui inițiator să trimită un token cu propria identitate pe inel
  - Nodurile nu mai au voie să inițieze după ce au primit un token

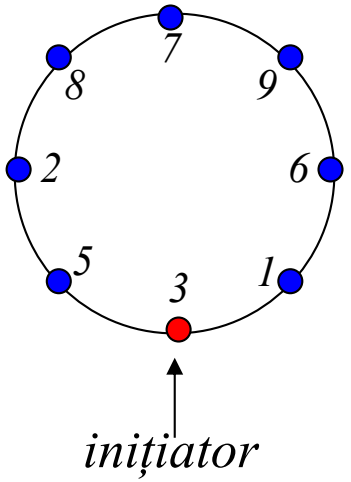
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



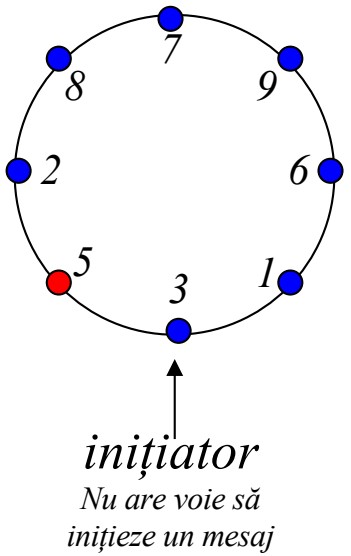
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



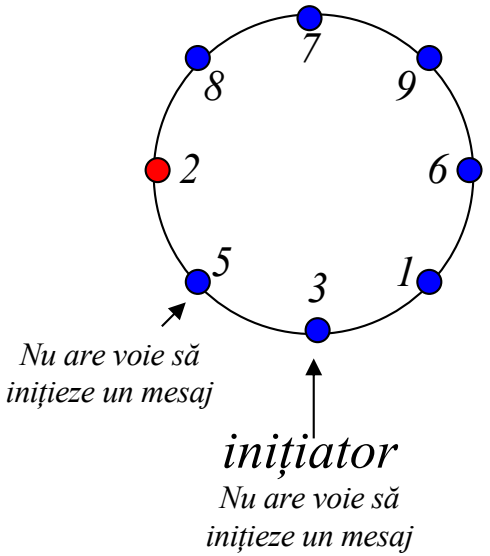
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



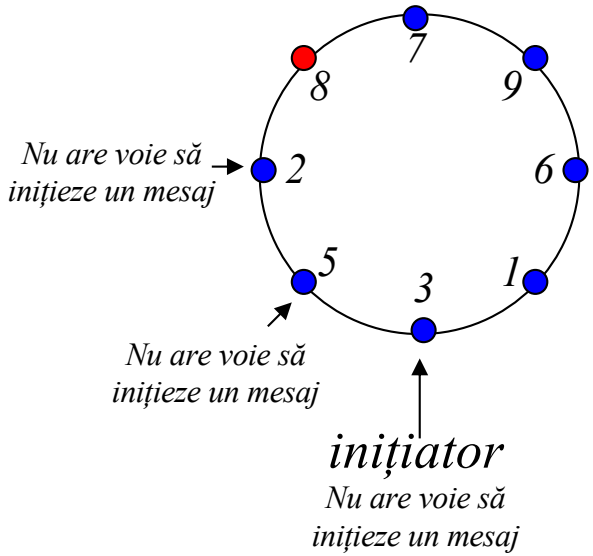
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



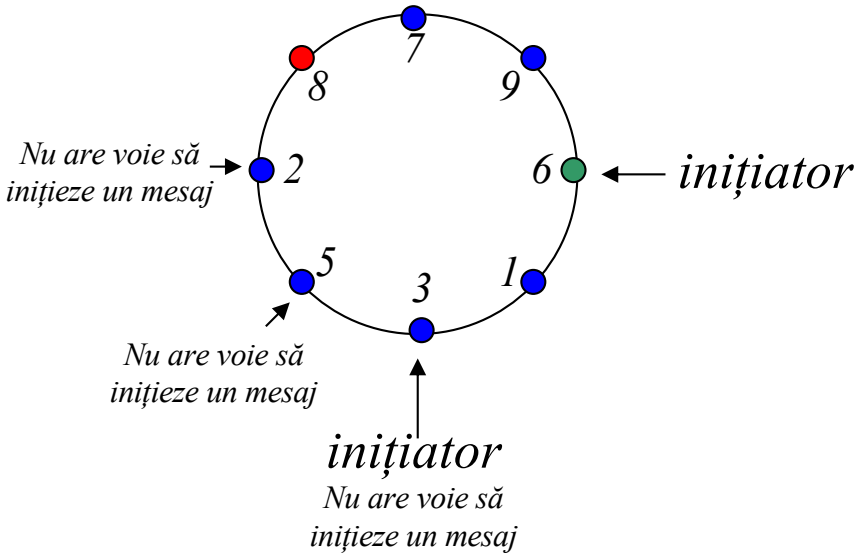
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



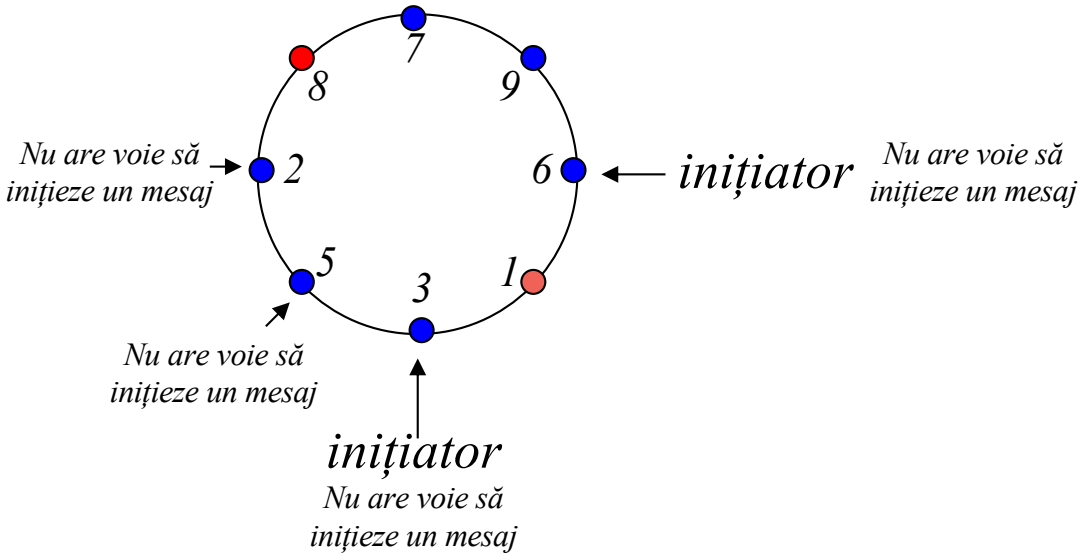
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



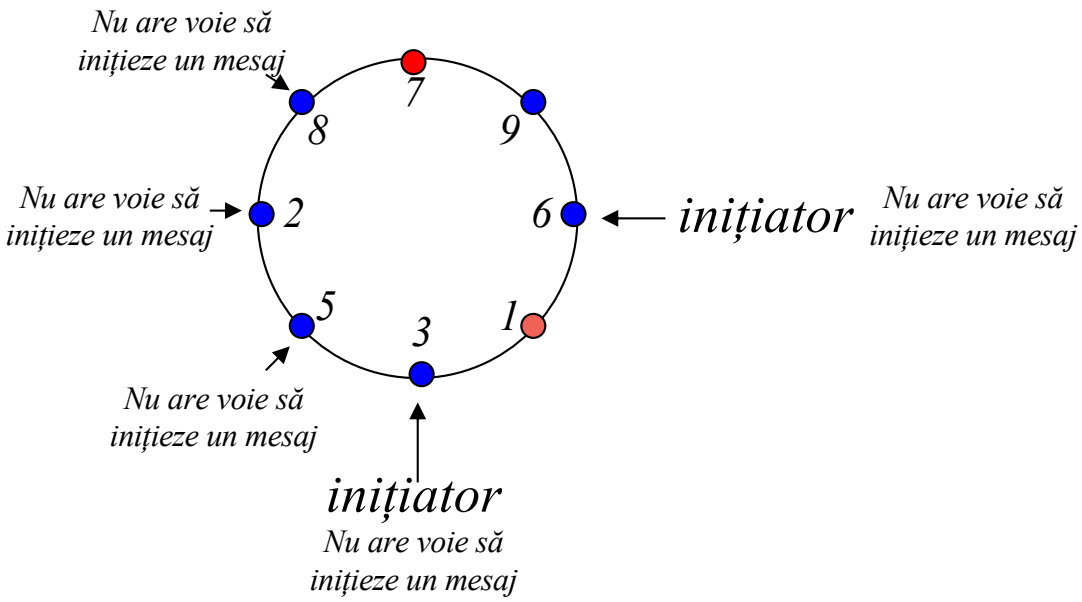
Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Exemplu

- Nodurile *nu* mai au voie să inițieze un mesaj după ce au primit un token...



Algoritmul LeLann

Algoritmul LeLann

Atunci când un nod primește propriul id, a văzut deja toate celelalte id-uri

- Presupunând canale FIFO

  ⇒ Fiecare proces poate reține o listă cu identificatorii pe care i-a primit...

Algoritmul **LeLann:**

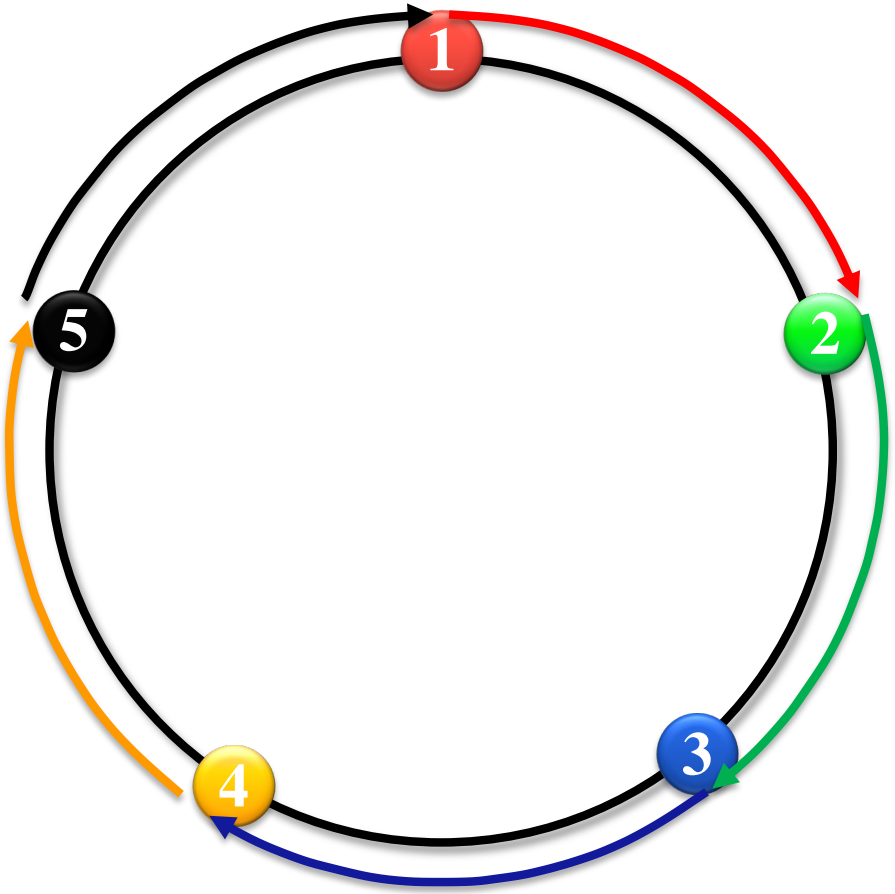
- Fiecare proces difuzează celorlalte un mesaj cu **id**-ul său
- Fiecare proces colectează numerele celorlalte procese
- Fiecare proces află maximul.
- Procesul al cărui număr este egal cu maximul devine lider.

Algoritmul LeLann

```
chan ch[1:n](token tok, int id_mare);
process Proc[p = 1 to n]{
   typedef SOP{multime identificatori};
   SOP List = {p};
   enum state (candidate, leader, lost);
   state stare = candidate;
   send ch[Urm](tok, p);
   receive ch[p](tok, q);
   while (q <> p) {
        List = List U {q};
        send ch[Urm](tok, q);
        receive ch[p](tok, q);
   }
   if (p == max(List)) stare = leader
   else state = lost
}
```

Număr de mesaje =? $O(N^2)$

Algoritmul LeLann



Nod	ID-uri colectate	Status	
1	{5,4,3,2,1}	Lost	5
2	{1,5,4,3,2}	Lost	5
3	{2,1,5,4,3}	Lost	5
4	{3,2,1,5,4}	Lost	5
5	{4,3,2,1,5}	Leader	5

Algoritmul Lelann-Chang-Robert

Operating Systems

R. Stockton Gaines
Editor

An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes

Ernest Chang
University of Toronto

Rosemary Roberts
University of Waterloo

This note presents an improvement to LeLann's algorithm for finding the largest (or smallest) of a set of uniquely numbered processes arranged in a circle, in which no central controller exists and the number of processes is not known a priori. This decentralized algorithm uses a technique of selective message extinction in order to achieve an average number of message passes of order (n log n) rather than $O(n^2)$.

Key Words and Phrases: decentralized algorithms, distributed systems, operating systems

CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

Given a random circular arrangement of uniquely numbered processes where no a priori knowledge of the number of processes is known, and no central controller is assumed, we would like a method of designating by consensus a single unique process. The algorithm we propose works equally well for finding either the highest numbered or the lowest numbered process. Let us, without loss of generality, consider highest finding.

A situation in which this algorithm is important has been presented by LeLann [1]. In his example, a circle of controllers in which the control token is lost causes every controller to time out, and an election to find a new emitter for the control token is performed. LeLann's algorithm requires every controller to send a message bearing its number. Each controller thus collects, through the messages seen, the numbers of the other controllers in the circle. Every controller sorts its list, and the controller whose own number is the highest on its list is elected.

LeLann's algorithm, in a circle with n controllers, requires total messages passed proportional to n^2, written $O(n^2)$, where a message pass is a SEND of a message from a controller. This is clearly so, since each of the n controllers sends a message which is passed to all other nodes. Our algorithm requires, on the average, $O(n \log n)$ message passes.

The Algorithm

Each process is assumed to know its own number, and initially it generates a message with its own number, passing it to the left. A process receiving a message compares the number on the message with its own. If its own number is lower, the process passes the message (to its left). If its own number is higher, the process throws the message away, and if equal, it is the highest numbered process in the system.

Proposition: This algorithm detects one and only one highest number.

Argument: By the circular nature of the configuration and the consistent direction of messages, any message

Chang, E., & Roberts, R. (1979). An improved algorithm for decentralized extrema-finding in circular configurations of processes. *Communications of the ACM*, *22*(5), 281-283.

Algoritmul Lelann-Chang-Roberts

- Mesajele sunt transmise în inel **în sensul acelor de ceasornic.**
  - Fiecare proces transmite procesului din dreapta un mesaj cu identificatorul său
  - Un proces care primește un mesaj **m** îl compară cu identificatorul propriu *(id)*:
    - **if** m > id -> transmite m în dreapta
    - **if** m < id -> elimină m
    - **if** m = id -> procesul curent devine lider

- ***Propoziție:***
  - Algoritmul detectează un singur cel mai mare număr

Algoritmul Lelann-Chang-Robert (2)

```
chan ch[1:n](token tok, int id_mare);

process Proc[p = 1 to n] {
    enum state(candidate, leader, lost);
    state stare = candidate;
    send ch[Urm](tok, p);
    while (state <> lider) {
        receive ch[p](tok, q);
        if (q == p) stare = leader;
        else if (q > p) {
            if (stare == candidate)
                stare = lost;
            send ch[Urm](tok, q);
        }
 }   }
```

Algoritmul Lelann-Chang-Roberts (3)

- Observații:
  - un proces **lost** ramâne în buclă pentru a pasa alte mesaje
  - doar procesul cu id maxim termină
  - pentru a debloca alte procese, liderul poate trimite un mesaj special

- *Algoritm modificat*:

  Când nu toate procesele sunt inițiatori:

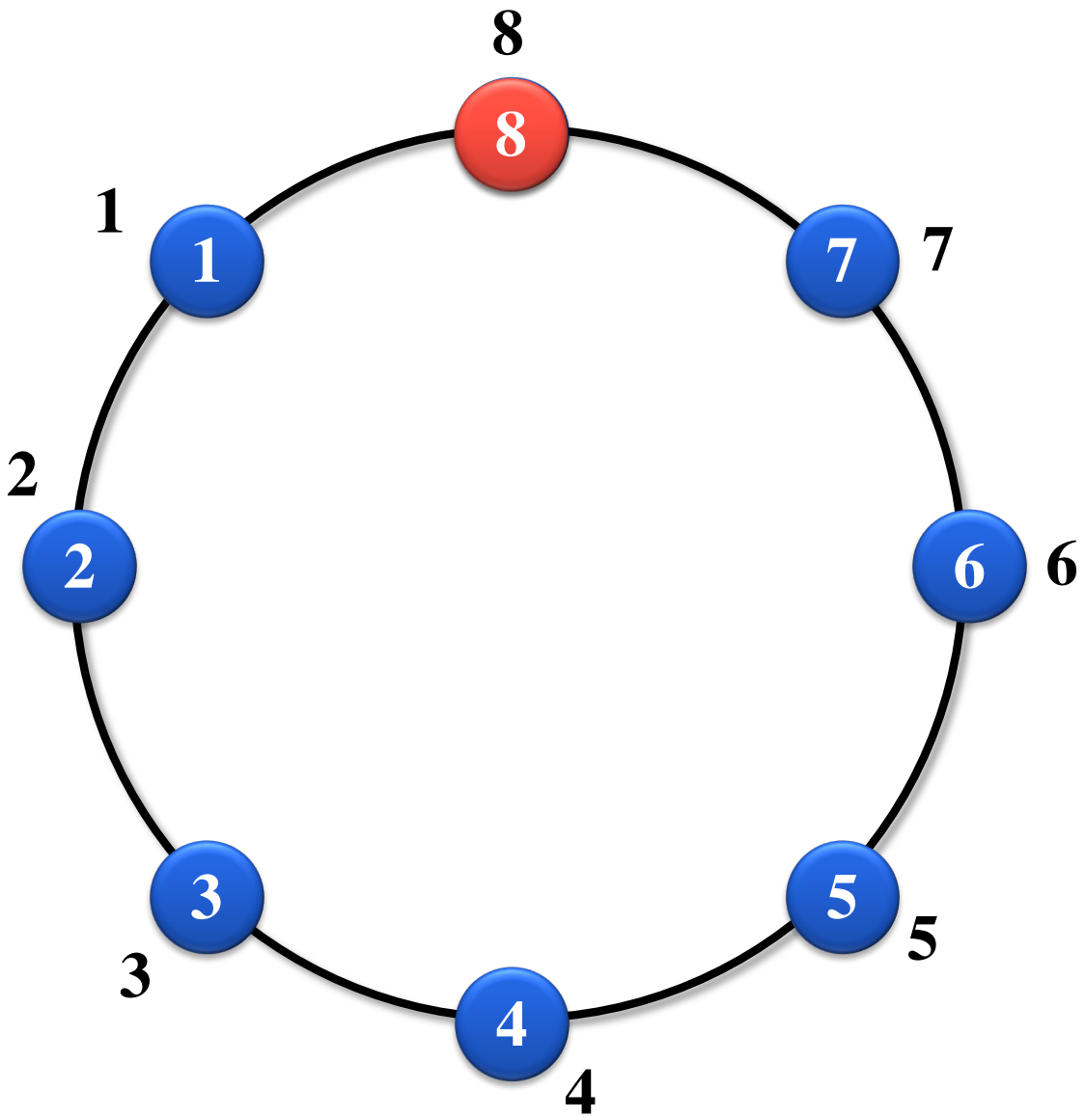
  - Cel puțin un proces inițiază alegerea și se auto-marchează
  - Când un mesaj **m** ajunge la un proces nemarcat, acesta :
    - va genera un mesaj cu **id** propriu
    - se va marca
    - va continua cu prelucrarea mesajului **m**

Algoritmul Lelann-Chang-Roberts
Performanță

- Timp
  - Dacă toate procesele pornesc simultan, timpul este $\boldsymbol{O(N)}$, unde $\boldsymbol{N}$ este numărul de procese.
  - Dacă procesul cu numărul maxim pornește primul, timpul este $\boldsymbol{O(N)}$. Altfel, marcajul ia cel mult $\boldsymbol{N-1}$ pași să ajungă la procesul cu id-ul maxim (și apoi $\boldsymbol{N}$ pași pentru propagare), deci timpul este $\boldsymbol{O(N)}$.

- Numărul de mesaje
  - *Cazul cel mai bun.*
    - Procesele sunt ordonate crescător în sensul acelor de ceasornic. $\boldsymbol{2N-1}$ mesaje.
  - *Cazul cel mai rău*.
    - Procesele sunt ordonate descrescător în sensul acelor de ceasornic. $\sum_{i=1}^{N} \boldsymbol{i} = \frac{\boldsymbol{N(N+1)}}{\boldsymbol{2}}$ mesaje.

Algoritmul Lelann-Chang-Roberts



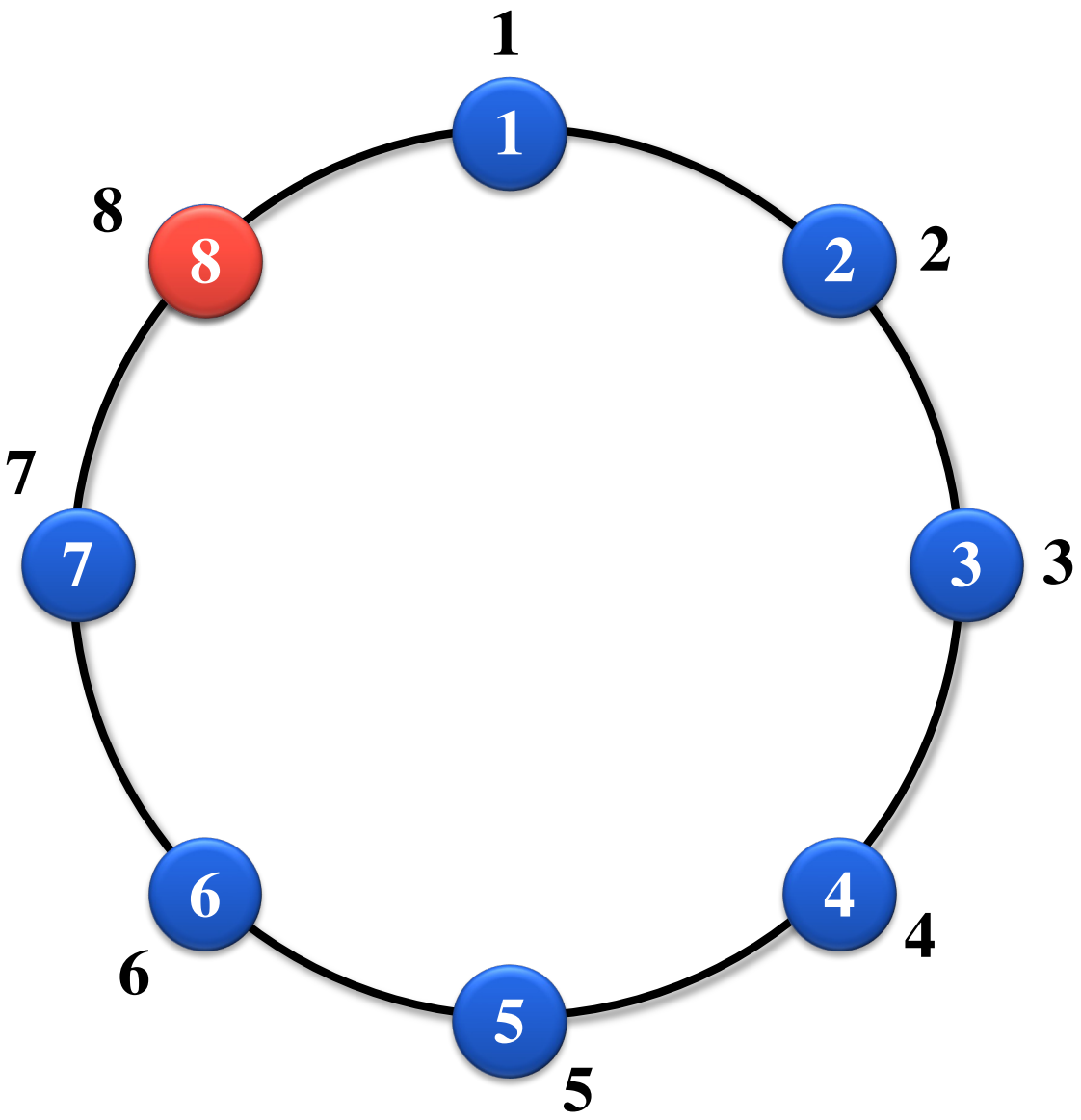
Cazul cel mai rău

candidate

lost

leader

Algoritmul Lelann-Chang-Roberts



Cazul cel mai bun

candidate

lost

leader

Algoritmul Hirschberg-Sinclair

Technical Note
Operating Systems

R. Stockton Gaines
Editor

Decentralized Extrema-Finding in Circular Configurations of Processors

D.S. Hirschberg and J.B. Sinclair
Rice University

This note presents an efficient algorithm, requiring $O(n \log n)$ message passes, for finding the largest (or smallest) of a set of n uniquely numbered processors arranged in a circle, in which no central controller exists and the number of processors is not known a priori.

Key Words and Phrases: decentralized algorithms, distributed system, operating systems
CR Categories: 4.32, 4.35, 5.25, 5.32

Introduction

We are given n processors that are loosely coupled in a circular arrangement and work asynchronously. Each of the processors has an associated unique value (of which it alone is aware) and none of the processors has a priori knowledge of the number of processors in the circle. The problem is to designate by consensus a unique processor from the circle. The total number of data transmissions (messages passed) among the n processors

pass messages in either or both directions, that these directions are distinguished, that processors can detect from which direction a received message originated, but that "left" may not mean the same to all processors. We propose an algorithm that requires $O(n \log n)$ messages in the worst case. The algorithm as given elects the processor with the highest value.

In the algorithm given below, a processor can initiate messages in both directions by a *sendboth* directive. A processor can pass a (possibly modified) message in a circular manner by a *sendpass* directive. A processor can send a responsive message back in the direction from which that processor received a message by a *sendecho* directive.

The Algorithm

```
To run for election:
    status ← "candidate"
    maxnum ← 1
    WHILE status = "candidate" DO
        sendboth ("from", myvalue, 0, maxnum)
        await both replies (but react to other messages)
 IF either reply is "no" THEN status ← "lost"
        maxnum ← 2*maxnum
    OD
On receiving message ("from", value, num, maxnum):
    IF value < myvalue THEN sendecho ("no", value)
    IF value > myvalue THEN DO
        status ← "lost"
        num ← num + 1
        IF num < maxnum THEN sendpass ("from", value, num,
        maxnum)
            ELSE sendecho ("ok", value)
    OD
    IF value = myvalue THEN status ← "won"
On receiving message ("no", value) or ("ok", value)
    IF value ≠ myvalue THEN sendpass the message
        ELSE this is a reply the processor was awaiting
```

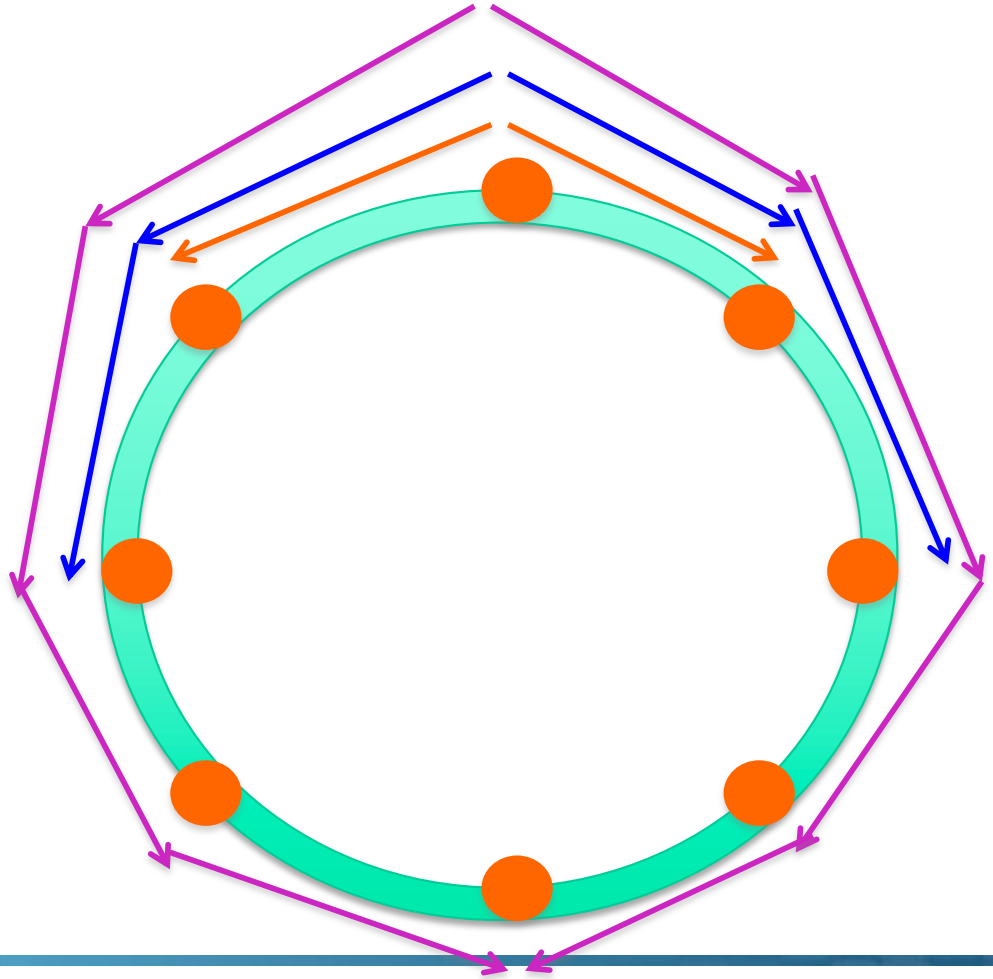
The processors initiate messages that are passed in both directions along paths of predetermined lengths (which are successive powers of 2). Processors on the path read the message. If a processor determines, from reading the message, that it cannot win the election, then

Hirschberg, D. S., & Sinclair, J. B. (1980). Decentralized extrema-finding in circular configurations of processors. *Communications of the ACM*, *23*(11), 627-628.

Algoritmul Hirschberg-Sinclair

- Complexitate $\boldsymbol{O(N\,logN)}$ în cel mai defavorabil caz.
- Lucrează pe un **inel bidirecțional**.
- Procesele pot detecta din ce direcție vine un mesaj și pot trimite un răspuns în acea direcție
- Operații de comunicare folosite de procese:
  - Un proces poate iniția mesaje în ambele direcții prin *sendboth*.
  - Un proces poate pasa un mesaj *(eventual modificat)* prin *sendpass*.
  - Un proces poate trimite un răspuns în direcția din care a primit un mesaj prin *sendecho*.

- Algoritmul opereaza in faze.
- In **faza k**, un proces initiaza mesaje care sunt transmise in ambele directii pe cai de lungime 2^k (1, 2, 4, 8 …)



- Procesele de pe o cale analizeaza mesajul
  - Daca un proces decide ca nu poate castiga alegerea el va pasa mesajul si nu va mai initia mesaje proprii
  - Daca procesul decide ca sursa nu poate castiga alegerea transmite un ecou negativ sursei
  - Ultimul proces din calea 2^k transmite un ecou pozitiv sursei (daca nu poate castiga alegerea)
- Un proces care primeste propriul mesaj castiga
  - in acest caz, calea “acopera” toate procesele
- El va trimite apoi un mesaj pentru a anunta celelalte procese ca a castigat.

Algoritmul Hirschberg-Sinclair (2)

Execuția pentru alegere:

```
    stare = "candidate"
    maxnum = 1
    while (stare == "candidate“) {
        sendboth ("from", myvalue, 0, maxnum);
        await both replies (but react to other
                            messages);
        if (either reply is "no“)
            stare = "lost“;
        maxnum = 2 * maxnum;
  }
```

Algoritmul Hirschberg-Sinclair (3)

La recepție mesaj ("from",value,num,maxnum):

```
if (value < myvalue)
    sendecho ("no", value)
else if (value > myvalue) {
        stare = "lost";
        num = num + 1;
        if (num < maxnum)
           sendpass ("from",value, num, maxnum)
        else
           sendecho ("ok", value)
    }
else
     stare = "won";
```

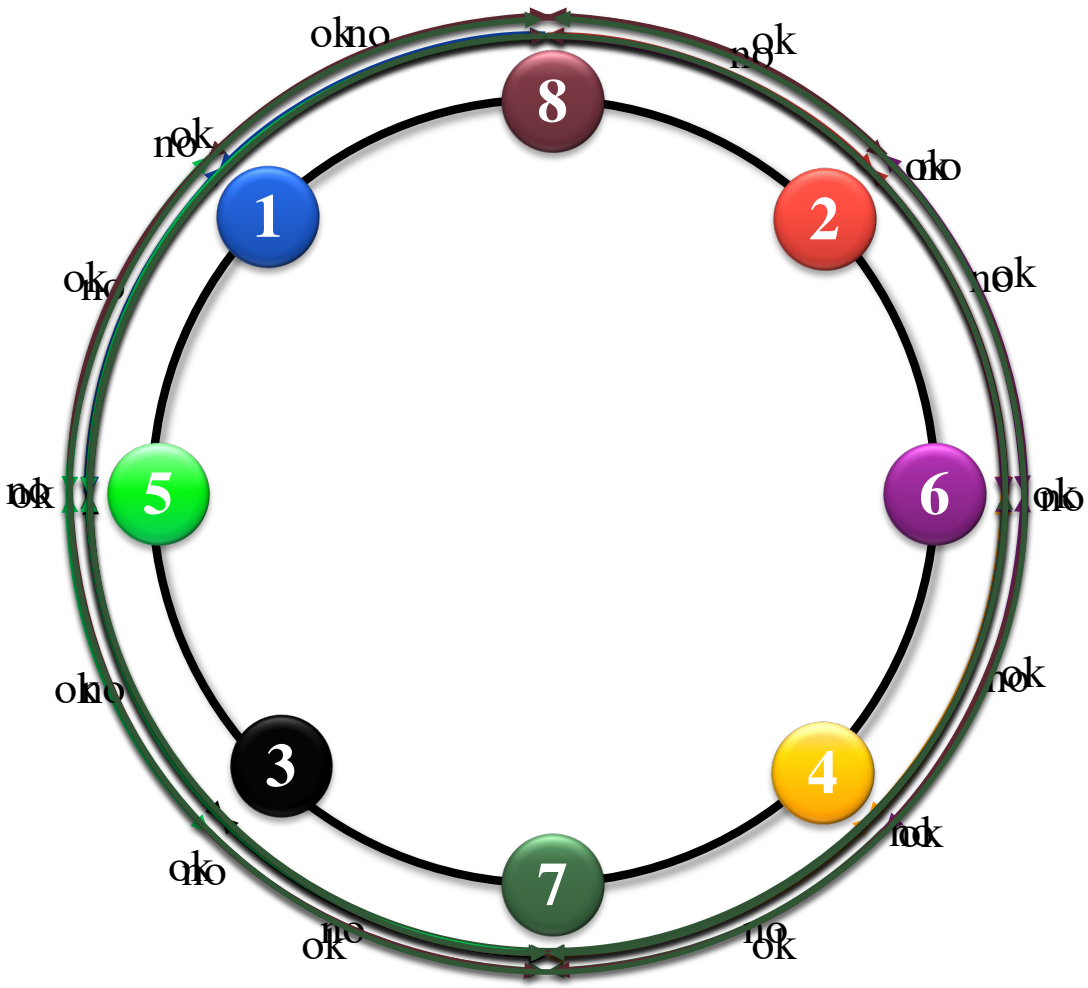
Algoritmul Hirschberg-Sinclair (4)

La recepție mesaj ("no",value) sau ("ok",value):

```
if (value <> myvalue) sendpass the message
else this is a reply the processor was
awaiting
```

- Netratat în algoritm – acțiunile unui proces neinițiator :
  - la primirea unui mesaj devine conștient de alegerea în curs
  - poate deveni *candidate* dacă identitatea are o valoare mai mare decât sursa mesajului.

Algoritmul Hirschberg-Sinclair (5)



Complexitate

Numar de mesaje

Un proces initiaza mesaje pe cai de lungime 2^i numai daca el a primit OK in faza pentru distanta 2^{i-1} (in oricare directie).

In orice grup de $2^{i-1} + 1$ procese consecutive, cel mult unul poate initia mesaje pe cai de lungime 2^i ➔

- toate cele n procese initiaza cai de lungime 1
- cel mult sup[n/2] procese vor initia cai de lungime 2,
- cel mult sup[n/3] de lungime 4,
- cel mult sup[n/5] de lungime 8 etc.

Pentru fiecare cale de lungime 2^i un proces declanseaza cel mult 4 mesaje (2 dus, 2 intors)

Vor fi $4*2^i$ transmiteri declansate de un proces

Complexitate

Numarul total de mesaje este cel mult

$$4*(l*n + 2*[n/2] + 4*[n/3] + 8*[n/5] + \ldots + 2^{i}*[n/(2^{i-1} + 1)] + \ldots).$$

- Fiecare termen (= lung cale * nr procese) mai mic de 2n
- Sunt cel mult 1+log n termeni
  - in log n faze, calea “acopera” n procese

➔ numar mesaje O(n log n)

Complexitate in timp

Fiecare mesaj este transmis intr-o unitate de timp

Un proces trimite in paralel in cele doua sensuri

Mesaje de la procese diferite se transmit in paralel

$$Timp = 2*(1 + 2 + 4 + 8 + \ldots + 2^{i} + \ldots + n)$$

Daca n este putere a lui 2 → $Timp = 2*(2^{\log n+1} -1)/(2-1) = 4*n-2$

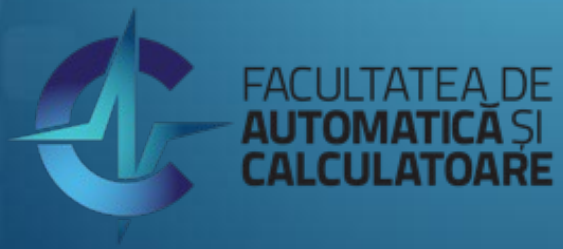
In general Timp = O(n)

Sumar

- Alegerea unui lider cu algoritmi undă
- Algoritmul tree
- Alegerea liderului – topologie inel
- Algoritmul LeLann
- Algoritmul LeLann-Chang-Robert
- Algoritmul Hirschberg-Sinclair

Algoritmi Paraleli și Distribuiți
Stabilirea Topologiei

Prof. Ciprian Dobre
ciprian.dobre@upb.ro



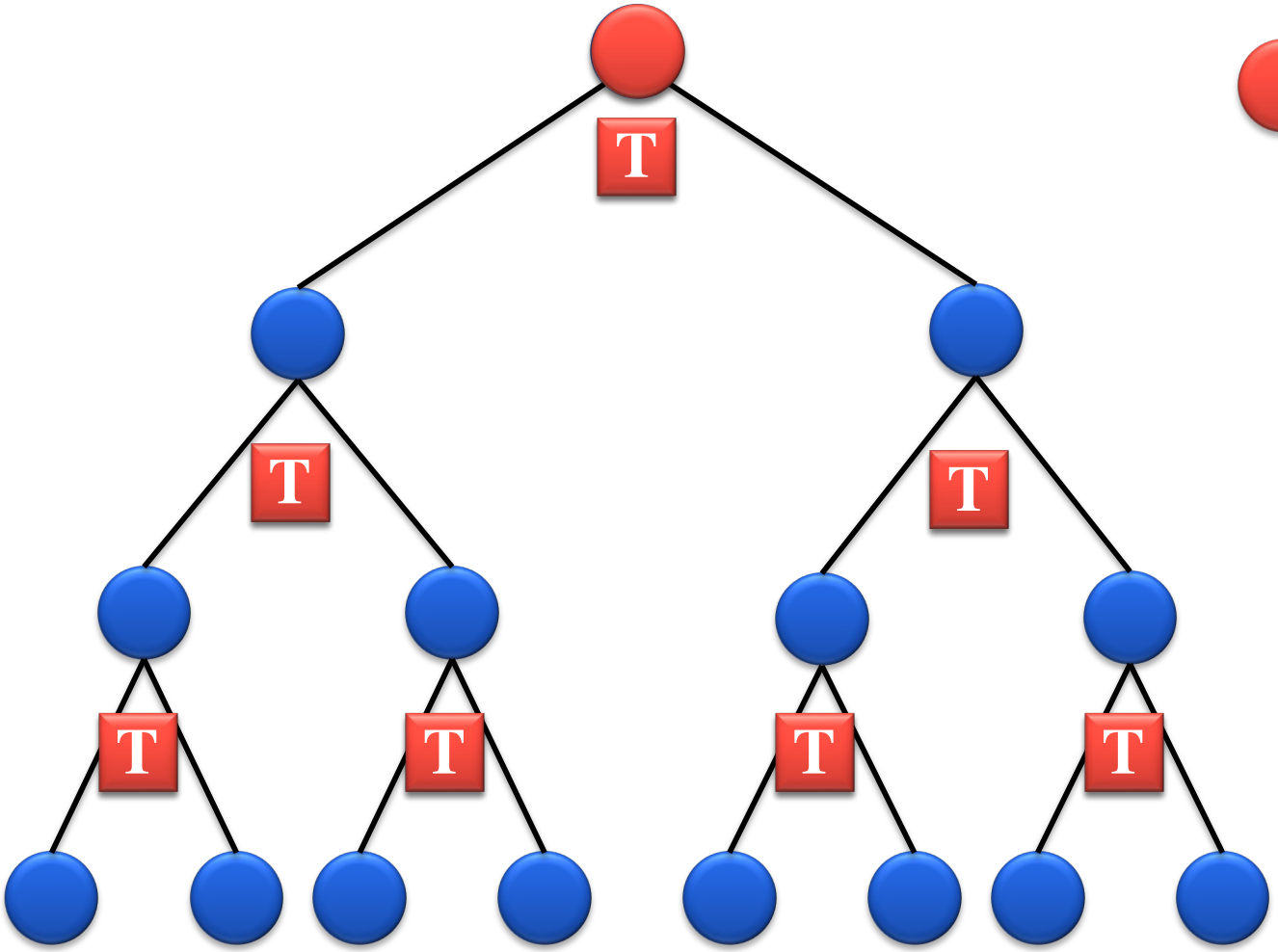
Difuzarea mesajelor folosind sondaje

```
typedef tip_arb int[1:N,1:N];
chan sondaj[1:N](tip_arb,tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to N]{
   tip_arb arb, T;
   tip_mesaj m;
   bool top[1:N,1:N];

   if (inițiator == p) {
      calculeaza T pe baza top;
      m = mesaj_de_transmis
   } else
      receive sondaj[p](arb,m)

   for [q = 1 to N st q este fiul lui p in arb]
      send sondaj[q](arb,m);
}
```

top este o matrice cu topologia arborelui

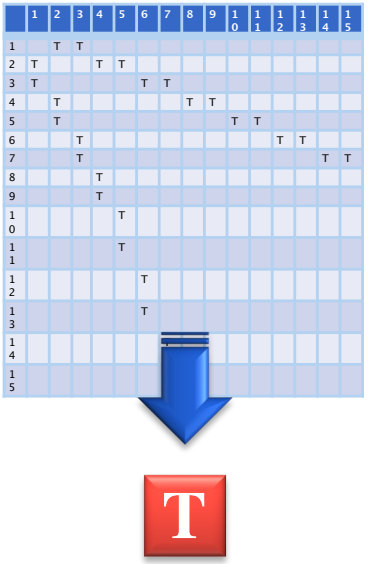
Numărul de mesaje = $N - 1$

Difuzarea mesajelor folosind sondaje



Inițiator

Matrice de adiacență



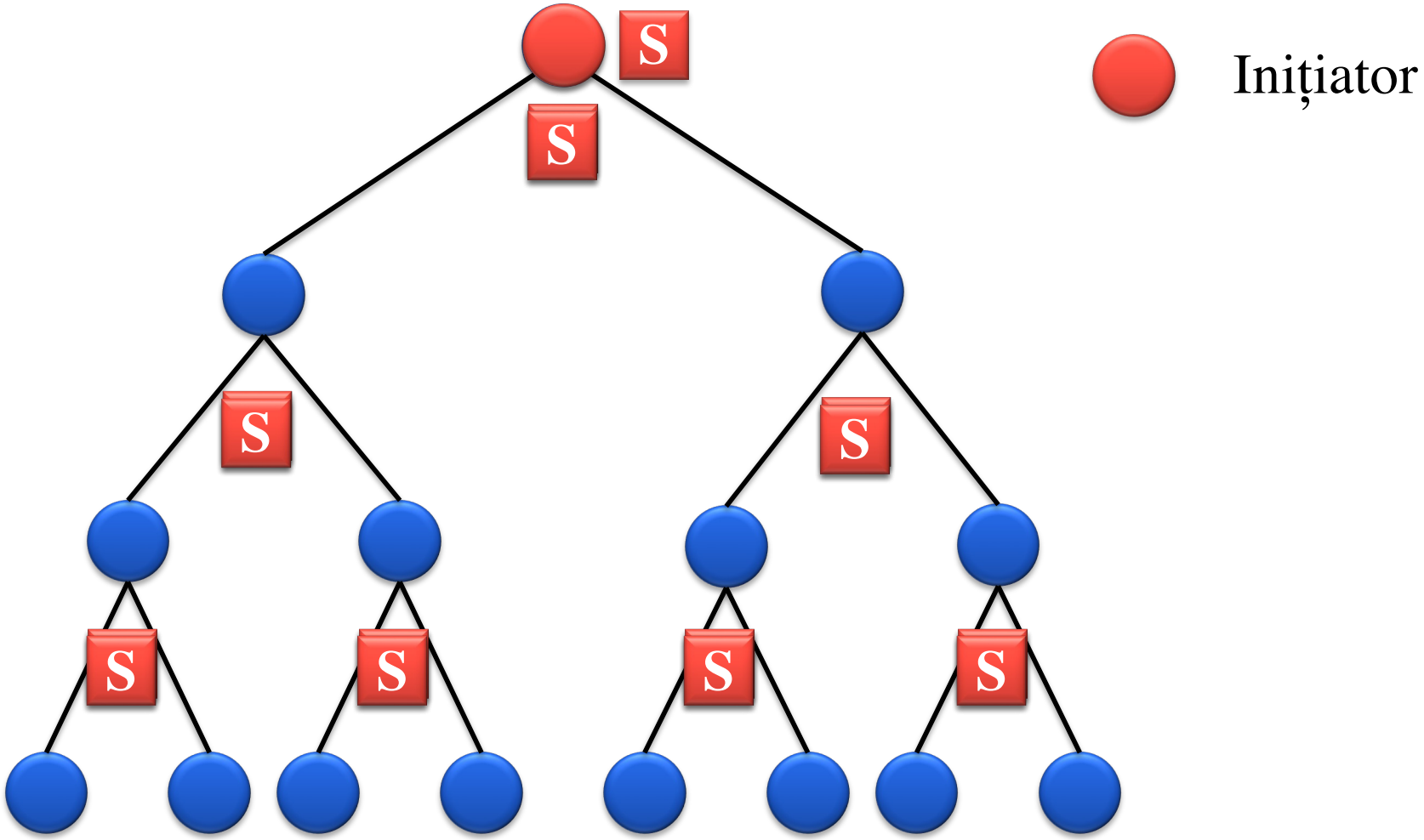
Difuzarea mesajelor prin inundare

```
chan sondaj[1:n](tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to n]{
    bool leg[1:n] = vecinii_lui_p;
    int num = numărul_vecinilor, răspunsuri = num;
    tip_mesaj m;

    if (p <> inițiator) {
        receive sondaj[p](m);
        răspunsuri = răspunsuri - 1;
    else
        m = mesaj_de_transmis;

    for [q = 1 to n st leg[q]]
        send sondaj[q](m)

    for [q = 1 to răspunsuri]
        receive sondaj[p](m)
}
```

Numărul de mesaje = $2m$

(m = numărul legăturilor)

Difuzarea mesajelor prin inundare



Stabilirea topologiei unei reţele de procese

Definiţii

- Colecție de procese **Nod(p:1..N)**
- Vecinii nodului p **leg[1:N]:**
  - $leg[q] = TRUE$ dacă q este vecin cu p,
  - altfel $leg[q] = FALSE$.
- Relație simetrică:
  - pentru p, $leg[q] = TRUE \leftrightarrow$ pentru q, $leg[p] = TRUE$

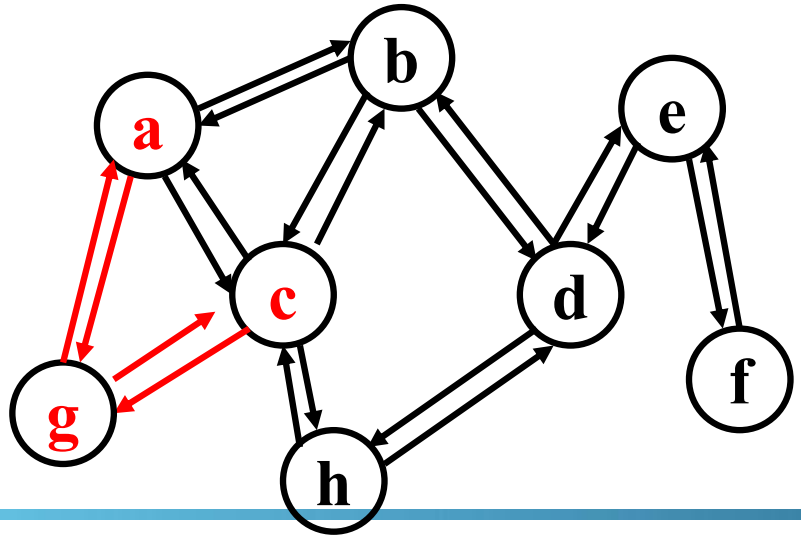
Problema

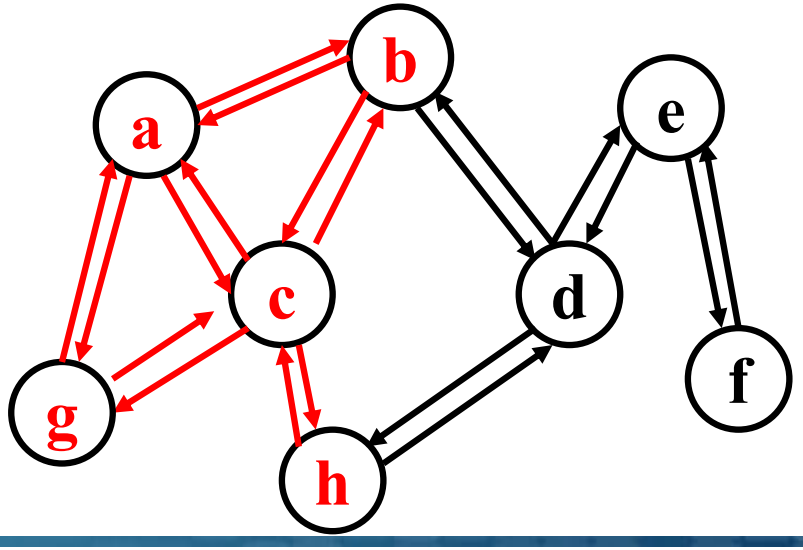
- Topologia reprezentată prin matricea de adiacenţe:
  - $top[i,j] = TRUE$, dacă i este vecin cu j
  - $top[i,j] = FALSE$, în caz contrar.
- Calculată pe baza cunoştinţelor locale
  - pentru orice p,q: $1 \leq p,q \leq N : top[p,q] = leg_p[q]$

Algoritmul pulsatiilor

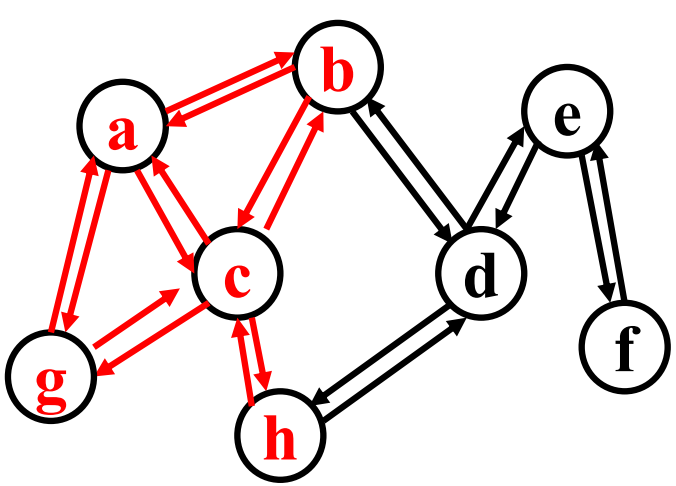
Algoritmul pulsaţiilor

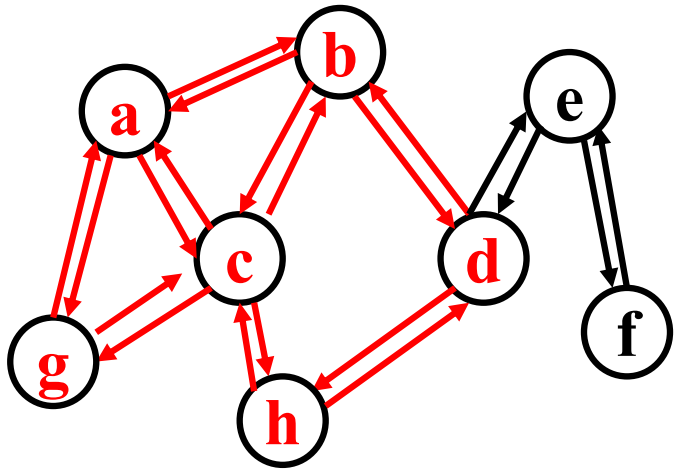
- Fiecare proces calculează singur topologia, folosind informaţiile provenite de la vecini
- Fiecare nod transmite vecinilor matricea sa de adiacenţă, top şi primeşte matricea de adiacenţă a fiecărui vecin
  - după un rund complet, fiecare nod va dispune de informaţii de la nodurile vecine (figura stg. pentru nodul g)
  - După două runde complete, orice nod va avea informaţii de la noduri aflate la distanţa 2 (figura dr.), etc.
  - Dupa r runde sunt completate liniile din top corespunzătoare nodurilor q aflate la o distanţă <= r

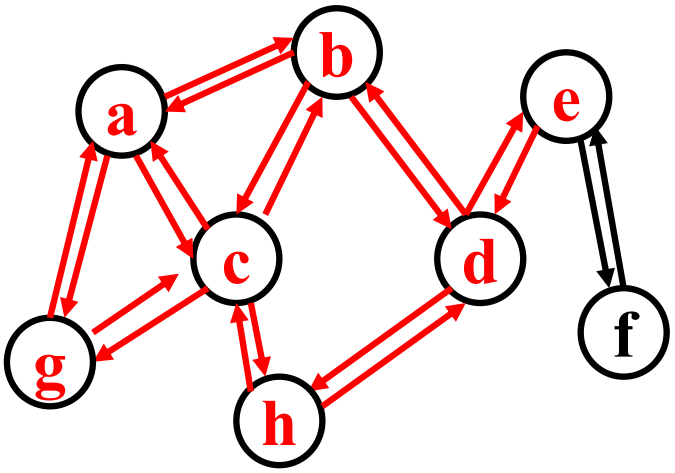


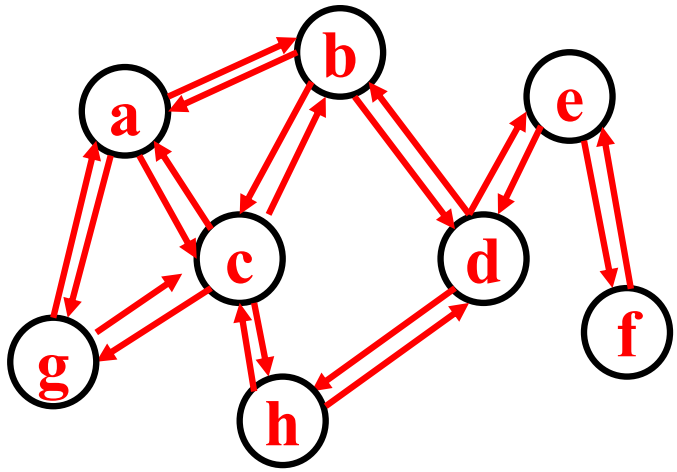


Algoritmul pulsaţiilor









Următorul predicat este satisfăcut pentru fiecare proces p:

RUND: oricare ar fi q cu 1<=q<=N: dist(p,q) <=r → top[q,*] este completat

dist(p,q) este distanţa de la p la q, adică lungimea căii celei mai scurte între cele două noduri.

Algoritmul pulsaţiilor

```
typedef tip_top bool[1:N,1:N];
typedef tip_leg bool[1:N];
chan topologie[1:N](tip_top);

process Nod[p=1 to n]{ tip_leg leg = vecinii_lui_p;
  int r = 0;
  tip_top top = ([N*N]FALSE), top_nou;
  top[p,1:N] = leg;
  /* actualizează liniile vecinilor */
  while (r < D) {
    /* transmite topologia curentă tuturor vecinilor */
    for [q = 1 to N st leg[q]]
      send topologie[q](top)
    for [q = 1 to n st leg[q]]{
      receive topologie[p](top_nou);
      top = top or top_nou;
    }
    r = r + 1;
  }
}
```

D = diametrul rețelei

Algoritmul pulsaţiilor (2)

- Neajunsuri:
  - În general, diametrul rețelei nu este cunoscut
  - Traficul inutil
- Rețea conexă
  - un nod cunoaşte întreaga topologie dacă fiecare linie ***top*** are cel puţin un element TRUE ⇒ mai execută un rund și poate termina
  - evitare blocare ⇒ un proces comunică doar cu vecinii care nu au terminat

Algoritmul pulsaţiilor (3)

```
chan topo[1:N](tip_top);
process Nod[p=1 to N] {
    tip_leg leg = vecinii_lui_p;
    tip_leg activ = leg;                    /* vecinii
   activi */
    tip_top top = ([N * N]FALSE);
    bool gata = FALSE;
    int transm;
    bool qgata;
    tip_top top_nou;

    /* actualizează vecinii lui p */
    top[p, 1:N] = leg;
```

Algoritmul pulsaţiilor (4)

```
 while (NOT gata) {
       /* transmite topologia curentă tuturor
vecinilor */
       for [q = 1 to N st leg[q]]
          send topologie[q](p, FALSE, top)
       /* recepționează topologiile de la vecini */
       for [q = 1 to N st leg[q]] {
          receive topologie[p](transm, qgata,
top_nou);
          top = top OR top_nou;
          if (qgata) activ[transm] = FALSE;
       }
       if (toate liniile din top au un element TRUE)
          gata = TRUE;
 }

 /* transmite top vecinilor activi
    în ultimul rund */
 for [q = 1 to N st activ[q]]
     send topologie[q](p,TRUE,top)
```

se executa D+1 runde

numar mesaje nu depaseste:

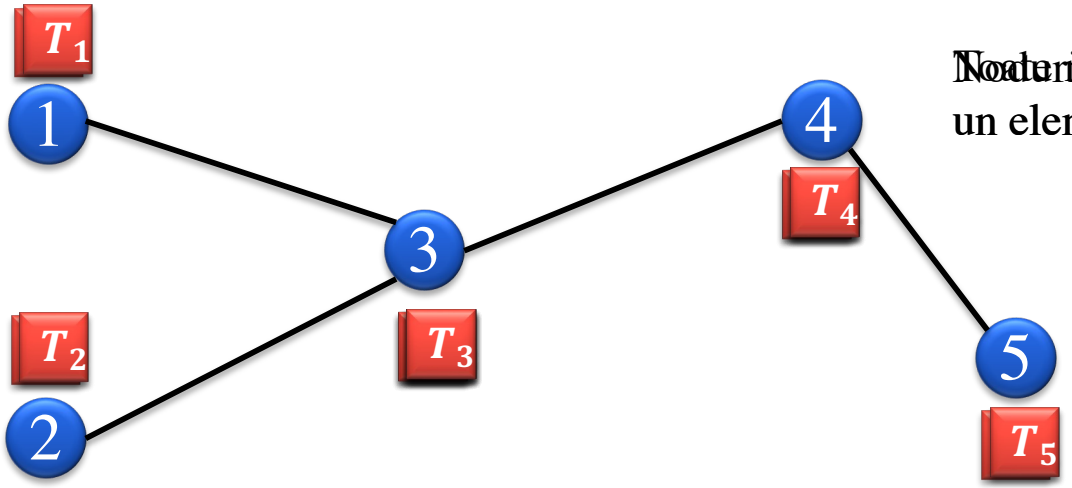
Numărul de mesaje = $2m(D+1)N$

m = numărul legăturilor

N = numărul de noduri

D = diametrul rețelei

Algoritmul pulsaţiilor (5)



Nodurile 3 și 4 au cel puțin un element ***true*** pe fiecare linie

Sau logic între tabela de adiacență proprie și cele primite de la vecini

$topologie[1..N]$

1	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

2	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

3	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

4	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

5	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

1	1	2	3	4	5
			F		

2	1	2	3	4	5
			F		

3	1	2	3	4	5
	T	T		F	

4	1	2	3	4	5
			F		T

5	1	2	3	4	5
				F	

$activ[1..N]$

Algoritmi cu mesaje de sondaj cu ecou

Algoritmi cu mesaje de sondaj cu ecou

- Ideea :
  - un **Iniţiator** colectează informaţii despre topologia locală a tuturor celorlalte noduri, determină topologia întregii reţele şi o difuzează nodurilor.
- **Cazul topologiilor arbore ⇒ sursa (initiatorul) este rădacina arborelui**
  - Două faze :
    - mesajele de sondaj se propagă de la inițiator spre frunze
    - mesajele de ecou *(care conțin topologii parțiale)* se propagă de la frunze spre inițiator.

Algoritmi cu mesaje de sondaj cu ecou (2)

```
const int inițiator = indexul nodului
inițiator;

typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondaj[1:N](int transm);
chan ecou[1:M]  (tip_top top);
chan ecou_final (tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = N + 1;
    tip_top top_nou;

    top[p, 1:n] = leg;
```

Algoritmi cu mesaje de sondaj cu ecou (3)

```
if (p <> inițiator)
   receive sondaj[p](părinte);

/* transmite sondaje celorlalte noduri vecine, copiii
lui p */
for [q = 1 to N st (leg[q] AND (q <> părinte))]
   send sondaj[q](p);


/* primește ecourile și fă reuniunea lor */
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
   receive ecou[p](top_nou);
   top = top OR top_nou;
}


if (p <> inițiator)
   send ecou[părinte](top);
}
```

Algoritmi cu mesaje de sondaj cu ecou (4)

- **Cazul general**
  - după ce primeşte un prim mesaj de sondaj, un nod îl propagă tuturor celorlalţi vecini
  - nodul aşteaptă ecouri de la vecini
  - **există cicluri** $\Longrightarrow$ nodul poate primi și sondaje
    - topologia locală va fi transmisă doar ca ecou la primul sondaj
    - pentru restul sondajelor se transmite o topologie nulă
    - sondajele și ecourile trebuie recepţionate pe acelaşi canal

Algoritmi cu mesaje de sondaj cu ecou (5)

```
/* index inițiator */
const inițiator = indexul nodului inițiator;

enum fel{sondă, ecou};
typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondă-ecou[1:N](fel tip_mesaj, int transm, tip_top
top);
chan ecou_final(tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = n + 1;
    tip_top top_nou;
```

Algoritmi cu mesaje de sondaj cu ecou (6)

```
/* actualizează vecinii lui p */
top[p, 1:N] = leg;

fel k; int transm;
int nr_ecouri = 0;

if (p <> inițiator)
   receive sondă-ecou[p](k, părinte, top_nou);

/* transmite sondaje celorlalte noduri vecine, copiii lui p
*/
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
   send sondă-ecou[q](sondă, p, 0);  /* topologie nulă */
   nr_ecouri = nr_ecouri + 1;
}
```

Algoritmi cu mesaje de sondaj cu ecou (7)

```
   /* primește ecourile și fă reuniunea lor, sau sonde și
ignoră */
   while (nr_ecouri > 0) {
      receive sondă-ecou[p](k, transm, top_nou);

      if (k == sondă)
         send sondă-ecou[transm](ecou, p, 0);
      else if (k == ecou) {
         top = top OR top_nou;
         nr_ecouri = nr_ecouri - 1;
      }
   }

   if (p <> inițiator)
      send sondă-ecou[părinte](ecou, p, top);
```

Complexitate

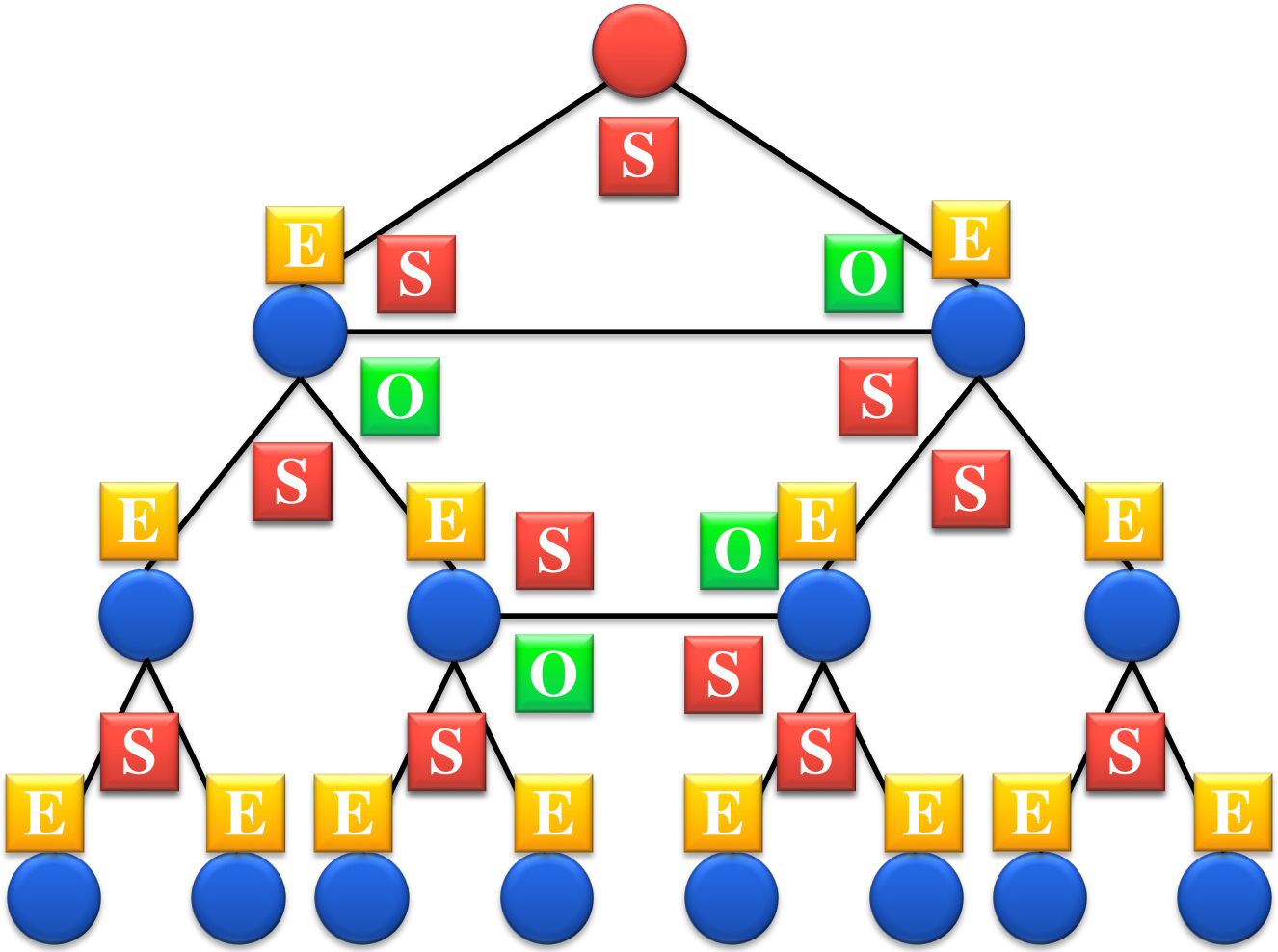
Numar mesaje

Fiecare legătură (aparţinând arborelui de acoperire) corespunzătoare primei sonde poartă două mesaje (sonda şi ecoul)

Celelalte, câte patru (două sonde şi două ecouri)

Transmiterea topologiei de la iniţiator la noduri necesită alte N mesaje.

Algoritmi cu mesaje de sondaj cu ecou (8)

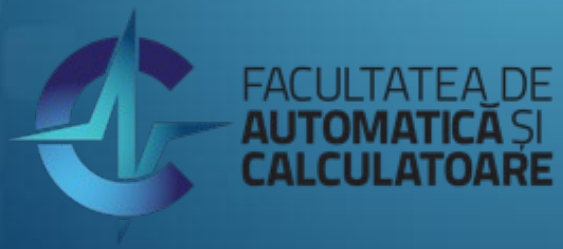


Sumar

- Difuzarea mesajelor prin inundare
- Difuzarea mesajelor folosind sondaje
- Problematica stabilirii topologiei
- Algoritmul pulsațiilor
- Algoritmi cu mesaje de sondaj cu ecou

Algoritmi Paraleli și Distribuiți
Terminarea programelor distribuite

Prof. Ciprian Dobre
ciprian.dobre@upb.ro

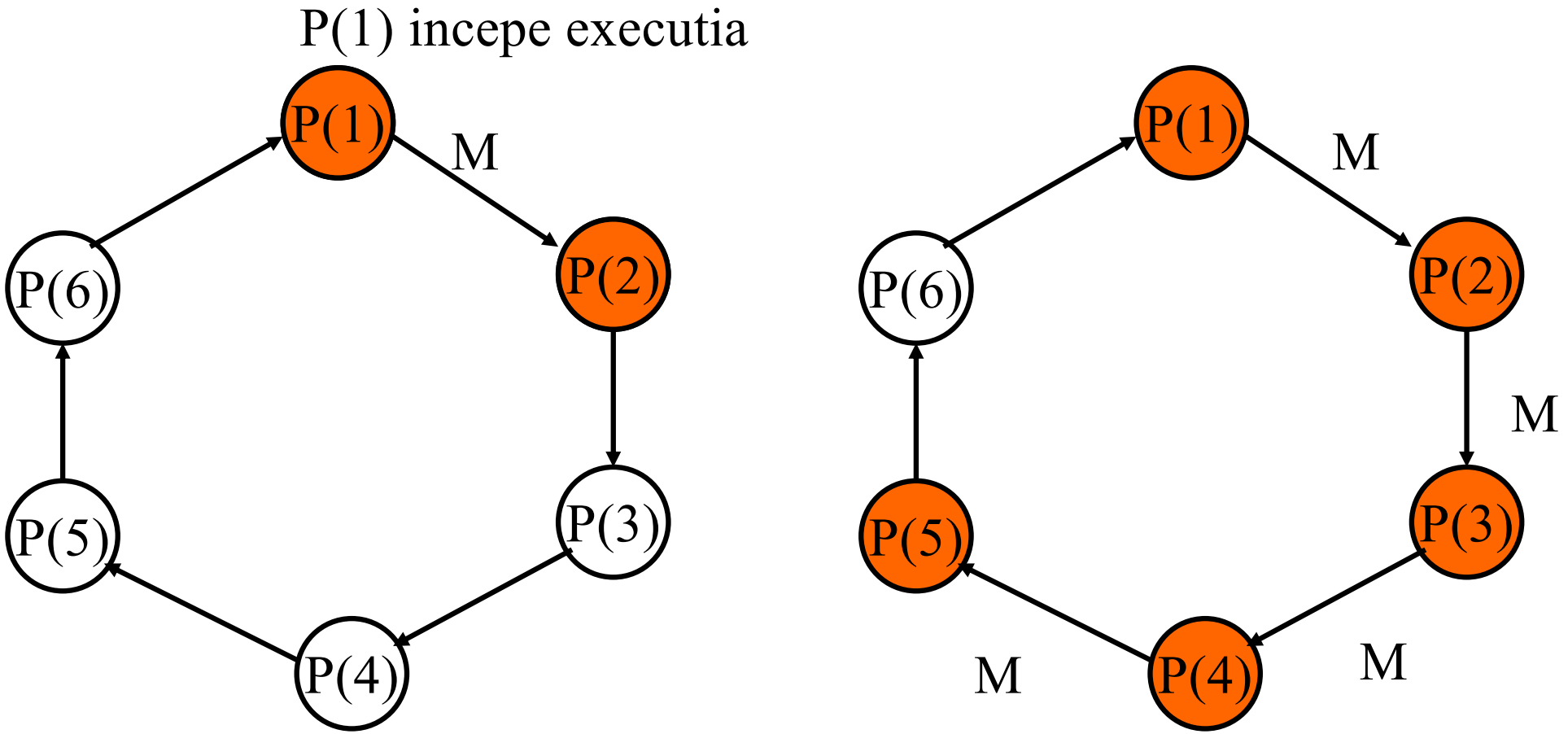


Terminarea programelor distribuite

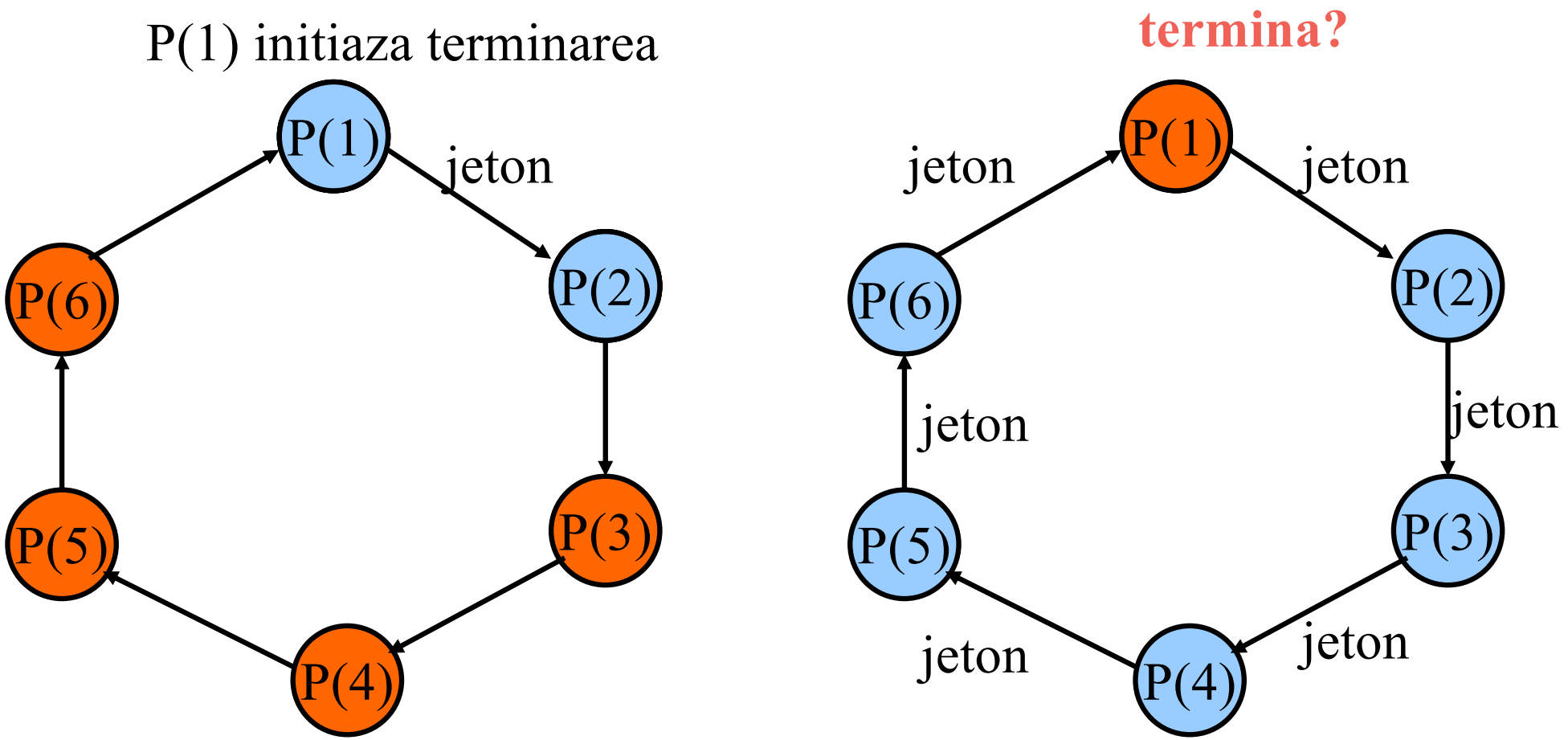
- **Problema:** detecția terminării
- Algoritmi paraleli:
  - toate procesele se termină
    - cicluri infinite?
  - procese terminate sau blocate și nu există I/E în curs
  - simplu pentru un singur procesor - coada *ready* goală

- Algoritmi distribuiți:
  - fiecare proces să fie liber
    - a terminat execuția *sau*
    - se află în aşteptarea unui mesaj (**receive**)
  - pe canale să nu existe mesaje în tranzit

Procese organizate în inel



Procese organizate în inel



inainte de a primi jeton, P(1) poate fi reactivat de un mesaj de la P(6)!

Procese organizate în inel (tehnica jetoanelor) (1)

- Inițiatorul terminării este procesul P[1]
- acțiunile lui P[1] cînd devine liber prima dată:

```
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

- acțiunile lui P[i=1 to N] la recepția unui mesaj obişnuit:

```
culoare[i] = roșu;
```

Tehnica jetoanelor (2)

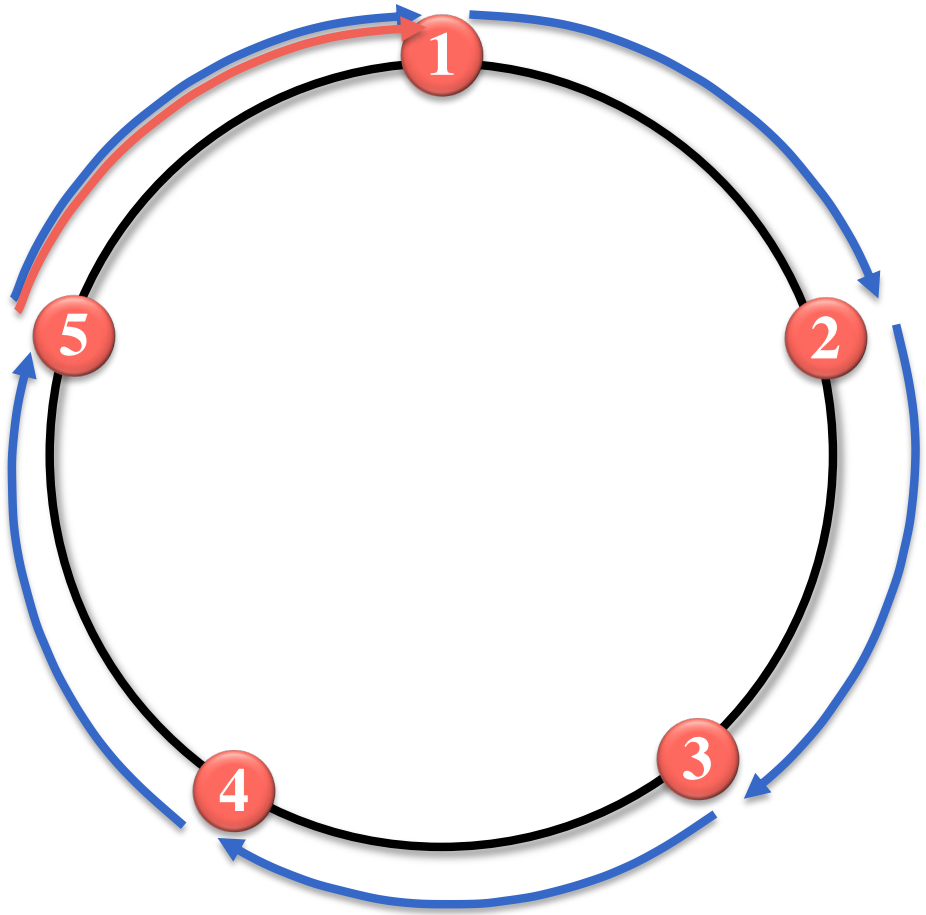
- acțiunile lui P[i=2 to N] la primirea unui jeton (și după ce devine liber):

```
culoare[i] = albastru;
jeton = jeton +1;   /* nesemnificativ */
send ch[(i + 1) mod n](jeton);
```

- acțiunile lui P(1) la recepția jetonului:

```
if (culoare[1] == albastru) {
  anunță terminarea; stop
}
culoare[1] = albastru;
jeton = 0;
send ch[2](jeton);
```

Tehnica jetoanelor (3)

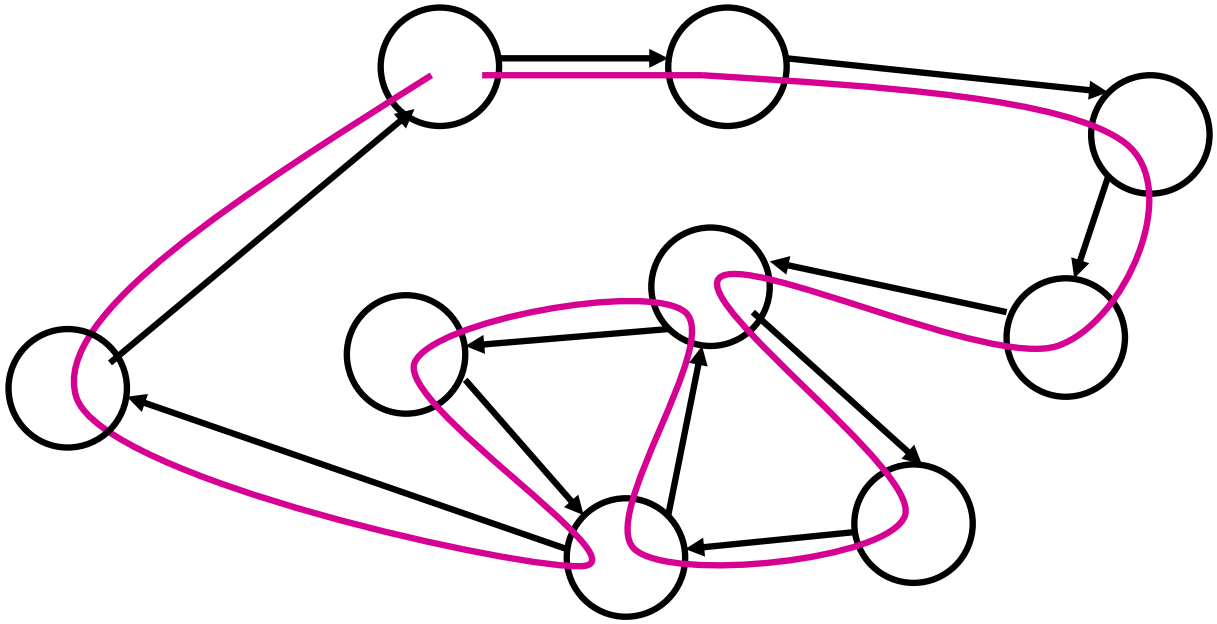


Jeton
1

STOP

Terminarea în cazul general (tehnica jetoanelor)(1)

- topologie generală de **graf**
- **c** un ciclu care include toate arcele grafului; fie **nc** lungimea lui



Terminarea în cazul general (tehnica jetoanelor)(2)

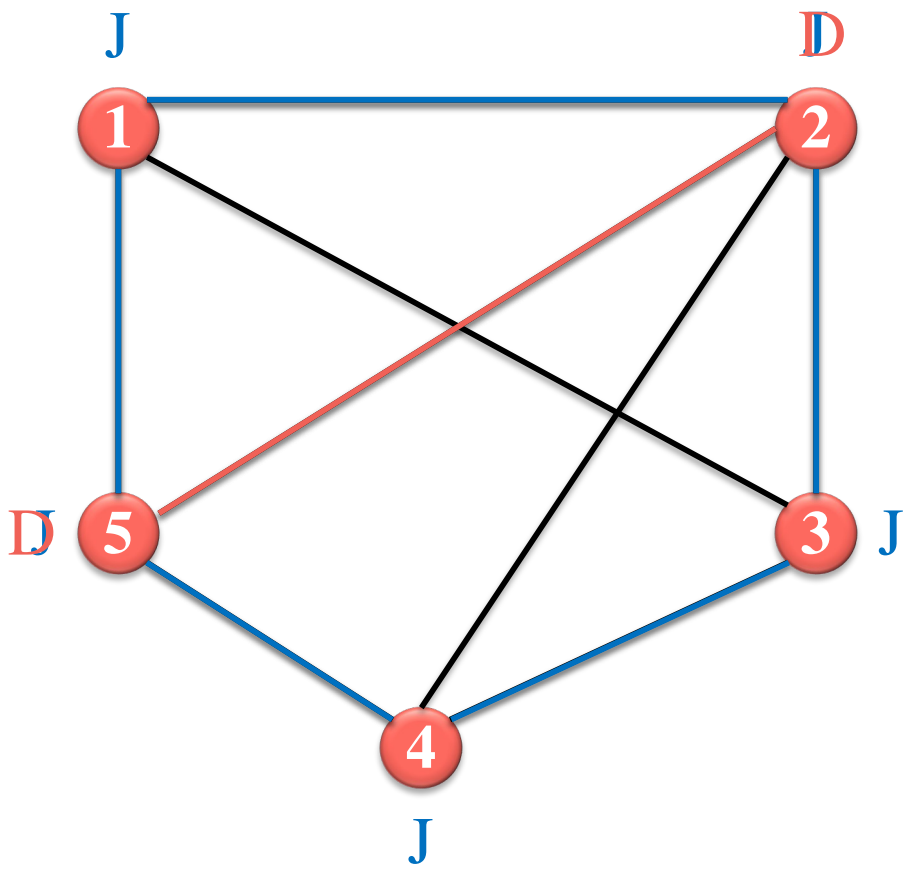
- un jeton care se transmite după mesajele de date
- are inițial valoarea 0
- **acțiunile lui P[i=1 to N] la primirea unui mesaj uzual:**

```
culoare[i] = roșu;
```

- **acțiunile lui P[i=1 to N] la recepția jetonului:**

```
if (jeton == nc) {
        if (culoare[i] == albastru) {
                anunță terminarea; stop }
        else if (culoare[i] == roșu) {
                culoare[i] = albastru; jeton = 0; }
} else if (culoare[i] == albastru) jeton = jeton +1;
setează j la canalul următor din ciclul c;
send ch[j](jeton);
```

Terminarea în cazul general (tehnica jetoanelor)(3)



nc = 5

Jeton
0

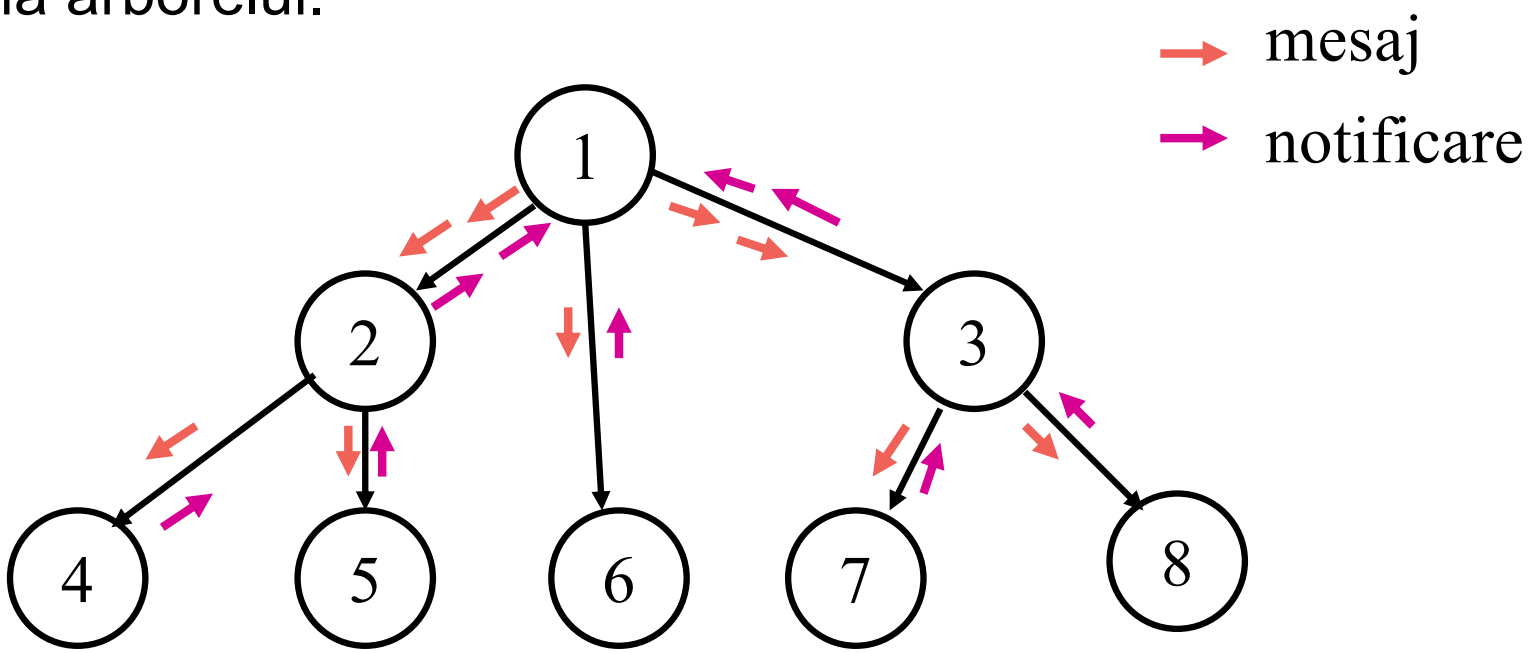
STOP

Terminare (topologii arbore)

- Tehnica confirmarii mesajelor -

Terminarea in topologii arbore

- Când un proces frunză se termină, el anunţă părintele său.
- Când un nod oarecare al arborelui se termină, el aşteaptă notificarea terminării de la toţi fiii săi şi apoi îşi anunţă părintele.
- Notificările se propagă astfel pînă la procesul sursa, aflat în rădăcina arborelui.



Confirmarea mesajelor (1)

- Funcția de determinare a stării canalelor se poate împărți între procesele programului: fiecare proces verifică canalele pe care a trimis mesaje de date altor procese (canalele sale de ieşire).

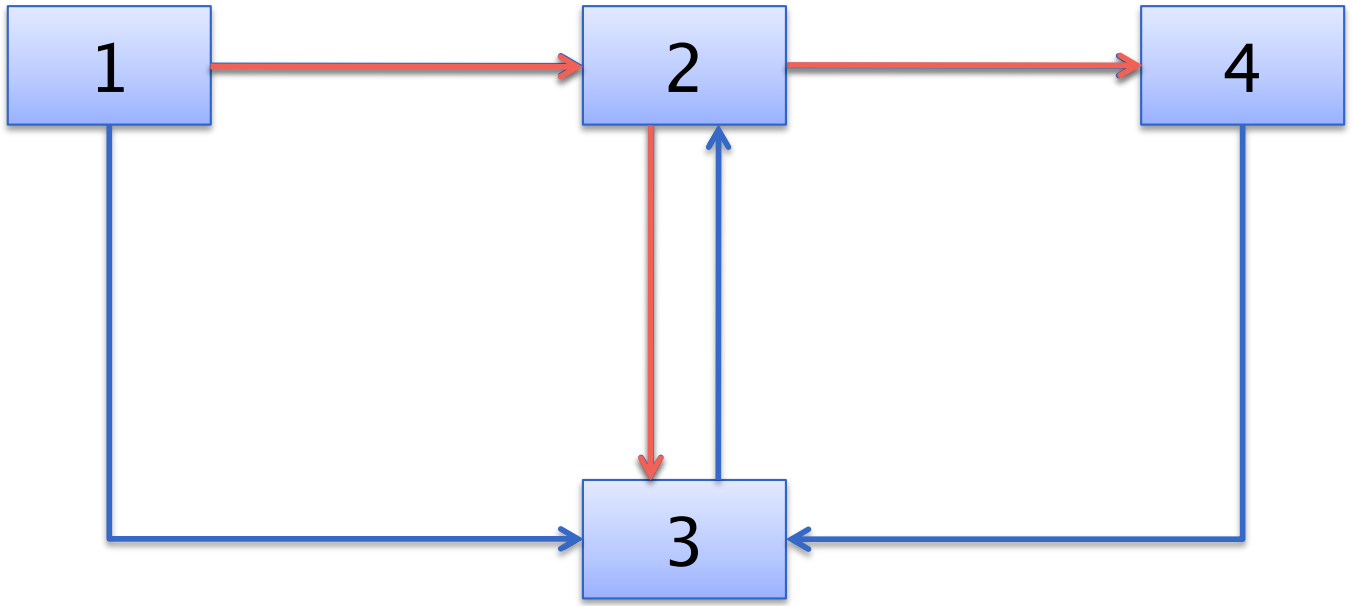
- Pe un canal
  - se trimit **mesaje de date** și se primesc **semnale de confirmare** *sau*
  - se primesc **mesaje de date** și se trimit **semnale de confirmare**

- **Confirmarea mesajelor**: dacă pentru fiecare mesaj de date transmis pe un canal procesul primeşte de la receptor un semnal de confirmare, atunci canalul respectiv este gol.

- Presupune existența unui proces **sursă**:
  - care nu are legături de intrare *şi*
  - poate accesa orice alt proces pe o cale oarecare din graf

Confirmarea mesajelor (2)

- Exemplu:
  - Procesul sursă: 1
  - Arbore de acoperire



Algoritmul Dijkstra-Scholten (1)

Volume 11, number 1 INFORMATION PROCESSING LETTERS 29 August 1980

TERMINATION DETECTION FOR DIFFUSING COMPUTATIONS *

Edsger W. DIJKSTRA
Burroughs, 5671 AL Nuenen, The Netherlands

C.S. SCHOLTEN
Philips Research Laboratories, 5656 AA Eindhoven, The Netherlands

Received 2 June 1980

Concurrency, termination detection, distributed control, separation of concerns, message-based systems, correctness proving, diffusing computations, networks, cornets, activation tree, nondeterminacy

The following seems to capture the quintessence of a situation that is not unusual in distributed processing. Consider a finite, directed graph. If the graph contains an edge from node A to node B, we call B 'a successor of A', and A 'a predecessor of B'. We assume the existence of a node without incoming edges; this node will be called 'the environment' (because it acts as such with respect to the rest of the graph). The other nodes will be called 'the internal nodes'.

For each node its initial state will be called 'the neutral state'. A so-called 'diffusing computation' is started when the environment sends – of its own accord, so to speak – a message to one or more of its successors; it is supposed to do this just once. After reception of its first message, an internal node is free to send messages to its successors. It is this feature proper – such that, when the diffusing computation proper has thus terminated, the fact of this completion will eventually be signalled back to the environment. Besides a node's ability to receive messages from its predecessors and to send messages to its successors, we assume each node also to be able to receive 'signals' from its successors and to send 'signals' to its predecessors; in other words, each edge is assumed to be able to accommodate two-way traffic, but only messages of the computation proper in the one direction and signals in the opposite direction. We shall impose that in the total computation – i.e. from the moment that the environment sent its messages to the rest of the graph until it has received the completion signal – each edge will have carried as many messages in the one direction as it has carried signals in the op-

Dijkstra, Edsger W., and Carel S. Scholten. "Termination detection for diffusing computations." *Information Processing Letters* 11.1 (1980): 1-4.

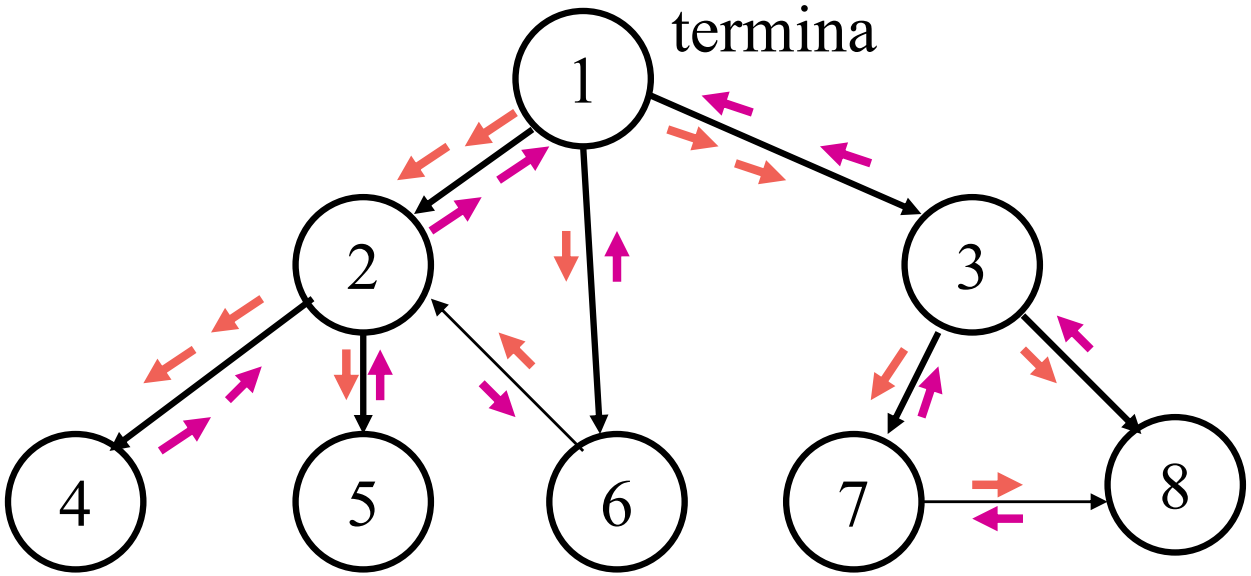
Algoritmul Dijkstra-Scholten (1)

- Bazat pe transmiterea unor semnale de confirmare
- Cazul unei rețele cu topologie **arborescentă**:
  - Când un proces *frunză* se termină, el îl anunță pe *părintele* său.
  - Când un *nod* oarecare al arborelui termină, el aşteaptă notificarea terminării de la toți *fiii* săi și apoi își anunță *părintele*.
  - Notificările se propagă până la procesul sursă.
- Cazul unui **graf dirijat aciclic** de procese
  - Deficitul unei legături = diferența dintre numărul de mesaje de date transmise şi numărul de semnale de confirmare primite pe acea legătură
  - Când un nod doreşte să termine:
    - aşteaptă primirea unor semnale pe legăturile de ieşire a datelor, semnale care reduc deficitele la zero (pe acele legături).
    - trimite pe fiecare legătură de intrare a datelor un număr de semnale egal cu deficitul (legăturii respective).

Algoritmul Dijkstra-Scholten (2)

- **Cazul general**: cicluri în graful proceselor
- Rezolvare: generare **arbore de acoperire**:
  - fiecare legătură generată la prima recepție (adică prima legătură de intrare, numită **prim** – de la părinte)
- Pentru terminare:
  - trimitere semnale pentru toate legăturile de intrare, mai puțin **prim**
  - recepție semnale pentru toate legăturile de ieşire
  - trimitere semnale spre **prim**.
- Schema de semnalare este separată și suplimentară față de prelucrarea propriu-zisă

Algoritmul Dijkstra-Scholten



Algoritmul Dijkstra-Scholten (3)

LORDUL

SERVITORUL

Algoritmul Dijkstra-Scholten (4)

- Comunică doar cu alți lorzi și cu propriul servitor
- Comunică cu lorzii prin scrisori (prin poștă) și cu servitorul prin semnale de comunicare (la telefon)
- Își anunță servitorul când trimite/primește o scrisoare (spunându-i și destinația/sursa)
- După ce se plictisește întreabă servitorul dacă a terminat treaba ca să poată merge la vânătoare

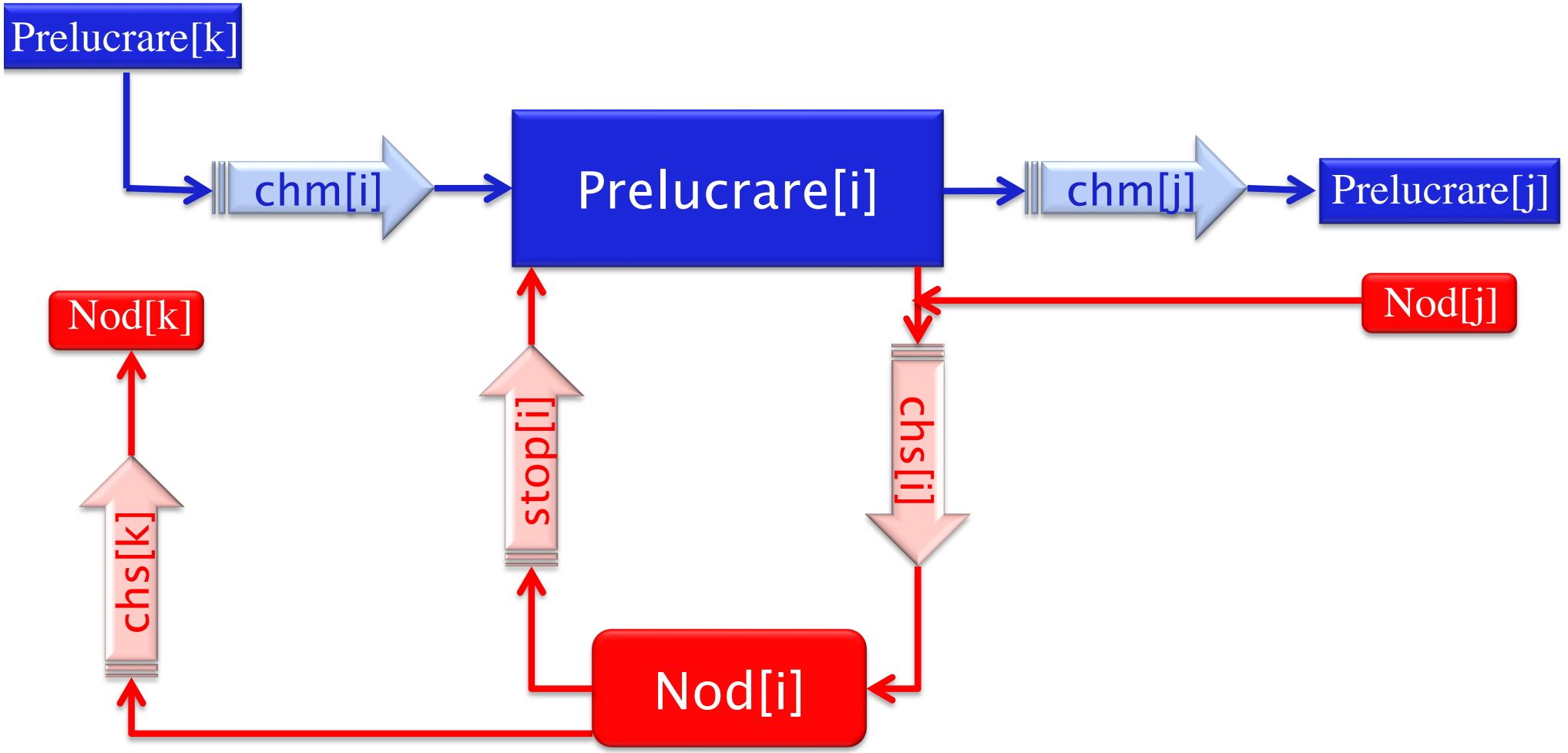
- Comunică doar cu alți servitori și cu propriul lord
- Comunică doar prin semnale (la telefon)
- Trimite semnale de confirmare pentru scrisorile primite și așteaptă semnale de confirmare pentru scrisorile trimise
  - Scrisoarea primită de la lordul$_i$ se confirmă servitorului$_i$
- Răspunde afirmativ doar dacă a primit și trimis toate confirmările pentru mesajele trimise și primite, altfel îl roagă pe lord să aștepte

Algoritmul Dijkstra-Scholten (5)

În continuare

- Lord = Prelucrare *(prelucrează mesaje de date)*
- Servitor = Nod *(gestionează semnalele de confirmare cu alte noduri și semnalele de comunicare cu Prelucrare)*
- Canalul poștal = canalul chm
  - chm_i este „adresa” lui $Prelucrare_i$
  - $Prelucrare_i$ primește mesaje pe chm_i și îi trimite lui $Prelucrare_j$ mesaje pe chm_j
- Canalul telefon pentru Nod = canalul chs
  - chs_i este „numărul de telefon” al lui Nod_i
  - Nod_i primește semnale pe chs_i (de la alte noduri sau de la $Prelucrare_i$) și trimite semnale de confirmare lui Nod_j pe chs_j
- Canalul telefon pentru Prelucrare = canalul stop
  - $stop_i$ este „numărul de telefon” al lui $Prelucrare_i$
  - $Prelucrare_i$ primește semnale de comunicare (da/nu) pe $stop_i$ de la Nod_i

Algoritmul Dijkstra-Scholten (6)



Algoritmul Dijkstra-Scholten (7)

```
enum fel{semnal, transmitere, receptie, terminare};
typedef struct {bool există;
                int deficit} arc;
chan chs[1:N](fel op, int id);
chan chm[1:N](int id, tip_date date);
chan stop[1:N](bool);

process Prelucrare[i=1 to N]{
   tip_date data; int id; bool term;
   inițializări;
   term = false;
   while (NOT term) {
      receive chm[i](id, data);                  /* primește mesaj */
      send chs[i](recepție, id);                 /* semnalează nod - recepție mesaj */
      realizează prelucrare
      ....
      send chm[j](i, data);                      /* trimite mesaj */
      send chs[i](transmitere, j);               /* semnalează nod - transmitere mesaj */
      ...
      if (decizie de terminare) term = true;
   }
   term =false;
   while (NOT term) {
      send chs[i](terminare, i);                 /* anunță nod că a terminat */
      receive stop[i](term);                     /* așteaptă să termine nodul */
   }
   stop;
```

Algoritmul Dijkstra-Scholten (8)

```
process Nod[i=1 to N] {
   arc in[1:N]; /* arcele de intrare */
   int nsemnale = 0;      /* număr semnale așteptate pentru mesajele trimise */
   int prim = 0;
   var id: int; fel k;
   while (true) {
      receive chs[i](k, id);
      if (k == recepție) { if (prim == 0) prim = id fi;
                           in[id].deficit = in[id].deficit + 1; }
      else if (k == transmitere) nsemnale = nsemnale+1;
      else if (k == semnal) nsemnale = nsemnale-1;
      else if (k == terminare){
      for [j = 1 to N st (in[j].există AND (j <> prim))] /* se confirmă mesajele primite */
         while (in[j].deficit > 0) { send chs[j](semnal, i);
                                     in[j].deficit = in[i].deficit - 1;
         }
      }
      if (nsemnale <> 0) send stop[i](false);            /* mai sunt semnale de primit */
      else if (nsemnale == 0) {
         if ((i <> sursa) AND (prim <> 0)) {              /* se confirmă lui prim */
            while (in[prim].deficit > 0) {
               send chs[prim](semnal, i);
               in[prim].deficit = in[prim].deficit - 1;
            }
          }
          send stop[i](true);
       }}
   }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

Prelucrare dorește terminarea

Detecția terminării folosind marcaje

DETECTING TERMINATION OF DISTRIBUTED COMPUTATIONS USING MARKERS

Jayadev Misra
Department of Computer Sciences, University of Texas, Austin, 78712

Abstract

A problem of considerable importance in designing computations by process networks, is detection of termination. We propose a very simple algorithm for termination detection in an arbitrary network using a single marker We show an application of this scheme in solving the problem of token loss detection and token regeneration in a token ring.

Introduction

We study the problem of detecting termination of computation in a network of processes. If every process in a network is *idle*, i.e. waiting for messages in order to carry out further computations, and there are no messages in transit, i.e. all messages that have been sent have been received, then no process will carry out any further computation. It is often important to detect such a situation. Multiphase algorithms [15] in which a phase is to be initiated only upon completion of the previous phase, requires termination detection of a phase. Francez, Rodeh and Sintzoff [11] suggest that it may be easier to devise a distributed algorithm in two steps: (1) design an algorithm that maintains the desired safety properties and eventually guarantees a terminating *global* state; (2) superimpose a termination detection algorithm on the basic algorithm. Deadlock detection [8] which is related to termination detection is of fundamental importance in distributed data bases. Detection of token loss in a token ring can be shown to be a termination detection problem.

We suggest an algorithm for termination detection in an arbitrary network of processes. The algorithm uses a single marker which repeatedly traverses the edges of the network until it detects termination. We make no assumption about the network structure, process behavior or message delays. Our only assumption is that a process receives (only and all) messages from another process in the order sent by the sender. Our solution is symmetric among the processes, i.e. process id's are not used in the solution. As an application of the scheme in this paper, we give a simple and efficient algorithm for detecting token loss and regenerating the token in a token ring network. The marker algorithm has also been applied [14] in *avoiding* deadlocks in distributed simulations.

There has been a considerable amount of work in termination detection. For a specific class of computations, called *diffusing computations*, Dijkstra and Scholten [2] proposed a very elegant algorithm. Their approach has been extended and applied to a variety of problems [4,5,6,7]. The major drawbacks of their approach are: (1) termination detection algorithm must be initiated whenever the diffusing computation starts and (2) the number of messages used for termination detection is equal to the number of messages in the diffusing computation itself. Our solution does not suffer from these drawbacks.

A series of papers has been published by Francez and co-workers [9,11,12] leading to a marker type algorithm. The algorithm proposed in this paper refines, simplifies and removes some of the restrictions of their approach. Independently Gouda [1] and Dijkstra [3] have proposed similar schemes, again with certain restrictions. All of these schemes use the marker to determine if a process has remained "continuously idle" over an interval of time; the idea of continuous idleness also appears in [8] for deadlock detection.

Recently Chandy and Lamport [10] have proposed a very elegant and general scheme for detecting *stable properties* of a network; a property (proposition) P is stable if it remains true once it becomes true. Clearly properties such as "network computation has terminated," and "a subset of processes are deadlocked" are all stable properties. Even though we treat the narrower problem of termination detection, our solution is simpler and more efficient for this specific problem.

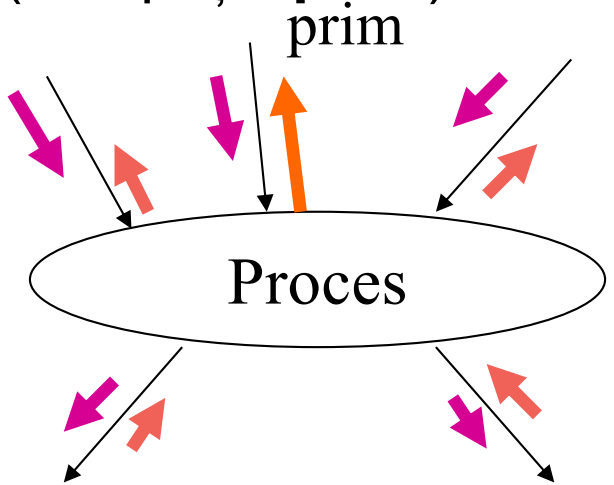
Problem Description

*This work was supported by a grant from IBM.

Misra, Jayadev. "Detecting termination of distributed computations using markers." *Proceedings of the second annual ACM symposium on Principles of distributed computing*. ACM, 1983.

Detecția terminării folosind marcaje (1)

- Procesele transmit mesaje de marcaj la sfârşitului comunicației.
- Procesele primesc semnale de confirmare doar pentru marcaje.
- Pentru terminare:
  - aşteaptă marcaje pe legăturile de intrare
    - *(să termine cei de la care a primit mesaje)*
  - transmite marcaje pe legăturile de ieşire
    - *(anunță propria terminare celor cărora le-a trimis mesaje)*
  - transmite semnale pe legăturile de intrare (mai puțin **prim**)
    - *(confirmă marcajele primite)*
  - aşteaptă semnale pe legăturile ieşire
    - *(așteaptă confirmări pentru marcajele trimise)*
  - transmite semnal pentru **prim**



Detecția terminării folosind marcaje (2)

- Pentru fiecare arc se țin două atribute: ***activ*** și ***marcaj.***

- Pentru fiecare **arc de intrare**, se înregistrează dacă:
  - s-a primit un mesaj (se pune ***activ*** pe true);
  - s-a primit un marcaj (se pune ***marcaj*** pe true);
  - s-a transmis un semnal de confirmare (se pun ambele pe false).

- Pentru fiecare **arc de ieşire**, se înregistrează dacă:
  - s-a transmis un mesaj (se pune ***activ*** pe true);
  - s-a transmis un marcaj (se pune ***marcaj*** pe true);
  - s-a primit un semnal de confirmare (se pun ambele pe false).

Detecția terminării folosind marcaje (3)

```
const marcaj = mesaj-special-de-marcaj-sfârșit-date;
typedef enum {semnal, transmitere, recepție,
              recepție-marcaj, terminare} fel;
typedef struct {bool există;
              bool activ;        /*inițial false*/
              bool marcaj} arc;    /*inițial false*/
chan chs[1:N](fel op, int id);
chan chm[1:N](id:int, tip_date date);
chan stop[1:N](bool);
process Nod[i=1 to N]{
  bool term = false;
  arc in[1:N], out[1:N];
  int nsemnale = 0;/*număr semnale așteptate pe out*/
  int prim = 0;
  int nmarcaje;    /*număr marcaje așteptate pe in*/
  int id; fel k;
```

Detecția terminării folosind marcaje (4)

```
while (true) {
    receive chs[i](k, id);
    if (k == recepție) {
      if (prim == 0) prim = id;
      if (NOT in[id].activ) { in[id].activ = true;
                              nmarcaje = nmarcaje + 1;
      }
    else if (k == recepție-marcaj) {
       in[id].marcaj = true; nmarcaje = nmarcaje - 1;
       if (id == prim) term = true; }
    else if (k == transmitere) {
      if (NOT out[id].activ) { out[id].activ = true;
                               nsemnale = nsemnale + 1; }
    else if (k == semnal) {
      out[id].activ = false; out[id].marcaj = false;
      nsemnale = nsemnale - 1;
    }
```

semnal de la Prelucrare: a primit un mesaj

semnal de la Prelucrare: a primit un marcaj

semnal de la Prelucrare: a trimis un mesaj

semnal de confirmare de la alt nod

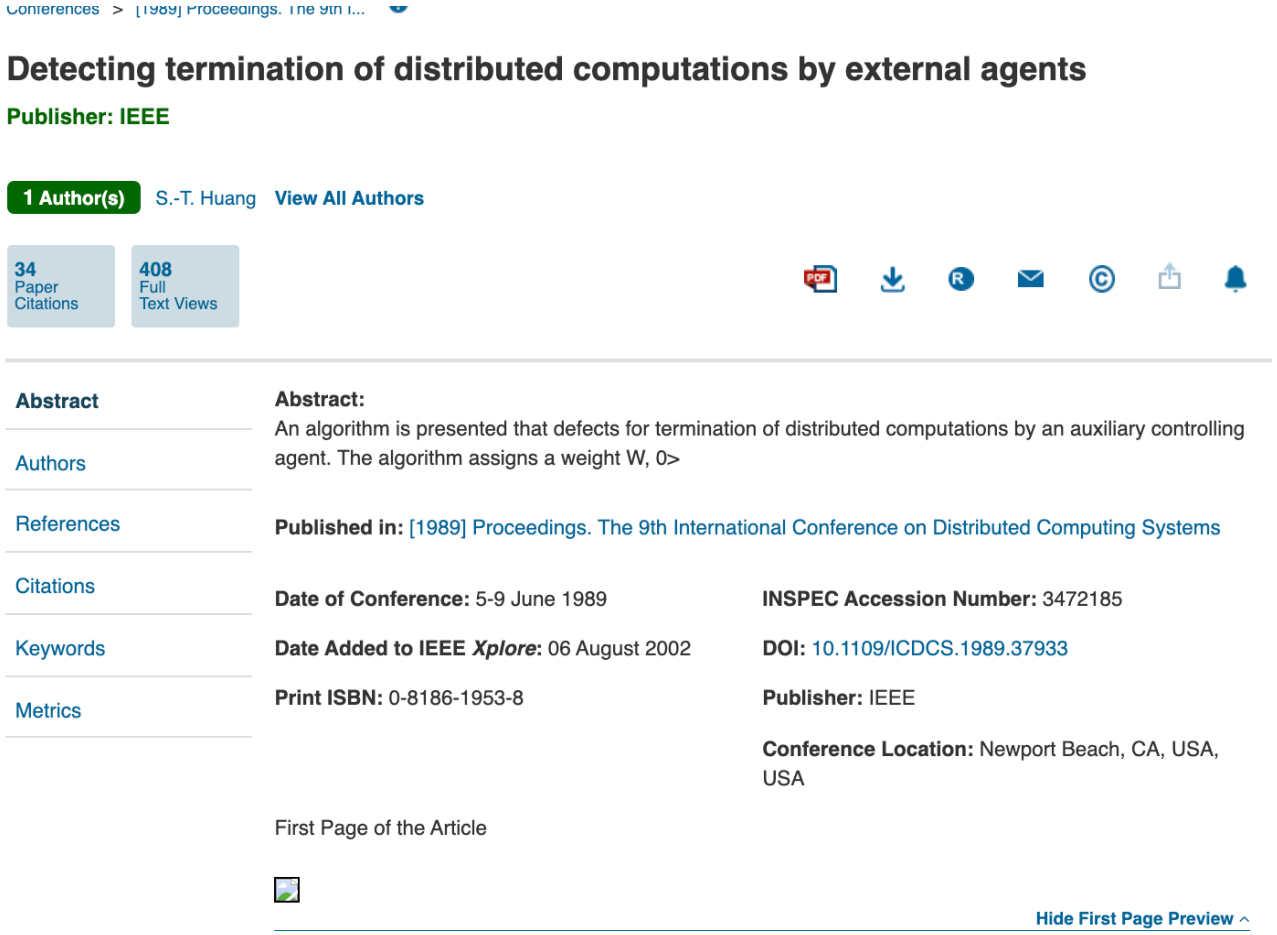
Detecția terminării folosind marcaje (5)

```
else if (k == terminare) {
      if (NOT term) send stop[i](false);
      else if (term) {
         for [j = 1 to N st (j <> prim AND out[j].activ AND
                             NOT out[j].marcaj)] {
          send chm[j](i, marcaj); out[j].marcaj = true;
         }
         if (nmarcaje <> 0) send stop[i](false);
         else if (nmarcaje == 0) {
             for [j = 1 to n st (in[j].există AND (j<>prim)
                                 AND in[j].activ)] {
                send chs[j](semnal, i);
                in[j].activ = false; in[j].marcaj = false;
             }
             if (nsemnale <> 0) send stop[i](false);
             else if (nsemnale == 0) {
                  if ((i <> sursa) AND (prim <> 0)) {
                     send chs[prim](semnal, i);
                     in[prim].activ = false;
                     in[prim].marcaj = false;
                  }
                  send stop[i](true);
   } } } }
```

Detecția terminării folosind marcaje (6)

```
process Prelucrare[i=1 to N] {
  tip_date data; int id; bool term;
  initializari;
  term = false;
  while (NOT term) {
     receive chm[i](id, data);
     if (data==marcaj) send chs[i](receptie-marcaj,id);
     else if (data<>marcaj) {
          send chs[i](receptie, id);
          realizează prelucrare
          ....
          send chm[j](i, data);
          send chs[i](transmitere, j);
          ...
          if (decizie de terminare) term = true;
     }
  }
  term =false;
  while (NOT term) {
     send chs[i](terminare, i);
     receive stop[i](term);
  }
  stop;
```

Algoritmul lui Huang



Conferences > [1989] Proceedings. The 9th I...

Detecting termination of distributed computations by external agents

Publisher: IEEE

1 Author(s) S.-T. Huang **View All Authors**

34 Paper Citations

408 Full Text Views

Abstract
Authors
References
Citations
Keywords
Metrics

Abstract:
An algorithm is presented that defects for termination of distributed computations by an auxiliary controlling agent. The algorithm assigns a weight W, 0>

Published in: [1989] Proceedings. The 9th International Conference on Distributed Computing Systems

Date of Conference: 5-9 June 1989

Date Added to IEEE *Xplore*: 06 August 2002

Print ISBN: 0-8186-1953-8

INSPEC Accession Number: 3472185

DOI: 10.1109/ICDCS.1989.37933

Publisher: IEEE

Conference Location: Newport Beach, CA, USA, USA

First Page of the Article

Hide First Page Preview

Huang, S-T. "Detecting termination of distributed computations by external agents." *[1989] Proceedings. The 9th International Conference on Distributed Computing Systems*. IEEE, 1989.

Algoritmul lui Huang (1)

- Modelul se bazează pe următoarele:
  - procesele pot fi active sau libere (idle)
  - doar procesele active transmit mesaje
  - procesele libere pot deveni active la recepția unui *mesaj de calcul*
  - procesele active pot deveni libere în orice moment
  - **terminarea**: toate procesele sunt libere şi nu există mesaje de date în tranzit
  - topologia este un graf conex

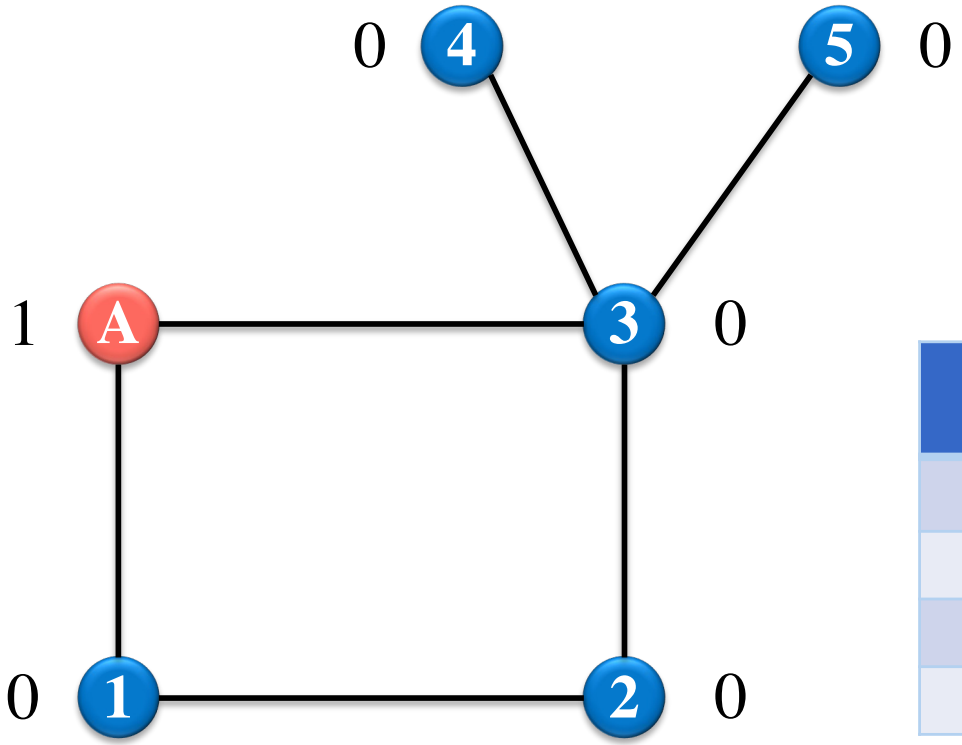
Algoritmul lui Huang (2)

- **Ideea:**
  - Un *agent de control* are inițial ponderea 1.
  - Toate celelalte procese sunt libere și au inițial ponderea 0.
  - Calculul porneşte atunci când *agentul de control* trimite un mesaj de date unui proces.
  - Un proces liber devine activ la recepția unui mesaj de date.
- **Notații:**
  - B(DW)
    - Mesaj de date cu ponderea DW
    - Trimis doar de *agentul de control* sau de un proces activ
  - C(DW)
    - Mesaj de control cu ponderea DW
    - Trimis de un proces activ *agentului de control* când devine liber
  - W
    - Ponderea curentă a unui proces

Algoritmul lui Huang (3)

1. **Send B(DW)**:
   - Găseşte W1, W2 astfel încât W1 > 0, W2 > 0, W1 + W2 = W
   - Pune W = W1 şi transmite B(DW = W2)
2. **Receive B(DW)**:
   - W = W + DW
   - **if** liber -> devine activ **fi**
3. **Send C(DW)**:
   - send C(DW = W) *agentului de control*
   - devine liber
4. **Receive C(DW) (doar *agentul de control*)**:
   - W = W + DW
   - **if** W = 1 -> declară “terminarea” **fi**

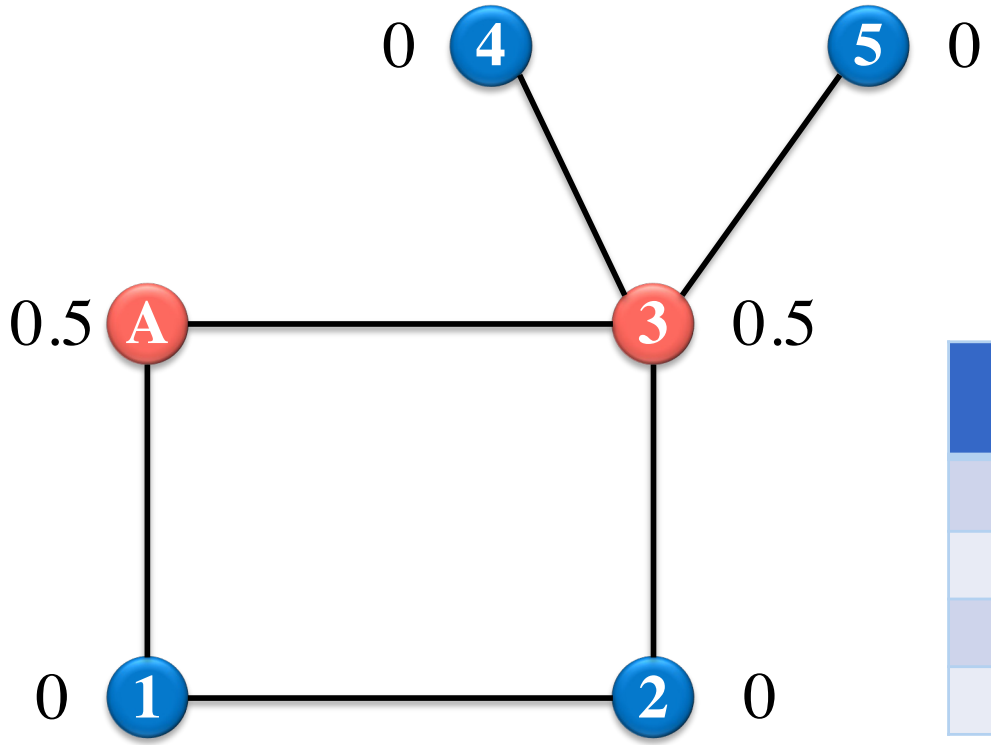
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
A	3	B(DW=0.5)

TIMP
0

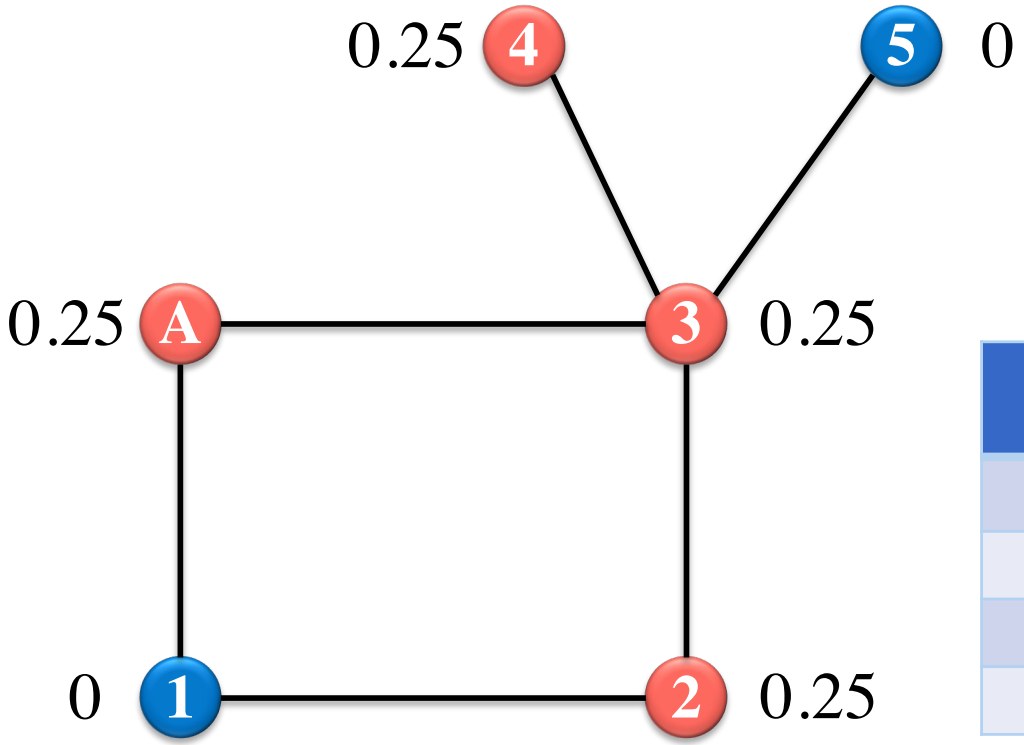
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
A	2	B(DW=0.25)
3	4	B(DW=0.25)

TIMP
1

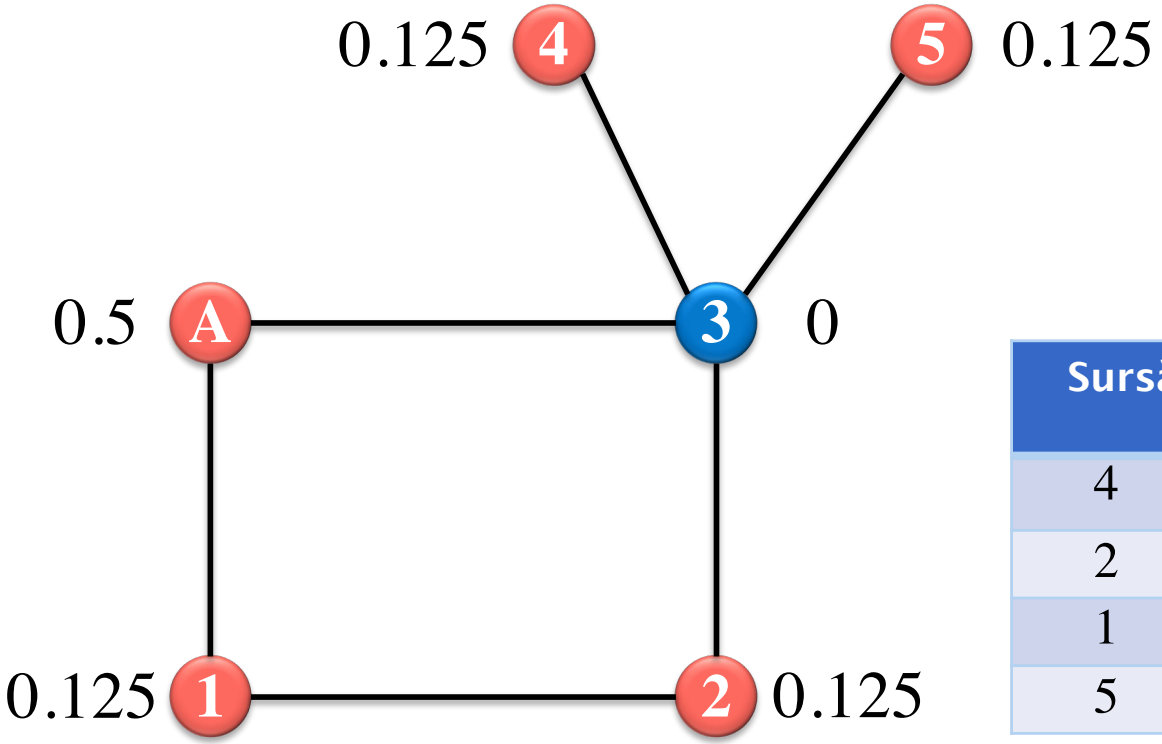
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
4	1	B(DW=0.125)
3	A	C(DW=0.25)
2	5	B(DW=0.125)

TIMP
2

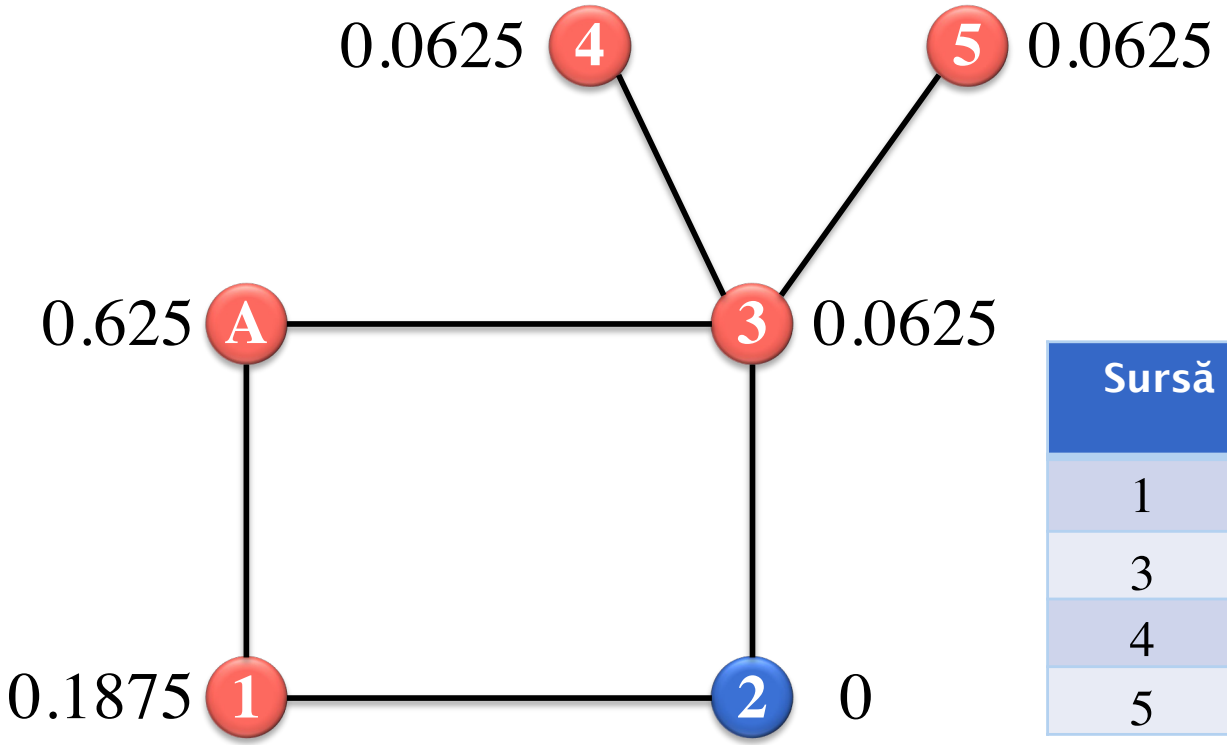
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
4	1	B(DW=0.0625)
2	A	C(DW=0.125)
1	3	B(DW=0.0625)
5	1	B(DW=0.0625)

TIMP
3

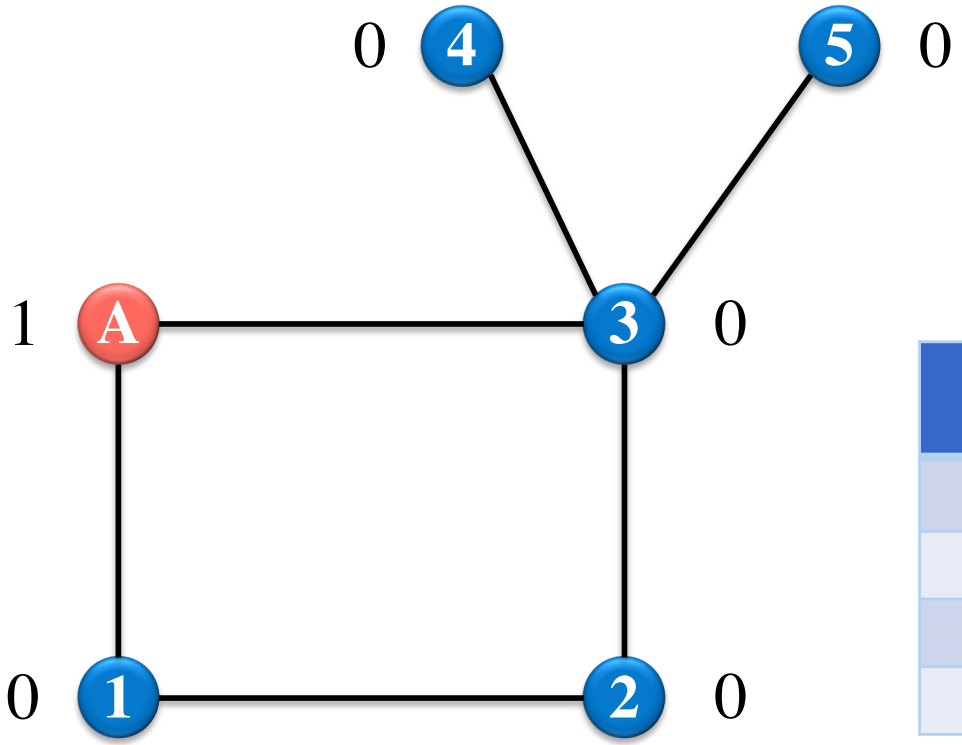
Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj
1	A	C(DW=0.1875)
3	A	C(DW=0.0625)
4	A	C(DW=0.0625)
5	A	C(DW=0.0625)

TIMP
4

Algoritmul lui Huang (4)



Sursă	Destinație	Mesaj

TIMP
5

Sumar

- Problematica terminării programelor distribuite
- Cazul proceselor organizate în inel
- Tehnica jetoanelor pentru cazul general
- Confirmarea mesajelor
- Algoritmul Dijkstra-Scholten
- Detecția terminării folosind marcaje
- Algoritmul lui Huang